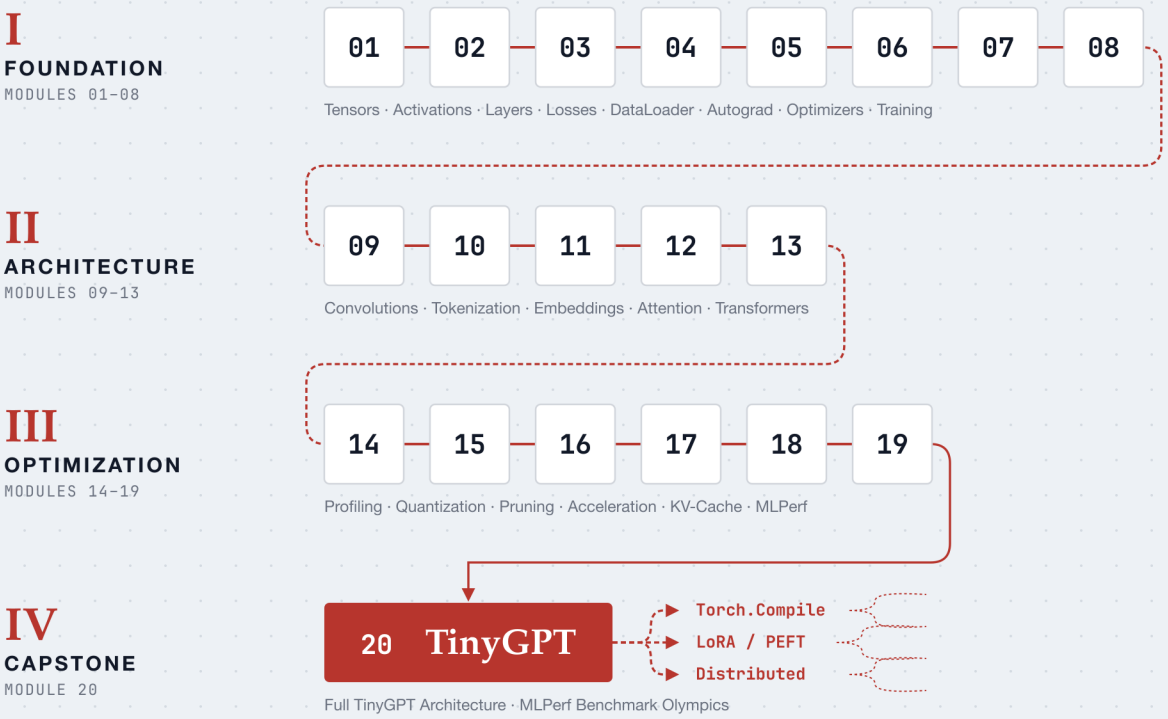
MACHINE LEARNING SYSTEMS SERIES

Tiny🔥Torch

Don't just **`import torch`**. *Build it.*

From Tensors to Transformers

THE FULL RUNTIME STACK — 20 MODULES / 4 TIERS



Vijay Janapa Reddi

Harvard University

TinyTorch: From Tensors to Transformers
Engineering Deep Learning Systems from Scratch

Licensing
The educational content in this book is licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0).

The TinyTorch software and code examples are released under the MIT License. See the repository's `LICENSE.md` and `tinytorch/LICENSE` for the applicable notices and terms.

Course Website: mlsysbook.ai/tinytorch/
Source Repository: github.com/harvard-edge/cs249r_book

First Edition Monograph (*Release Candidate*)
Compiled: October 27, 2026

Table of contents

Overview **1**

Preface **3**
Building vs. Engineering . . . 3
The Heritage: From Lions' Commentary and MIT's xv6 to Harvard's TinyTorch 3
The Architectural Choice: An Executable Specification . . . 4
Why a Generative Transformer is the Central Workload . . . 5
Training the Next Generation of Systems Architects . . . 5
The Journey Ahead . . . 6
Author's Note: On Craft, Systems Pedagogy, and Progressive Disclosure . . . 6
Acknowledgements . . . 7

Introduction **9**
The Evolution of Frameworks: From Static Graphs to Modern Compilers . . . 10
How a Framework Engineer Thinks: From Concept to Execution . . . 11
Progressive Disclosure . . . 13
What TinyTorch Is . . . 17
How Each Chapter Is Organized . . . 22
What You Need to Know Coming In . . . 23
Reading, Experimenting, and Returning . . . 23
Running and Testing the Code . . . 24

I The Core Engine

1 Tensors: Strided Memory and Multi-Dimensional Arrays **27**
1.1 The Problem . . . 30
1.2 The Idea . . . 31
1.3 The Code . . . 34
1.4 A Worked Trace . . . 44
1.5 From TinyTorch to Production . . . 45
1.6 Summary . . . 47
1.7 Further Reading . . . 47

2 Activation Functions: Non-Linearity and Numerical Stability **49**
2.1 The Problem . . . 50
2.2 The Idea . . . 52
2.3 The Code . . . 56
2.4 A Worked Trace . . . 61
2.5 From TinyTorch to Production . . . 63
2.6 Summary . . . 64
2.7 Further Reading . . . 65

3 Layers: Parameters, State, and Initialization **67**
3.1 The Problem . . . 68
3.2 The Idea . . . 69
3.3 The Code . . . 73
3.4 A Worked Trace . . . 81
3.5 From TinyTorch to Production . . . 82
3.6 Summary . . . 84
3.7 Further Reading . . . 84

4 Losses: Cross-Entropy and Numerical Stability **87**
4.1 The Problem . . . 88
4.2 The Idea . . . 89
4.3 The Code . . . 91
4.4 A Worked Trace . . . 99
4.5 From TinyTorch to Production . . . 101
4.6 Summary . . . 103
4.7 Further Reading . . . 103

5 Dataloaders: Batching and Memory Management **105**
5.1 The Problem . . . 106
5.2 The Idea . . . 107
5.3 The Code . . . 109
5.4 A Worked Trace . . . 114
5.5 From TinyTorch to Production . . . 119
5.6 Summary . . . 121
5.7 Further Reading . . . 122

6 Automatic Differentiation: Recording the Graph and Walking It Backward **123**
6.1 The Problem . . . 124
6.2 The Idea . . . 125
6.3 The Code . . . 128
6.4 A Worked Trace . . . 139
6.5 From TinyTorch to Production . . . 142
6.6 Summary . . . 143
6.7 Further Reading . . . 143

7 Optimizers: Momentum, AdamW, and Decoupled Weight Decay **145**
7.1 The Problem . . . 146
7.2 The Idea . . . 146
7.3 The Code . . . 151
7.4 A Worked Trace . . . 160
7.5 From TinyTorch to Production . . . 161
7.6 Summary . . . 163
7.7 Further Reading . . . 163

8 The Training Engine: The Five-Step Loop **165**
8.1 The Problem . . . 166
8.2 The Idea . . . 167
8.3 The Code . . . 169
8.4 A Worked Trace . . . 182
8.5 From TinyTorch to Production . . . 186
8.6 Summary . . . 188
8.7 Further Reading . . . 189

Synthesis I: From Perceptrons to Rumelhart's MLP **191**

Part 1: The 1958 Perceptron (Forward Pass Without Training) . . . 192
Part 2: The 1969 XOR Crisis (Linear Separability Limits) . . . 194
The Idea . . . 195
The Code . . . 196
A Worked Trace . . . 201
From TinyTorch to Production . . . 203
Summary . . . 205
Further Reading . . . 205

II Deep Architectures

9 Convolutions: Sliding Windows and the im2col Trick 209
9.1 The Problem . . . 210
9.2 The Idea . . . 211
9.3 The Code . . . 215
9.4 A Worked Trace . . . 227
9.5 From TinyTorch to Production . . . 230
9.6 Summary . . . 231
9.7 Further Reading . . . 232

10 Byte-Pair Encoding: Compressing Language into Subword Tokens 233
10.1 The Problem . . . 234
10.2 The Idea . . . 235
10.3 The Code . . . 236
10.4 A Worked Trace . . . 247
10.5 From TinyTorch to Production . . . 249
10.6 Summary . . . 250
10.7 Further Reading . . . 251

11 Embeddings and Positional Encoding: Turning Token IDs into Vectors 253
11.1 The Problem . . . 254
11.2 The Idea . . . 255
11.3 The Code . . . 256
11.4 A Worked Trace . . . 269
11.5 From TinyTorch to Production . . . 272
11.6 Summary . . . 274
11.7 Further Reading . . . 275

12 Attention Mechanisms: Scaled Dot-Product and Causal Masking 277
12.1 The Problem . . . 278
12.2 The Idea . . . 278
12.3 The Code . . . 281
12.4 A Worked Trace . . . 288
12.5 From TinyTorch to Production . . . 290
12.6 Summary . . . 292
12.7 Further Reading . . . 292

13 The Transformer: Assembling GPT from Attention Blocks 295
13.1 The Problem . . . 296
13.2 The Idea . . . 298
13.3 The Code . . . 301

13.4 A Worked Trace . . . 309
13.5 From TinyTorch to Production . . . 313
13.6 Summary . . . 315
13.7 Further Reading . . . 316

14 Synthesis II: Convolution, Attention, and Generated Text 317
14.1 Convolution on TinyDigits . . . 318
14.2 Attention on Structured Sequences . . . 319
14.3 The Problem . . . 321
14.4 The Idea . . . 322
14.5 The Code . . . 325
14.6 A Worked Trace . . . 332
14.7 From TinyTorch to Production . . . 333
14.8 Summary . . . 335
14.9 Further Reading . . . 335

III Systems & Acceleration

15 Profiling: The Roofline Model 339
15.1 The Problem . . . 340
15.2 The Idea . . . 341
15.3 The Code . . . 344
15.4 A Worked Trace . . . 353
15.5 From TinyTorch to Production . . . 357
15.6 Summary . . . 359
15.7 Further Reading . . . 359

16 INT8 Quantization: Compressing Floats into 8-Bit Integers 361
16.1 The Problem . . . 362
16.2 The Idea . . . 363
16.3 The Code . . . 366
16.4 A Worked Trace . . . 376
16.5 From TinyTorch to Production . . . 379
16.6 Summary . . . 381
16.7 Further Reading . . . 382

17 Model Compression: Pruning, Low-Rank SVD, and Distillation 383
17.1 The Problem . . . 384
17.2 The Idea . . . 385
17.3 The Code . . . 389
17.4 A Worked Trace . . . 399
17.5 From TinyTorch to Production . . . 405
17.6 Summary . . . 407
17.7 Further Reading . . . 407

18 Acceleration: Kernel Fusion and Cache Tiling 409
18.1 The Problem . . . 410
18.2 The Idea . . . 412
18.3 The Code . . . 414
18.4 A Worked Trace . . . 424
18.5 From TinyTorch to Production . . . 428

18.6 Summary . . . 429
18.7 Further Reading . . . 429

19 The KV Cache: Remembering Keys and Values During Generation 431
19.1 The Problem . . . 432
19.2 The Idea . . . 433
19.3 The Code . . . 436
19.4 A Worked Trace . . . 448
19.5 From TinyTorch to Production . . . 452
19.6 Summary . . . 453
19.7 Further Reading . . . 454

20 Benchmarking: Measuring Reliable Speedups 455
20.1 The Problem . . . 456
20.2 The Idea . . . 457
20.3 The Code . . . 460
20.4 A Worked Trace . . . 473
20.5 From TinyTorch to Production . . . 475
20.6 Summary . . . 477
20.7 Further Reading . . . 478

21 The Capstone: From an Optimization to a Checkable Report 479
21.1 The Problem . . . 480
21.2 The Idea . . . 481
21.3 The Code . . . 482
21.4 A Worked Trace . . . 492
21.5 From TinyTorch to Production . . . 499
21.6 Summary . . . 500
21.7 Further Reading . . . 500

22 Synthesis III: The MLPerf Optimization Olympics 501
22.1 The Problem . . . 503
22.2 The Idea . . . 504
22.3 The Code . . . 505
22.4 A Worked Trace . . . 511
22.5 From TinyTorch to Production . . . 514
22.6 Summary . . . 516
22.7 Further Reading . . . 516

IV Extensions & Accelerators

23 Extending TinyTorch 519
23.1 The Core Framework and the Extensibility Dilemma . . . 519
23.2 Why Not Just Use PyTorch? The Minix and xv6 Lesson . . . 520
23.3 The Four Native Extension Points . . . 521
23.4 The Flat Extension Catalog . . . 524
23.5 Evaluating an Extension: From Micro-benchmarks to End-to-End Milestones 540
23.6 Building on the Foundations: Next-Level Extensions . . . 542
23.7 From TinyTorch to Production . . . 546
23.8 Summary . . . 547
23.9 Further Reading . . . 548

Conclusion: Building Systems That Learn **549**
What We Built: The Complete Arc 549
Five Enduring Principles of Machine Learning Systems 551
The Bridge to Industrial Engines: From TinyTorch to PyTorch and JAX 552
The Road Ahead: Beyond Single-Node TinyTorch 555
The Builder's Manifesto 556

Overview

TinyTorch is a reference implementation for studying how a machine learning framework works. Its Python modules expose the operations that produce predictions, the record of those operations used to compute gradients, and the state that changes during training. This book accompanies that code in the spirit of classic systems commentaries: follow an execution path, explain the decisions it embodies, and check the explanation against the implementation.

Reading and building reinforce each other. A small trace makes a function understandable; a test checks whether that understanding survives an edge case; an integrated model shows whether the functions compose. The complete instructor solutions provide the reference for this process. NumPy supplies array storage and numerical operations, while TinyTorch implements the framework mechanisms around them.

This book is one half of a pair. The website at mlsysbook.ai/tinytorch is where you do the work: it carries the module pages, the API you fill in, the `tito` commands, the tests, and the historical milestones. This book is where the work is explained. Each chapter follows the same shape (The Problem, The Idea, The Code, A Worked Trace, From TinyTorch to Production), so a page on the site can hand you to the chapter that explains the mechanism behind it.

The progression in table 1 starts with individual operations and ends with a framework whose behavior and costs can be investigated.

Table 1: **The Four Parts of TinyTorch**: The implementation progresses from core operations to integrated experiments, followed by a production comparison.

Part	Focus	What you build
Part I: The Core Engine	Memory, gradients, optimization	`Tensor`, activations, `Linear`, `CrossEntropyLoss`, `DataLoader`, autograd, `AdamW`, the training loop
Part II: Deep Architectures	Images, text, transformers	`Conv2d`, `BPETokenizer`, `Embedding`, `MultiHeadAttention`, `TransformerBlock`, a small GPT
Part III: Systems & Acceleration	Measuring and speeding up	Profiling, simulated INT8 quantization, compression, NumPy acceleration experiments, `KVCache`, benchmarking, the capstone
Part IV: Extensions & Accelerators	Beyond NumPy	Optional C++, Triton, and Apple GPU kernels, measured; no additional TinyTorch module

The chapters keep the reference implementation distinct from production comparisons. A simulated storage saving is identified as an estimate; a timing result comes from an

experiment. Each part combines local tests with an integrated workload, so the reader can examine both an individual mechanism and the contract it must honor when another module calls it. The book concludes with the Conclusion, synthesizing the enduring systems principles of deep learning and surveying the road ahead toward planetary-scale distributed runtimes.

Preface

Building vs. Engineering

The technology industry is in a headlong rush to build artificial intelligence systems. Organizations scramble to assemble models, fine-tune weights, deploy pipelines, and capture the commercial potential of machine learning. Yet a crucial distinction separates building from engineering.

To build is to assemble components until an output appears. To engineer is to understand how those components interact under the physical laws of the machine: to measure memory movement, calculate arithmetic intensity, enforce numerical invariants, and predict where execution will stall when scale changes. Much of what passes for AI development treats the underlying runtime as a black box, optimizing a loss curve while remaining blind to the memory bus.

This book is anchored on a different principle: **track the byte, not just the loss.**

A machine learning framework is systems software. When a model trains or generates a token, floating-point numbers do not compute themselves in mathematical ether. They move across physical silicon. They stream through memory buses, contend for cache lines, saturate execution units, and stall on memory bandwidth. The practitioner who understands only the loss curve is helpless when an accelerator runs out of memory, when an inference pipeline delivers 5 percent of hardware peak throughput, or when a float32 exponential overflows to infinity and a loss turns into NaN. As machine learning matures into an engineering discipline, durable value belongs to those who understand the engine room.

This book is designed as the code-level, executable companion to the broader machine learning systems curriculum at **MLSysBook.ai** (*Machine Learning Systems*). While MLSysBook presents the architectural concepts, hardware landscape, and distributed scaling principles across the field, TinyTorch opens the hood on the software runtime itself, walking line by executable line through how a complete framework is built.

The Heritage: From Lions' Commentary and MIT's xv6 to Harvard's TinyTorch

We embrace a historical precedent because history proves that this pedagogical model works.

In 1976, John Lions wrote his famous commentary on the Unix Sixth Edition kernel, teaching operating systems by walking through actual lines of code. Thirty years later in 2006, Frans Kaashoek, Robert Morris, and Russ Cox at MIT revitalized that tradition by creating **xv6**: a modern re-implementation of Unix Sixth Edition. Rather than forcing students to navigate the millions of lines of C and assembly that constitute modern Linux, the MIT team presented a clean, self-contained kernel small enough to read in a semester, yet complete enough to boot on physical hardware, handle interrupts, manage page

tables, and schedule processes. A generation of computer systems engineers learned how an operating system functions by reading every line of xv6.

TinyTorch, which we developed at Harvard University, adopts that exact philosophy for machine learning systems.

In traditional systems curricula (operating systems, compilers, or computer architecture), it is a rite of passage to build the entire system from scratch. A student in an OS class builds a miniature kernel (like Pintos, TinyOS, or xv6); a compilers student builds a toolchain from lexing to assembly; an architecture student builds a working processor. Yet, in modern machine learning education, this systems-building tradition is genuinely lacking. Students write a tremendous amount of deep learning code, scripting isolated layers or assembling pre-built modules, but they rarely build the underlying system that collectively orchestrates them from the ground up.

Compounding this educational gap, modern deep learning frameworks have become massive software infrastructure. The PyTorch repository spans millions of lines of Python dispatch wrappers, C++ ATen tensor engines, cuDNN epilogues, and CUDA kernel templates. A newcomer attempting to understand how `loss.backward()` works inside PyTorch is immediately lost in an ocean of template metaprogramming, reference counting, and vendor-specific device drivers.

TinyTorch provides the executable commentary. It is a complete, self-contained machine learning framework implemented from scratch. It contains no external deep learning dependencies. A reader can study its tensor memory layouts, write its forward activations, construct its computational graph, implement reverse-mode automatic differentiation, and trace its optimizers line by line. Nothing is hidden; no operation is relegated to a pre-compiled binary.

The Architectural Choice: An Executable Specification

Engineering education requires honest boundaries. TinyTorch makes deliberate design choices to maximize clarity, and a reader should understand those choices from page one.

TinyTorch uses Python for framework orchestration and NumPy arrays for raw buffer storage. NumPy supplies C-contiguous memory allocations, strided addressing, and vectorized floating-point arithmetic. Python supplies the inspectable logic: how an autograd tape records operations, how parameter discovery traverses an object hierarchy, how a data loader collates batches, and how a key-value cache manages write pointers.

We deliberately do not write the core of TinyTorch in raw CUDA or C++ template code. Writing custom CUDA thread-block synchronization and PTX assembly in the foundational chapters would trap the reader in syntax and vendor compilation flags, obscuring the architectural contracts of the runtime. TinyTorch serves as an **executable architectural specification**. It allows a student to run the entire system in a standard Python debugger, place breakpoints on gradient accumulation steps, and inspect memory strides directly.

To keep the educational contracts transparent, TinyTorch maintains deliberate pedagogical firewalls, such as synchronous CPU execution and simulated quantization, which are detailed in the Introduction. To bridge this boundary, every chapter concludes with a dedicated section titled *From TinyTorch to Production*, detailing how the mechanisms modeled in TinyTorch translate to the C++ ATen runtime, NVIDIA cuDNN, CUTLASS, and Triton. Part IV (Chapter 23) then leaves NumPy for the first time, walking TinyTorch's

optional C++, Triton, and Apple GPU extensions to measure what each one gains and costs.

Why a Generative Transformer is the Central Workload

A systems textbook requires a workload demanding enough to stress the software runtime. While toy classifiers on synthetic datasets can verify that backpropagation computes correct numerical derivatives, they cannot expose the physical memory wall. A multi-layer perceptron with a few thousand parameters fits entirely inside a CPU's L1 data cache; it experiences no memory-bandwidth starvation, requires no context caching, and reveals nothing about sequence scaling.

For this reason, **TinyGPT serves as the unifying workload of TinyTorch**. From subword tokenization and multi-headed self-attention to autoregressive generation, memory profiling, INT8 quantization, and Key-Value caching, the framework is built and optimized around a complete generative language model. By anchoring the curriculum to a modern transformer, TinyTorch ensures that every optimization technique confronts the exact computational and memory dynamics that govern modern artificial intelligence.

Training the Next Generation of Systems Architects

Ultimately, our goal in writing this book is not merely to help readers master today's tools. Frameworks and APIs inevitably change. Libraries that dominate the industry today will eventually be superseded by new runtimes, new programming models, and new hardware architectures.

We wrote TinyTorch to train the next generation of engineers who can build the **next PyTorch**, the **next JAX**, or the next runtime that takes their place.

It is natural to ask, in an era where AI coding assistants can generate boilerplate scripts and assemble neural networks in seconds, what the value of building a system from scratch truly is. If the AI can write the code, why learn how the engine works?

The answer lies in the limits of automation. Generative AI is remarkably effective at assembling known APIs, but it struggles with novel architectural design, strict numerical invariants, and physical hardware constraints. As AI abstracts away the routine coding tasks of the past, the bottleneck in technology development shifts. The engineers of the future will not be hired to write superficial deep learning scripts: they will be hired to reason about memory physics, to debug complex latency stalls, and to architect the systems that the AI itself runs on.

The world needs more AI systems engineers. The practitioner who only knows how to prompt a language model for a PyTorch script is ultimately replaceable. The systems architect who understands how that script translates into strided memory allocations, backward-mode tape execution, and hardware-accelerated kernels is indispensable.

The fundamentals are what endure: how memory buffers are laid out and strided, how computational graphs record dependencies and execute reverse-mode differentiation, how cache lines pack quantized values, and how memory bandwidth governs execution throughput. By mastering these first principles in TinyTorch, students graduate not as passive consumers of existing software, but as the systems architects equipped to build the future of artificial intelligence.

The Journey Ahead

This book is designed to be read with the source code open. The companion repository contains the complete, tested reference implementation. The Introduction that follows details the five-part chapter structure, the visual signposts and callout tiers, the full curriculum roadmap across our four architectural tiers, the six reading flows, and the interactive development environment.

Author's Note: On Craft, Systems Pedagogy, and Progressive Disclosure

I learned computer systems by building them.

In computer science, that is how we master complex abstractions: we learn operating systems by writing a kernel like xv6; we learn compilers by writing an AST parser and a code generator; we learn computer architecture by designing a pipelined processor.

Yet in machine learning, that tactile, build-it-from-scratch pedagogy was missing. Most people learn deep learning as passive consumers of massive frameworks, importing a library, calling `.fit()`, and treating the underlying machinery as an opaque black box. I wanted to bring the foundational systems way of learning to a field that desperately needed it.

My goal was to give students and learners a framework that clicks together like a precision Lego set. Granted, systems engineers do not usually invent the original mathematical algorithms; the backpropagation formulation, the attention mechanism, and adaptive optimizers were created by pioneering researchers. But there is tremendous intellectual merit, craft, and discipline in understanding how those pieces actually fit together into a performant, coherent engine. I wanted learners to hold every piece in their hands: to lay out raw tensor memory, attach the autograd tape, wire up the optimizer, assemble multi-headed attention, and watch a complete GPT transformer come to life from components they engineered themselves.

Designing that progression was one of the hardest parts of this project. Progressive disclosure in a complex software stack is deceptively difficult. Every single module had to be self-contained and immediately runnable on day one, yet snap cleanly into the next module without circular dependencies, hidden global state, or hand-waving. I spent countless days and nights brainstorming, sifting through design possibilities, and writing prototypes only to tear them down when the progression did not feel natural for learners. Every abstraction boundary in this book, from keeping `Layer` minimal with explicit `forward()` and `parameters()`, to designing the four native extension points, was shaped through that iteration.

On AI Assistance and Modern Engineering

To bring this curriculum together at the depth and rigor it required, I worked side by side with AI coding assistants. I want to be transparent about what that collaboration actually looked like:

- **The architecture, pedagogical progression, and design decisions were mine.** I decided how the pieces should fit together, what should be included in the core vs. the extensions, and how to sequence the concepts so learners build real systems intuition.

- **I used AI as an untiring pair programmer and sounding board.** We used AI to help scaffold initial code blocks, generate repetitive boilerplate, write test harnesses, and translate concepts into C++ and Triton kernels.
- **We brainstormed and stress-tested relentlessly.** I used the AI to debate trade-offs, challenge over-engineered ideas (like stripping out artificial "Track A/B" groupings in favor of a flat extension catalog), and refine explanations until they were direct and clear.
- **Exhaustive verification.** We used AI to verify mathematical equations against numerical baselines in PyTorch and NumPy, construct unit test suites, run regression checks, and inspect rendered figures and tables in the compiled PDF to ensure everything met high visual and technical standards.

This is what modern engineering looks like: the human provides the domain judgment, the architectural blueprint, and the pedagogical vision; the AI provides speed, exhaustiveness, and rapid iteration.

A thought for students and learners: As you work through TinyTorch, you will likely use AI tools like Cursor, Copilot, Claude, or Gemini. I encourage you to use them. But be deliberate about how you do.

If you use AI to simply generate solutions for the exercises, you skip the struggle, and the struggle is where the learning happens. You miss the visceral understanding of stride arithmetic, memory bandwidth bottlenecks, and cache locality.

Use AI the way we did, as a collaborator. Ask it to explain why an unaligned vector load carries a performance penalty. Have it help you write tricky unit tests. Use it to review your code for edge cases. But make sure that you understand how every nut and bolt fits together, because in systems engineering, real mastery comes from knowing how the machine works from the ground up.

Vijay Janapa Reddi

Acknowledgements

This curriculum grew out of machine learning systems research and teaching at Harvard University. We are grateful to the students, teaching fellows, and colleagues whose questions, classroom feedback, and debugging sessions shaped the pedagogical boundaries of TinyTorch.

Introduction

The modern era of artificial intelligence is defined by an explosion of excitement and capability. Models converse, write software, synthesize biomolecules, and reason across complex domains in real time. Yet beneath the headlines, demos, and billion-parameter benchmarks lies an unsung hero: the software glue that holds the entire machine learning ecosystem together. That glue is the **deep learning framework**.

In computing history, the compiler was the quintessential systems breakthrough. In the early days of computing, programmers manually wrestled with assembly instructions and machine registers until optimizing compilers arrived as the foundational bridge, transforming abstract human logic (expressions, control flow, and data structures) into the physical realities of processor execution pipelines. Today, the machine learning framework plays that exact historical role. It is the modern compiler and runtime operating system for artificial intelligence. It takes high-level mathematical intent, such as tensors, non-linear activations, dynamic computational graphs, and loss objectives, and bridges them directly to physical silicon: streaming through memory buses, fitting within cache hierarchies, saturating execution units, and coordinating parallel hardware accelerators. Without the framework, deep learning is merely blackboard mathematics; with it, mathematics becomes physical, scalable execution.

When a practitioner writes a standard training step:

```
y = model(x)
loss = criterion(y, target)
loss.backward()
optimizer.step()
```

that deceptively simple syntax conceals a massive orchestration cascade. A layer retrieves persistent parameter tensors from memory, vector kernels dispatch floating-point arithmetic, an automatic differentiation tape records a dynamic directed acyclic graph (DAG), loss functions guard against IEEE 754 underflow, and an optimizer translates gradient vectors into parameter trajectory updates.

This book follows those execution paths through TinyTorch's reference implementation. The source defines what runs; the commentary explains why it is architected that way. In the tradition of classic systems commentaries, TinyTorch provides an open, self-contained reference framework where every layer, tape, and kernel can be inspected, traced, and executed on physical hardware.

The Evolution of Frameworks: From Static Graphs to Modern Compilers

The software that powers deep learning has undergone relentless churn over the past two decades. Understanding how and why frameworks evolved illuminates why their inner architectures look the way they do today:

1. **The First Generation: Static Graphs and Declarative Engines (2007–2015).** Early frameworks, including Theano, Caffe, and early TensorFlow (v1.x with `tf.Session` and `sess.run()`), required developers to define the entire computational graph upfront before feeding a single byte of data. This "define-and-run" paradigm allowed the framework to optimize graph execution globally, fuse operations, and plan static memory reuse before runtime. However, it came at a catastrophic cost to developer ergonomics: debugging required inspecting symbolic placeholder nodes rather than concrete values, standard Python `if` statements and `for` loops could not be used inside graphs, and runtime errors pointed to impenetrable C++ engine traces rather than the user's Python lines.
2. **The Second Generation: Dynamic Graphs and Eager Execution (2015–2020).** Chainer pioneered, and PyTorch popularized, the "define-by-run" approach. In eager execution, tensors are standard Python objects, operations execute immediately upon invocation, and a dynamic **autograd tape** records the graph on the fly as data flows forward. Developers could set standard Python breakpoints (`pdb`), use native Python control flow, print intermediate tensor shapes, and rapidly prototype dynamic architectures like recurrent networks and tree-LSTMs. The developer ergonomics accelerated experimentation, and eager execution quickly captured the global machine learning research community.
3. **The Third Generation: The Compiler Convergence (2020–Present).** Yet eager execution introduced a severe systems penalty: dispatching thousands of micro-operations one by one from Python saturates the CPU, prevents inter-operator fusion, and drowns the hardware memory bus in temporary memory allocations. Modern frameworks (such as JAX with functional transformations and XLA, along with PyTorch 2.0 with `torch.compile`, TorchDynamo, and Triton) have converged on a synthesis: **eager developer ergonomics backed by whole-program just-in-time compilation**. The runtime captures Python execution into an **Intermediate Representation (IR)** graph, analyzes data dependencies across operator boundaries, fuses memory-bound kernels, and generates optimized machine code for physical accelerators.

The Universal Invariants Beneath the Churn

Libraries, syntax, and APIs will inevitably continue to change. Over the past fifteen years, frameworks have risen to dominance only to be superseded: Torch yielded to Caffe; Caffe yielded to TensorFlow; TensorFlow yielded to PyTorch; and JAX, Mojo, and specialized MLIR compilers continue to push the boundaries of systems performance.

Yet beneath all of this churn, **every deep learning framework that has ever been built relies on the exact same five foundational systems pillars**:

1. **Strided Memory Layouts:** How contiguous bytes in physical DRAM or high-bandwidth memory (HBM) are indexed via dimension shapes, strides, and offsets without copying.

2. **Parallel Execution and Kernel Dispatch:** How mathematical operations (GEMM, convolutions, activations) lower into vectorized hardware instructions and contract inner dimensions across cache hierarchies.
3. **Reverse-Mode Automatic Differentiation:** How a directed acyclic graph (DAG) of dependencies records operations and traverses backward to evaluate the chain rule and accumulate parameter sensitivities.
4. **Stateful Parameter Lifecycle and Optimization:** How persistent weights are distinguished from ephemeral activations, discovered across container hierarchies, and updated along loss trajectories via momentum and adaptive moments.
5. **Memory Lifecycle and Caching Architecture:** How memory allocation overhead, cache line reuse (such as Key-Value context caching), buffer lifetimes, and IEEE 754 numerical stability conquer the physical memory wall.

If you only learn a specific framework's high-level API by memorizing the parameters of `torch.nn.Linear` or `tf.keras.layers.Dense`, your knowledge becomes obsolete with the next industry transition. But if you master the **five structural pillars** that govern the runtime underneath, you can understand any framework built on them, and you have the model needed to engineer new ones.

This is the purpose of **TinyTorch**. Rather than forcing you to wade through millions of lines of C++ template metaprogramming, device drivers, and vendor-specific wrappers, TinyTorch isolates these five universal pillars in their purest, most transparent executable form.

How a Framework Engineer Thinks: From Concept to Execution

To understand that engine, we must shift our perspective. A machine learning practitioner looks at a script and asks: *Is the loss decreasing, and is the model converging?* A framework engineer looks at the exact same script and asks: *Where was that memory buffer allocated? Is this tensor contiguous in memory, or will non-contiguous strides force an expensive clone before kernel dispatch? Is Python interpreter dispatch overhead dominating the actual hardware execution time? Did this operation return a zero-copy strided view or allocate a fresh array? Can this operation safely mutate a buffer in place without corrupting saved values on the autograd tape? Does the autograd tape retain unnecessary intermediate tensors in memory? Will broadcasting across the batch dimension require an accumulation reduction in the backward pass?*

Framework engineering is the discipline of **invariants**: the non-negotiable physical and mathematical contracts governing tensor shapes, memory strides, buffer lifetimes, and numerical precision that ensure execution is both bit-accurate and hardware-efficient.

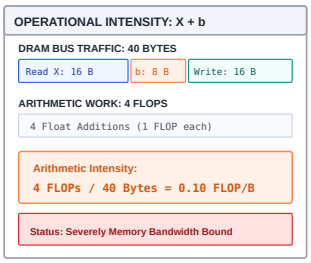
Instead of abstract math, we trace physical bytes streaming through silicon. To see how a framework engineer approaches an operation, consider a concrete micro-trace: adding a bias vector to a mini-batch of activations.

In logical coordinates, an activation matrix X has shape `(2, 2)` with values `[[1.0, 2.0], [3.0, 4.0]]`, and a bias vector b has shape `(2,)` with values `[10.0, 20.0]`. In physical hardware, DRAM knows nothing of 2D grids; it is a flat, 1D address space: - Activation tensor X occupies a contiguous 16-byte buffer: four 32-bit floats at byte offsets `0x00`, `0x04`, `0x08`, and `0x0C`, with row-major strides of `(8, 4)` bytes. - Bias vector b occupies an 8-byte buffer: two 32-bit floats at byte offsets `0x00` and `0x04`, with stride `(4,)` bytes.

When $X + b$ executes, how does the hardware evaluate the broadcast across both rows without duplicating the bias in DRAM? The framework constructs a zero-stride view with logical strides `(0, 4)` bytes. Iterating down the row axis adds zero bytes to the bias memory pointer. The CPU execution units fetch the identical 8 bytes directly from L1 data cache for each row: zero extra bytes allocated, zero memory bus waste.

Now calculate the physical memory traffic: - Read activations: 16 bytes streamed from DRAM. - Read bias: 8 bytes read once and retained in L1 cache. - Write output: 16 bytes written back to DRAM. - Total memory traffic: 40 bytes transferred across the memory bus. - Arithmetic computation: 4 floating-point additions (4 FLOPs).

The operational intensity is $4 \text{ FLOPs}/40 \text{ bytes} = 0.1$ FLOPs/byte. On hardware capable of 100 GFLOPs per second but constrained by 20 GB/s memory bandwidth, this operation runs at a tiny fraction of peak execution speed: it is hopelessly memory bandwidth bound. This is how a framework engineer reads code: not just checking that the output matches `[[11.0, 22.0], [13.0, 24.0]]`, but tracking the bytes, calculating the operational intensity, and auditing the memory bus.



Operational intensity micro-trace: 4 FLOPs over 40 DRAM bytes yields 0.1 FLOPs/byte, placing simple tensor addition deep inside the memory-bound regime.

In Module 01, `Tensor.__add__` passes the operands to `Add.apply`. The inherited `Function.apply` extracts their NumPy arrays, calls the operation's `forward` method, and wraps the returned array in a new Tensor. NumPy performs the broadcast addition, reusing the bias across the rows without requiring us to repeat it in the input. Neither operand changes; the result has its own storage. Following these calls separates the public expression from the mechanism that computes it, without requiring the rest of the Tensor class to be understood at once.

The caller can therefore rely on three facts: the values are the rowwise sums, the output shape is `(2, 2)`, and the inputs remain unchanged. Such a property that must remain true is an *invariant*. A useful reading order makes these obligations explicit:

1. **Find entry point**: Find the entry point and its caller. Record the input shapes and the result the caller expects.
2. **Follow values**: Follow the values and state through the calls that serve this example. Distinguish a returned value from a mutation of an existing object.
3. **State the invariant**: State the invariant that must hold when the call returns, such as a matching shape or an unchanged parameter.
4. **Trace a small input**: Trace a small input by hand, then compare it with the code and its tests. Change one assumption to expose the boundary of the contract.
5. **Compare cost**: Once the mechanism is clear, identify its cost and compare it with the alternative in the production bridge. Check which costs the implementation measures and which it estimates.

The source is organized to support more than the current example. Module 01 contains fields reserved for gradients, but they need not be understood to follow addition. Module 06 extends operation dispatch to record the graph needed for differentiation. Deferring those fields is a reading strategy: the forward computation provides a stable starting point for the later explanation of learning.

Contracts Across Module Boundaries: The Broadcast-Reduction Invariant

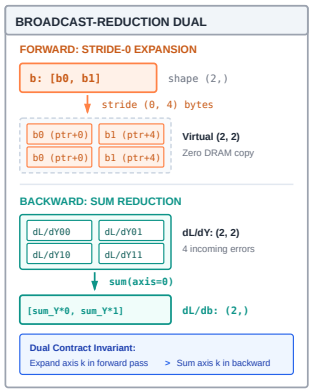
The bias example also illustrates a fundamental systems principle: every operation establishes a contract that downstream execution depends on.

In forward execution, broadcasting expands the bias from shape `(2,)` to `(2, 2)` by projecting virtual strides across the batch dimension. But notice the downstream

obligation this creates: parameters in physical memory have fixed shapes. The bias buffer contains exactly two numbers. When the framework eventually traverses backward during learning to update parameters, it encounters four error contributions (one for each element in the output grid). If the framework failed to sum those contributions back across the broadcast dimension, it would attempt to update a 2-element parameter with a 4-element tensor, corrupting memory.

This is the **Broadcast-Reduction Dual Invariant**: *any forward operation that expands a dimension through broadcasting incurs a strict contract to reduce that dimension through summation during the backward pass.*

We defer the mathematical mechanics of automatic differentiation until Module 06. However, the architectural lesson applies on Day 1: a framework operation cannot be designed in isolation. It must be engineered to honor the contracts of its consumers, preserving shapes, strides, and memory invariants across the entire pipeline.



The Broadcast-Reduction Dual Invariant: Expanding a dimension forward via stride 0 requires summing along that dimension in reverse-mode autograd to preserve physical parameter shape.

Progressive Disclosure

A tensor addition can be studied before a trainable model exists. The first part therefore develops forward computation before deriving its reverse traversal and connecting gradients to updates. The next part reuses that training machinery for images and sequences. Measurement and optimization follow a working baseline, so a proposed improvement has something concrete to preserve and compare against.

The source sometimes contains supporting fields or hooks needed by a later module. The commentary identifies the path needed now and returns to those hooks when their role can be explained. Production comparisons follow the local mechanism; they are not prerequisites for running it.

The roadmap in table 2 names what each chapter builds and the question it answers. Modules 01 through 20 use the source directory with the matching two-digit prefix under `tinytorch/src/`, such as `01_tensor/` for Chapter 1 and `18_memoization/` for Chapter 19. Syntheses I, II, and III integrate modules 01–08, 09–13, and 14–20, respectively; Chapter 23 adds no source module.

Table 2: **The TinyTorch Roadmap**: Modules 01 through 20 map to source directories under `tinytorch/src/`; synthesis chapters integrate earlier modules, and Chapter 23 measures the optional hardware extensions.

Chapter	What you build	The question it answers
01. Tensors	A NumPy-backed array wrapper with shape operations	How do tensor operations preserve values and shapes, and where are arrays copied?
02. Activations	The nonlinear functions (ReLU, sigmoid, GELU) applied between layers	Why does a stack of linear layers collapse into one, and what prevents the collapse?
03. Layers	Linear layers with weights and biases, plus dropout	How are parameters organized and initialized, and what changes between training and inference?

Chapter	What you build	The question it answers
04. Losses	Cross-entropy and its relatives, computed without overflow	How do predictions become one number to minimize, without floating point blowing up?
05. DataLoader	Datasets, per-epoch shuffling, and collating samples into batches	How do separate samples become a stream of batches, and why does the order matter?
06. Autograd	Reverse-mode automatic differentiation on a recorded tape	What does `loss.backward()` actually do?
07. Optimizers	Gradient descent with momentum, and AdamW	Why is plain gradient descent slow, and what do adaptive optimizers change?
08. Training Loop	The zero, forward, loss, backward, step loop, with clipping and schedules	When must gradients be cleared, computed, and applied to parameters?
Synthesis I	XOR and handwritten digits, learned by an MLP (Milestones 01–03)	Can the components learn a nonlinear decision rule and classify small images?
09. Convolutions	Two-dimensional convolution and pooling with explicit loops	How do local windows share weights and accumulate gradients?
10. Tokenization	Byte-pair encoding, which turns text into integer tokens	How does text become numbers, and why subwords rather than characters or words?
11. Embeddings	Lookup tables from token IDs to vectors, plus position encodings	How does an integer token become something a network can learn from, and how does the model know word order?
12. Attention	Scaled dot-product attention with a causal mask	How does each token decide which earlier tokens to look at?
13. Transformers	The GPT-2 style block, stacked into a model	How do attention, feed-forward layers, normalization, and residual connections fit together?
Synthesis II	CNNs and a generative language model trained and sampled from (Milestones 04–05)	Does each architecture earn its place against a baseline, and can the transformer generate text?
14. Profiling	Timing and counting tools, and the roofline model	Where does the time go, and is an operation limited by arithmetic or by memory?
15. Quantization	Integer-code quantization and float32 reconstruction	How does rounding change values, and which storage savings are simulated?

Chapter	What you build	The question it answers
16. Compression	Pruning, low-rank factorization, and distillation	How do you shrink a model, and what accuracy does each method cost?
17. Acceleration	Fusion and tiling experiments in NumPy	Which overheads change when operations are combined, and which remain?
18. The KV Cache	Caching attention keys and values during generation	Which earlier computations can generation reuse, and what work still grows with context?
19. Benchmarking	Measurement discipline: warmup, repetition, and percentiles	How do you measure a speedup you can trust?
20. The Capstone	An evaluation harness for optimization experiments	Do individual improvements survive composition and measurement?
Synthesis III	The MLPerf Optimization Olympics: accuracy, memory, and latency measured together, with the Pareto frontier computed from them (Milestone 06)	Did the optimizations pay off on all three axes at once?
21. Extensions	Custom C++, Metal, and Triton kernels, timed against your Module 17 kernels in Milestone 07	What does leaving NumPy gain, and what does it cost?

The roadmap is also a dependency guide. The optimization chapters require a working model and a measurement method; the architecture chapters require tensor operations and a training loop. A synthesis chapter tests the handoff between those pieces, rather than introducing another independent abstraction.

Reading Flows and Evidence of Understanding

The framework develops across six interconnected execution flows. Each flow begins with a concrete computation and finishes with a contract that the next flow can rely upon. table 3 summarizes the path to trace through each flow and the concrete evidence that demonstrates mastery.

Table 3: **The Six Reading Flows of TinyTorch**: Execution paths to follow and the concrete evidence of understanding expected at each synthesis milestone.

Flow	Modules	Execution path to follow	Evidence of understanding
Forward computation	Chapter 1 through Chapter 5	Tensor operation → activation → layer → loss, with batches supplied by a dataset and loader	Trace values and shapes; distinguish parameters from activations and array storage from Tensor wrappers.
Learning	Chapter 6 through Chapter 8, Synthesis I	Recorded operation → backward traversal → parameter gradients → optimizer update	Explain broadcast reduction, graph lifetime, gradient accumulation, and normalization by the actual sample count.
Vision	Chapter 9	Image batch → convolution → pooling and normalization → loss	Trace one window and the contributions to its gradients; explain shared parameters and receptive fields.
Language	Chapter 10 through Chapter 13, Synthesis II	Text → token IDs → embeddings → attention → transformer outputs	Keep token, batch, head, and feature axes distinct; trace a causal attention computation before generation.
Measurement & optimization	Chapter 15 through Chapter 18	Baseline measurement → one transformation → correctness comparison → measurement	Separate elapsed time from analytical counts; distinguish an algorithmic demonstration from changed physical execution.
Reuse & evaluation	Chapter 19 through Chapter 23, Synthesis III	Cache update → cached attention → controlled comparison → integrated evaluation	Identify cached state, reset, and capacity behavior; explain remaining context work and verify metrics.

What TinyTorch Is

TinyTorch has twenty source modules under `tinytorch/src/`, corresponding to Modules 01 through 20. Each source file combines executable Python with notebook prose, solution markers, inline tests, and demonstrations. Generated notebooks live under `modules/`; exported library code lives under the nested `tinytorch/` package. Changes to the instructor reference belong in `src/`, from which these other forms are rebuilt.

A TinyTorch Tensor wraps a NumPy array. NumPy performs elementwise arithmetic, matrix multiplication, and reductions; TinyTorch supplies the framework logic around those operations. The constructor creates float32 array storage. Consequently, a NumPy operation that can return a view does not establish that the enclosing Tensor operation is a zero-copy view. The tensor chapter follows the actual constructor and operation path to make that boundary visible.

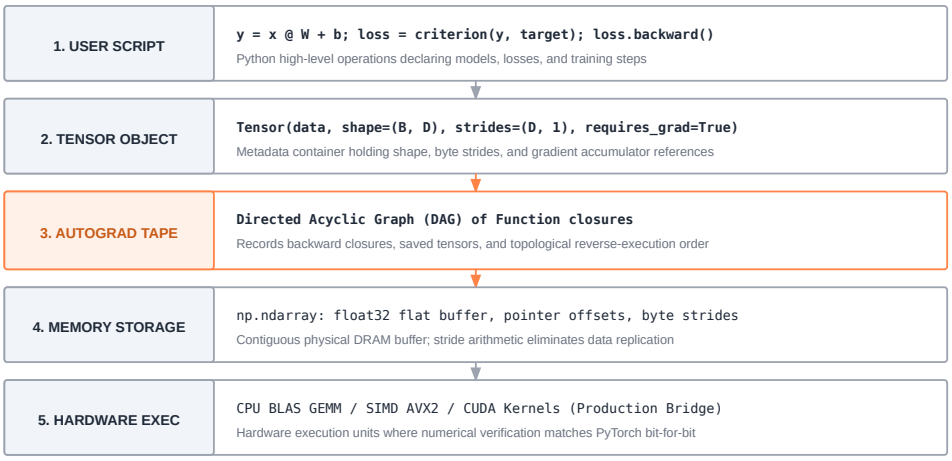


Figure 1: **The physical execution stack**: A TinyTorch Tensor wraps a CPython object containing shape and stride metadata, referencing a contiguous C-level float32 memory buffer mapped across OS virtual memory pages down to hardware CPU cache lines (L1/L2/L3) and DRAM.

The book's registered code listings are extracted from source symbols. The extraction removes notebook and exercise scaffolding so the commentary can discuss a function without reproducing an assignment cell in full. Small illustrative examples and worked traces accompany these listings. The source remains the authority for the complete implementation, including validation and tests outside the displayed excerpt.

The modules and the book are organized in four parts, summarized in table 4.

Table 4: **The Four Parts of TinyTorch**: The core engine supports architectures, optimization experiments, and a final production comparison.

Part	Chapters	What it covers
Part I: The Core Engine	1 to 8, Synthesis I	Tensors, activations, layers, losses, data loading, automatic differentiation, optimizers, and the training loop. Everything needed to train a neural network from scratch.
Part II: Deep Architectures	9 to 13, Synthesis II	Convolutions for images. Tokenization, embeddings, attention, and the transformer block for text, culminating in training TinyGPT from scratch.
Part III: Systems and Acceleration	14 to 20, Synthesis III	Measuring where time goes, then testing ways to reduce it: quantization, compression, operator fusion, the KV cache, and honest benchmarking in the MLPerf capstone.
Part IV: Extensions	21	What compiled kernels and GPUs gain over NumPy, measured, and what they cost.

Figure 2 follows the twenty source modules through their integration in the capstone. Its last two boxes subdivide Part III; the production comparisons in Part IV add no source module. The six milestone labels name historical workloads in the code repository, which the book groups into three synthesis chapters; a seventh milestone, custom kernels, accompanies Chapter 23.

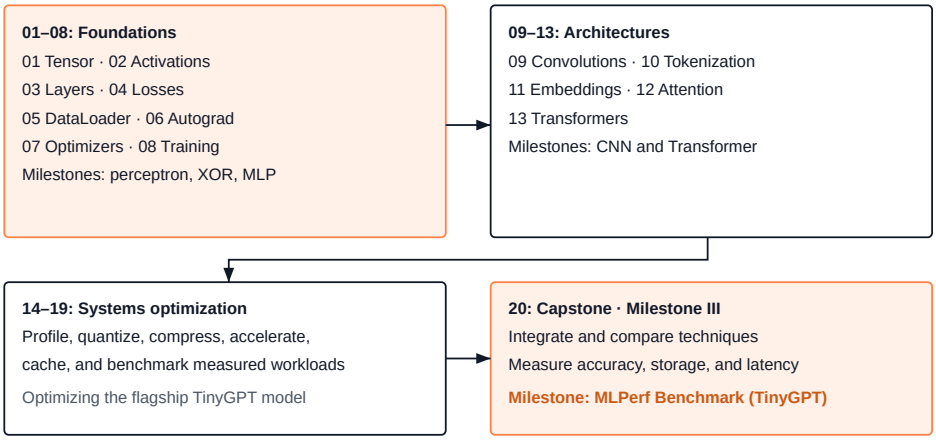


Figure 2: **From Operations to an Integrated Framework**: Foundational modules support image and language architectures; optimization experiments then lead to the capstone comparison.

TinyGPT: The Flagship Workload and Dual Pathways

A systems curriculum requires an application workload demanding enough to stress every layer of the software runtime. A simple feedforward classifier on synthetic points can verify that backpropagation computes correct numerical derivatives, but it cannot expose the physical memory wall. A small two-layer network fits entirely inside a CPU's L1 cache; it experiences no memory-bandwidth starvation, requires no sequence-length scaling, and exercises no dynamic state reuse.

For this reason, **TinyGPT is the main workload of the curriculum**.

TinyGPT is the main workload because its sequence length and attention mechanism stress the memory hierarchy hardest, but the systems principles you learn apply to every workload. The exact same profiling, INT8 quantization, pruning, kernel lowering (such as `im2col` spatial GEMM expansion), and operator fusion techniques apply directly to computer vision models like LeNet CNNs and dense edge networks. In the capstone Optimization Olympics, you evaluate all three workload classes side by side.

The curriculum is structured around a deliberate two-stage arc:

1. **The First Half (Modules 01–13): Building and Training TinyGPT.** You do not have to wait until the end of the book to experience generative AI. In Part I (Modules 01–08), you build the core execution engine: strided tensor storage, activations, parameter containers, losses, mini-batch loaders, reverse-mode autograd, and AdamW. In Part II (Modules 09–13), you construct tokenization, embeddings, multi-head causal attention, and stacked transformer blocks. By **Module 13 and Synthesis II**, you train TinyGPT from scratch on Shakespeare and watch it generate coherent text one token at a time. The working generative model arrives right at the midpoint of your journey.
2. **The Second Half (Modules 14–20): Accelerating TinyGPT and Vision.** Once you have models generating predictions and text, you face the reality of production machine learning systems: naive models are slow, memory-bound, and wasteful. In standard autoregressive sampling, every new token recomputes attention across the entire preceding context, so memory traffic grows as $O(S^2)$. Part III subjects your models to systematic systems performance engineering:
   - Profiling Roofline memory walls across linear, convolutional, and attention layers (Module 14)
   - Quantizing projection weights to INT8 (Module 15)
   - Pruning redundant weights and low-rank factorization (Module 16)
   - Fusing memory-bound operations and lowering convolutions to GEMM (Module 17)
   - Eliminating prefix recomputation with a Key-Value cache (Module 18)
   - Benchmarking under MLPerf discipline in the Optimization Olympics (Modules 19–20, Synthesis III)

Dual Pathways and the Two-Semester Curriculum

This two-stage architecture enables two distinct, fully supported learning pathways through the book, which naturally map to a two-semester academic sequence:

- **Pathway 1 / Semester 1: Building Generative AI from Scratch (Modules 01 to 13).** Starting from first principles, you build every layer of the stack from raw array memory up to a generative language model. This track is designed for students and practitioners who want to understand, end to end, how deep learning software

works. The semester culminates in Milestone 05: training TinyGPT on Shakespeare, generating Python source code with TinyCopilot under compiler gates, and closing the generative autoregressive loop.

- **Pathway 2 / Semester 2: Deep Learning Systems and Acceleration (Modules 14 to 20 and Chapter 23).** Systems programmers, compiler engineers, and hardware architects who already understand deep learning mathematics often want to focus exclusively on systems performance: cache dynamics, memory bandwidth saturation, quantization arithmetic, kernel fusion, and inference serving. Because TinyTorch provides pre-exported, validated reference libraries (`tinytorch.core` and `tinytorch.models.transformer`), readers on this track can take our out-of-the-box reference TinyGPT and CNN models and jump directly into Module 14. You immediately begin profiling memory bottlenecks, quantizing tensors, and building KV caches without needing to code earlier layers by hand. The semester culminates in Milestone 06: competing in the MLPerf Optimization Olympics across dense, vision, and generative LLM serving divisions.

The Milestone Progression: Recreating Deep Learning History

Alongside the modular chapters, the repository provides seven executable milestone workloads via the Tito CLI (`tito milestone run 01` through `07`); the seventh, custom kernels, belongs to Chapter 23. Each milestone recreates a pivotal historical breakthrough using exclusively the code you have built up to that point. Table 5 details the progression, the required modules, and how the repository workloads map to the book's three synthesis chapters.

Table 5: **The Six Historical Milestones of TinyTorch**: Chronological breakthroughs in machine learning history, executed via `tito milestone run 01` through `06` (Milestone 07, custom kernels, accompanies Chapter 23), and their alignment with the book's three synthesis chapters.

Milestone ID	Historical Era	Required Modules	Core Investigation	Book Synthesis Chapter
01	**The Perceptron (1958)**	01–03	Forward pass with random weights on linearly separable data; demonstrates that without training the decision boundary, and so the accuracy, is left to chance.	Synthesis I (Synthesis I, Part 1)
02	**The XOR Crisis (1969)**	01–03	Minsky and Papert's linear separability proof; proves that no single-layer hyperplane can separate XOR coordinates.	Synthesis I (Synthesis I, Part 2)

Milestone ID	Historical Era	Required Modules	Core Investigation	Book Synthesis Chapter
03	**The MLP Revival (1986)**	01–07	Rumelhart et al.'s backpropagation; hidden units fold the plane to solve XOR with 100% accuracy and scale to TinyDigits.	Synthesis I (Synthesis I, Part 3)
04	**The CNN Revolution (1998)**	01–07, 09	LeNet spatial parameter sharing and pooling; evaluates parameter count reduction vs. loop arithmetic on images.	Synthesis II (Synthesis II, Part 1)
05	**The Transformer Era (2017–2020)**	01–08, 10–13	Vaswani et al. (2017) and Brown et al. (GPT-3, 2020); trains TinyGPT on Shakespeare, validates sequence routing, trains TinyCopilot on Python code with AST compiler gating, and investigates conversational overfitting.	Synthesis II (Synthesis II, Part 2)
06	**MLPerf to ChatGPT Serving (2018–Present)**	01–04, 06, 07, 09, 11–19	Systems benchmarking from MLPerf discipline (Reddi et al., 2018) to modern production LLM serving (ChatGPT, 2022); measures TinyGPT, CNNs, and MLPs under KV-caching and INT8 quantization and reports the Pareto frontier computed from those measurements.	Synthesis III

Implementation Boundaries to Preserve

TinyTorch serves as an *executable architectural specification* rather than a competitor to multi-million-line production runtimes. To keep the framework inspectable, we maintain deliberate pedagogical firewalls:

- **NumPy storage vs. zero-copy views**: TinyTorch's `Tensor` constructor creates float32 NumPy array storage. A NumPy view inside an operation does not imply that the returned `Tensor` shares that storage without copying.
- **Synchronous single-thread loading**: The reference `DataLoader` forms batches synchronously. Multiprocess worker queues, asynchronous prefetching, pinned host memory, and non-blocking device transfers belong to the production bridge.

- **In-memory micro-datasets vs. distributed streaming corpora**: TinyTorch bundles curated, closed-world micro-datasets (such as TinyDigits, TinyPy, and TinyShakespeare) under 450 KB directly in the repository. This guarantees that training epochs complete in seconds on a single CPU core and intermediate activations fit on a single terminal screen for debugging. Multi-gigabyte web crawls, distributed sharding, and remote cloud object stores belong to production data engines.
- **Explicit convolution loops**: Reference convolution exposes sliding-window spatial loops to make weight sharing and receptive fields transparent. Vectorized `im2col` GEMM expansions and Winograd kernels are discussed as systems alternatives.
- **Simulated INT8 quantization**: Quantization demonstrates integer codes, dynamic scale factors, and zero-point alignment while retaining float32 containers, isolating numerical mechanics before discussing bit-level hardware packing.
- **Compression and distillation**: Knowledge distillation combines differentiable student loss with frozen teacher logits. Low-rank factorization illustrates parameter efficiency without conflating it with a full LoRA serving harness.
- **Algorithmic fusion vs. hardware SRAM residency**: NumPy operator fusion experiments demonstrate intermediate memory traffic reductions. They do not claim physical GPU register residency or compiler kernel generation.
- **Static KV cache allocation**: The Key-Value cache preallocates memory up to a maximum context window and advances a write head. It is not an elastic ring buffer or paged virtual memory manager.
- **Honest benchmarking**: Measurement discipline separates cold-start warmup effects from steady-state percentiles. Microbenchmark speedups do not automatically compose into end-to-end workload speedups.

How Each Chapter Is Organized

Every chapter in TinyTorch follows a deliberate **Hourglass Architecture**:

1. **The Problem (The ML "Why")**: We motivate why the mathematical operation or physical necessity arises in deep learning before writing code.
2. **The Idea (Systems Invariants)**: We establish representations, data structures, shapes, memory layouts, and correctness contracts.
3. **The Code**: At the narrowest point of the hourglass, we inspect the exact source implementation, extracted from the repository at build time so it cannot drift from what you run.
4. **A Worked Trace**: We follow a small, hand-verifiable numerical example through that code, so you can check your understanding against real values rather than against a description of them.
5. **From TinyTorch to Production**: We expand wide again, examining how production engines (PyTorch ATen, cuDNN, CUTLASS, Triton, vLLM) scale the concept across distributed clusters and specialized silicon.

The chapter summary establishes what later chapters rely on, followed by further reading pointing to the original foundational papers.

Navigating the Text: Signposts and Callouts

Because systems engineering relies on progressive disclosure, the margins contain continuous cross-referencing signposts:

- ↩ **`Jump back`**: Directs you back to where a prerequisite concept, memory layout, or invariant was first derived (e.g., Chapter 1 or Chapter 6).
- ↪ **`Jump ahead`**: Previews where a baseline component is revisited, optimized, or accelerated in later chapters (e.g., Chapter 15 or Chapter 19).

Four semantic callout tiers provide specialized engineering perspectives throughout each chapter:

- **Napkin Math** (`.callout-notebook`): Quantitative calculations for memory footprints, FLOP budgets, cache line utilization, and roofline operational intensity.
- **Systems Perspectives** (`.callout-perspective`): Production insights detailing how real-world frameworks (PyTorch ATen C++, CUDA, cuBLAS) engineer the equivalent subsystem.
- **Checkpoints** (`.callout-checkpoint`): Non-negotiable correctness invariants, common silent bugs (such as accidental gradient accumulation), and edge-case validation.
- **War Stories** (`.callout-war-story`): Real-world production incidents, numerical catastrophes (NaN overflows), and architectural lessons from large-scale model deployments.

What You Need to Know Coming In

Following `x @ weight` requires familiarity with Python method calls and matrix multiplication. Following `loss.backward()` additionally requires partial derivatives and the chain rule. The starting prerequisites are working Python, including classes and indexing, and basic matrix arithmetic. The tensor chapter develops shapes and broadcasting through examples. Derivatives and probability enter when learning and prediction require them; their framework-specific uses are explained where they arise.

Systems discussion starts from arrays in memory and the distinction between storing a value and computing it. The chapters explain the mechanisms needed by the reference implementation. Familiarity with an elementary derivative or a probability distribution will help when those concepts appear, but the opening chapters do not require an understanding of automatic differentiation or sampling. No GPU programming, C++, or compiler construction is needed to execute TinyTorch. Those topics appear later as production comparisons.

Reading, Experimenting, and Returning

A worked trace can be read on paper and then checked in a notebook. The narrative track explains the design and follows selected calls; the lab track puts those calls beside assertions, demonstrations, and experiments. Keeping the source open alongside the chapter makes it possible to inspect a helper when its behavior becomes relevant without losing the chapter's main path.

The current instructor notebooks contain complete solutions. Reading them is an active task: predict an output before executing a cell, identify the assertion that checks the prediction, and try a boundary case that could invalidate it. Reimplementing a function provides another exercise, with the reference and tests available for comparison.

On a first reading, stay with the chapter's chosen path until you can explain its input, output, and state. On a return visit, begin with the summary, reconstruct the worked trace without running it, and then follow the source calls that resolve any uncertainty. A useful test of understanding is to change the example's shape or reuse one input and predict what else must change. The chapter order supplies prerequisites; the summaries and source symbols provide entry points for rereading.

Running and Testing the Code

The inline tests exercise functions close to their definitions. The separate `tests/` tree checks additional numerical cases and interactions between modules. Both matter: a loss must compute the intended number and preserve the information needed to differentiate it; a training loop must normalize accumulated gradients correctly even when the last batch is smaller.

After completing the repository's TinyTorch environment setup, the following commands run from the TinyTorch package root, the directory containing `src/`, `tools/`, and `pyproject.toml`. The export command regenerates notebooks and the library from the instructor sources, so it belongs in a reference checkout rather than a notebook containing unexported learner edits.

```
# Use the activated TinyTorch development environment.
python3 tools/check_reference.py
python3 -m tito.main dev export --all
python3 tools/release_check.py
```

The first command builds a temporary package from source and checks targeted regressions. The release check validates structure, runs the test suite, and executes notebooks in order in fresh interpreters, making only earlier modules available to each notebook. These checks provide evidence for particular contracts; they do not establish that every possible input or optimization is correct.

A reader can begin without running the suite. The first chapter starts with the array inside a Tensor, follows an operation to its result, and establishes the shape and storage behavior on which the rest of the framework depends.

PART I

The Core Engine

🔥 Chapter 1

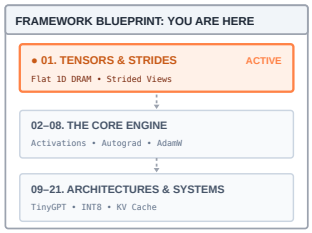
Tensors: Strided Memory and Multi-Dimensional Arrays

Every machine learning system, from the smallest training script to production infrastructure like PyTorch or JAX, is fundamentally an engine for streaming, transforming, and storing numerical arrays. Whether ingesting a batch of images, projecting token embeddings through transformer attention heads, or evaluating predictions against loss targets, the universal medium of exchange across the framework is the **tensor**.

Before writing a single line of framework code, we must answer a fundamental question: **How is a tensor used in deep learning?**

A deep learning model is an organized dataflow pipeline where tensors fulfill three distinct operational roles (figure 1.1):

1. **Input Data (Sensory Coordinates)**: How real-world phenomena enter the network. An image batch is a 4D tensor of shape (B, C, H, W), with batch size B, color channels C (such as RGB), height H, and width W. A batch of natural language text is a 3D tensor of shape (B, S, D), with batch size B, sequence length S tokens, and embedding hidden dimension D.
2. **Model Parameters (Persistent Learned Knowledge)**: The weights and biases that define the network's transformations. A linear projection layer (Chapter 3) stores a 2D weight tensor of shape $(D_{\text{in}}, D_{\text{out}})$ and a 1D bias vector of shape $(D_{\text{out}},)$. A convolutional layer (Chapter 9) stores a 4D weight tensor of shape $(C_{\text{out}}, C_{\text{in}}, K, K)$ holding spatial filter banks.
3. **Intermediate Activations (Ephemeral Lifetimes)**: As inputs cascade forward through layers, each intermediate transformation produces an *activation tensor*. Unlike persistent parameters that remain allocated throughout execution, activations are ephemeral: they exist solely to convey information forward to subsequent layers and loss metrics (Chapter 4), after which their allocated memory buffers can be reclaimed or overwritten.



Framework Datapath: Module 01 lays the physical 1D DRAM storage and strided coordinate foundation for the entire framework.

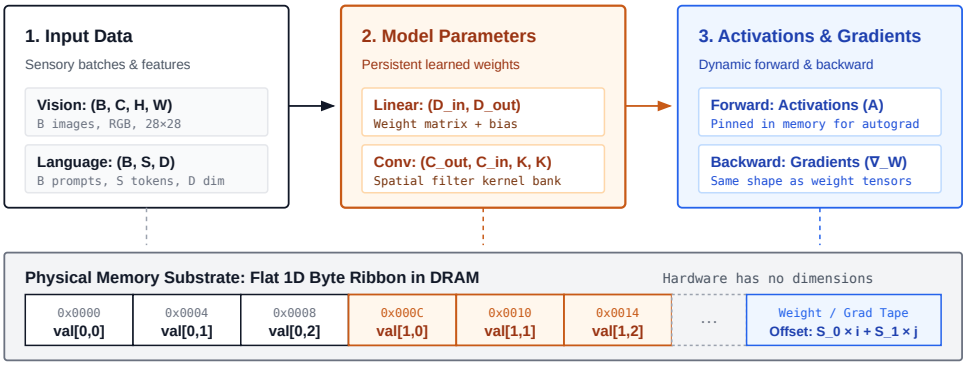


Figure 1.1: The three roles of tensors in a machine learning system. Input data, model parameters, and intermediate activations/gradients all use the tensor as their common medium of exchange, projecting logical coordinate axes onto a single physical substrate: a flat 1D byte ribbon in physical DRAM.

A **tensor** is thus a multidimensional array of numerical values organized along a defined sequence of coordinate axes. A single number is a scalar (a 0-dimensional tensor); an ordered list of numbers is a vector (1-dimensional); a rectangular grid is a matrix (2-dimensional); and higher-order tensors represent batches of sequence or spatial data. The tensor's **shape** is an integer tuple specifying how many entries each dimension contains.

In everyday deep learning, creating, reshaping, and indexing tensors in PyTorch feels instantaneous and effortless:

```
import torch

# Create a 2D tensor and inspect its memory layout
x = torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
y = x.t()  # Transpose: rows become columns[^fn-fortran-blas-strides]

[^fn-fortran-blas-strides]: Historical stride conventions originate in
    Fortran BLAS and LAPACK, where matrices are stored in column-major
    order with a leading dimension stride (`LDA`). Modern C and C++
    libraries (such as NumPy and PyTorch ATen) use row-major order by
    default, where the rightmost coordinate varies fastest in memory.
    TinyTorch inherits row-major contiguous memory from NumPy ndarray
    buffers.

print("x shape:", x.shape, "strides:", x.stride())        # (2, 3),
    strides: (3, 1)
print("y shape:", y.shape, "strides:", y.stride())        # (3, 2),
    strides: (1, 3)
print("Shares storage?", x.data_ptr() == y.data_ptr())    # True!
    Zero-copy view
```

Underneath that simple snippet, a sophisticated memory management engine is operating:

1. **Flat 1D physical storage:** The physical DRAM or GPU memory subsystem has no concept of multidimensional grids. Memory is a flat, one-dimensional ribbon of sequential byte addresses. The framework allocates a contiguous block of six 32-bit floats (24 bytes) in physical memory.
2. **Strided coordinate translation:** Instead of rearranging numbers in memory when transposing `x` to `y`, the framework performs **zero data copies**. It merely swaps the *strides* from `(3, 1)` to `(1, 3)`. The transposed tensor is an ephemeral view pointing to the identical underlying memory address (`data_ptr`).
3. **Storage vs. view decoupling:** The logical coordinate system (i, j) is completely decoupled from the physical byte layout. Stride arithmetic translates logical coordinates into physical byte offsets on the fly.

Module 01 builds this mechanism from first principles. NumPy describes arrays with shapes and strides; its **views** are array objects that reinterpret existing storage. TinyTorch wraps arrays in `Tensor` objects, and its constructor copies into a fresh float32 buffer. Consequently, `np.transpose(x.data)` shares storage with `x`, while `x.transpose()` does not. The operation path in figure 1.2 locates this copying boundary. Following addition and transpose through `Function.apply` will give us the interface that every later chapter uses.

Building this abstraction presents our first fundamental systems engineering challenge. If we implement arrays naively using native high-level language constructs like Python nested lists, memory footprints explode by nearly an order of magnitude and hardware caches thrash helplessly across pointer chains.

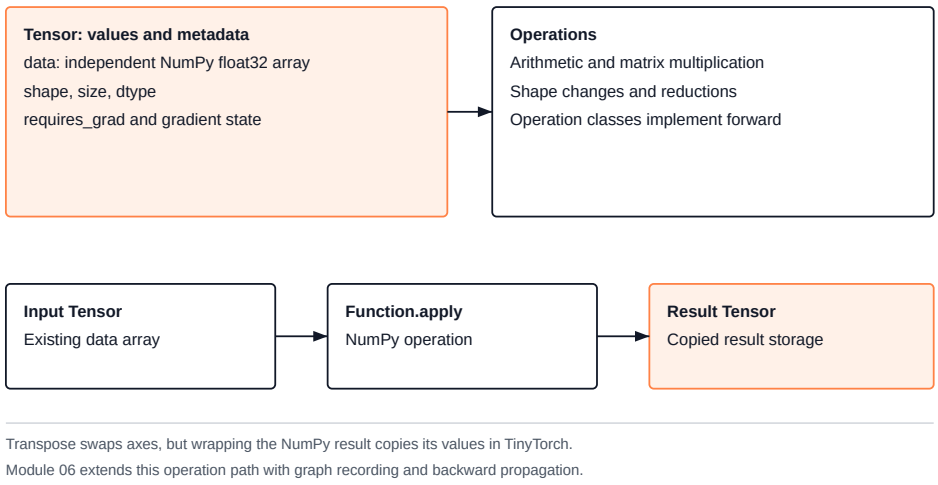


Figure 1.2: Tensor operations pass arrays through Function.apply and wrap the result in independent storage. The Tensor also reserves fields that Chapter 6 will use for gradients.

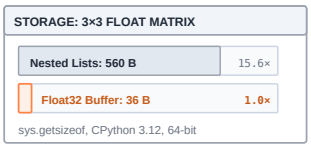
1.1 The Problem

The obvious way to hold a matrix in Python is a list of lists. It reads well, it indexes as `matrix[row][col]`, and it is how most of us wrote our first matrix multiply.

```
import sys

matrix = [
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
    [7.0, 8.0, 9.0],
]

payload = 9 * 4                       # nine 32-bit floats, 4 bytes each
list_bytes = (sys.getsizeof(matrix)
              + sum(sys.getsizeof(row) for row in matrix)
              + sum(sys.getsizeof(x) for row in matrix for x in row))
print(f"float32 payload: {payload} bytes, nested-list objects:
    {list_bytes} bytes")
```



The listing's output on CPython 3.12 (64-bit macOS): nested lists take 560 B, 15.6× the 36 B float32 buffer. List headers dominate this tiny matrix; at 1024×768 with distinct values the measured ratio falls to about 8× (25.2 MB vs. 3.1 MB).

The first count is always 36 bytes. The second depends on the Python implementation and includes the outer list, the three row lists, and the nine float objects. It does not measure a process's total memory use. Nor is 36 bytes the total cost of a NumPy array or Tensor; those objects also have metadata. The useful comparison is how storage grows as we add values. A float32 array adds four bytes per value to its data buffer, whereas nested lists hold references to separately represented Python numbers. Converting to float32 also changes numerical precision, which is acceptable for the exact integers in this example.

Problem: For a moderate batch tensor of shape $(1024, 768)$ in float32, how much memory is consumed by a contiguous array descriptor vs. nested Python lists?

Flat buffer calculation:

$$\text{Elements} = 1024 \times 768 = 786{,}432$$

$$\text{Storage} = 786{,}432 \times 4 \text{ bytes} \approx 3.15 \text{ MB}$$

Nested list overhead: Each Python `float` object contains a reference count and type pointer (24 bytes on 64-bit CPython). The inner lists require 8 bytes per pointer plus list container metadata (56 bytes). The outer list requires 1024 pointers ($8{,}192$ bytes, about 8.2 KB) plus 56 bytes:

$$\text{List payload} \approx 786{,}432 \times (24 + 8) + 1024 \times 64 \approx 25.23 \text{ MB}$$

Systems insight: Contiguous float32 storage achieves an $8.0\times$ memory reduction over nested Python lists while eliminating $786{,}432$ pointer indirections that thrash hardware data caches during matrix operations. Later in Chapter 16, we model what packing these 4-byte floats into 1-byte INT8 codes would save; TinyTorch itself keeps the codes in float32 storage.

The organization causes a second cost. Transposing a list of lists requires new row lists and a reference to each value in its new position. The float objects themselves need not be duplicated, but the new arrangement still requires work proportional to the

number of entries. With an array descriptor, the same rearrangement can be expressed by changing shape and strides. TinyTorch deliberately copies when it wraps the resulting view; understanding both steps prevents us from calling its transpose free.

The comparison in table 1.1 reflects a shared cause. A list of lists makes the *shape* part of the storage. The rows are physical objects, so changing the rows changes the storage. What we want is the opposite arrangement, in which the numbers sit in one flat run of memory and the shape is a separate, cheap description of how to read them.

Table 1.1: What changes when the shape is separated from the storage.

Property	Nested Python lists	TinyTorch `Tensor`
Where the numbers live	Scattered `float` objects, reached through two layers of pointers	One flat buffer, values side by side
Storage per value	A reference plus a Python float object; implementation-dependent	4 bytes in the float32 data buffer
Cost of a transpose	New row lists holding references to the values	NumPy changes metadata; the `Tensor` wrapper then copies the values
Cost of adding a vector to every row	A copy of the vector per row, or a Python loop	Broadcast reads avoid duplicating the vector; the result still allocates

1.2 The Idea

Store the numbers in a single flat buffer, in row order, and keep three pieces of metadata alongside it. The **shape** says how many entries each dimension has. The **strides** say how far to move in the buffer when you advance one step along each dimension. The **offset** says where in the buffer the tensor's first element sits. Indexing becomes arithmetic on these three. Some changes of shape can then be expressed by editing metadata alone; others require the values to be copied.

1.2.1 Row-major layout and strides

Take a matrix of shape $(2, 3)$ and write its rows one after another into the buffer, so the buffer is contiguous[1] and row-major.[2] Moving one column to the right means moving one slot in the buffer. Moving one row down means skipping a whole row, three slots. The strides are therefore $(3, 1)$.

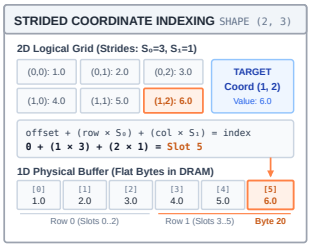
The same reasoning extends to any number of dimensions. For a shape $(D_0, D_1, \ldots, D_{N-1})$, the stride of the last dimension is 1, and each earlier stride is the next stride multiplied by the next dimension's size, because advancing one step there has to skip everything nested inside it:

$$S_{N-1} = 1, \qquad S_k = S_{k+1} \times D_{k+1} \quad \text{for } k = N-2, \ldots, 0$$

[1] **Contiguous** memory occupies consecutive addresses. In this chapter, a C-contiguous array is one whose logical entries occur consecutively when visited in row-major order. Owning a consecutive block of memory does not by itself establish that traversal order.

[2] **Row-major** order stores a matrix one complete row after another, so the last index varies fastest. Column-major order stores one complete column after another. The nested-list constructor used here creates row-major storage.

For shape $(2, 3, 4)$ this gives $S_2 = 1$, $S_1 = 4$, and $S_0 = 12$. Advancing one matrix in the batch skips 12 elements, one row skips 4, one column skips 1.



Strided Coordinate Mapping: Logical coordinate (1, 2) projects directly to 1D DRAM buffer slot 5 with zero memory allocation.

1.2.2 The coordinate mapping equation

With strides in hand, locating any coordinate $(i_0, i_1, \ldots, i_{N-1})$ in the buffer is one dot product:

$$\text{index} = \text{offset} + \sum_{k=0}^{N-1} i_k \times S_k$$

This equation describes an array view. We first use it to explain what NumPy can do without moving values, then follow the module's wrapper to see when TinyTorch copies those values. Shape describes the logical coordinates; strides describe their physical layout. Neither alone tells us whether another object shares the storage.

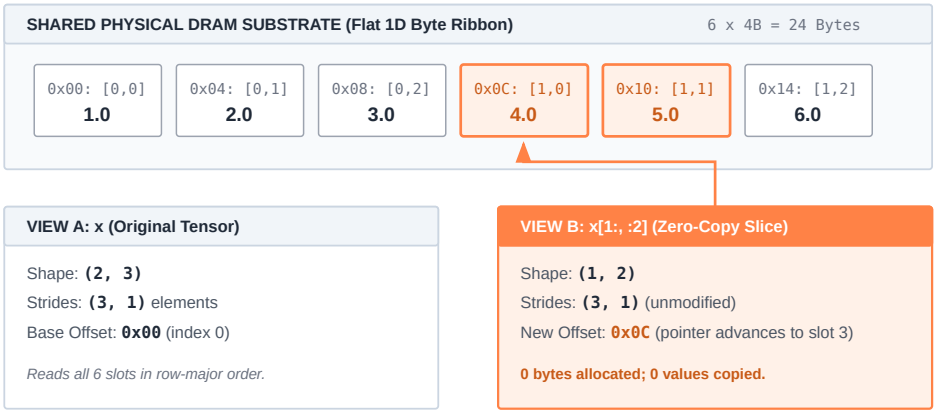


Figure 1.3: **Zero-copy slicing and stride manipulation**: Slicing advances the base memory offset pointer without allocating new memory, while strided views navigate the flat 1D byte ribbon in row-major or transposed order.

↪ **Jump ahead:** Chapter 3 *Broadcast strides allow parameter bias vectors to add across mini-batches without memory copies.*

1.2.3 Transposition is a swap

Deep learning architectures constantly reorient tensor axes, from transposing weight matrices (xW^T) in linear projections to permuting attention head dimensions in Transformers. Rather than copying gigabytes of buffer memory across DRAM, frameworks execute this operation in zero time by simply manipulating metadata. A transpose exchanges rows and columns. In the mapping equation, that is the same as exchanging the roles of i_0 and i_1, which is the same as swapping the two shape entries and the two stride entries. The matrix with shape $(2, 3)$ and strides $(3, 1)$ becomes a view with shape $(3, 2)$ and strides $(1, 3)$ over the *same* buffer.[3] Nothing is copied. The transposed view walks the buffer in column order because its strides tell it to. In Chapter 18, we will see that while transposing by swapping strides costs zero operations, reading that transposed tensor row by row causes cache misses on non-contiguous memory, motivating cache-tiled loops that partition matrices into sub-blocks to fit within hardware caches.

[3] PyTorch tracks storage sharing through `c10::StorageImpl`, a refcounted allocation wrapped by `c10::TensorImpl` view instances. In NumPy, `x.base` similarly points to the owning array when `x` is an ephemeral view.

1.2.4 Broadcasting is a zero stride

Real-world dataflow graphs routinely combine tensors of mismatched dimensions, such as adding a 1D parameter bias across a 2D batch of activations or scaling channel statistics during normalization. Because physically duplicating data across memory would waste DRAM bandwidth and capacity, runtimes implement **broadcasting** by walking expanded dimensions with a stride of zero. A bias vector `[10, 20, 30]` must supply the same values to both rows of our matrix. We can describe those reads with shape $(2, 3)$ and strides $(0, 1)$. Moving to the next row skips zero slots, so both rows read the original three values. For a vector of length D used across B rows, the mapping is

$$\text{index} = i \times 0 + j \times 1 = j$$

and the row index i has no effect on where we read. Every row maps onto the same D values. The vector now *behaves* as a (B, D) matrix at zero extra memory, and it did so through the same equation that handles every other tensor. NumPy and PyTorch call this **broadcasting**.

For the forward pass, the contract is simple: compatible dimensions either agree or one of them is 1, and each output coordinate reads the corresponding value from each operand. The `Add` listing below relies on this rule when it adds one bias vector to several rows. A later consequence, developed in Chapter 6, is that contributions from those rows must be summed when computing the bias's gradient; no backward machinery is needed to use broadcasting here. Specifically, the multivariable chain rule mandates that any axis expanded via stride 0 in the forward pass must be summed along that same axis in the backward pass via `_reduce_broadcast_grad`.

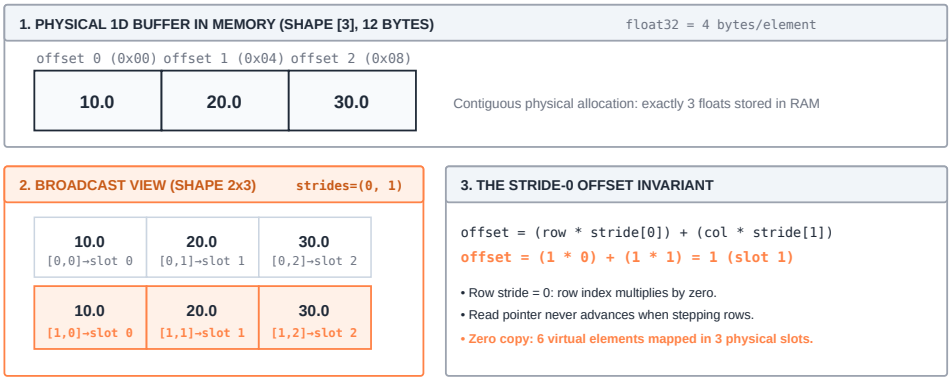


Figure 1.4: **Broadcasting with stride zero**: An operand vector of shape `(3,)` is broadcast across batch rows by assigning stride `0` along the row axis. The coordinate mapping equation reads the identical memory buffer `[10, 20, 30]` across multiple rows without allocating or duplicating any physical storage.

1.2.5 Contiguity is a property you can check

High-performance numerical backends (such as cuBLAS GEMM kernels and vectorized SIMD loops) require tensor elements to be laid out sequentially in physical memory to maximize memory bus bandwidth. When zero-copy operations like transpositions or broadcasts disrupt this linear order, frameworks must inspect buffer layout before

dispatching optimized routines. After a transpose or a broadcast, the buffer is unchanged but the view no longer walks it in plain row order. Our transposed matrix has shape $(3, 2)$ and strides $(1, 3)$, whereas row-major strides for that new shape would be $(2, 1)$. The broadcast view repeats locations because its row stride is zero. For arrays whose dimensions all exceed one, matching the formula for the array's own shape establishes C-contiguity. A dimension of length one is an exception because we never step along it; its stride does not affect the traversal. Empty arrays are another special case. NumPy's array flags documentation describes these cases, so `C_CONTIGUOUS` is the authoritative check. NumPy can read these layouts through their strides. Writes need more care: a broadcast view can map several coordinates to one location,[4] and `np.broadcast_to` returns a read-only view. TinyTorch avoids sharing the returned tensor with its input by copying at construction. Its `contiguous()` method additionally requests row-major layout, which is a separate property from owning the buffer.

[4] Writing to a stride-0 array is illegal in production systems because multiple logical coordinates resolve to the identical physical address. Concurrent parallel writes cause non-deterministic race conditions; PyTorch explicitly forbids in-place writes on stride-0 views.

1.2.6 Reductions remove an axis

From calculating loss objectives to computing layer statistics in LayerNorm and BatchNorm, deep learning pipelines depend continuously on **reductions** to aggregate tensor values. These operations collapse high-dimensional activations along targeted coordinate axes while preserving the geometry of remaining dimensions. Broadcasting adds an axis by walking it with stride zero. A reduction does the opposite: it collapses one, combining every entry along that axis into a single value. `Sum`, `Mean`, and `Max` are the three the module builds, and they share one parameter that decides the output's shape.

For a matrix of shape (R, C), `axis=0` collapses the rows and leaves one value per column, shape $(C,)$. `axis=1` collapses the columns and leaves one value per row, shape $(R,)$. `axis=None`, the default, collapses everything to a scalar. The axis you name is the axis that disappears.

That disappearance is the source of a shape error that recurs across deep learning implementations. When calculating layer normalizations or subtracting per-row maximum values in activation functions (Chapter 2), the reduced tensor must broadcast back across the original input shape. If a matrix of shape (R, C) reduces along axis 1 to rank-1 shape $(R,)$, standard array broadcasting rules align dimensions from right to left: $(R,)$ aligns with the trailing column dimension C, triggering an immediate shape mismatch $(R, C) - (R,)$. Setting `keepdims=True` retains the collapsed axis with length one, so (R, C) reduces to $(R, 1)$ rather than $(R,)$, ensuring the result broadcasts cleanly across the original dimensions without transposition. Chapter 4 relies directly on this rank preservation when centering logits.

The arithmetic cost is linear in the entries read, since every element is touched once whichever axis is named. What changes is the access pattern: collapsing the last axis of a row-major array walks contiguous memory, while collapsing the first walks with a stride, reading one element per row. Same arithmetic, a different number of cache lines.

1.3 The Code

The shape-and-stride description belongs to NumPy's `ndarray`. Module 01 adds an owned array and a common operation interface around it. We can read the constructor first to establish what a returned Tensor guarantees, then follow the calls that produce one.

1.3.1 The Tensor class

The constructor establishes the array and the fields used by the Tensor methods.

```
def __init__(self, data, requires_grad=False):
    """Create a new tensor from data.
    """
    if isinstance(data, Tensor):
        data = data.data
    elif isinstance(data, (list, tuple)) and len(data) > 0 and
    ↪   isinstance(data[0], Tensor):
        data = np.stack([t.data for t in data])
    self.data = np.array(data, dtype=np.float32)
    self.shape = self.data.shape
    self.size = self.data.size
    self.dtype = self.data.dtype
    self.requires_grad = requires_grad   # Module 06 (autograd) will
    ↪   use this
    self.grad = None                     # Module 06 (autograd) will
    ↪   use this
    self._grad_fn = None                 # Module 06 (autograd) will
    ↪   use this
    self._graph_released = False         # Module 06 (autograd) will
    ↪   use this
```

SOURCE CODE MAPPING MOD 01

FILE: src/01_tensor/01_tensor.py
EXPORT: tinytorch.core.tensor
SYMBOLS: Tensor, Function.apply

Extracted & verified at build time ✓ tested

Source Code Mapping: Module 01 extracts Tensor, Function.apply, and strided coordinate indexing directly from the working repository.

The first branch unwraps an existing Tensor to its `.data`; the second stacks a list of Tensors along a new first axis. For example, two shape-`(3,)` tensors become one shape-`(2, 3)` array. The `np.array` call then copies the supplied values into float32 storage. The nine numbers in the nested-list example occupy 36 bytes of data. The shape, size, and dtype lines are conveniences that copy fields the array already carries. The four lines after them reserve state for later learning operations. A tensor needs room for a gradient, and for a record of the operation that produced it, and Module 06 fills those in; until then they are bookkeeping fields that the forward computation does not use. The NumPy array handles layout and its starting address; the Tensor does not duplicate those fields. In the worked trace, `A.data.strides` reports the array's strides in bytes rather than elements.

One property of the constructor matters more than it looks. It always copies. Whatever array it is handed, `np.array` allocates a fresh buffer for the result, so separately constructed tensors own separate buffers. Returning an existing tensor, or assigning to the public `.data` attribute, is outside that constructor guarantee. That simplification prevents an ordinary returned operation result from modifying its input through shared storage. Direct writes through `.data` or `.numpy()` can still change a tensor; copying does not make the object immutable. The transpose listing below shows the cost of this ownership rule.

1.3.2 What the array is doing: a model of strides

Model scope: The next listing is not in Module 01. It is the arithmetic NumPy performs inside the array that `Tensor` wraps, written out so it can be read. The worked trace checks it against NumPy's own numbers.

For the six-value matrix, `A[1, 2]` must find slot $1 \times 3 + 2 \times 1 = 5$, whose value is 6. A transpose must make that same value available at coordinate `(2, 1)` by swapping the shape and strides. Four functions express the calculation. `contiguous_strides` computes row-order strides; `physical_index` maps a valid coordinate to a slot; `transpose` exchanges two axes; and `broadcast_to` supplies zero strides where a dimension is repeated. This small model assumes positive dimension sizes and valid coordinates. It describes array reads, without implementing storage or mutation.

```
def contiguous_strides(shape):
    """Row-major strides, in elements, computed from the last dimension leftward."""
    strides = [1] * len(shape)
    for k in range(len(shape) - 2, -1, -1):
        strides[k] = strides[k + 1] * shape[k + 1]
    return tuple(strides)

def physical_index(strides, offset, indices):
    """The mapping equation: one slot number for an N-D coordinate."""
    return offset + sum(i * s for i, s in zip(indices, strides))

def transpose(shape, strides, dim0, dim1):
    """Swap two dimensions. The buffer is untouched."""
    shape, strides = list(shape), list(strides)
    shape[dim0], shape[dim1] = shape[dim1], shape[dim0]
    strides[dim0], strides[dim1] = strides[dim1], strides[dim0]
    return tuple(shape), tuple(strides)

def broadcast_to(shape, strides, target):
    """Stretch size-1 dimensions to the target shape with stride 0. No copy."""
    pad = len(target) - len(shape)
    if pad < 0:
        raise ValueError("broadcasting cannot remove dimensions")
    shape, strides = (1,) * pad + tuple(shape), (0,) * pad + tuple(strides)
    out = []
    for dim, stride, want in zip(shape, strides, target):
        if dim == want:
            out.append(stride)
        elif dim == 1:
            out.append(0)               # stretched: reread the same slot
        else:
            raise ValueError(f"cannot broadcast {shape} to {target}")
    return tuple(target), tuple(out)
```

`physical_index` never looks at the data or asks which kind of view it is serving. The strides carry that information, so a transposed or broadcast view takes the same path as a fresh one. Neither `transpose` nor `broadcast_to` allocates a new data buffer. Broadcasting `[10, 20, 30]` across two rows still reads only three stored values.

NumPy reports strides in bytes. A $(2, 3)$ array of 4-byte floats therefore shows `(12, 4)` where the model says `(3, 1)`, and its transpose shows `(4, 12)`. Dividing

NumPy's strides by `data.itemsize` converts them to the model's element units. `flags["C_CONTIGUOUS"]` checks row order, and `np.ascontiguousarray` provides that order when needed. The worked trace checks both properties separately from storage sharing.

1.3.3 Operations as objects

The naive way to give `Tensor` a `+` is to hardcode array arithmetic directly inside `__add__`. TinyTorch instead adopts the classic **Command Pattern/Operational Dispatch**: separating tensor data storage from computational operations. Each operation is encapsulated as a distinct class with a `forward` method that evaluates transformations on underlying NumPy buffers, and an execution interface (`Function.apply`) that unifies operator dispatch. This decouples mathematical operations into modular, extensible units rather than cluttering the core `Tensor` class with hundreds of mathematical methods. Each call instantiates an *operation object* bringing the transformation and its operand references together. While the initial dispatch simply computes and returns the output buffer, this uniform boundary ensures that when automatic differentiation arrives in Chapter 6, the runtime can seamlessly record operations on an execution tape without changing the `Tensor` interface. Here is the base class, from Module 01.

```
class Function:
    """
    Base class for every operation the framework can differentiate.
    """

    def __init__(self, *inputs, **params):
        self.inputs = inputs                # the Tensors this operation consumed
        for name, value in params.items():  # axis, shape, key, ... for the operation
            setattr(self, name, value)

    def forward(self, *arrays):
        """Compute the result array from the input arrays. Each operation implements this."""
        raise NotImplementedError(f"{type(self).__name__}.forward is not implemented")

    def backward(self, grad_output):
        """Return one gradient array per input. Module 06 implements this."""
        raise NotImplementedError(f"Module 06 implements {type(self).__name__}.backward")

    @classmethod
    def apply(cls, *inputs, **params):
        """Run the operation on Tensors and return a new Tensor."""
        node = cls(*inputs, **params)
        arrays = [t.data for t in inputs]
        return Tensor(node.forward(*arrays))
```

The constructor stores the input tensors and any parameters the operation needs, an axis for a reduction, a shape for a reshape. The `apply` classmethod is the only entry point the rest of the framework uses. It builds the operation object, unwraps every input to its NumPy array, calls `forward` on the arrays, and wraps the result in a new `Tensor`. `forward` receives arrays rather than Tensors. The final wrap goes through the constructor above, so every result from this path owns a fresh buffer. A helper turns scalars into tensors so that `x + 1` works.

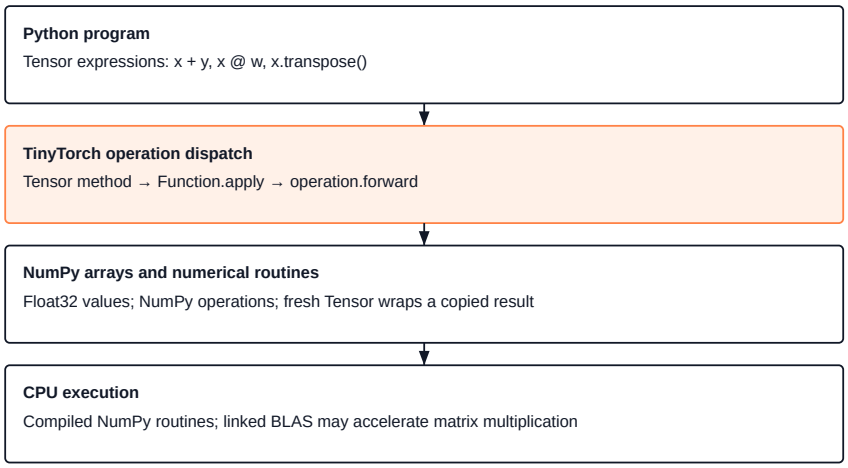


Figure 1.5: The TinyTorch execution stack. High-level Python expressions dispatch through Function.apply into concrete NumPy array operations, which execute on the CPU using compiled numerical routines and linked BLAS libraries.

```
def _as_tensor(x):
    """Wrap a scalar or array as a Tensor; pass a Tensor through
    ↪  unchanged."""
    return x if isinstance(x, Tensor) else Tensor(x)
```

With those two in place, the operator method on `Tensor` is one line, and the operation class is barely more.

```
def __add__(self, other):
    """Add two tensors element-wise with broadcasting (the Add
    ↪  operation below)."""
    return Add.apply(self, _as_tensor(other))
```

```
class Add(Function):
    """Element-wise addition c = a + b, with NumPy broadcasting."""

    def forward(self, a, b):
        """
        Add two arrays element-wise with broadcasting support.
        """
```

```
        return a + b
```

Follow `A + b` for `A.shape == (2, 3)` and `b = Tensor([10, 20, 30])`. Python calls `A.__add__(b)`. `_as_tensor(b)` returns the existing Tensor unchanged, and `Add.apply(A, b)` stores those two inputs on an operation object. `apply` passes their `.data` arrays to `Add.forward`. NumPy's addition reads the bias once per output coordinate according to the broadcasting rule, producing `[[11, 22, 33], [14, 25, 36]]`. The final `Tensor(...)` copies that result and returns shape `(2, 3)`. Neither input is mutated.

The broadcast avoids allocating a repeated bias matrix; it does not avoid allocating the sum. In this implementation, NumPy produces the sum array and Tensor construction then copies it. An explicit `np.broadcast_to` view in the worked trace makes the repeated reads visible, but `Add.forward` calls only `a + b`. NumPy implements the broadcasting within that operation, as described in its broadcasting guide.

1.3.4 Matrix multiplication

The other operation a linear layer needs is matrix multiplication. For two matrices, Module 01 makes the output-row and output-column loops explicit and delegates each inner dot product to NumPy.

```
class MatMul(Function):
    """Matrix multiplication c = a @ b. Shapes were validated by
    ↪ Tensor.matmul."""

    def forward(self, a, b):
        """
        Multiply two matrices.
        """
        # Educational implementation: explicit loops to show what
        ↪ matrix multiplication does
        # This is intentionally slower than np.matmul to demonstrate
        ↪ the value of vectorization

        # Handle 2D matrices with explicit loops (educational)
        if len(a.shape) == 2 and len(b.shape) == 2:
            M, K = a.shape
            _, N = b.shape   # b's row count is K; Tensor.matmul
↪ already checked it
            result_data = np.zeros((M, N), dtype=a.dtype)

            # Explicit nested loops - students can see exactly what's
            ↪ happening!
            # Each output element is a dot product of a row from A and
            ↪ a column from B
            for i in range(M):
                for j in range(N):
                    # Dot product of row i from A with column j from B
                    result_data[i, j] = np.dot(a[i, :], b[:, j])
        else:
```

```
            # Anything other than 2D @ 2D (a 1D operand, or batched
            ↪ 3D+ inputs)
            # goes to np.matmul. The mechanism is the same dot product
            ↪ per element.
            result_data = np.matmul(a, b)

        return result_data
```

The entry path is `A @ B` → `Tensor.__matmul__` → `Tensor.matmul`. Before constructing a `MatMul` operation, `_validate_matmul_shapes` requires a Tensor operand, rejects scalars, and checks the contracted dimensions. Our shape-`(2, 3)` matrix can multiply a shape-`(3, 2)` matrix, but cannot multiply another shape-`(2, 3)` matrix. A mismatch raises before the output loops begin. Batched inputs also depend on NumPy's batch-broadcasting rules.

For `A @ A.transpose()`, `M = 2`, `K = 3`, and `N = 2`. The zero-filled result has shape `(2, 2)`. At `i = 0, j = 1`, the inner call reads `[1, 2, 3]` and `[4, 5, 6]`, computes $1 \times 4 + 2 \times 5 + 3 \times 6 = 32$, and assigns `result_data[0, 1]`. Repeating for the other coordinates gives `[[14, 32], [32, 77]]`. `Function.apply` wraps that array, leaving both operands unchanged. Inputs involving a vector or a batch take the `np.matmul` branch instead of the two explicit output loops.

The dot product accepts strided inputs, so a view does not need to be copied merely to be read correctly. Layout can still affect the time required to read it. A row of a row-major array is contiguous; its column skips a row between values. In this particular product, the right operand has already been transposed, so its columns are contiguous. We must inspect each operand's strides before drawing a performance conclusion.

1.3.5 Transpose, ownership, and row order

The product needs shape `(3, 2)` from a `(2, 3)` input. `Tensor.transpose` builds the axis list `[0, 1]`, swaps its last two entries to `[1, 0]`, and passes `(1, 0)` to `Permute.apply`. An explicit pair such as `transpose(0, 2)` swaps those axes instead. For a scalar or vector with no explicit axes, there are no two dimensions to exchange, so the method takes the `Copy.apply` path and preserves the shape.

```
def transpose(self, dim0=None, dim1=None):
    """Transpose tensor dimensions.
    """
    if dim0 is None and dim1 is None:
        if len(self.shape) < 2:
            return Copy.apply(self)
        else:
            axes = list(range(len(self.shape)))
            axes[-2], axes[-1] = axes[-1], axes[-2]
    else:
        if dim0 is None or dim1 is None:
            provided = f"dim0={dim0}" if dim1 is None else
↪   f"dim1={dim1}"
            missing = "dim1" if dim1 is None else "dim0"
            raise ValueError(
                f"Transpose requires both dimensions to be specified\n"
```

```
                f"  ✗ Got {provided}, but {missing} is None\n"
                f"  💡 Either provide both dims or neither (default
                ↪  swaps last two)\n"
                f"  🔧 Use transpose({dim0 if dim0 is not None else
                ↪  0}, {dim1 if dim1 is not None else 1}) or just
                ↪  transpose()"
            )
        axes = list(range(len(self.shape)))
        axes[dim0], axes[dim1] = axes[dim1], axes[dim0]
    return Permute.apply(self, axes=tuple(axes))
```

```
class Permute(Function):
    """Reorder axes. Parameter: axes, the new order of the old axes
    ↪  (Tensor.transpose computes it)."""

    def forward(self, a):
        """
        Permute the axes of the array.
        """
        return np.transpose(a, self.axes)
```

`np.transpose` implements the axis rearrangement illustrated by the model. It returns a view with the shape and stride entries swapped and the buffer untouched, and the worked trace confirms that the view shares memory with the original. Then `apply` wraps that view in a new `Tensor`, and the constructor copies. The TinyTorch result therefore has independent storage, even though the intermediate NumPy view does not. The copy is not a plain row-major one, though. For this transpose, `np.array` preserves column-major order, so the new buffer keeps strides `(4, 12)`. The returned Tensor owns its memory and is still not C-contiguous. This example does not imply that copying preserves every possible slice's exact strides.

Independent buffer does not imply row-major layout: When `Tensor.transpose` executes, `np.transpose` returns a view with swapped strides sharing memory with the input. TinyTorch wraps this view in a fresh `Tensor`, and its constructor copies the values into a newly allocated buffer (`Buffer B` in figure 1.6).

Crucially, this copy allocates an **independent memory buffer** (`np.shares_memory(At.data, A.data) == False`), but NumPy's `np.array` constructor preserves the strided access pattern of the view. The strides of `At.data` remain `(4, 12)` in bytes, which is column-major order. The tensor owns its memory, yet `At.data.flags["C_CONTIGUOUS"]` is `False`. To arrange the physical elements sequentially in row-major order, the student must call `At.contiguous()`, producing `Buffer C`.

The constructor has already paid for an independent buffer, but that buffer is still in column-major order. `contiguous()` answers a different request: arrange the same logical values in row-major order. Some consumers can use arbitrary strides, and matrix libraries can often accept a transpose directly; others require a particular layout. Copying for ownership and copying for layout therefore belong on separate lines of the cost ledger.

```
def contiguous(self):
```

```
    """Return a contiguous copy of the tensor data
    ↪ (PyTorch-compatible)."""
    return Copy.apply(self)
```

```
class Copy(Function):
    """A contiguous copy of the array (used by contiguous() and 1D
    ↪ transpose)."""

    def forward(self, a):
        """
        Copy the array into contiguous memory.
        """
        return np.ascontiguousarray(a).reshape(a.shape)
```

`Copy.forward` calls `np.ascontiguousarray`. For a non-contiguous input, that call writes the values into row-major order; for an already contiguous input, NumPy can return the same array. The final `.reshape(a.shape)` preserves a scalar's shape `()`, since `np.ascontiguousarray` alone would promote it to `(1,)`. `Function.apply` then constructs a `Tensor`, adding one copy whether or not a layout conversion already occurred. Thus `Tensor.contiguous()` always returns a separate tensor, even when no layout conversion was needed. The method is intentionally simple, but it is not a no-cost check.

The three results in figure 1.6 have identical logical values but differ in storage sharing or row order. Buffer B makes the ownership copy visible; Buffer C shows the reordered output of `contiguous()`.

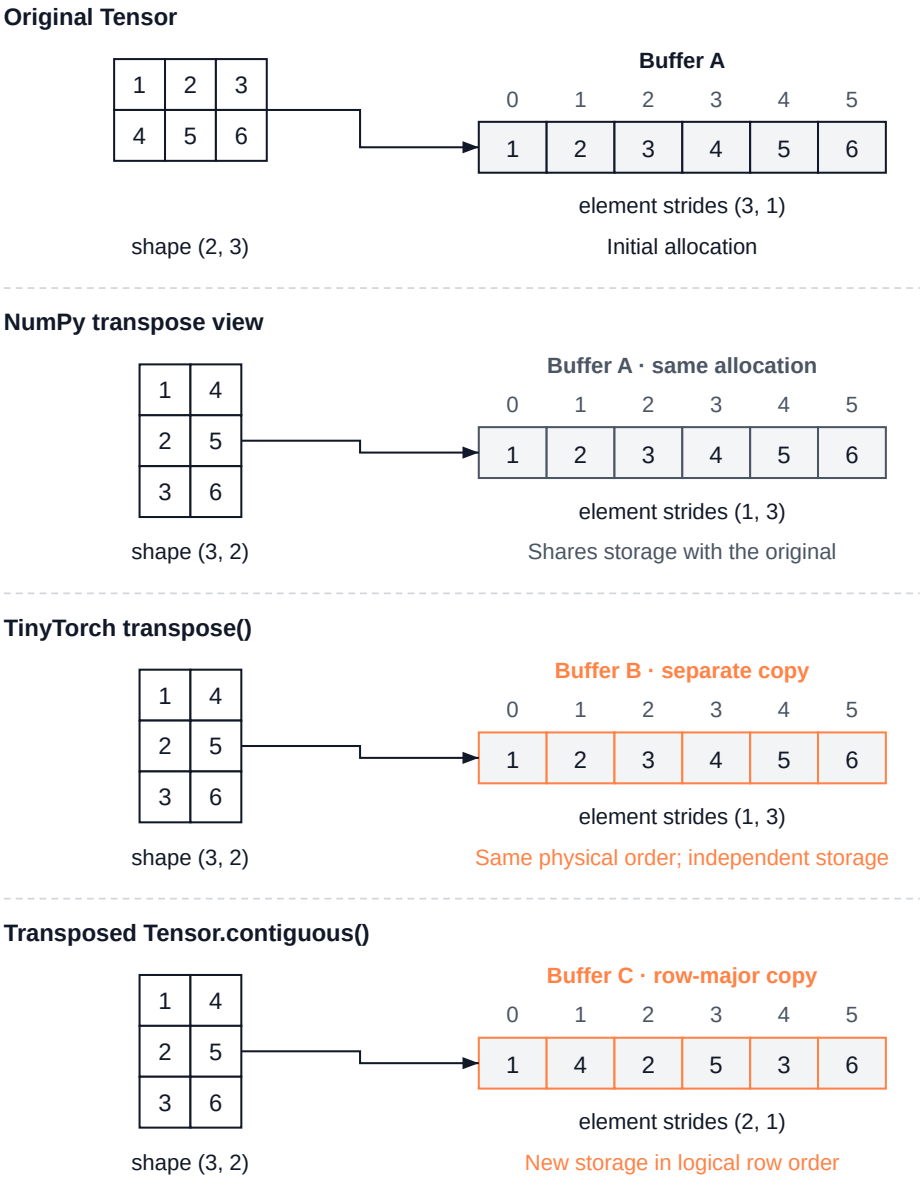


Figure 1.6: **Logical shape and physical storage**: NumPy transposes this array by sharing Buffer A with swapped strides. TinyTorch wraps the transpose in independent Buffer B with the same physical element order. The result of contiguous() uses Buffer C in logical row order.

The other shape operations follow the same path. `reshape` checks the requested element count and calls `Reshape.apply`; indexing calls `Slice.apply`; both wrap their array result in a new tensor. TinyTorch's `view` is an alias for `reshape`, so it does not promise the storage sharing that a production `view` API may promise. These methods preserve values and change their organization, while arithmetic such as `Add` changes values. Following each method to `apply` makes that distinction visible without reading the whole class at once.

1.4 A Worked Trace

The following script exercises the model and the module together on a 2×3 matrix. It needs the model functions from above and an installed `tinytorch` package.

```
import numpy as np
from tinytorch.core.tensor import Tensor

A = Tensor([[1.0, 2.0, 3.0],
            [4.0, 5.0, 6.0]])
print("A       ", A.shape, A.data.strides,
    A.data.flags["C_CONTIGUOUS"])
print("model   ", contiguous_strides(A.shape))

V = np.transpose(A.data)                        # what Permute.forward
    does
print("view    ", V.shape, V.strides, V.flags["C_CONTIGUOUS"])
print("model   ", transpose(A.shape, contiguous_strides(A.shape), 0,
    1))
print("V[1,0]  ", V[1, 0], " V[2,1]", V[2, 1])
print("shared  ", np.shares_memory(V, A.data))

C = np.ascontiguousarray(V)                     # what Copy.forward
    does
print("copy    ", C.ravel().tolist(), C.strides)

b = np.array([10.0, 20.0, 30.0], dtype=np.float32)
B = np.broadcast_to(b, (2, 3))                  # expose the reads used
    by broadcasting
print("bias    ", B.shape, B.strides, " B[1,2] =", B[1, 2])
print("model   ", broadcast_to(b.shape, contiguous_strides(b.shape),
    (2, 3)))

print("A + b   ", (A + Tensor(b)).data.tolist())
At = A.transpose()
print("A @ A.T ", (A @ At).data.tolist())
print("A.T     ", At.data.strides, At.data.flags["C_CONTIGUOUS"],
    np.shares_memory(At.data, A.data))
print("contig  ", At.contiguous().data.strides,
    At.contiguous().data.flags["C_CONTIGUOUS"])
```

```
A       (2, 3) (12, 4) True
model   (3, 1)
view    (3, 2) (4, 12) False
model   ((3, 2), (1, 3))
V[1,0]  2.0  V[2,1] 6.0
shared  True
copy    [1.0, 4.0, 2.0, 5.0, 3.0, 6.0] (8, 4)
bias    (2, 3) (0, 4)  B[1,2] = 30.0
model   ((2, 3), (0, 1))
A + b   [[11.0, 22.0, 33.0], [14.0, 25.0, 36.0]]
```

```
A @ A.T  [[14.0, 32.0], [32.0, 77.0]]
A.T      (4, 12) False False
contig   (8, 4) True
```

Every model line agrees with the NumPy output above it once byte strides are divided by four. In lines 4 to 7, the fresh array `A` has strides `(12, 4)`, matching `contiguous_strides` element strides $(3, 1)$ for 4-byte floats, and `A.data.flags["C_CONTIGUOUS"]` confirms it is contiguous. In lines 9 to 13, `V = np.transpose(A.data)` constructs a strided view with swapped strides `(4, 12)` (element strides $(1, 3)$) without touching buffer memory; line 13 confirms `np.shares_memory(V, A.data)` is `True`. In line 12, the mapping equation resolves `V[1, 0]` to physical slot $1 \times 1 + 0 \times 3 = 1$ (value `2.0`) and `V[2, 1]` to slot $2 \times 1 + 1 \times 3 = 5$ (value `6.0`). In lines 15 to 16, `np.ascontiguousarray` allocates a *second* physical buffer in which the transposed values sit in row order, `[1, 4, 2, 5, 3, 6]`, with the strides `(8, 4)` that the formula gives a $(3, 2)$ shape.

The bias trace in lines 18 to 21 is the stride-0 mechanism in the open. After padding, the vector has shape $(1, 3)$ and strides $(0, 1)$; stretching the first dimension from 1 to 2 leaves the zero in place, and NumPy reports the same thing as `(0, 4)` in line 20. Every coordinate of the $(2, 3)$ view resolves to one of three slots, as the repeated indices in table 1.2 show.

Table 1.2: Stride-0 broadcast of a length-3 vector to shape $(2, 3)$. Six coordinates, three slots, no copy.

Coordinate (i, j)	$i \times 0 + j \times 1$	Slot	Value
$(0, 0)$	0	0	`10.0`
$(0, 1)$	1	1	`20.0`
$(0, 2)$	2	2	`30.0`
$(1, 0)$	0	0	`10.0`
$(1, 1)$	1	1	`20.0`
$(1, 2)$	2	2	`30.0`

In line 23, `A + b` uses the same broadcast reads inside `Add.forward`, and the result `[[11, 22, 33], [14, 25, 36]]` shows each row receiving the same bias. In lines 24 to 25, the matrix product multiplies **A** by its own transpose, $\begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix}$, with `MatMul.forward` reading its right operand through the transposed strides and never noticing. Lines 26 to 27 trace the module's own transpose rather than the raw view. `A.transpose()` owns a separate buffer, so `shares_memory` is false in line 26, but that buffer keeps the transposed strides `(4, 12)` and is not contiguous; line 27 shows it takes `contiguous()` to produce the row-order `(8, 4)` layout. The first call copies for ownership; the second requests row order and wraps that result in another owned buffer.

1.5 From TinyTorch to Production

The transposed NumPy view in the trace illustrates an alternative to TinyTorch's ownership rule. PyTorch's transpose operations normally return views sharing the original

storage. Changes through such a view can therefore affect its base tensor. PyTorch's `view()` also requires shared storage and raises when the requested shape cannot be represented with compatible strides. `reshape()` may return either a view or a copy. Its `contiguous()` returns the input when it already has the requested layout, and copies otherwise. These contracts are described in the PyTorch tensor views documentation.

A copy becomes necessary when the layout a consumer requires differs from the one available. The cost is proportional to the amount of data moved, while changing a view's metadata does not copy the entries. Some numerical routines accept transposed inputs directly, so a transpose alone is not evidence that a copy is necessary. TinyTorch's `np.dot` calls already show this distinction. NumPy reads their operands through the available strides.

Layout directly governs hardware memory bandwidth utilization. Modern CPU architectures transfer data between main memory and execution units in fixed chunks, typically a 64-byte L1 cache line holding 16 single-precision float32 values. Traversing a 2D matrix in row-major sequential order accesses all 16 floats sequentially in each cache line before requesting the next line from memory, achieving 100 percent cache line utilization. Conversely, walking column-major across rows with large strides touches only a single 4-byte float per cache line before jumping to the next line, discarding the remaining 60 bytes and wasting 93.75 percent of the fetched memory bus bandwidth. We return to these physical cache-line dynamics quantitatively in Part III, measuring hardware cache misses against the Roofline model in Chapter 15 and implementing cache-tiled loops to restore sequential spatial locality in Chapter 18.

The ATen storage contract: In PyTorch's native C++ core (ATen), a `torch::Tensor` is an envelope around a `c10::TensorImpl`. The `TensorImpl` stores sizes, strides, and a storage offset pointing into a refcounted `c10::Storage` heap allocation:

```
[Tensor Envelope] → [TensorImpl: sizes, strides, offset] → [Storage: raw byte
↪  buffer]
```

Zero-copy view mutation: When code executes `t.transpose(0, 1)` or `t[::2]`, ATen creates a new `TensorImpl` pointing to the exact same `Storage` instance in $O(1)$ time, modifying only strides and offsets. No byte copies traverse PCIe or CPU memory buses.

The safety invariant: Because views share memory, an in-place mutation immediately mutates the underlying `Storage`. When an operation involves a stride-0 broadcasted tensor, PyTorch detects that multiple logical coordinates map to identical physical addresses and raises `RuntimeError: unsupported operation: more than one element of the written-to tensor refers to a single memory location`, preventing corrupted concurrent memory writes.

Table 1.3: Ownership and layout contracts at the Tensor boundary.

Operation	TinyTorch (Module 01)	PyTorch comparison
Storage and layout	An owned float32 NumPy array; `data.strides` uses bytes	Tensor metadata describes storage and element strides
Transpose	NumPy view followed by an owned Tensor copy	A view sharing storage
Broadcasted addition	Repeated input reads; allocated sum and wrapper copy	Repeated input reads; allocated sum
Contiguity check	`data.flags["C_CONTIGUOUS"]`	`.is_contiguous()`

Operation	TinyTorch (Module 01)	PyTorch comparison
Request row order	`Copy` requests row order; Tensor construction always copies	`.contiguous()` can return the input unchanged
Change shape	`reshape()` and `view()` both return owned copies	`view()` requires shared storage; `reshape()` may copy

The distinction in table 1.3 explains why similarly named methods can have different costs. Before inserting `contiguous()`, identify the required layout and inspect the existing one. Before writing through a result, establish whether it shares storage. A shape tells us where an entry belongs logically; strides tell us how to reach it; ownership tells us whether a write can affect another object.

1.6 Summary

Module 01 gives later code two contracts. A `Tensor` holds float32 data with a definite shape, and each operation reaches NumPy through `Function.apply`. NumPy's strides explain how transpose and broadcast views read existing storage; the wrapper explains why TinyTorch's returned tensors generally own fresh storage instead. The worked trace separates those questions with `data.strides` and `np.shares_memory`. This distinction lets us reason about a result's values, layout, and copying independently. The next step in Chapter 2 uses the same operation boundary to transform each value without changing its shape.

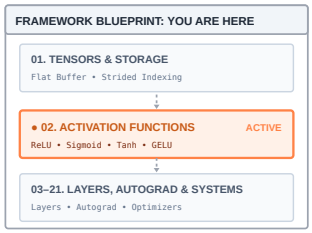
1.7 Further Reading

- **Harris et al.** (2020). "Array programming with NumPy." *Nature*. The design behind the strides, views, and broadcasting this chapter rests on.
- **Lawson et al.** (1979). "Basic Linear Algebra Subprograms for Fortran usage." *ACM TOMS*. The Level 1/2/3 split that explains why matrix multiplication is the operation the whole stack is tuned around.
- **Baydin et al.** (2018). "Automatic differentiation in machine learning: a survey." *JMLR*. Background for Chapter 6, including why reverse mode makes memory the binding constraint.

🔥 Chapter 2

Activation Functions: Non-Linearity and Numerical Stability

The core premise of deep learning is the ability to learn complex, non-linear representations from data. Yet if a neural network consists solely of linear transformations, matrix multiplications and bias additions, it suffers a fatal mathematical collapse: the composition of any number of linear maps is itself just a single linear map ($(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 = \mathbf{x}\mathbf{W}_{\text{eff}} + \mathbf{b}_{\text{eff}}$, with effective weight $\mathbf{W}_{\text{eff}} = \mathbf{W}_1\mathbf{W}_2$ and bias $\mathbf{b}_{\text{eff}} = \mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2$). Without non-linearity, a 100-layer deep network has no more expressive power than a single linear layer. To bend representation space and learn intricate decision boundaries, deep architectures require **activation functions**.



Framework Blueprint: Module 02 introduces non-linear activation functions that bend representation space between linear transformations.

In PyTorch, applying an activation function is a concise one-liner:

```python
import torch
import torch.nn.functional as F

x = torch.tensor([-2.0, -0.5, 0.0, 1.5])
relu_out = F.relu(x)
gelu_out = F.gelu(x)

print("ReLU:", relu_out)  # [0.0, 0.0, 0.0, 1.5]
print("GELU:", gelu_out)  # [-0.0455, -0.1543, 0.0, 1.3998]
```

Underneath that simple call, the runtime coordinates three distinct systems concerns:

1. **Elementwise 1D kernel execution:** The runtime dispatches an elementwise unary kernel across the contiguous float32 buffer, reading input values and writing transformed activations into a newly allocated output tensor of identical shape and strides.
2. **Memory-bandwidth saturation:** Because an activation performs only a few arithmetic operations per float read and written, its arithmetic intensity is low. On modern accelerators with fast FP32 units, the wall-clock execution time is dominated almost entirely by the memory bus bandwidth required to move bytes to and from DRAM.
3. **Execution contracts and activation lifetimes:** The runtime must allocate and manage output buffers for downstream consumption, while maintaining activation records that downstream optimization and automatic differentiation engines will require during training (Chapter 6).

This chapter builds the non-linear foundation of TinyTorch, implementing the core activation zoo (ReLU, Sigmoid, Tanh, GELU, and Softmax) through `Function.apply`.

Before building these operators, we must inspect the mathematical collapse that occurs when non-linearities are omitted.

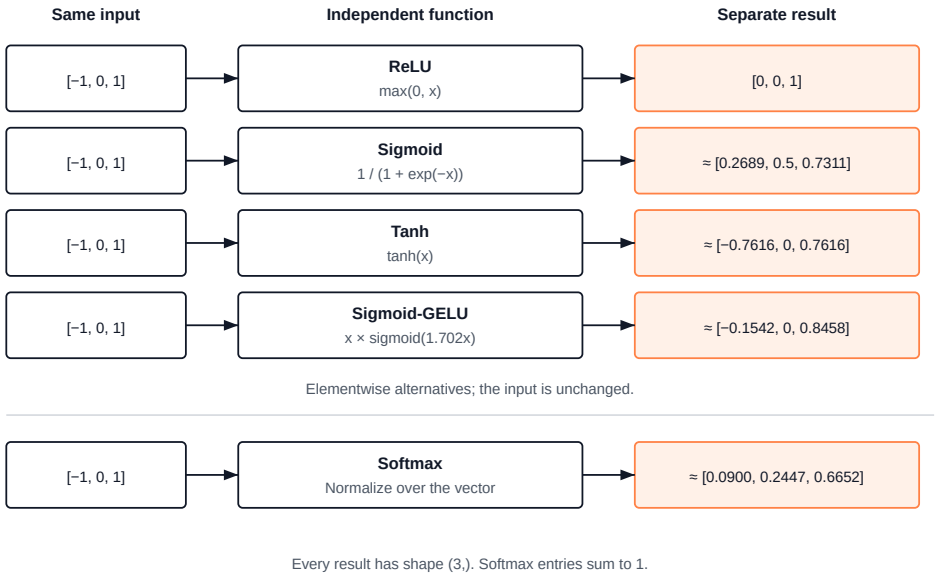


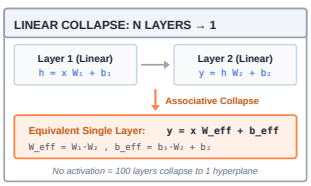
Figure 2.1: Independent activation results for the same input. The first four functions act element by element; Softmax normalizes across the vector. The GELU row shows TinyTorch's sigmoid approximation.

2.1 The Problem

↩ **Jump back:** Chapter 1 *Elementwise activation kernels operate directly over contiguous 1D memory buffers.*

With one input, consider a first layer that computes $h = 2x + 1$ and a second that computes $y = 3h - 2$. At $x = 1$, the intermediate value is $h = 3$ and the result is $y = 7$. Substitution gives $y = 6x + 1$, a single multiply-and-add with the same result. Adding the second layer changed the coefficients, but not the kind of function available.

The layer traditionally called linear includes a bias, so mathematically it is an affine map, a linear map plus a constant shift. It maps an input row vector $\mathbf{x}$ to $\mathbf{xW} + \mathbf{b}$, where $\mathbf{W}$ is a matrix of weights and $\mathbf{b}$ is a vector of biases. Stack two of them, feeding the output of the first into the second:



Stacking linear layers yields an equivalent single linear layer ($\mathbf{W}_{eff} = \mathbf{W}_1\mathbf{W}_2$), collapsing model depth into a single flat hyperplane.

$$\mathbf{h} = \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1, \qquad \mathbf{y} = \mathbf{h}\mathbf{W}_2 + \mathbf{b}_2$$

Substitute the first equation into the second and multiply out:

$$\mathbf{y} = (\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 = \mathbf{x}\,(\mathbf{W}_1\mathbf{W}_2) + (\mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2)$$

The right-hand side has the form $\mathbf{x}\mathbf{W}_{\text{eff}} + \mathbf{b}_{\text{eff}}$ with $\mathbf{W}_{\text{eff}} = \mathbf{W}_1\mathbf{W}_2$ and $\mathbf{b}_{\text{eff}} = \mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2$. That is a single linear layer. Nothing in the argument depends on there being exactly two layers. Apply it again and a third layer folds into the same form, and so on for a hundred. Matrix multiplication is associative, so the product of any number of weight matrices is just another weight matrix.

Here is the collapse in code, using the `Tensor` from Chapter 1 with its `@` and `+`, and two-element vectors so that every number can be checked by hand. The input and the biases are written as 1×2 matrices (one row) so that `@` applies to them directly.

```
from tinytorch.core.tensor import Tensor

x = Tensor([[1.0, 2.0]])
W1, b1 = Tensor([[2.0, 0.0], [1.0, 3.0]]), Tensor([[0.5, -0.5]])
W2, b2 = Tensor([[1.0, 2.0], [3.0, 1.0]]), Tensor([[1.0, 0.0]])

# Two layers, one after the other.
h = x @ W1 + b1
y_deep = h @ W2 + b2

# One layer, with the weights folded together ahead of time.
W_eff = W1 @ W2
b_eff = b1 @ W2 + b2
y_folded = x @ W_eff + b_eff

print("two layers:", y_deep.data.tolist())    # [[22.0, 14.5]]
print("one layer: ", y_folded.data.tolist())  # [[22.0, 14.5]]
print("W_eff:", W_eff.data.tolist(), " b_eff:", b_eff.data.tolist())
```

The two paths agree exactly for these small values. Algebraically the equivalence holds for every input; floating-point evaluations can differ because folding changes the order of arithmetic. In table 2.1 the intermediate values sit side by side so we can confirm the fold by hand. The middle row shows what the second layer receives; the last row shows that the folded weights $\mathbf{W}_{\text{eff}} = \begin{bmatrix} 2 & 4 \\ 10 & 5 \end{bmatrix}$ and $\mathbf{b}_{\text{eff}} = [0, 0.5]$ land on the same answer in one step.

Table 2.1: Two linear layers in sequence vs. the single layer they fold into, on the input $\mathbf{x} = [1, 2]$.

Path	Input	Weights, bias	Output
Layer 1	$\mathbf{x} = [1, 2]$	$\mathbf{W}_1, \mathbf{b}_1$	$\mathbf{h} = [4.5, 5.5]$
Layer 2	$\mathbf{h} = [4.5, 5.5]$	$\mathbf{W}_2, \mathbf{b}_2$	$\mathbf{y} = [22, 14.5]$
Folded	$\mathbf{x} = [1, 2]$	$\mathbf{W}_1\mathbf{W}_2,\ \mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2$	$\mathbf{y} = [22, 14.5]$

The reason this hurts is what a single linear layer cannot do. Take the four points of the XOR truth table, $(0, 0)$ and $(1, 1)$ labeled 0, $(0, 1)$ and $(1, 0)$ labeled 1, and ask a linear function $f(x_1, x_2) = ax_1 + bx_2 + c$ to score the 1s above some threshold t and the 0s below it. Adding the two "below" conditions gives $a + b + 2c < 2t$; adding the two "above" conditions gives $a + b + 2c > 2t$. Both cannot hold, so no linear function separates the points, and by the collapse above no *stack* of linear functions can either. A hundred layers and a million weights are no better than a single line through the plane. Synthesis I revisits this example with a two-layer network that solves it, once we have the missing piece.

2.2 The Idea

The missing piece is a function σ that is applied to every element of a layer's output and is not linear. With it between the layers,

$$\mathbf{h} = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1), \qquad \mathbf{y} = \mathbf{h}\mathbf{W}_2 + \mathbf{b}_2$$

and the substitution trick fails, because $\sigma(\mathbf{x}\mathbf{W}_1)\mathbf{W}_2$ cannot be rewritten as $\mathbf{x}$ times some matrix. A one-element example makes the point. With ReLU, the function that replaces negatives by zero, $\text{ReLU}(-3 \times -2) = 6$ while $\text{ReLU}(-3) \times (-2) = 0$. The function does not commute with multiplication, so it cannot be pushed through the weights and absorbed. Each layer now bends the space it receives before the next layer draws a line through it, which is how a network builds a boundary that one linear layer cannot represent. ReLU networks produce boundaries made from straight pieces; smooth activations can produce smooth curves.

To see this folding in action, consider a two-neuron hidden layer with weights and biases:

$$\mathbf{W}_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{b}_1 = [0, -1]$$

Operating on an input row $\mathbf{x} = [x_1, x_2]$, the affine step produces $[x_1 + x_2, x_1 + x_2 - 1]$. Applying ReLU yields hidden features $\mathbf{h} = [h_1, h_2] = [\max(0, x_1 + x_2), \max(0, x_1 + x_2 - 1)]$. Let us trace the four XOR truth-table inputs into this folded space:

- $(0, 0) \to [\max(0, 0), \max(0, -1)] = [0, 0]$ (Class 0)
- $(1, 0) \to [\max(0, 1), \max(0, 0)] = [1, 0]$ (Class 1)
- $(0, 1) \to [\max(0, 1), \max(0, 0)] = [1, 0]$ (Class 1)
- $(1, 1) \to [\max(0, 2), \max(0, 1)] = [2, 1]$ (Class 0)

Notice the spatial collapse: the two off-diagonal points $(1, 0)$ and $(0, 1)$ map to the identical location $[1, 0]$, while $(0, 0)$ remains at the origin and $(1, 1)$ moves to $[2, 1]$. As shown in figure 2.2, a single downstream linear decision boundary $h_1 - 2h_2 = 0.5$ now cleanly partitions the positive class $[1, 0]$ (evaluating to $1 - 0 = 1.0 > 0.5$) from both negative classes $[0, 0]$ (evaluating to $0.0 < 0.5$) and $[2, 1]$ (evaluating to $2 - 2(1) = 0.0 < 0.5$). The hinge of the ReLU physically folded the 2D plane along the $x_1 + x_2 = 1$ crease, untangling an inseparable topological configuration.

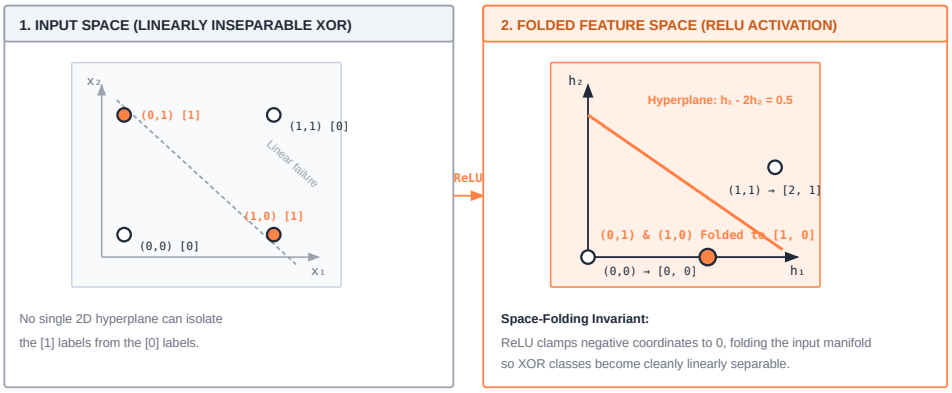
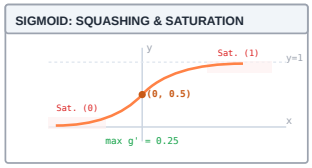


Figure 2.2: **Space folding with ReLU**: Linear layers in sequence collapse into a single affine transformation unable to partition linearly inseparable patterns such as XOR. An intervening elementwise $\text{ReLU}(z) = \max(0, z)$ folds the negative quadrant onto the coordinate axes, bending the representation space so a downstream hyperplane can separate classes.

Breaking the collapse is only the first requirement. The function also has to be inexpensive to evaluate across a whole array and responsive enough that changing the incoming values can change the result. A nearly flat function loses that responsiveness. Chapter 6 will turn this observation into derivatives and gradients, the quantities used to decide how weights should change during training. Here the output values and the shapes of the curves are enough to compare the choices in table 2.2.

Table 2.2: Four elementwise choices. Here σ denotes sigmoid; Softmax is discussed separately because it couples entries.

Activation	Rule	Output behavior	Limitation
Sigmoid	$1/(1+e^{-x})$	Squashes toward 0 and 1	Nearly flat in both tails
Tanh	$\tanh(x)$	Squashes toward -1 and 1	Nearly flat in both tails
ReLU	$\max(0, x)$	Keeps positives, zeros negatives	Discards every negative input
GELU (Module 02)	$x\,\sigma(1.702x)$	Smooth gate; small negatives can pass	More arithmetic than ReLU; an approximation to exact GELU



Sigmoid squashes inputs to $(0, 1)$ with maximum gradient 0.25 at origin, saturating in extreme tails.

2.2.1 Sigmoid and saturation

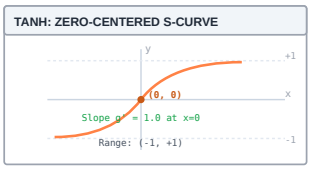
Historically the standard hidden-layer activation in early neural networks, the **sigmoid** function remains indispensable in modern deep learning for binary classification output heads and gating mechanisms inside recurrent architectures (such as LSTMs and GRUs). By squashing unbounded pre-activations into a bounded probability range, it maps zero to 0.5, -1 to about 0.269, and 1 to about 0.731. Its mathematical range is $(0, 1)$,

useful when an output represents a probability. The endpoints are approached but never reached in exact arithmetic; float32 can round sufficiently extreme results to exactly 0 or 1.

At the ends of the curve, large input changes produce small output changes. For example, increasing the input from 5 to 6 moves the output only from about 0.9933 to 0.9975. This flattening is called **saturation**.[5] At extreme inputs, float32 evaluations of exponential functions also encounter severe hardware limits. In single-precision IEEE 754 float32, the maximum finite positive value is $(2 - 2^{-23}) \times 2^{127} \approx 3.4028235 \times 10^{38}$. Because $\ln(3.4028235 \times 10^{38}) \approx 88.72284$, any unshielded evaluation of e^x for $x > 88.72$ overflows to $+\infty$. Symmetrically, below $x \approx -87.34$ the result of e^x falls under the smallest normal float32 value and becomes subnormal, and below about -104 it underflows to exactly 0.0, causing downstream unshielded logarithms to produce $-\infty$ or NaN. Chapter 6 explains why saturated activations can weaken the signal used to update earlier weights, a problem called vanishing gradients. Saturation is already visible in the forward values, without implementing training.

[5] In IEEE 754 single-precision float32, the subnormal minimum is $2^{-149} \approx 1.4 \times 10^{-45}$ and the normal minimum is $2^{-126} \approx 1.175 \times 10^{-38}$. Some accelerator code paths enable Flush-To-Zero (FTZ), for example CUDA builds with fast-math, and treat subnormals as zero; under FTZ, e^x becomes 0.0 below about -87.34. NumPy on a CPU keeps subnormals, so TinyTorch reaches exact zero only below about -104.

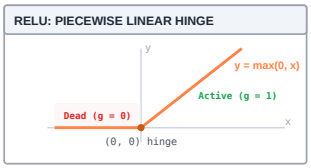
2.2.2 Tanh



Tanh provides a zero-centered $(-1, 1)$ range with unit gradient at origin, retaining saturation cliffs in extreme tails.

Prominent in recurrent neural networks (such as LSTM and GRU hidden state updates) and energy-based models, the **hyperbolic tangent** improves upon sigmoid by offering zero-centered outputs that prevent systematic directional bias during gradient updates. Algebraically, it rescales sigmoid according to $\tanh(x) = 2\sigma(2x) - 1$. It maps zero to zero, -1 to about -0.762, and 1 to about 0.762. Unlike sigmoid, it can pass both signs to the next layer. Its mathematical range is $(-1, 1)$, although float32 can round to either endpoint. Changing the range does not remove saturation; at large positive or negative inputs this curve also becomes nearly flat. A symmetric function alone does not guarantee that a particular batch of outputs has mean zero.

2.2.3 ReLU



ReLU eliminates negative saturation via a hard zero hinge, preserving gradient flow ($g = 1$) for positive inputs.

The breakthrough activation that unlocked modern deep networks—popularized in landmark vision architectures like AlexNet and ResNet—is the **rectified linear unit (ReLU)**. By replacing expensive transcendental exponentials with a simple threshold and preserving a constant unit gradient for positive values, ReLU allowed networks to scale to hundreds of layers without vanishing gradients. Mathematically, it gives up on squashing: $\text{ReLU}(x) = \max(0, x)$. Positive inputs pass through unchanged and negative inputs become zero. Increasing a positive input from 5 to 6 therefore increases the output by exactly 1. Increasing a negative input from -6 to -5 changes nothing. The implementation needs only an elementwise maximum with zero.

The zero branch has a cost. If every example gives a unit a negative input, its output carries no information about changes within that branch. Such a unit is called a **dead unit**. Chapter 6 will explain when it receives no learning signal; changes elsewhere in the network can still change its inputs, so a currently inactive unit is not necessarily permanently inactive. If inputs are distributed symmetrically around zero, ReLU also sets about half of them to zero. Chapter 3 uses this effect to reason about the scale of a layer's starting weights.

Mechanism: If a layer's weights and bias project representations entirely into negative space across a dataset, the unit outputs 0.0 everywhere. Because the output remains constant zero regardless of input variation, all downstream layers receive zero information

from this feature, collapsing the effective representation capacity of the network. When training begins (Chapter 6), a zero slope on the negative branch means upstream weights receive no corrective learning signals.

Invariant check: Saturated units (in sigmoid/tanh) or dead units (in ReLU) collapse effective model capacity. Tracking the percentage of completely inactive activations across a batch provides an immediate diagnostic for representation collapse.[6]

[6] To prevent dead units, Maas et al. (2013) proposed Leaky ReLU ($\max(\alpha x, x)$ with $\alpha = 0.01$) and Clevert et al. (2015) introduced the Exponential Linear Unit (ELU), ensuring a nonzero derivative across negative inputs.

2.2.4 GELU

Standard across modern Transformer architectures—including BERT, GPT, and Vision Transformers—the **Gaussian error linear unit (GELU)** bridges deterministic non-linear activation with stochastic regularization. By smoothly gating inputs rather than abruptly truncating them at zero, it avoids the dead-neuron pathology of ReLU while maintaining nonzero gradient flow across small negative pre-activations. A gate is a multiplier that controls how much of a value passes through. ReLU uses a hard gate, either 0 or 1. GELU replaces it with a smooth multiplier. The exact definition is $x\Phi(x)$, where $\Phi(x)$ is the fraction of a standard normal bell curve lying to the left of x. That bell curve is centered at zero with standard deviation one; $\Phi(0) = 0.5$ and $\Phi(1) \approx 0.8413$. In contrast to stochastic regularizers like Dropout that randomly zero activations with Bernoulli trials during training, GELU gates values deterministically by their cumulative probability: each activation is weighted by the likelihood that standard Gaussian noise falls below it ($x \cdot P(X \leq x)$). This probability interpretation defines the shape of the smooth gate, but applying GELU does not draw a random sample; it is an entirely deterministic mathematical function.

TinyTorch approximates the gate with the sigmoid already introduced:

$$\text{GELU}_{\text{TinyTorch}}(x) = x\,\sigma(1.702x).$$

At $x = -1$, the gate is about 0.1542, giving an output of -0.1542. At $x = 1$, it is about 0.8458, giving 0.8458. Negative inputs can therefore affect the next layer, unlike ReLU's zero branch. Very negative inputs still produce outputs close to zero, so smoothness does not promise strong responsiveness everywhere. The approximation also differs from exact GELU, whose outputs at -1 and 1 are about -0.1587 and 0.8413.[7] The worked trace follows the sigmoid form that the code actually computes.

[7] The scaling factor $1.702 \approx \sqrt{8/\pi} \times 1.066$ is the classical logistic coefficient that minimizes the uniform Chebyshev discrepancy $\sup_{x\in\mathbb{R}} |\sigma(c \cdot x) - \Phi(x)|$, guaranteeing that the sigmoid gate deviates from the standard Gaussian cumulative normal distribution by less than 0.0095 across the entire real line.

2.2.5 Which activation goes last

Hidden layers need a nonlinearity. The last layer needs whatever form the loss expects, which depends on the task more than on any activation's merits.

- **Regression.** Leave the output linear. A target can be any real number, and Chapter 4's mean squared error compares raw outputs with it.
- **One yes-or-no label.** End with a sigmoid, so the output is one probability in $(0, 1)$. TinyTorch's `BinaryCrossEntropyLoss` expects exactly that, and clips it away from 0 and 1 before taking logarithms.
- **One label among many.** Return the raw scores, the **logits**, and let the loss normalize them. TinyTorch's `CrossEntropyLoss` applies a max-shifted log-softmax internally, which stays finite where exponentiating first would overflow. Apply softmax yourself only when a caller needs probabilities, at inference time.

The third rule is the one that fails quietly. A model that ends in softmax and trains with `CrossEntropyLoss` runs without an error,[8] because the loss internally applies

[8] This represents one of the most common silent failure modes in beginner PyTorch pipelines. Because PyTorch's `nn.CrossEntropyLoss` internally composes `LogSoftmax` and `NLLLoss`, feeding it softmax probabilities evaluates $\log(\text{softmax}(p))$, squashing the logit differences the loss can see and capping how low the loss can go.

`LogSoftmax` and treats those probabilities as raw logits (as detailed in Chapter 4). Every score it receives then lies strictly in $(0, 1)$, so the correct class can never pull far ahead in exponentiated space. With ten classes, the best loss the model can ever achieve is $-\ln\left(e/(e+9)\right) \approx 1.46$, however well the network separates the inputs. In addition, computing Softmax twice creates redundant memory round-trips and distorts backward gradients, corrupting the learning trajectory without raising a single exception.

2.3 The Code

2.3.1 Two classes per activation

A layer might turn a batch of two examples into a 2×3 array. Applying `ReLU()` to that array changes its values but leaves the next layer with the same two rows and three columns. The same rule works on a vector or a larger array because each output depends only on the input at that position.

Module 02 writes each activation as a pair of classes, and the pair is the pattern every later module follows. The first is an *operation*, a subclass of Chapter 1's `Function` whose `forward` takes a NumPy array and returns one. Chapter 6 will teach the framework to retain this object as a record of the call and add a `backward` method for computing gradients. The second is a *layer*, a plain class that Chapter 3's containers can hold: it reports no parameters, its `forward` takes a `Tensor` and hands it to the operation through `apply`, and `__call__` makes `y = f(x)` work. Here is the sigmoid pair, the layer first.

>_ SOURCE CODE MAPPING MOD 02

FILE: src/02_activations/02_activations.py
EXPORT: tinytorch.core.activations
SYMBOLS: ReLU, Sigmoid, GELU, Softmax

Extracted & verified at build time ✓ tested

Source Code Mapping: Module 02 extracts elementwise activations (ReLU, Sigmoid, Tanh, GELU) and Softmax normalization directly from the working repository.

```python
class Sigmoid:
    """
    Sigmoid activation: σ(x) = 1/(1 + e^(-x))
    """

    def parameters(self):
        """No weights to train, so Module 03's layers and Module 07's optimizers will have nothing to collect."""
        return []

    def forward(self, x: Tensor) -> Tensor:
        """Apply Sigmoid through its operation, so Module 06 will be able to record it for gradients."""
        return SigmoidFunction.apply(x)

    def __call__(self, x: Tensor) -> Tensor:
        """Allows the activation to be called like a function."""
        return self.forward(x)
```

Three things to notice. `parameters()` returns an empty list because Chapter 3's containers will ask every component for its learnable parameters, and an activation has none. `forward` contains no arithmetic at all; `SigmoidFunction.apply` unwraps the tensor, runs the operation on the array, and wraps the result in a fresh `Tensor`, exactly as Chapter 1 showed for `Add`. Neither class examines the shape. An elementwise function does not care whether the array is a vector or a four-dimensional batch of images, and NumPy walks whatever strides the array has, so a transposed input needs no special handling.

↪ **Jump ahead:** Chapter 18 *Operator fusion combines activation arithmetic with linear layers, slashing DRAM round-trips.*

2.3.2 The four operations

With the wrapping settled, each operation's `forward` is its formula written on an array. ReLU and tanh are one NumPy call each.

```
class ReLUFunction(Function):
    """
    The ReLU operation. forward() works on NumPy arrays; Module 06
    adds backward().
    """
    def forward(self, x):
        """
        Apply ReLU activation element-wise.
        """
        # Apply ReLU: max(0, x)
        result = np.maximum(0, x)
        return result
```

```
class TanhFunction(Function):
    """
    The Tanh operation. forward() works on NumPy arrays; Module 06
    adds backward().
    """
    def forward(self, x):
        """
        Apply tanh activation element-wise.
        """
        # Apply tanh using NumPy
        result = np.tanh(x)
        return result
```

Sigmoid needs an additional numerical safeguard. The formula $1/(1+e^{-v})$ asks for e^{-v}, and a hidden unit's input can be any size at all, including large magnitudes when the initial weights have a poor scale. Feed it $v = -1000$ and the naive transcription computes e^{1000}, far beyond the largest number a 64-bit float can hold. Python's `math.exp` raises `OverflowError` on the spot. NumPy returns `inf` with a warning and then evaluates $1/(1+\infty) = 0$, which hides an overflowing intermediate behind a plausible final value. Tests that raise on overflow expose the problem. Here is the trick. The same function has a second form, $e^{v}/(1+e^{v})$, whose exponent is $+v$, so it overflows for large positive v instead. Choose whichever form keeps the exponent non-positive, the first when $v \geq 0$ and the second when $v < 0$, and neither branch can overflow for any input.

```
class SigmoidFunction(Function):
    """
    The Sigmoid operation. forward() works on NumPy arrays; Module 06
    adds backward().
    """
    def forward(self, x):
```

```
        """
        Apply sigmoid activation element-wise.
        """
        # Both branches use the same bounded exponential, so neither
        ↪ overflows.
        z = np.exp(-np.abs(x))
        result = np.where(x >= 0, 1.0 / (1.0 + z), z / (1.0 + z))
        return result
```

The crucial line computes `z = exp(-abs(x))`. Every exponent is nonpositive, so `z` stays between zero and one. For nonnegative inputs, the result is `1 / (1 + z)`; for negative inputs it is `z / (1 + z)`. `np.where` selects between these results per element. NumPy evaluates both candidate arrays before selecting, but both are safe because they share the bounded `z`. This implementation does not need to silence overflow in a discarded sigmoid branch. At zero either division gives 0.5; at -1000, `z` rounds to zero and the negative branch returns zero safely.

GELU is defined as $x\,\Phi(x)$, where the Gaussian fraction can be computed using Python's `math.erf`, the error function. Module 02 instead approximates that fraction. It uses the approximation $x\,\sigma(1.702x)$, the sigmoid form, because it reuses the operation just written and expresses the gate using NumPy array operations.

```
class GELUFunction(Function):
    """
    The GELU operation. forward() works on NumPy arrays; Module 06
    ↪ adds backward().
    """
    def forward(self, x):
        """
        Apply GELU activation element-wise.
        """
        # Overflow in the scaled gate means saturation, which sigmoid
        ↪ handles.
        with np.errstate(over="ignore"):
            sig = SigmoidFunction().forward(1.702 * x)
        with np.errstate(invalid="ignore"):
            out = x * sig
        if np.any(np.isneginf(x)):
            out = np.where(np.isneginf(x), 0.0, out)
        return out
```

This is a different numerical function from the exact GELU. Its approximation error and derivative must therefore be checked against the sigmoid form actually implemented. Notice that the call is `SigmoidFunction().forward`, the array-level method, not `Sigmoid()` on a tensor. Inside an operation everything is NumPy; tensor wrapping happens at the boundary that `apply` guards. The temporary `SigmoidFunction` call is array arithmetic inside GELU, not a separate Tensor operation. The `errstate` around scaling allows extreme finite inputs to reach infinite gate arguments, where sigmoid saturates safely. The second guard handles the otherwise invalid product $-\infty \times 0$, and the final `where` supplies its intended limiting output, zero.

For `y = Sigmoid()(x)`, read the calls in order: `Sigmoid.__call__` delegates to `Sigmoid.forward`, which selects `SigmoidFunction.apply`; `apply` passes `x.data` to the operation and constructs `y` from its result. The wrapper decides which operation runs, the operation decides the numerical formula, and `apply` decides how arrays cross the tensor boundary. A test can check the formula directly on an array, then check shape preservation and unchanged input through the public wrapper. These are different failure points, so one test should not stand in for both.

2.3.3 Calling one

Chapter 3's `Sequential` will call an activation exactly the way it calls a layer, `y = f(x)`. Before that container relies on the interface, we can check both values and ownership. Transposing a 2×3 input changes its logical shape to $(3, 2)$; the activation must follow those new positions rather than flattening the old row order. NumPy handles the array strides. TinyTorch's transpose then wraps its result in independently owned storage, so the transposed Tensor is not a zero-copy alias of the original.

The example builds a 2×3 tensor, applies `ReLU`, and then applies it again to the transpose.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.activations import ReLU

x = Tensor([[-3.0, -1.0, -0.75],
            [ 0.0,  1.0,  3.0]])

relu = ReLU()
y = relu(x)
print(y.shape, y.data.tolist())           # (2, 3) [[0.0, 0.0, 0.0],
↪   [0.0, 1.0, 3.0]]

xt = x.transpose()                       # shape (3, 2), independent
↪   storage
yt = relu(xt)
print(yt.shape, yt.data.tolist())         # (3, 2) [[0.0, 0.0], [0.0,
↪   1.0], [0.0, 3.0]]
print(x.data.tolist())                    # input untouched
assert not np.shares_memory(x.data, xt.data)
assert not np.shares_memory(xt.data, yt.data)
```

The transposed call's output is the transpose of the first output, and neither activation call changes its input. The assertions check ownership separately from numerical equality. The return contract is a fresh Tensor with the same shape, containing transformed values and sharing no storage with the input. NumPy first computes the result array, then `apply` constructs the Tensor's own copy. That copy makes ownership simple, but it also adds memory traffic.

2.3.4 Softmax couples a row

Ubiquitous in multi-class classification heads and Transformer attention mechanisms (softmax$(QK^T/\sqrt{d})$), the **Softmax** function maps unbounded vectors of real scores into valid categorical probability distributions. By coupling all entries along an axis through an

exponential sum, it enables competitive normalization where increasing the probability of one candidate necessarily depresses the others. For scores `[1, 2, 3]`, Softmax returns approximately `[0.0900, 0.2447, 0.6652]`. The outputs are nonnegative and sum to one, so they can represent relative probabilities among three choices. Unlike sigmoid, increasing one score changes the other outputs too because every entry shares the same denominator.

`Softmax.forward(x, dim=-1)` calls `SoftmaxFunction.apply(x, dim=dim)`. The `dim` argument becomes an attribute of that operation instance; it chooses which axis to normalize. For an input with shape $(2, 3)$ and `dim=-1`, the operation handles each row independently. Its maximum and sum arrays have shape $(2, 1)$ because `keepdims=True` retains the reduced axis. NumPy can then broadcast those values across their original rows.

Core Invariant: Safe Exponentiation

$\max(z)$ subtraction keeps $z_i - m \leq 0$

Exponentials never exceed $e^0 = 1.0$, preventing float32 overflow cliff.

```python
class SoftmaxFunction(Function):
    """
    The Softmax operation. forward() works on NumPy arrays; Module 06 adds backward().
    """
    dim = -1

    def forward(self, x):
        """
        Apply softmax activation along specified dimension.
        """
        # Numerical stability: subtract max to prevent overflow
        x_max = np.max(x, axis=self.dim, keepdims=True)
        safe_max = np.where(np.isneginf(x_max), 0.0, x_max)
        # An extreme negative difference may overflow to -inf, whose exp is zero.
        with np.errstate(over="ignore"):
            x_shifted = x - safe_max

        # Compute exponentials
        exp_values = np.exp(x_shifted)

        # Sum along dimension
        exp_sum = np.sum(exp_values, axis=self.dim, keepdims=True)
        safe_sum = np.where(exp_sum == 0, 1.0, exp_sum)

        # Normalize to get probabilities
        result = exp_values / safe_sum
        return result
```

The formula exponentiates each score and divides by the row's exponential sum. To avoid asking for an enormous exponential, the code subtracts the row maximum first. For `[1, 2, 3]`, the shifted scores are `[-2, -1, 0]`, the exponentials are approximately `[0.1353, 0.3679, 1]`, and their sum is `1.5032`. Division gives the probabilities above. Subtracting a common constant $m = \max_k x_k$ does not change the normalized ratios, since its exponential factor cancels between numerator and denominator:

$$\frac{e^{x_i-m}}{\sum_j e^{x_j-m}} = \frac{e^{x_i}e^{-m}}{\sum_j (e^{x_j}e^{-m})} = \frac{e^{-m}e^{x_i}}{e^{-m}\sum_j e^{x_j}} = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

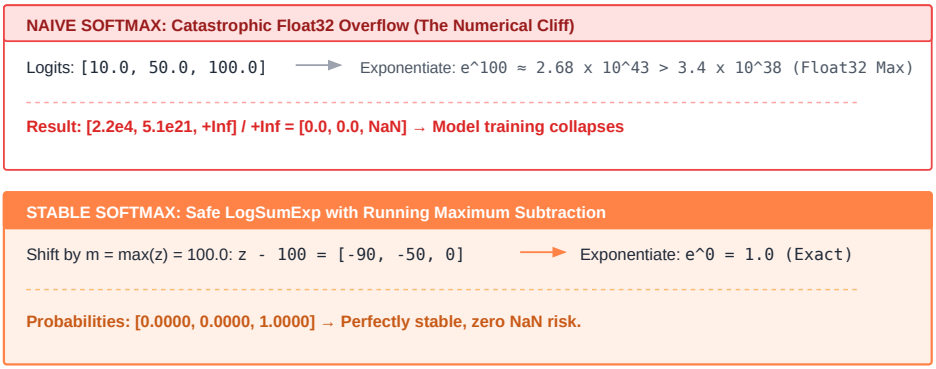


Figure 2.3: **The Softmax numerical cliff**: Naive exponentiation of large logits triggers float32 overflow ($> 3.4 \times 10^{38}$), returning NaN and collapsing training. Subtracting the running maximum shifts exponents into the non-positive domain (≤ 0), ensuring finite, stable probability evaluation.

A difference between extreme finite scores may round to negative infinity; `errstate` permits that result because its exponential is correctly zero.

A score of `-inf` excludes an entry because its exponential is zero. For a fully excluded row, the maximum is also `-inf`. The `safe_max` guard replaces that maximum with zero before subtraction, avoiding `-inf - -inf`. The resulting exponentials are all zero; `safe_sum` replaces their zero sum with one so division leaves zeros. This row means no available choices, not a probability distribution. Chapter 12 will use excluded entries for attention masks. Positive infinity and NaN scores are outside this contract.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.activations import Softmax

scores = Tensor([[1.0, 2.0, 3.0],
                 [-np.inf, -np.inf, -np.inf]])
probabilities = Softmax()(scores, dim=-1)
np.testing.assert_allclose(probabilities.data[0],
                           [0.09003057, 0.24472847, 0.66524096],
    atol=1e-7)
np.testing.assert_array_equal(probabilities.data[1], [0.0, 0.0, 0.0])
assert probabilities.shape == scores.shape
```

2.4 A Worked Trace

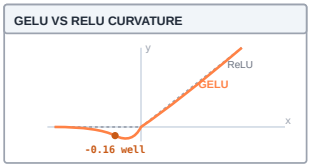
A useful trace follows the intermediate value that selects the output. Sigmoid forms a bounded exponential; GELU forms a sigmoid gate and multiplies by the original input.

The following program sends six values through the array-level operations, just as `apply` does. Every array has shape `(6,)`.

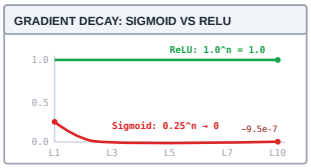
```
1  import numpy as np
2  from tinytorch.core.activations import SigmoidFunction, ReLUFunction,
   ↪   GELUFunction
3
4  v = np.array([-3.0, -1.0, -0.75, 0.0, 1.0, 3.0], dtype=np.float32)
5  s = SigmoidFunction().forward(v)
6  r = ReLUFunction().forward(v)
7  gate = SigmoidFunction().forward(1.702 * v)
8  g = GELUFunction().forward(v)
9  np.testing.assert_allclose(g, v * gate)
10
11 print("    x  sigmoid   relu  GELU gate     GELU")
12 for i in range(len(v)):
13     print(f"{v[i]:5.2f} {s[i]:8.4f} {r[i]:6.2f}"
14           f" {gate[i]:10.4f} {g[i]:8.4f}")
```

In line 4, `v` defines six sample points spanning negative, zero, and positive domains. Lines 5 to 6 compute standard Sigmoid and ReLU activations via their `forward` methods. In lines 7 to 8, `gate` evaluates the sigmoid factor $\sigma(1.702x)$ and `g` computes the complete GELU activation; line 9 validates the identity $g = v \cdot \text{gate}$ with `assert_allclose`.

In table 2.3 (printed by lines 11 to 14), the GELU output is the product of the first and fourth columns. At -1, a small positive gate (0.1542) lets a small negative value through (-0.1542); at zero, the gate is 0.5 but the product is still zero. These are consequences of the multiplication, not special cases added to the function.



TinyTorch's sigmoid-form GELU dips to about -0.164 near x = -0.75 (exact GELU: -0.170) before approaching zero.



Sigmoid's maximum slope (0.25) compounds across 10 layers to 9.5×10^{-7}, causing vanishing gradients. ReLU maintains unit slope (1.0), preserving signal flow.

Table 2.3: Forward values and the intermediate gate in Module 02's sigmoid-form GELU.

Input x	Sigmoid	ReLU	GELU gate $\sigma(1.702x)$	GELU output
-3.00	0.0474	0	0.0060	-0.0181
-1.00	0.2689	0	0.1542	-0.1542
-0.75	0.3208	0	0.2181	-0.1636
0.00	0.5000	0	0.5000	0.0000
1.00	0.7311	1	0.8458	0.8458
3.00	0.9526	3	0.9940	2.9819

The trace also distinguishes a formula from its name. Exact GELU at -1 is approximately -0.1587, while the implementation gives -0.1542. A regression test should compare to the sigmoid form actually chosen. A test against the exact form needs an explicit approximation tolerance.

There is a short preview of learning in these values. A local slope measures how much the output changes per small input change. Sigmoid's largest slope is 0.25; ReLU's positive branch has slope 1 and its negative branch has slope zero. Ten factors of 0.25 multiply to about 9.5×10^{-7}, while ten factors of 1 remain 1. This isolates the activations' contribution to shrinking a change as it travels through a computation. A complete network also contains weights. Chapter 6 introduces derivatives and tracks all those

factors together, rather than treating this activation-only calculation as a prediction of a network's gradient.

2.5 From TinyTorch to Production

A ReLU on six float32 values produces a 24-byte result. NumPy reads the input and writes its result array; TinyTorch then copies that result into the returned Tensor. Scaling this same interface to millions of elements makes ownership and temporary storage part of the cost, even though the numerical rule is still a maximum with zero.

Production frameworks can change that storage policy. PyTorch's `ReLU(inplace=True)` overwrites its input rather than allocating a separate result. That option trades ownership safety for reuse of storage; another caller may still need the original values. TinyTorch's ReLU always returns independently owned output. PyTorch ReLU documentation.

A second alternative is **fusion**, evaluating neighboring operations together. A matrix multiplication followed by ReLU can, in principle, apply the maximum before writing each result to memory. It avoids writing an unactivated array only to read it back immediately. This requires an implementation that can combine the operations; calling TinyTorch's ordinary matrix multiplication and ReLU separately does not fuse them. Chapter 18 will study such transformations, and Chapter 23 will discuss implementations outside NumPy.

For an array of shape $(32, 512, 3072)$ stored in float32, the element count is $32 \times 512 \times 3072 = 50{,}331{,}648$. At four bytes per element, one array occupies $201{,}326{,}592$ bytes, about 201 MB.

An ideal separate ReLU pass reads one input array and writes one output array, about 403 MB in total. TinyTorch also copies the NumPy result into its Tensor, adding another array read and write. That gives about 805 MB of logical array traffic for this reference path, before any other framework bookkeeping.

These are byte counts, not measured transfers from physical memory or a latency prediction. Caches, allocation, and the execution environment affect elapsed time. The count explains why reducing temporary arrays can matter even when the arithmetic is cheap. In-place evaluation saves an allocation but still reads and writes the values; fusion can eliminate the intermediate pass itself.

For such a simple function, memory movement can dominate arithmetic. An operation is called **memory-bound** when moving its data is the limiting cost. In Chapter 15, we will plot ReLU and Sigmoid on the Roofline model. Because an elementwise activation performs only 1 FLOP per 8 bytes of DRAM traffic (4 bytes read plus 4 bytes written), its operational intensity is 0.125 FLOP/byte, placing it deep within the memory-bandwidth-bound region of all modern processors. That diagnosis requires measurement on a particular implementation and workload; it does not follow from a universal GPU timing ratio. Sigmoid and GELU make this especially worth checking in TinyTorch because their NumPy expressions create several temporary arrays, not one shared loop for the entire activation.

The memory bandwidth wall: In modern deep learning accelerators, arithmetic units (e.g., NVIDIA Tensor Cores) execute floating-point operations orders of magnitude faster than high-bandwidth memory (HBM) can supply operands. A separate activation kernel spends most of its execution time waiting on memory round-trips:

$$\text{Time} \approx \frac{\text{Bytes Read} + \text{Bytes Written}}{\text{Peak Memory Bandwidth}}$$

Kernel fusion: Compilers like PyTorch Inductor or Triton fuse the activation directly into the epilogue of the preceding GEMM (matrix multiplication). Instead of writing the unactivated result to DRAM and reading it back, the GPU keeps intermediate matrix products in fast on-chip registers (RF) or shared memory (SRAM), applies the activation elementwise, and writes only the final activated tensor back to HBM (see figure 18.2 in Chapter 18 for a hardware trace of register vs. DRAM traffic).

Numerical choices also remain visible at the production boundary. PyTorch's `GELU` defaults to $x\Phi(x)$ and offers the tanh approximation

$$0.5x\left(1 + \tanh\left(\sqrt{2/\pi}\left(x + 0.044715x^3\right)\right)\right).$$

Neither is TinyTorch's sigmoid approximation. Matching an API name therefore does not establish numerical identity; comparisons need to name the formula as well as the input dtype. PyTorch GELU documentation.

Table 2.4: The reference makes array ownership and numerical formulas explicit. Production alternatives must preserve the intended observable behavior.

Concern	TinyTorch reference	Production alternative
Visiting elements	NumPy operations follow array shapes and strides	A device implementation can combine operations in one loop or kernel
Output ownership	`apply` copies the result into a fresh Tensor	In-place operations reuse input storage when allowed
Intermediate storage	Sigmoid and GELU create several array temporaries	Fusion can avoid materializing intermediate arrays
GELU formula	$x\,\sigma(1.702x)$	Exact Gaussian form or a specified approximation
Sigmoid stability	Both candidate divisions share bounded $e^{-\lvert x\rvert}$	Implementation may use another stable numerical formula

The distinction in table 2.4 gives two separate checks for an optimization. First compare the intended numerical behavior, including approximation choices. Then compare memory ownership and measurements. An implementation that is faster because it overwrites a still-needed input has changed the program's contract.

2.6 Summary

The caller writes `y = activation(x)`. The wrapper selects an operation; `Function.apply` unwraps `x.data`; the operation computes on arrays; and Tensor construction gives `y` its own storage. Every activation preserves shape and leaves the input unchanged. The four

elementwise functions transform each position independently, while Softmax uses a shared maximum and sum along its chosen axis.

The numerical mechanisms are small but consequential. ReLU keeps positives and zeros negatives. Tanh and sigmoid saturate; sigmoid avoids overflow by keeping its exponent nonpositive. GELU multiplies the input by an approximate sigmoid gate. Softmax subtracts a maximum before exponentiating and returns zeros for fully excluded slices. These rules let a reader reconstruct the implementation rather than memorize its API.

Chapter 3 adds the tensors an activation does not own: trainable weights and biases. Its containers can compose those layers with the activation wrappers because both accept a Tensor, return a Tensor, and answer `parameters()`. The next design problem is collecting the trainable values and choosing their initial scale.

2.7 Further Reading

- **Glorot, Bordes, and Bengio** (2011). "Deep sparse rectifier neural networks." *AISTATS*. The case for ReLU in deep networks, including the exact zeros this chapter's ReLU produces.
- **Hendrycks and Gimpel** (2016). "Gaussian error linear units (GELUs)." *arXiv*. The source of GELU and of both approximations, including the sigmoid form $x\,\sigma(1.702x)$ the module implements.

🔥 Chapter 3

Layers: Parameters, State, and Initialization

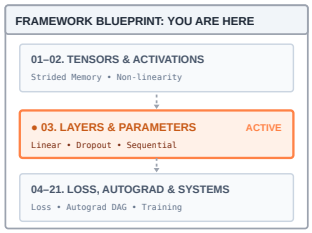
Deep learning models are not solitary mathematical equations; they are modular compositions of stateful, parameterized building blocks. As architectures grow in depth, raw tensor math must be organized into clean modules that bundle weights, biases, and forward computation rules. Furthermore, deep networks require systematic numerical initialization to prevent signal variance from vanishing or exploding across layers, as well as distinct operational behaviors during training vs. inference (such as stochastic zeroing in Dropout).

In PyTorch, defining and executing a parameterized layer takes only a few lines:

```python
import torch
import torch.nn as nn

# Linear projection layer with variance-scaled initialization
layer = nn.Linear(in_features=784, out_features=128)
x = torch.randn(32, 784)  # Batch of 32 vectors
y = layer(x)              # Forward: y = x @ W^T + b

print("Weight shape:", layer.weight.shape)  # (128, 784)
print("Bias shape:", layer.bias.shape)      # (128,)
print("Output shape:", y.shape)             # (32, 128)
```



Framework Blueprint: Module 03 encapsulates stateful weights and modular execution into composable layer blocks.

Underneath that clean interface, the framework resolves several fundamental systems responsibilities:

1. **Persistent state vs. ephemeral activations:** The layer creates persistent parameter tensors (`weight` and `bias`) that live across training iterations, distinct from the transient activation tensors allocated during each forward step.
2. **Variance-scaled parameter initialization:** Rather than initializing weights with raw unscaled random noise (which causes activations to explode or vanish exponentially with depth), the constructor scales initial weight variance inversely with fan-in.
3. **Hierarchical parameter discovery:** The layer registers its weights in an internal container hierarchy so optimizers can traverse the model and retrieve all trainable parameters in a single call (`model.parameters()`).

The memory buffers of Chapter 1 and activations of Chapter 2 provide the foundation; this chapter encapsulates stateful parameters into reusable, composable layer blocks. Before building these abstractions, we must inspect what breaks when multi-layer networks are implemented with raw, unmanaged tensors.

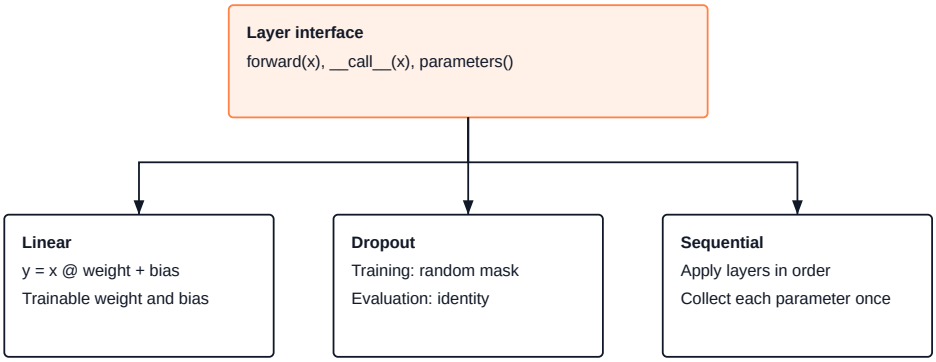


Figure 3.1: **A shared layer interface**: Linear owns parameters, Dropout changes behavior with the training flag, and Sequential composes their forward calls and collects parameters.

3.1 The Problem

↩ **Jump back:** Chapter 2 *Weight initialization variance scaling accounts for non-linear activation compression (e.g., ReLU).*

Write a three-layer network with nothing but tensors. You allocate `W1`, `b1`, `W2`, `b2`, `W3`, `b3`, write the forward pass as three matrix multiplies with a ReLU between each, and the code runs. Then three things go wrong.

First, the numbers. The weights need starting values, and the obvious choice is standard normal noise, `randn`. The loop in listing 3.1 uses a hidden width of 1024 to make the change in activation spread visible as depth grows.

Pause & Predict: Unscaled Variance Across Depth

Before running the code below, consider a 15-layer network where each layer has width $D = 1024$, weights are drawn from $\mathcal{N}(0, 1)$, and inputs have unit variance. What happens to the activation variance? Will it grow linearly, stay bounded, or explode?

Listing 3.1 Unscaled initialization: A ReLU stack exposes growth in pre-activation variance.

```python
import numpy as np

rng = np.random.default_rng(0)
D = 1024
h = rng.standard_normal((256, D)).astype(np.float32)
for layer in range(1, 16):
    W = rng.standard_normal((D, D)).astype(np.float32)
    z = h @ W
    h = np.maximum(0.0, z)  # ReLU
    if layer in (1, 5, 10, 15):
        variance = np.var(z, dtype=np.float64)
        print(f"layer {layer:2d}: variance of z = {variance:.2e}")
```

On my machine, with seed 0, that prints the following.

```
layer  1: variance of z = 1.03e+03
layer  5: variance of z = 6.78e+13
layer 10: variance of z = 2.34e+27
layer 15: variance of z = 8.50e+40
```

Every layer multiplies the spread of the signal by about 500. By layer 15 the pre-activations are around 10^{20} in size, and at that rate they pass the largest number float32 can hold (3.4×10^{38}) near layer 29 and turn into `inf`, then `NaN`. Shrink the weights instead, say `randn * 0.001`, and the same table runs the other way, toward zero. There is a correct scale, it depends on the layer's width, and a bare tensor does not know it.

Second, bookkeeping. When you hand the network to an optimizer (Chapter 7) it needs every trainable tensor and nothing else. With six loose variables you pass a list by hand. With sixty you forget one, and that layer silently never learns.

Third, mode. Some layers do one thing while training and another when the trained network is used. If that switch is a global flag you set by hand, you will forget it, and the deployed model will behave like a noisy training run.

All three are the same gap. A tensor holds numbers; it does not hold the knowledge of how it is meant to be used. A layer does.

3.2 The Idea

3.2.1 A layer owns parameters and implements a forward rule

↪ **Jump ahead:** Chapter 16 *Compacts 32-bit floating-point layer weight matrices into 8-bit integers (INT8).*

In modern frameworks, architectures like ResNet and Transformers are structured as hierarchies of `nn.Module` abstractions rather than loose collections of arrays. Encapsulating stateful tensors and computation rules into discrete layers allows optimizers to automatically discover learnable parameters and execute forward dataflow without manual variable tracking. For distinct layers, the design resembles a tree. Each `Linear` holds its weight and optional bias; each container holds references to its child layers. Asking the root for parameters means asking each child for the tensors it owns. The result is a new Python list containing references to the original tensors, so an optimizer can

later change the same values that the forward pass reads. A second copy of the numbers would break that connection.

Sharing adds an important case to that picture. A child can occur twice, or appear inside a nested container as well as directly in the parent. `Sequential.parameters()` keeps the first occurrence of each tensor by object identity. Two different tensors with equal values remain two parameters; the same tensor reached twice remains one. The forward pass still executes every child occurrence. Parameter ownership and execution order therefore require different traversals.

3.2.2 The right starting scale

Calibrated weight initialization is essential for training deep feedforward and residual networks; without it, signal variances either explode exponentially or collapse to zero across successive layers. Frameworks calibrate initial weight variance inversely with layer width to preserve activation variance from input to output. Consider a layer with four inputs per output. With independent, centered inputs of variance 1 and independent, centered weights of variance 1, each product contributes variance 1, and summing four products gives variance 4. Giving each weight variance $1/4$ instead keeps the sum's variance at 1. Variance is the average squared distance from the mean; it measures the spread we saw growing in the loop.

For a general width, take one output $y = \sum_{j=1}^{D_{\text{in}}} x_j w_j$, where D_{in} is the layer's **fan-in**,[9] the number of inputs each output reads. Under the same independence and zero-mean assumptions, the variance of each product is the product of the variances, and the variances of the independent terms add:

$$\text{Var}(y) = D_{\text{in}} \cdot \text{Var}(x) \cdot \text{Var}(w).$$

This exact variance scaling principle reappears in Chapter 12 when computing scaled dot-product attention: the inner product between query and key vectors of dimension d_k sums d_k independent products, scaling variance by d_k. Dividing the dot product by $\sqrt{d_k}$ scales its standard deviation back to 1.0, preventing softmax from saturating into one-hot distributions.

With $\text{Var}(w) = 1$ and $D_{\text{in}} = 1024$, the first linear operation multiplies the input variance by 1024. For a symmetric input distribution, ReLU halves the second moment, $\mathbb{E}[x^2]$ (the average squared value), rather than exactly halving the variance: its output no longer has mean zero. With independent zero-mean weights in the next layer, that second moment determines the next pre-activation variance. This gives the approximate factor of 512 seen in the loop.

To hold the variance steady, set $D_{\text{in}} \cdot \text{Var}(w) = 1$, or $\text{Var}(w) = 1/D_{\text{in}}$. This is LeCun's rule and it is what the TinyTorch course module uses. Kaiming initialization accounts for the ReLU halving by doubling the weight variance:[10]

$$\text{Var}(w) = \frac{2}{D_{\text{in}}}, \qquad \sigma_w = \sqrt{\frac{2}{D_{\text{in}}}}.$$

Here we call the multiplier on weight *variance* the **gain**, matching the first exercise; a gain on standard deviation would be its square root. The appropriate multiplier depends on the activation and its input distribution. Gain 1 preserves the linear case and is the module's default, rather than a universal prescription for tanh. Xavier Glorot's earlier rule, $\text{Var}(w) = 2/(D_{\text{in}} + D_{\text{out}})$, splits the difference between keeping the forward signal

[9] **Fan-in** is the number of inputs feeding one output unit (`in_features` in `Linear`). **Fan-out** is the number of outputs each input feeds. Summing over fan-in inputs scales forward signal variance by D_{in}.

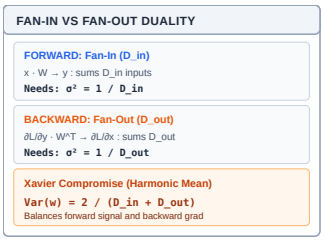
[10] For a zero-mean symmetric input z, $\text{ReLU}(z) = \max(0, z)$ zeroes out the negative half-space, halving the expected second moment: $\mathbb{E}[(\text{ReLU}(z))^2] = \frac{1}{2}\mathbb{E}[z^2]$. To preserve variance through the subsequent matrix multiplication, the weight variance must double from $1/D_{\text{in}}$ to $2/D_{\text{in}}$ (He et al., 2015).

steady (which wants $1/D_{\text{in}}$) and keeping the backward gradient steady (which wants $1/D_{\text{out}}$; Chapter 6 will introduce that backward computation).

Why does backward propagation through a layer demand $1/D_{\text{out}}$? In the forward pass, each output feature sums over D_{in} incoming terms ($y_i = \sum_{j=1}^{D_{\text{in}}} x_j W_{ji}$). In the reverse direction, backward signal flow transposes the weight matrix (W^T), so each input coordinate accumulates incoming signals across all D_{out} downstream outputs ($s_j = \sum_{i=1}^{D_{\text{out}}} s_i W_{ji}$). Reverse flow treats the layer's fan-out as its effective fan-in. When $D_{\text{in}} \neq D_{\text{out}}$, no single fixed variance can hold both forward and reverse signals constant simultaneously, which is why Glorot proposed the harmonic mean. Any of these rules can be realized with a normal distribution of the right standard deviation or a uniform one on $[-a, a]$, whose variance is $a^2/3$. For a fixed ratio of input to output width, each rule in table 3.1 reduces the standard deviation as the inverse square root of width. The activation determines the appropriate constant.

Table 3.1: **Weight initialization rules**: The scales follow assumptions about the activation and its input distribution; they do not guarantee constant variance in an arbitrary network.

Scheme	Meant for	Weight variance	Uniform bound a
LeCun	linear, tanh (course module default)	$1/D_{\text{in}}$	$\sqrt{3/D_{\text{in}}}$
Xavier (Glorot)	sigmoid, tanh	$2/(D_{\text{in}} + D_{\text{out}})$	$\sqrt{6/(D_{\text{in}} + D_{\text{out}})}$
Kaiming (He)	ReLU	$2/D_{\text{in}}$	$\sqrt{6/D_{\text{in}}}$



Xavier Glorot's harmonic mean balances forward signal preservation with backward gradient stability across dimension changes.

Rerunning the loop from The Problem with the weights scaled three ways gives table 3.2. Reset `rng` to seed 0 before each run, and multiply each newly drawn `W` by the scale in the column heading. The unscaled column grows and the small-scale column shrinks. Kaiming initialization keeps the pre-activation variance near 2 here. The first layer receives raw inputs of variance 1, so its factor of 2 doubles the variance once; subsequent layers approximately compensate for the ReLU between them.

Table 3.2: **Measured variance across depth**: Variance of the pre-activation z at four depths for a 1024-wide ReLU network under three weight scales, from one run of the loop in The Problem with seed 0. Variance is accumulated in float64, allowing the measurement to exceed float32's range even while the activations themselves remain finite. The figures are rounded observations, not exact predictions for other random draws.

Layer	`randn`	`randn * 0.001`	Kaiming, `randn * sqrt(2/1024)`
1	1.0×10^{3}	1.0×10^{-3}	2.0
5	6.8×10^{13}	6.8×10^{-17}	1.9
10	2.3×10^{27}	2.3×10^{-33}	1.9
15	8.5×10^{40}	8.5×10^{-50}	2.0

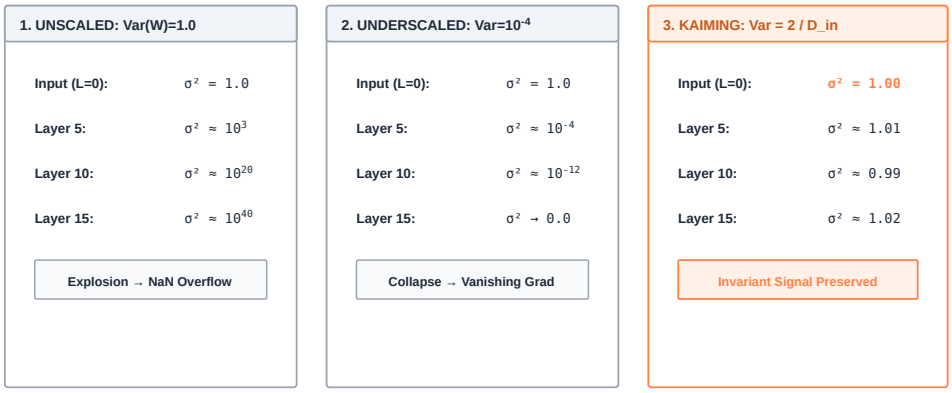
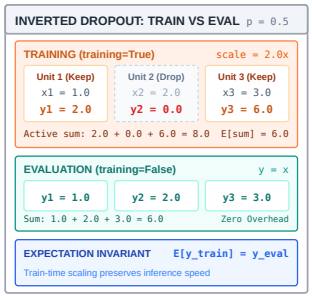


Figure 3.2: **The variance waterfall across depth**: Without proper initialization, signal variance either explodes exponentially (causing numerical overflow and NaN activations) or collapses to zero (causing vanishing signals). Kaiming initialization ($\text{Var}(w) = 2/D_{\text{in}}$) counterbalances ReLU's half-space truncation to preserve stable forward activation variance across deep stacks.

3.2.3 Dropout, and why a layer needs a mode



Inverted Dropout: Training scales active activations by $1/(1-p)$ so that evaluation executes as a zero-overhead identity pass.

Popularized in deep convolutional networks and ubiquitous in Transformer feed-forward sublayers, **dropout** acts as a powerful regularizer by preventing neurons from co-adapting to noise. Because training requires stochastic unit masking while production inference demands deterministic predictions, introducing dropout forces neural network frameworks to maintain distinct execution modes (`train` vs. `eval`). For an activation of value 3 and dropout probability $p = 0.5$, half of the random outcomes discard the value and half retain it. Returning either 0 or 3 would average to 1.5. Returning either 0 or 6 instead averages to 3, matching the value passed through at evaluation. That correction is the arithmetic behind dropout, a training technique intended to reduce overfitting. Each activation is independently dropped, so later computations cannot always rely on the same inputs being present.

The correction can be placed on either side of training. One convention leaves surviving values unchanged during training and scales them down at evaluation. **Inverted dropout**, which TinyTorch implements, instead divides surviving training activations by the probability of keeping them, $1 - p$. Evaluation can then return the original input without performing any scaling or array passes:

$$y_{\text{train}} = \frac{x \cdot m}{1-p}, \quad m \in \{0, 1\} \text{ with } P(m = 1) = 1 - p, \qquad y_{\text{eval}} = x.$$

For $0 \leq p < 1$, the expected value of m is $1 - p$, so the expected value of y_{train} is exactly x. Training sees a noisy version of what eval sees, with the same average, and the eval path becomes an exact zero-overhead identity. The price is that `Dropout` must know which of the two it is doing. That knowledge is the layer's `training` flag, which a single call can override with a `training` argument.

Mechanism: Inverted dropout scales activations by $\frac{1}{1-p}$ during training.[11] At test time, dropout must be turned off ($y_{\text{eval}} = x$) so predictions remain deterministic.

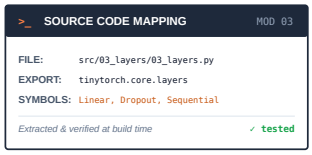
Failure mode: If evaluation runs with `training=True`, inferences become stochastic and activations are scaled up by $\frac{1}{1-p}$ (e.g., $2\times$ when $p = 0.5$). Conversely, if training

[11] In standard dropout (Srivastava et al., 2014), activations are kept unscaled during training and multiplied by $1 - p$ during inference. Inverted dropout divides by $1 - p$ during training instead, keeping the test-time forward pass an exact zero-overhead identity read without modifying tensor values.

runs with `training=False`, the regularization effect vanishes and downstream weights overfit without gradient noise.

3.3 The Code

Module 03 builds on the `Tensor` operators from Chapter 1 and the callable activations from Chapter 2. `Linear.forward` performs its arithmetic through `matmul` and `+`; `Dropout.forward` multiplies tensors after drawing a mask with NumPy. It reads `x.data.shape` only to size that mask. Chapter 6 will teach the tensor operations to record the information needed for gradients, allowing these layer forward methods to remain unchanged.



Source Code Mapping: Module 03 extracts Linear affine transformations, Dropout regularization, and Sequential containers directly from the working repository.

3.3.1 The base class

A caller needs two independent operations on a model. `model(x)` computes an output from the current parameter values; `model.parameters()` returns the tensors available for learning. Chapter 7's optimizer will take that list and update its entries, while Chapter 8 will use it for training and checkpoints. `Layer` defines the common call and parameter methods without allocating weights itself.

```python
class Layer:
    """
    Base class for all neural network layers.
    """

    # A class attribute, so every subclass starts in training mode even when
    # its __init__ never calls super().__init__().
    training = True

    def forward(self, x):
        """
        Forward pass through the layer.
        """
        raise NotImplementedError(
            f"forward() not implemented in {self.__class__.__name__}\n"
            f"  ✗ The Layer base class requires subclasses to implement forward()\n"
            f"  🔔 forward() defines how input data is transformed by this layer\n"
            f"  🔧 Add this method to your class:\n"
            f"     def forward(self, x):\n"
            f"         # Your transformation logic here\n"
            f"         return transformed_x"
        )

    def __call__(self, x, *args, **kwargs):
        """Allow layer to be called like a function."""
        return self.forward(x, *args, **kwargs)
```

```
27
28      def train(self, mode=True):
29          """Put this layer and every layer inside it in training mode
            ↪   (or eval if mode=False)."""
30          return set_training_mode(self, mode)
31
32      def eval(self):
33          """Put this layer and every layer inside it in evaluation
            ↪   mode."""
34          return set_training_mode(self, False)
35
36      def parameters(self):
37          """
38          Return list of trainable parameters.
39          """
40          return []  # Base class has no parameters
41
42      def __repr__(self):
43          """String representation of the layer."""
44          ...
```

`forward` is abstract, `__call__` turns a layer into something you can call and passes any extra arguments through, and `parameters()` returns an empty list until a subclass says otherwise. Notice what is not here. There is no registry of parameters. Module 03 takes the direct route. Each layer that owns trainable tensors lists them in its own `parameters()`, right next to the code that allocated them, and a container collects its children's lists and removes repeated tensor references. Synthesis I's XOR network does exactly this; its `parameters()` returns `self.hidden.parameters() + self.output.parameters()`. The list lives where the tensors live, so it stays short and readable, and the price is that a forgotten entry fails silently: the layer keeps its random weights, the loss may still fall because other layers change, and nothing raises. That silence is the reason PyTorch automates the registration, and the production section shows how. The mode question, which layers need to know whether the network is training, gets one piece of stored state. Line 8 makes `training = True` a class attribute, so every layer starts in training mode even if its constructor never calls `super().__init__()`. `train()` and `eval()` (lines 28 to 34) hand the layer to `set_training_mode`, a module-level function that sets `training` on the layer and on every attribute, or list, tuple, or dict entry, that has a `forward` method, visiting each object once so shared or circular references cannot loop. The walk finds children by inspecting attributes rather than a registry, so it also reaches a hand-written model class that never inherited from `Layer`. A layer that cares about the mode reads its own flag, and a caller can still pass `training=` for one call; `__call__` forwards that argument, and it takes precedence over the stored flag.

3.3.2 Linear

`Linear(2, 3)` packages the opening calculation. Its constructor allocates a `(2, 3)` weight matrix and a `(3,)` bias once; calling the layer with a `(1, 2)` row then returns a `(1, 3)` result. A later call reads the same tensors. The constructor chooses random starting values and their scale, while `forward` only reads them.

The constructor derives the initialization scale from its own width. Module 03 uses LeCun's rule from table 3.1, $\mathrm{Var}(w) = 1/D_{\text{in}}$, so `Linear` draws its weight with standard deviation $\sqrt{1/D_{\text{in}}}$ (the module's `INIT_SCALE_FACTOR` is 1) and applies $y = xW + b$. The weight is stored as `(in_features, out_features)`, so a batch `x` of shape `(B, in_features)` multiplies it directly. The bias starts at zero; it adds nothing to the initial weighted sum. `rng` is the module's NumPy generator, created once with seed 7, and each constructor consumes the next draw from that generator. Repeated calls therefore create different weights; constructing a new layer does not reset the random sequence.

```python
class Linear(Layer):
    """
    Linear (fully connected) layer: y = xW + b
    """

    def __init__(self, in_features, out_features, bias=True):
        """
        Initialize linear layer with proper weight initialization.
        """
        self.in_features = in_features
        self.out_features = out_features

        # LeCun-style initialization for stable gradients
        scale = np.sqrt(INIT_SCALE_FACTOR / in_features)
        weight_data = rng.standard_normal((in_features, out_features))
    * scale
        self.weight = Tensor(weight_data, requires_grad=True)

        # Initialize bias to zeros or None
        if bias:
            bias_data = np.zeros(out_features)
            self.bias = Tensor(bias_data, requires_grad=True)
        else:
            self.bias = None

    def forward(self, x):
        """
        Forward pass through linear layer.
        """
        # Linear transformation: y = xW
        output = x.matmul(self.weight)

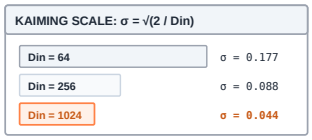
        # Add bias if present
        if self.bias is not None:
            output = output + self.bias

        return output

    def parameters(self):
        """
        Return list of trainable parameters.
        """
        params = [self.weight]
```

```
        if self.bias is not None:
            params.append(self.bias)
        return params

    def __repr__(self):
        """String representation for debugging."""
        ...
```



Kaiming weight $\sigma = \sqrt{2/D_{in}}$ scales inversely with the square root of fan-in.

Notice that `scale` comes from `in_features` alone. With a ReLU after every layer, LeCun's scale lets the next pre-activation variance shrink rather than hold steady, which is the gap Kaiming's gain of 2 closes. The module implements one simple default; the first exercise changes the gain and measures its effect instead of assuming that one initializer suits every network. PyTorch's `nn.Linear` stores the weight the other way around, as `(out_features, in_features)`, and uses a different default distribution; the bridge section takes up both. Notice also that the weights are created with `requires_grad=True`. As defined on `Tensor` in Chapter 1, learnable parameters declare their need for gradients at construction so that when Chapter 6 activates backward graph recording, operations seamlessly capture the parameters without requiring manual flags.

The forward pass is two tensor operations, `matmul` and `+`, and that choice is doing quiet work. Each returns a fresh `Tensor`. Once Chapter 6 adds gradient recording to tensor operations, the same forward code can connect the result to `weight` and `bias`. The addition of `self.bias` relies on the broadcasting from Chapter 1: the bias has shape `(out_features,)`, the product has shape `(B, out_features)`, and NumPy reads the same bias values for all B rows without allocating a repeated bias matrix. However, executing this as two separate operations incurs a concrete memory traffic penalty: the intermediate matrix product **xW** of shape (B, D_{out}) ($4 \times B \times D_{\text{out}}$ bytes in float32) must be written out to main memory, only to be immediately reread across the memory bus to add the bias and write a second (B, D_{out}) array. For a modest batch size of 64 and hidden width 4096, this unfused intermediate round-trip forces over 2 MB of unnecessary DRAM traffic per layer invocation. We eliminate this round-trip through operator fusion in Chapter 18. That forward efficiency creates an obligation for Chapter 6: because the bias influenced every row in the batch, the backward engine will have to sum those gradients across the batch to keep the parameter's shape.

Core Invariant: Linear Contraction

$$(B, D_{\text{in}}) \times (D_{\text{in}}, D_{\text{out}}) + (D_{\text{out}},) \to (B, D_{\text{out}})$$

Inner dimension D_{in} contracts via row-major GEMM; bias broadcasts with stride 0.

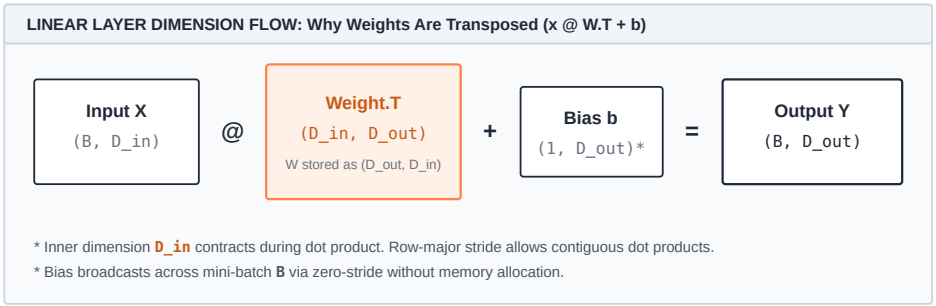


Figure 3.3: **Linear layer dimension flow and weight layout**: In a forward linear projection, the inner dimension D_{in} contracts via matrix multiplication while the batch dimension B is preserved. Bias broadcasting adds $(D_{\text{out}},)$ across all rows in zero-stride memory without replication.

3.3.3 Dropout

`Dropout` gives the `training` flag its job. For the three-element hidden row `[1, 2, 3]`, a keep-drop-keep draw at `p=0.5` produces the scaled mask `[2, 0, 2]` and output `[2, 0, 6]`. On evaluation the same layer returns `[1, 2, 3]`. The probability belongs to the layer, while the mask belongs to one training call.

The constructor validates `p` before storing it. The forward method first settles the mode: its `training` argument defaults to `None`, which falls back to the layer's own `self.training` (lines 41 and 42), while an explicit `True` or `False` overrides the flag for that call. It then delegates its first decision to `_should_apply_dropout`. If that helper says no, the method returns before drawing random numbers. Otherwise it handles `p=1` separately, then asks `_generate_dropout_mask` for an input-shaped mask. That helper draws uniform values, compares each with `1-p`, converts the resulting booleans to float32, and folds the scale into the mask. The final tensor multiplication both zeros and scales the input elements.

```
class Dropout(Layer):
    """
    Dropout layer for regularization.
    """

    def __init__(self, p=0.5):
        """
        Initialize dropout layer.
        """
        if not DROPOUT_MIN_PROB <= p <= DROPOUT_MAX_PROB:
            raise ValueError(
                f"Invalid dropout probability: {p}\n"
                f"  ✗ p must be between {DROPOUT_MIN_PROB} and
                ↪ {DROPOUT_MAX_PROB}\n"
                f"  🔔 p is the probability of DROPPING a neuron (not
                ↪ keeping it!)\n"
                f"     p=0.0 means keep all neurons (no dropout)\n"
                f"     p=0.5 means drop 50% of neurons randomly\n"
                f"     p=1.0 means drop all neurons (zero output)\n"
                f"  🔧 Common values: Dropout(0.1) for light,
                ↪ Dropout(0.3) for moderate, Dropout(0.5) for
                ↪ aggressive"
            )
        self.p = p

    def _should_apply_dropout(self, training):
        """
        Determine whether dropout should be applied.
        """
        return training and self.p > DROPOUT_MIN_PROB

    def _generate_dropout_mask(self, shape):
        """
        Generate a random dropout mask with inverted scaling.
        """
```

```
        keep_prob = 1.0 - self.p
        binary_mask = (rng.random(shape) <
↪   keep_prob).astype(np.float32)
        scale = 1.0 / keep_prob
        return Tensor(binary_mask * scale)

    def forward(self, x, training=None):
        """
        Forward pass through dropout layer.
        """
        if training is None:
            training = self.training

        if not self._should_apply_dropout(training):
            return x

        if self.p == DROPOUT_MAX_PROB:
            return x * 0.0  # Keep the operation path; Module 06 will
            ↪   propagate zero gradients.

        mask = self._generate_dropout_mask(x.data.shape)
        return x * mask

    def __call__(self, x, training=None):
        """Allows the layer to be called like a function."""
        return self.forward(x, training)

    def parameters(self):
        """Dropout has no parameters."""
        return []

    def __repr__(self):
        ...
```

The return paths have different ownership behavior. Evaluation and `p=0` return `x` itself, so the output is the same object and no random state is consumed. Training with `p=1` returns `x * 0.0`, which gives a new all-zero tensor for finite inputs and avoids dividing by zero. For `0<p<1`, the method returns `x * mask`, also a new tensor. None of these branches overwrites the input. The multiplication in both training branches will let Chapter 6 propagate zero gradients for dropped elements; replacing it with an unrelated zero tensor would sever that connection.

3.3.4 Sequential

The worked example needs one object that accepts the input row, calls two linear layers with dropout between them, and returns the final row. `Sequential` supplies that object. It accepts either individual children or a list of children and stores their references in `self.layers`. It does not inherit from `Layer`, but provides the same callable, `forward`, and `parameters` interface.

```python
class Sequential:
    """
    Container that chains layers together sequentially.
    """

    training = True  # Same default as Layer

    def __init__(self, *layers):
        """Initialize with layers to chain together."""
        # Accept both Sequential(layer1, layer2) and
        ↪  Sequential([layer1, layer2])
        if len(layers) == 1 and isinstance(layers[0], (list, tuple)):
            self.layers = list(layers[0])
        else:
            self.layers = list(layers)

    def forward(self, x, training=None):
        """Forward pass through all layers sequentially.
        """
        if training is None:
            training = self.training
        for layer in self.layers:
            # Only layers whose forward takes a `training` flag
            ↪  (Dropout) receive it
            if 'training' in
            ↪  inspect.signature(layer.forward).parameters:
                x = layer.forward(x, training=training)
            else:
                x = layer.forward(x)
        return x

    def __call__(self, x, training=None):
        """Allow model to be called like a function."""
        return self.forward(x, training=training)

    def train(self, mode=True):
        """Put the container and every layer in it in training mode
        ↪  (or eval if mode=False)."""
        return set_training_mode(self, mode)

    def eval(self):
        """Put the container and every layer in it in evaluation
        ↪  mode."""
        return set_training_mode(self, False)

    def parameters(self):
        """Collect each parameter once, even when layers share a
        ↪  weight."""
        params = []
        seen = set()
        for layer in self.layers:
            for param in layer.parameters():
```

```
                if id(param) not in seen:
                    params.append(param)
                    seen.add(id(param))
        return params

    def __repr__(self):
        ...
```

Calling `Sequential.__call__` forwards the input and any `training` argument to `Sequential.forward`, which runs its children in order. Follow the assignment `x = layer.forward(...)`: after each iteration, `x` holds the previous child's output, so the next child must accept that shape. The container does not reshape or repair it.

The `inspect.signature` test handles a second contract. `Dropout.forward` accepts `training`, while `Linear.forward` does not, so the container inspects the signature before making each call. It does not catch a `TypeError` from inside the layer and retry: a genuine layer error must reach the caller. This makes a failing child visible at the point where composition breaks. A nested `Sequential` also accepts `training`, so the value reaches dropout at every depth. When the call supplies no argument, the container resolves it from its own flag (lines 19 and 20), which `model.eval()` set along with every child's, and passes that value down explicitly. An explicit `model(x, training=False)` overrides the stored flags for that call without changing them, so a later call with no argument returns to whatever mode `train()` or `eval()` last set. Signature inspection adds noticeable Python overhead on every forward pass, typically 1 to 5 microseconds of CPython bytecode introspection per layer. For small batch sizes or shallow networks, this reflection latency can rival or exceed the execution time of the matrix multiplication itself. Production engines avoid dynamic reflection entirely, relying on cached method dispatch tables or static C++ polymorphic dispatch. Building a small model shows the container doing its two jobs.

```
from tinytorch.core.layers import Linear, Dropout, Sequential

model = Sequential(Linear(4, 3), Dropout(0.5), Linear(3, 2))
print([p.shape for p in model.parameters()])
```

```
[(4, 3), (3,), (3, 2), (2,)]
```

Four parameter tensors came back, weight before bias within each layer and layer order within the container, and `Dropout` contributed none. In practice a `ReLU` from Chapter 2 sits between the two `Linear` layers; it is left out here so the trace below stays short, and Synthesis I builds the full version.

Parameter collection follows `self.layers` in that same order, but its `seen` set records `id(param)` before appending. For `shared = Linear(2, 2)` and `Sequential(shared, Sequential(shared))`, the forward pass applies `shared` twice, while `parameters()` returns only `[shared.weight, shared.bias]`. An optimizer should update those tensors once per step. Returning them twice would confuse reuse in the forward computation with ownership of two independent parameters. This deduplication is also an architectural requirement for modern language models. In Transformer architectures (as explored in Chapter 13), **weight tying** shares the exact same weight matrix between input token embeddings and the final output vocabulary projection head. Deduplication ensures that `optimizer.step()` (Chapter 7) updates the shared weights exactly once

per iteration, rather than erroneously applying duplicate gradient steps or corrupting optimizer momentum buffers.

3.4 A Worked Trace

We trace that same model with weights chosen by hand so every number can be checked on paper. Parameter discovery first. Table 3.3 lists what `model.parameters()` returns and what it costs in memory at four bytes per float32 value. The byte count covers array payloads, excluding Python objects and list overhead. Listed tensors already carry `requires_grad=True`; the operations that record gradients arrive in Chapter 6.

Table 3.3: **Parameter storage**: Parameter ledger for `Sequential(Linear(4, 3), Dropout(0.5), Linear(3, 2))`.

Layer	Attribute	Shape	Values	Listed	Bytes
`Linear(4, 3)`	`weight`	$(4, 3)$	12	yes	48
`Linear(4, 3)`	`bias`	$(3,)$	3	yes	12
`Dropout(0.5)`	none		0		0
`Linear(3, 2)`	`weight`	$(3, 2)$	6	yes	24
`Linear(3, 2)`	`bias`	$(2,)$	2	yes	8
Total	4 tensors		**23**		**92**

Now the forward pass. We overwrite the random weights with small integers and halves, feed one input row, and supply a particular dropout mask, keep-drop-keep. The explicit mask in listing 3.2 traces one possible training result; it does not claim that the next random call to `model(x)` will select that mask.

```
h1       [[1. 2. 3.]]
h_drop   [[2. 0. 6.]]
y_train  [[5. 2.]]
y_eval   [[4.5 3.5]]
```

In listing 3.2, lines 3 to 12 overwrite the model's random parameters with exact weights and biases for hand tracing. Line 14 supplies a single input vector `x`, and line 15 evaluates the first affine layer `h1 = lin1(x)`. Check the first hidden unit by hand: its column of `W1` is $[0.5, 0.5, 1.0, 0.0]$, so xW_1 gives $0.5 + 1.0 - 1.0 + 0 = 0.5$, and adding the bias 0.5 makes 1.0 (printed in line 16 as `[[1. 2. 3.]]`). Before adding the bias, the full product is `[0.5, 1.0, 4.0]`; the bias `[0.5, 1.0, -1.0]` produces `[1.0, 2.0, 3.0]`. The input shape is `(1, 4)`, the hidden and mask shapes are `(1, 3)`, and the final output shape is `(1, 2)`.

In lines 18 to 21, the inverted dropout mask keeps the first and third units and scales them by $1/(1-0.5) = 2$, yielding `h_drop = [[2. 0. 6.]]` (line 20) and a training prediction `y_train = [[5. 2.]]` (line 21). Line 23 switches to `training=False`, bypassing dropout to produce `y_eval = [[4.5 3.5]]`. Table 3.4 contrasts the two computational paths through the second layer.

Listing 3.2 A fixed dropout draw: The same parameter tensors support both training and evaluation paths.

```
from tinytorch.core.tensor import Tensor

lin1, drop, lin2 = model.layers
lin1.weight.data[:] = [[0.5, -1.0,  0.0],
                       [0.5,  0.5,  1.0],
                       [1.0,  0.0, -1.0],
                       [0.0,  2.0,  2.0]]
lin1.bias.data[:]   = [0.5, 1.0, -1.0]
lin2.weight.data[:] = [[1.0, -1.0],
                       [1.0,  1.0],
                       [0.5,  0.5]]
lin2.bias.data[:]   = [0.0, 1.0]

x = Tensor([[1.0, 2.0, -1.0, 0.5]])
h1 = lin1(x)
print("h1      ", h1.data)

mask = Tensor([[2.0, 0.0, 2.0]])            # the draw we assume:
    keep, drop, keep
h_drop = h1 * mask
print("h_drop  ", h_drop.data)
print("y_train ", lin2(h_drop).data)

print("y_eval  ", model(x, training=False).data)
```

Table 3.4: **One dropout draw and evaluation**: The same input passes through the same weights on the two paths. The training output depends on the mask; the use-time output does not.

Stage	`training=True`	`training=False`
Input x	$[1.0, 2.0, -1.0, 0.5]$	$[1.0, 2.0, -1.0, 0.5]$
$h_1 = xW_1 + b_1$	$[1.0, 2.0, 3.0]$	$[1.0, 2.0, 3.0]$
Dropout, $p = 0.5$	mask $[2, 0, 2]$ gives $[2.0, 0.0, 6.0]$	identity, $[1.0, 2.0, 3.0]$
$y = hW_2 + b_2$	$[5.0, 2.0]$	$[4.5, 3.5]$

The two outputs differ, as they should; training saw a noisy version of the network. The claim from The Idea is that they agree on average. With three units and $p = 0.5$ there are only eight equally likely masks, so listing 3.3 averages over all of them to check the algebra.

Listing 3.3 All eight masks: An exhaustive average checks the affine output expectation.

```
import itertools
import numpy as np

outputs = []
for bits in itertools.product([0.0, 2.0], repeat=3):   # all 8 masks,
    equally likely
    outputs.append(lin2(h1 * Tensor([list(bits)])).data)
print("mean of y_train over all masks:", np.mean(outputs, axis=0))
```

```
mean of y_train over all masks: [[4.5 3.5]]
```

The average equals the evaluation output here because the operation after dropout

the stored flag. Both designs need a path from the root model to the components whose behavior or values matter. The Module documentation describes that registration and mode contract.

The shared interface does not imply identical weight layout or initialization. PyTorch's Linear documentation specifies a weight of shape `(out_features, in_features)` and the calculation $xW^T + b$.[12] Its default uniform bound is $1/\sqrt{D_{\text{in}}}$, giving variance $1/(3D_{\text{in}})$. TinyTorch stores `(in_features, out_features)` and uses a normal draw with variance $1/D_{\text{in}}$. Transferring weights between those conventions requires a transpose, and matching dimensions alone does not establish matching initial values.

[12] PyTorch's `nn.Linear` stores weights as `(out_features, in_features)` so that low-level BLAS and cuBLAS routines (such as `cublasSgemm`) can stream output rows as contiguous blocks in row-major memory buffers without requiring explicit transposition of weight matrices in GPU SRAM.

PyTorch's Dropout documentation specifies the same inverted scaling and evaluation identity. That mathematical contract does not depend on how a backend schedules the work. TinyTorch exposes separate NumPy steps for the random draw, comparison, scaling, and multiplication, plus Tensor allocations, materializing a full 32-bit float mask in DRAM. In contrast, production GPU runtimes like cuDNN and Triton use counter-based pseudo-random number generators (such as Philox) to generate random bits on-the-fly in processor registers. Rather than storing a 32-bit float or 8-bit boolean per activation, production backends pack dropout decisions into 1-bit boolean masks (packing 32 elements per 32-bit word) or simply recompute the mask during backward passes from the original Philox seed and offset, reducing memory bus traffic by a factor of 32. The local contract remains the one we traced: preserve the input's expected value during training and pass it through at evaluation.

This is a storage and operation count, not a timing measurement. `Linear(4096, 4096)` stores $4096 \times 4096 \times 4 = 67{,}108{,}864$ bytes of float32 weights and $4096 \times 4 = 16{,}384$ bytes of bias. The parameter payload is about 67.1 MB. A batch of 16 produces 16×4096 output values, or $262{,}144$ bytes. These counts exclude temporary arrays and Python objects.

The matrix product contains $16 \times 4096 \times 4096 = 268{,}435{,}456$ multiply-accumulate terms. Counting one multiply and one add per term gives approximately 537 million floating-point operations; adding the bias requires another $65{,}536$ additions. Increasing batch size increases arithmetic and output storage while leaving parameter storage unchanged. Whether the measured call is limited by computation or memory movement depends on the backend and its reuse of those weights; the storage count alone cannot decide that. Chapter 15 formalizes this trade-off using the Roofline model to compute operational intensity (FLOPs per byte transferred), while Chapter 18 experiments with cache loop tiling to improve the reuse of each loaded block.

In Chapter 17 and Chapter 23, we address the massive memory footprint of these large linear layers. Instead of fine-tuning the full 67 MB weight matrix, Low-Rank Adaptation (LoRA) decomposes the weight update ΔW into two low-rank matrices $B \times A$ of rank $r \ll D_{\text{in}}$ (for instance, $r = 8$). Initializing A randomly and B to zero guarantees that $\Delta W = 0$ at the start of training while slashing trainable parameter memory by over 99%.

The dynamic range cliff: In standard IEEE 754 single-precision (`float32`), numbers down to $\sim 1.4 \times 10^{-45}$ are representable. In half-precision (`float16`), the minimum positive subnormal is 5.96×10^{-8} and minimum normal is 6.10×10^{-5}.

Initialization hazards: For wide layers (e.g., $D_{\text{in}} = 16384$), Kaiming standard deviation is $\sigma = \sqrt{2/16384} \approx 0.011$. If small initialization is combined with low precision, products can underflow directly to zero during forward execution, effectively pruning random pathways before training begins. Modern distributed training frameworks (like

Megatron-LM or DeepSpeed) initialize weights in FP32 before casting to BF16 (which shares FP32's 8-bit exponent) to preserve the dynamic range of small variance weights.

The comparison suggests concrete debugging checks. If a parameter never changes, verify that the model returns the exact tensor read by its forward pass. If the first outputs are too large or too small, inspect the initializer and measure intermediate values before blaming training. If repeated evaluation calls differ, inspect the `training` flag of each dropout layer and any argument that overrides it. Each check follows state already visible in the reference.

3.6 Summary

A `Linear` constructor allocates weight and optional bias once; `forward` reads those tensors and returns a new result. Its fan-in controls the initial weight scale. `Sequential` executes every child in order but returns each distinct parameter only once. `model.train()` and `model.eval()` set the `training` flag on it and on every layer inside it, and an explicit `training` argument overrides that flag for one call. `Dropout` preserves the input object during evaluation and creates a masked result during training; for $p < 1$, its scaling preserves each input element in expectation.

Those contracts are enough to reconstruct the worked path from `(1, 4)` to `(1, 3)` to `(1, 2)`, identify the 23 stored parameter values, and explain why one training output differs from evaluation. Chapter 4 takes the next step. A final linear layer supplies one raw score per class; a loss function must combine those scores with the correct label into a scalar that measures the prediction error.

With `Tensor` (Module 01), activations (`Sigmoid`, Module 02), and `Linear` layers (Module 03) complete, you have built the entire forward execution engine. You can now run your first historical milestone from the terminal:

```
tito milestone run 01
```

This milestone executes Frank Rosenblatt's 1958 single-layer perceptron on linearly separable 2D data using random weights. You will observe that while the forward pass computes without error, the accuracy is whatever a random line happens to give, anywhere from 0% to 100%, and changes from run to run. What the milestone grades is the arithmetic: YOUR output must match $\sigma(XW + b)$ computed in NumPy from the same weights. The fundamental takeaway: **a forward pass alone is not intelligence**. Building model architectures is straightforward; making them learn requires loss functions, automatic differentiation, and optimizers (Chapter 4 through Chapter 8). See Synthesis I for the full historical analysis.

3.7 Further Reading

- **Glorot and Bengio** (2010). "Understanding the difficulty of training deep feedforward neural networks." *AISTATS*. The variance argument behind fan-in scaling and the Xavier rule.
- **He et al.** (2015). "Delving deep into rectifiers." *ICCV*. The same argument redone for ReLU, which gives the factor of two in Kaiming initialization.

- **Srivastava et al.** (2014). "Dropout: a simple way to prevent neural networks from overfitting." *JMLR*. The dropout layer and the rescaling that keeps its expected output unchanged.

🔥 Chapter 4

Losses: Cross-Entropy and Numerical Stability

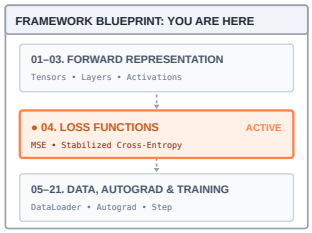
A neural network produces raw, unbounded scores across possible outputs, but learning requires a scalar objective that quantifies prediction error: how far is the model's prediction from ground truth? Different learning tasks require distinct mathematical loss formulations: mean squared error for continuous regression, binary cross-entropy for probabilities, and categorical cross-entropy for multi-class classification.

In PyTorch, computing cross-entropy on multi-class logits is a single stable function call:

```
import torch
import torch.nn.functional as F

# Logits with large values that would overflow naive exponentials
logits = torch.tensor([[2.0, 1.0, 0.1], [1000.0, 1001.0, 1002.0]])
targets = torch.tensor([0, 2])

loss = F.cross_entropy(logits, targets)
print("Stable scalar loss:", loss.item())  # 0.4123, no NaN or Inf!
```



Framework Blueprint: Module 04 evaluates network predictions against targets while guarding against numerical overflow.

Underneath that single call, the runtime protects computation from the hazards of IEEE 754 floating-point arithmetic:

1. **The LogSumExp max-subtraction shift:** In raw float32, e^{1000} immediately overflows to `+inf`, causing division to produce `nan`. The runtime subtracts the row maximum ($\max_j z_j$) before exponentiation, bounding all exponents to ≤ 0 so they never overflow.
2. **Operator fusion across softmax and log-likelihood:** Rather than materializing a probability tensor and then taking its logarithm, the runtime computes log-softmax directly, so no probability array is formed.
3. **Scalar reduction invariant:** A loss function must collapse high-dimensional prediction arrays across batch and class coordinates into a single 0-D scalar. This reduction guarantees that the training objective represents a global scalar potential, preventing downstream memory explosions during graph execution.

This chapter constructs TinyTorch's loss library, defusing the numerical traps of cross-entropy to provide stable scalar objectives that Chapter 6 and Chapter 7 turn into parameter updates. Before building the stable implementation, we must inspect how textbook mathematical recipes immediately fail in float32.

figure 4.1 contrasts a direct exponential with the max-shifted path. The worked trace will follow each intermediate value through the implementation.

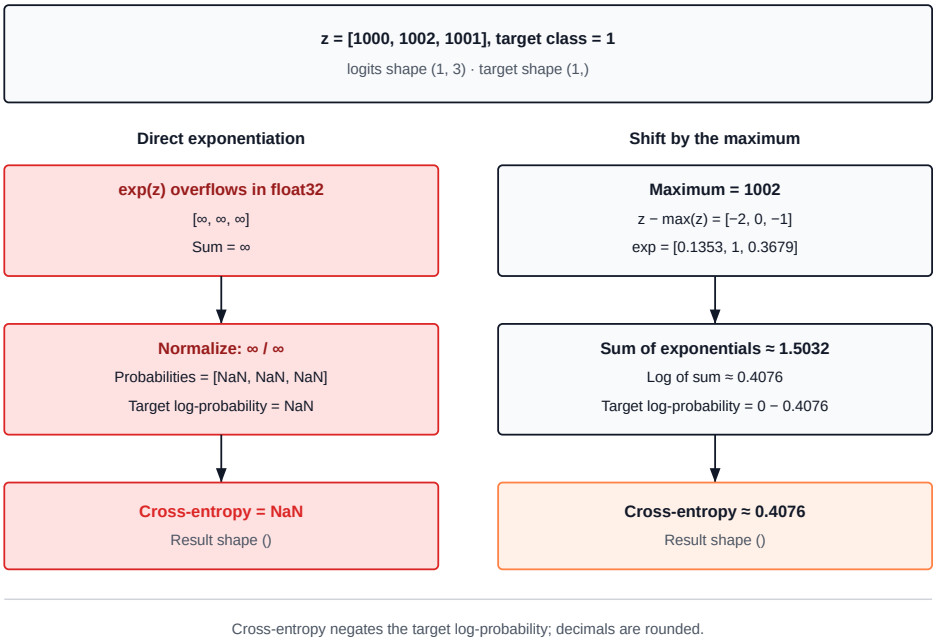
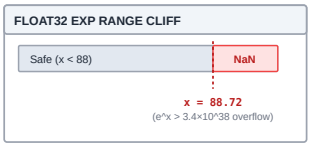


Figure 4.1: Stable cross-entropy on large logits. For `[1000, 1002, 1001]` and target class 1, direct float32 exponentiation overflows. Subtracting the maximum gives `[-2, 0, -1]` and a finite loss of approximately 0.4076.

4.1 The Problem

↩ **Jump back:** Chapter 2 *Softmax normalization and the max-subtraction shift prevent catastrophic floating-point overflow.*

A classifier with three classes produces three numbers. Suppose, early in training, they come out as `[100.0, 101.0, 99.0]`. The standard recipe for turning them into probabilities is to exponentiate each one and divide by the sum. Here is that recipe, verbatim, in float32.



In float32, e^z overflows to ∞ at $z = 88.72$, necessitating the max-shift stabilization.

```
import numpy as np

z = np.array([100.0, 101.0, 99.0], dtype=np.float32)
e = np.exp(z)                 # [inf, inf, inf]
print(e / np.sum(e))          # [nan nan nan]
```

NumPy prints an overflow warning and carries on, and every probability comes out as `nan`. The reason is the range of a 32-bit float. The largest finite float32 is about 3.4×10^{38}, which is $e^{88.72}$, so `exp` of anything above 88.72 returns `inf`. The sigmoid in Chapter 2 met the same wall and selected the branch whose exponent was nonpositive; this chapter applies the same idea to a whole row of numbers at once. Once one term is `inf`, the sum is `inf`, and `inf / inf` is not a number. The other direction fails just as quietly. Below about -104, `exp` returns an exact `0.0`, so logits of `[-1000, -1002, -1001]` give a sum of zero and `0 / 0`, which is also `nan`.

Logits of 100 are not exotic. Recall from Chapter 2 that a direct exponential overflows float32 to infinity for any input above 88.72, which is why TinyTorch's Softmax subtracts the row maximum first. Furthermore, Chapter 3 showed how fast badly scaled weights inflate activations; a few layers of that and the outputs easily enter the hundreds. A nonfinite loss is a reason to stop and inspect the batch before updating weights. Nonfinite intermediate values can propagate into the derivatives and weight updates introduced in Chapter 6 and Chapter 7. The scalar alone does not tell us which intermediate first failed, so the useful debugging boundary is the input logits and the operations that turn them into a loss.

So the need is concrete. We want a loss for classification that avoids preventable overflow when the input logits and their differences are representable. Stable algebra cannot recover a value that was already converted to `inf` before the loss received it.

4.2 The Idea

4.2.1 A loss is one number, and lower is better

↪ **Jump ahead:** Chapter 8 *How scalar loss reductions drive backward gradient propagation and parameter updates.*

Every supervised training pipeline—from continuous noise prediction in diffusion models to coordinate regression in robotics—culminates in an objective function that quantifies task performance. By reducing multidimensional batch predictions into a single scalar value, this objective defines the scalar potential that drives reverse-mode gradient updates. A loss function compares predictions with targets and returns an error score. The losses here are nonnegative, with smaller values indicating better agreement. MSE can reach zero exactly; classification loss approaches zero as the probability of the correct answer approaches one. For regression, where the target is a number, the natural choice is **mean squared error**:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

Here N is the number of predictions, $\hat{y}_i$ is the model's i-th prediction, and y_i is the true value. Squaring makes the error positive and punishes large misses more than small ones. Dividing by N keeps the units as squared error per prediction, so a larger batch does not automatically produce a larger loss. Different samples can still have different mean errors. `MSELoss` will be a few lines, although very large differences can still overflow when squared.[13] Averaging also fixes the meaning of the returned number: duplicating a batch leaves its mean loss unchanged.

[13] In float32 arithmetic, squaring differences between large numbers of similar magnitude (for instance, $10000.02 - 10000.00$) risks catastrophic cancellation in the 24-bit mantissa. High-precision loss kernels accumulate sums in float64 before dividing by N to preserve numerical precision.

4.2.2 Cross-entropy scores the probability of the right answer

The ubiquitous objective powering language model pretraining (such as next-token prediction in GPT and LLaMA) and multi-class image classification is **cross-entropy loss**. Grounded in maximum likelihood estimation, it applies an increasingly steep gradient penalty as the probability assigned to the ground-truth target approaches zero. For classification the model outputs a vector $\mathbf{z} = [z_1, \ldots, z_C]$ of C raw scores, one per class. These are the **logits**, and they can be any real numbers. To read them as probabilities we apply the **softmax** function:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Exponentiating makes every entry positive, and dividing by the total makes them sum to one. The largest logit gets the largest probability, and the gap between logits controls how confident the distribution is; that is the "soft" in softmax.

Given the true class t, **cross-entropy** is the negative logarithm of the probability the model assigned to it:

$$\mathcal{L}_{\text{CE}} = -\log p_t$$

A few values make the shape of this function clear. If the model gives the right class probability 0.99, the loss is 0.01. At 0.5 it is 0.69. At 0.1 it is 2.30, and at 0.01 it is 4.61. The loss is near zero for a confident correct answer and grows without bound for a confident wrong one, which is exactly the pressure we want training to apply. Here and throughout the chapter, `log` means the natural logarithm, the inverse of the exponential. Statisticians call this quantity the negative log-likelihood,[14] and over a batch of N examples we average it, for the same reason MSE divides by N.[15]

The three loss functions in TinyTorch partition the core supervised learning workloads as summarized in table 4.1.

[14] **Negative log-likelihood** (NLL): the negative logarithm of the probability a model assigns to the outcome that actually occurred. For class-index targets without additional weighting or smoothing, this is the log-softmax, select, negate, and average computation implemented below.

[15] Mathematically, cross-entropy relates to Kullback-Leibler (KL) divergence by $H(p, q) = H(p) + D_{\text{KL}}(p \parallel q)$, where p is the target distribution and q is the model prediction. Because one-hot targets have zero entropy ($H(p) = 0$), minimizing cross-entropy directly minimizes the KL divergence between predictions and true labels.

Table 4.1: **Loss function taxonomy**: Architectural contracts, input domains, and numerical stabilization strategies across TinyTorch losses.

Loss Class	Target Type	Input Domain	Stability Mechanism	Primary Task
`MSELoss`	Continuous $\mathbb{R}$	Unbounded predictions	Elementwise mean	Regression
`BinaryCrossEntropyLoss`	Probabilities $[0, 1]$	Probabilities in $[0, 1]$	Epsilon clipping $[\epsilon, 1 - \epsilon]$	Binary classification
`CrossEntropyLoss`	Class indices $\{0, \ldots, C - 1\}$	Unbounded logits $\mathbb{R}^C$	Max-shifted log-softmax & gather	Multi-class classification

Pause & Predict: The Silent Log-Softmax Cancellation

Before deriving the max-shift stabilization, consider what happens when evaluating $\log(\text{softmax}(\mathbf{z}))$ for extreme logits $\mathbf{z} = [1000, 1002, 1001]$. If intermediate probabilities underflow to 0.0 or overflow to `inf`, what happens when you take the logarithm? Can mathematical rearrangement compute the exact logarithm without ever materializing an intermediate exponential?

4.2.3 The fix: shift before you exponentiate

Production deep learning runtimes (such as PyTorch's native ATen kernels and Triton fused-softmax operators) protect classification objectives against floating-point overflow

using the **max-subtraction trick**. Because single-precision IEEE 754 float32 overflows to $+\infty$ for any exponent exceeding 88.72, evaluating raw logits in deep networks would routinely crash training with NaN values. Softmax has a property that saves it. Multiply the numerator and denominator by the same constant e^{-c} and nothing changes:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{-c}\, e^{z_i}}{e^{-c} \sum_j e^{z_j}} = \frac{e^{z_i - c}}{\sum_j e^{z_j - c}}$$

So we may subtract any constant c from every logit before exponentiating, and the probabilities are identical on paper. On hardware they are not identical, and the choice $c = \max(\mathbf{z})$ is the one that makes the computation safe. After the shift, every logit is at most zero, so every $e^{z_i - c}$ is at most $e^0 = 1$ and exponentiation cannot overflow. And the largest logit becomes exactly zero, so the sum contains at least one term equal to 1 and can never be zero. Tiny terms may still underflow to `0.0`, but a term that small was contributing nothing next to a 1 anyway. Both failures in the previous section are gone.

4.2.4 Take the logarithm before you divide

High-performance deep learning systems never compute cross-entropy by chaining separate Softmax, division, and logarithm operations; instead, they execute a single fused **Log-Softmax** routine. This algebraic fusion eliminates intermediate DRAM round-trips while preventing small probabilities from underflowing to zero, which would cause downstream logarithms to evaluate to $-\infty$. The loss never needs the probabilities themselves; it needs $\log p_t$. Taking the logarithm of the shifted softmax turns the division into a subtraction:

$$\log p_i = (z_i - c) - \log \sum_{j=1}^{C} e^{z_j - c}$$

The vector of all C values of $\log p_i$ is the **log-softmax** of $\mathbf{z}$, and the scalar $c + \log \sum_j e^{z_j - c}$ is called the **log-sum-exp** of $\mathbf{z}$. It is a safe way to compute $\log \sum_j e^{z_j}$, and the cross-entropy loss is simply the log-sum-exp minus the target logit. That rearrangement is the algebra. The code keeps one more detail straight, which is that $z_t - c$ must be computed first, as its own small number, before the log of the sum is subtracted. The worked trace shows why, with a case where the other order is wrong in the second decimal place.

The invariant: In computing $\log \sum_j e^{z_j}$, shifting by $c = \max_j(z_j)$ ensures that every exponent $z_j - c \leq 0$, bounding all terms $e^{z_j - c} \in [0, 1]$. Because the maximum element produces $e^0 = 1$, the sum is guaranteed to satisfy $\sum_j e^{z_j - c} \geq 1$, completely precluding both overflow (∞) and underflow to zero (0).

Precision rule: Always evaluate $(z_t - c) - \log \sum_j e^{z_j - c}$ instead of $z_t - (c + \log \sum_j e^{z_j - c})$. In float32, combining large c with small log-sums obliterates low-order mantissa bits.

4.3 The Code

Module 04 follows the pattern Chapter 2 set. Each loss is a pair of classes: an operation, a `Function` subclass whose `forward` takes NumPy arrays and returns a numerical result,

and a thin wrapper that takes tensors and hands them to the operation through `apply`. The wrapper is what the caller uses. For `MSELoss()(predictions, targets)`, Python enters `MSELoss.__call__`, which calls `forward`, which delegates to `MSEFunction.apply`. That shared method passes the tensors' arrays to `MSEFunction.forward` and wraps its numerical result in a `Tensor`. Chapter 6 will add the record of how the result was computed and the derivative rules that use it.

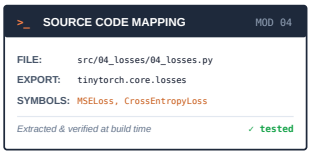
4.3.1 A loss is one scalar in a `Tensor`

The loss is the boundary between scoring predictions and deciding how to improve them. This module enforces a strict architectural contract: the loss must collapse multi-dimensional predictions into a single 0-dimensional scalar `Tensor` of shape `()`.

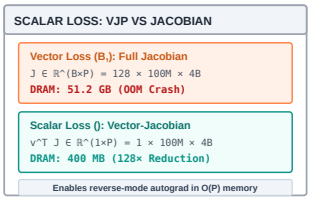
This scalar reduction is not merely a design preference; it is a foundational systems imperative for scalable automatic differentiation. If a loss function returned a vector of per-sample errors of shape $(B,)$ for a batch size $B = 128$ evaluating a modest model with $P = 100 \times 10^6$ parameters, reverse-mode automatic differentiation would be forced to compute and materialize a full Jacobian matrix of partial derivatives of shape (B, P). That Jacobian would require $128 \times 100{,}000{,}000 \times 4$ bytes ≈ 51.2 GB of DRAM allocations! By collapsing the objective to a single scalar through batch averaging ($\frac{1}{B}\sum_{i=1}^{B} \mathcal{L}_i$), reverse-mode accumulation collapses the entire backward pass to a single vector-Jacobian product, reducing the required gradient memory from 51.2 GB to a single parameter-sized buffer of 400 MB.

In Chapter 6, we formalize this through the **scalar loss contract**: because the objective function $\mathcal{L}$ evaluates to a 0-D scalar, reverse accumulation seeds the backward graph traversal with the scalar identity seed $\partial\mathcal{L}/\partial\mathcal{L} = 1.0$. This allows TinyTorch to propagate gradient vectors directly backward through every layer without allocating multi-gigabyte Jacobian matrices.

The cheaper thing to return would be a bare Python float, and for this chapter a float would work. It stops working in Chapter 6, where `backward()` walks from the loss back through every operation that produced it, and a float has no place to record where it came from. So every loss returns a `Tensor` holding a single value, and `apply` guarantees it: whatever the operation's `forward` returns, even a NumPy scalar, comes back wrapped. `MSELoss` is the smallest pair that satisfies the contract.



Source Code Mapping: Module 04 extracts MSELoss, numerically stabilized CrossEntropyLoss, and BinaryCrossEntropyLoss directly from the working repository.



Scalar Loss Reduction Invariant: Collapsing multi-sample predictions to a scalar prevents materializing a 51.2 GB Jacobian during reverse-mode autograd.

```
class MSEFunction(Function):
    """
    The MSELoss operation. forward() works on NumPy arrays; Module 06 adds backward().
    """

    def forward(self, predictions, targets):
        """
        Compute mean squared error between predictions and targets.
        """
        if predictions.shape != targets.shape or predictions.size == 0:
            raise ValueError("MSELoss requires matching, nonempty prediction and target shapes")
```

```
        # Step 1: Compute element-wise difference
        diff = predictions - targets

        # Step 2: Square the differences
        squared_diff = diff ** 2

        # Step 3: Take mean across all elements
        mse = np.mean(squared_diff)

        return mse
```

```
class MSELoss:
    """Mean Squared Error loss for regression tasks."""

    def __init__(self):
        """Initialize the loss function."""
        pass

    def forward(self, predictions: Tensor, targets: Tensor) -> Tensor:
        """Compute the loss through its operation, so Module 06 will
        ↪ be able to record it for gradients."""
        return MSEFunction.apply(predictions, targets)

    def __call__(self, predictions: Tensor, targets: Tensor) ->
    ↪ Tensor:
        """Allows the loss function to be called like a function."""
        return self.forward(predictions, targets)
```

The output has shape `()`, NumPy's shape for a scalar. The loss object has no learned parameters or running statistics. Subtraction, squaring, and averaging create results without changing either input array; another loss can safely score the same predictions. Chapter 6 will use this scalar tensor as the starting point for computing derivatives. Notice that `np.mean` averages *all elements*. For predictions and targets of shape `(B, D)`, it divides by `B * D`, not just by `B`. The intended contract is matching shapes; a target of shape `(B,)` against predictions `(B, 1)` would broadcast into `(B, B)` before the mean and score the wrong pairs. The function rejects mismatched or empty shapes before subtraction, so NumPy cannot silently turn this mistake into a different objective. The caller can therefore rely on one scalar mean over corresponding elements, or an error explaining that the input shapes do not match.

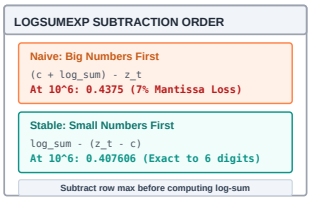
4.3.2 `LogSoftmax`, the max shift in code

For a batch with shape `(2, 3)`, each row holds three class scores. The loss must normalize each row separately: the first example's scores must not affect the second example's probabilities. Later classifiers will increase the number of rows and classes, but this row-wise computation stays the same.

The naive way first computes `e = np.exp(z)`, then `np.log(e / np.sum(e))`. The Problem showed the loud failure, `nan` for any logit past 88.7. There is a quiet one too. Even when nothing overflows, a probability that underflows to `0.0` goes into `log` and comes

out `-inf`. In float32, `log(softmax([-110, 0]))` prints `[-inf, 0.]`, where the stable log-probabilities round to `[-110, 0]`; the naive form has thrown away a perfectly finite number. The composition also creates separate intermediate arrays for exponentiation, division, and logarithms.

The fix is The Idea's two moves, in that order. Subtract each row's maximum, so every exponential is at most 1 and every sum is at least 1. Then take the log of the sum and subtract it from the shifted logits, so no probability is ever formed and nothing is ever divided. The operation normalizes along the selected axis `dim` and returns log-probabilities of the same shape. For `(N, C)` classification scores, `dim=-1` selects the class axis; a one-line function wraps it for callers holding a `Tensor`.



Subtraction Ordering: Evaluating `x - max_vals - log_sum_exp` left-to-right preserves 24-bit mantissa precision; grouping terms causes catastrophic cancellation.

```python
class LogSoftmax(Function):
    """
    The log-softmax operation along an axis. forward() works on NumPy arrays; Module 06 adds backward().
    """
    dim = -1

    def forward(self, x):
        """
        Compute log-softmax with numerical stability.
        """
        # Step 1: Find max along dimension for numerical stability
        max_vals = np.max(x, axis=self.dim, keepdims=True)

        # Step 2: Subtract max to prevent overflow
        shifted = x - max_vals

        # Step 3: Compute log(sum(exp(shifted)))
        log_sum_exp = np.log(np.sum(np.exp(shifted), axis=self.dim, keepdims=True))

        # Step 4: Return log_softmax = input - max - log_sum_exp
        result = x - max_vals - log_sum_exp

        return result
```

```python
def log_softmax(x: Tensor, dim: int = -1) -> Tensor:
    """Compute log-softmax of a Tensor along dim (see LogSoftmax)."""
    return LogSoftmax.apply(x, dim=dim)
```

Notice two things. The `keepdims=True` keeps the per-row maximum as a column of shape `[N, 1]` rather than a flat vector of length `N`, so that subtracting it from the `[N, C]` logits lines up one maximum with each row; that is the broadcasting rule from Chapter 1 doing the work. And the last line is `x - max_vals - log_sum_exp`, which Python evaluates left to right, so the shift happens first and the log of the sum is subtracted from a number that is already small. Written as `x - (max_vals + log_sum_exp)` it would be equal on paper and wrong in float32, for the reason the trace makes visible. The maximum and the logarithm of the sum have shape `(N, 1)`; the shifted logits and the returned log-probabilities have shape `(N, C)`. `log_softmax(x, dim)` passes the axis

through `apply`, which stores it on the new operation as `self.dim` before calling `forward`. No input is overwritten.

4.3.3 `CrossEntropyLoss` gathers instead of multiplying

Labels identify classes by index: with three classes, each target must be 0, 1, or 2. Chapter 5 will build the loader that keeps these labels paired with the corresponding inputs. With log-probabilities in hand, the loss has to pick, for row `i`, the entry in column `target[i]`, negate it, and average over the batch.

The textbook formula does not say "pick." It says $-\sum_i y_i \log p_i$, where a one-hot[16] target puts a 1 at the correct class and zeros elsewhere. For target class 2 among three classes, that vector is `[0, 0, 1]`. Multiplying by it retains only the third log-probability. Expanding targets into an $[N, C]$ matrix of zeros with a single 1 per row, multiplying it into the log-probabilities elementwise, and summing across columns incurs $N \times C$ multiplications to keep N numbers. For vocabulary sizes in language models ($C = 32{,}000$) and batch sizes $N = 2048$, allocating an explicit one-hot matrix demands $2048 \times 32{,}000 \times 4$ bytes ≈ 262 MB of memory and executes 65.5 million multiplications, of which 99.997 percent multiply by zero. Can we select the target entries directly without allocating that matrix?[17]

NumPy can. Indexing an array with two integer arrays of the same length pairs them up elementwise, so `log_probs[np.arange(N), target_idx]` pairs row `i` with column `target_idx[i]` and selects `N` entries into a new vector of shape `(N,)`. For `N=2` and targets `[0, 1]`, the pairs are `(row 0, column 0)` and `(row 1, column 1)`. Computing the log-probabilities still visits all `N*C` logits; the saving is in avoiding hundreds of megabytes of intermediate one-hot memory traffic and eliminating redundant arithmetic.

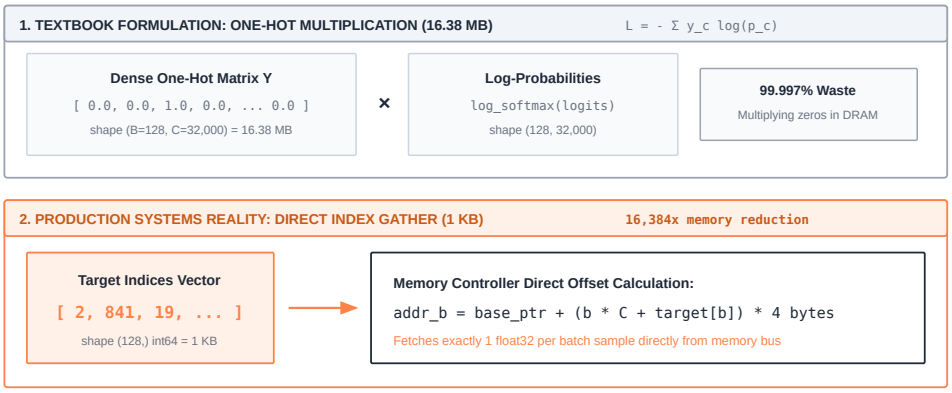


Figure 4.2: **Sparse index gathering vs. one-hot expansion**: Expanding target labels into dense one-hot vectors allocates $O(B \times C)$ memory and performs $B \times C$ redundant multiplications by zero. Direct integer index gathering ($O(B)$ memory) fetches the target logit via row-offset indexing without constructing intermediate matrices.

```
class CrossEntropyFunction(Function):
    """
    The CrossEntropyLoss operation. forward() works on NumPy arrays;
    Module 06 adds backward().
```

[16] **One-hot**: a vector with a 1 in the position of the correct class and 0 everywhere else, so a label of class 2 among 5 classes becomes `[0, 0, 1, 0, 0]`. For a batch of N examples it takes $N \times C$ numbers to store what N integers already say, which is why the code indexes with the integers instead.

[17] In hardware terms, direct integer index gathering translates to indirect memory addressing (`log_probs[i * C + target[i]]`). Instead of streaming $B \times C$ floats across the memory bus to multiply by sparse zeros, the gather reads only B selected values instead of multiplying through $B \times C$ one-hot entries (the log-softmax itself still reads every logit, as the next paragraph notes).

```
    """

    def forward(self, logits, targets):
        """
        Compute cross-entropy loss between logits and target class indices.
        """
        if logits.ndim < 2 or logits.size == 0 or targets.shape != logits.shape[:-1]:
            raise ValueError("CrossEntropyLoss requires nonempty logits (..., classes) and targets (...)")
        if not np.all(np.isfinite(targets)) or np.any(targets != np.floor(targets)):
            raise ValueError("CrossEntropyLoss targets must be finite integer class indices")

        num_classes = logits.shape[-1]
        if num_classes == 0:
            raise ValueError("CrossEntropyLoss requires nonempty logits (..., classes) and targets (...)")

        flat_logits = logits.reshape(-1, num_classes)
        flat_targets = targets.reshape(-1)
        batch_size = flat_logits.shape[0]

        out_of_range = (flat_targets < 0) | (flat_targets >= num_classes)
        if np.any(out_of_range):
            bad_values = np.unique(flat_targets[out_of_range])
            raise ValueError(
                f"CrossEntropyLoss target index out of range: {bad_values.tolist()}\n"
                f"  Valid range for {num_classes} classes is [0, {num_classes - 1}]"
            )

        # Validate before casting: conversion would silently truncate fractional labels.
        target_indices = flat_targets.astype(int)
        log_probs = LogSoftmax().forward(flat_logits)

        # Select correct class log-probabilities using advanced indexing
        selected_log_probs = log_probs[np.arange(batch_size), target_indices]

        # Step 3: Return negative mean (cross-entropy is negative log-likelihood)
        cross_entropy = -np.mean(selected_log_probs)

        return cross_entropy
```

```python
class CrossEntropyLoss:
    """Cross-entropy loss for multi-class classification."""

    def __init__(self):
        """Initialize the loss function."""
        pass

    def forward(self, logits: Tensor, targets: Tensor) -> Tensor:
        """Compute the loss through its operation, so Module 06 will
        ↪  be able to record it for gradients."""
        return CrossEntropyFunction.apply(logits, targets)

    def __call__(self, logits: Tensor, targets: Tensor) -> Tensor:
        """Allows the loss function to be called like a function."""
        return self.forward(logits, targets)
```

Targets represent integer class IDs, but their storage is float32: `Tensor([0, 1])` contains `[0.0, 1.0]`. `targets.astype(int)` creates the integer array required by the gather. The expected shape is `(N,)`, paired with logits of shape `(N, C)`. Before conversion, the operation checks that the original labels are finite whole numbers and rejects IDs outside the class range. Nothing builds a one-hot matrix merely to multiply most of it by zero. The range check exists because NumPy treats a negative index as counting from the end, so a corrupted label of `-1` would silently select the last class instead of failing. After validation and conversion, the operation calls `LogSoftmax().forward` on the array directly. Calling the tensor wrapper here would add an unnecessary Tensor boundary inside an array computation. The selected log-probabilities form a temporary `(N,)` vector; negating its mean returns a scalar to `apply`, which wraps it for the caller. This forward method does not cache the log-probabilities or change the logits or targets. Chapter 6 will add a derivative rule that recomputes the probabilities from the saved inputs.

4.3.4 `BinaryCrossEntropyLoss`, finite logs through clipping

The binary loss has a different input contract. With probabilities `[0.9, 0.2]` and targets `[1, 0]`, the first example contributes `-log(0.9)` and the second contributes `-log(0.8)`. Their mean is about `0.1643`. A single formula handles both cases: `-(y*log(p) + (1-y)*log(1-p))`. Target 1 keeps the first term; target 0 keeps the second.

```python
class BinaryCrossEntropyFunction(Function):
    """
    The BinaryCrossEntropyLoss operation. forward() works on NumPy
    ↪  arrays; Module 06 adds backward().
    """

    def forward(self, predictions, targets):
        """
        Compute binary cross-entropy loss.
        """
        if predictions.shape != targets.shape or predictions.size ==
        ↪  0:
```

```
            raise ValueError("BinaryCrossEntropyLoss requires matching, nonempty prediction and target shapes")
        for values in (predictions, targets):
            if not np.all(np.isfinite(values)) or np.any((values < 0) | (values > 1)):
                raise ValueError("BinaryCrossEntropyLoss predictions and targets must be finite values in [0, 1]")

        # Step 1: Clamp predictions to avoid numerical issues with log(0) and log(1)
        eps = EPSILON
        clamped_preds = np.clip(predictions, eps, 1 - eps)

        # Step 2: Compute binary cross-entropy
        # BCE = -(targets * log(preds) + (1-targets) * log(1-preds))
        log_preds = np.log(clamped_preds)
        log_one_minus_preds = np.log(1 - clamped_preds)

        bce_per_sample = -(targets * log_preds + (1 - targets) * log_one_minus_preds)

        # Step 3: Return mean across all samples
        bce_loss = np.mean(bce_per_sample)

        return bce_loss
```

The first check rejects mismatched or empty shapes. The next checks require finite probabilities and targets in `[0, 1]`; intermediate target values are allowed, and weight the two logarithms accordingly. `np.clip` then creates a new probability array bounded by `1e-7` and `1-1e-7`, leaving the input unchanged.[18] Both `p` and `1-p` are positive at the logarithms. The two logged arrays and `bce_per_sample` have the input shape, and `np.mean` reduces every element to one scalar, including every output in a `(B, D)` input.

[18] In exact real analysis, $\lim_{p\to 0^+} p \ln p = 0$. In IEEE 754 floating-point hardware, however, $\ln(0.0) = -\infty$, and the arithmetic expression $0.0 \times (-\infty)$ evaluates immediately to NaN. Bounding $p \in [\epsilon, 1-\epsilon]$ guarantees strictly positive arguments to $\ln(\cdot)$ and finite intermediate values.

```
class BinaryCrossEntropyLoss:
    """Binary cross-entropy loss for binary classification."""

    def __init__(self):
        """Initialize the loss function."""
        pass

    def forward(self, predictions: Tensor, targets: Tensor) -> Tensor:
        """Compute the loss through its operation, so Module 06 will be able to record it for gradients."""
        return BinaryCrossEntropyFunction.apply(predictions, targets)

    def __call__(self, predictions: Tensor, targets: Tensor) -> Tensor:
        """Allows the loss function to be called like a function."""
        return self.forward(predictions, targets)
```

The wrapper follows the same path as MSE: `__call__` delegates to `forward`, which enters the operation through `apply` and returns the wrapped scalar. Clipping keeps

boundary inputs finite but makes the loss constant beyond its thresholds: predictions `4e-8` and `6e-8` against target 1 both produce about `16.1181`. Increasing the confidence of that wrong prediction no longer changes its score. Chapter 6 will differentiate this actual clipped function, including its flat regions. Cross-entropy on logits avoids this probability-clipping limit by retaining the target's log-probability directly.

4.3.5 Checking the scalar contract

Here is a short run of MSE and cross-entropy, on numbers small enough to check by hand. It is good practice to keep a sanity check like this next to every loss implementation, because a loss that returns a plausible-looking wrong number is the kind of bug that trains for a week before anyone notices.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.losses import MSELoss, CrossEntropyLoss

logits = Tensor(np.array([[2.0, 1.0, 0.1],
                          [0.5, 1.5, 0.8]], dtype=np.float32))
targets = Tensor(np.array([0, 1]))
loss = CrossEntropyLoss()(logits, targets)
print(f"cross-entropy: {loss.data:.4f}")   # 0.5200
print("shapes:", logits.shape, targets.shape, loss.shape)  # (2, 3)
    (2,) ()
repeated = CrossEntropyLoss()(
    Tensor(np.concatenate([logits.data, logits.data])),
    Tensor(np.concatenate([targets.data, targets.data])),
)
assert np.allclose(repeated.data, loss.data)  # the objective is a
    mean

preds = Tensor(np.array([1.0, 2.0, 3.0], dtype=np.float32))
truth = Tensor(np.array([1.5, 2.5, 2.8], dtype=np.float32))
print(f"mse: {MSELoss()(preds, truth).data:.4f}")  # 0.1800
```

The shape print follows the interface through the call: two rows of class scores and two IDs become a scalar with shape `()`. Duplicating both inputs leaves that scalar unchanged, checking the reduction independently of the softmax arithmetic. The MSE value is $(0.5^2 + 0.5^2 + 0.2^2)/3 = 0.18$. The cross-entropy value comes from the row-by-row trace that follows.

4.4 A Worked Trace

Table 4.2 takes the first row of the batch above, logits $\mathbf{z} = [2.0, 1.0, 0.1]$ with target $t = 0$, and follows it through `LogSoftmax.forward` one line at a time.

Table 4.2: Hand trace of `log_softmax` and `CrossEntropyLoss` on one row of logits $[2.0, 1.0, 0.1]$ with target class 0.

Step	Computation	Value
1. Row maximum	$c = \max(\mathbf{z})$	2.0
2. Shift	$\mathbf{z} - c$	$[0.0, -1.0, -1.9]$
3. Exponentiate	$e^{\mathbf{z}-c}$	$[1.0000, 0.3679, 0.1496]$
4. Sum	$\sum_j e^{z_j - c}$	1.5174
5. Log of the sum	$\log 1.5174$	0.4170
6. Log-probabilities	$(\mathbf{z} - c) - 0.4170$	$[-0.4170, -1.4170, -2.3170]$
7. Pick target, negate	$-\log p_0$	0.4170

Exponentiating the log-probabilities gives the probabilities $[0.659, 0.242, 0.099]$, which sum to one as they must. The model puts 66 percent on the right class and pays a loss of 0.417 for its remaining doubt. The second row, $[0.5, 1.5, 0.8]$ with target 1, works through the same steps to a loss of 0.623, and the batch mean is $(0.417 + 0.623)/2 = 0.520$, which matches the printed value.

The same seven steps on the logits that broke the naive formula show the shift doing its job. With $\mathbf{z} = [1000, 1002, 1001]$ and $t = 1$, the naive and stable paths diverge at the very first exponential.

Table 4.3: The naive and max-shifted paths on the extreme logits $[1000, 1002, 1001]$ with target class 1.

Step	Naive softmax	Max-shifted log-softmax
Exponentiate	$e^{1000}, e^{1002}, e^{1001} \to$ `inf`	$c = 1002$, shifted $[-2, 0, -1]$, $e^{\cdot} = [0.1353, 1.0, 0.3679]$
Sum	`inf`	1.5032
Normalize or take log	`inf / inf` $=$ `nan`	$\log 1.5032 = 0.4076$
Loss	$-\log($`nan`$) =$ `nan`	$-(0 - 0.4076) = 0.4076$

Table 4.3 shows that the shifted exponential stage handles only values between 0.1353 and 1, and the answer, 0.4076, is the same loss the row would have if all three logits were 998 smaller. That is the shift invariance from The Idea, now visible as arithmetic.

One last detail hides in the final row. The loss can be written as $\mathrm{LSE}(\mathbf{z}) - z_t = 1002.4076 - 1002$, or as $-((z_t - c) - \log \mathrm{sum}) = -(0 - 0.4076)$. On paper these are equal. In float32 they are not, because a float32 carries about seven significant digits, and 1002.4076 has already spent four of them on the integer part. Adding 1002 to 0.4076 and then subtracting 1002 throws away the digits that matter.

```
import numpy as np

```

```
for base in (1e3, 1e6):
    z = np.array([base, base + 2.0, base + 1.0], dtype=np.float32)
    c = np.max(z)
    log_sum = np.log(np.sum(np.exp(z - c)))      # 0.40760604 in every case
    lse_first = (c + log_sum) - z[1]             # big numbers first
    shift_first = log_sum - (z[1] - c)           # small numbers first
    print(f"logits near {base:.0e}: {lse_first:.6f} vs {shift_first:.6f}")
# logits near 1e+03: 0.407593 vs 0.407606
# logits near 1e+06: 0.437500 vs 0.407606
```

In the snippet above, line 4 constructs test logits with large baseline magnitudes (10^3 and 10^6), and line 5 finds the row maximum c. Line 6 computes the stabilized log-sum of exponents. Line 7 evaluates the naive arithmetic ordering `lse_first = (c + log_sum) - z[1]` ("big numbers first"), while line 8 computes TinyTorch's stabilized formulation `shift_first = log_sum - (z[1] - c)` ("small numbers first").

As reported by line 9 in the trailing comments (lines 10 to 11), at logits near a thousand the error already corrupts the fifth decimal place. At logits near a million, where float32 can only represent multiples of 0.0625, the "big numbers first" form is off by 0.03 (a 7 percent error in the loss), while the stabilized form remains accurate to six digits. This is why `LogSoftmax` subtracts the maximum before the log of the sum and why `CrossEntropyFunction` gathers from that array rather than computing the log-sum-exp as a separate scalar. Overflow is the loud failure; loss of precision is the quiet one, and the same subtraction order fixes both.

4.5 From TinyTorch to Production

For ordinary class-index targets, PyTorch's `nn.CrossEntropyLoss` implements the same objective as the listing: log-softmax followed by target selection and reduction. Its API also accepts class weights, ignored labels, smoothed labels, and probability targets. Those options change how class or sample contributions are combined; probability targets require a weighted sum across classes. PyTorch CrossEntropyLoss reference.

The input distinction is easy to test before training. For logits `[10, 0]` and target 0, the loss is about `0.0000454`. Passing their softmax probabilities, approximately `[0.999955, 0.0000454]`, to the same loss makes it interpret those two probabilities as new logits and produces about `0.3133`. Both calls return plausible finite numbers, but they score different distributions. A classifier using this loss should therefore return raw logits.

The NumPy listing exposes the memory work as well as the arithmetic. It computes a row maximum, allocates shifted logits, exponentiates them, sums along the class axis, and forms the final log-probabilities with two subtractions. The elementwise results have the size of the logits; the row reductions are smaller. The implementation even evaluates `x - max_vals` again in its final expression. These allocations and repeated reads are visible costs, although source inspection alone does not establish which cost dominates elapsed time on a particular machine.

A production **fused kernel** combines several array operations into one device routine. Instead of writing every intermediate array to main device memory (DRAM) and reading it back, it retains intermediate values in fast on-chip SRAM. The Triton softmax tutorial

demonstrates this for rows that fit locally; fusion eliminates intermediate memory traffic rather than the arithmetic itself. Chapter 18 will examine how to combine operations this way. Triton fused-softmax tutorial.

The vocabulary VRAM bottleneck: In modern LLMs such as LLaMA, vocabulary size V ranges from 32,000 to 128,000 (TinyGPT's character vocabulary is far smaller). For batch size $B = 16$ and sequence length $S = 2048$, allocating an unfused FP32 logits tensor of shape (B, S, V) requires $16 \times 2048 \times 32{,}000 \times 4$ bytes ≈ 4.19 GB. Materializing separate intermediate tensors for shifted logits, exponentials, and log-softmax can consume over 16 GB of VRAM just to compute the loss!

Online LogSumExp fusion: Production engines (such as Flash-Cross-Entropy or Triton kernels) stream through the vocabulary directly from the final linear layer projection, computing the running maximum and LogSumExp in fast GPU registers (SRAM) without ever materializing the massive (B, S, V) logit tensor in DRAM.

The earlier sparse index gather avoids allocating an intermediate one-hot target matrix of this magnitude, while normalization still processes every class score.

Precision is a separate concern. A narrow floating-point format stores fewer significant digits and may have a smaller range. The max shift protects the exponential arguments, but it cannot restore information lost before the loss receives its logits. PyTorch's CUDA autocast mode, which selects arithmetic types for eligible operations, runs eligible `softmax`, `log_softmax`, and `cross_entropy` calls in float32. That policy applies within the specified execution context; it is not a promise about every backend or explicitly typed call. TinyTorch uses float32 tensors throughout. PyTorch autocast operation reference.

Table 4.4: What carries over from the TinyTorch loss to `torch.nn.CrossEntropyLoss` and what production adds.

	TinyTorch	PyTorch
Input	Logits, shape `[N, C]`	Logits, shape `(N, C)` or with extra spatial dimensions
Targets	Integer class indices	Indices or class probabilities; `ignore_index`, `label_smoothing`, class weights
Stability	Max shift inside `log_softmax`	Same
Passes over the logits	Separate reductions and elementwise passes; several full-size temporaries	Specialized kernels can reduce memory traffic
Precision	float32 throughout	Eligible loss calls promoted under CUDA autocast

Table 4.4 summarizes the interface and execution differences. Three checks carry over. Pass logits to cross-entropy and check the shape and label range before tuning training. Keep the calculation in log-space, so an underflowed probability cannot become an infinite loss. Finally, compare with a uniform baseline: equal logits assign each class probability $1/C$, giving loss $\log C$, about 2.30 for ten classes. Random initialization need not produce exactly equal logits, but this baseline gives the returned number a useful

scale. Agreement with that baseline does not prove that labels and predictions are paired correctly. A large or nonfinite loss calls for inspecting both the logits and the target contract before changing the optimizer.

4.6 Summary

A caller enters a loss wrapper with predictions and targets and receives a scalar `Tensor` with shape `()`. MSE checks matching, nonempty shapes and averages squared differences over every element. Cross-entropy checks `(N, C)` logits and `(N,)` class indices, computes row-wise log-probabilities, selects one value per row, and returns their negative mean. Neither forward computation changes its inputs. Binary cross-entropy accepts probabilities instead and clips them before taking logarithms, creating flat regions near the endpoints.

The numerical invariant is the max shift: for representable finite differences, every shifted logit is nonpositive and at least one is zero. Exponentials are therefore at most 1, and their row sum is at least 1. Keeping the target log-probability in log-space also avoids taking `log(0)`. The precision trace adds an ordering rule: compute `x - max_vals` before subtracting the log of the sum. Production implementations can reduce memory traffic while preserving this objective. Chapter 5 now asks where the matched batches come from and how to assemble them without losing the pairing between each prediction's input and its label.

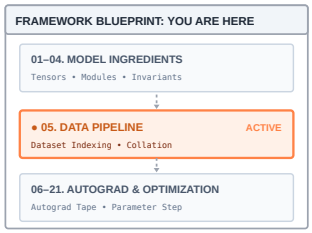
4.7 Further Reading

- **Lin et al.** (2017). "Focal loss for dense object detection." *ICCV*. A reshaped cross-entropy that shows how the choice of loss changes which examples drive the gradient.
- **Müller, Kornblith, and Hinton** (2019). "When does label smoothing help?." *NeurIPS*. What happens to cross-entropy's targets and to the learned representation when the one-hot label is softened.

🔥 Chapter 5

Dataloaders: Batching and Memory Management

High-throughput deep learning requires feeding compute engines with dense, contiguous batches of data. Real-world datasets rarely arrive as ready-to-compute contiguous tensors in main memory: raw inputs may consist of individual sensor readings, text files, or image streams stored across disk or networked storage. Processing inputs one sample at a time fails to saturate modern vector hardware, leaving memory buses and compute units idle. Conversely, loading and materializing an entire multi-gigabyte dataset into a single monolithic tensor exhausts system DRAM. Mini-batching resolves this tension: it groups discrete samples into fixed-size contiguous tensors that saturate matrix multiplication units, while random index shuffling ensures independent sample order across training epochs.



Framework Blueprint: Module 05 feeds the compute engine by batching, shuffling, and collating samples into contiguous memory.

In PyTorch, data loading and batching are encapsulated in a standard iterator abstraction:

```python
import torch
from torch.utils.data import TensorDataset, DataLoader

# Dataset of 100 feature vectors and labels
features = torch.randn(100, 16)
labels = torch.randint(0, 2, (100,))
dataset = TensorDataset(features, labels)

# Batching, shuffling, and iterator orchestration
loader = DataLoader(dataset, batch_size=32, shuffle=True)
for batch_x, batch_y in loader:
    print("Batch shape:", batch_x.shape, batch_y.shape)  # (32, 16), (32,)
    break
```

Underneath that clean `for` loop, the data pipeline coordinates three crucial systems responsibilities:

1. **Index permutation without data duplication:** To shuffle the dataset at the start of every epoch, the loader permutes an array of integer indices rather than copying or rearranging the underlying feature tensors in memory.
2. **Batch collation into contiguous memory:** The collate function extracts individual sample tensors and stacks them into a single contiguous row-major batch tensor $(B, \dots)$ so vectorized matrix-multiplication kernels can process them at high arithmetic throughput.

3. **Iterator state and boundary handling:** The runtime manages epoch cursor state, gracefully yielding the final partial batch when the dataset size is not an exact multiple of the batch size.

This chapter builds TinyTorch's data loading pipeline, establishing the contiguous batch stream that feeds the forward passes of Chapter 3 and powers the end-to-end training loop in Chapter 8. Before building this pipeline, we must inspect the performance and convergence penalties incurred when data is fed naively.

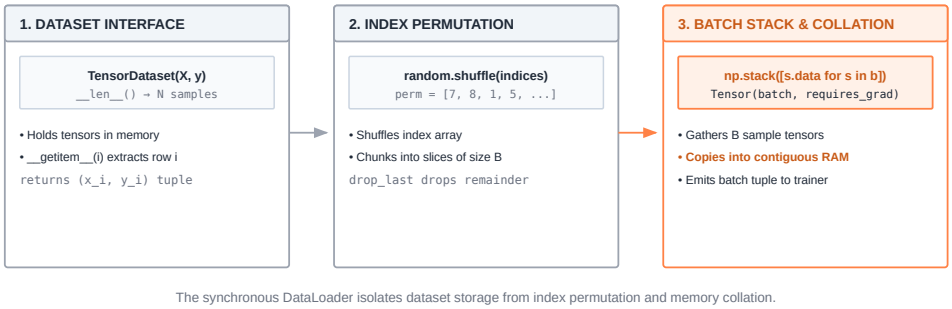


Figure 5.1: TinyTorch's synchronous data path. TensorDataset implements indexed sample access; DataLoader selects indices and its iterator fetches and stacks the corresponding fields. The caller receives one tuple of Tensor batches, and the final batch may be short.

5.1 The Problem

↩ **Jump back:** Chapter 1 *Batch collation packs scattered dataset samples into contiguous row-major tensor batches.*

Suppose the training data is a list of samples and the labels arrive sorted by class. The obvious loop feeds them one at a time, in file order:

```
samples = [([i, i + 0.5], 0 if i < 5 else 1) for i in range(10)]   # x_i, y_i

for x, y in samples:          # one sample per step, in stored order
    pass                      # forward(x), loss(y), update
```

This loop misses two opportunities: sharing work across samples and mixing classes within each batch.

The first is waste. Chapter 3 built `Linear.forward` as a matrix multiply on a (B, D) input, and every call carries setup work, including Python dispatch and entering the array operation. Some of that cost is paid regardless of B, while the result size and numerical work grow with the batch. At $B = 1$ that fixed cost is paid once per sample. At $B = 64$ it is paid once per 64 samples, and the arithmetic itself is the same. Feeding one row at a time throws away the batching that the whole forward pass was designed around.

The second is bias. Group the loop above into fours and look at the labels each group carries:

```python
batches = [samples[i:i + 4] for i in range(0, 10, 4)]
print([[y for _, y in batch] for batch in batches])
# [[0, 0, 0, 0], [0, 1, 1, 1], [1, 1]]
```

The first batch is entirely class 0. Its loss measures only performance on that class, so using it to guide an update gives no information about class 1. Long runs of one class can pull the model toward recent samples and away from earlier ones. Chapter 6 and Chapter 7 will explain how a loss determines an update. For now, the problem is visible in the labels alone: each successive batch presents a different part of the task. Disk-backed samples add a separate cost, the wait for reading and decoding, which the production section returns to.

Problem: An accelerator consumes batches of size $B = 64$ with $D = 768$ float32 values every 2.5 ms $(400$ steps/s$)$. What host data bandwidth is required to prevent GPU starvation?

Data consumption rate:

$$\text{Batch size} = 64 \times 768 \times 4 \text{ bytes} = 196{,}608 \text{ bytes} \approx 192 \text{ KB}$$

$$\text{Throughput} = 196{,}608 \text{ bytes} \times 400 \text{ steps/s} \approx 78.6 \text{ MB/s}$$

The I/O bottleneck: If samples are read individually from non-contiguous storage (random SSD reads at 0.1 ms per sample), loading 64 samples sequentially takes 64×0.1 ms $= 6.4$ ms, starving the 2.5 ms GPU step and dropping GPU utilization to $\frac{2.5}{2.5+6.4} \approx 28\%$. Production profilers such as PyTorch Profiler and Nsight Systems (see the production bridge of Chapter 15) expose this condition as periodic idle gaps on the accelerator timeline while the host prepares the next batch.

5.2 The Idea

The separation in table 5.1 lets sample access and batching change independently. The order policy determines which samples share a batch.

↪ **Jump ahead:** Chapter 8 *The DataLoader mini-batch stream powers the unified five-step training execution loop.*

Table 5.1: The three roles of a data pipeline. Each one can be replaced without touching the other two.

Role	Question it answers	Interface
Dataset	How many samples are there, and what is sample i?	`__len__()` returns N; `__getitem__(i)` returns $(\mathbf{x}_i, y_i)$
Order	In what sequence are the indices visited?	A list of the integers 0 through $N-1$, optionally shuffled once per epoch
DataLoader	How do B consecutive samples become one tensor?	Walks the order in strides of B, stacks each stride into (B, D)

A **Dataset** is any object that can count itself and answer index lookups. It knows nothing about batches or order. That is what makes the split useful. A dataset of ten rows

in memory and a dataset of ten million images on disk present the same two methods, and the loader on top of them does not change.

Pause & Predict: The Cost of Sampling with Replacement

Suppose an epoch consists of N training steps on a dataset of N images. If you sample images uniformly at random *with replacement* instead of shuffling a permutation *without replacement*, what fraction of the dataset do you expect will never be seen during that epoch? (Hint: consider the limit $\lim_{N\to\infty}(1-1/N)^N$.)

An **epoch** is one complete pass over the dataset, every sample visited exactly once. At the start of each pass the loader writes down the indices $0, 1, \dots, N-1$ and, when `shuffle=True`, shuffles them.[19] The shuffled list is a **permutation**, written π, of the indices. Position k of the epoch visits sample $\pi(k)$, and because π is a rearrangement rather than a redraw, every index appears exactly once. This is sampling *without replacement*.[20] Drawing indices at random *with* replacement would visit some samples twice and others not at all in a given epoch, and the loss would be tuned to whichever samples happened to be drawn more often.

Shuffling removes the fixed class ordering. Under an ideal uniform shuffle, a batch at a fixed position is a uniformly selected subset of the dataset. Write ℓ_i for the loss on sample i and $\mathcal{B}$ for the set of B indices in one batch. The batch loss is the average $\frac{1}{B}\sum_{i\in\mathcal{B}}\ell_i$. For example, with losses `[1, 3, 5]` and batch size two, the possible subset means are 2, 3, and 4. Their average is 3, the full-dataset mean. The symbol $\mathbb{E}$ expresses this average over possible selections. For a uniform subset,

$$\mathbb{E}\left[\frac{1}{B}\sum_{i\in\mathcal{B}}\ell_i\right] = \frac{1}{N}\sum_{i=1}^{N}\ell_i,$$

with model parameters held fixed. Actual training changes the parameters between batches, and batches from one permutation are dependent because samples are not repeated. The equation therefore describes a sampling property, not a promise of independent updates or balanced classes in every batch. The worked trace includes an all-zero-label batch even after shuffling.

Invariant 1: Exact coverage: Under `shuffle=True`, the loader constructs a permutation $\pi \in S_N$. Across all $\lceil N/B \rceil$ batches, every sample $i \in \{0, \dots, N-1\}$ is visited exactly once per epoch without replacement.

Invariant 2: Alignment: Collation must never detach inputs from targets. Row k of batch feature tensor **X** must correspond strictly to index k of batch label tensor **Y**.

Batching fixes the waste problem, and the operation that does it is **collation**: take B samples, each a tensor of shape $(D,)$, and stack them into one tensor of shape (B, D). The rows do not have to come from adjacent places in the dataset; the copy is what makes them adjacent in memory. In this implementation, sample retrieval and tensor construction also copy. Following those calls in the next section separates the necessary collation step from the wrapper's additional allocations.



Index permutation π is shuffled each epoch and sliced into non-overlapping mini-batches of size B.

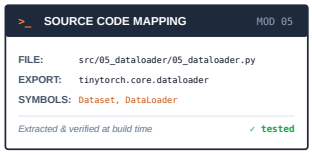
[19] In Python's standard library, `random.shuffle` implements the Durstenfeld-Knuth-Fisher-Yates in-place shuffle algorithm. It operates in $O(N)$ time complexity and guarantees that each of the $N!$ possible permutations has equal probability $(1/N!)$ under ideal pseudo-random generator entropy.

[20] Under sampling with replacement, the probability that a specific sample is not drawn in N independent uniform selections is $(1-1/N)^N \to 1/e \approx 0.3679$. Thus, $\approx$ 36.8% of the dataset remains completely unvisited in each epoch. By the coupon collector's problem, observing all N distinct samples with replacement requires on average $\approx N\ln N + \gamma N$ draws, making permutation shuffling without replacement vastly more sample-efficient.

5.3 The Code

The listings use the copying `Tensor` constructor from Chapter 1. Reading a row and constructing a batch both create fresh input tensors for the model. These copies do not connect the batch back to its source tensor for differentiation; Chapter 6 will distinguish data preparation from recorded model operations. Read the code in the order one request executes: `__iter__` chooses indices, `TensorDataset.__getitem__` returns samples, and `_collate_batch` groups and stacks their fields.

5.3.1 The dataset contract



Source Code Mapping: Module 05 extracts Dataset sample indexing, DataLoader shuffling, and batch collation directly from the working repository.

In production frameworks like PyTorch, the map-style `Dataset` abstraction cleanly decouples data storage and sample retrieval from downstream batching and model execution. By exposing a uniform key-value retrieval interface, it allows the surrounding runtime to interact identically with in-memory arrays, compressed disk archives, or remote object stores. Start with what the rest of the framework needs from a dataset. The training loop we write in Chapter 8 says `for x, y in loader`, and it should say exactly that whether the samples are ten rows in a Python list, sixty thousand handwritten digits in one array in RAM, or ten million JPEG files on disk that each have to be decoded. The model, the loss, and the loop must not change when the data does.

The naive design bakes the data source into the loader, so a loader for in-memory arrays and a loader for image folders become two classes with two copies of the batching logic. So the contract is two methods, `__len__` and `__getitem__`, and nothing else, and Module 05 writes it down as an abstract base class so that forgetting one of them fails at construction rather than in the middle of an epoch.

```
class Dataset(ABC):
    """
    Abstract base class for all datasets.
    """

    @abstractmethod
    def __len__(self) -> int:
        """
        Return the total number of samples in the dataset.
        """
        pass

    @abstractmethod
    def __getitem__(self, idx: int):
        """
        Return the sample at the given index.
        """
        pass
```

`TensorDataset` is the dataset for data that is already in memory. It holds any number of tensors that agree on their first dimension, and sample i is row i of each of them.

```python
class TensorDataset(Dataset):
    """
    Dataset wrapping tensors for supervised learning.
    """

    def __init__(self, *tensors):
        """
        Create dataset from multiple tensors.
        """
        if len(tensors) == 0:
            raise ValueError("TensorDataset needs at least one tensor")

        # Store all tensors
        self.tensors = tensors

        # Validate all tensors have same first dimension
        first_size = len(tensors[0].data)  # Size of first dimension
        for i, tensor in enumerate(tensors):
            if len(tensor.data) != first_size:
                raise ValueError(
                    f"Tensor size mismatch in TensorDataset\n"
                    f"  ❌ Tensor 0 has {first_size} samples, but Tensor {i} has {len(tensor.data)} samples\n"
                    f"  🔔 All tensors must have the same size in their first dimension (the sample dimension)\n"
                    f"  🔧 Check your data: features.shape[0] should equal labels.shape[0]\n"
                    f"     Example fix: labels = labels[:{first_size}] or features = features[:{len(tensor.data)}]"
                )

    def __len__(self) -> int:
        """
        Return number of samples (size of first dimension).
        """
        return len(self.tensors[0].data)

    def __getitem__(self, idx: int) -> Tuple[Tensor, ...]:
        """
        Return tuple of tensor slices at given index.
        """
        original_idx = idx
        if idx < 0:
            idx += len(self)

        if idx >= len(self) or idx < 0:
            raise IndexError(f"Index {original_idx} out of range for dataset of size {len(self)}")

        # Return tuple of slices from all tensors
```

```
46          return tuple(Tensor(tensor.data[idx]) for tensor in
            ↪   self.tensors)
```

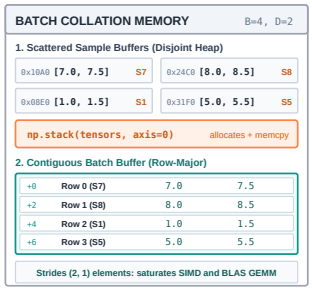
Look at what `__getitem__` costs. `tensor.data[idx]` is NumPy row indexing, which is a view, the zero-copy trick from Chapter 1; then `Tensor(...)` wraps it, and Chapter 1's constructor copies. So fetching a sample copies its D floats into a fresh tensor, 784 floats for a digit, about 3 KB. Module 05 accepts that copy on purpose. The copied sample can be modified without changing the stored dataset. That ownership rule is simple to inspect, but the copy can matter when lookup itself is cheap, as it is for this in-memory dataset. Notice the negative-index handling, which mirrors Python lists so that `dataset[-1]` is the last sample, and the tuple return, which is why the loop can unpack `x, y` without the loader knowing there are two of them. The loader only needs indexed access, even though `TensorDataset` also holds references to the source tensors in its `tensors` attribute. The order of visits is someone else's decision.

5.3.2 Collation

Modern accelerators and BLAS libraries achieve peak throughput only when operating on contiguous memory buffers, yet individual dataset samples reside in scattered, non-adjacent heap allocations. PyTorch delegates this packing step to `collate_fn`, which aggregates individual sample tuples into dense, row-major batch tensors ready for vectorized GEMM execution. Now the other side of the contract. `Linear.forward` from Chapter 3 computes `x.matmul(W)`, and its input needs a batch dimension. Our loader produces a contiguous array, with row i at positions iD through $iD + D - 1$ and element strides $(D, 1)$. NumPy can multiply strided arrays too; the need here is to assemble separate sample buffers into one array. What the dataset hands us is B separate small tensors, each in its own buffer. Four samples drawn as 7, 8, 1, 5 live in four unrelated allocations. No stride trick makes four buffers look like one $(4, 2)$ tensor.

The naive way is to skip the batch and call `forward` four times, once per sample. That pays the per-call fixed cost from The Problem four times over and leaves us averaging four separate losses by hand, which is the $B = 1$ loop we set out to escape.

Collation allocates a buffer and places each sample's values in its row. `np.stack` performs that step. The next line, `Tensor(batched_data)`, copies the stacked array again (applying the defensive copy semantics of the `Tensor` constructor from Chapter 1). Including `TensorDataset.__getitem__`, each feature value is copied three times on this path: into a sample tensor, into the stacked array, and into the returned batch tensor. For `B` samples of `D` features, that is `3 * B * D` copied values, excluding labels and other fields. This counts work; it does not compare elapsed copy time with matrix arithmetic. The extra copies are the cost of applying the same ownership rule at each boundary.



Batch Collation: Shuffled sample allocations live at scattered heap addresses; `_collate_batch` allocates a single contiguous buffer with strides $(D, 1)$ to saturate SIMD vectorization and BLAS GEMM kernels.

```
1  def _collate_batch(self, batch: List[Tuple[Tensor, ...]]) ->
   ↪   Tuple[Tensor, ...]:
2      """
3      Collate individual samples into batch tensors.
4      """
5      if len(batch) == 0:
6          return ()
7
```

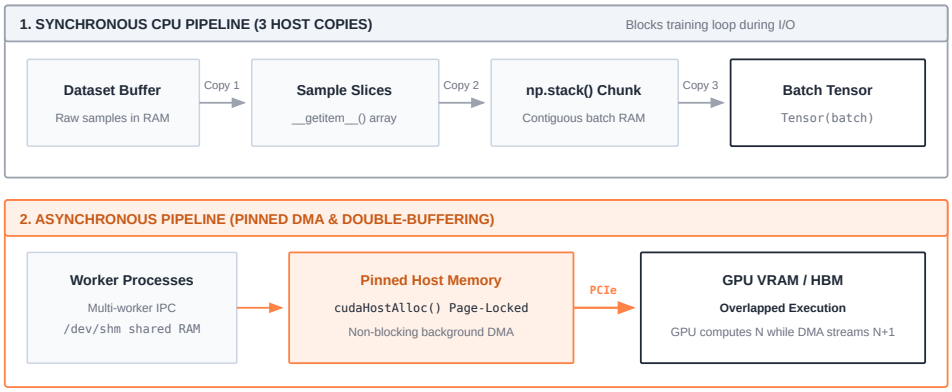
```
    # Determine number of tensors per sample
    num_tensors = len(batch[0])

    # Group tensors by position
    batched_tensors = []
    for tensor_idx in range(num_tensors):
        # Extract all tensors at this position
        tensor_list = [sample[tensor_idx].data for sample in batch]

        # Stack into batch tensor
        batched_data = np.stack(tensor_list, axis=0)
        batched_tensors.append(Tensor(batched_data))

    return tuple(batched_tensors)
```

`np.stack(..., axis=0)` allocates a `(B, D)` array and writes each sample's row at offset iD, which is the stride equation from Chapter 1 with strides $(D, 1)$ carried out in C. The method groups by position first, so a dataset that returns `(x, y)` yields one stacked `X` and one stacked `Y`, and a dataset that returns three tensors per sample yields three batches, without the loader having to know. Every sample must have the same number of tensor fields, and the shapes at each field position must agree for `np.stack` to succeed. Variable-length sequences require a dataset that pads them to a common length, or a different collation policy.



Double-buffering and page-locked host memory allow I/O data transfers to overlap perfectly with GPU kernel execution.

Figure 5.2: **Memory lifecycle and copy pipeline in DataLoader**: From in-memory `TensorDataset` storage through index shuffling, each sample retrieval copies values into an individual sample `Tensor`. Collation via `np.stack` allocates a fresh contiguous C-level array, and the final `Tensor` constructor copies again into the batch buffer delivered to the training loop.

5.3.3 The loader

In production deep learning pipelines, `DataLoader` serves as the runtime orchestrator that bridges static datasets to execution hardware, managing index permutations and batch dispatch. By yielding batches lazily via an iterator instead of materializing all batches in memory upfront, it bounds memory consumption while providing fresh pseudo-random shuffles across epochs. The last piece owns the order. Chapter 8's loop is

going to read `for epoch in range(E): for x, y in loader:` and expect two things from that inner line, a fresh shuffle when enabled and one batch at a time.

The naive way to get batches is to build them all up front, shuffle the dataset once, cut it into $\lceil N/B \rceil$ tensors, and hand back the list. That list is a second copy of the whole dataset sitting next to the first, another 47 million floats in the 60,000 by 784 case, and it has to be rebuilt every epoch. The naive way to get a random order is worse. Draw N indices at random with replacement and roughly a third of the samples never come up in that epoch, because the chance that a particular index is missed in N draws tends to $1/e$ as N grows.

The generator instead holds one list of N indices, along with the samples for the batch it is preparing. With shuffling enabled, it rearranges that list in place at the start of each pass, walks it in strides of B, and asks the dataset for the samples of one stride only when the loop reaches it. The index list costs memory proportional to the dataset length, even with `shuffle=False`. A Python list stores references to integer objects, not packed eight-byte integers. On typical 64-bit CPython, approximately 8 bytes per reference plus 28 bytes per integer gives about 3.6 MB for 100,000 indices, excluding small list overhead; runtime object sizes can differ. The sampled tensors and stacked arrays add temporary storage proportional to a batch, while the dataset itself remains wherever its implementation stores it. No list of all collated batches is built.

One detail remains. Unless B divides N, the last stride is short. With $N = 10$ and $B = 4$ the batches have 4, 4, and 2 samples, so an epoch has $\lceil N/B \rceil = 3$ batches. In integer arithmetic that ceiling is `(N + B - 1) // B`. The short batch is real data, and dropping it would mean those samples are never seen in that epoch; the loader keeps it, and the layers handle the smaller B because nothing in `Linear.forward` assumed a particular batch size.

```
class DataLoader:
    """
    Data loader with batching and shuffling support.
    """

    def __init__(self, dataset: Dataset, batch_size: int, shuffle: bool
    ↪   = False):
        """
        Create DataLoader for batched iteration.
        """
        if isinstance(batch_size, bool) or not isinstance(batch_size,
        ↪   (int, np.integer)) or batch_size < 1:
            raise ValueError("batch_size must be a positive integer")
        self.dataset = dataset
        self.batch_size = batch_size
        self.shuffle = shuffle

    def __len__(self) -> int:
        """
        Return number of batches per epoch.
        """
        # Calculate number of complete batches
        return (len(self.dataset) + self.batch_size - 1) //
        ↪   self.batch_size
```

```
    def __iter__(self) -> Iterator[Tuple[Tensor, ...]]:
        """
        Return iterator over batches.
        """
        # Create list of indices
        indices = list(range(len(self.dataset)))

        # Shuffle if requested
        if self.shuffle:
            random.shuffle(indices)

        # Yield batches
        for i in range(0, len(indices), self.batch_size):
            batch_indices = indices[i:i + self.batch_size]
            batch = [self.dataset[idx] for idx in batch_indices]

            # Collate batch - convert list of tuples to tuple of
            ↪ tensors
            yield self._collate_batch(batch)

    def _collate_batch(self, batch: List[Tuple[Tensor, ...]]) ->
    ↪ Tuple[Tensor, ...]:
        """
        Collate individual samples into batch tensors.
        """
        ...
```

Two things to notice. First, `__iter__` is a generator, so the index list is rebuilt and the permutation drawn fresh every time a `for` loop starts over the loader; that is what "shuffle once per epoch" means in code, and it is how Chapter 8 can request a fresh shuffle on each pass through its outer loop. Creating the generator does not execute its body; the first `next` request creates and optionally shuffles the indices. Second, a batch is never materialized before it is needed. The generator retains its index list and current sample list while suspended at `yield`; the caller holds the returned batch. The next request resumes the loop and prepares the next batch synchronously. There is no background work between yields, and the caller can increase memory use by retaining old batches. `random.shuffle` mutates the index list, leaving the dataset's stored tensors in place. It consumes Python's random-generator state, so reusing a seed requires resetting that state before iteration. A completed pass visits every index once; stopping early consumes only the requested batches. An empty dataset yields no batches, and construction rejects a nonpositive or noninteger batch size.

5.4 A Worked Trace

Ten samples with two features each, $\mathbf{x}_i = [i,\ i + 0.5]$, and labels stored sorted, five zeros followed by five ones. The features are chosen so that every row announces its own index, making the collated batch easy to inspect visually.

```
import random
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.dataloader import TensorDataset, DataLoader

N, D = 10, 2
inputs = Tensor(np.array([[i, i + 0.5] for i in range(N)],
    dtype=np.float32))
targets = Tensor(np.array([0 if i < 5 else 1 for i in range(N)],
    dtype=np.float32))

dataset = TensorDataset(inputs, targets)
x3, y3 = dataset[3]
print(x3.shape, x3.data.tolist(), y3.data.tolist())          # (2,)
    [3.0, 3.5] 0.0
print(np.shares_memory(x3.data, inputs.data))                # False:
    the fetch copied
```

In line 10, `dataset = TensorDataset(inputs, targets)` wraps the ten generated samples. In line 11, `x3, y3 = dataset[3]` indexes the fourth row. Sample 3 is a fresh tensor of shape $(2,)$ holding row index 3 (`[3.0, 3.5]`), and its label is a scalar tensor `0.0` (printed in line 12). Crucially, line 13 shows `shares_memory` is `False`: each indexed lookup copies data out of the underlying storage to ensure memory isolation. Without shuffling, the loader reproduces the sorted-batch problem from the opening; the label rows of the three batches are `[0, 0, 0, 0]`, `[0, 1, 1, 1]`, and `[1, 1]`, and `len(loader)` is $\lceil 10/4 \rceil = 3$.

With `shuffle=True` the order changes. In lines 1 to 2 of the second snippet, `random.seed(0)` sets the generator seed and `loader = DataLoader(dataset, batch_size=4, shuffle=True)` initializes the shuffled iterator. The loop in lines 3 to 5 traces two consecutive epochs:

```
random.seed(0)
loader = DataLoader(dataset, batch_size=4, shuffle=True)
for epoch in range(2):
    for X, Y in loader:
        print(epoch, X.shape, X.data.strides, X.data.ravel().tolist(),
            Y.data.tolist())
```

With seed 0, execution produces the permutation $\pi_1 = [7, 8, 1, 5, 3, 4, 2, 0, 9, 6]$ in this runtime. Seeding reproduces the sequence within the same runtime; the exact shuffle sequence need not be stable across Python versions. The loader cuts it into groups of four, as table 5.2 records.

Table 5.2: Epoch 1 with seed 0. Every index appears exactly once across the three batches, and the last batch is short.

Batch	Slice of π_1	Sample indices	Labels	Shape
0	`[0:4]`	$[7, 8, 1, 5]$	$[1, 1, 0, 1]$	$(4, 2)$
1	`[4:8]`	$[3, 4, 2, 0]$	$[0, 0, 0, 0]$	$(4, 2)$

Batch	Slice of π_1	Sample indices	Labels	Shape
2	`[8:12]`	$[9, 6]$	$[1, 1]$	$(2, 2)$

Batch 0's buffer, read flat, is `[7.0, 7.5, 8.0, 8.5, 1.0, 1.5, 5.0, 5.5]` with shape $(4, 2)$ and strides `(8, 4)` in bytes, which is $(2, 1)$ in elements. These are the four scattered samples from the collation subsection. Sample 7 began at element offset 14 of the dataset's buffer and sample 1 at offset 2; each was copied by the fetch, by `np.stack`, and by the final `Tensor` construction, and now they sit at offsets 0 and 4 of one new buffer, adjacent, in the order the permutation drew them, eight floats in all. The label tensor is `[1.0, 1.0, 0.0, 1.0]` with shape $(4,)$. Batch 1 happens to be all class 0, which is the kind of thing a random draw of four from ten will do now and then; the difference from the sorted loop is that the next epoch draws a fresh permutation, which may change the grouping.

The next pass gives $\pi_2 = [9, 4, 8, 6, 0, 1, 7, 2, 3, 5]$. In table 5.3, the same ten rows now share batches in different groups.

Table 5.3: Epoch 2 with the same loader. A fresh permutation changes both the order and which samples share a batch.

Batch	Slice of π_2	Sample indices	Labels	Shape
0	`[0:4]`	$[9, 4, 8, 6]$	$[1, 0, 1, 1]$	$(4, 2)$
1	`[4:8]`	$[0, 1, 7, 2]$	$[0, 0, 1, 0]$	$(4, 2)$
2	`[8:12]`	$[3, 5]$	$[0, 1]$	$(2, 2)$

Each index appears exactly once in a completed epoch, and features and labels keep the same index throughout retrieval and collation. Those are invariants. A fresh shuffle can change pairings, as it did in the two tables, but it can also repeat a previous pairing or even a whole permutation. Randomness gives a distribution over orders, not a guarantee that consecutive orders differ.

The short batch exposes a contract that the training loop must preserve. If the three batch-mean losses are `L0`, `L1`, and `L2`, the epoch mean is `(4*L0 + 4*L1 + 2*L2) / 10`, not `(L0 + L1 + L2) / 3`. The loader supplies all ten samples; the trainer in Chapter 8 must give each sample equal weight when reporting loss or accumulating an update. Keeping the final batch is only useful if the consumer handles its actual size.

5.4.1 A transform belongs at sample lookup

In vision and multimodal pipelines, data augmentation libraries such as `torchvision.transforms` apply stochastic alterations—like flips, crops, and color jitter—on the fly rather than expanding dataset size on disk. Executing these operations inside sample lookup ensures that the network encounters unique, randomly perturbed views of each sample across different training epochs without ballooning memory footprint. Up to this point, datasets have supplied flat feature vectors of shape $(D,)$ collated into (B, D) batches. Perception tasks, by contrast, operate on multi-dimensional tensors like images with spatial dimensions and color channels. An image adds axes to the same contract. In a channels-last image, shape `(2, 3, 2)` means two rows, three columns, and two values

per pixel. Flipping horizontally must reverse the three columns while keeping each pixel's two values together. Shape alone cannot identify the channel axis: the same tuple could describe two channels of height three and width two. The explicit `layout="HWC"` argument resolves that ambiguity.

```
from tinytorch.core.dataloader import RandomHorizontalFlip,
    RandomCrop, Compose

image = np.arange(12).reshape(2, 3, 2)
flip = RandomHorizontalFlip(p=1.0, layout="HWC")
flipped = flip(image)
print(image[0].tolist())    # [[0, 1], [2, 3], [4, 5]]
print(flipped[0].tolist())  # [[4, 5], [2, 3], [0, 1]]
assert flipped.shape == image.shape
```

The flip's entry point first unwraps a Tensor to its NumPy array so both input types follow one implementation. `is_tensor` remembers which wrapper to restore on return.

```
def __call__(self, x: Union[Tensor, np.ndarray]) -> Union[Tensor,
    np.ndarray]:
    """
    Apply random horizontal flip to input.
    """
    is_tensor = isinstance(x, Tensor)
    data = x.data if is_tensor else x
    layout = self.layout
    if layout is None:
        if data.ndim == 2:
            layout = "HW"
        elif data.ndim == 3:
            layout = "CHW" if data.shape[0] <= 4 else "HWC"
    if layout is None or data.ndim != (2 if layout == "HW" else 3):
        raise ValueError(
            f"RandomHorizontalFlip input dimensions do not match image
            ↪ layout\n"
            f"  ✗ Got {data.ndim}D input with shape {data.shape} and
            ↪ layout={self.layout!r}\n"
            f"  🔄 HW requires 2 dimensions; CHW and HWC require 3
            ↪ dimensions\n"
            f"  🔧 Pass a single image and set layout='HW', 'CHW', or
            ↪ 'HWC' to match its axes"
        )
    if rng.random() < self.p:
        axis = -2 if layout == "HWC" else -1

        flipped = np.flip(data, axis=axis).copy()
        return Tensor(flipped) if is_tensor else flipped
    return x
```

Validation happens before the random draw, so an invalid number of dimensions fails even when `p=0`. For `HWC`, `axis=-2` selects width; for `HW` and `CHW`, width is the final axis. `np.flip` reverses that axis, and `.copy()` gives the flipped result independent storage.

If the probability check does not select a flip, `return x` preserves the original object. A caller that always needs independent storage must therefore establish it before the transform, as `TensorDataset.__getitem__` does.

Cropping composes padding, random position selection, and extraction. With the image above and one pixel of padding, `_pad_image` creates shape `(4, 5, 2)`. A `(2, 3)` crop has top and left positions in `[0, 2]`, inclusive: the padded height minus the target height is two, and the padded width minus the target width is also two.

```
def __call__(self, x: Union[Tensor, np.ndarray]) -> Union[Tensor,
    np.ndarray]:
    """
    Apply random crop after padding.
    """
    is_tensor = isinstance(x, Tensor)
    data = x.data if is_tensor else x

    target_h, target_w = self.size

    # Step 1: Pad the image (handles format detection internally)
    padded = _pad_image(data, self.padding, self.layout)

    # Step 2: Determine padded spatial dims and sample crop position
    if data.ndim == 2:
        padded_h, padded_w = padded.shape
        top, left = _random_crop_region(padded_h, padded_w, target_h,
    target_w)
        cropped = padded[top:top + target_h, left:left + target_w]
    elif self.layout == "CHW" or (self.layout is None and
        data.shape[0] <= 4):
        # Channels-first: (C, H, W)
        padded_h, padded_w = padded.shape[1], padded.shape[2]
        top, left = _random_crop_region(padded_h, padded_w, target_h,
    target_w)
        cropped = padded[:, top:top + target_h, left:left + target_w]
    else:
        # Channels-last: (H, W, C)
        padded_h, padded_w = padded.shape[0], padded.shape[1]
        top, left = _random_crop_region(padded_h, padded_w, target_h,
    target_w)
        cropped = padded[top:top + target_h, left:left + target_w, :]

    return Tensor(cropped) if is_tensor else cropped
```

`target_h, target_w` unpacks the constructor's requested output size. The three branches then select the spatial axes for the layout. In the `HWC` branch, `padded[top:top + target_h, left:left + target_w, :]` leaves the channels untouched. `_random_crop_region` uses an exclusive upper bound of `padded_h - target_h + 1`, and likewise for width, so a crop can reach either padded edge. The slice is a view of the newly padded array; returning a Tensor copies that view, while a NumPy caller receives it directly. Neither path writes into the input image.

The accepted layouts are `HW`, `CHW`, and `HWC`. Omitting `layout` keeps the module's convenience heuristic: a three-dimensional image with first dimension at most four is treated

as `CHW`, otherwise as `HWC`. Our short image would be misinterpreted under that heuristic, so explicit layout is part of its input contract. These transforms process individual two- or three-dimensional images, not a batch with an additional leading axis.

The place to invoke the transform is the dataset's `__getitem__`. The loader then calls it once for each requested sample, including on later epochs:

```
class AugmentedDataset(TensorDataset):
    def __init__(self, images, labels, transform):
        super().__init__(images, labels)
        self.transform = transform

    def __getitem__(self, idx):
        image, label = super().__getitem__(idx)
        return self.transform(image), label

transform = Compose([
    RandomHorizontalFlip(p=0.5, layout="HWC"),
    RandomCrop((2, 3), padding=1, layout="HWC"),
])
images = Tensor(np.stack([image, image + 12]))
labels = Tensor([0, 1])
image_loader = DataLoader(
    AugmentedDataset(images, labels, transform), batch_size=2
)
X, Y = next(iter(image_loader))
assert X.shape == (2, 2, 3, 2)  # batch, height, width, channels
assert Y.data.tolist() == [0.0, 1.0]
```

The copied sample isolates the stored image from transformations. `Compose` passes the result of the flip to the crop; it does not know about epochs or labels. Applying the pipeline once before creating the dataset would instead reuse the same transformed images on every pass. Fresh random draws need not produce different images, but placing the call at lookup gives each visit that opportunity. These draws use the module's NumPy generator, whereas shuffling uses Python's `random`; seeding one does not reset the other.

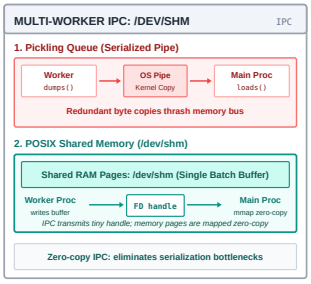
5.5 From TinyTorch to Production

For datasets accessed by index, called map-style datasets in its API, PyTorch keeps indexed lookup separate from sampling and collation. Its tensor dataset indexes stored tensors directly, allowing a row view rather than TinyTorch's copied sample. Tensor fields are then stacked into batches. A configurable sampler and collation function support policies beyond the tuple-of-tensors contract used here. PyTorch data loading documentation.

When `__getitem__` reads and decodes disk-backed files, synchronous execution forces the model to wait during I/O. Background worker processes decouple ingestion from compute: a background producer prepares batch $N + 1$ while the accelerator executes batch N. PyTorch exposes this through `num_workers` and `prefetch_factor`. TinyTorch leaves execution synchronous so every `next` call traces deterministically from index selection to memory layout. PyTorch data loading documentation.

Dataset scale introduces an equally fundamental systems boundary. TinyTorch assumes the training dataset fits comfortably inside host RAM, allowing instant, random-access integer indexing (`dataset[i]`). In production, datasets scaling to terabytes or petabytes (such as Common Crawl text or ImageNet-scale image corpora) cannot be materialized in memory or indexed by random integers without thrashing the storage subsystem. Industrial runtimes transition from map-style datasets to streaming iterable datasets (such as WebDataset or Apache Arrow shards), reading sequential compressed tar or parquet archives directly from remote object storage over the network.

Worker processes also need to hand completed tensors back to the caller. **Shared memory** lets two processes access the same region of RAM; PyTorch's multiprocessing support uses this for CPU tensor storage passed through its queues. Moving storage into shared memory can itself require a copy, so sharing does not mean every part of the handoff is free. PyTorch multiprocessing notes.



Multi-worker IPC: Worker processes bypass pipe serialization by writing batches directly to `/dev/shm`, sending a small handle (a file descriptor passed over a Unix socket) so the main process maps the same pages.

Inter-process communication (IPC): When `num_workers > 0`, Python's `multiprocessing` spawns worker processes. `multiprocessing.Queue` pickles each object and sends the bytes through a pipe, so an unshared tensor is copied at least twice on its way to the main process.

POSIX shared memory (`/dev/shm`): PyTorch bypasses pickle serialization for tensors by writing tensor memory into POSIX shared memory (`shm_open`) or memory-mapped files. The worker sends only a small handle (a file descriptor plus storage metadata) over the IPC channel; the main process reconstructs the tensor zero-copy by mapping the identical physical memory pages.

A later transfer to an accelerator introduces a critical hardware boundary. Standard host memory allocated via `malloc` or NumPy is pageable: the operating system can swap pages out to disk or migrate physical pages across virtual addresses at any time. When an accelerator (such as a GPU) copies data over PCIe, it uses Direct Memory Access (DMA) engines that require fixed physical page addresses. If host memory is pageable, the driver must first copy the batch into an internal, page-locked staging buffer before initiating the DMA transfer over PCIe, paying a double-copy penalty. **Pinned memory** (page-locked memory) avoids this by locking host pages to fixed physical DRAM addresses; the CUDA driver allocates or registers such buffers itself (`cudaHostAlloc`, `cudaHostRegister`), much as the POSIX `mlock` call locks pages.[21] With `pin_memory=True`, a PyTorch loader thread copies each collated batch into page-locked host memory, so a later `.to(device, non_blocking=True)` can be served by the GPU's DMA engine without an extra staging copy. Neither option exists in TinyTorch's synchronous NumPy loader, which isolates memory mechanics before physical hardware transfer boundaries. PyTorch pinned-memory and nonblocking-transfer guide.

The Linux fork hazard: On Linux/POSIX systems, starting workers via `fork` causes child processes to inherit parent address space marked Copy-on-Write (CoW).

CPython refcount thrashing: Every time a worker process touches a Python object (e.g., iterating a large in-memory dataset list), CPython increments its internal reference count. This in-place integer mutation triggers Linux page-fault handlers, copying entire 4 KB memory pages from parent to child.[22] A parent process with 30 GB of RAM can quickly trigger Out-Of-Memory (OOM) fatal kills across 8 workers despite allocating no new data structures. Using `spawn` or flat byte buffers (like `numpy.memmap`) prevents refcount-induced memory duplication.

The comparison in table 5.4 locates each extension at the boundary it changes. Indexed access remains usable whether retrieval runs in the caller or in a worker.

[21] The POSIX `mlock()` system call instructs the operating system kernel to disable paging for a specified virtual address range, locking pages into physical RAM. GPU DMA engines require physical address stability because PCIe bus transfers bypass the CPU memory management unit (MMU).

[22] Because Python's reference counts are stored inline in object headers, even read-only iteration mutates refcounts, triggering Linux kernel copy-on-write page faults that duplicate memory pages across every forked worker.

Table 5.4: What the two loaders share and where they differ. The same indexed-dataset pattern supports configurable sampling and scheduling; TinyTorch keeps the request synchronous.

Concern	TinyTorch (this chapter)	PyTorch `torch.utils.data`
Storage scale	In-memory arrays in host RAM	In-memory or streaming shards (`IterableDataset`, WebDataset)
Order	`random.shuffle(indices)` inside `__iter__`	`Sampler` classes; `generator=` for a seeded, reproducible order
Lookup	`TensorDataset` copies one row per fetch, in one process	Main-process or worker lookup; tensor rows can be views
Collation	`np.stack` inside `_collate_batch`	Configurable collation; tensor fields are stacked
Handoff	None (same process)	Shared-memory tensors between worker and main process
Host to device	None	Pinned batches can support asynchronous device transfers
Short last batch	Always kept	Kept by default; `drop_last=True` discards it

This architectural comparison provides a framework to diagnose data bottlenecks. First measure whether time is spent reading samples, collating them, or copying tensors. Increasing workers resolves I/O bottlenecks but cannot fix excessive buffer copies. Discarding a short batch guarantees fixed shapes, but permanently drops valid training data. Reproducibility requires controlling every random seed across indexing and transformations.

5.6 Summary

You built the three foundational components of a data pipeline: a `Dataset` that exposes `__len__` and indexed sample lookups, a per-epoch permutation that randomizes visiting order without replacement, and a `DataLoader` that walks indices in strides of B and collates samples into contiguous row-major batch tensors. A feature vector becomes a row of a (B, D) tensor; an image preserves its spatial and channel dimensions after the leading batch axis. Shuffling guarantees uniform subset sampling across epochs while collation enforces strict feature-label alignment. When dataset length is not divisible by batch size, the final batch is shorter, requiring the training loop to scale loss contributions by actual sample count. Transforms placed at sample lookup allow dynamic data augmentation without mutating underlying storage. Production loaders preserve this modular separation while adding multi-process workers, shared memory, and asynchronous device transfers. Chapter 6 assumes this batch stream and derives parameter gradients.

5.7 Further Reading

- **Goyal et al.** (2017). "Accurate, large minibatch SGD: training ImageNet in 1 hour." *arXiv*. How batch size and learning rate move together, which Chapter 7 returns to.
- **Krizhevsky, Sutskever, and Hinton** (2012). "ImageNet classification with deep convolutional neural networks." *NeurIPS*. A training pipeline where data loading and augmentation ran concurrently with GPU compute.
- **Murray et al.** (2021). "tf.data: A Machine Learning Data Processing Framework." *VLDB*. A systems analysis of asynchronous, pipelined data loaders that prevent GPU starvation.

🔥 Chapter 6

Automatic Differentiation: Recording the Graph and Walking It Backward

The foundational algorithm of modern deep learning is backpropagation: computing the exact partial derivative of a scalar loss with respect to every trainable weight in a network via the multivariable chain rule. These gradients tell the optimizer precisely how to adjust millions of parameters to decrease error.

In PyTorch, automatic differentiation is an effortless four-line ritual:

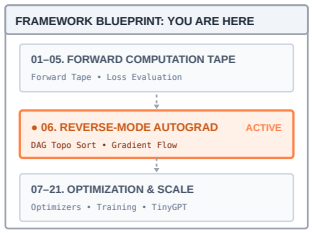
```
import torch

# Leaf parameters requiring gradients
x = torch.tensor([[1.0, 2.0]], requires_grad=True)
W = torch.tensor([[0.5, -0.5], [1.0, 0.5]], requires_grad=True)

# Forward pass and scalar loss
loss = torch.relu(x @ W).sum()

# Backward automatic differentiation
loss.backward()

print("Gradient dL/dW:\n", W.grad)
```



Framework Blueprint: Module 06 builds the computation DAG and executes reverse-mode automatic differentiation via topological sort.

Underneath that clean surface, the framework executes a sophisticated runtime engine:

1. **Dynamic tape recording:** As `x @ W`, `relu`, and `sum` execute during the forward pass, the runtime dynamically allocates operational graph nodes, constructing a Directed Acyclic Graph (DAG) that records parent-child dependencies and saves intermediate activations needed for local derivatives.
2. **Reverse topological traversal:** Calling `loss.backward()` triggers a topological sort of the graph from the scalar loss node back to the leaf inputs, guaranteeing that every downstream consumer contributes its partial derivative before an upstream tensor propagates its accumulated gradient.
3. **In-place accumulation and memory cleanup:** Gradients accumulate in-place directly into leaf `.grad` buffers (`param.grad += ...`), while non-leaf intermediate graph edges are unhooked so the runtime can immediately reclaim activation memory.

This chapter constructs TinyTorch's reverse-mode automatic differentiation engine, connecting the forward computations of Chapter 1 through Chapter 5 with the backward gradient updates required by Chapter 7. Before building this engine, we must inspect why naive numerical or manual alternatives fail.

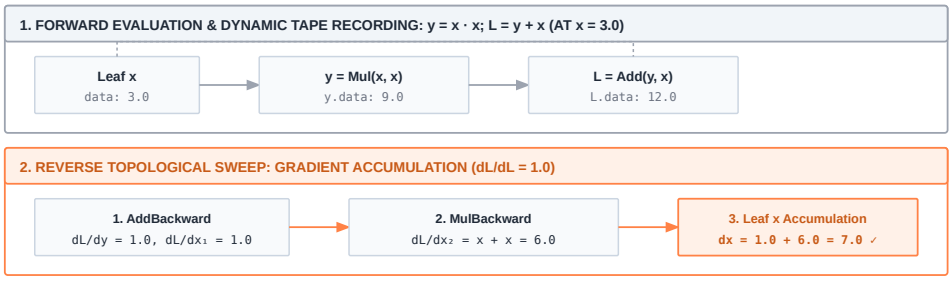


Figure 6.1: **Forward records and backward contributions**: The upper row computes y = x × x and L = y + x. In the lower row, addition contributes 1 directly to x; multiplication contributes 3 through each input position. All three contributions are needed for the gradient of 7.

6.1 The Problem

↩ **Jump back:** Chapter 1 *Reverse topological traversal writes gradient updates directly into leaf tensor* `.grad` *buffers.*

Gradient descent, which Chapter 7 turns into a real optimizer, needs one number per weight: the partial derivative of the loss with respect to that weight. For N weights, the gradient contains N values. Each parameter tensor stores its part in an array of the same shape, so a 2×2 weight matrix needs a 2×2 gradient.

The obvious way to get a derivative is to nudge a weight and see what happens to the loss. For the small network the rest of the chapter uses (a 1×2 input, a 2×2 weight matrix, a ReLU, and a sum), listing 6.1 computes one estimate per weight.

Two things are wrong with this, and both get worse as the model grows. The first is cost. Estimating four derivatives took five forward passes, one per weight plus a baseline. A model with N weights needs $N + 1$ forward passes for a single gradient, and training needs a fresh gradient at every step. The baseline can be shared, but each perturbed weight still needs its own model evaluation. That cost grows with the number of weights even though the desired output remains one scalar loss.

The second is precision. The printed answer is already off in the fourth decimal place, and shrinking `eps` does not fix it. The numerator subtracts two losses that differ only slightly, and float32 keeps about seven significant digits, so the subtraction throws most of them away.[23] Choose `eps` too large and the estimate is biased; choose it too small and it is noise. The useful range depends on the scale of the inputs and the precision. Finite differences remain valuable as an independent check on a small derivative rule, but this sensitivity and the repeated forward passes make them unsuitable as the training engine.

Handwritten derivative formulas work for the four-weight example, but every change to the model then requires a corresponding change to those formulas. A reusable engine must instead compose the local rules for whichever operations executed. Shared subcomputations should contribute to the derivative without forcing the engine to reevaluate the whole model for each weight.

[23] **Catastrophic cancellation**: subtracting two nearly equal floating-point numbers discards the leading digits they share, leaving only the few that differ. Dividing that small difference by `eps` magnifies the effect of rounding in the two loss evaluations.

Listing 6.1 A finite-difference baseline: Caching the unperturbed loss gives five evaluations for four weights.

```python
import numpy as np

x = np.array([[1.0, 2.0]], dtype=np.float32)
W = np.array([[0.5, -0.5], [1.0, 0.5]], dtype=np.float32)

def loss_fn(W):
    return np.float32(np.sum(np.maximum(0.0, x @ W)))

eps = np.float32(1e-3)
numeric = np.zeros_like(W)
baseline = loss_fn(W)
for i in range(W.shape[0]):
    for j in range(W.shape[1]):
        W_plus = W.copy()
        W_plus[i, j] += eps
        numeric[i, j] = (loss_fn(W_plus) - baseline) / eps    # one extra forward pass per weight

print(np.round(numeric, 4))
# [[0.9999 0.9999]
#  [2.0001 1.9999]]
```

6.2 The Idea

6.2.1 One rule per operation

↪ **Jump ahead:** Chapter 7 *How the optimizer transforms computed gradient vectors into stable parameter update trajectories.*

In production frameworks like PyTorch and JAX, automatic differentiation decomposes complex neural network graphs into modular, isolated operations registered with their own vector-Jacobian products (VJPs). This modular decoupling ensures the runtime engine never needs global knowledge of the network architecture to propagate exact gradients backward. In the opening example, increasing y by a small amount increases $L = y + x$ by the same amount. The derivative of L with respect to y is therefore 1. Through the path $y = x^2$, the derivative of y at $x = 3$ is 6, giving a contribution 1×6 to the derivative of L. This multiplication is the **chain rule**. When L depends on x only through y,

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}.$$

In the opening example the direct $+x$ supplies a separate contribution, which is added to the contribution through y. Each factor in the formula describes one part of that path. The first factor, $\partial L/\partial y$, is about everything downstream of y and nothing upstream. The second, $\partial y/\partial x$, is about one operation and nothing else. If every operation can answer the question "given the gradient of the loss with respect to my output, what is the gradient with respect to my inputs," then no operation ever needs to know what the rest of the network looks like.

That question has a short answer for every operation we use. Addition passes the gradient through unchanged to both inputs, because $\partial(a + b)/\partial a = 1$. ReLU passes it through where the input was positive and blocks it where the input was not, because

its derivative is 1 for positive z and 0 for negative z. At zero the ordinary derivative is undefined; TinyTorch chooses zero. Summing a tensor to a scalar sends the same gradient to every element. Each rule needs only what the operation saw during the forward pass: ReLU needs the sign of its input, and matrix multiplication needs both of its inputs.

Pause & Predict: Vector-Jacobian Products vs. Full Jacobians

For a hidden linear layer with batch size $B = 128$, input dimension $D_{\text{in}} = 1024$, and output dimension $D_{\text{out}} = 4096$, how many elements would the full Jacobian tensor $\partial \mathbf{Y}/\partial \mathbf{W}$ contain? What is its memory footprint in float32? Why does reverse-mode autodiff compute the vector-Jacobian product directly without ever constructing this tensor?

Matrix multiplication is the one rule worth writing down. For $Y = AB$ with A of shape $M \times K$ and B of shape $K \times N$, let $G = \partial L/\partial Y$ be the incoming gradient, which has the same shape as Y. Then

$$\frac{\partial L}{\partial A} = GB^T, \qquad \frac{\partial L}{\partial B} = A^T G.$$

[24]

For the chapter's concrete input $A = [1, 2]$ and incoming gradient $G = [1, 1]$, $A^T G$ is $[[1, 1], [2, 2]]$. Each weight in the first row receives the input value 1, and each weight in the second row receives 2, because those values multiply the weights during forward execution. A shape check confirms that the general rule returns one derivative per input element. G is $M \times N$ and B^T is $N \times K$, so GB^T is $M \times K$, the shape of A. Likewise $A^T G$ is $K \times N$, the shape of B. When both inputs require gradients, backward performs two matrix multiplications, each with the same leading arithmetic count as the forward product. If only the weights require gradients, the rule skips the input-gradient product. This explains why backward work can be comparable to forward work without implying a fixed elapsed-time ratio for every model.

6.2.2 Reverse order

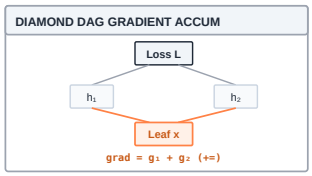
Modern deep learning frameworks universally rely on reverse-mode automatic differentiation because neural networks optimize a single scalar objective over millions of trainable parameters. Propagating cotangents backward from the scalar loss to leaf inputs computes every parameter gradient in a single reverse sweep, avoiding the prohibitive $O(N)$ passes required by forward-mode differentiation. During the forward pass each result is computed from earlier results, so the tensors and operations form a graph with a direction (from inputs toward the loss) and no cycles (a tensor cannot depend on something computed after it). Reverse mode starts at the loss, where the gradient is trivially $\partial L/\partial L = 1$, and moves toward the inputs.[25]

The order matters. A tensor's backward rule turns the gradient at that tensor into gradients at its parents, so the rule must not run until the gradient at that tensor is complete. If the tensor feeds two later operations, both of them must have reported back first. The backward traversal therefore needs the reverse of a **topological order**: consumers precede the tensors they read. A depth-first walk from the loss that lists each tensor only after all of its parents produces the opposite order (parents first), and reversing that list gives us the one we want.

[24] When evaluating $\mathbf{Y} = \mathbf{AB}$, the full Jacobian $\partial \mathbf{Y}/\partial \mathbf{B}$ is a 4D tensor of shape (M, N, K, N). In industrial networks, materializing this full Jacobian would consume gigabytes of DRAM. Reverse-mode automatic differentiation never constructs the Jacobian tensor; it evaluates the vector-Jacobian product (VJP) $\mathbf{A}^T\mathbf{G}$ directly, computing the exact parameter gradient in $O(MKN)$ operations while maintaining the original matrix shape.

[25] **Forward-mode vs. reverse-mode AD**: For a function $f : \mathbb{R}^N \to \mathbb{R}^M$, forward-mode automatic differentiation propagates tangent vectors with cost proportional to the number of inputs N. Reverse-mode automatic differentiation propagates cotangent vectors with cost proportional to the number of outputs M. Because deep neural networks optimize a single scalar loss ($M = 1$) over millions of parameters ($N \gg 1$), reverse-mode AD computes all N parameter gradients in a single backward pass ($O(1)$ sweeps), whereas forward-mode would require N separate forward passes.

6.2.3 Used twice means add



Diamond DAGs require gradient accumulation: $\nabla_x = \nabla_{y_1} \frac{\partial y_1}{\partial x} + \nabla_{y_2} \frac{\partial y_2}{\partial x}$.

Architectural motifs like residual skip connections in ResNets and shared projection weights in Transformers routinely fan out a single tensor into multiple downstream consumers. To correctly evaluate the multivariable chain rule across such diamond dependencies, the autograd engine must accumulate incoming cotangent vectors into a shared buffer rather than overwriting them. When a tensor feeds two operations, the loss depends on it along two paths, and the multivariable chain rule says the total derivative is the sum over paths:

$$\frac{\partial L}{\partial x} = \sum_k \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial x},$$

where the y_k are the tensors computed directly from x. In code the incoming contributions must be added. The implementation accumulates them in `pending` before adding the complete result into `x.grad`. The smallest example is `x + x`. The same tensor is both inputs of one addition, so its gradient is the incoming gradient twice. An engine that overwrote each pending contribution would silently report half the true value, and the mistake would not raise an error. It would just train worse. The same accumulation is what makes a layer whose input is added back to its output work, and it introduces one kind of accumulation: combining paths within a backward call. A separate choice keeps `.grad` between calls, so the training loop must clear it when starting a new gradient estimate.

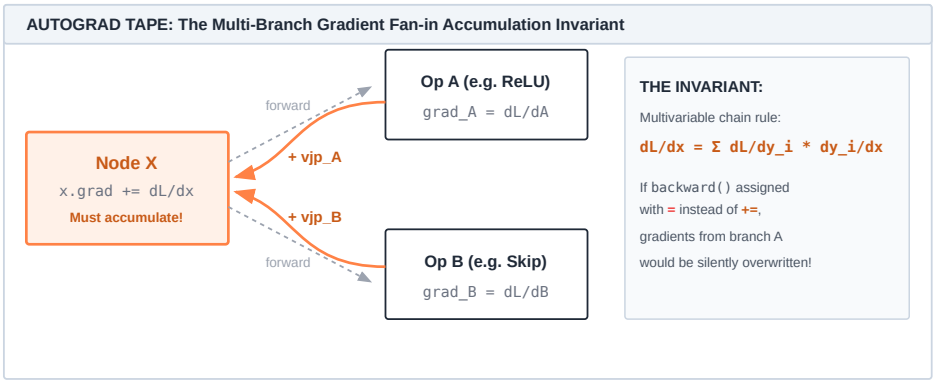


Figure 6.2: **Gradient fan-in accumulation across branches**: When a tensor fans out during forward execution (e.g. into parallel residual branches or multi-head attention), reverse-mode autograd must accumulate incoming vector-Jacobian products (`x.grad += dL/dx`) rather than assigning them, honoring the multivariable chain rule.

6.2.4 What the design costs

The multiply in the opening example needs the forward value $x = 3$ to produce its two contributions of 3. ReLU needs its input's sign; matmul needs its operands. TinyTorch keeps references to the input tensors on every operation node and also saves the output tensor. Keeping a reference does not duplicate its array, but it prevents that array from being reclaimed while the node remains reachable. Training therefore retains intermediate results that prediction may no longer need.

The reference keeps all inputs even when a particular rule only needs a shape. After backward releases the links, intermediates with no other references can be reclaimed; any tensors still held by the caller remain alive. During prediction, `no_grad()` prevents these graph links from being retained. The difference is about object lifetime, not a promise that inference always needs exactly one layer's memory.

Problem: For a 12-layer TinyGPT model with batch size $B = 16$, sequence length $S = 1024$, and hidden dimension $D = 768$, what is the activation memory retained on the forward tape?

Per-layer activation footprint (counting only four $B \times S \times D$ tensors per layer and excluding the $S \times S$ attention maps):

$$\text{Layer activations} \approx 4 \times (B \times S \times D) \times 4 \text{ bytes} \approx 4 \times (16 \times 1024 \times 768) \times 4 \approx 201 \text{ MB}$$

Total tape footprint:

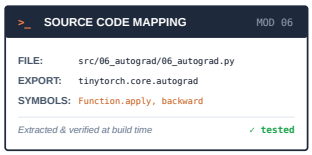
$$\text{Total activation memory} \approx 12 \text{ layers} \times 201 \text{ MB} \approx 2.4 \text{ GB}$$

The excluded attention maps are larger still: each holds $B \times h \times S^2$ values, about 805 MB per layer in float32 with $h = 12$ heads, so a full count reaches tens of gigabytes.

Systems insight: While model parameters for this configuration occupy only $\sim$ 340 MB, forward activation memory on the autograd tape exceeds parameter memory by nearly $7\times$, making activation management the primary bottleneck during training. In Chapter 23, we implement activation checkpointing to break this linear memory growth: by discarding intermediate layer activations during the forward pass and selectively recomputing them during the backward traversal, memory drops from $O(L)$ to $O(\sqrt{L})$ at the cost of one additional forward evaluation.

6.3 The Code

The tensor implementation in Chapter 1 left three fields on every `Tensor` with nothing to do: `requires_grad`, `grad`, and `_grad_fn`. It also routed every operation through one method, `Function.apply`, which computed a result and forgot how. This chapter is where all four earn their keep.



Source Code Mapping: Module 06 extracts automatic differentiation, the backward execution tape, and gradient accumulation directly from the working repository.

6.3.1 An operation that remembers its inputs

Consider the entry point for the opening multiply. `x * x` calls `Tensor.__mul__`, which delegates to `Mul.apply(x, x)`. Chapter 1 supplied `Function.apply` as the common path for every operation. It constructs a fresh `Mul` node whose `inputs` tuple contains two references to the same tensor. Calling `forward` with the inputs' NumPy arrays produces the value 9.

If the framework kept only that result, it would lose both the operation and its two input positions. A per-call node preserves them without requiring a separate backward function for the whole model. Its class supplies the rule, while its instance supplies the values from this particular call.

```python
class Function:
    """
    Base class for every operation the framework can differentiate.
    """

    def __init__(self, *inputs, **params):
        self.inputs = inputs                # the Tensors this operation consumed
        for name, value in params.items():  # axis, shape, key, ... for the operation
            setattr(self, name, value)

    def forward(self, *arrays):
        """Compute the result array from the input arrays. Each operation implements this."""
        raise NotImplementedError(f"{type(self).__name__}.forward is not implemented")

    def backward(self, grad_output):
        """Return one gradient array per input. Module 06 implements this."""
        raise NotImplementedError(f"Module 06 implements {type(self).__name__}.backward")

    @classmethod
    def apply(cls, *inputs, **params):
        """Run the operation on Tensors and return a new Tensor."""
        node = cls(*inputs, **params)
        arrays = [t.data for t in inputs]
        return Tensor(node.forward(*arrays))
```

Read `apply` again with this chapter's question in mind. It builds a node that holds the inputs, runs `forward`, and then drops the node. After `apply` returns, no result points back to that node. Keeping the record makes the result a starting point for a later backward traversal. This is the version of `apply` that Module 06 installs in place of the one above.

```python
@method_of(Function)
@classmethod
def apply(cls, *inputs, **params):
    """
    Run the operation and, when a gradient is wanted, remember it on the output.
    """
    node = cls(*inputs, **params)
    out = Tensor(node.forward(*[t.data for t in inputs]))
    node.output = out
    if _GRAD_TRACKING_ENABLED and any(t.requires_grad for t in inputs):
        out.requires_grad = True
        out._grad_fn = node
    return out
```

The output is a new `Tensor` wrapping the array returned by `forward`. The node keeps this tensor as `node.output`, because some derivative rules reuse the output value. If tracking is enabled and any input requires a gradient, `apply` sets `out.requires_grad = True` and stores the node as `out._grad_fn`. For `x * x`, following that link reaches a `Mul` whose two input entries both refer to `x`. Another operation can extend the chain without knowing how the earlier result was computed.

When a tensor's array is mutated in place (e.g., `x.data[0] = 5` or `x.data += 1`), the numerical values saved during forward execution are destroyed. If `backward()` runs later, operations like `Mul` or `ReLU` read the modified values rather than the activations present when the graph was recorded, silently producing corrupt gradients.

Always ensure parameter updates and buffer overwrites occur *after* `loss.backward()` has finished traversing the graph. `no_grad()` stops new recording; it does not protect values an existing graph already references.

The `if` line controls whether this record survives the call. A prediction using trainable weights still records operations unless tracking is disabled. Inside `no_grad()`, the output has `requires_grad=False` and no `_grad_fn`; leaving the context restores the previous tracking state. Setting a layer to evaluation mode is a separate decision about behavior such as dropout, and does not disable autograd.

A tensor constructed directly, such as `x` or a weight, has no producing operation. Such a tensor is a **leaf** of this record structure. Backward stops there after accumulating its gradient. TinyTorch also stores `.grad` on visited intermediates, which makes the trace inspectable; the production bridge distinguishes that choice from PyTorch's default.

What each node needs to send a gradient back to its inputs is the other half of the operation, the `backward` that Chapter 1 left raising `NotImplementedError`. The rest of this section writes those halves, one class at a time, on the same classes that already implement `forward`.

6.3.2 The broadcast obligation: reducing gradients

In framework tensor runtimes, operations like layer bias additions and channel scaling expand lower-dimensional tensors across batch axes via zero-copy broadcasting. Because forward broadcasting implicitly replicates parameter values across multiple output coordinates, the backward pass incurs an obligation to sum incoming gradients across those broadcasted axes to recover the parameter's original shape. For a concrete bias addition, let the activation batch be `[[1, 2, 3], [4, 5, 6]]` with shape `(2, 3)` and the bias be `[10, 20, 30]` with shape `(3,)`. Forward returns `[[11, 22, 33], [14, 25, 36]]`. NumPy broadcasting reuses each bias entry across rows without requiring an explicitly tiled bias array; the addition still creates a result, and TinyTorch wraps it in its own float32 storage. An upstream gradient `[[1, 2, 3], [4, 5, 6]]` must therefore give the bias `[5, 7, 9]`, summing the two uses of each entry.

If the framework blindly pushed that gradient into the bias, two things would break. First, the shapes would mismatch, crashing the first elementwise update. Second, and worse, the bias gradient would fail to accumulate the contributions from every example in the batch. By the multivariable chain rule, because b_j was added to every row i of the output, its total gradient is the sum across the batch:

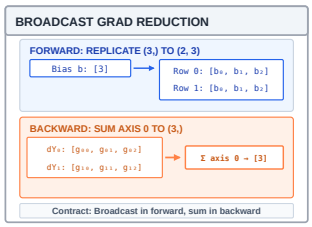
$$\frac{\partial L}{\partial b_j} = \sum_{i=0}^{B-1} \frac{\partial L}{\partial y_{i,j}} \frac{\partial y_{i,j}}{\partial b_j} = \sum_{i=0}^{B-1} \frac{\partial L}{\partial y_{i,j}} \times 1 = \sum_{i=0}^{B-1} (\nabla \mathbf{Y})_{i,j}$$

The mathematical summation across the batch becomes an array reduction across the leading axes. TinyTorch enforces this contract with `_reduce_broadcast_grad`, which sums out extra leading dimensions and collapses singleton axes back to the parameter's original shape.

```
def _reduce_broadcast_grad(grad, original_shape):
    """
    Reduce gradient to match original tensor shape after broadcasting.
    """
    # Step 1: Remove leading dimensions that weren't in original
    ↪   tensor
    # Example: grad (32, 128) with original (128,) → sum over axis 0
    while grad.ndim > len(original_shape):
        grad = grad.sum(axis=0)

    # Step 2: Collapse dimensions where original had size 1
    # Example: grad (10, 5) with original (10, 1) → sum over axis 1
    ↪   with keepdims
    for i in range(len(original_shape)):
        if i < grad.ndim and original_shape[i] == 1 and grad.shape[i]
        ↪   > 1:
            grad = grad.sum(axis=i, keepdims=True)

    return grad
```



Broadcast gradient reduction: Replicating forward requires summing cotangents across batch axes in backward.

With this helper in place, the backward half of `Add` handles scalars, vectors, and broadcast tensors with the same local rule. The `@method_of(Add)` line above the definition is how Module 06 writes a method onto a class that Module 01 defined; it is the same as writing `backward` inside `class Add`, without editing the earlier module.

```
@method_of(Add)
def backward(self, grad_output):
    """
    Gradient computation for tensor addition.
    """
    a, b = self.inputs
    grad_a = grad_b = None

    # Gradient for first input
    if isinstance(a, Tensor) and a.requires_grad:
        grad_a = grad_output
        # Handle broadcasting: reduce gradient to match original shape
        grad_a = _reduce_broadcast_grad(grad_a, a.data.shape)

    # Gradient for second input
    if isinstance(b, Tensor) and b.requires_grad:
        grad_b = grad_output
        # Handle broadcasting: reduce gradient to match original shape
        grad_b = _reduce_broadcast_grad(grad_b, b.data.shape)

    return grad_a, grad_b
```

The rule itself is one line per input, `grad_a = grad_output`, because $\partial(a+b)/\partial a = 1$. Everything else in the listing is the obligation: check that the input wants a gradient at all, and reduce the gradient to the input's own shape. The walker at the end of this section applies the same reduction to whatever any rule returns, so a rule that forgets it still lands the right shape; `Add` does it here because addition is the rule where the obligation is easiest to see. In the concrete example the helper reduces axis 0, changing `(2, 3)` into `(3,)`. If the bias were stored as `(1, 3)`, it would instead preserve the singleton row axis. The returned gradient must match the original input shape in either case.

6.3.3 Matrix multiplication, ReLU, and sum

Matrix multiplication is where the weights are. Chapter 3's `Linear` layer is `x @ W` plus a bias, so every weight matrix in an MLP, and later every projection in Chapter 12's attention, receives its gradient through the matmul rule. It is also the rule where the naive approach is most obviously unaffordable. In the general case, an operation that maps a vector of length n to a vector of length m has an $m \times n$ matrix of local derivatives called the Jacobian,[26] and the textbook way to apply the chain rule is to build it and multiply. For a linear layer with batch 256 and width 4096, the output has about a million entries and W has nearly seventeen million, so $\partial Y/\partial W$ has roughly 1.8×10^{13} entries, about 70 TB in float32, for one layer of one training step.

[26] **Jacobian**: the matrix of all partial derivatives $\partial y_i/\partial x_j$ of a vector-valued function. For a layer with 1024 inputs and 1024 outputs it has a million entries per example, which is why autograd computes gradient-times-Jacobian products instead of the Jacobian itself.

The reference never constructs this array. The matmul rule from The Idea computes only the product that the chain rule needs. GB^T and A^TG are the products of the incoming gradient with that Jacobian, computed directly as two matrix multiplications the size of the forward one, and the gradient for W comes out at W's own size, 67 MB rather than 70 TB. This direct multiplication of an incoming gradient by local derivatives is called a **vector-Jacobian product**, and every backward rule in this chapter computes such a product. The incoming gradient summarizes the loss downstream, so the rule does not need every local partial derivative separately.

```
@method_of(MatMul)
def backward(self, grad_output):
    """
    Gradient computation for matrix multiplication.
    """
    a, b = self.inputs
    grad_a = grad_b = None

    # A vector behaves as a one-row left matrix or a one-column right
    ↪   matrix.
    # Restore those dimensions on the output gradient before applying
    ↪   the same
    # two matrix rules; remove them again before returning each input
    ↪   gradient.
    a_vector, b_vector = a.data.ndim == 1, b.data.ndim == 1
    a_matrix = a.data[np.newaxis, :] if a_vector else a.data
    b_matrix = b.data[:, np.newaxis] if b_vector else b.data
    matrix_grad = grad_output
    if b_vector:
        matrix_grad = np.expand_dims(matrix_grad, -1)
    if a_vector:
```

```
        matrix_grad = np.expand_dims(matrix_grad, -2)

    if a.requires_grad:
        grad_a = np.matmul(matrix_grad, np.swapaxes(b_matrix, -2, -1))
        if a_vector:
            grad_a = np.squeeze(grad_a, axis=-2)
    if b.requires_grad:
        grad_b = np.matmul(np.swapaxes(a_matrix, -2, -1), matrix_grad)
        if b_vector:
            grad_b = np.squeeze(grad_b, axis=-1)

    return grad_a, grad_b
```

The two-dimensional case is the two-line formula from The Idea, and the shapes work out for the same reason. The branches around it are there because the real module has to serve Chapter 12, where attention multiplies batches of matrices at once; `np.swapaxes` transposes only the last two axes so the batch axis is left alone, and vector operands are temporarily promoted to row or column matrices. Removing those added axes afterward lets the same matrix rule cover vector and batched cases without flattening a batch into one outer product.

ReLU and sum are simpler, and they are here because the worked trace needs them. ReLU supplies a nonlinear step, and sum produces a scalar whose backward seed can default to one. Notice what each rule reads from its node, because that is exactly what has to stay in memory until backward runs: matmul read both operands, ReLU reads its input, and sum uses its input to recover the shape and gradient requirement. The node nevertheless retains the full input tensor.

```
@method_of(ReLUFunction)
def backward(self, grad_output):
    """
    Gradient computation for ReLU activation.
    """
    tensor, = self.inputs

    if isinstance(tensor, Tensor) and tensor.requires_grad:
        # ReLU gradient: 1 if x > 0, else 0
        relu_grad = (tensor.data > 0).astype(np.float32)
        return grad_output * relu_grad,
    return None,
```

```
@method_of(Sum)
def backward(self, grad_output):
    """
    Gradient computation for tensor sum.
    """
    tensor, = self.inputs

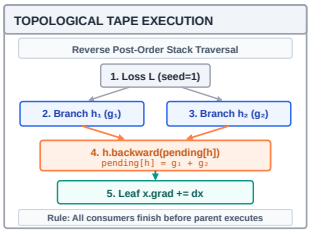
    if isinstance(tensor, Tensor) and tensor.requires_grad:
```

```
        # For axis-reduced sums, expand grad_output back along the
        ↪ summed
        # axis before broadcasting, so each row/column gets its own
        ↪ gradient.
        if self.axis is not None and not self.keepdims:
            grad_output = np.expand_dims(grad_output, axis=self.axis)
        return np.ones_like(tensor.data) * grad_output,
    return None,
```

The ReLU mask is the derivative of $\max(0, z)$ evaluated at the forward input. Sum broadcasts its scalar gradient back to the shape it reduced, and `expand_dims` restores an axis removed by `sum(axis=1)` so each row receives its own incoming gradient. A sum of a `(2, 3)` tensor along axis 1 has shape `(2,)`; an incoming `[2, 5]` becomes `[[2], [5]]`, then broadcasts to `[[2, 2, 2], [5, 5, 5]]`.

The same requirement applies to the loss rules from Chapter 4. For example, binary cross-entropy clips a probability into $[10^{-7}, 1 - 10^{-7}]$ before taking logarithms. Outside that interval its implemented forward value is constant, so the prediction gradient is zero; inside it, the rule differentiates the logarithmic expression. At the two clip boundaries the implementation chooses the derivative from the interior. A backward rule must match the implemented forward function, including such numerical choices.

6.3.4 Walking the graph backward



Reverse topological sort guarantees all consumer paths contribute to pending gradients before a parent node executes.

In production runtimes like PyTorch's C++ autograd engine (`torch::autograd::Engine`), backward execution coordinates dependencies across complex acyclic computation graphs. By executing nodes in reverse topological order, the engine guarantees that all downstream consumers finish depositing their cotangent contributions before an upstream node evaluates its vector-Jacobian product. The records are in place. What is left is the call the training loop of Chapter 8 will make once per step, `loss.backward()`, which has to visit every record and leave every leaf holding its gradient.

A tempting walk combines recursion with a visited set. Start at the loss, run its rule, and immediately recurse into each unvisited input. On the straight chain in the worked trace below that gives the right answer, which is what makes it dangerous. For `x = 2`, let `h = x * 2` and `L = h * 3 + h * 4`. The two branches contribute 3 and 4 at `h`; its multiply must send $(3 + 4) \times 2 = 14$ to `x`. If a recursive walk reaches `h` through the first branch and immediately runs its rule, it sends only $3 \times 2 = 6$. When the second branch arrives, the visited set suppresses another traversal of `h`, losing the remaining 8. Dropping the visited set while propagating the total `.grad` causes a different error: the later traversal also resends the first contribution. No error, a wrong number, and a model that trains a little worse than it should.

The fix is to run no rule until every consumer of its tensor has reported. A topological order guarantees exactly that, and a depth-first walk hands us one for free. The implementation therefore does four things: sorts the graph, seeds the gradient at the loss, walks the order once, and releases the graph.

```
@method_of(Tensor)
def backward(self, gradient=None, retain_graph=False):
    """
```

```
    Compute gradients via backpropagation.
    """

    # Only compute gradients if required
    if not self.requires_grad:
        return

    # Initialize gradient if not provided (for scalar outputs)
    if gradient is None:
        if self.data.size == 1:
            gradient = np.ones_like(self.data)
        else:
            raise ValueError(
                f"backward() called on non-scalar tensor without gradient argument.\n"
                f"  Tensor shape: {self.shape}\n"
                f"  Issue: For non-scalar outputs, you must provide the gradient from the next layer.\n"
                f"  Fix: Call backward(gradient) with the gradient tensor from the loss function."
            )

    if isinstance(gradient, Tensor):
        gradient = gradient.data
    gradient = np.asarray(gradient)
    if gradient.shape != self.shape:
        raise ValueError(
            f"backward gradient shape {gradient.shape} must match output shape {self.shape}. "
            "Supply one gradient value per output element."
        )

    # ---- Step 1: topological sort ------------------------------------
    # Depth-first from the output, appending each tensor only AFTER its
    # parents. Reversing that post-order gives an order in which every
    # consumer of a tensor appears before the tensor itself.
    topo_order = []
    seen = set()
    # Iterative depth-first post-order (a recursive visit would hit Python's
    # recursion limit on a few thousand chained operations).
    stack = [(self, False)]
    while stack:
        tensor, expanded = stack.pop()
        if expanded:
            topo_order.append(tensor)
            continue
        if id(tensor) in seen:
            continue
        # Check every intermediate before changing any gradients. A released
```

```
        # intermediate has no grad_fn, but it is not a new leaf:
        ↪ stopping there
        # would silently lose the gradient to the original inputs.
        if getattr(tensor, "_graph_released", False):
            raise RuntimeError(
                "Trying to backward through a graph that was released.
                ↪ "
                "Recompute the forward pass, or use retain_graph=True
                ↪ on the earlier backward()."
            )
        seen.add(id(tensor))
        stack.append((tensor, True))
        fn = tensor._grad_fn
        if fn is not None:
            for parent in fn.inputs:
                if isinstance(parent, Tensor) and
                ↪ parent.requires_grad:
                    stack.append((parent, False))
    topo_order.reverse()

    # ---- Step 2: seed the output
    ↪ ------------------------------------
    # pending[id(tensor)] holds the gradient accumulated from the
    ↪ consumers
    # visited so far. Keying by id() makes the identity lookup
    ↪ explicit. Tensor
    # hashes by identity today, so the tensor itself would work as a
    ↪ key, but an
    # elementwise __eq__ (which frameworks do add) would quietly break
    ↪ that.
    pending = {id(self): gradient}

    # ---- Step 3: one pass, in topological order
    ↪ ------------------------
    for tensor in topo_order:
        grad = pending.get(id(tensor))
        if grad is None:
            continue

        if not tensor.requires_grad:
            continue

        # Accumulate into this tensor's .grad
        if tensor.grad is None:
            tensor.grad = np.zeros_like(tensor.data)
        grad = _reduce_broadcast_grad(grad, tensor.grad.shape)
        if np.shape(grad) != tensor.grad.shape:
            raise ValueError(f"gradient of shape {np.shape(grad)} does
            ↪ not match tensor of shape {tensor.grad.shape}")
        tensor.grad += grad

        # Hand each parent its share
```

```
        fn = tensor._grad_fn
        if fn is None:
            continue
        parent_grads = fn.backward(grad)
        for parent, parent_grad in zip(fn.inputs, parent_grads):
            if not isinstance(parent, Tensor) or parent_grad is None:
                continue
            if not parent.requires_grad:
                continue
            parent_grad = _reduce_broadcast_grad(parent_grad,
    parent.data.shape)
            if np.shape(parent_grad) != parent.data.shape:
                raise ValueError(
                    f"{fn!r} returned a gradient of shape
                    {np.shape(parent_grad)} for an input of shape
                    {parent.data.shape}"
                )
            if id(parent) in pending:
                pending[id(parent)] = pending[id(parent)] +
    parent_grad
            else:
                pending[id(parent)] = parent_grad

    # ---- Step 4: release the graph, once
    ----------------------------
    # The graph holds references to every intermediate tensor, so
    without
    # this, memory grows with each training step. One sweep at the end
    is the
    # simplest place to do it. The topological order also makes
    per-node
    # release safe, since each grad_fn is read exactly once, and that
    is how
    # PyTorch frees memory as it walks. Same gradients either way.
    if not retain_graph:
        for tensor in topo_order:
            if tensor._grad_fn is not None:
                tensor._graph_released = True
            tensor._grad_fn = None
```

Before step 1 changes any gradients, the method checks the external seed. A one-element output may omit the seed; `ones_like` then gives it exactly the output shape. An explicit seed must have that same shape, even when NumPy could broadcast it. For an output of shape `(2, 3)`, a seed of shape `(3,)` is rejected: the caller must specify the contribution of each of the six output elements. This entry contract differs from `_reduce_broadcast_grad`, which reverses broadcasting performed by a known forward operation. Silently broadcasting an external seed would invent a caller intent that no operation recorded.

Step 1 is the depth-first walk. It uses an explicit stack rather than recursion so a long chain does not consume one Python call frame per operation. Each first visit pushes an expanded entry onto the stack and then pushes its parents above it, so `topo_order` lists

inputs before the tensors computed from them and reversing it puts the loss first and the leaves last. The `seen` set records `id(tensor)`, so tensor identity determines whether it was visited. Two references to the same tensor produce one entry in the order; two distinct tensors with equal values remain distinct.

Step 2 creates `pending = {id(self): gradient}`. In the scalar trace this gives the loss an incoming value of one, expressing that changing the loss changes itself by the same amount. No other tensor has a pending contribution yet.

Core Invariant: Fan-In Accumulation

$$x.\text{grad} \mathrel{+}= \sum_k \text{vjp}_k$$

Silent bug: assigning = instead of += drops multi-branch gradients without raising an error.

Step 3 maintains two distinct kinds of gradient state. The local `pending` dictionary starts fresh on every backward call; a tensor's `.grad` may already contain contributions from an earlier call. The rule receives the current call's `grad` from `pending`, never the historical total in `.grad`. Replaying a retained graph therefore adds a new derivative to the leaf without sending the old derivative through the graph a second time.

Step 3 is also where "used twice means add" lives. `pending` holds, for each tensor, the gradient accumulated so far from the consumers already visited. A tensor is only taken from `pending` when the walk reaches it, and by the order's guarantee every consumer has already added its share, so the addition into an existing `pending[id(parent)]` entry combines contributions before that parent is processed. Each node's `backward` returns one gradient per input, in the order the inputs were given, and the walker reduces each one to its input's shape before handing it on. Nothing in the loop knows what any operation does. It only knows the order.

Step 4 severs the `_grad_fn` links after the walk and marks tensors that had a producing operation with `_graph_released`. The marker distinguishes a consumed intermediate from an original leaf. Both now have no `_grad_fn`, but only the leaf is a valid stopping point for a later backward pass. Step 1 checks the marker on every reachable differentiable tensor before step 3 mutates any gradient. A new loss built from an old intermediate therefore raises instead of silently stopping short of its original inputs.

Passing `retain_graph=True` on the earlier call preserves the links so another backward call can traverse them. It does not make the derivatives differentiable: these rules compute NumPy arrays, so retaining the graph does not implement higher-order derivatives. For ordinary training, the next forward pass builds a new graph from the updated parameters. Retention is needed only when more than one backward traversal must use the same forward computation.

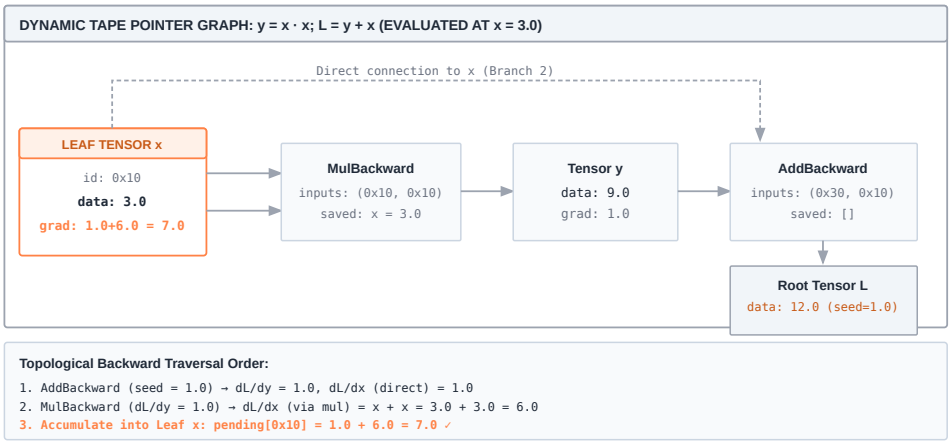


Figure 6.3: **The autograd pointer DAG and topological tape**: As operations execute forward, each `Tensor` retains a pointer to its creator node (`_grad_fn`), and each node stores references to its input tensors. Calling `loss.backward()` performs an iterative post-order depth-first stack traversal, reverses the order to produce a reverse topological sequence, and propagates vector-Jacobian products backward while accumulating in-place gradients into leaf `.grad` buffers.

6.4 A Worked Trace

The network from The Problem is small enough to check by hand. Take an input $\mathbf{x}$, a weight matrix $\mathbf{W}$, and compute a pre-activation, a ReLU, and a scalar loss:

$$\mathbf{x} = \begin{bmatrix} 1.0 & 2.0 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 0.5 & -0.5 \\ 1.0 & 0.5 \end{bmatrix}, \quad \mathbf{z} = \mathbf{x}\mathbf{W}, \quad \mathbf{a} = \mathrm{ReLU}(\mathbf{z}), \quad L = \sum \mathbf{a}.$$

The forward pass gives

$$\begin{aligned} \mathbf{z} &= \begin{bmatrix} 1.0(0.5) + 2.0(1.0) & 1.0(-0.5) + 2.0(0.5) \end{bmatrix} \\ &= \begin{bmatrix} 2.5 & 0.5 \end{bmatrix}, \\ \mathbf{a} &= \begin{bmatrix} 2.5 & 0.5 \end{bmatrix}, \qquad L = 3.0. \end{aligned}$$

Both entries of $\mathbf{z}$ are positive, so ReLU passes both through and its mask is all ones. As the forward pass runs, `apply` creates three nodes: a `MatMul` whose inputs are $(\mathbf{x}, \mathbf{W})$, a `ReLUFunction` whose input is $\mathbf{z}$, and a `Sum` whose input is $\mathbf{a}$. The depth-first walk from L lists the tensors as $[\mathbf{W}, \mathbf{x}, \mathbf{z}, \mathbf{a}, L]$, and reversing it gives the backward order $[L, \mathbf{a}, \mathbf{z}, \mathbf{x}, \mathbf{W}]$. The explicit stack pops the last scheduled parent first, placing $\mathbf{W}$ before $\mathbf{x}$ in the initial list. The order between the two leaves does not matter because leaves have no node to run, and table 6.1 follows the loop through the three that do.

Table 6.1: The backward loop on the four-weight network. Each rule receives this call's pending gradient and returns contributions for its inputs. Vector entries in the table represent row arrays of shape `(1, 2)`.

Step	Node	Gradient at this tensor	Local rule	Sent to inputs
seed	L	1.0	$\partial L/\partial L = 1$	(none yet)
1	$L = \sum \mathbf{a}$	1.0	broadcast to **a**'s shape	**a** gets $[1.0, 1.0]$
2	$\mathbf{a} = \mathrm{ReLU}(\mathbf{z})$	$[1.0, 1.0]$	multiply by mask $[1, 1]$	**z** gets $[1.0, 1.0]$
3	$\mathbf{z} = \mathbf{x}\mathbf{W}$	$[1.0, 1.0]$	$\mathbf{x}^T G$ and $G\mathbf{W}^T$	**W** gets $\begin{bmatrix} 1.0 & 1.0 \\ 2.0 & 2.0 \end{bmatrix}$, **x** gets $[0.0, 1.5]$
4, 5	**x**, **W**	(leaves)	no node	done

The two matmul results are worth doing by hand once. With $G = [1.0, 1.0]$,

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{x}^T G = \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} \begin{bmatrix} 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 1.0 \\ 2.0 & 2.0 \end{bmatrix},$$

$$\frac{\partial L}{\partial \mathbf{x}} = G\mathbf{W}^T = \begin{bmatrix} 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 \\ -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.0 & 1.5 \end{bmatrix}.$$

The zero in $\partial L/\partial \mathbf{x}$ has a plain meaning. Raising x_0 by a small amount raises z_0 by half that amount and lowers z_1 by the same half, and since L is $z_0 + z_1$, the two effects cancel. The gradient with respect to **W** matches the finite-difference estimate from The Problem to within its noise, which is the check that matters.

Running the real module on the same inputs reproduces the table (listing 6.2). Line 3 imports `tinytorch.core.autograd`, which completes the operation classes with their backward methods. The package's `__init__` already performs this import, so the line documents the dependency rather than changing behavior. In lines 5 to 6, `x` and `W` are declared with `requires_grad=True`. Lines 8 to 10 evaluate the forward sequence $z \to a \to L$, attaching operation nodes inspected in lines 11 to 12 (`<MatMulBackward>`, `<ReLUBackward>`, `<SumBackward>`). In line 14, `L.backward()` seeds the gradient at 1.0 and traverses the tape in reverse topological order, populating `W.grad` and `x.grad` in lines 15 to 17 to match the hand-derived matrix products in table 6.1 exactly.

The accumulation rule can be checked with a two-line experiment, and it is worth running because `x + x` is the smallest version of a residual connection, where a tensor is added back to something computed from it. In listing 6.3, line 1 creates `x` and line 2 evaluates `(x + x).sum().backward()`. The same tensor occupies both input slots of one `Add` node, so the tape traversal adds to its `pending` entry twice, producing `[[2. 2.]]` in line 3.

Residual connections ($y = x + F(x)$) feed a single activation tensor into multiple downstream consumers. If an autograd engine used assignment (`pending[parent] = incoming`) rather than accumulation (`pending[parent] += incoming`), the gradient

Listing 6.2 Tracing a matrix network: Three recorded operations produce the weight and input gradients.

```
from tinytorch.core.tensor import Tensor
from tinytorch.core.activations import ReLU
import tinytorch.core.autograd          # completes every operation
    with its backward half

x = Tensor([[1.0, 2.0]], requires_grad=True)
W = Tensor([[0.5, -0.5], [1.0, 0.5]], requires_grad=True)

z = x @ W
a = ReLU()(z)
L = a.sum()
print(L.data, z._grad_fn, a._grad_fn, L._grad_fn)
# 3.0 <MatMulBackward> <ReLUBackward> <SumBackward>

L.backward()
print(W.grad)           # [[1. 1.]
                        #  [2. 2.]]
print(x.grad)           # [[0.  1.5]]
```

Listing 6.3 One tensor in two input positions: The two additions to the pending gradient must both survive.

```
x = Tensor([[1.0, 2.0]], requires_grad=True)
(x + x).sum().backward()
print(x.grad)           # [[2. 2.]]
```

from the skip connection would overwrite the gradient flowing through the residual sub-block, silently zeroing out feature updates in deep networks.

Replace the `+` in the `pending` update inside `backward` with an assignment and this prints `[[1. 1.]]`, with no error to tell us it is wrong. A shared intermediate adds a second obligation: its local rule must propagate the combined gradient once. The trace in listing 6.4 checks accumulation across separate calls that share a forward intermediate. In lines 1 to 3, `h = x * 2` branches into `left` and `right`. Line 4 runs `left.backward(retain_graph=True)`, printing `x.grad = [6.]` in line 5. Line 6 runs `right.backward()`, accumulating an additional 8 to yield `x.grad = [14.]` in line 7. In lines 9 to 12, attempting to backpropagate through `h` a third time raises a `RuntimeError` because the unretained backward pass released `h`'s computational graph.

On the first call (line 4), `h` receives three and sends six to `x`. On the second (line 6), its fresh `pending` entry receives four and sends eight, even though `h.grad` also retains the earlier three. The second call releases `h`'s producing operation. The third call (line 10) reaches that marked intermediate during graph validation and stops before changing `x.grad`. Recomputing `h = x * 2` creates a usable graph; clearing `x.grad` alone does not restore the old one.

Listing 6.4 Gradient and graph lifetimes: Contributions accumulate across calls, but a released intermediate cannot be traversed again.

```
x = Tensor([2.0], requires_grad=True)
h = x * 2
left, right = (h * 3).sum(), (h * 4).sum()
left.backward(retain_graph=True)
print(x.grad)                          # [6.]
right.backward()
print(x.grad)                          # [14.], six from left plus eight
    from right

try:
    (h * 5).sum().backward()           # new root, but h's graph was
    released
except RuntimeError:
    print(x.grad)                      # [14.], validation preceded
        accumulation
```

6.5 From TinyTorch to Production

PyTorch also records executed operations and exposes a result's producing node through `grad_fn`.[27] Its backward engine applies local derivative rules to saved forward values. A production implementation can save only the values a rule needs; TinyTorch keeps every input and its output to make the record easy to inspect. PyTorch also checks version counters to detect changes to saved tensors. PyTorch autograd mechanics.

In PyTorch's native C++ core (`torch/csrc/autograd`), every `Tensor` maintains an internal `_version` counter incremented on every in-place mutation. When a backward rule requires saved forward activations (via `ctx.save_for_backward`), it records their version. If any saved tensor's version has changed by the time backward uses it, PyTorch raises: `RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation`.

PyTorch also executes the reverse traversal using a ready-queue worker thread pool, executing vector-Jacobian products in C++ without Python interpreter or GIL overhead.

That mutation check marks a boundary of this reference. TinyTorch nodes hold references to mutable tensors, not immutable snapshots. If `x` is `[3]` when `y = x * x` runs, changing `x.data` to `[4]` before backward makes the multiply read 4 and return a gradient of 8 for a forward calculation performed at 3. TinyTorch does not detect that mismatch. Parameter updates must follow the backward passes that need the old values; `no_grad()` disables recording but does not make an early mutation safe.

The public gradient-storage contract also differs. With its default arguments, PyTorch accumulates `.grad` on leaf tensors and requires `retain_grad()` to retain an intermediate's gradient. TinyTorch stores gradients on all visited differentiable tensors. Both accumulate across backward calls rather than clearing an existing gradient automatically. PyTorch's `backward` also offers `create_graph` for differentiating derivatives; TinyTorch's NumPy backward rules do not. PyTorch Tensor.backward.

Saved activations add a memory cost whose size depends on the workload. For example, an activation array of shape `(32, 128, 512)` contains 2,097,152 values and occupies 8,388,608 bytes in float32, about 8.4 MB. That is an array-size calculation,

[27] If a backward closure or tape node retains a direct Python reference to its own output tensor while that tensor retains a reference to the node via `_grad_fn`, a cyclic reference is formed. CPython's reference-counting memory manager cannot reclaim cyclic structures immediately upon variable scope exit, forcing the system to fall back on the periodic generational garbage collector. TinyTorch forms exactly this cycle (`node.output` and `out._grad_fn`) and breaks it when backward releases the graph; a graph that is never differentiated waits for the cyclic collector. PyTorch avoids the cycle by saving outputs through weak references (`c10::weak_intrusive_ptr`).

not a measured training peak. Parameters, accumulated gradients, operation records, temporary arrays, and later optimizer state also occupy memory. Keeping several such activations alive can therefore change whether a model fits, but neither activations nor parameters always dominate.

Production frameworks can trade recomputation for saved-array storage. **Activation checkpointing** keeps selected intermediate values and reruns portions of forward computation when backward needs omitted values. The exact memory saving and extra work depend on which portions are checkpointed; TinyTorch does not implement this policy in Module 06. In Chapter 23, we build a full activation checkpointing wrapper that discards activations across transformer layers to reduce forward tape memory from $O(L)$ to $O(\sqrt{L})$. PyTorch checkpoint documentation. The transferable invariant remains the same: a rule needs the values belonging to the forward computation being differentiated, whether the implementation retained those values or reconstructed them.

6.6 Summary

`Function.apply` connects an output tensor to the operation and inputs that produced it whenever gradient tracking is enabled. Each local `backward` reads that record and returns one contribution per input. `Tensor.backward` validates the seed and graph lifetime, starts a fresh `pending` dictionary, and visits tensors only after their consumers have contributed. Broadcast reduction restores every input's own shape before accumulation.

The two gradient lifetimes are the main contract to remember. `pending` belongs to one call and supplies the derivative propagated through each rule; `.grad` remains on tensors across calls until it is cleared. Graph links have a separate lifetime and are released by default, so clearing `.grad` does not make a consumed graph reusable. At return, differentiable parameters reached by the loss hold accumulated gradients; unused parameters can still have `grad=None`. The optimizer in Chapter 7 can now turn those gradients into updates, after the forward values needed by backward have served their purpose.

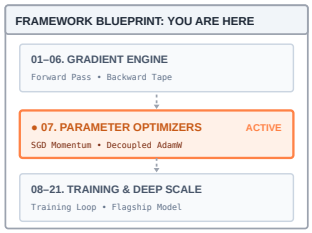
6.7 Further Reading

- **Griewank and Walther** (2008). "Evaluating Derivatives." *SIAM*. The standard reference on reverse-mode differentiation, including its memory cost and checkpointing.
- **Baydin et al.** (2018). "Automatic differentiation in machine learning: a survey." *JMLR*. Forward and reverse mode side by side, and why machine learning uses the reverse one.
- **Paszke et al.** (2019). "PyTorch: an imperative style, high-performance deep learning library." *NeurIPS*. The design of a production tape that records operations as they run, as this chapter's does.
- **Abadi et al.** (2016). "TensorFlow: a system for large-scale machine learning." *OSDI*. For contrast, how a framework builds and optimizes a static computational graph before executing it (Define-and-Run), unlike our dynamic Define-by-Run tape.

🔥 Chapter 7

Optimizers: Momentum, AdamW, and Decoupled Weight Decay

Gradient descent is the engine that drives neural learning, but raw gradients alone are often insufficient to train deep models effectively. High-dimensional loss landscapes are characterized by ill-conditioned ravines, saddle points, and extreme curvature variations where gradients are steep in some directions and nearly flat in others. Plain gradient descent oscillates uncontrollably across canyon walls while making glacial progress along the valley floor toward the optimum.



Framework Blueprint: Module 07 translates raw backward gradients into parameter trajectory updates through momentum and adaptive moments.

In PyTorch, optimizing parameters is encapsulated in an optimizer object with a two-method protocol:

```
import torch

model = torch.nn.Linear(10, 2)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3,
    weight_decay=1e-2)

# Inside the training iteration:
optimizer.zero_grad()                    # 1. Clear residual gradient
    buffers
loss = model(torch.randn(4, 10)).sum()   # 2. Forward pass
loss.backward()                          # 3. Accumulate gradients
optimizer.step()                         # 4. Update parameter
    trajectories
```

Underneath that straightforward step, the optimizer coordinates extensive per-parameter state:

1. **Auxiliary moment buffer tracking:** For every registered parameter, adaptive optimizers allocate and maintain persistent first-moment (momentum m_t) and second-moment (uncentered variance v_t) tensors, tripling the parameter memory footprint.
2. **Bias correction during early iterations:** Because running moments are initialized to zeros, early estimates are biased toward zero. The runtime computes exponential decay correction factors ($1-\beta_1^t$ and $1-\beta_2^t$) to rescale steps during initial warm-up.
3. **Decoupled weight decay (AdamW):** Rather than adding L_2 penalties to the gradient (which distorts moment estimates), the optimizer directly shrinks weight values ($W \leftarrow W(1-\eta\lambda)$) independently of the adaptive gradient scale.

This chapter builds TinyTorch's optimization suite (SGD with momentum, Adam, and AdamW), providing the parameter update engine that drives the unified training loop

in Chapter 8. Before building these algorithms, we must examine why plain gradient descent fails on stretched loss surfaces.

7.1 The Problem

↩ **Jump back:** Chapter 6 *Why `zero_grad()` must clear accumulated `.grad` buffers before the next backward pass.*

Take the smallest loss that has the shape of a real one. Two weights, x and y, and

$$L(x, y) = \tfrac{1}{2}\left(100\,x^2 + y^2\right).$$

The minimum is at the origin. The loss is a bowl, but a stretched one, a hundred times steeper in x than in y. Its gradient is $(100x,\ y)$, so at the same distance from the bottom the x component is a hundred times larger than the y component.

The simplest update subtracts a fixed fraction η of the gradient from each weight. Run that on the ravine with $\eta = 0.03$ in listing 7.1.

Listing 7.1 Ravine divergence: A shared learning rate destabilizes the steep coordinate.

```
def grad(x, y):
    """Gradient of L(x, y) = 0.5 * (100 x^2 + y^2)."""
    return 100.0 * x, y

x, y, lr = 1.0, 1.0, 0.03
for step in range(6):
    gx, gy = grad(x, y)
    x, y = x - lr * gx, y - lr * gy
    print(f"step {step + 1}: x = {x:9.3f}   y = {y:.4f}")
```

The y coordinate behaves. It shrinks by three percent a step, from 1.0 to 0.833 after six steps. The x coordinate explodes: -2, 4, -8, 16, -32, 64. Each step multiplies x by $1 - 0.03 \times 100 = -2$, so it flips sign and doubles. Any η above 0.02 makes the magnitude grow, because $|1 - 100\eta|$ exceeds one. At exactly 0.02, x alternates between 1 and -1 without approaching zero.

Lower the learning rate to $\eta = 0.019$ and x is tamed. It still flips sign every step, but each flip shrinks it by ten percent, and after fifty steps it is 0.005. Now y is the problem. It shrinks by 1.9 percent per step, so after 100 steps it is still 0.147 and after 250 steps it is 0.008. One knob serves two masters. The steep direction sets a ceiling on η, and the flat direction wants a floor a hundred times higher.

The two-coordinate example isolates a problem that also arises in networks: one shared learning rate must serve directions with different curvature, meaning different rates of change of the gradient. The optimizers below use gradient history to respond to that mismatch. None guarantees convergence for an arbitrary loss or learning rate.

7.2 The Idea

7.2.1 Gradient descent, and why it is called stochastic

In modern deep learning runtimes, gradient-based optimization drives parameter updates across every architecture from standard multilayer perceptrons to billion-parameter foundation models. Because computing exact loss gradients over an entire multi-gigabyte dataset is computationally and memory-prohibitive, frameworks evaluate loss on small mini-batches, turning classical gradient descent into an efficient stochastic approximation. Write the parameters as one long vector θ and the gradient of the loss with respect to them as g. Plain gradient descent is

$$\theta \leftarrow \theta - \eta\, g,$$

where η is the **learning rate**, the fraction of the gradient applied per step. In training, g is computed on one mini-batch rather than the whole dataset, so it is a noisy estimate of the true gradient. That noise is why the method is called **stochastic gradient descent** (SGD). The noise is a second reason, besides the ravine, to average gradients over several steps instead of trusting each one alone.

↪ **Jump ahead:** Chapter 8 *Coordinates in-place optimizer parameter mutation within the unified training lifecycle.*

7.2.2 Momentum: let consistent gradients add up

Widely adopted in competitive vision models such as ResNets and ConvNeXts, momentum augments stochastic gradient descent by carrying an exponentially decayed history of past update vectors. This running velocity buffer dampens high-frequency oscillations across steep ravine walls while accelerating parameter progress along gentle, consistent slopes. Suppose two successive gradients are both 2, the learning rate is 0.1, and the retained fraction of the previous update is 0.9. Starting with velocity zero, the first velocity is 2 and the second is $0.9 \times 2 + 2 = 3.8$. The weight moves by 0.2, then 0.38. If the second gradient were -2, the second velocity would instead be -0.2, giving a much smaller reversal. **Momentum** implements this memory with a velocity v:

$$v \leftarrow \beta\, v + g, \qquad \theta \leftarrow \theta - \eta\, v.$$

[28]

The coefficient β, usually 0.9, is the fraction of the previous velocity that survives into the next step. Unrolling the recurrence shows what v holds after t steps:

$$v_t = g_t + \beta\, g_{t-1} + \beta^2 g_{t-2} + \cdots$$

For a constant gradient g, the terms reinforce and v approaches $g/(1-\beta)$, ten times g at $\beta = 0.9$. For an alternating gradient of fixed magnitude, they partly cancel and the velocity's eventual magnitude is $|g|/(1+\beta)$. Those two idealized sequences explain how momentum can reinforce a consistent direction and soften reversals. In a real optimization run the gradient changes with the weights, so the factors are not universal speedups or guarantees against oscillation.

[28] Classical momentum (Polyak, 1964) computes velocity updates using the gradient at the current position $g_t = \nabla\mathcal{L}(\theta_t)$. In contrast, Nesterov accelerated gradient evaluates the gradient at an anticipated look-ahead position; in this chapter's convention ($\theta \leftarrow \theta - \eta v$), that is $g_t = \nabla\mathcal{L}(\theta_t - \eta\beta v_{t-1})$. This look-ahead acts as an anticipatory predictive brake, damping oscillation when approaching high-curvature ravine walls.

Pause & Predict: The Hidden Memory Cost of AdamW

If a transformer model contains 1 billion parameters stored in float32 (4 GB of DRAM), how much memory is required strictly for optimizer state during AdamW

training? (Hint: compute first and second moment buffers.) What fraction of the total parameter plus optimizer memory does the optimizer consume?

7.2.3 Adam: a step size for every parameter

As the workhorse optimizer powering deep architectures from recurrent networks to modern transformers, Adam (Adaptive Moment Estimation) addresses the severe ill-conditioning of non-convex loss landscapes. By maintaining independent running averages of both past gradients and squared gradients, it automatically computes an individual adaptive step size for every parameter coordinate. Consider a two-element gradient $[100, 1]$. Plain SGD at learning rate 0.1 would move the coordinates by $[10, 0.1]$. Adam's first bias-corrected estimates are $\hat{m} = [100, 1]$ and $\hat{v} = [10000, 1]$, so division by $\sqrt{\hat{v}}$ gives $[1, 1]$. Ignoring the small numerical-stability term, both coordinates move by 0.1. Adam obtains these scales from two running averages, one of the gradient and one of its square:

$$m \leftarrow \beta_1 m + (1 - \beta_1)\, g, \qquad v \leftarrow \beta_2 v + (1 - \beta_2)\, g^2.$$

The first, m, tracks the direction. The second, v, tracks the scale. Both are averages, so β_1 and β_2 are the fractions of history retained, with defaults 0.9 and 0.999. The squaring in v is elementwise, so each parameter gets its own scale.

Both averages start at zero, which biases them low at first. At the very first step, $m = 0.1\, g$, a tenth of the true gradient. Adam corrects this by dividing by $1 - \beta^t$, where t counts updates of this parameter from one:[29]

$$\hat{m} = \frac{m}{1 - \beta_1^t}, \qquad \hat{v} = \frac{v}{1 - \beta_2^t}.$$

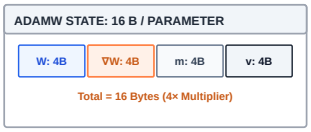
At $t = 1$ the denominator for m is $1 - 0.9 = 0.1$, which restores the full gradient. As t grows the denominators approach one and the correction fades. The update is then

$$\theta \leftarrow \theta - \eta\, \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon},$$

with ϵ a tiny constant, 10^{-8} by default, that prevents division by zero when the squared-gradient estimate is zero. A missing gradient, `None`, skips the update altogether; a present array of zeros still runs the update.

The thing to see in that formula is the ratio. For a parameter whose gradient is steady, $\hat{m} \approx g$ and $\sqrt{\hat{v}} \approx |g|$, so the ratio is about ± 1 and the parameter moves by about η per step regardless of how large its gradient is. On the ravine, this normalization makes the first movements of x and y nearly equal despite their different gradient magnitudes. Later movements depend on the histories stored in both averages. The price is two extra numbers per parameter, m and v, kept alive between steps.

[29] **Derivation of bias correction**: Unrolling the recurrence $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ starting from $v_0 = 0$ yields $v_t = (1 - \beta_2) \sum_{i=1}^{t} \beta_2^{t-i} g_i^2$. Taking expectations under the assumption that true second moments $\mathbb{E}[g_i^2]$ are stationary yields $\mathbb{E}[v_t] = \mathbb{E}[g^2](1 - \beta_2) \sum_{i=1}^{t} \beta_2^{t-i} = \mathbb{E}[g^2](1 - \beta_2^t)$. Dividing by $1 - \beta_2^t$ produces an exact unbiased estimator $\mathbb{E}[\hat{v}_t] = \mathbb{E}[g^2]$ even at step $t = 1$.



AdamW quadruples parameter memory footprint: 16 bytes per scalar weight in FP32.

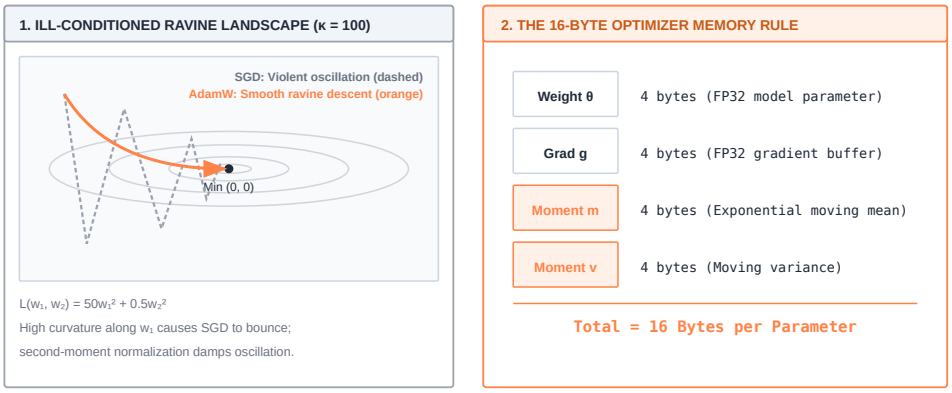


Figure 7.1: **Optimization trajectories in an ill-conditioned ravine**: On an anisotropic loss surface ($100x^2 + y^2$), plain SGD oscillates violently across the steep walls while making negligible progress along the gentle valley floor. AdamW computes coordinate-wise adaptive step sizes from first and second gradient moments, driving directly toward the global minimum. The right panel counts the 16 bytes per parameter that AdamW keeps in float32: weight, gradient, and two moments.

7.2.4 Weight decay and adaptive scaling

In production training pipelines, weight decay serves as a crucial regularizer that prevents parameter values from growing uncontrollably and overfitting training data. However, the classical approach of folding an L_2 penalty directly into the gradient interacts poorly with adaptive optimizers, where second-moment normalization distorts the intended shrinkage rate. With weight 1.0, learning rate 0.1, and decay coefficient 0.01, a shrinkage step gives 0.999. **Weight decay** uses this small movement toward zero to discourage large weights. Its rule is

$$\theta \leftarrow (1 - \eta\lambda)\,\theta,$$

so a weight stays large only if the gradient keeps pushing it there. The coefficient λ defaults to 0.01 in TinyTorch's AdamW. For plain gradient descent this shrink is identical to adding $\lambda\theta$ to the gradient, because $\eta(g + \lambda\theta) = \eta g + \eta\lambda\theta$. The added term is the gradient of $\frac{\lambda}{2}\|\theta\|^2$, so this form is called an **L2 penalty**, and it is the form TinyTorch's SGD uses. The equivalence to an independent shrinkage step holds for plain SGD; with momentum, the L2 term also enters the velocity buffer.

Under Adam they are not the same. Adding $\lambda\theta$ to the gradient changes both running averages, then the result passes through the adaptive division. The amount of shrinkage can no longer be read as $\eta\lambda\theta$ independently of gradient history. AdamW keeps the L2 term out of m and v and applies the shrink to the old weight before subtracting the adaptive update:

$$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\,\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}.$$

[30] In standard Adam with L2 regularization, the weight penalty $\lambda\theta$ is added directly to g_t, entering the second moment accumulator v_t. Coordinates with large gradients receive suppressed decay, while coordinates with infrequent or small gradients receive excessively strong decay. Loshchilov & Hutter (2019) showed that decoupling weight decay from the gradient moments restores the intended shrinkage $\theta \leftarrow (1 - \eta\lambda)\theta$ and improves Adam's generalization on image-classification benchmarks.

For the weight above, an adaptive update of 0.1 therefore gives $0.999 - 0.1 = 0.899$. Reversing the operations would give $(1 - 0.1) \times 0.999 = 0.8991$, a different algorithm. The update paths in figure 7.2 make the placement of decay explicit.

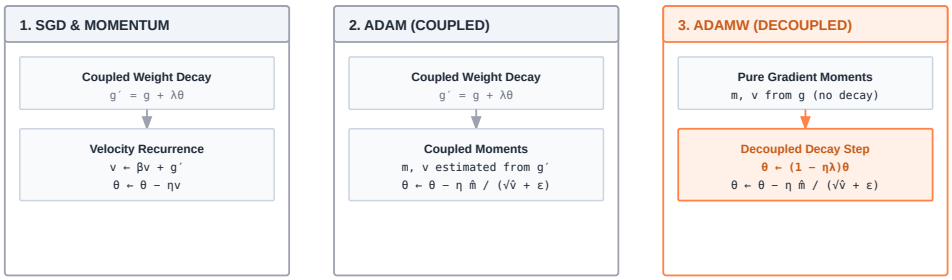


Figure 7.2: **Where decay enters the update**: SGD and Adam add decay to the gradient before updating their history. AdamW forms moments from the unmodified gradient, shrinks the old weight, and then subtracts the adaptive update.

7.2.5 What a learning rate means

Selecting and tuning the learning rate η is the single most sensitive hyperparameter decision in deep learning systems, directly governing training stability and convergence speed. Because SGD and adaptive optimizers apply the learning rate to fundamentally different mathematical quantities—unnormalized gradient vectors vs. normalized, scale-invariant direction vectors—the effective meaning of η changes between optimizer families. The two families give η different units, which is why their customary values differ by about a factor of ten. For SGD the step is ηg, so a safe rate depends on the curvature of the loss. The ravine above diverged along its steep axis for any η above $2/100$; a loss with ten times that curvature would need a rate ten times smaller. There is no universal SGD rate, only a rate for a given loss. For Adam the normalized step $\hat{m}/(\sqrt{\hat{v}} + \epsilon)$ has magnitude close to one when a coordinate's gradients agree in sign and less when they do not, so η approximately bounds how far any parameter moves in one step, whatever the scale of its gradient (Kingma and Ba, 2015). That is why an Adam rate carries between models more readily than an SGD rate. TinyTorch's defaults follow the usual convention, 0.01 for SGD and 0.001 for Adam and AdamW, and the course milestones use those values: SGD at 0.01 for the digit networks of Synthesis I and Synthesis II, and Adam at 0.001 for the attention routing task; the TinyGPT runs raise AdamW to between 0.002 and 0.008.

The batch size moves the rate as well. Averaging B independent per-example gradients divides the variance of the estimate by B, so a larger batch gives a more reliable direction and can support a larger step. Goyal and colleagues (2017) turned this into a rule for SGD: when the batch grows by a factor of k, multiply the learning rate by k, and ramp the rate up over the first few epochs. With it they trained ResNet-50 on ImageNet at batches of up to 8,192 images without losing accuracy, and accuracy fell at larger batches. The rule is an empirical regularity for a family of models, not a law, and it is the reason a learning rate reported without its batch size is incomplete.

7.3 The Code

7.3.1 The base class, and why `zero_grad()` sets `None`

An optimizer separates the update rule from the loop that obtains gradients. The loop clears old gradients with `zero_grad()`, computes the next loss, calls `backward()`, and then calls `step()`. It can change optimizer classes without changing this sequence. The base class owns a list of parameter references and supplies gradient clearing; each subclass supplies the update rule.

Because `backward()` accumulates gradients via `+=`, omitting `optimizer.zero_grad()` before backward causes gradients to sum continuously across consecutive mini-batches. Step 1 computes ∇L_1; step 2 updates with $\nabla L_1 + \nabla L_2$; step k updates with $\sum_{i=1}^{k} \nabla L_i$. The update drifts away from the current batch's gradient and can oscillate or diverge, without raising any runtime error.

Clearing must remove the previous objective's gradient without removing the optimizer's history. The `backward()` traversal from Chapter 6 accumulates into `.grad`, so a second batch otherwise adds to the previous batch's gradient. Assigning `None` releases the parameter's reference to its gradient array instead of filling it with zeros.[31] TinyTorch's backward traversal allocates a zero buffer when it first reaches the parameter, then adds the incoming contribution. The avoided work is the explicit clearing pass; the next backward pass still initializes storage.

The constructor copies the supplied parameter list, retains the original tensors, and rejects duplicates so the same tensor cannot be updated twice in one step. It preserves existing parameter gradient state without clearing any gradients already present. `zero_grad()` clears the gradients. `step()` is left for each subclass to define.

```
>_ SOURCE CODE MAPPING                    MOD 07

FILE:     src/07_optimizers/07_optimizers.py
EXPORT:   tinytorch.core.optimizers
SYMBOLS:  SGD, Adam, AdamW

Extracted & verified at build time        ✓ tested
```

Source Code Mapping: Module 07 extracts SGD with momentum, Adam moment trackers, and AdamW decoupled weight decay directly from the working repository.

[31] In PyTorch, calling `zero_grad` with `set_to_none=True` frees gradient memory buffers rather than filling them with zeros. This eliminates memory bandwidth overhead by skipping zero-fill memory writes; subsequent backward operations allocate and write gradients without reading previous zeros across the memory bus.

```python
class Optimizer:
    """
    Base class for all optimizers.
    """

    def __init__(self, params: List[Tensor]):
        """
        Initialize optimizer with parameters to optimize.
        """
        # Own the list so later caller edits cannot misalign parameters and state.
        self.params = list(params)
        if len({id(param) for param in self.params}) != len(self.params):
            raise ValueError("Optimizer parameters must not contain duplicates")

        # Parameters track gradients through layer definitions. Do NOT reset param.grad:
        # a caller may have run backward() before building the optimizer.
        for param in self.params:
            if not hasattr(param, 'grad'):
                param.grad = None
```

```
        self.step_count = 0  # For algorithms that need step counting

    def zero_grad(self):
        """
        Clear gradients from all parameters.
        """
        for param in self.params:
            param.grad = None

    def step(self):
        """
        Update parameters based on gradients.
        """
        raise NotImplementedError(
            f"Abstract method step() not implemented\n"
            f"  ❌ {self.__class__.__name__} inherits from Optimizer but doesn't define step()\n"
            f"  💡 Each optimizer must implement its own update rule (SGD, Adam, etc.)\n"
            f"  🔧 Override step() in your optimizer subclass:\n"
            f"      def step(self):\n"
            f"          for param in self.params:\n"
            f"              if param.grad is not None:\n"
            f"                  param.data -= self.lr * self._extract_gradient(param)"
        )
```

Parameters must have `requires_grad=True` before the forward pass that will be differentiated. TinyTorch layer parameters (such as `Linear.weight` and `Linear.bias`) enable this flag at construction, while arbitrary tensors default to `requires_grad=False`. The optimizer does not mutate `requires_grad`; it simply stores references to the tensors passed to it. `Function.apply` inspects `requires_grad` during the forward pass to build the autograd graph. If a tensor was evaluated with `requires_grad=False`, running `backward()` produces no gradient for it, and the optimizer step leaves its data untouched. The optimizer also does not discover parameters by traversing the computation graph: a tensor omitted from `params` will never be updated, even if autograd computed its gradient. Listing 7.2 demonstrates how `requires_grad` controls gradient generation and optimizer updates.

Listing 7.2 Enable recording before forward: Constructing an optimizer cannot recover an earlier unrecorded graph.

```
from tinytorch.core.tensor import Tensor
from tinytorch.core.optimizers import SGD
import tinytorch.core.autograd

# Case 1: Tensor without requires_grad produces no gradient
w_frozen = Tensor([1.0], requires_grad=False)
opt_frozen = SGD([w_frozen], lr=0.1)
loss_frozen = (w_frozen * w_frozen).sum()
loss_frozen.backward()
print(w_frozen.grad)                # None: unrecorded graph
opt_frozen.step()
print(w_frozen.data)                # [1.]: parameter untouched

# Case 2: Trainable parameter records graph and receives update
w = Tensor([1.0], requires_grad=True)
opt = SGD([w], lr=0.1)
loss = (w * w).sum()
loss.backward()
print(w.grad)                       # [2.]
opt.step()
print(w.data)                       # [0.8]: updated by lr * grad
```

The final assignment inside `step()` changes the array on `w`, while the model and optimizer still refer to the same `Tensor` object. Constructing a new parameter tensor and leaving the optimizer's list unchanged would break that connection. Parameter identity, gradient lifetime, and optimizer state together determine whether the next forward pass observes the update.

The optimizer keeps a global `step_count` for calls to `step()`, while Adam and AdamW keep `update_counts` beside their moment buffers. Each parameter ages only when it has a gradient and its moments are updated. A parameter skipped with `grad=None` therefore uses the right bias correction when it next participates. These counters survive gradient clearing along with the moments. One helper is attached to the base class from its own cell. The backward traversal leaves gradients as arrays, while manually assigned gradients may be tensors, so the helper accepts either representation.

```
@method_of(Optimizer)
def _extract_gradient(self, param: Tensor) -> np.ndarray:
    """
    Extract gradient data as a NumPy array from a parameter.
    """
    grad = param.grad
    if isinstance(grad, Tensor):
        return grad.data
    else:
        return grad
```

A zero gradient and a missing gradient have different meanings. All three optimizers skip `None`. A zero array still permits stored momentum to move the weight, and AdamW still applies decay. Clearing gradients preserves the buffers and their ages; it does not reset the optimizer.

7.3.2 SGD with momentum

SGD's loop adds one array to the state we already know. For a parameter tensor of shape $(2, 3)$, its gradient and optional momentum buffer also have shape $(2, 3)$. The scalar learning rate and momentum coefficient apply elementwise across those arrays. The list index associates each parameter with its own history.

The constructor checks the learning rate, momentum coefficient, and decay coefficient before initializing parameter buffers. Invalid negative or nonfinite values raise an error before any parameter is changed. The buffer list starts with `None` entries; a tensor receives a velocity array only when it first has a gradient.

```
class SGD(Optimizer):
    """
    Stochastic Gradient Descent with momentum.
    """

    def __init__(self, params: List[Tensor], lr: float =
    ↪   DEFAULT_LEARNING_RATE_SGD, momentum: float = 0.0, weight_decay:
    ↪   float = 0.0):
        """
        Initialize SGD optimizer.
        """
        for name, value in (("lr", lr), ("momentum", momentum),
        ↪   ("weight_decay", weight_decay)):
            if not np.isfinite(value) or value < 0:
                raise ValueError(f"{name} must be finite and
                ↪   nonnegative")
        super().__init__(params)

        self.lr = lr
        self.momentum = momentum
        self.weight_decay = weight_decay

        # Initialize momentum buffers (created lazily)
        self.momentum_buffers = [None for _ in self.params]

    def has_momentum(self) -> bool:
        """
        Check if this optimizer uses momentum.
        """
        ...

    def get_momentum_state(self) -> Optional[List]:
        """
        Get momentum buffers for checkpointing.
        """
        ...
```

```
33
34      def set_momentum_state(self, state: Optional[List]) -> None:
35          """
36          Restore momentum buffers from checkpointing.
37          """
38          ...
39
40      def step(self):
41          """
42          Perform SGD update step with momentum.
43          """
44          for i, param in enumerate(self.params):
45              if param.grad is None:
46                  continue
47
48              # Extract gradient using shared helper
49              grad_data = self._extract_gradient(param)
50
51              # Apply weight decay
52              if self.weight_decay != 0:
53                  grad_data = grad_data + self.weight_decay * param.data
54
55              # Update momentum buffer
56              if self.momentum != 0:
57                  if self.momentum_buffers[i] is None:
58                      # Initialize momentum buffer
59                      self.momentum_buffers[i] =
                        ↪   np.zeros_like(param.data)
60
61                  # Update momentum: v = momentum * v_prev + grad
62                  self.momentum_buffers[i] = self.momentum *
                    ↪   self.momentum_buffers[i] + grad_data
63                  grad_data = self.momentum_buffers[i]
64
65              # Update parameter: param = param - lr * grad. Cast back
                ↪   to the
66              # parameter's dtype so a float64 lr or gradient cannot
                ↪   silently
67              # promote float32 weights (and double their memory).
68              param.data = (param.data - self.lr *
    ↪   grad_data).astype(param.data.dtype, copy=False)
69
70          # Increment step counter
71          self.step_count += 1
```

The order inside the loop is the order of The Idea. Weight decay folds into the gradient first (the coupled L2 form), momentum accumulates second, and the parameter moves last. The momentum buffers are allocated on the first step rather than in the constructor, so a parameter that never receives a gradient needs no velocity array. Once allocated, a buffer persists across steps, which is the whole point. It is state that outlives a batch, and the three elided helpers exist so that the training engine's checkpoints can save and restore it. A parameter with `grad=None` is skipped. This describes an unused parameter only

if the caller cleared stale gradients before the batch. Whole-array NumPy expressions perform the arithmetic and may allocate intermediate arrays. The final assignment casts the result back to the parameter's dtype and replaces `param.data`, preserving the Tensor object that the model owns. Without the cast, a float64 learning rate or gradient would silently promote float32 weights to float64 and double their storage.

Run SGD on the ravine from $(1, 1)$ with $\eta = 0.019$ and `momentum=0.9`, supplying $(100x, y)$ as a float32 gradient on each step. After 100 steps, $(x, y) \approx (0.005154, 0.005234)$; plain SGD at the same rate gives $(0.000027, 0.146859)$. Momentum has reduced the slowly changing y coordinate much further, although its x coordinate is farther from zero at that particular step. For this quadratic, eliminating velocity gives $x_{t+1} = (1 + \beta - 100\eta)x_t - \beta x_{t-1}$. Its stability condition is $0 < 100\eta < 2(1 + \beta)$, so with $\beta = 0.9$ the upper boundary is 0.038. At that boundary the coefficient is -1.9, and substituting $x_t = (-1)^t$ satisfies the recurrence. An alternating component can therefore persist without decay; this is not a convergent run.

7.3.3 AdamW

The standard optimizer for modern foundation models—including GPT, LLaMA, and Vision Transformers—is **AdamW** (Loshchilov & Hutter, 2019). By applying weight decay directly to parameter buffers rather than blending it into gradient moment accumulators, AdamW restores true scale-invariant regularization across all weights. AdamW replaces SGD's one velocity buffer with a pair of arrays per parameter tensor. A $(2, 3)$ weight therefore has $(2, 3)$ first- and second-moment arrays, but only one integer update count: all entries of that tensor receive a gradient together. The elementwise arithmetic supplies a different scale for each scalar weight without requiring a separate learning-rate setting for each one.

On the same ravine, start AdamW at $(1, 1)$ with $\eta = 0.1$ and `weight_decay=0.0` to isolate adaptive scaling. After 100 steps, both coordinates are approximately 0.002937. This is a reproducible result for this loss and these settings, not evidence that AdamW permits an arbitrary learning rate. The gradient history and the numerical-stability term still affect the trajectory.

With nonzero decay, `step()` must keep the two paths separate. It extracts the gradient, calls `_update_moments`, shrinks the old parameter, and subtracts the adaptive update. The constructor creates empty buffer slots and zero update counts; the helper fills the slots lazily and advances their ages only for tensors that participate.

```
class AdamW(Optimizer):
    """
    AdamW optimizer with decoupled weight decay.
    """

    def __init__(self, params: List[Tensor], lr: float =
    ↪   DEFAULT_LEARNING_RATE_ADAM, betas: tuple = (DEFAULT_BETA1,
    ↪   DEFAULT_BETA2), eps: float = DEFAULT_EPS, weight_decay: float =
    ↪   DEFAULT_WEIGHT_DECAY_ADAMW):
        """
        Initialize AdamW optimizer.
        """
```

```
        for name, value in (("lr", lr), ("eps", eps), ("weight_decay",
        ↪   weight_decay)):
            if not np.isfinite(value) or value < 0:
                raise ValueError(f"{name} must be finite and
                ↪   nonnegative")
        if len(betas) != 2 or any(not np.isfinite(beta) or not 0 <=
        ↪   beta < 1 for beta in betas):
            raise ValueError("betas must contain two finite values in
            ↪   [0, 1)")
        super().__init__(params)

        self.lr = lr
        self.beta1, self.beta2 = betas
        self.eps = eps
        self.weight_decay = weight_decay

        # Initialize moment buffers (same as Adam)
        self.m_buffers = [None for _ in self.params]
        self.update_counts = [0 for _ in self.params]  # Age of each
        ↪   parameter's moments
        self.v_buffers = [None for _ in self.params]
```

```
@method_of(AdamW)
def _update_moments(self, i: int, grad_data: np.ndarray) -> tuple:
    """
    Update first and second moment estimates with bias correction for
    ↪   AdamW.
    """
    # Initialize buffers if needed
    if self.m_buffers[i] is None:
        self.m_buffers[i] = np.zeros_like(grad_data)
        self.v_buffers[i] = np.zeros_like(grad_data)

    # Update biased first moment estimate
    self.m_buffers[i] = self.beta1 * self.m_buffers[i] + (1 -
    ↪   self.beta1) * grad_data

    # Update biased second moment estimate
    self.v_buffers[i] = self.beta2 * self.v_buffers[i] + (1 -
    ↪   self.beta2) * (grad_data ** 2)

    # A parameter with grad=None skipped this update. Its moments are
    ↪   younger
    # than the optimizer's global step count, so correct using their
    ↪   own age.
    self.update_counts[i] += 1
    bias_correction1 = 1 - self.beta1 ** self.update_counts[i]
    bias_correction2 = 1 - self.beta2 ** self.update_counts[i]

    # Compute bias-corrected moments
    m_hat = self.m_buffers[i] / bias_correction1
```

```
    v_hat = self.v_buffers[i] / bias_correction2

    return m_hat, v_hat
```

```
@method_of(AdamW)
def step(self):
    """
    Perform AdamW update step by composing helpers with decoupled weight decay.
    """
    # Increment step counter first
    self.step_count += 1

    for i, param in enumerate(self.params):
        if param.grad is None:
            continue

        # Extract gradient using shared helper
        grad_data = self._extract_gradient(param)

        # Update moments using PURE gradients (no weight decay mixed in)
        m_hat, v_hat = self._update_moments(i, grad_data)

        # Remember the dtype so float32 weights come back out as float32.
        dtype = param.data.dtype

        # Decay the old weight, not the adaptive update we are about to add.
        if self.weight_decay != 0:
            param.data = param.data * (1 - self.lr * self.weight_decay)

        # Apply gradient-based update independently of decay.
        param.data = param.data - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
        param.data = param.data.astype(dtype, copy=False)
```

The call chain preserves three useful invariants. The per-parameter update count advances before bias correction, which divides by $1 - \beta^t$ and would divide by zero at $t = 0$. The gradient handed to `_update_moments` is the raw gradient, with no weight decay mixed in, so m and v never see the L2 term. The decay is a separate multiply on `param.data` before the adaptive step, exactly the decoupled formula, and the result is cast back to the dtype the parameter had on entry. The bias-corrected values are computed fresh each step and discarded; only the raw `m_buffers` and `v_buffers` are stored, along with their update counts.

7.3.4 The original Adam, for contrast

TinyTorch's `Adam` uses the same moment equations, but it adds $\lambda\theta$ before calling the helper. To isolate the role of the denominator, first hold the normalization fixed in listing 7.3.

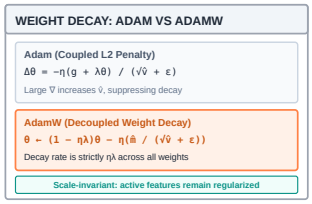
Listing 7.3 Fixed-normalization comparison: This arithmetic illustration holds the denominator fixed.

```
import math

theta, lr, wd = 10.0, 0.001, 0.01
m_hat, v_hat = 1.0, 100.0          # a weight whose gradients have been large

# Adam with L2: the decay term rides inside the normalized step
decay_adam = lr * (wd * theta) / math.sqrt(v_hat)
# AdamW: the decay term is applied to the weight directly
decay_adamw = lr * wd * theta

print(f"shrinkage from decay, Adam+L2: {decay_adam:.6f}")
print(f"shrinkage from decay, AdamW:   {decay_adamw:.6f}")
```



Decoupled Weight Decay: Adam couples decay to gradient variance, suppressing shrinkage on active weights, whereas AdamW applies uniform parameter decay.

Holding the normalization fixed in this arithmetic example gives a decay contribution of 0.00001 through the normalized path and 0.0001 through direct shrinkage. This is an illustration of the scaling, not a simulation of an Adam step. In the actual coupled update the decay term also changes both running moments, so its cumulative effect cannot be isolated by this division alone. Here is Module 07's `Adam.step`, the method that does it. It folds the decay into the gradient before the moments, and it has no separate shrink step.

```
@method_of(Adam)
def step(self):
    """
    Perform Adam update step by composing helpers.
    """
    # Count optimizer calls; each parameter separately counts its moment updates.
    self.step_count += 1

    for i, param in enumerate(self.params):
        if param.grad is None:
            continue

        # Extract gradient using shared helper
        grad_data = self._extract_gradient(param)

        # Apply weight decay to a copy, so param.grad is never mutated
        if self.weight_decay != 0:
            grad_data = grad_data + self.weight_decay * param.data

```

```
        # Update moments and get bias-corrected estimates
        m_hat, v_hat = self._update_moments(i, grad_data)

        # Update parameter, keeping its dtype (float32 stays float32)
        update = self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
        param.data = (param.data - update).astype(param.data.dtype,
    copy=False)
```

With `weight_decay=0.0` the two optimizers are identical, and their `_update_moments` methods are the same code. With nonzero decay, `Adam` feeds $\lambda\theta$ into both moment updates, whereas `AdamW` keeps the moment estimates independent of decay. Module 07 keeps `Adam` and `AdamW` as two separate classes with the moment code repeated rather than a subclass pair, so that a student building either one sees the whole rule in one place.

7.4 A Worked Trace

Take a single weight, $\theta_0 = 1.0$, that receives the same gradient $g = 0.5$ on two consecutive steps. Use $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.1$, $\lambda = 0.01$, and $\epsilon = 10^{-8}$. Every number below can be checked by hand.

At the first step, $m_1 = 0.1 \times 0.5 = 0.05$ and $v_1 = 0.001 \times 0.5^2 = 0.00025$. At the second, $m_2 = 0.9 \times 0.05 + 0.05 = 0.095$ and $v_2 = 0.999 \times 0.00025 + 0.00025 = 0.00049975$. Dividing by $1 - \beta_1^t$ and $1 - \beta_2^t$ gives $\hat{m} = 0.5$ and $\hat{v} = 0.25$ on both steps. The adaptive subtraction is approximately 0.1; the small ϵ prevents it from being exactly 0.1 in real arithmetic. Table 7.1 follows the state in execution order.

Table 7.1: **Two AdamW updates**: Decay multiplies the old weight by 0.999 before the adaptive subtraction. Displayed values are rounded.

Update t	Stored m_t	Stored v_t	Old weight	After decay	After update
1	0.05000	0.00025000	1.000000	0.999000	0.899000
2	0.09500	0.00049975	0.899000	0.898101	0.798101

Listing 7.4 executes the same updates through Module 07's `AdamW`, with the gradient set by hand to stand in for `backward()`. The one-element weight, gradient, and moment arrays all have shape `(1,)`.

Listing 7.4 Execute the AdamW trace: The printed weight includes decay before the adaptive subtraction.

```
1  from tinytorch.core.tensor import Tensor
2  from tinytorch.core.optimizers import AdamW
3
4  w = Tensor([1.0])
5  opt = AdamW([w], lr=0.1, betas=(0.9, 0.999),
6              eps=1e-8, weight_decay=0.01)
7  for t in (1, 2):
8      w.grad = Tensor([0.5])              # manually wrapped; backward()
   ↪  stores an array
9      opt.step()
10     m, v = opt.m_buffers[0][0], opt.v_buffers[0][0]
11     print(f"t={t}: m={m:.5f} v={v:.8f} theta={w.data[0]:.4f}")
12     opt.zero_grad()
```

In listing 7.4, line 4 initializes weight `w = Tensor([1.0])`, and lines 5 to 6 configure `AdamW` with learning rate $\eta = 0.1$, $\beta = (0.9, 0.999)$, and weight decay $\lambda = 0.01$. The loop in lines 7 to 12 executes two consecutive update cycles. In line 8, a synthetic gradient `w.grad = Tensor([0.5])` is assigned. Line 9 executes `opt.step()`, which updates the first and second moment estimates, applies weight decay (0.999θ), and adjusts the parameter. Line 10 extracts the internal moment values `m` and `v`, and line 11 prints the state (`t=1: m=0.05000 v=0.00025000 theta=0.8990` and `t=2: m=0.09500 v=0.00049975 theta=0.7981`). Finally, line 12 calls `opt.zero_grad()` to wipe `w.grad` before the next iteration.

After either call, the weight has changed, the two raw moment arrays and their age remain available for the next call, and the assigned gradient is still present until `zero_grad()` clears it in line 12. The normalized update and the decayed intermediate weight are temporary values. The model continues to hold the same `Tensor`; its next forward pass reads the new array.

A checkpoint must preserve the age together with the moments. Suppose another parameter is unused for the first three calls and receives its first gradient on call four. Its global optimizer count is four, but its moment age is one, so it divides by $1 - \beta$, not $1 - \beta^4$. `get_momentum_state()` copies only the buffers; it is not a complete checkpoint. The Trainer introduced in Chapter 8 will also save `update_counts`, `step_count`, and hyperparameters. Restoring those fields together makes the next update match an uninterrupted run. The buffer restore helpers check all shapes before replacing any state, preventing a mismatched array from broadcasting into a different parameter shape.

7.5 From TinyTorch to Production

A pair of moments and an update count is enough to recognize the same algorithm in a production framework. PyTorch's AdamW documentation describes decoupled decay and the same default learning rate, moment coefficients, and epsilon as TinyTorch. Its SGD documentation uses the same velocity recurrence when dampening and Nesterov momentum are disabled. PyTorch's Adam defaults to coupled decay but also exposes a

decoupled option; the class name alone is therefore insufficient to identify a production configuration's rule.

Gradient clearing carries the same semantic distinction. PyTorch's `zero_grad()`, with `set_to_none=True`, leaves unused parameters with no gradient, allowing their updates to be skipped. A zero gradient tensor still enters the optimizer's update path. TinyTorch makes that distinction visible in the first `if` of each loop.

Memory provides a reason to change execution while retaining the equations. Count one float32 weight, its gradient, and two float32 moments: $4 + 4 + 4 + 4 = 16$ bytes per scalar weight. The moments alone account for 8 bytes. This is an array-storage model for a point in training when all gradients and moments exist, not a universal training-memory requirement. Activations, temporary expressions, counters, and Python objects add storage; clearing gradients removes one of the counted arrays until backward recreates it.

Problem: For a 7-billion parameter language model ($N = 7 \times 10^9$) trained with AdamW in FP32, what is the static memory required for model parameters, gradients, and optimizer state?

Per-parameter memory ledger:

$$\begin{aligned} \text{Weights } (\theta) &: 1 \times 4 \text{ bytes} = 4 \text{ bytes} \\ \text{Gradients } (g) &: 1 \times 4 \text{ bytes} = 4 \text{ bytes} \\ \text{First moment } (m) &: 1 \times 4 \text{ bytes} = 4 \text{ bytes} \\ \text{Second moment } (v) &: 1 \times 4 \text{ bytes} = 4 \text{ bytes} \\ \hline \textbf{Total static footprint} &: \textbf{16} \text{ bytes per parameter} \end{aligned}$$

Total model footprint:

$$\text{Memory} = 7 \times 10^9 \times 16 \text{ bytes} = 112 \text{ GB}$$

Systems insight: Weights and gradients require $28\,\text{GB} + 28\,\text{GB} = 56\,\text{GB}$, but AdamW's momentum buffers double that requirement to 112 GB. A single 80 GB NVIDIA A100 cannot even hold the static training state of a 7B model without optimizer offloading or ZeRO state sharding. Production systems compress these moment states further; 8-bit optimizers (Dettmers et al., 2022) store them in about 2 bytes per parameter using block-wise quantization, a technique outside TinyTorch. In Chapter 23, we implement parameter-efficient fine-tuning via LoRA, which freezes the original weight matrices and trains low-rank adapter matrices, eliminating AdamW state for over 98% of the model parameters.

NumPy evaluates the update as several array expressions, with temporary arrays between operations. Production implementations can combine work across tensors or fuse several operations into one device program, reducing intermediate storage and launch overhead. PyTorch exposes `foreach` and `fused` implementations in its AdamW interface. Their performance depends on the workload and device; neither changes the intended update equation. Chapter 18 will examine how execution choices affect repeated array operations.

A per-parameter ("for-loop") optimizer implementation launches several small kernels for each parameter tensor (decay, two moment updates, square root, division, update). For a 70-layer model with hundreds of weight matrices, launching thousands of small kernels saturates the CPU-GPU command queue.

Production systems use `torch.optim.AdamW(..., fused=True)`, which compiles the entire weight update into a single fused GPU kernel per parameter group. This cuts intermediate memory traffic and the number of kernel launches; the size of the saving depends on the model and device.

The mapping in table 7.2 separates the mathematical state from the implementation that holds it.

Table 7.2: **Optimizer correspondence**: The same state and missing-gradient contract appear behind different storage and execution choices.

Contract	TinyTorch	PyTorch
Moment state	`m_buffers`, `v_buffers`	`exp_avg`, `exp_avg_sq`
Moment age	per-tensor `update_counts`	per-tensor `step`
Array storage	NumPy arrays in host memory	tensors on CPU or an accelerator
Missing gradient	skip the parameter	skip the parameter
Learning rate	one mutable `lr` attribute	settings for parameter groups

When comparing configurations, record where decay enters the rule and which arrays the memory estimate includes. In AdamW, the shrink factor is $1 - \eta\lambda$, so changing the learning rate also changes the shrinkage applied per update. That consequence follows directly from the multiplication in `step()`.

7.6 Summary

An optimizer reads each parameter's gradient and changes the array held by that same Tensor. SGD may retain a velocity; Adam and AdamW retain two moments and an update count for each parameter tensor. Adam couples decay to those moments, while AdamW shrinks the old weight separately before the adaptive subtraction. `grad=None` skips the entire parameter update, whereas `zero_grad()` clears gradients without clearing history. These are the contracts the training loop can rely on. Chapter 8 will arrange forward computation, gradient clearing, backward, and stepping, then preserve the optimizer's history when a run is saved and resumed.

7.7 Further Reading

- **Kingma and Ba** (2015). "Adam: a method for stochastic optimization." *ICLR*. The moment estimates, the bias correction, and the bound on the effective step used above.
- **Loshchilov and Hutter** (2019). "Decoupled weight decay regularization." *ICLR*. Why decay folded into Adam's gradient is not the same as decay applied to the weights, the difference AdamW makes.
- **Sutskever et al.** (2013). "On the importance of initialization and momentum in deep learning." *ICML*. Momentum's role in training deep networks, with the velocity form this chapter implements.

- **Goyal et al.** (2017). "Accurate, large minibatch SGD: training ImageNet in 1 hour." *arXiv*. The linear scaling rule and warmup discussed with learning rates above.

🔥 Chapter 8

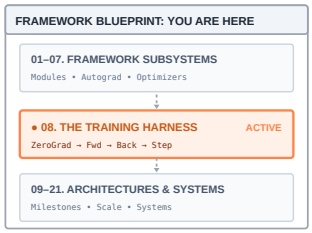
The Training Engine: The Five-Step Loop

Supervised machine learning is the practical execution of empirical risk minimization: repeatedly exposing a parameterized model to training data, evaluating loss, computing gradients, and steering weights toward optimal generalization. However, learning is not a loose script; it is a precisely choreographed state machine where data pipelines, forward models, autograd graphs, and optimizers interact in lockstep.

In PyTorch, the fundamental training step is a five-line contract:

```
import torch

for epoch in range(num_epochs):
    for batch_x, batch_y in loader:
        optimizer.zero_grad()                # 1. Clear residual gradients
        outputs = model(batch_x)             # 2. Forward pass
        loss = criterion(outputs, batch_y)   # 3. Evaluate loss
        loss.backward()                      # 4. Backward differentiation
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()                     # 5. Update parameter weights
```



Framework Blueprint: Module 08 enforces the canonical training step cycle and coordinates convergence across epochs.

Underneath that rhythmic sequence, the framework orchestrates complex state transitions and memory lifecycles:

1. **The accumulation invariant:** Because autograd accumulates derivatives into leaf buffers (`param.grad += ...`, the multivariate chain rule invariant established in Chapter 6), `.grad` buffers never clear themselves. Omitting `optimizer.zero_grad()` causes subsequent batch gradients to pile on top of earlier ones, corrupting update trajectories into runaway sums.
2. **Global gradient norm bounding:** Gradient clipping computes the collective L_2 norm across all registered parameter tensors ($\sqrt{\sum \|\mathbf{g}_i\|_2^2}$) and rescales gradients if they exceed a threshold, preventing sudden loss explosions.
3. **Graph detachment and metric memory safety:** To track running training loss without exhausting GPU memory, the loop extracts scalar metrics with `loss.item()`, severing the computation graph reference so intermediate activations can be reclaimed.

This chapter synthesizes the foundational modules of Chapter 1 through Chapter 7 into the unified `Trainer` engine, establishing the end-to-end execution loop of TinyTorch. Before building this engine, we must inspect the silent failures that occur when any link in this five-step chain is broken.

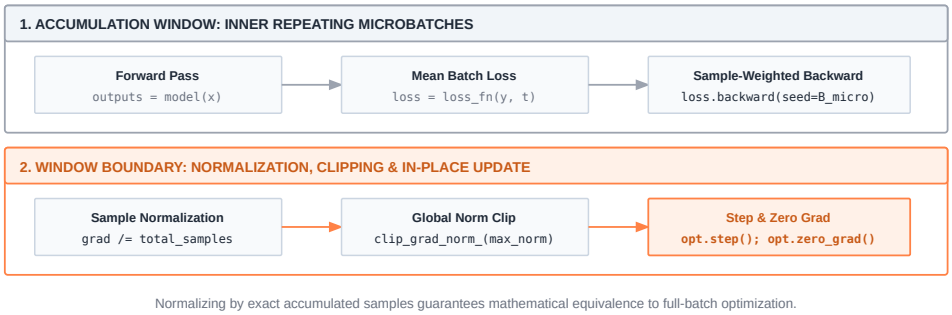


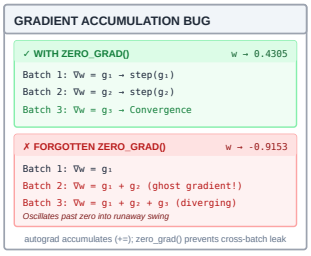
Figure 8.1: **An accumulation window**: Forward, loss, and backward repeat while parameters stay fixed. At the window boundary, the Trainer averages by actual samples, optionally clips, updates parameters, and clears gradients.

8.1 The Problem

↩ **Jump back:** Chapter 7 *In-place optimizer updates and gradient clipping bound parameter trajectories during training.*

Chapter 6 built `backward()` so that it *adds* into `param.grad` rather than overwriting it. That choice was forced. When two branches of a network share a parameter, its gradient is the sum of both contributions, and adding is how the sum happens. The price is that nothing ever clears `.grad` on its own. Without a clear, the second batch's gradient lands on top of the first, the third on top of both, and the optimizer steps along the running total.

No exception is raised. The loss simply stops behaving. The smallest example that shows it is a single weight w with loss $\frac{1}{2}w^2$, whose gradient is w itself, trained with plain gradient descent at learning rate 0.1.



Gradient Contamination: Because autograd accumulates derivatives into leaf buffers, omitting zero_grad causes old gradients to corrupt subsequent updates.

```
step 1   correct w = 0.9000   forgotten w = +0.9000
step 2   correct w = 0.8100   forgotten w = +0.7100
step 3   correct w = 0.7290   forgotten w = +0.4490
step 4   correct w = 0.6561   forgotten w = +0.1431
step 5   correct w = 0.5905   forgotten w = -0.1771
step 6   correct w = 0.5314   forgotten w = -0.4796
step 7   correct w = 0.4783   forgotten w = -0.7341
step 8   correct w = 0.4305   forgotten w = -0.9153
```

The correct column walks steadily toward the minimum at zero. The forgotten column crosses zero between steps four and five, then keeps swinging. For this loss, the accumulated gradient creates an undamped oscillation rather than convergence. Other models can oscillate, diverge, or produce nonfinite values. None of those outcomes is needed to identify the bug: the second update already uses a gradient from two different parameter values.

Calling `optimizer.step()` before `loss.backward()` has the same ownership problem. It uses whatever gradients remain, or skips parameters whose gradients are `None`. Leaving dropout active during evaluation changes the predictions being measured. These

Listing 8.1 Uncleared gradients: The old gradient changes the next update even for one weight.

```
import numpy as np

w_ok, w_bad = 1.0, 1.0
grad_bad = 0.0       # the .grad buffer that never gets cleared
lr = 0.1
for step in range(1, 9):
    # correct loop: zero_grad() ran, so the buffer holds only this
    ↪  step's gradient
    grad_ok = w_ok
    w_ok = w_ok - lr * grad_ok
    # forgotten zero_grad(): backward() adds this step's gradient to
    ↪  the old ones
    grad_bad = grad_bad + w_bad
    w_bad = w_bad - lr * grad_bad
    print(f"step {step}   correct w = {w_ok:.4f}   "
          f"forgotten w = {w_bad:+.4f}")
```

mistakes need not raise an exception, so the Trainer establishes the order and mode at the entry to each pass.

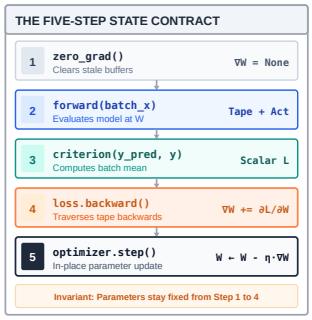
8.2 The Idea

Pause & Predict: The Order of the 5-Step Loop

Why must `optimizer.zero_grad()` strictly precede `loss.backward()`, and why must `loss.backward()` strictly precede `optimizer.step()`? What catastrophic failure mode occurs if `optimizer.step()` is called before `loss.backward()`? What happens if `zero_grad()` is executed after `step()` instead of before `backward()`?

8.2.1 Five steps, and why the order is forced

In production deep learning frameworks like PyTorch, the iterative training loop orchestrates a strict dataflow state machine that synchronizes tensors across memory allocations, autodiff graphs, and optimizer states. Because each phase depends directly on the uncorrupted buffers produced by its immediate predecessor, executing these operations out of sequence introduces catastrophic silent failures such as updating on stale gradients or differentiating through mutated weights. Each step of the loop reads state that the previous step wrote, and that dependency is what pins the order down.



The Five-Step State Contract: Each step consumes the exact memory state produced by its predecessor. Parameters must remain fixed until backward completes.

↪ **Jump ahead:** Synthesis I

Deploying the five-step training engine to train a multi-layer perceptron from scratch.

Table 8.1: **The five-step state contract**: Each operation reads the state produced by its prerequisites.

Step	Call	Reads	Writes
1	`optimizer.zero_grad()`	nothing	every `param.grad` becomes `None`
2	`outputs = model(inputs)`	`param.data`, the batch	activations and the tape (Chapter 6)
3	`loss = loss_fn(outputs, targets)`	`outputs`, `targets`	one scalar, the root of the tape
4	`loss.backward()`	the tape	every `param.grad`, by accumulation
5	`optimizer.step()`	`param.grad`, optimizer buffers	`param.data`, optimizer buffers

The contract is about which computation an update represents. *Every optimizer step must use the intended objective evaluated at one unchanged set of parameter values.* For a single batch, that objective is the batch mean. Later, accumulation combines several batch means with sample-count weights before stepping.[32] Clearing separates those intentional windows; skipping the clear mixes an old objective into the new one. Moving the parameters before backward finishes is another violation because local rules still read operands from the earlier forward pass. The `Trainer` centralizes this ordering so every epoch observes the same boundaries.

[32] When GPU memory constraints prevent allocating a large batch size B in DRAM, gradient accumulation simulates large-batch optimization. The training harness runs k sequential forward and backward micro-steps with batch size $B_{\text{micro}} = B/k$ without calling `zero_grad()` between them. Autograd's accumulation rule naturally sums the gradients $\nabla \mathcal{L} = \sum_{j=1}^{k} \nabla \mathcal{L}_j$. Dividing by k before calling `optimizer.step()` reproduces the exact mathematical update of batch size B.

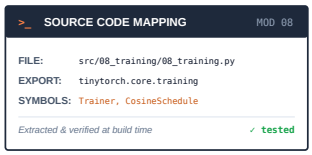
8.2.2 What the loop remembers

Production training frameworks like PyTorch Lightning and Hugging Face Accelerate separate the ephemeral computational graph of an individual mini-batch from the persistent lifecycle state of the broader training run. Maintaining this explicit boundary ensures that transient forward activations are promptly garbage-collected while global step counters, learning rate schedules, and layer execution modes persist intact across epochs. The five steps handle one batch. Around them sit two more loops, one over the batches in a dataset (one pass is an epoch) and one over epochs. The `Trainer` owns the state that outlives a single batch: how many epochs and optimizer steps have completed, the current learning rate, and a history of losses for plotting. It stores `training_mode` and, at the start of each pass, calls `set_training_mode` from Chapter 3, which sets `training` on the model and on every layer stored inside it. A Dropout layer whose `training` argument is omitted falls back to that flag, so it follows the pass whether it sits in a `Sequential` or in a hand-written model. The `_forward` helper also passes the mode explicitly to models whose `forward` signature accepts `training`, such as `Sequential`.

The model and optimizer retain the parameter objects; the optimizer also retains its update buffers. The Trainer holds references to those collaborators, rather than creating a second set of weights. Batch counters and sample sums are local to one `train_epoch` call, while `epoch`, `step`, and `history` remain on the Trainer when that call returns. These different lifetimes explain both accumulation and the checkpoint boundary.

8.3 The Code

The first two pieces stand alone. The Trainer then needs a model with `forward()` and `parameters()`, a loss with `forward()`, an optimizer, and an iterable of `(inputs, targets)` batches. The DataLoader from Chapter 5 supplies that iterable; a Python list of batches works as well. Importing Module 08 enables the backward rules from Chapter 6.



Source Code Mapping: Module 08 extracts the canonical training loop, cosine learning rate schedules, and gradient clipping directly from the working repository.

8.3.1 The schedule

In modern large-scale model training—from foundation transformers to vision backbones—dynamic learning rate schedulers like cosine annealing (`torch.optim.lr_scheduler.CosineAnnealingLR`) are essential for convergence. Rather than holding step sizes fixed, schedulers adjust the optimization rate dynamically to encourage broad exploratory steps early in training before smoothly decaying to facilitate precise settling into sharp minima. With $\eta = 0.1$, a gradient of 2 moves a weight by 0.2 under plain SGD. With $\eta = 0.01$, the same gradient moves it by 0.02. A schedule changes this multiplier as training progresses without changing the optimizer's update rule. It can allow larger early updates and smaller later ones; whether that helps depends on the objective and the chosen endpoints.

`CosineSchedule` connects a maximum rate to a minimum rate along half a cosine wave over T epochs:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}\left(\eta_{\max} - \eta_{\min}\right)\left(1 + \cos\frac{\pi t}{T}\right)$$

At $t = 0$ the cosine is 1 and $\eta_0 = \eta_{\max}$. At $t = T$ the cosine is -1 and $\eta_T = \eta_{\min}$. In between, the rate falls slowly at first, fastest in the middle, and slowly again at the end, so the run spends a long stretch near the top rate, a long stretch near the floor, and passes quickly through the middle. `CosineSchedule` is that formula wrapped in a class, so the `Trainer` can ask for a rate by epoch number. Its defaults are the module's constants, a peak of 0.1, a floor of 0.01, and 100 epochs.

```
class CosineSchedule:
    """
    Cosine annealing learning rate schedule.
    """
    def __init__(self, max_lr: float = DEFAULT_MAX_LR, min_lr: float =
    ↪   DEFAULT_MIN_LR, total_epochs: int = DEFAULT_TOTAL_EPOCHS):
        self.max_lr = max_lr
        self.min_lr = min_lr
        self.total_epochs = total_epochs

    def get_lr(self, epoch: int) -> float:
        """Get learning rate for current epoch."""
        if epoch >= self.total_epochs:
            return self.min_lr

        # Cosine annealing formula
        cosine_factor = (1 + np.cos(np.pi * epoch /
↪   self.total_epochs)) / 2
        # Return a plain Python float: an np.float64 lr would promote
        ↪   float32 weights
```

```
        return float(self.min_lr + (self.max_lr - self.min_lr) *
        ↪   cosine_factor)
```

The guard at the top of `get_lr` returns the floor once `epoch >= total_epochs`. The formula's result is returned as a Python float, because an `np.float64` learning rate would promote float32 weights to float64 at the next update. Without it, continuing to evaluate the cosine would eventually raise the rate again. The Trainer asks for `get_lr(self.epoch)` before processing any batches, so every optimizer update in that epoch uses the same scheduled rate. With `total_epochs=4`, four calls to `train_epoch` use indices 0 through 3; index 4 reaches the floor only if training continues for another epoch.

8.3.2 Clipping

Standard practice in training Transformers and recurrent networks, gradient clipping (`torch.nn.utils.clip_grad_norm_`) guards training stability against sudden gradient explosions when navigating sharp loss cliffs. By computing the joint Euclidean norm across all model parameters and rescaling them proportionally when that norm exceeds a maximum threshold, the runtime constrains update magnitude while preserving the composite gradient's directional geometry. Suppose two parameter tensors have gradients $[3, 4]$ and $[0, 12]$. Their separate lengths are 5 and 12. Clipping each independently to length 2.5 multiplies the first by 0.5 and the second by about 0.2083, changing their relative contributions. The combined gradient points in a different direction.

Global clipping treats the gradients as one long vector. Its length is 13, so one coefficient, $2.5/13$, scales every entry. For plain SGD without momentum or decay, this also bounds the update length by the learning rate times 2.5. For Adam or momentum SGD, the clipping operation bounds the current gradient; the optimizer's stored state still participates in the eventual update. The global norm is

$$\|\mathbf{g}\|_{\text{global}} = \sqrt{\sum_{i=1}^{P} \|\mathbf{g}_i\|^2}$$

where $\mathbf{g}_i$ is the gradient of the i-th of P parameter tensors and $\|\cdot\|$ is the ordinary Euclidean length. If that global length exceeds a threshold $\gamma_{\max}$, scale every gradient tensor by the same factor $\gamma_{\max}/\|\mathbf{g}\|_{\text{global}}$. Because all tensors are scaled together, the direction of the long vector is unchanged; only its length is clamped to $\gamma_{\max}$. `clip_grad_norm` does this in two passes over the parameters. The first sums squared gradients to get the global norm; the second scales, but only if the norm is over the limit.

```
def clip_grad_norm(parameters: List, max_norm: float = 1.0) -> float:
    """
    Clip gradients by global norm to prevent exploding gradients.
    """
    if not np.isfinite(max_norm) or max_norm < 0:
        raise ValueError("max_norm must be finite and nonnegative")
    parameters = list(parameters)
    if not parameters:
        return 0.0

```

```
    # Collect all gradients and compute global norm
    total_norm = 0.0
    for param in parameters:
        if param.grad is not None:
            # Handle both Tensor gradients and numpy array gradients
            if isinstance(param.grad, np.ndarray):
                grad_data = param.grad
            else:
                # grad set by hand as a Tensor; take its array
                grad_data = param.grad.data
            # Accumulate in float64 so large finite float32 gradients do
            ↪ not
            # overflow when squared and incorrectly get clipped to
            ↪ zero.
            total_norm += np.sum(grad_data.astype(np.float64) ** 2)

    total_norm = np.sqrt(total_norm)

    # Clip if necessary
    if total_norm > max_norm:
        # A Python float scales float32 gradients without promoting them
        ↪ to float64
        clip_coef = float(max_norm / total_norm)
        for param in parameters:
            if param.grad is not None:
                # Handle both Tensor gradients and numpy array
                ↪ gradients
                if isinstance(param.grad, np.ndarray):
                    param.grad = param.grad * clip_coef
                else:
                    # grad set by hand as a Tensor; take its array
                    param.grad.data = param.grad.data * clip_coef

    return float(total_norm)
```

Parameters whose `.grad` is `None` contribute nothing to the norm. For the remaining parameters, the function accepts either the NumPy gradient array produced by backward or a Tensor assigned by hand. It replaces the array stored in `param.grad`, or in `param.grad.data` for the Tensor case, with scaled values. The coefficient is converted to a Python float, so float32 gradients stay float32 after scaling. An external reference to the old array does not become a reference to the replacement. Squaring in float64 avoids overflowing the norm calculation for large finite float32 gradients.[33] The returned value is the norm before clipping, so a caller can record how much scaling the update required.

8.3.3 The Trainer and the five steps

`train_epoch` is the public entry for training. It clears gradients, chooses the epoch's learning rate, and repeatedly calls `_process_batch`. That helper performs forward, loss, and backward. When an update is due, `_optimizer_update` averages and optionally clips the gradients, calls `optimizer.step()`, then clears the buffers. This split lets the Trainer process several batches before changing any parameters.

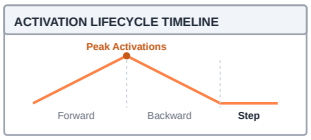
[33] In a deep network with tens of millions of parameters, summing squared float32 gradients ($\sum g_i^2$) can exceed float32's maximum finite bound of $\approx 3.4 \times 10^{38}$ during transient gradient spikes, triggering a premature inf overflow. Accumulating in IEEE 754 double precision (`float64`) provides dynamic range up to $\approx 1.8 \times 10^{308}$, preventing accumulator overflow before square-root extraction.

A larger batch requires more live activation storage during backward. Processing four samples together, for example, keeps intermediates for all four until their backward work completes. Processing two samples at a time can release each smaller graph after backward while retaining only the parameter gradients needed to combine their contributions.

The `.grad` buffers can hold the contributions from several **micro-batches**, smaller batches processed before one optimizer update. Run forward, loss, and backward on each micro-batch, releasing its graph afterward, while leaving parameters fixed and preserving their accumulated gradients. Only when the window is complete does the optimizer step. For equal-size batches the desired mean gradient is

$$\mathbf{g} = \sum_{j=1}^{k} \frac{1}{k} \nabla \mathcal{L}_j = \frac{1}{k} \sum_{j=1}^{k} \nabla \mathcal{L}_j$$

Here each loss is already a mean over its micro-batch. Dividing each contribution by a configured window length works only if the window contains that many equally sized batches. A short final window needs a smaller denominator. Unequal batches need different weights, or one example in the smaller batch would count more than an example in the larger one.



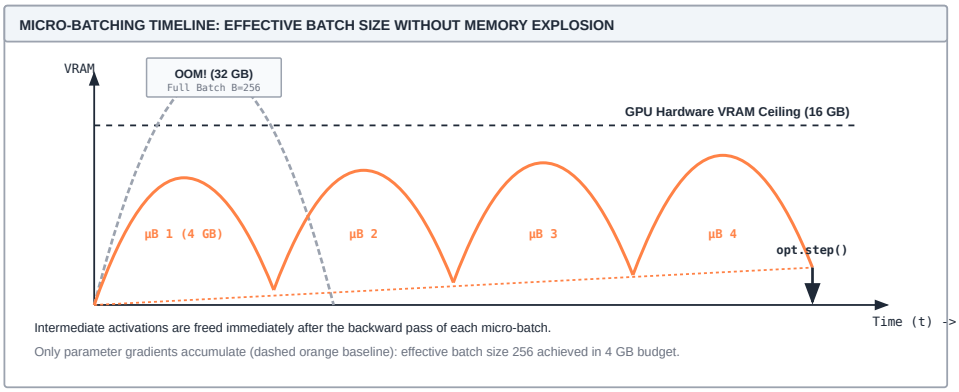
Microbatching caps peak activation RAM to a single microbatch while gradients accumulate.

The implementation avoids both assumptions by accumulating sample sums. `_process_batch` seeds backward with the current batch's sample count, undoing the averaging in the loss. `_optimizer_update` divides the accumulated parameter gradients by the number of samples actually seen in the window. The batch counter determines when to step; the sample counter determines how much to divide. This requires a loss whose scalar value is a batch mean, as the earlier loss modules provide, rather than an arbitrary summed loss.

A common bug in custom training loops divides accumulated gradients by the number of accumulation steps (k) rather than the true number of samples processed. If the final micro-batch has fewer samples (e.g., a batch of 6 following batches of 32), naive division over-weights the final samples by more than $5\times$.

Always accumulate sample-summed gradients and divide by $\sum N_{\text{samples}}$ across the window before clipping and stepping.

The equivalence to a single concatenated batch also assumes that splitting does not change the forward computation for a sample. A deterministic linear model satisfies that assumption. A layer that computes statistics across its batch, or a change in random masks, can make the two forward executions differ even when accumulation weights are correct. Accumulation saves the live activation storage of the larger batch; it does not remove these differences in model behavior.



Gradient accumulation decouples mathematical batch size from hardware VRAM constraints via sequential micro-batch activation clearing.

Figure 8.2: **Microbatch memory timeline and activation lifecycle**: Splitting a batch into microbatches caps peak active memory at a single microbatch's forward activations. During the backward pass, activations are discarded as gradients accumulate into persistent parameter `.grad` buffers. When the accumulation window concludes, the optimizer performs a normalized parameter update and clears gradients.

```python
class Trainer:
    """
    Complete training orchestrator for neural networks.
    """

    # ── Private serialization helpers (provided, not student-implemented) ──
    # These are pickle plumbing for checkpoint save/load. The training
    # concepts you'll implement are in the public methods below.

    def _get_model_state(self) -> Dict[int, np.ndarray]:
        """Extract model parameters for checkpointing."""
        ...

    def _set_model_state(self, state: Dict[int, np.ndarray]) -> None:
        """Restore model parameters from checkpoint."""
        ...

    def _get_optimizer_state(self) -> Dict[str, Any]:
        """Extract optimizer state for checkpointing."""
        ...

    def _set_optimizer_state(self, state: Dict[str, Any]) -> None:
        """Restore optimizer state from checkpoint."""
        ...

    def _get_scheduler_state(self) -> Optional[Dict[str, Any]]:
        """Extract scheduler state for checkpointing."""
        ...

```

```
    def _set_scheduler_state(self, state: Optional[Dict[str, Any]]) ->
    ↪   None:
        """Restore scheduler state from checkpoint."""
        ...

    def _forward(self, inputs: Tensor) -> Tensor:
        """Run the model, passing the training flag to a forward() that
        ↪   accepts one.
        """
        try:
            takes_flag = 'training' in
    ↪   inspect.signature(self.model.forward).parameters
        except (TypeError, ValueError):
            takes_flag = False
        if takes_flag:
            return self.model.forward(inputs,
            ↪   training=self.training_mode)
        return self.model.forward(inputs)
```

The class body supplies checkpoint helpers and `_forward`. The latter inspects `model.forward` and passes `training=self.training_mode` only when the method accepts that argument. A `Sequential` receives it; a bare `Linear` needs only the input. A model whose `forward` takes no flag still switches Dropout correctly, because `set_training_mode` has already set `training` on its layers. This dispatch keeps the training loop independent of the model container. The constructor and the two halves of the loop each live in their own source cell.

```
def trainer_init(self, model: Any, optimizer: Any, loss_fn: Any,
                 scheduler: Optional[Any] = None,
                 grad_clip_norm: Optional[float] = None) -> None:
    """
    Initialize trainer with model and training components.
    """
    self.model = model
    self.optimizer = optimizer
    self.loss_fn = loss_fn
    self.scheduler = scheduler
    self.grad_clip_norm = grad_clip_norm

    # Training state
    self.epoch = 0
    self.step = 0
    self.training_mode = True

    # History tracking
    self.history = {
        'train_loss': [],
        'eval_loss': [],
        'learning_rates': []
    }
```

```
def _trainer_process_batch(self, inputs: Tensor, targets: Tensor) ->
    float:
    """
    Process one batch: forward pass, loss computation, backward pass.
    """
    batch_size = inputs.shape[0]
    if batch_size == 0:
        raise ValueError("Training batches must contain at least one
        sample")

    # Forward pass (training flag on, so Dropout draws a mask)
    outputs = self._forward(inputs)

    # For sequence models (e.g. language models where outputs is [B,
    S, V] and targets is [B, S]),
    # flatten sequence and batch dimensions to compute per-token loss.
    if outputs.data.ndim == 3 and targets.data.ndim == 2:
        vocab_size = outputs.data.shape[-1]
        loss = self.loss_fn.forward(outputs.reshape(-1, vocab_size),
    targets.reshape(-1))
    else:
        loss = self.loss_fn.forward(outputs, targets)

    # Accumulate sample sums; the optimizer update will form the
    window mean.
    loss.backward(np.ones_like(loss.data) * batch_size)
    return float(loss.data) * batch_size
```

```
def _trainer_optimizer_update(self, sample_count: int = 1) -> None:
    """
    Average accumulated gradients, clip if enabled, and step the
    optimizer.
    """
    if sample_count <= 0:
        raise ValueError("An optimizer update needs a positive sample
        count")
    # Reuse the same parameters for normalization and clipping, even
    when
    # parameters() returns a generator.
    params = list(self.model.parameters())
    for param in params:
        if param.grad is not None:
            param.grad /= sample_count
    if self.grad_clip_norm is not None:
        clip_grad_norm(params, self.grad_clip_norm)

    self.optimizer.step()
    self.optimizer.zero_grad()
```

Follow the values across the two helpers. In `_process_batch`, it reads `inputs.shape[0]`, calls `_forward`, and requests the scalar batch mean from `loss_fn.forward`. Multiplying `np.ones_like(loss.data)` by `batch_size` supplies a seed with exactly the loss shape

required by Chapter 6. The method leaves sample-summed derivatives in `.grad` and returns the corresponding sample-summed loss as a Python float. It neither steps the optimizer nor clears the gradients.

`_optimizer_update(sample_count)` consumes those sums. It first materializes `model.parameters()` as a list, because normalization and clipping must be able to visit the same parameters even if the model returns a generator. It divides each existing gradient by `sample_count`, computes the optional clipping norm on that mean, updates the weights, and clears the buffers. Normalizing before clipping is essential: a threshold applied to a sum would become more restrictive merely because the window contained more samples. The constructor also marks model parameters trainable, but the optimizer must still own the same parameter objects. A flag can enable recording; it cannot add a missing weight to the optimizer's list.

Notice that `_trainer_process_batch` returns `float(loss.data) * batch_size` rather than the `loss` Tensor itself. In Python and PyTorch, retaining a reference to a loss Tensor in a long-lived list (such as `losses.append(loss)` or `history.append(loss)`) preserves the root of the autograd computation graph.

Because `loss._grad_fn` references the operation that produced it, and that operation references all intermediate activation buffers saved for backward, appending the Tensor prevents the garbage collector from freeing the forward graph. Memory consumption grows linearly with every mini-batch until the process crashes with Out-Of-Memory. Always extract a raw scalar float via `float(loss.data)` (or `loss.item()` in PyTorch) before accumulating loss metrics across iterations.

8.3.4 One epoch

The five steps handle one batch, and a dataset is thousands of them. Something has to walk the `DataLoader` from Chapter 5, count the batches waiting for an optimizer step, and set this epoch's learning rate before the first batch. That is `train_epoch`.

The naive loop for accumulation fires an update whenever the batch index is a multiple of k. It is one line, and it has a hole. If the loader yields 1001 batches and $k = 4$, the last batch's gradient is computed, added into `.grad`, and never stepped on. Either it is thrown away, or, worse, it is still sitting in the buffers when the next epoch's first `backward()` adds to it, and the first update of the new epoch then includes a gradient computed on last epoch's parameters. That is the contract violation from The Idea, once per epoch, silently.

`train_epoch` starts by clearing old gradients, then maintains both `pending` and `pending_samples`. Every batch increments the first by one and the second by its actual size. A full window calls `_optimizer_update(pending_samples)` and resets both counters. After the loop, a nonempty tail calls the same helper with its own sample count. No gradient is discarded, and a tail receives a full mean-gradient update rather than an update weakened by the nominal window length.

```
def trainer_train_epoch(self, dataloader: Iterable[Tuple[Tensor,
↪   Tensor]],
                        accumulation_steps: int = 1) -> float:
    """
    Train for one epoch through the dataset.
    """
```

```
    if isinstance(accumulation_steps, bool) or not isinstance(accumulation_steps, (int, np.integer)) or accumulation_steps < 1:
        raise ValueError("accumulation_steps must be a positive integer")
    set_training_mode(self.model, True)    # model and every layer inside it
    self.training_mode = True
    self.optimizer.zero_grad()

    # Update scheduler at the start of each epoch so LR is set before training begins
    if self.scheduler is not None:
        current_lr = self.scheduler.get_lr(self.epoch)
        self.optimizer.lr = current_lr
        self.history['learning_rates'].append(current_lr)

    total_loss = 0.0
    total_samples = 0
    pending_samples = 0
    pending = 0                        # batches waiting for an optimizer step

    for inputs, targets in dataloader:
        total_loss += self._process_batch(inputs, targets)
        batch_size = inputs.shape[0]
        total_samples += batch_size
        pending_samples += batch_size
        pending += 1

        # Update parameters every accumulation_steps batches
        if pending == accumulation_steps:
            self._optimizer_update(pending_samples)
            pending_samples = 0
            pending = 0
            self.step += 1

    # A short tail (batch count not divisible by accumulation_steps) still gets its step,
    # so no gradient is thrown away or carried into the next epoch
    if pending > 0:
        self._optimizer_update(pending_samples)
        self.step += 1

    avg_loss = total_loss / max(total_samples, 1)
    self.history['train_loss'].append(avg_loss)

    self.epoch += 1
    return avg_loss
```

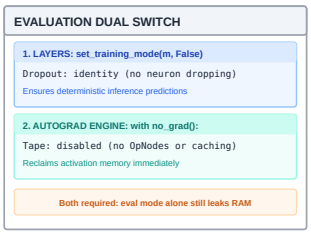
The returned loss uses a separate denominator: total samples in the epoch. `total_loss` adds the sample-summed values returned by `_process_batch`; dividing

by `total_samples` gives every observed sample equal weight. It is the mean of losses measured as training progressed, not a new evaluation of the final parameters. Changing accumulation can change the parameter trajectory and therefore the observed losses, but it does not change the reporting weights merely by regrouping the same values. On a successful return, `step` has increased once per optimizer update and `epoch` once for the completed call. The pending counters no longer exist, and all optimizer-owned gradients have been cleared. An empty loader produces the reference implementation's `0.0` loss placeholder and advances the epoch without an update; it is not evidence of a perfectly fitted dataset.

8.3.5 Evaluation

In production deep learning pipelines, validation loops evaluate model generalization by executing forward passes under strict inference semantics (`model.eval()` and `torch.no_grad()`). This operational dual switch deactivates stochastic training behaviors like dropout while disabling autograd tape recording to eliminate unnecessary activation memory overhead. A binary classifier returning probabilities `[[0.9], [0.1]]` for labels `[[1], [0]]` predicts both labels correctly. Taking `argmax` over each one-column row would instead return zero twice. Evaluation therefore needs a metric appropriate to the loss, as well as a forward pass that leaves the training state undisturbed.

`evaluate` sets `training_mode` to `False` and calls `set_training_mode(self.model, False)`, which clears `training` on the model and every layer stored inside it, then calls `_forward` and `loss_fn.forward` inside `no_grad()`. The first change disables dropout in any model, `Sequential` or hand-written; the second prevents autograd from retaining a graph for a backward pass that will never occur. These are separate controls. Turning off recording alone does not turn off dropout.



Evaluation Dual Switch: `set_training_mode(model, False)` switches off dropout in every layer of the model; `no_grad` disables tape recording to reclaim memory.

```python
def trainer_evaluate(self, dataloader: Iterable[Tuple[Tensor, Tensor]]) -> Tuple[float, float]:
    """
    Evaluate model on dataset without updating parameters.
    """
    set_training_mode(self.model, False)  # model and every layer inside it
    self.training_mode = False

    total_loss = 0.0
    correct = 0
    total = 0
    total_samples = 0

    for inputs, targets in dataloader:
        # Forward pass only, with the training flag off and no graph recorded
        with no_grad():
            outputs = self._forward(inputs)
            loss = self.loss_fn.forward(outputs, targets)

        batch_size = inputs.shape[0]
```

```
            total_loss += float(loss.data) * batch_size
            total_samples += batch_size

            # The loss defines the task: multiple outputs can also be
            ↪ regression.
            if isinstance(self.loss_fn, CrossEntropyLoss):
                predictions = np.argmax(outputs.data, axis=-1)
                correct += np.sum(predictions == targets.data)
                total += predictions.size
            elif isinstance(self.loss_fn, BinaryCrossEntropyLoss):
                predictions = outputs.data >= 0.5
                correct += np.sum(predictions == (targets.data >= 0.5))
                total += predictions.size

        avg_loss = total_loss / total_samples if total_samples > 0 else
        ↪ 0.0
        # correct is a NumPy integer sum, so the division yields
        ↪ np.float64.
        # Cast it so both returned values are plain Python floats.
        accuracy = float(correct / total) if total > 0 else 0.0

        self.history['eval_loss'].append(avg_loss)

        return avg_loss, accuracy
```

The evaluation loss uses the same sample weighting as the training report. CrossEntropyLoss computes class accuracy by comparing the largest-logit index with each integer target. BinaryCrossEntropyLoss thresholds probabilities and targets at `>= 0.5` and counts correct binary entries, including every label in a multilabel output. Regression and custom losses return `0.0` as an unused accuracy placeholder. Output width cannot identify the task: a two-column output may represent two regression values.

There is no backward or optimizer update, so existing parameter gradients and weights remain unchanged. The pass still allocates the arrays needed to calculate predictions and losses; `no_grad()` removes their retention for backward, not the forward computation itself. On return, the Trainer remains in evaluation mode and appends the loss to `history['eval_loss']`. The next `train_epoch` explicitly switches training back on.

8.3.6 Checkpoints

In large-scale distributed training runs that span days or weeks across hardware clusters, robust checkpointing (`torch.save`) is essential for fault tolerance against node preemption and hardware failures. Resuming an interrupted run without altering optimization dynamics requires serializing not merely model weights, but the complete state machine—including optimizer moment accumulators, step counters, and scheduler phases. After two Adam updates, restoring only the weights discards the moment estimates and their ages. The next call to `step()` then follows a different trajectory even on identical data. A **checkpoint** preserves the state needed to continue the supported computation: parameters, optimizer state and hyperparameters, schedule settings, clipping configuration, mode, counters, and history. (The word is unrelated to the activation checkpointing of Chapter 6, which trades recomputation for memory inside one backward pass.)

The naive save is `open(path, 'wb')` followed by `pickle.dump`. Opening for writing truncates the old file to zero bytes first, then the new bytes stream in over the next seconds, and a training job is exactly the kind of process that gets killed during those seconds. A kill that lands mid-write leaves a truncated file that `pickle.load` rejects, and the previous checkpoint is already gone, because the open destroyed it. The crash the checkpoint existed to protect against has now cost the run and the checkpoint together.

The implementation writes to `path + ".tmp"`, closes that file, and calls `os.replace` to replace the checkpoint name. Because the temporary file sits beside the destination, the replacement stays on the same filesystem. Readers see the old complete file or the new complete file instead of a half-written destination. A failed write leaves an existing checkpoint untouched; the exception handler also attempts to remove the temporary file.

Atomic replacement is narrower than durability after power loss. This code does not call `fsync` on the file or directory, and its fixed temporary name assumes one writer per checkpoint path. The mechanism protects the destination from interrupted serialization; it is not a transaction protocol for concurrent writers or a guarantee that all bytes have reached persistent storage.

`save_checkpoint` gathers the state into one dictionary, serializes it with `pickle`, Python's mechanism for writing object data to a byte stream, and performs that write-then-rename. `load_checkpoint` reverses it, copying parameter values back into the live tensors so the model, optimizer, schedule, and counters agree.

```
def trainer_save_checkpoint(self, path: str) -> None:
    """
    Save parameters, optimizer, scheduler, and epoch history for
    ↪ resumption.
    """
    checkpoint = {
        'epoch': self.epoch,
        'step': self.step,
        'model_state': self._get_model_state(),
        'optimizer_state': self._get_optimizer_state(),
        'scheduler_state': self._get_scheduler_state(),
        'history': self.history,
        'training_mode': self.training_mode,
        'grad_clip_norm': self.grad_clip_norm
    }

    Path(path).parent.mkdir(parents=True, exist_ok=True)
    # Write to a temp file first, then atomically replace the target.
    # Writing directly to `path` would leave a partially-written
    ↪   (corrupt)
    # checkpoint in place if the process is killed, the disk fills up,
    ↪   or
    # pickling fails partway through, silently destroying whatever
    ↪   good
    # checkpoint was there before. os.replace() is atomic on both
    ↪   POSIX
    # and Windows: `path` is always either the old complete checkpoint
    ↪   or
    # the new one, never a partial write.
```

```
    tmp_path = f"{path}.tmp"
    try:
        with open(tmp_path, 'wb') as f:
            pickle.dump(checkpoint, f)
        os.replace(tmp_path, path)
    except BaseException:
        try:
            os.remove(tmp_path)
        except OSError:
            pass
        raise
```

```
def trainer_load_checkpoint(self, path: str) -> None:
    """
    Load training state from checkpoint.
    """
    with open(path, 'rb') as f:
        checkpoint = pickle.load(f)

    self.epoch = checkpoint['epoch']
    self.step = checkpoint['step']
    self.history = checkpoint['history']
    self.training_mode = checkpoint['training_mode']
    self.grad_clip_norm = checkpoint.get('grad_clip_norm',
    ↪   self.grad_clip_norm)
    set_training_mode(self.model, self.training_mode)

    # Restore states
    if 'model_state' in checkpoint:
        self._set_model_state(checkpoint['model_state'])
    if 'optimizer_state' in checkpoint:
        self._set_optimizer_state(checkpoint['optimizer_state'])
    if 'scheduler_state' in checkpoint:
        self._set_scheduler_state(checkpoint['scheduler_state'])
```

The dictionaries come from the helpers on the class. The model helper copies every parameter's array, keyed by position in `parameters()`; the schedule helper records the three numbers a `CosineSchedule` holds; and the optimizer helper is the one worth reading, because it decides which of Chapter 7's state survives a restart.

```
def _get_optimizer_state(self) -> Dict[str, Any]:
    """Extract optimizer state for checkpointing."""
    state = {}
    # Moment estimates and their age must resume together.
    ↪   Hyperparameters
    # also belong to the saved run, not the freshly constructed
    ↪   optimizer.
    for name in ('lr', 'step_count', 'momentum', 'beta1', 'beta2',
    ↪   'eps', 'weight_decay'):
        if hasattr(self.optimizer, name):
```

```
            state[name] = getattr(self.optimizer, name)
    if hasattr(self.optimizer, 'update_counts'):
        state['update_counts'] = self.optimizer.update_counts.copy()
    if hasattr(self.optimizer, 'has_momentum') and
    ↪   self.optimizer.has_momentum():
        momentum_state = self.optimizer.get_momentum_state()
        if momentum_state is not None:
            state['momentum_buffers'] = momentum_state
    return state
```

The entry stores optimizer hyperparameters, the global call counter, per-parameter update counts, and the arrays returned by `get_momentum_state()`. For SGD those arrays are velocities; for Adam and AdamW they are the two moment buffers. Loading restores this state on an already constructed optimizer. The model helper checks parameter counts and shapes before replacing their data arrays with copies. It preserves the Tensor objects, so the optimizer's references still reach the model's weights. Matching architecture, parameter order, and optimizer class remain the caller's responsibility. Restoring `grad_clip_norm` also matters: a resumed Trainer with default clipping disabled must not take an unclipped update when the saved run used clipping. An older checkpoint without this field leaves the current configuration unchanged.

The supported save point is an epoch boundary, after pending gradients have been applied and cleared. With the same next batch and a deterministic model, restored SGD, Adam, and AdamW take the same next update as uninterrupted training, including bias correction and decay. The file does not preserve random generator state, an unfinished accumulation window, loader position, or later nonparameter buffers such as running normalization statistics. Older checkpoints that lack optimizer ages cannot promise exact continuation. The resume listing in the worked trace (listing 8.5) checks the supported boundary before considering these additional states. A failed load is not a rollback operation: counters and other fields may already have changed before a later compatibility check raises, so an incompatible checkpoint should be rejected rather than treated as a partial resume.

8.4 A Worked Trace

The smallest run that exercises all five steps is a model with one weight, one training sample, and mean squared error, traced in table 8.2. Take $w = 0.5$, input $x = 1$, target $y = 1$, and plain gradient descent at $\eta = 0.1$. The loss is $(wx - y)^2$ and its gradient with respect to w is $2(wx - y)\,x$.

Table 8.2: **A one-weight update**: Each iteration's forward reads the w that the previous update wrote.

Iteration	1. `zero_grad`	2. forward wx	3. loss $(wx-y)^2$	4. backward $\partial\mathcal{L}/\partial w$	5. step $w - \eta\,\text{grad}$
1	`grad = None`	0.500	0.2500	$2(0.5-1)(1) = -1.000$	$0.5 + 0.1 =$ **0.600**

Iteration	1. `zero_grad`	2. forward wx	3. loss $(wx-y)^2$	4. backward $\partial\mathcal{L}/\partial w$	5. step $w - \eta\,\text{grad}$
2	`grad = None`	0.600	0.1600	$2(0.6-1)(1) = -0.800$	$0.6 + 0.08 =$ **0.680**
3	`grad = None`	0.680	0.1024	$2(0.68-1)(1) = -0.640$	$0.68 + 0.064 =$ **0.744**

The residual $wx - y$ shrinks by a factor of 0.8 each time, so the loss is multiplied by 0.64 each iteration and w approaches 1. In listing 8.2, lines 8 to 10 initialize `model = Linear(1, 1, bias=False)` with $w = 0.5$ and construct a `Trainer` with `SGD(lr=0.1)` and `MSELoss()`. Lines 11 to 12 create a single-element `TensorDataset` and `DataLoader`. In lines 13 to 15, the loop trains for three epochs, reproducing the exact sequence from table 8.2 ($w \to 0.600, 0.680, 0.744$). The Trainer adds bookkeeping, not arithmetic.

Listing 8.2 A one-weight training run: Three epochs follow the arithmetic in the worked trace.

```
1  from tinytorch.core.tensor import Tensor
2  from tinytorch.core.layers import Linear
3  from tinytorch.core.losses import MSELoss
4  from tinytorch.core.optimizers import SGD
5  from tinytorch.core.dataloader import TensorDataset, DataLoader
6  from tinytorch.core.training import Trainer
7
8  model = Linear(1, 1, bias=False)
9  model.weight.data[:] = 0.5
10 trainer = Trainer(model, SGD(model.parameters(), lr=0.1), MSELoss())
11 dataset = TensorDataset(Tensor([[1.0]]), Tensor([[1.0]]))
12 loader = DataLoader(dataset, batch_size=1)
13 for it in range(1, 4):
14     loss = trainer.train_epoch(loader)
15     print(f"epoch {it}: loss = {loss:.4f}  w -> {model.weight.data[0,
       ↪   0]:.3f}")
```

```
epoch 1: loss = 0.2500  w -> 0.600
epoch 2: loss = 0.1600  w -> 0.680
epoch 3: loss = 0.1024  w -> 0.744
```

A short accumulation window exposes a bug that the one-sample default cannot. In listing 8.3, lines 1 to 3 initialize the same model to zero with `SGD(lr=0.1)`. Lines 4 to 7 define two unequal batches: batch 0 has shape `(2, 1)` (two examples with target 1) and batch 1 has shape `(1, 1)` (one example with target 3). Line 8 calls `trainer.train_epoch(batches, accumulation_steps=4)`. The first batch contributes a weighted gradient sum of -4, and the second contributes -6. Line 9 prints the resulting epoch-averaged loss $11/3 = 3.666667$ and weight 0.333333 (from step $\eta \cdot 10/3$). In line 11, `model.weight.grad` is confirmed to be `None`, verifying that the window update successfully cleared pending gradients.

The first batch has shape `(2, 1)` and the second `(1, 1)`; both produce a scalar loss and a weight gradient of shape `(1, 1)`. The state in table 8.3 shows what survives each helper return while the weight stays at zero.

Listing 8.3 Unequal batches in one window: The final partial window averages three samples before updating.

```
model = Linear(1, 1, bias=False)
model.weight.data[:] = 0.0
trainer = Trainer(model, SGD(model.parameters(), lr=0.1), MSELoss())
batches = [
    (Tensor([[1.0], [1.0]]), Tensor([[1.0], [1.0]])),
    (Tensor([[1.0]]), Tensor([[3.0]])),
]
loss = trainer.train_epoch(batches, accumulation_steps=4)
print(f"loss={loss:.6f}, weight={model.weight.data[0, 0]:.6f}")
# loss=3.666667, weight=0.333333
print(trainer.step, model.weight.grad)
# 1 None
```

Table 8.3: **One unequal accumulation window**: The gradient and loss sums use the same sample weights. The optimizer changes the weight only after the final division.

Point in the call	Pending batches	Pending samples	Weight gradient	Loss sum
Epoch entry, after clearing	0	0	`None`	0
After first batch	1	2	-4	2
After second batch	2	3	-10	11
Inside the tail update, after averaging	2	3	$-10/3$	11
Epoch return	local state ends	local state ends	`None`	reported as $11/3$

Only two batches arrived, although the configured window length was four. The tail still gets one update using all three samples, and the reported loss is `(2 * 1 + 1 * 9) / 3`. `None` confirms that the update cleared the gradients. This trace checks the meaning of the counters as well as the arithmetic, which a falling loss curve alone would not establish.

The clipping example from the code discussion uses two nonzero gradient tensors. Take $\mathbf{g}_1 = [3, 4]$ and $\mathbf{g}_2 = [0, 12]$ with $\gamma_{\max} = 2.5$. Both tensors contribute to the global norm in table 8.4.

Table 8.4: **Global-norm clipping**: One coefficient preserves the direction of the concatenated vector; its new length is 2.5. Entries are rounded to four decimal places.

Quantity	Value
Sum of squares	$3^2 + 4^2 + 0^2 + 12^2 = 169$
Global norm	$\sqrt{169} = 13.00$
Clip coefficient	$2.5/13 \approx 0.1923$
$\mathbf{g}_1$ after clipping	$[0.5769, 0.7692]$, norm 0.9615

Quantity	Value
$\mathbf{g}_2$ after clipping	$[0, 2.3077]$

The cosine schedule with $\eta_{\max} = 0.10$, $\eta_{\min} = 0.01$, and $T = 4$ gives five boundary values in table 8.5. The first four training epochs use indices 0 through 3; the fifth uses the floor at index 4.

Table 8.5: **Cosine schedule boundaries**: The drop is smallest at the ends and largest in the middle.

Epoch t	$\cos(\pi t/4)$	$\frac{1}{2}(1+\cos)$	$\eta_t = 0.01 + 0.09 \times$ factor
0	$+1.0000$	1.0000	**0.1000**
1	$+0.7071$	0.8536	**0.0868**
2	0.0000	0.5000	**0.0550**
3	-0.7071	0.1464	**0.0232**
4	-1.0000	0.0000	**0.0100**

The one-weight, clipping, and schedule calculations can also be checked directly with NumPy.

Listing 8.4 Checking the arithmetic: NumPy reproduces the gradient, clipping, and schedule calculations.

```
import numpy as np

# One weight, one sample, mean squared error, plain SGD with lr = 0.1
w, x, y, lr = 0.5, 1.0, 1.0, 0.1
for it in range(1, 4):
    grad = None                     # 1. zero_grad
    pred = w * x                    # 2. forward
    loss = (pred - y) ** 2          # 3. loss
    grad = 2 * (pred - y) * x       # 4. backward
    w = w - lr * grad               # 5. step
    print(f"iteration {it}: pred = {pred:.3f}  loss = {loss:.4f}  "
          f"grad = {grad:+.3f}  w -> {w:.3f}")

# Global-norm clipping on two gradient tensors, max_norm = 2.5
g1, g2 = np.array([3.0, 4.0]), np.array([0.0, 12.0])
global_norm = np.sqrt(np.sum(g1 ** 2) + np.sum(g2 ** 2))
coef = min(1.0, 2.5 / global_norm)
print(f"global norm {global_norm:.2f}, coefficient {coef:.4f}")
print(f"g1 -> {(g1 * coef).tolist()}")

# Cosine schedule, max_lr = 0.10, min_lr = 0.01, four epochs
for t in range(5):
    lr_t = 0.01 + 0.5 * (0.10 - 0.01) * (1.0 + np.cos(np.pi * t / 4))
    print(f"epoch {t}: lr = {lr_t:.4f}")
```

A restart should preserve the next update, not merely recover a plausible weight. Reuse the Trainer and the unequal batches from the accumulation trace, whose weight is now one third. Set clipping to `0.5`, save, and load into a fresh model with a different learning rate and no clipping configured.

Listing 8.5 Checking a resumed update: A fresh Trainer recovers the saved clipping configuration and next update.

```
import tempfile
from pathlib import Path

trainer.grad_clip_norm = 0.5
restored_model = Linear(1, 1, bias=False)
restored = Trainer(
    restored_model, SGD(restored_model.parameters(), lr=0.9),
    MSELoss())
with tempfile.TemporaryDirectory() as directory:
    path = Path(directory) / "resume.pkl"
    trainer.save_checkpoint(path)
    restored.load_checkpoint(path)

assert restored.optimizer.lr == trainer.optimizer.lr == 0.1
assert restored.grad_clip_norm == 0.5
expected_loss = trainer.train_epoch(batches, accumulation_steps=4)
actual_loss = restored.train_epoch(batches, accumulation_steps=4)
np.testing.assert_allclose(actual_loss, expected_loss)
np.testing.assert_allclose(restored_model.weight.data,
    model.weight.data)
print(f"resumed weight={restored_model.weight.data[0, 0]:.6f}")
# resumed weight=0.383333
```

The next mean gradient is $-8/3$, which clipping changes to -0.5. SGD adds 0.05 to the saved weight. Both Trainers finish with `epoch=2`, `step=2`, and cleared gradients. Without restoring the clipping threshold, the new Trainer would use the larger, unclipped update even though it loaded the correct weight and learning rate.

8.5 From TinyTorch to Production

The reference loop makes two decisions separately: what objective a gradient represents, and how to execute the operations that produce it. Production systems can change execution without changing the first decision. A gradient accumulation implementation still needs the correct sample denominator, whether its arrays live in NumPy or on an accelerator. Chapter 15 measures elapsed time, operation counts, and memory for model forward passes; production profilers extend the same measurements to accelerator timelines.

Evaluation retains the separation between mode and recording. PyTorch's autograd notes distinguish `model.eval()`, which changes layer behavior, from `torch.no_grad()`, which disables graph recording.[34] The resulting memory reduction depends on which

[34] A common bug among deep learning practitioners is confusing `model.eval()` with `torch.no_grad()`. Calling `model.eval()` mutates layer operational modes (disabling dropout regularization and freezing batch normalization running statistics), but leaves autograd tape recording active, unnecessarily allocating intermediate forward graphs. Conversely, `with torch.no_grad():` disables the autograd tape to reclaim memory, but leaves dropout actively masking features. Robust validation passes require invoking both simultaneously.

intermediates the training graph would otherwise retain, so it does not establish a fixed ratio between evaluation and training batch sizes.

PyTorch's global clipping implementation can apply operations across lists of tensors through a `foreach` path when supported. This reduces per-tensor dispatch overhead, but it is not a guarantee that the entire clipping operation uses one GPU kernel. With mixed precision, a separate scaling mechanism may multiply the loss to keep small derivatives representable. The accumulated gradients must then be unscaled before comparing their norm to the clipping threshold, as shown in the mixed-precision examples. This is the same requirement as normalizing sample sums before clipping in TinyTorch: the threshold must see gradients in the intended units.

The choice of 16-bit numerical format fundamentally alters the training loop mechanics:

- **IEEE FP16 (5 exponent bits, 10 mantissa bits):** With only 5 exponent bits, FP16 has a narrow dynamic range (6×10^{-5} to $65{,}504$). Gradients smaller than $2^{-24} \approx 6 \times 10^{-8}$ underflow directly to zero. Production engines must wrap the training loop in a dynamic loss scaler (`torch.amp.GradScaler("cuda")`):
  1. Multiply forward loss by scale factor S (e.g., $65{,}536$) before backward so small gradients remain representable.
  2. Unscale gradients ($\mathbf{g} \leftarrow \mathbf{g}/S$) before gradient clipping and optimizer stepping.
  3. If nonfinite values (`inf/nan`) appear, skip `optimizer.step()` and halve S to re-stabilize the trajectory.
- **Brain Floating Point (BF16, 8 exponent bits, 7 mantissa bits):** BF16 shares the exact same 8-bit exponent and dynamic range as single-precision FP32 (10^{-38} to 10^{38}), trading off mantissa precision. Because gradients do not underflow its wide exponent range, BF16 training does not require loss scaling or a `GradScaler`, simplifying modern distributed training loops.[35]

A schedule's unit is also part of its contract. PyTorch's CosineAnnealingLR advances on calls to `scheduler.step()`. A caller can place those calls at epoch or update boundaries, with the horizon expressed in the corresponding units. TinyTorch makes a simpler choice, looking up a rate by the epoch counter and holding it fixed throughout that pass.

For checkpointing, PyTorch's general checkpoint example saves the model and optimizer state dictionaries together with training metadata. Reproducing a particular run can require additional state, such as random generators and the loader's position, as well as deterministic execution. Saving those fields is a separate concern from protecting the destination file during a write. TinyTorch exposes both boundaries with an explicit state dictionary and temporary-file replacement; it does not implement a complete snapshot of the process.

[35] Even when running forward and backward matrix multiplications in 16-bit precision (BF16 or FP16), industrial training harnesses must maintain an FP32 master copy of model weights in optimizer state (Micikevicius et al., 2018). Small parameter updates ηg_t are frequently smaller than the spacing between representable 16-bit floating-point numbers ($\epsilon_{\text{BF16}} \approx 7.8 \times 10^{-3}$); applying updates directly in 16-bit would round down to zero, stalling gradient descent entirely.

Table 8.6: **The same contracts at different scales**: Execution choices and saved state vary, while an update still requires correctly weighted gradients at fixed parameters.

Concern	TinyTorch `Trainer`	PyTorch
The five steps	`_process_batch` + `_optimizer_update`	Written by hand, same order

Concern	TinyTorch `Trainer`	PyTorch
Evaluation	`set_training_mode(model, False)` (or `model.eval()` on a `Layer`/`Sequential`), plus `training=False` through `_forward`, inside `no_grad()`	`model.eval()` + `torch.no_grad()`
Clipping	Global norm, NumPy passes over tensors	Global norm, optional multi-tensor path
Schedule	Epoch-indexed lookup before training	Caller chooses epoch or update units
Accumulation	Sum sample-weighted gradients; divide by actual window samples before clipping	Same weighting required; dividing by k assumes equal full batches
Checkpoint	Parameters, optimizer buffers and ages, configuration, counters, history	State dictionaries plus application-selected run state

The comparison in table 8.6 gives a useful review order for a training program. First identify where parameters can change and where gradients are cleared. Then check the sample denominator, the units seen by clipping, and the state restored before the next update. A loss curve can suggest a problem; these boundaries identify the computation the program actually performed.

8.6 Summary

A call to `train_epoch` clears old gradients, sets the epoch's learning rate, and enters `_process_batch` for each batch. That helper returns a sample-summed loss and leaves sample-summed derivatives in parameter gradients. `_optimizer_update` divides by actual window samples, clips if configured, updates the shared weights, and clears gradients. This boundary handles both unequal batches and a short final window.

Evaluation changes mode and disables recording while preserving weights and existing gradients. Checkpointing preserves parameters, optimizer buffers and ages, hyperparameters, clipping, schedule settings, counters, and history at an epoch boundary. With a matching deterministic model and the same next batch, this state preserves the next update; random execution and loader position require additional state. Milestone I uses these contracts to test the complete engine on XOR and handwritten digits. Synthesis II applies the same five-step contract to train TinyGPT. The next layer mechanisms can then build on a loop whose state transitions are explicit.

You have now built the entire learning stack (Modules 01–08). You can run:

```
tito milestone run 02  # Recreate the 1969 XOR crisis (max 75%
    accuracy)
tito milestone run 03  # Solve XOR (all 4 rows, hidden layer must
    learn), then TinyDigits (fails below 75% test accuracy)
```

These milestones recreate Minsky and Papert's 1969 limit (Milestone 02, which needs only Modules 01–03) and the breakthrough of Rumelhart, Hinton, and Williams (1986, Milestone 03, which needs Modules 01–07). Turn to Synthesis I to walk through the mathematical proof and execution trace of this synthesis.

8.7 Further Reading

- **Pascanu, Mikolov, and Bengio** (2013). "On the difficulty of training recurrent neural networks." *ICML*. Exploding gradients and the norm clipping that `clip_grad_norm` implements.
- **Loshchilov and Hutter** (2017). "SGDR: stochastic gradient descent with warm restarts." *ICLR*. The cosine schedule behind `CosineSchedule`.

Synthesis I: From Perceptrons to Rumelhart's MLP

The History and the Problem



Framework Blueprint: Synthesis I synthesizes all Part I mechanisms into a complete neural network to solve the non-linear XOR boundary.

In 1958, Frank Rosenblatt introduced the perceptron, sparking initial optimism for trainable artificial intelligence. But in 1969, Marvin Minsky and Seymour Papert proved mathematically that single-layer perceptrons could not learn the non-linear XOR (exclusive or) decision boundary. A single linear hyperplane cannot separate points whose positive labels sit on opposite diagonals in coordinate space. Because real-world classification boundaries are fundamentally non-linear, this limitation paralyzed neural network research and precipitated the first "AI Winter," halting funding and progress for more than a decade.

The Multi-Layer Breakthrough

The resolution required folding the coordinate space itself. In 1986, Rumelhart, Hinton, and Williams demonstrated that multi-layer perceptrons (MLPs) equipped with non-linear hidden units, trained through reverse-mode automatic differentiation (backpropagation), could resolve the XOR crisis. Hidden layers project inputs into transformed intermediate representations where previously inseparable patterns become linearly separable.

What Was Solved

By chaining affine projections with non-linear activations and executing exact reverse-mode gradient updates through computational graphs, neural networks acquired the capacity to learn non-linear decision boundaries autonomously. Synthesis I integrates Modules 01 through 08 (`tinytorch/core/tensor` through `tinytorch/core/training`) to construct and train that complete multi-layer network, verifying that our foundational engine resolves the historical crisis that once halted the field.

In modern deep learning, creating a multi-layer perceptron to learn non-linear decision surfaces is trivial:

```
import torch
import torch.nn as nn

# The multi-layer perceptron that resolves the XOR crisis
mlp = nn.Sequential(
    nn.Linear(2, 4),
    nn.ReLU(),
    nn.Linear(4, 1)
)
```

```
x = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
predictions = mlp(x)
print("Predictions shape:", predictions.shape)  # (4, 1)
```

Underneath that short declaration, the framework synthesizes multiple core mechanisms into a single learning machine:

1. **Geometric space warping:** The hidden layer projects the 2D input coordinates into a 4D hidden subspace, where the non-linear ReLU activation bends and folds the coordinates so a final linear hyperplane can cleanly separate the classes.
2. **Modular container pipelining:** The `Sequential` container automatically threads activations forward and routes training signals across parameter layers.
3. **Multi-layer backward chain rule:** Autograd stitches multiple affine transformations and non-linear gates into an end-to-end graph, computing exact parameter gradients across depth.

This milestone synthesizes the foundations of TinyTorch into a complete working deep learning system, demonstrating that our modular kernel can learn non-linear representations from scratch. Before validating our multi-layer network, we must inspect the mathematical barrier that blocked early single-layer perceptrons.

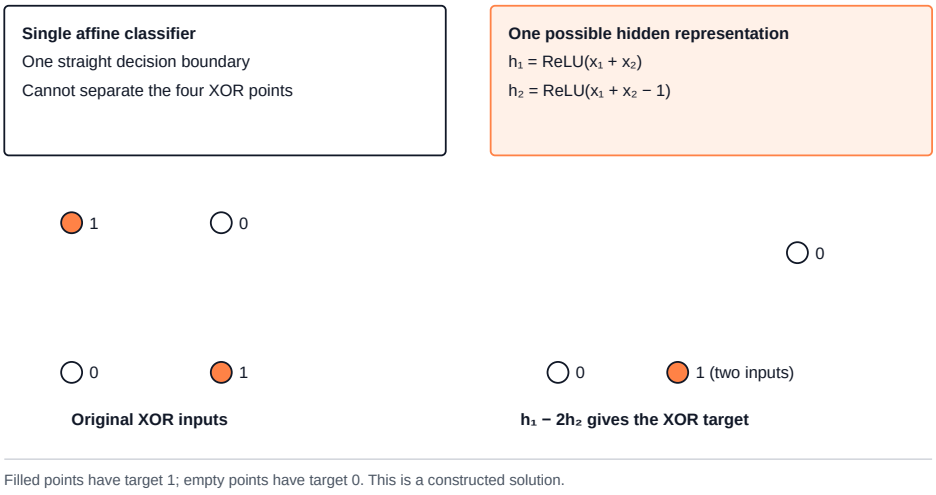


Figure 8.3: **Separating XOR after a hidden layer**: The four input corners become three hidden points; the two positive inputs share one point that a linear output can separate.

↩ **Jump back:** Chapter 3 *Composing Linear layers and ReLU activations to bend representation space.*

Part 1: The 1958 Perceptron (Forward Pass Without Training)

Frank Rosenblatt's 1958 perceptron is one `Linear` layer feeding one squashing function (`Sigmoid`). With Modules 01 through 03 in hand, the model class fits in about a dozen lines of Python, and the rest of this excerpt draws the data and runs one forward pass:

```python
# milestones/01_1958_perceptron/01_rosenblatt_forward.py (trimmed)
import numpy as np
rng = np.random.default_rng()
from tinytorch.core.tensor import Tensor        # Module 01: YOU built this!
from tinytorch.core.layers import Linear        # Module 03: YOU built this!
from tinytorch.core.activations import Sigmoid  # Module 02: YOU built this!

class Perceptron:
    def __init__(self, input_size=2, output_size=1):
        self.linear = Linear(input_size, output_size)
        self.activation = Sigmoid()

    def forward(self, x):
        x = self.linear(x)
        x = self.activation(x)
        return x

    def __call__(self, x):
        return self.forward(x)

# in main():
cluster1 = rng.normal([2, 2], 0.5, (5, 2))    # Class 1: top-right
cluster2 = rng.normal([-2, -2], 0.5, (5, 2)) # Class 0: bottom-left
X = np.vstack([cluster1, cluster2]).astype(np.float32)
y = np.array([1, 1, 1, 1, 1, 0, 0, 0, 0, 0], dtype=np.float32)  # True labels

import tinytorch.core.layers as _layers
_layers.rng = np.random.default_rng()

model = Perceptron(input_size=2, output_size=1)

input_tensor = Tensor(X)
predictions = model(input_tensor)

pred_classes = (predictions.data > 0.5).astype(int).flatten()
accuracy = (pred_classes == y).mean()
```

Running this forward pass (`tito milestone run 01`) draws two clusters of five points each, centered at $(2, 2)$ for class 1 and $(-2, -2)$ for class 0, and scores them with a perceptron whose weights are random. Neither the data nor the weights are seeded. The script replaces the layers module's seeded generator, so every run draws a new line. The accuracy can land anywhere from 0% to 100%. The clusters are separable, so a random line often gets every point right or every point backwards, and neither outcome is learning. The forward pass computes without error, but because the weights are drawn randomly from initialization, the decision boundary lands at an arbitrary orientation. Since the accuracy is luck, the script grades the arithmetic instead: it recomputes $\sigma(XW + b)$ in NumPy from the model's own weights and exits with status 1 if your `Linear` and `Sigmoid` disagree.

This demonstrates the foundational lesson of deep learning systems: **a forward pass alone does not produce intelligence**. Writing model architectures is straightforward; making them learn requires loss functions, automatic differentiation, and optimizers (Chapter 4 through Chapter 8).

Part 2: The 1969 XOR Crisis (Linear Separability Limits)

Can we fix the perceptron simply by choosing the weights by hand? For linearly separable clusters, yes: picking $w_1 = 1, w_2 = 1, b = 0$ separates the script's two clusters, centered at $(-2, -2)$ and $(2, 2)$, with 100% accuracy.

In 1969, Marvin Minsky and Seymour Papert published their famous critique showing that for non-linear boundaries, no weight configuration exists that can separate the data.[36] XOR (exclusive or) is the smallest function that defeats the single-layer perceptron. It outputs 1 when its two inputs differ and 0 when they agree: $(0, 0) \to 0$, $(0, 1) \to 1$, $(1, 0) \to 1$, and $(1, 1) \to 0$. The two positive corners sit on one diagonal and the two negative corners on the other. Draw any line you like through the square and at least one corner ends up on the wrong side.

[36] Minsky and Papert proved that single-layer perceptrons of limited order cannot compute global predicates such as parity (XOR is its two-input case) and connectedness. The authors acknowledged that multilayer networks escape these limits, but they doubted that a practical learning procedure for them would be found; they did not prove that none existed. Their book is often credited with contributing to a decline in neural network funding that lasted until Rumelhart, Hinton, and Williams popularized backpropagation in 1986.

The script in `milestones/02_1969_xor/01_xor_crisis.py` tests several candidate lines before the proof. It builds the perceptron from the existing modules, sets two weights and one bias by hand, and scores the four rows:

```python
# milestones/02_1969_xor/01_xor_crisis.py (trimmed)
import numpy as np
from tinytorch import Tensor, Linear, Sigmoid

XOR_INPUTS = np.array([
    [0.0, 0.0],
    [0.0, 1.0],
    [1.0, 0.0],
    [1.0, 1.0]
])

XOR_TARGETS = np.array([
    [0.0],   # 0 XOR 0 = 0
    [1.0],   # 0 XOR 1 = 1
    [1.0],   # 1 XOR 0 = 1
    [0.0],   # 1 XOR 1 = 0
])

class SingleLayerPerceptron:
    def __init__(self):
        self.linear = Linear(2, 1)
        self.sigmoid = Sigmoid()

    def set_weights(self, w1, w2, b):
        # Weight shape is (in_features, out_features) = (2, 1)
        self.linear.weight.data[...] = [[w1], [w2]]
        self.linear.bias.data[...] = b

```

```
    def __call__(self, x):
        return self.sigmoid(self.linear(x))

def evaluate_on_xor(model):
    X = Tensor(XOR_INPUTS)
    predictions = model(X)
    pred_classes = (predictions.data > 0.5).astype(int)
    accuracy = (pred_classes == XOR_TARGETS).mean()
    return accuracy, predictions.data.flatten()

# in main(), the first four of ten hand-picked lines:
model = SingleLayerPerceptron()
configurations = [
    (1.0, 1.0, -0.5, "Diagonal line (positive slope)"),
    (1.0, 1.0, -1.5, "Diagonal line (shifted)"),
    (-1.0, 1.0, 0.0, "Diagonal line (negative slope)"),
    (1.0, -1.0, 0.0, "Diagonal line (other direction)"),
]
for i, (w1, w2, b, description) in enumerate(configurations):
    model.set_weights(w1, w2, b)
    accuracy, preds = evaluate_on_xor(model)
```

The four settings score 75, 25, 75, and 75 percent. The code uses `> 0.5`, so a score exactly at zero belongs to class zero; the proof uses the opposite tie convention. Neither convention makes XOR separable. The full script tries ten hand-picked lines and one hundred random ones, and the best of the 110 gets three rows out of four. Notice what is absent. There is no optimizer and no loss in this script, and it needs only modules 01 through 03. The failure is not a training failure. No weight setting represents all four labels, so changing the training procedure cannot repair this model. The script checks every one of those 110 forward passes against the same arithmetic in NumPy, and it treats a line that scores all four rows as a bug: a single layer that "solves" XOR can only come from broken code, so the milestone exits with status 1.

The Idea

Pause & Predict: The Dimensionality of Separation

A single-layer perceptron operates in 2D space and fails on XOR because no single straight line can isolate the two positive diagonal points. If you project 2D coordinates into a 4D space using a linear layer without an activation, can a hyperplane separate XOR? Why does a non-linear activation like ReLU fundamentally change what is geometrically separable?

↪ **Jump ahead:** Chapter 13 *Scaling from 2-layer MLPs to deep, autoregressive Transformer foundation models.*

Why one line cannot do it

The proof fits in four lines. For the perceptron to get every row right, its weights must satisfy all four conditions at once:

$$
\begin{aligned}
(0,0) \to 0 &\implies b < 0 \\
(0,1) \to 1 &\implies w_2 + b \geq 0 \\
(1,0) \to 1 &\implies w_1 + b \geq 0 \\
(1,1) \to 0 &\implies w_1 + w_2 + b < 0
\end{aligned}
$$

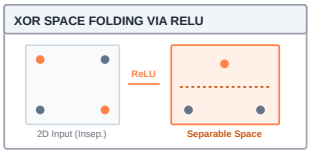
Adding the second and third lines gives $w_1 + w_2 + 2b \geq 0$. Adding the first and fourth gives $w_1 + w_2 + 2b < 0$. No numbers satisfy both, so no perceptron computes XOR. In the vocabulary of the field, XOR is not linearly separable.[37]

[37] **Linearly separable**: a labeled set of points is linearly separable when one straight line (in higher dimensions, one flat plane) puts every positive point on one side and every negative point on the other. AND and OR are linearly separable with two inputs; XOR is the smallest function that is not, which is why it became the test case.

A hidden layer folds the plane

The escape is to stop classifying the raw inputs and classify a transformed version of them instead. Insert a layer of neurons between input and output, each computing its own `Linear` sum followed by a ReLU, and let the output neuron draw its line in the space of those hidden values. Two hidden units are enough for XOR, and a specific pair makes the mechanism visible. Both read the same sum $s = x_1 + x_2$:

$$
h_1 = \text{ReLU}(s), \qquad h_2 = \text{ReLU}(s - 1), \qquad y = h_1 - 2h_2
$$



A hidden layer folds the 2D plane: ReLU projects non-separable XOR vertices into a separable 3-point geometry.

Unit h_1 switches on as soon as at least one input is on. Unit h_2 switches on only when both are. The output neuron rewards h_1 and punishes h_2 hard enough that the two cancel exactly when both inputs are on. Follow the four corners into the hidden plane and the fold appears. Corner $(0,0)$ lands at $(0,0)$, both positive corners land on the same point $(1,0)$, and corner $(1,1)$ lands at $(2,1)$. The two positive rows, which no line could isolate in the input square, are now a single point with the negatives on either side of it along the h_1 axis, and the line $h_1 - 2h_2 = 0.5$ separates them. ReLU did the work. A layer without it would compose two linear maps into one linear map, and the plane would stay flat.

How the hidden weights receive a learning signal

The hand-built weights establish that a solution exists. Training must find a solution using only the targets at the output. A hidden neuron has no label of its own, but its effect on the loss passes through the output layer. Chapter 6 supplies the mechanism: the tape records the hidden `Linear`, the ReLU, and the output `Linear` as ordinary operations. `loss.backward()` applies the chain rule along that path until every participating hidden weight has a gradient. The worked trace below follows one such gradient before relying on a full training run.

The Code

The XOR network and its data

The XOR truth table itself is a complete finite task, so fitting its four rows is a valid first integration check. The solved script adds a second demand: nearby inputs should receive the same labels. It draws twenty-five samples around each corner with Gaussian noise of standard deviation 0.1, giving one hundred points, then shuffles them. These samples encourage the decision boundary to hold over a neighborhood. Testing the exact corners checks the intended truth table, while a separate draw of noisy inputs would test generalization within the chosen noise model.

The model is the two-layer shape from The Idea with two changes from the hand-built construction. The hidden layer has four units rather than two. Two units suffice when choosing the weights analytically, but additional units give gradient descent more possible active paths from a random start; the next subsection shows what happens when the slack runs out. The hidden units use ReLU rather than the sigmoid of the 1986 paper, because Chapter 2 showed that a sigmoid passes at most a quarter of the gradient through and ReLU passes the gradient unchanged on its positive branch and blocks it on its negative branch. After the output layer comes a sigmoid, and the reason is the loss. Binary cross-entropy, the two-class form of Chapter 4's cross-entropy, scores a probability against a 0 or 1 target, and a `Linear` layer emits any real number, so the sigmoid is what turns the score into something the loss can read.

```
# adapted from milestones/02_1969_xor/02_xor_solved.py (corner loop and forward pass condensed)
import numpy as np
from tinytorch import Tensor, Linear, ReLU, Sigmoid, BinaryCrossEntropyLoss, SGD

rng = np.random.default_rng(7)

def generate_xor_data(n_samples=100):
    per_corner = n_samples // 4
    xs, ys = [], []
    for (a, b), target in [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 0)]:
        xs.append(rng.standard_normal((per_corner, 2)) * 0.1 + np.array([a, b]))
        ys.append(np.full((per_corner, 1), float(target)))
    X, y = np.vstack(xs), np.vstack(ys)
    order = rng.permutation(n_samples)
    return Tensor(X[order]), Tensor(y[order])

class XORNetwork:
    def __init__(self, hidden_size=4):
        self.hidden = Linear(2, hidden_size)
        self.relu = ReLU()
        self.output = Linear(hidden_size, 1)
        self.sigmoid = Sigmoid()

    def __call__(self, x):
        return self.sigmoid(self.output(self.relu(self.hidden(x))))

    def parameters(self):
        return self.hidden.parameters() + self.output.parameters()
```

The model is seventeen numbers: a 2 by 4 weight matrix and 4 biases in the hidden layer, a 4 by 1 weight matrix and 1 bias in the output layer. Notice `parameters()`. `XORNetwork` is a plain Python class, not a Chapter 3 `Layer`, so nothing walks its attributes for it; it has to hand the optimizer the concatenation of the two layers' lists itself. That list is all the optimizer ever learns about the model's structure. A production container can

```
>_ SOURCE CODE MAPPING                M-01
FILE:     milestones/02_1969_xor/ & 03_1986_mlp/
EXPORT:   tinytorch.milestones.xor_mlp
SYMBOLS:  MLP, XORNet, train_xor
Extracted & verified at build time    ✓ tested
```

Source Code Mapping: Milestone I combines Modules 01–07 to build and train a multi-layer perceptron from scratch.

discover these tensors automatically; the optimizer still receives parameter references, not a description of the forward computation.

The loop

The first call inside the loop is `model(X)`. Recording is already enabled because the `Linear` layer constructors initialize weights and biases with `requires_grad=True`. The optimizer holds references to the four parameter tensors and tracks their update buffers. Forward, loss, backward, step, and clear then follow the cycle from Chapter 8. The step that is easiest to drop is the clear. Chapter 6's `backward()` adds into `.grad` rather than assigning to it, so if the gradients are not cleared between one backward pass and the next, the next update uses accumulated gradients from different parameter values. That changes the update direction as well as its magnitude. Nothing needs to raise an error, so comparing one controlled update is a better diagnostic than waiting for the loss to fail.

The training function below writes that cycle directly. It returns the model for the next experiment; the full milestone script returns a history dictionary for its display. The XOR class exposes `__call__`, which this loop uses. `Trainer._forward` instead calls a model's `forward` method, so handing this exact class to `Trainer` would require providing that method. The manual loop makes this interface boundary visible.

```
def train_network(model, X, y, epochs=500, lr=0.5):
    loss_fn = BinaryCrossEntropyLoss()
    optimizer = SGD(model.parameters(), lr=lr)
    for epoch in range(epochs):
        predictions = model(X)                  # forward
        loss = loss_fn(predictions, y)          # loss
        loss.backward()                         # backward, through the
↪   hidden layer
        optimizer.step()                        # update
        optimizer.zero_grad()                   # clear, ready for the
↪   next epoch
        if (epoch + 1) % 100 == 0:
            accuracy = ((predictions.data > 0.5).astype(int) ==
↪   y.data).mean()
            print(f"epoch {epoch + 1:3d}  loss {float(loss.data):.4f}
            ↪   accuracy {accuracy:.0%}")
    return model

import tinytorch.core.layers as layers
layers.rng = np.random.default_rng(11)          # see below

X, y = generate_xor_data(100)
model = train_network(XORNetwork(hidden_size=4), X, y)
```

The trailing `zero_grad()` is valid here because the freshly constructed parameters begin with `grad=None`, and every iteration clears after its update. Calling this function on a model with existing gradients would require clearing before the first forward pass as well. The printed predictions and loss belong to the forward pass before the update; measuring the updated model would require another forward pass. The script also reseeds the random number generator that `Linear` uses for its initial weights.[38] Initialization determines which ReLU paths are initially active. With only four hidden

[38] **Seed**: the starting value of a pseudo-random number generator. Two runs with the same seed draw the same sequence of "random" numbers, which makes a training run reproducible; two runs with different seeds start from different initial weights and can end in different places.

units, some random starts put a unit permanently on the dead side of its ReLU, and a run can settle at 75 percent accuracy with one corner near probability 0.5. Inspecting hidden pre-activations and gradients distinguishes the dying ReLU failure from a broken backward path. The default seed, 11, is chosen so that the output layer alone cannot solve the task: its initial hidden features do not separate the four corners, so the hidden layer has to learn. The script therefore passes only when the trained model gets all four truth-table rows right, the hidden layer received a nonzero gradient, and the hidden weights moved by at least a quarter of their initial norm. A seeded run repeats exactly, so a failure at the default seed points at the code, not at luck. Only a run started with a different `--seed` can land on the 75 percent plateau with correct code, which happens for about one seed in six.

If all weights in a hidden layer are initialized to identical values (such as all zeros or all ones), every hidden unit computes the exact same forward pre-activation and receives the exact same gradient during `backward()`. The units update identically and remain clones forever, effectively reducing an N-unit hidden layer to a single unit.

Nonzero randomized initialization (such as He/Kaiming normal) is required to break permutation symmetry across neurons.

With that seed, the excerpt starts at loss 0.7468 and 38 percent accuracy, reaches 100 percent accuracy by epoch 100 at loss 0.1773, and ends after 500 epochs at loss 0.0060. Fed the four exact corners, the trained network answers 0.005, 0.996, 0.997, and 0.002. These values check that the seeded training path reaches a solution; the hand-built trace below explains why such a solution exists.

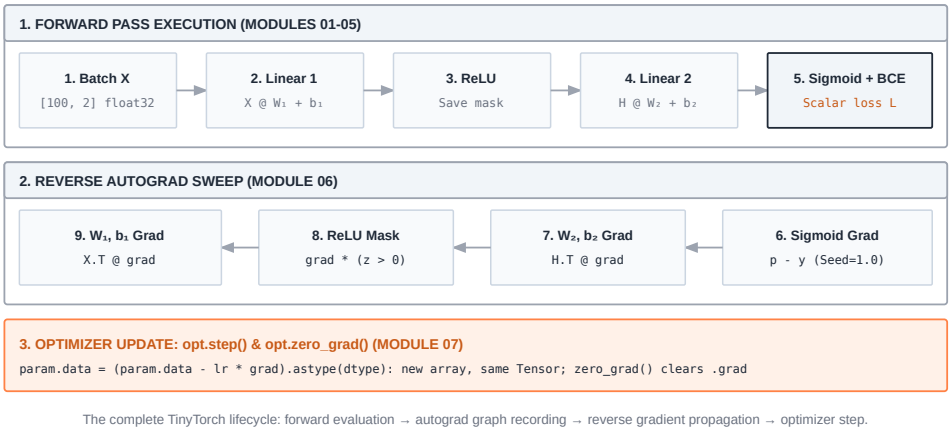


Figure 8.4: **One training iteration, step by step**: A schematic of the forward operations, the reverse vector-Jacobian products, and the optimizer update for the XOR network. It shows order and data dependencies, not measured time or memory.

Digits

The third script moves from a truth table to images, and three things have to change for the same two-layer shape to cope. TinyDigits ships with the repository as two pickled arrays: 1,000 training images and 200 test images in the bundled split used here, each 8 by 8 pixels with values normalized to the interval from 0 to 1, labeled 0 through 9. An 8 by 8 image flattens to a 64-vector, the hidden layer has 32 units, and the output has 10, one per digit.

The first change is the output. A digit classifier has ten answers, not two, and the two-class trick of one sigmoid and one probability does not stretch to cover it; ten separate sigmoids would give ten probabilities that need not sum to one and ten losses with nothing tying them together. Chapter 4's `CrossEntropyLoss` takes the ten raw scores and an integer label and does the softmax and the logarithm inside, with the max shift that keeps a score of 100 from overflowing float32. The model therefore emits ten raw scores, there is no sigmoid, and the prediction is the index of the largest score.

The second change is how the data arrives. XOR feeds all one hundred points through the network at once, giving one gradient step per epoch. With 1,000 digit images, `DataLoader` batches of 32 produce 32 steps per epoch: 31 full batches and a final batch of eight. Each `CrossEntropyLoss` averages over the current batch, including that shorter final batch. Twenty epochs therefore make 640 updates. The loader changes how often parameters move without changing the model's forward interface.

The third change is what we measure. Training loss checks whether updates fit the examples presented to the optimizer. Held-out accuracy checks predictions on other examples from the same dataset. Neither substitutes for the other, and a high held-out score on a small dataset does not establish general digit recognition. The listing reports held-out accuracy so the two roles stay distinct; repeatedly tuning settings to this split would turn it into validation data and require a fresh test split.

```
# adapted from milestones/03_1986_mlp/01_rumelhart_tinydigits.py (loading, evaluation, and display condensed)
import pickle
import numpy as np
from tinytorch import Tensor, Linear, ReLU, CrossEntropyLoss, SGD
from tinytorch.core.dataloader import TensorDataset, DataLoader

def load_split(name):                                   # paths relative to the repo root
    with open(f"datasets/tinydigits/{name}.pkl", "rb") as f:
        d = pickle.load(f)                              # {"images": (N, 8, 8), "labels": (N,)}
    return Tensor(d["images"].astype(np.float32)),
        Tensor(d["labels"].astype(np.int64))

train_images, train_labels = load_split("train")
test_images, test_labels = load_split("test")

class DigitMLP:
    def __init__(self, input_size=64, hidden_size=32, num_classes=10):
        self.fc1 = Linear(input_size, hidden_size)
        self.relu = ReLU()
        self.fc2 = Linear(hidden_size, num_classes)

    def __call__(self, x):
        if len(x.data.shape) > 2:                       # (N, 8, 8) -> (N, 64)
            x = x.reshape(x.shape[0], -1)
        return self.fc2(self.relu(self.fc1(x)))

```

```
    def parameters(self):
        return [self.fc1.weight, self.fc1.bias, self.fc2.weight,
        ↪   self.fc2.bias]

def evaluate_accuracy(model, images, labels):
    predictions = np.argmax(model(images).data, axis=1)
    return 100.0 * (predictions == labels.data).mean()

model = DigitMLP()
train_loader = DataLoader(TensorDataset(train_images, train_labels),
                          batch_size=32, shuffle=True)
loss_fn = CrossEntropyLoss()
optimizer = SGD(model.parameters(), lr=0.01)

for epoch in range(20):
    for batch_images, batch_labels in train_loader:
        logits = model(batch_images)
        loss = loss_fn(logits, batch_labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    print(f"epoch {epoch + 1:2d}  test accuracy
    ↪   {evaluate_accuracy(model, test_images, test_labels):.1f}%")
```

`DigitMLP.__call__` changes an `(N, 8, 8)` batch to `(N, 64)` before the first `Linear`. The reference `Tensor.reshape` wraps the NumPy result in a new Tensor, which owns copied float32 storage. It also records the reshape when a gradient connection is needed. Reconstructing a Tensor directly from `.data` would discard that connection; using the Tensor operation keeps the example safe if a trainable image transform later precedes the classifier. `parameters()` returns the four original parameter tensors, so an update changes the same weights the next forward pass reads.

The model has $64\times32+32+32\times10+10 = 2{,}410$ parameters, or 9,640 bytes of float32 parameter data. The bundled training images occupy $1{,}000 \times 64 \times 4 = 256{,}000$ bytes before Tensor and Python overhead. Dataset variants and random initialization change the learning curve, so the split sizes can be checked directly with `train_images.shape` and `test_images.shape`. A useful run records those sizes, the seed, the loss, and held-out accuracy. The invariant under test is that batches reach the loss and the resulting gradients reach both layers, not that every run attains a particular percentage. The script still sets a floor. It exits with status 1 when test accuracy falls below 75%, well under the roughly 82% a working implementation reaches.

A Worked Trace

The hand-built two-unit network from The Idea is small enough to run in your head, and running it is the fastest way to see the fold. The listing is plain Python with no framework, so it runs anywhere.

```
corners = [(0, 0), (0, 1), (1, 0), (1, 1)]
targets = [0, 1, 1, 0]
```

```
3
4  def relu(v):
5      return v if v > 0 else 0.0
6
7  for (x1, x2), t in zip(corners, targets):
8      s = x1 + x2                        # both hidden units read the same
   ↪  sum
9      z1, z2 = s, s - 1.0          # pre-activations
10     h1, h2 = relu(z1), relu(z2)        # the fold
11     y = h1 - 2 * h2                # output neuron
12     print(f"x=({x1},{x2})  z=({z1:+.1f},{z2:+.1f})
       ↪   h=({h1:.1f},{h2:.1f})  y={y:.1f}  target={t}")
```

In lines 1 to 2, `corners` and `targets` define the four XOR vertices. Inside the loop (lines 7 to 12), line 8 forms $s = x_1 + x_2$ and line 9 computes the two pre-activations $z_1 = s$ and $z_2 = s - 1.0$. Line 10 applies the ReLU activation (h_1, h_2), creating the space-fold where $(0, 1)$ and $(1, 0)$ both collapse to $(1.0, 0.0)$. Line 11 combines the hidden units via $y = h_1 - 2h_2$, and line 12 prints the output matching table 8.7.

Table 8.7: **A constructed XOR solution**: The hand-built two-unit network on the four XOR corners. The two positive rows collapse onto the same hidden point.

Input (x_1, x_2)	Target	Pre-activation (z_1, z_2)	Hidden (h_1, h_2)	Output $y = h_1 - 2h_2$
$(0, 0)$	0	$(0.0, -1.0)$	$(0.0, 0.0)$	0.0
$(0, 1)$	1	$(1.0, 0.0)$	$(1.0, 0.0)$	1.0
$(1, 0)$	1	$(1.0, 0.0)$	$(1.0, 0.0)$	1.0
$(1, 1)$	0	$(2.0, 1.0)$	$(2.0, 1.0)$	0.0

The hidden column in table 8.7 matches the construction in figure 8.3. Rows two and three are identical after the ReLU, and row four is the only one where h_2 is nonzero, so the output neuron only has to subtract it away. Every number is exact, which is what makes this a good first check when your own `Linear` and `ReLU` produce something surprising.

The hand-built fold establishes representational capacity. A separate one-example backward trace establishes how a hidden weight receives credit. In lines 1 to 5 of the backward script below, `trace_model` is configured with deterministic weights ($W_h = \mathbf{1}_{2\times 2}$, $W_o = [1.0, -1.0]^T$) and zero biases. Lines 6 to 7 prepare the optimizer and clear gradients. In line 8, binary cross-entropy loss is computed on input `[[1.0, 0.0]]` with target `[[1.0]]`. Line 9 calls `loss.backward()`, propagating through the linear-ReLU-linear graph. Lines 10 to 11 print the resulting weight gradients: `output.weight.grad` is `[[-0.5], [-0.5]]`, and `hidden.weight.grad` is `[[-0.5, 0.5], [0.0, 0.0]]`. Line 12 performs the parameter update `optimizer.step()`, and line 13 confirms that the first row of hidden weights adjusts to `[[1.05, 0.95], [1.0, 1.0]]`.

```
1  trace_model = XORNetwork(hidden_size=2)
2  trace_model.hidden.weight.data[:] = [[1.0, 1.0], [1.0, 1.0]]
3  trace_model.hidden.bias.data[:] = 0.0
4  trace_model.output.weight.data[:] = [[1.0], [-1.0]]
```

```
trace_model.output.bias.data[:] = 0.0
optimizer = SGD(trace_model.parameters(), lr=0.1)
optimizer.zero_grad()
loss = BinaryCrossEntropyLoss()(trace_model(Tensor([[1.0, 0.0]])),
    Tensor([[1.0]]))
loss.backward()
print(trace_model.output.weight.grad)       # [[-0.5], [-0.5]]
print(trace_model.hidden.weight.grad)       # [[-0.5, 0.5], [0.0, 0.0]]
optimizer.step()
print(trace_model.hidden.weight.data)       # [[1.05, 0.95], [1.0,
    1.0]]
```

The output matmul multiplies the incoming -0.5 gradient by its saved weights, yielding hidden gradients of -0.5 and $+0.5$. Both ReLU inputs are positive, so both gradients pass through. The hidden matmul forms the outer product with the saved input $(1, 0)$: the first row updates while the second receives zero. The hidden layer needs no target of its own; its derivative follows the exact path influencing the supervised output. Increasing the first hidden activation raises the output through its positive weight, whereas decreasing the second raises it through its negative weight. Gradient descent applies both adjustments, verifying graph composition and parameter identity before extended training.

The trained four-unit network finds a fold of its own. Feeding the four exact corners through the seeded model gives the hidden activations in table 8.8; they differ from the hand-built construction while preserving its labels.

Table 8.8: **A learned XOR solution**: Hidden activations of the trained four-unit XOR network at the four corners. The output score is $z = \mathbf{w}_2 \cdot \mathbf{h} + b_2$ with learned weights $\mathbf{w}_2 = (3.59, -4.20, 2.40, -2.98)$ and bias $b_2 = -0.78$.

Input	Hidden (h_1, h_2, h_3, h_4)	Output score z	Probability $\sigma(z)$
$(0, 0)$	$(0.00, 0.00, 0.21, 1.71)$	-5.38	0.005
$(0, 1)$	$(0.00, 0.00, 2.66, 0.00)$	$+5.60$	0.996
$(1, 0)$	$(2.45, 0.38, 0.00, 0.16)$	$+5.92$	0.997
$(1, 1)$	$(0.00, 2.65, 2.44, 0.00)$	-6.04	0.002

The learned units have readable jobs. Unit 1 responds to $(1, 0)$ and unit 3 most strongly to $(0, 1)$, and both carry positive output weights. Unit 3 also fires for $(1, 1)$, so unit 2, which fires almost only for $(1, 1)$ and carries the largest negative weight, is how the network cancels that row. Unit 4 fires mainly for $(0, 0)$ and is also weighted negative. Nobody told the network to decompose XOR this way. It is one of many folds that work, and a different seed finds a different one.

From TinyTorch to Production

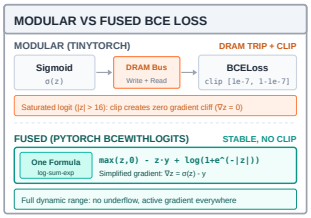
The corresponding PyTorch experiment uses a container to discover its parameters and a fused loss to consume raw scores. The update loop remains explicit. The conversion below copies the earlier TinyTorch XOR tensors into PyTorch tensors before running the comparison.

```python
import torch, torch.nn as nn

X_torch = torch.tensor(X.data, dtype=torch.float32)
y_torch = torch.tensor(y.data, dtype=torch.float32)
model = nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1))
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
for epoch in range(500):
    optimizer.zero_grad()
    loss = loss_fn(model(X_torch), y_torch)
    loss.backward()
    optimizer.step()
```

Table 8.9: **The same update contract in two frameworks**: The XOR and digits experiments in TinyTorch and in PyTorch.

Piece	TinyTorch milestone	PyTorch
Model container	`XORNetwork` class with a hand-written `parameters()`	`nn.Sequential`; `parameters()` walks the module tree
Output and loss	`Sigmoid` then `BinaryCrossEntropyLoss` with the probability clipped to $[10^{-7}, 1-10^{-7}]$	`BCEWithLogitsLoss`, sigmoid and loss fused into one numerically stable function
Weight init	LeCun-scaled normal, seeded from a module-level generator	Uniform within bounds set by input width; seeded by `torch.manual_seed`
Loop	Five steps written by hand	Five steps written by hand
Batches	`DataLoader` over in-memory arrays, one process	`DataLoader` can configure workers and prefetching
Arithmetic	NumPy on the CPU	CPU, or GPU kernels launched per operation



Modular vs. Fused Loss: Separating Sigmoid and BCELoss writes an intermediate probability tensor and clips gradients, whereas BCEWithLogits evaluates one log-sum-exp formula that stays finite without clipping.

The loss comparison in table 8.9 reveals a crucial trade-off between pedagogical modularity and systems performance. To maintain clear abstraction boundaries, TinyTorch separates the `Sigmoid` activation from the `BCELoss` module. However, executing them as separate operations introduces two significant runtime vulnerabilities:

1. **Numerical instability**: Evaluating $\sigma(z) = 1/(1+e^{-z})$ directly in float32 causes exponentials to saturate for large magnitudes, forcing TinyTorch to manually clip probabilities to $[10^{-7}, 1-10^{-7}]$ to prevent $\log(0)$ crashes. While clipping avoids `NaN`, it creates zero gradients on clipped activations, permanently stalling weight updates in saturated regimes.
2. **Intermediate memory traffic**: Separating the operations writes an intermediate probability tensor of shape $(B, 1)$ out to main memory, only to immediately read it back across the bus into the loss kernel.

Production engines address these issues by merging the two operations: `torch.nn.BCEWithLogitsLoss` combines the sigmoid and binary cross-entropy

algebra into a single numerically stable expression:

$$\mathcal{L} = \max(z, 0) - z \cdot y + \log(1 + e^{-|z|})$$

This formulation keeps every term finite without epsilon clipping and simplifies the gradient to $\sigma(z) - y$; the PyTorch loss documentation attributes its advantage to this log-sum-exp stability. Whether the operations also run as one kernel depends on the backend and compiler.

Cybenko (1989) and Hornik (1991) proved that single-hidden-layer feedforward networks with non-linear activations can approximate any continuous function on compact subsets of $\mathbb{R}^n$ arbitrarily well.

However, the theorem guarantees *representational existence*, not *efficient learnability*: an MLP may require exponentially many hidden neurons ($O(2^n)$) to approximate functions that deep, structured architectures learn with polynomial parameter counts using gradient descent.[39]

The core training contract remains invariant: forward passes evaluate parameters, backward passes compute gradients, and optimizers apply updates. Production frameworks alter batching, scheduling, or memory layouts while preserving this contract. When training stalls, systematic diagnosis precedes structural changes: inspect hidden preactivations, verify backward propagation with deterministic traces, and test initialization seeds before adjusting hyperparameters.

[39] While the Universal Approximation Theorem guarantees uniform approximation of continuous functions by single-hidden-layer networks, Barron (1993) showed that for functions with a bounded Fourier-moment (Barron) norm, a one-hidden-layer sigmoidal network with n units reaches squared error $O(1/n)$ independent of the input dimension d, whereas any linear combination of n fixed basis functions cannot beat order $(1/n)^{2/d}$. Outside that function class, the width a shallow network needs can grow exponentially with d.

Summary

Eight core modules now participate in a single unified training loop. The XOR experiment shows how multi-layer networks solve linearly inseparable tasks via non-linear space folding, while the backward trace verifies exact gradient flow across layers without explicit intermediate targets. In Chapter 13, we discover that the two-layer multi-layer perceptron built in this synthesis never disappears in modern foundation models: it serves as the Feedforward Network (FFN) sublayer in every transformer block, projecting token representations into expanded dimensions before non-linear gating. Chapter 9 preserves this identical training contract while introducing spatial weight sharing for images.

Further Reading

- **Rosenblatt** (1958). "The perceptron: a probabilistic model for information storage and organization in the brain." *Psychological Review*. The single-layer model this milestone starts from.
- **Minsky and Papert** (1969). *Perceptrons: An Introduction to Computational Geometry*, MIT Press. The analysis of what a single linear boundary cannot represent, XOR included.
- **Rumelhart, Hinton, and Williams** (1986). "Learning representations by back-propagating errors." *Nature*. Backpropagation through a hidden layer, the method that solves XOR here.

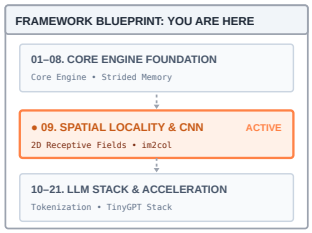
PART II

Deep Architectures

🔥 Chapter 9

Convolutions: Sliding Windows and the im2col Trick

Visual and spatial data exhibit two fundamental physical properties: spatial locality and translation equivariance. Neighboring pixels share strong local correlations, and a visual feature, such as an edge, corner, or texture, maintains identical semantic meaning regardless of where it appears in an image. Fully connected dense layers ignore this spatial structure: flattening an image into a 1D vector gives every pixel its own weight to every hidden unit, causing parameter counts to explode with image size while discarding 2D geometry.



Framework Blueprint: Module 09 implements convolution and pooling as sliding-window loops inside autograd; the chapter models the im2col lowering to GEMM that Chapter 18 builds.

In PyTorch, applying a 2D convolution is expressed in a concise module declaration:

```
import torch
import torch.nn as nn

# 1 input channel, 4 filters of size 3x3, stride 1, padding 1
conv = nn.Conv2d(in_channels=1, out_channels=4, kernel_size=3,
    padding=1)
images = torch.randn(8, 1, 28, 28)  # Batch of 8 single-channel images
feature_maps = conv(images)

print("Weight shape:", conv.weight.shape)       # (4, 1, 3, 3)
print("Output shape:", feature_maps.shape)     # (8, 4, 28, 28)
```

Underneath that simple call, the runtime orchestrates several complex systems mechanisms:

1. **Drastic parameter reduction through weight sharing:** Instead of learning separate weights for every pixel coordinate, the layer slides a tiny (3×3) filter bank across the entire image. Four filters across one channel require only 36 weight floats (plus 4 biases), slashing parameter memory by orders of magnitude.
2. **The `im2col` lowering transformation:** While nested loops sliding across 2D patches are conceptually straightforward, they are slow on hardware. Production frameworks extract overlapping 2D spatial patches into a large column matrix (`im2col`), lowering the 4D convolution into a single high-throughput General Matrix Multiply (GEMM).
3. **Autograd gradient scattering:** During backpropagation, output gradients at every spatial coordinate accumulate back into the shared kernel weights, while gradients scatter backward across overlapping input receptive fields.

This chapter expands TinyTorch into spatial representation learning. It implements 2D convolution and max pooling as direct loops inside the autograd framework, then lowers

the convolution to a matrix multiply with `im2col`. Before building these kernels, we must inspect the parameter explosion that occurs when dense layers are applied directly to images.

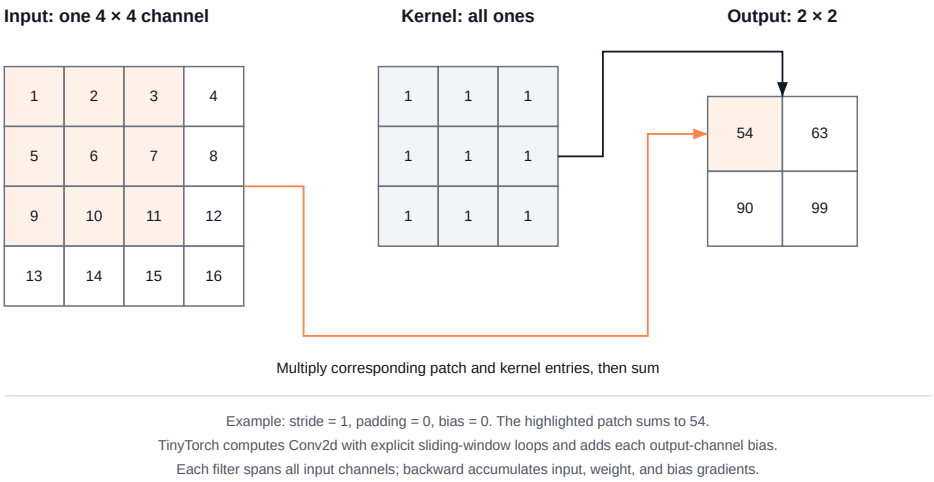


Figure 9.1: **One filter, four windows**: The highlighted input patch supplies the first output. Moving the same kernel across the remaining positions produces the other three values.

9.1 The Problem

↩ **Jump back:** Chapter 3 *The `im2col` lowering transformation converts spatial sliding windows into standard GEMM operations.*

Take a modest color photo, 224×224 pixels with three color channels. Flattened, that is $224 \times 224 \times 3 = 150{,}528$ input numbers. A `Linear` layer from those inputs to 1,024 hidden units needs a weight for every (input, output) pair, so its weight matrix has $150{,}528 \times 1{,}024 \approx 154$ million entries. In float32 that is about 617 MB for the first layer alone, before a single deeper layer or a single gradient buffer. Napkin math, but the conclusion holds at any reasonable image size. A dense first layer on raw pixels is enormous.

Size is the smaller of the two problems. The larger one is that a `Linear` layer has no idea the input was ever a picture. Every input index is just a column of W. A fixed permutation of the input pixels can be undone by applying the matching permutation to the columns of its weight matrix. The set of functions the layer can represent is unchanged, because nothing in $XW^T + b$ encodes that pixel $(3, 4)$ is next to pixel $(3, 5)$. The layer has to discover neighborhoods from data, and it has to discover them separately at every location. A vertical edge in the top-left corner and the same edge shifted ten pixels right touch disjoint sets of weights, so the network learns the detector twice, or as many times as there are places an edge can appear.

We want a layer that spends its weights on *what* a pattern looks like, not on *where* it is.

9.2 The Idea

Pause & Predict: Parameter Sharing vs. Dense Connections

A dense linear layer connecting a $224 \times 224 \times 3$ image to 64 feature maps requires over 9.6 million weights. A 3×3 convolutional layer producing the same 64 feature maps requires only 1,728 weights, regardless of image size. If both layers can perform linear operations on the pixels, what structural assumption enables the convolution to discard 99.98% of the weights? What happens when that assumption fails (for example, if image pixels are randomly scrambled)?

↪ **Jump ahead:** Chapter 18 *How hardware accelerators optimize convolution memory layouts and high-throughput GEMM tiling.*

9.2.1 A filter that slides

Standard vision backbones from AlexNet and ResNet to modern ConvNeXt models process images using spatial convolutions rather than dense connections. By constraining interactions to local neighborhoods and reusing the same kernel across all spatial coordinates, convolutions drastically reduce parameter counts while baking in translation equivariance. A **convolution** replaces the giant weight matrix with a small grid of weights, called a **filter** (or kernel), typically 3×3 or 5×5. The filter is placed over the top-left corner of the image, the overlapping pixels are multiplied by the matching filter weights, the products are summed, and the sum becomes one output pixel. Then the filter slides one step to the right and the same weights produce the next output pixel, and so on across and down the image.

With input X of height H and width W, a $K \times K$ filter w, and a scalar bias b, the output at row h and column c is

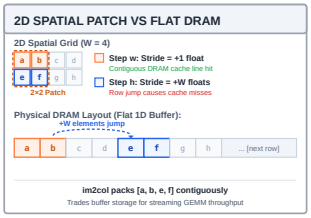
$$Y[h, c] = b + \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} X[h + i,\ c + j] \cdot w[i, j].$$

Each output pixel sees only a $K \times K$ neighborhood of the input, its **receptive field**, and every output pixel is computed with the same $K^2 + 1$ numbers. That sharing is the whole point. At stride 1, shifting the input shifts the output correspondingly wherever the compared windows do not encounter a changed boundary. Finite-image padding and larger strides limit this correspondence. This property is called **translation equivariance**, and it means a filter that learns to detect a vertical edge detects it everywhere at no extra cost in parameters.

9.2.2 Channels, padding, stride

In production convolutional backbones such as ResNets and U-Nets, feature extractors expand channel capacity while managing spatial dimensionality through geometric padding and strides. These hyperparameters govern how spatial resolution shrinks through the network hierarchy while preventing border information loss. Real images have several channels (red, green, blue), and a useful layer produces several output planes, called **feature maps**, one per filter. This non-contiguous row striding is the 2D spatial manifestation of the stride arithmetic introduced in Chapter 1: whereas horizontal steps along width exploit consecutive 64-byte cache line loads with unit stride, vertical steps jump by entire row strides, causing cache line eviction if rows exceed L1 cache capacity. TinyTorch, like PyTorch, stores an image batch as a 4D tensor with shape $[N, C, H, W]$, meaning N images, each with C channels of $H \times W$ pixels. A filter then

spans all input channels, so a layer with C_{in} input channels and C_{out} filters holds a weight tensor of shape $[C_{\text{out}}, C_{\text{in}}, K, K]$ plus one bias per filter. Output pixel (h, c) of filter o sums over the input channels too:



Spatial Locality vs. Strided DRAM: Stepping along width accesses contiguous memory, while stepping vertically jumps W elements. The im2col transformation unrolls patches to restore contiguous vector loads.

$$Y[n, o, h, c] = b[o] + \sum_{\text{ch}=0}^{C_{\text{in}}-1} \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} X[n, \text{ch}, hS + i,\ cS + j] \cdot W[o, \text{ch}, i, j].$$

The channel formula indexes the padded input when padding is enabled. Two choices determine which windows it visits. **Padding** P adds a border of P zero pixels around each input plane, allowing windows to extend beyond its edges. The amount of padding determines whether the output shrinks, stays the same size, or grows. **Stride** S is how far the filter jumps between output pixels; a stride of 2 visits every other position. The number of filter positions that fit along one axis, and therefore the output size, is

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1,$$

and the same for width. The formula is worth memorizing, because it is the source of most shape errors in convolutional networks. For a 3×3 filter, padding 1 and stride 1 keep the size $(32 \to 32)$, padding 0 shrinks it by two $(32 \to 30)$, and stride 2 with padding 0 roughly halves it $(32 \to 15)$.

Count the parameters. A 3×3 layer taking 3 channels to 64 filters holds $64 \times 3 \times 3 \times 3 + 64 = 1{,}792$ numbers, and it works on a 224×224 photo or a 2000×2000 one without changing. The dense layer from The Problem needed 154 million and was tied to one image size.

9.2.3 Max pooling

Pioneered in architectures like AlexNet and VGG, spatial pooling downsamples intermediate feature maps to compress memory footprint and compute costs while conferring local translation invariance. By aggregating regional activations without introducing learnable parameters, pooling layers aggressively widen the network's effective receptive field. **Max pooling** is the second layer we build, and it shrinks the planes a convolution produces. A 2×2 window with stride 2 slides over each channel independently and keeps only the largest value in each window, halving both height and width. It has no weights. It also makes the network a little tolerant to small shifts, since moving a strong activation one pixel within its window does not change the pooled output. The output-size formula is the same one as for convolution.

9.2.4 How far a stack of layers can see

Deep vision backbones rely on hierarchical feature abstraction, where higher layers must integrate context across entire objects or scenes rather than isolated edges. Calculating the theoretical receptive field ensures that deep networks built from small, compute-efficient 3×3 kernels achieve sufficient global context without blowing up parameter counts. One convolution's output pixel sees a $K \times K$ patch. Stack layers and the patch grows, because each output of a later layer combines several outputs of the earlier one. A layer adds $K - 1$ positions of reach at its own depth, and one position at that depth spans as many input pixels as the product of the strides below it. The **receptive field** r_ℓ of layer ℓ, measured in input pixels along one axis, therefore follows

$$r_\ell = r_{\ell-1} + (K_\ell - 1)\prod_{i<\ell} S_i, \qquad r_0 = 1.$$

Three stacked 3×3 convolutions at stride 1 see 3, then 5, then 7 pixels. One 7×7 filter would see the same patch with 49 weights per pair of input and output channels, where the stack uses $3 \times 9 = 27$ and applies a nonlinearity between each layer; that trade is the design argument behind the VGG family of networks. Pooling layers count as well. Synthesis II's network runs a 3×3 convolution and then 2×2 pooling at stride 2, so its receptive field is 3 after the convolution and $3 + (2-1) \cdot 1 = 4$ after the pool. A 3×3 convolution placed after that pool would add $(3-1) \cdot 2 = 4$ more, reaching 8 of the image's 8 pixels.

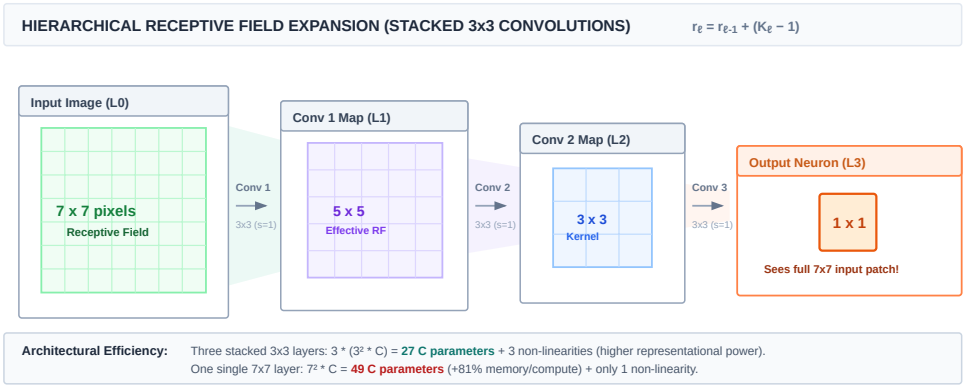
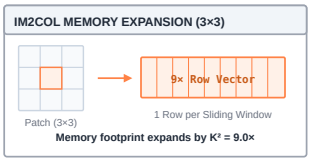


Figure 9.2: **Hierarchical receptive field expansion in stacked convolutions**: A single output neuron at Layer 3 integrates information across an effective 7×7 spatial patch of the input image through intermediate 3×3 kernel stages. Stacking three 3×3 convolutions uses only 27 parameter weights per channel pair compared to 49 for a single 7×7 filter, while introducing three non-linearities.

9.2.5 The im2col trick

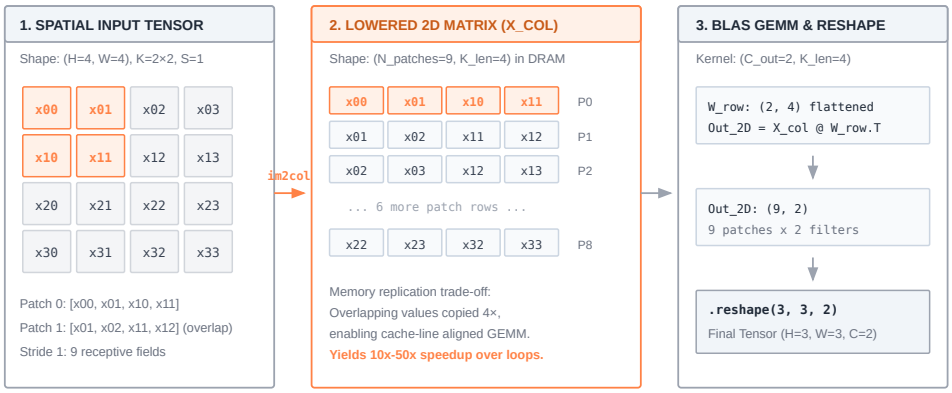
Production deep learning runtimes like cuDNN and oneDNN rarely execute 2D convolutions using nested spatial loops, which suffer from severe memory striding and poor arithmetic intensity. Instead, frameworks lower convolutions into a high-throughput General Matrix Multiply (GEMM) by unrolling receptive fields into a 2D patch matrix. Look again at the single-output formula. Each output pixel is a dot product between two lists of $C_{\text{in}}K^2$ numbers, the flattened patch under the filter and the flattened filter. That means the entire layer is a matrix multiply in disguise. Write every patch the filter visits as one row of a matrix, with $N \cdot H_{\text{out}} \cdot W_{\text{out}}$ rows of length $C_{\text{in}}K^2$, and write every filter as one column of a second matrix, with $C_{\text{in}}K^2$ rows and C_{out} columns. Their product has one row per output position and one column per filter, and reshaping it gives the $[N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}}]$ output. Building the patch matrix is called **im2col** ("image to column"), and the four steps are

1. unroll every $K \times K \times C_{\text{in}}$ patch into a row of X_{col}, shape $[NH_{\text{out}}W_{\text{out}},\ C_{\text{in}}K^2]$;
2. flatten every filter into a column of W_{col}, shape $[C_{\text{in}}K^2,\ C_{\text{out}}]$;
3. compute $Y_{\text{col}} = X_{\text{col}}W_{\text{col}}$, shape $[NH_{\text{out}}W_{\text{out}},\ C_{\text{out}}]$;
4. add the bias and reshape to $[N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}}]$.



im2col unrolls spatial 2D patches into GEMM matrix rows, expanding memory footprint by $K_h \times K_w = 9\times$.

The matrix multiplication performs the same dot products. In Chapter 18, we optimize this lowered GEMM operation using 2D cache tiling and SIMD vectorization, showing how hardware hierarchies transform memory-bound unrolled loops into peak arithmetic throughput. The Code section counts the extra storage and separates that analytical cost from elapsed time, which depends on the workload and machine.



The im2col lowering transforms spatial 2D sliding windows into contiguous row vectors, trading memory for peak BLAS GEMM FLOPs.

Figure 9.3: **Lowering 2D convolution to GEMM via im2col**: Sliding receptive field patches of shape $C_{\text{in}} \times K \times K$ are unrolled into rows of an input matrix X_{col}, while filters are flattened into columns of W_{col}. High-performance BLAS matrix multiplication computes all overlapping dot products simultaneously, which is then reshaped into the 4D output tensor.

9.2.6 The backward pass

A single term makes the backward rule concrete. If a weight $w = 2$ multiplies an input $x = 3$ and the output receives gradient $g = 4$, the contribution to the weight gradient is $gx = 12$ and the contribution to the input gradient is $gw = 8$. Convolution repeats this product rule across its windows and adds contributions wherever an input or shared weight appears again. Call the incoming gradient $G = \partial \mathcal{L}/\partial Y$, with the same shape as the output. Since Y is a sum of terms $X[\ldots] \cdot W[\ldots]$, each output pixel contributes its gradient value times the input pixel it multiplied to the weight's gradient, and its gradient value times the weight to the input pixel's gradient:

$$\frac{\partial \mathcal{L}}{\partial W[o, \text{ch}, i, j]} = \sum_{n,h,c} G[n, o, h, c] \cdot X[n, \text{ch}, hS + i, cS + j],$$

$$\frac{\partial \mathcal{L}}{\partial b[o]} = \sum_{n,h,c} G[n, o, h, c].$$

The input gradient is the mirror image, scattering $G[n, o, h, c] \cdot W[o, \text{ch}, i, j]$ back onto input pixel $(hS + i, cS + j)$. The window loops accumulate input and weight gradients; a separate reduction sums the bias gradient.[40] Max pooling has no weights, and its gradient goes entirely to the one input pixel that won each window; the others receive zero.

[40] In lowered matrix formulations, input gradient accumulation is computed via the **col2im** operation, the adjoint of im2col. Because sliding windows overlap, col2im cannot simply reshape G_{col}; it must accumulate overlapping entries using a scatter-add $(+=)$ reduction into the spatial input buffer. This is the spatial manifestation of the multivariate chain rule ("Used twice means add") from Chapter 6: an input pixel shared across multiple receptive fields must sum their incoming adjoints.

9.3 The Code

Consider `conv = Conv2d(1, 2, 3, stride=2, padding=1)` applied to an input of shape `(1, 1, 4, 4)`. Calling `conv(x)` enters `Conv2d.__call__`, then `forward`. The layer validates the four axes and computes the spatial output size before calling `Conv2dFunction.apply(x, weight, bias, layer=self)`. The two filters have shape `(2, 1, 3, 3)`, and the result will have shape `(1, 2, 2, 2)`. The batch axis survives; the output-channel axis counts filters; the final two axes count valid window positions.

`apply` is the boundary established in Chapter 6. It gives `Conv2dFunction.forward` NumPy arrays for the arithmetic while retaining the input tensors as the parents of the output. The operation calls the layer's shape, padding, and convolution helpers through `self.layer`. Each helper can therefore be tested independently, yet the complete call produces a tensor whose gradient record still points to the image, weights, and bias. Calling a helper directly and wrapping its array in a new `Tensor` would preserve the output values but omit that record.

9.3.1 Output size and padding

Before any filter slides, the layer must settle its output dimensions and its border behavior.

The size question comes first because every layer allocates its output before it computes a single value, and the layer after it needs that shape to allocate its own. Get it wrong by one and the network fails three layers later with an error that names the wrong layer. `_compute_output_shape` is the formula from The Idea, and `forward` rejects a nonpositive spatial output before entering the loops. A mismatched input-channel count also raises at this boundary, so a caller gets an error at the layer responsible for the mismatch.

```
>_ SOURCE CODE MAPPING                                MOD 09

FILE:     src/09_convolutions/09_convolutions.py
EXPORT:   tinytorch.core.spatial
SYMBOLS:  Conv2d, MaxPool2d, im2col

Extracted & verified at build time                    ✓ tested
```

Source Code Mapping: Module 09 extracts Conv2d spatial convolution, im2col buffer unrolling, and MaxPool2d downsampling directly from the working repository.

```python
def _compute_output_shape(self, in_h: int, in_w: int) -> tuple[int, int]:
    """
    Calculate output spatial dimensions for convolution.
    """
    kernel_h, kernel_w = self.kernel_size
    stride_h, stride_w = self.stride
    out_height = (in_h + 2 * self.padding - kernel_h) // stride_h + 1
    out_width = (in_w + 2 * self.padding - kernel_w) // stride_w + 1
    return out_height, out_width
```

The border question is what padding is for. A 3×3 filter with no padding shrinks the image by two pixels per layer, so a 32×32 input is 12×12 after ten layers, and the pixels near the edge get less attention than the ones in the middle (a corner pixel is covered by exactly one filter position, a center pixel by nine). For this kernel, padding 1 preserves the output size and lets windows include every edge pixel, although boundary pixels still appear in fewer windows than interior pixels. The naive way to handle a border is a bounds check inside the innermost loop, an `if` on every one of the roughly 87 million multiplies a 224×224 color photo needs from a single 64-filter layer. The better way is to build the padded image once, before the loops, and then let every loop run as if there were no edge at all.

```
def _apply_padding(self, x_data: np.ndarray) -> np.ndarray:
    """
    Zero-pad the spatial dimensions of the input numpy array.
    """
    if self.padding > 0:
        return np.pad(x_data,
                      ((0, 0), (0, 0),
                       (self.padding, self.padding),
                       (self.padding, self.padding)),
                      mode='constant', constant_values=0)
    else:
        return x_data
```

Only the two spatial axes get padded; batch and channel are untouched. With positive padding, `np.pad` creates a separate array and leaves the caller's input unchanged. With padding 0, the helper returns the input array itself, which the convolution only reads. The loop writes into a separate output array; padding is temporary storage for this call, not a trainable parameter.

9.3.2 The direct convolution

The direct implementation is a readable reference for every alternative. The im2col model and gradient checks below must reproduce its values. For one 224×224 image through `Conv2d(3, 64, 3, padding=1)`, the loops perform $224 \times 224 \times 64 \times 3 \times 3 \times 3 = 86{,}704{,}128$ multiply-adds. Four loops pick an output position; three more form its dot product. This count describes work, not elapsed time. Python indexing and loop overhead make the implementation useful for tracing small examples rather than predicting production throughput.

```
def _convolve_loops(self, padded: np.ndarray, batch_size: int, out_h:
↪   int, out_w: int) -> np.ndarray:
    """
    The core convolution: sliding window dot products over the input.
    """
    out_channels = self.out_channels
    in_channels = self.in_channels
    kernel_h, kernel_w = self.kernel_size
    stride_h, stride_w = self.stride

    output = np.zeros((batch_size, out_channels, out_h, out_w))

    for b in range(batch_size):
        for out_ch in range(out_channels):
            for oh in range(out_h):
                for ow in range(out_w):
                    in_h_start = oh * stride_h
                    in_w_start = ow * stride_w

                    conv_sum = 0.0
                    for k_h in range(kernel_h):
                        for k_w in range(kernel_w):
```

```
                            for in_ch in range(in_channels):
                                input_val = padded[b, in_ch,
                                                   in_h_start + k_h,
                                                   in_w_start + k_w]
                                weight_val = self.weight.data[out_ch,
    in_ch, k_h, k_w]
                                conv_sum += input_val * weight_val

                    output[b, out_ch, oh, ow] = conv_sum

    return output
```

For the `(1, 1, 4, 4)` stride-2 example, `_apply_padding` makes a `(1, 1, 6, 6)` array. At output position `(oh=1, ow=0)`, the window starts at padded position `(2, 0)`, because each output index is multiplied by stride 2. Kernel position `(k_h=1, k_w=2)` then reads padded position `(3, 2)`, which is original image position `(2, 1)` after subtracting the one-pixel border. The outer `out_ch` loop changes the filter used with that same window. Every visit adds a product into `conv_sum`; only after the window is complete does the helper write one output value.

The operation that autograd records is a thin wrapper around the helpers. Its `forward` receives the arrays, asks the layer for the output shape, pads, convolves, and adds the bias to each output channel.

```
def forward(self, x: np.ndarray, weight: np.ndarray, bias: np.ndarray
    | None = None) -> np.ndarray:
    """
    Convolve the (already validated) input. The layer that owns the
    weights
    is passed as `layer`, so the student-written helpers below do the
    work:
    _apply_padding and _convolve_loops.
    """
    batch_size = x.shape[0]
    out_height, out_width =
    self.layer._compute_output_shape(x.shape[2], x.shape[3])
    padded_input = self.layer._apply_padding(x)
    output = self.layer._convolve_loops(padded_input, batch_size,
    out_height, out_width)
    if bias is not None:
        for out_ch in range(self.layer.out_channels):
            output[:, out_ch, :, :] += bias[out_ch]
    return output
```

The `self.layer` it reaches for is a parameter of the operation, passed as `layer=self` when the layer calls `apply`, which is how a `Function` that works on bare arrays can borrow the stride, padding, and helpers that live on the layer.

9.3.3 Pooling

With padding chosen to preserve spatial size, a convolution hands the next layer a plane the same size as the one it received, and that gets expensive to keep doing. Run the

numbers for the first layer of a photo network. The 224×224 color input holds 150,528 numbers. After `Conv2d(3, 64, 3, padding=1)` there are $64 \times 224 \times 224 = 3{,}211{,}264$ of them, 12.8 MB in float32 for a single image, and a second 3×3 layer with 64 input and 64 output channels does about 1.85 billion multiplies per image, 21 times what the first layer did, because its dot products are 576 long instead of 27. Left at full resolution, every later layer pays that. There is a second problem. A 3×3 filter widens what one output pixel can see by only two pixels per layer, so at full resolution it takes over a hundred layers before any output has seen the whole 224-wide picture.

The cheap fix is to shrink the planes between convolutions. Max pooling slides a 2×2 window with stride 2 over each channel and keeps the largest value in each window. That quarters the stored activations for even spatial dimensions and costs no weights. It also doubles the distance between neighboring pooled positions in the original image, so a subsequent convolution reaches farther across that image. In code it replaces the dot product with a running `max` and runs per channel, so it has no channel loop inside the window.

```
def _maxpool_loops(self, padded: np.ndarray, batch_size: int,
    channels: int,
                   out_h: int, out_w: int) -> np.ndarray:
    """
    The core max pooling: find maximum value in each window.
    """
    kernel_h, kernel_w = self.kernel_size
    output = np.zeros((batch_size, channels, out_h, out_w))

    for b in range(batch_size):
        for c in range(channels):
            for oh in range(out_h):
                for ow in range(out_w):
                    in_h_start = oh * self.stride[0]
                    in_w_start = ow * self.stride[1]

                    max_val = -np.inf
                    for k_h in range(kernel_h):
                        for k_w in range(kernel_w):
                            input_val = padded[b, c,
                                               in_h_start + k_h,
                                               in_w_start + k_w]
                            max_val = max(max_val, input_val)

                    output[b, c, oh, ow] = max_val

    return output
```

`MaxPool2d(2)` defaults to stride `(2, 2)`, while `MaxPool2d((2, 3))` defaults to `(2, 3)`. Height and width use their own stride, so rectangular windows tile without accidental overlap. An explicit integer stride applies to both axes. The layer follows `__call__` → `forward` → `_compute_pool_output_shape` → `MaxPool2dFunction.apply`. The operation pads if needed and calls `_maxpool_loops`; a kernel that cannot fit raises before an empty output can reach the next layer.

The module retains the input through `Function.apply`; its backward method scans each window again to recover the winning position. It selects the first valid maximum in row-major order when values tie. The gradient uses += because overlapping windows can select the same input more than once. Max-pool padding contains negative infinity rather than zero, so it cannot defeat a finite negative activation. Backward excludes padding from its candidates, including when real input values are also negative infinity.

Average pooling changes the reduction, not the geometry. For a window `[[1, 2], [3, 4]]`, max pooling returns 4 and sends a unit gradient to the lower-right element; average pooling returns 2.5 and sends 0.25 to each element. `AvgPool2d` always divides by the full kernel area, including zero-padding positions. A corner window containing one real value 4 and three padded zeros therefore returns 1, and a unit upstream gradient gives that real input 0.25. The padded shares have no input owner and are removed when backward crops the gradient. Both pooling layers return an empty parameter list; their operation records exist to train earlier layers.

9.3.4 im2col

Module 09 stops at the loops. The two functions in this section are the chapter's own NumPy model of the trick GPU libraries use, and the worked trace checks that they reproduce the module's numbers. Chapter 18 builds im2col into the package and times it against these loops.

The direct loops spend time on more than arithmetic. Each multiply requires Python to execute loop control and NumPy scalar indexing. The array accesses are structured, but writing the computation as scalar Python operations does not hand an entire dot product to NumPy's compiled matrix-multiplication routine. im2col changes that execution boundary.

Look at the direct routine again. The three innermost loops form a dot product between two lists of $C_{\text{in}}K^2$ numbers, the flattened patch under the filter and the flattened filter, and every output pixel is one of those dot products. We can move those dot products into one matrix operation, paying the Python loop overhead per copied patch instead of per multiply. Copy each patch the filter visits into the row of a matrix, flatten each filter into a column, and let one `matmul` do every dot product at once. The `matmul` operation from Chapter 1 already delegates matrix arithmetic to NumPy; the chapter model uses the same numerical primitive.

For a 32×32 color image with sixteen 3×3 filters and padding 1, both formulations perform 442,368 multiplies. The patch matrix contains 27,648 numbers, compared with 3,072 in the input: a $9\times$ memory footprint expansion because neighboring patches replicate interior pixels up to $K^2 = 9$ times. Why would silicon architectures gladly accept a nine-fold memory expansion? Because the copy turns every window into a row of one contiguous matrix, a single tuned GEMM routine can do all the dot products, reusing each loaded value many times from cache or registers. In this NumPy model the larger gain is leaving the per-multiply Python loop; how much the lowering pays on hardware depends on the library and the shape, which Chapter 18 measures.

Core Invariant: Spatial Unfolding

Patch $(C_{\text{in}}, K, K) \rightarrow$ Row in X_{col}

Converts nested 4D spatial sliding windows into a single BLAS GEMM at the expense of K^2 memory expansion.

The model in listing 9.1 walks the same output positions as the loops, but instead of multiplying it copies the patch into a row. The order inside a row (channel, then filter row, then filter column) is the order `ravel` produces and the order `reshape` gives the flattened filters, so the dot products pair the right numbers.

The multiplies all happen inside the `@`. Everything around it is copying and reshaping. `patches` holds $NH_{\text{out}}W_{\text{out}}C_{\text{in}}K^2$ numbers where the input held $NC_{\text{in}}HW$, which gives

Listing 9.1 An array-only convolution model: The patch buffer exposes the storage cost of turning windows into a matrix.

```
import numpy as np

def im2col(x, K, stride=1, padding=0):
    """Return patches (one row per position), H_out, and W_out."""
    x = np.pad(x, ((0, 0), (0, 0), (padding, padding), (padding,
    ↪ padding)))
    N, C, H, W = x.shape
    H_out = (H - K) // stride + 1
    W_out = (W - K) // stride + 1
    patches = np.empty((N * H_out * W_out, C * K * K), dtype=x.dtype)
    r = 0
    for n in range(N):
        for oh in range(H_out):
            for ow in range(W_out):
                patch = x[n, :, oh * stride:oh * stride + K,
                          ow * stride:ow * stride + K]
                patches[r] = patch.ravel()
                r += 1
    return patches, H_out, W_out

def conv2d_im2col(x, weight, bias, stride=1, padding=0):
    """x: (N, C_in, H, W); weight: (C_out, C_in, K, K); bias:
    ↪ (C_out,)."""
    N, C_out, K = x.shape[0], weight.shape[0], weight.shape[2]
    patches, H_out, W_out = im2col(x, K, stride, padding)
    product = patches @ weight.reshape(C_out, -1).T + bias
    planes = product.reshape(N, H_out, W_out, C_out)
    return planes.transpose(0, 3, 1, 2)
```

the ninefold storage expansion in the example above. The last line uses NumPy views to arrange the output axes: the product comes out with the output channel as its last axis, and a `reshape` plus a `transpose` present it as $[N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}}]$ without moving a number. This helper supports square filters and scalar stride/padding. It is an array-only comparison model; the module also accepts rectangular kernels, and its tensor operations retain the gradient graph.

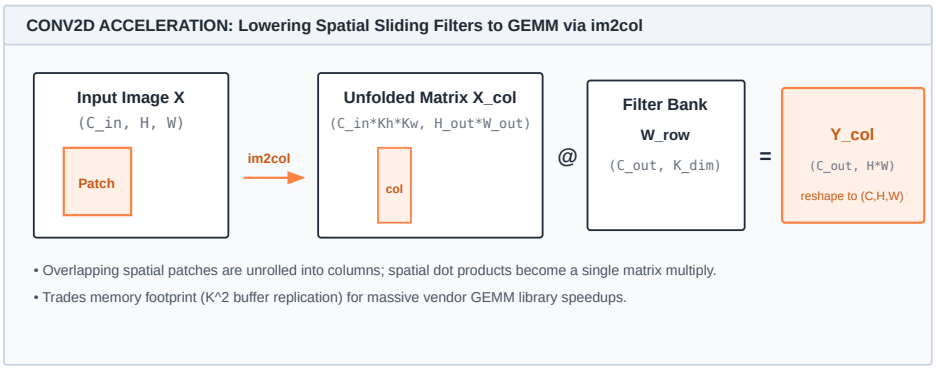


Figure 9.4: **Lowering 2D convolution to GEMM via im2col**: Spatial receptive fields across input feature maps are unfolded into contiguous rows/columns. The resulting matrix product executes with optimized level-3 BLAS libraries, trading temporary buffer replication for maximum compute density.

Problem: For a batch of $B = 32$ feature maps with $C_{\text{in}} = 64$ channels and spatial dimensions 112×112, what is the memory overhead of lowering a 3×3 convolution via explicit im2col?

Input tensor storage:

$$\text{Input memory} = 32 \times 64 \times 112 \times 112 \times 4 \text{ bytes} \approx 102.8 \text{ MB}$$

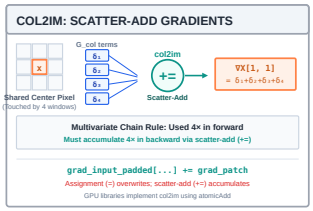
Lowered im2col matrix storage:

$$\text{im2col matrix} = (32 \times 112 \times 112) \times (64 \times 3 \times 3) \times 4 \text{ bytes} \approx 924.8 \text{ MB}$$

Systems insight: Explicit im2col duplicates input values by $K_h \times K_w = 9.0\times$ in memory. For deep vision models, explicit im2col buffers quickly overwhelm device RAM, motivating production libraries (cuDNN) to use *implicit GEMM* where addresses are computed in hardware registers on the fly without materializing the patch matrix in DRAM.

9.3.5 The gradients

The training loop needs the gradient of every parameter at each step. A centered finite-difference check would rerun the forward pass twice per tested parameter: 3,584 forwards to check all 1,792 parameters of `Conv2d(3, 64, 3)`. That makes numerical differentiation useful for checking a small sample, not for implementing each update. The product rule from The Idea computes all gradients by revisiting the windows once. The operation returns one gradient per tensor input it consumed: the image, the weights, and, when present, the bias.



Gradient Scatter-Add: Overlapping receptive fields share input pixels in forward execution. In backpropagation, `col2im` must accumulate incoming terms via += rather than overwriting.

```
def backward(self, grad_output: np.ndarray) -> tuple[np.ndarray, ...]:
    """
    Compute gradients for convolution inputs and parameters.
    """
    x, weight = self.inputs[0], self.inputs[1]
```

```
    bias = self.inputs[2] if len(self.inputs) > 2 else None
    (stride_h, stride_w), padding, kernel_size = self.layer.stride,
    self.layer.padding, self.layer.kernel_size

    batch_size, out_channels, out_height, out_width =
    grad_output.shape
    _, in_channels, in_height, in_width = x.shape
    kernel_h, kernel_w = kernel_size

    # Apply padding to input if needed (for gradient computation)
    if padding > 0:
        padded_input = np.pad(x.data,
                              ((0, 0), (0, 0), (padding, padding),
    (padding, padding)),
                              mode='constant', constant_values=0)
    else:
        padded_input = x.data

    # Initialize gradients
    grad_input_padded = np.zeros_like(padded_input)
    grad_weight = np.zeros_like(weight.data)
    grad_bias = None if bias is None else np.zeros_like(bias.data)

    # Compute gradients using explicit loops (educational approach)
    for b in range(batch_size):
        for out_ch in range(out_channels):
            for out_h in range(out_height):
                for out_w in range(out_width):
                    # Position in input
                    in_h_start = out_h * stride_h
                    in_w_start = out_w * stride_w

                    # Gradient value flowing back to this position
                    grad_val = grad_output[b, out_ch, out_h, out_w]

                    # Distribute gradient to weight and input
                    for k_h in range(kernel_h):
                        for k_w in range(kernel_w):
                            for in_ch in range(in_channels):
                                # Input position
                                in_h = in_h_start + k_h
                                in_w = in_w_start + k_w

                                # Gradient w.r.t. weight
                                grad_weight[out_ch, in_ch, k_h, k_w]
    += (
                                    padded_input[b, in_ch, in_h, in_w]
    * grad_val
                                )

                                # Gradient w.r.t. input
```

```
                                grad_input_padded[b, in_ch, in_h,
    ↪ in_w] += (
                                    weight.data[out_ch, in_ch, k_h,
    ↪ k_w] * grad_val
                                )

    # Compute gradient w.r.t. bias (sum over batch and spatial
    ↪ dimensions)
    if grad_bias is not None:
        for out_ch in range(out_channels):
            grad_bias[out_ch] = grad_output[:, out_ch, :, :].sum()

    # Remove padding from input gradient
    if padding > 0:
        grad_input = grad_input_padded[:, :,
                                       padding:-padding,
                                       padding:-padding]
    else:
        grad_input = grad_input_padded

    # One gradient per input: (x, weight) or (x, weight, bias).
    if bias is None:
        return grad_input, grad_weight
    return grad_input, grad_weight, grad_bias
```

Where the forward pass read `input_val * weight_val` and wrote to `output`, the backward pass reads `grad_val` and accumulates into two separate buffers. `grad_weight` receives `input_val * grad_val`; `grad_input_padded` receives `weight_val * grad_val`. Their shapes match the weights and padded input. Neither operation changes the parameter values; the optimizer does that later. Notice the bias gradient. Because the bias vector of shape `[C_out]` was broadcast across the batch dimension N and both spatial dimensions $(H_{\text{out}}, W_{\text{out}})$ during the forward pass, its derivative sums upstream gradients over those three axes, expressed by `grad_output[:, out_ch, :, :].sum()`. The broadcast-reduction contract reappears inside spatial convolution. Notice also the last few lines, which strip the padding border off the input gradient, because the zeros we added have no owner to receive a gradient. The tensor's `backward()` reaches this method through the record that `apply` made, without the caller knowing.

9.3.6 The layer

Finally the layer object, because the routines above are helpers and operations and the rest of TinyTorch works with layers. The optimizer from Chapter 7 asks each layer for `parameters()`, and `Sequential` from Chapter 3 calls `forward`, so `Conv2d` needs the same two methods `Linear` has. The one decision hiding in the constructor is the initial scale of the weights, and the wrong choice here produces a wrong number, not a slow one. This convolution uses a normal distribution with standard deviation $\sqrt{2/\text{fan_in}}$, whose factor of 2 compensates for the ReLU that follows, and the trap is what fan-in means for a filter. It is not the number of pixels in the image. Each output pixel sums $C_{\text{in}}K^2$ products, 27 for the first layer of a color network, so that is the fan-in.[41] Use the image size instead, 150,528 for a 224×224 photo, and the weights come out about 75 times too

[41] **Kaiming (He) initialization** sets weight variance $\text{Var}(W) = 2/\text{fan_in}$ where $\text{fan_in} = C_{\text{in}} \cdot K_h \cdot K_w$. The factor of 2 accounts for half the activations being zeroed out by subsequent ReLU nonlinearities ($\mathbb{E}[\max(0, x)^2] = \frac{1}{2}\text{Var}(x)$), maintaining constant activation variance across arbitrarily deep convolutional networks.

small; the first layer's activations inherit that smaller scale. Repeating such under-scaling through a network can leave very small signals at the loss.

```
def __init__(self, in_channels: int, out_channels: int, kernel_size: int | tuple[int, int],
             stride: int | tuple[int, int] = 1, padding: int = 0, bias: bool = True) -> None:
    """
    Initialize Conv2d layer with proper weight initialization.
    """
    self.in_channels = in_channels
    self.out_channels = out_channels

    # Handle kernel_size as int or tuple
    if isinstance(kernel_size, int):
        self.kernel_size = (kernel_size, kernel_size)
    else:
        self.kernel_size = kernel_size

    # Stride as (height, width), like kernel_size and MaxPool2d's stride
    if isinstance(stride, int):
        self.stride = (stride, stride)
    else:
        self.stride = tuple(stride)
    self.padding = padding

    # He initialization for ReLU networks
    kernel_h, kernel_w = self.kernel_size
    fan_in = in_channels * kernel_h * kernel_w
    std = np.sqrt(2.0 / fan_in)

    # Weight shape: (out_channels, in_channels, kernel_h, kernel_w)
    self.weight = Tensor(rng.normal(0, std,
                         (out_channels, in_channels, kernel_h, kernel_w)),
                         requires_grad=True)

    # Bias initialization
    if bias:
        self.bias = Tensor(np.zeros(out_channels), requires_grad=True)
    else:
        self.bias = None
```

```
def forward(self, x: Tensor) -> Tensor:
    """
    Forward pass through Conv2d layer.
    """
    # Step 1: Validate input
    validate_4d_input(x, "Conv2d")

```

```
    batch_size, in_channels, in_height, in_width = x.shape
    if in_channels != self.in_channels:
        raise ValueError(f"Conv2d expected {self.in_channels} input channels, got {in_channels}")
    out_height, out_width = self._compute_output_shape(in_height, in_width)
    if out_height <= 0 or out_width <= 0:
        raise ValueError("Conv2d kernel must fit within the padded input")

    # Steps 3-5: pad, convolve, add bias. The operation runs the helpers
    # (via `layer=self`) and Module 06's apply() records it for backward.
    if self.bias is not None:
        return Conv2dFunction.apply(x, self.weight, self.bias, layer=self)
    return Conv2dFunction.apply(x, self.weight, layer=self)
```

```
def parameters(self) -> list[Tensor]:
    """Return trainable parameters."""
    params = [self.weight]
    if self.bias is not None:
        params.append(self.bias)
    return params
```

`Conv2d(3, 64, 3)` has exactly the 1,792 numbers counted earlier, and `parameters()` hands them to an optimizer the same way `Linear` did. `forward` validates the input, computes the output shape, and hands the tensors and itself to `Conv2dFunction.apply`, which records the operation so that a later `backward()` can find the gradient method above. The fan-in counts the same local connections used by the forward loop, so changing image size does not change the initialization scale.

9.3.7 Batch normalization and layer state

Standard in vision architectures like ResNet and DenseNet, batch normalization stabilizes deep network optimization by standardizing feature distributions across mini-batches during training. Unlike purely stateless layers, it must track non-differentiable running statistics across iterations to enable deterministic inference during evaluation. For one channel containing `[[0, 1], [2, 3]]`, the mean is 1.5 and the variance is 1.25. With `gamma=1`, `beta=0`, and the default small denominator offset, `BatchNorm2d` returns approximately `[[-1.342, -0.447], [0.447, 1.342]]`. Even one image has several values per channel, so a batch of one does not generally normalize to zeros.

The layer performs this calculation separately for each channel. For a tensor `(N, C, H, W)`, `_get_stats` reduces axes `(0, 2, 3)` and leaves one mean and variance per channel. `BatchNorm2dFunction.forward` reshapes those arrays to `(1, C, 1, 1)`, normalizes each activation, and applies learned scale `gamma` and shift `beta`. The input and output shapes match; the tensor broadcasting contract applies to whole image planes.

There are two kinds of persistent state on the layer. `gamma` and `beta` are tensors returned by `parameters()`, so the optimizer updates them from gradients. `running_mean` and

`running_var` are NumPy arrays updated by `_get_stats` during each training forward call. With the default momentum 0.1, the new running value combines 0.9 of the old value and 0.1 of the current batch statistic. For the four-value example, the initial running mean 0 becomes 0.15, and the initial running variance 1 becomes 1.025. TinyTorch uses the population variance from `np.var` for both normalization and this update. These arrays do not belong in the optimizer's parameter list. A training forward changes them even if no backward or optimizer step follows.

```
def _get_stats(self, x: Tensor) -> tuple[np.ndarray, np.ndarray]:
    """
    Get mean and variance for normalization (batch or running stats).
    """
    if self.training:
        # Compute batch statistics per channel
        # Mean over batch and spatial dimensions: axes (0, 2, 3)
        batch_mean = np.mean(x.data, axis=(0, 2, 3))  # Shape: (C,)
        batch_var = np.var(x.data, axis=(0, 2, 3))    # Shape: (C,)

        # Update running statistics (exponential moving average)
        self.running_mean = (1 - self.momentum) * self.running_mean +
        ↪   self.momentum * batch_mean
        self.running_var = (1 - self.momentum) * self.running_var +
        ↪   self.momentum * batch_var

        return batch_mean, batch_var
    else:
        # Use running statistics (frozen during eval)
        return self.running_mean, self.running_var
```

Calling `bn.eval()` changes which statistics the next forward uses. It reads the running arrays and leaves them unchanged; `bn.train()` restores batch statistics and running updates. Evaluation mode does not disable autograd. The operation receives the mode as an argument to `apply` and still has a backward rule, but that rule treats the running statistics as constants. A model containing BatchNorm must propagate its mode changes to these layers before validation, or validation data will change the statistics and each prediction will depend on the other examples in its batch.

The backward calculation explains why the mode is part of the operation record. In training, changing one activation changes its channel's mean and variance, which changes every normalized activation in that channel. `BatchNorm2dFunction.backward` therefore combines the direct gradient with two reductions, `sum_grad` and `sum_grad_norm`, that account for those dependencies. In evaluation only the direct scale by `gamma * inv_std` remains. Both modes sum over `(0, 2, 3)` for `grad_beta` and sum `grad_output * normalized` over those axes for `grad_gamma`. The saved `normalized_data` and `inv_std` let backward use the quantities from this particular forward call.

These spatial layers connect to the existing training loop through their tensor outputs and parameter lists. A classifier can pass `conv(x)` through normalization, activation, and pooling, then call `pooled.reshape(batch_size, -1)` before a `Linear` layer. Using `Tensor(pooled.data.reshape(...))` instead would detach the features; the classifier could still receive gradients while the convolution stopped learning. Milestone 04 checks for exactly this failure. After a complete forward, the loss traverses the recorded opera-

tions backward, and the optimizer updates the parameters returned by the convolution, normalization, and classifier layers.

9.4 A Worked Trace

One image, one channel, 3×3 pixels, one 2×2 filter with bias 1, no padding, stride 1:

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}, \qquad w = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \qquad b = 1.$$

The output is $\lfloor (3-2)/1 \rfloor + 1 = 2$ pixels on each side. The filter visits four positions, and each row of the trace in table 9.1 is also one row of the im2col matrix.

Table 9.1: **Four windows, one shared filter**: Each output pixel is a dot product between the flattened patch and the flattened filter $[1, 0, -1, 1]$. The four patch rows stacked are exactly X_{col}.

Output (h, c)	Input slice	Patch as a row	Dot product plus bias	$Y[h, c]$
$(0, 0)$	[0:2, 0:2]	$[1, 2, 0, 1]$	$1(1) + 2(0) + 0(-1) + 1(1) + 1$	3
$(0, 1)$	[0:2, 1:3]	$[2, 0, 1, 3]$	$2(1) + 0(0) + 1(-1) + 3(1) + 1$	5
$(1, 0)$	[1:3, 0:2]	$[0, 1, 2, 1]$	$0(1) + 1(0) + 2(-1) + 1(1) + 1$	0
$(1, 1)$	[1:3, 1:3]	$[1, 3, 1, 0]$	$1(1) + 3(0) + 1(-1) + 0(1) + 1$	1

The result is $Y = \begin{bmatrix} 3 & 5 \\ 0 & 1 \end{bmatrix}$, and a 2×2 max pool over it keeps the single value 5.

The backward pass is just as checkable. Send back a gradient of 1 at every output pixel. Weight $w[0, 0]$ multiplied the top-left pixel of each of the four patches, so its gradient is $1+2+0+1 = 4$. Weight $w[0, 1]$ saw the top-right pixel of each patch, $2+0+1+3 = 6$. The bias gradient is the sum of the four incoming ones. For the input, the center pixel $X[1, 1]$ was touched by all four filter positions, once under each weight, so its gradient is $1 + 0 + (-1) + 1 = 1$; the corner $X[2, 0]$ was touched only by $w[1, 0]$ and gets -1. The script in listing 9.2 runs the module's layer, checks the im2col model against it, and lets `backward()` reach the gradient method through the tape.

In listing 9.2, lines 6 to 8 initialize a single 3×3 image tensor `X` with `requires_grad=True`. Lines 9 to 11 set up `conv = Conv2d(1, 1, 2)` with the trace kernel $\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$ and bias 1.0. Line 13 executes the forward convolution `Y = conv(X)`, printing `[[3.0, 5.0], [0.0, 1.0]]` in line 14. Lines 15 to 16 verify that the layer's output matches `conv2d_im2col` within floating-point tolerance via `np.allclose`. In line 17, `MaxPool2d(2)(Y)` extracts the winning activation `[[5.0]]`.

Listing 9.2 Tracing four convolution windows: The layer and im2col model agree before the recorded backward pass accumulates gradients.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.spatial import Conv2d, MaxPool2d
import tinytorch.core.autograd

X = Tensor(np.array([[[[1.0, 2.0, 0.0],
                       [0.0, 1.0, 3.0],
                       [2.0, 1.0, 0.0]]]]), requires_grad=True)
conv = Conv2d(1, 1, 2)
conv.weight.data[:] = [[[[1.0, 0.0], [-1.0, 1.0]]]]
conv.bias.data[:] = [1.0]

Y = conv(X)
print(Y.data[0, 0].tolist())  # [[3.0, 5.0], [0.0, 1.0]]
modeled = conv2d_im2col(X.data, conv.weight.data, conv.bias.data)
assert np.allclose(Y.data, modeled)
print(MaxPool2d(2)(Y).data[0, 0].tolist())  # [[5.0]]

Y.backward(Tensor(np.ones_like(Y.data)))
print(conv.weight.grad[0, 0].tolist())   # [[4.0, 6.0], [4.0, 5.0]]
print(conv.bias.grad.tolist())           # [4.0]
print(X.grad[0, 0].tolist())
# [[1.0, 1.0, 0.0], [0.0, 1.0, 1.0], [-1.0, 0.0, 1.0]]
```

In line 19, `Y.backward(...)` injects unit gradients across all four output pixels. Lines 20 to 23 print the resulting analytical gradients: `conv.weight.grad` receives the column-patch sums `[[4.0, 6.0], [4.0, 5.0]]` (line 20), `conv.bias.grad` sums all four outputs to `[4.0]` (line 21), and `X.grad` reflects kernel overlap across pixels (lines 22 to 23).

The preceding backward call differentiated the sum of the four convolution outputs. Pooling defines a different scalar objective. For `MaxPool2d(2)(conv(X)).sum()`, only the top-right convolution output, value 5, wins. Its incoming gradient is 1 and the other three receive zero, so the weight gradient is exactly that winning patch, `[[2, 0], [1, 3]]`, and the bias gradient is 1.

The two destinations in figure 9.5 come from the same product rule. The selected patch supplies the weight gradient, while the filter supplies the input gradient at that patch's location.

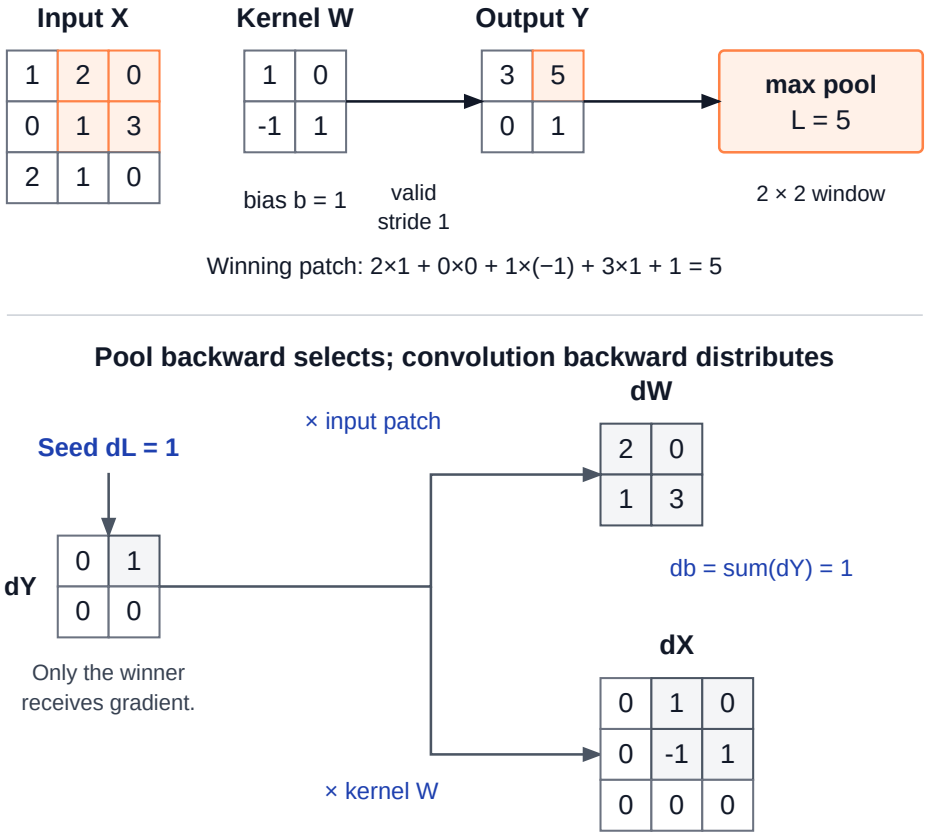


Figure 9.5: **Routing the winning gradient**: Max pooling selects the output value 5. Its unit gradient reaches one convolution window, whose input patch supplies the weight gradient and whose kernel supplies the input gradient.

Listing 9.3 Tracing a pooled objective: A fresh graph routes a unit gradient through the winning window.

```
# Build a fresh graph after the preceding backward released its records.
X.grad = None
for parameter in conv.parameters():
    parameter.grad = None
pooled = MaxPool2d(2)(conv(X))
pooled.sum().backward()
assert np.allclose(conv.weight.grad[0, 0], [[2, 0], [1, 3]])
assert np.allclose(conv.bias.grad, [1])
assert np.allclose(X.grad[0, 0], [[0, 1, 0], [0, -1, 1], [0, 0, 0]])
```

In listing 9.3, clearing parameter gradients prevents the two objectives from accumulating into the same buffers. Recomputing `conv(X)` is separately necessary because the

earlier `Y.backward(...)` released its operation records. This distinction is the graph-lifetime rule from Chapter 6 in a complete spatial chain. The array-only im2col model can check forward values, but it does not create a TinyTorch graph and cannot replace this training path.

9.5 From TinyTorch to Production

A production implementation can preserve the window computation while changing how it executes. NVIDIA's cuDNN library includes implicit matrix-multiplication and transform-based convolution methods. **GEMM** means general matrix multiplication; *implicit* means the patch matrix is formed as values are needed instead of stored in full. Shape and available workspace influence the choice. NVIDIA's convolution guide explains these alternatives. table 9.2 compares their mechanisms rather than prescribing a universal winner.

Table 9.2: **Alternative convolution formulations**: Each preserves the intended convolution while changing its arithmetic or temporary storage. Actual performance depends on the implementation and workload.

Algorithm	Main trade-off	What it does
Explicit im2col + GEMM	Extra patch storage	Build the patch matrix and multiply, as in the chapter model.
Implicit GEMM	More indexing inside execution	Compute with patches without storing the full patch matrix.
Winograd	Transform work and numerical behavior	Transform small tiles to reduce multiplication count.
FFT	Transform work and storage	Transform inputs and filters, multiply, then transform back.

Modern deep learning hardware rarely materializes an explicit im2col matrix. NVIDIA cuDNN selects among several convolution algorithms based on kernel geometry and available GPU workspace; two matter here:

1. **Implicit GEMM**: Computes matrix addresses directly inside CUDA thread indexing formulas, achieving GEMM speed without the $9\times$ memory duplication penalty.[42]
2. **Winograd** $F(2 \times 2, 3 \times 3)$: Uses polynomial residue transforms to reduce 2D multiplications from $4 \times 9 = 36$ down to 16, yielding up to $2.25\times$ computational throughput on 3×3 filters at the expense of slight numerical precision.

The transform methods require mathematical tools beyond this implementation. Winograd changes the representation of small tiles to reduce multiplication count.[43] FFT methods transform larger arrays before multiplying.[44] Neither method is implemented by Module 09; the reference loops provide values against which such alternatives can be checked.

The storage calculation makes the reason for implicit GEMM concrete. A batch of 32 images with 64 channels and 112×112 pixels contains 25,690,112 values, about 103 MB at four bytes per value. With a 3×3 filter, padding 1, and stride 1, the explicit patch matrix has nine times as many entries, about 925 MB. These are buffer-size calculations,

[42] Rather than allocating an explicit $O(K^2 \cdot H_{\text{out}} \cdot W_{\text{out}})$ patch matrix in global DRAM, cuDNN's implicit GEMM computes index mappings on the fly within GPU registers. Each warp calculates virtual row and column coordinates directly from thread IDs, streaming pixels directly from L2 cache into shared memory and register tiles without writing intermediate unrolled patch buffers to DRAM.

[43] **Winograd convolution** applies minimal-filtering algorithms to small convolution tiles. The paper by Lavin and Gray develops these algorithms for convolutional networks and compares their arithmetic and execution costs.

[44] **FFT convolution** uses the fast Fourier transform to express convolution through pointwise products in a transformed representation. The input and output transforms add work and storage, so fewer products alone do not establish a speedup.

not measured process memory. Avoiding that full matrix removes a large temporary allocation while keeping the same virtual dot products; it does not remove the need to fetch the inputs.

The optional comparison in listing 9.4 preserves the small example's numerical contract. The output-size formula, the weight shape $[C_{\text{out}}, C_{\text{in}}, K, K]$, the parameter count, and the gradient formulas are the same in PyTorch, and `nn.Conv2d(1, 1, 2)` loaded with our filter returns our $[[3, 5], [0, 1]]$. The FLOP count is easiest to read off the im2col view, $2 \cdot NH_{\text{out}}W_{\text{out}} \cdot C_{\text{out}} \cdot C_{\text{in}}K^2$ (counting a multiply-add as two operations and excluding bias additions), which estimates the arithmetic work per layer before execution. Max pooling still routes each gradient to one winning pixel.

Listing 9.4 The same window in PyTorch: The optional comparison uses the identical input, filter, and bias.

```
# Optional production comparison; not a TinyTorch dependency.
import torch, torch.nn as nn

conv = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=2)
with torch.no_grad():
    conv.weight.copy_(torch.tensor([[[[1.0, 0.0], [-1.0, 1.0]]]]))
    conv.bias.copy_(torch.tensor([1.0]))

x = torch.tensor([[[[1.0, 2.0, 0.0], [0.0, 1.0, 3.0], [2.0, 1.0,
    0.0]]]])
print(conv(x).squeeze())          # tensor([[3., 5.], [0., 1.]])
```

Memory layout is another implementation choice. A tensor can retain logical shape `(N, C, H, W)` while changing which neighboring values sit next to each other in memory. PyTorch's `channels_last` format keeps those logical axes and changes the strides so channels are adjacent; it is not a request to pass an NHWC-shaped tensor to `Conv2d`. The PyTorch memory-format tutorial demonstrates that distinction. Module 09 uses its ordinary NumPy arrays. Chapter 15 later separates operation counts, modeled storage, and elapsed time; none of these alone establishes which layout or convolution algorithm will be fastest.

9.6 Summary

A call to `Conv2d` validates `(N, C, H, W)`, allocates its output, visits each window, and adds each filter's bias. `Function.apply` connects that output to the input and parameter tensors. Backward revisits the same windows, accumulating contributions into input and weight gradients and summing the broadcast bias gradient. Shared weights explain both the small parameter count and the need to add contributions from many output positions.

Pooling preserves batch and channel axes while changing the spatial dimensions. Max pooling selects a winner per window; average pooling divides among the full kernel area, including padded positions. Batch normalization preserves shape and maintains two distinct kinds of state: learned scale and shift, and running statistics updated by training

forwards. A returning reader can check the spatial chain by reproducing the worked values, then confirming that the same parameters receive gradients.

The im2col model gives the same forward computation a matrix shape at the cost of an explicit patch buffer. It does not replace the module's recorded operations or establish a universal speedup. Convolutions exploit rigid, 2D local grid geometry through translation equivariance and fixed receptive fields. But how do deep learning systems model open-ended, 1D sequential structures like human language or code, where dependencies span arbitrary distances and coordinates carry variable symbolic meaning? Chapter 10 pivots from 2D spatial pixel arrays to 1D discrete sequential data, introducing byte-pair encoding to transform raw unicode strings into the integer token identifiers that feed modern language models.

With `Conv2d` and `MaxPool2d` complete (the milestone needs Modules 01 to 07 and 09), you can run Yann LeCun's LeNet architecture on handwritten digits:

```
tito milestone run 04
```

This milestone tests spatial weight sharing against dense multi-layer perceptrons on TinyDigits, demonstrating why convolutional inductive biases reduce parameter counts from 2,410 down to 810. Before training, it checks YOUR `CrossEntropyLoss` on one batch against a NumPy computation. It passes when test accuracy reaches 75 percent and YOUR `Conv2d` filters receive a nonzero gradient and move at least 25 percent of their initial norm; accuracy alone cannot show that the convolution learned, because a classifier on frozen random filters still reaches about 81 percent. See Synthesis II for the complete experimental comparison.

9.7 Further Reading

- **LeCun et al.** (1998). "Gradient-based learning applied to document recognition." *Proceedings of the IEEE*. LeNet, weight sharing, and pooling, the design Synthesis II tests on TinyDigits.
- **Chellapilla, Puri, and Simard** (2006). "High performance convolutional neural networks for document processing." *IWFHR*. The unrolling of convolution into a matrix product that this chapter models as im2col.
- **Simonyan and Zisserman** (2015). "Very deep convolutional networks for large-scale image recognition." *ICLR*. Stacks of small filters in place of large ones, the receptive-field trade described above.

🔥 Chapter 10

Byte-Pair Encoding: Compressing Language into Subword Tokens

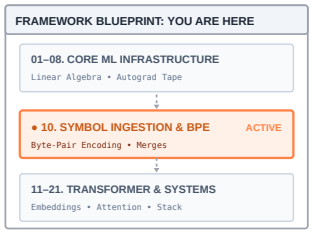
Language models operate over discrete symbolic text, yet neural networks compute strictly over continuous floating-point vectors. The fundamental challenge of natural language processing is bridging this representational divide: how do we convert variable-length human text into structured numerical sequences? Character-level tokenization produces long sequences, and attention's cost grows with the square of sequence length; word-level tokenization creates large vocabularies that have no ID for an unseen word.

In modern deep learning, subword tokenization turns strings into integer token arrays in a single method call:

```python
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")
text = "Machine learning systems from scratch"
tokens = tokenizer.encode(text)

print("Token IDs:", tokens)      # [35890, 4673, 3131, 422, 9081]
print("Subwords:", [tokenizer.decode([t]) for t in tokens])
# ['Machine', ' learning', ' systems', ' from', ' scratch']
```



Framework Blueprint: Module 10 bridges unstructured text and tensor mathematics through Byte-Pair Encoding subword tokenization.

Underneath that simple call, the tokenizer executes a statistical compression algorithm:

1. **Ordered merge rule traversal:** The engine breaks text into base characters, then iteratively scans and merges the most frequent adjacent pairs according to a pretrained priority table, compressing common words into single integer IDs.
2. **Elimination of out-of-vocabulary tokens:** By starting from all 256 byte values and building subwords hierarchically, a byte-level tokenizer can represent any text; a character alphabet covers novel words but not unseen characters.
3. **Sequence length compression:** Subword chunking reduces sequence lengths by $3\times$ to $4\times$ compared to character-level representations, directly mitigating the quadratic $O(S^2)$ memory and compute costs of downstream attention layers (Chapter 12).

This chapter constructs TinyTorch's BPE tokenization engine, establishing the text ingestion bridge that feeds the embedding tables in Chapter 11 and powers language modeling in Chapter 13. Before building the merge algorithm, we must inspect why naive character-level and word-level strategies fail in opposite directions.

10.1 The Problem

↩ **Jump back:** Chapter 5 *Tokenization acts as the front-end CPU ingestion stage that feeds numerical tensor batches.*

There are two obvious ways to turn text into integers, and both fail, in opposite directions.

The first is to number the characters. Every observed character gets an ID, but unseen characters remain a problem. A byte tokenizer can instead reserve an ID for each of the 256 byte values and represent any UTF-8 input as bytes. The cost is length. For the text `"the cat sat"`, character tokenization needs eleven IDs, including the spaces; word tokenization needs three. A longer sequence requires more positions to process. In Chapter 12, attention will introduce a table with one entry for every pair of token positions, making sequence length especially consequential; downstream in Chapter 19, this sequence length directly dictates the memory footprint of the autoregressive key-value cache.

The second is to number the words. Split on spaces, give each distinct word an ID, and a 1,000-word essay is 1,000 tokens. Now the vocabulary is the problem. The listing below builds a word vocabulary from one sentence and then encodes three others. Any word the vocabulary has not seen maps to a reserved ID, conventionally written `<unk>` for "unknown".

```
train_words = "the cat sat on the mat".split()
vocab = {word: i + 1 for i, word in
    enumerate(sorted(set(train_words)))}  # id 0 is <unk>

def encode_words(text):
    return [vocab.get(word, 0) for word in text.split()]

print(encode_words("the cat sat"))      # [5, 1, 4]
print(encode_words("the cats sat"))     # [5, 0, 4]   one extra letter
    and the word is gone
print(encode_words("The cat sat."))     # [0, 1, 0]   a capital and a
    period lose two words
```

One plural and the model reads nothing where "cats" was. A capital letter and a period erase two of three words. Real text is full of plurals, capitals, typos, names, code, and words coined last week, so increasing a word vocabulary cannot guarantee that the next word is present. Size has its own cost. The lookup operation in Chapter 11 turns each token ID into a row of a lookup table with one entry per vocabulary word and D numbers per row, so, for example, a hypothetical vocabulary of one million words at $D = 4{,}096$ in 32-bit floats is $10^6 \times 4{,}096 \times 4$ bytes, about 16 GB, before the model has read a single word.

↪ **Jump ahead:** Chapter 11 *How discrete token IDs index into continuous, high-dimensional vector representations via row-gather operations.*

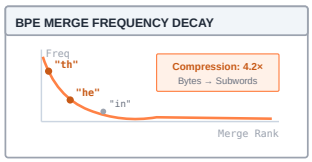
The two extremes trade one cost for the other. Characters give a tiny table and long sequences; words give short sequences and an enormous, leaky table. What we want is in between. Common words should be single tokens, so sequences stay short. Rare words should split into pieces the vocabulary does know, so an unseen word can still retain its known characters. The split itself should be learned from data rather than written by hand, because no one can list the useful pieces of every language.

Pause & Predict: The Byte-Level Vocab Guarantee

A character-level vocabulary has ~100 tokens but produces long sequences. A word-level vocabulary has >500,000 tokens but suffers from out-of-vocabulary (OOV) unknown words. How does byte-level Byte-Pair Encoding (BPE) guarantee both a compact vocabulary (~50,000 tokens) and that no input string can ever produce an out-of-vocabulary token?

10.2 The Idea

BPE began as a compression technique; Sennrich, Haddow, and Birch applied it to translation to represent rare words through subword units. The compression origin is the right way to think about it. A compressor looks for repeated patterns and gives them short codes. BPE looks for the most repeated adjacent pair of symbols and gives it a new symbol.



BPE pair frequency decay across merge ranks.

Start with an alphabet. Module 10 uses the characters that occur in the training words, with a marker glued onto each word's last character so that a letter ending a word is a different symbol from the same letter inside one; GPT-2 uses the 256 possible byte values instead, and the production section says why.[45] Write the training text as a sequence of those symbols. Then repeat one step. Count how often every adjacent pair occurs, pick the pair with the highest count, invent a new symbol for it, and replace every occurrence of the pair with that symbol. In symbols, with $\text{count}(a, b)$ the number of times symbol a is immediately followed by symbol b anywhere in the corpus,

$$(a^*, b^*) = \arg\max_{(a,b)} \text{count}(a, b), \qquad \text{new symbol } c = a^* b^*.$$

A new vocabulary entry can replace many occurrences of its pair; each nonoverlapping replacement shrinks a word by one symbol. The vocabulary ends up holding "es", "est", and so on up to whole common words, each earned by frequency, and the merges are recorded in the order they were made. That ordered list, together with the alphabet and ID mappings, specifies how text is represented. Training runs once, over a large corpus, and produces the list; each new vocabulary entry adds to the initial alphabet and reserved symbols. The requested `vocab_size` is a target for that total, not a count of merges; training may stop sooner when no pairs remain. It also retains the complete base alphabet when that alphabet alone exceeds the target. The public `vocab_size` reports the actual number of entries.

Encoding new text replays the list. Split the text into its base symbols and apply the merges in the order they were learned, each rule rewriting every place it matches, so that a pair learned earlier is always merged before one learned later. Because the merge list is fixed and the order is fixed, the same string always encodes to the same IDs, and a word the tokenizer never saw still encodes, because at worst it stays as single known-character symbols. That is most of the guarantee that word vocabularies could not give. The remainder depends on the alphabet: with characters, a character the corpus never contained still has nowhere to go and becomes `<UNK>`; with bytes there is nothing left to be unknown.

Decoding is trivial by comparison. Each ID names a string, so concatenate the strings and normalize whitespace. Word-final tokens already contain a trailing space. The

[45] In standard UTF-8, control characters (bytes 0 to 31, 127 to 159) and whitespace cannot be rendered cleanly in regex parsers. GPT-2 solves this by building an explicit invertible bijection `bytes_to_unicode()` that maps all 256 byte values directly into printable Unicode code points (such as shifting byte 32 space to `Ġ` U+0120), ensuring that BPE can run standard Python string regexes without corrupting control characters or byte sequences.

pipeline in figure 10.1 separates the state-changing training loop from encoding with fixed rules. The alphabet decides which characters survive; the merge order decides how they group.

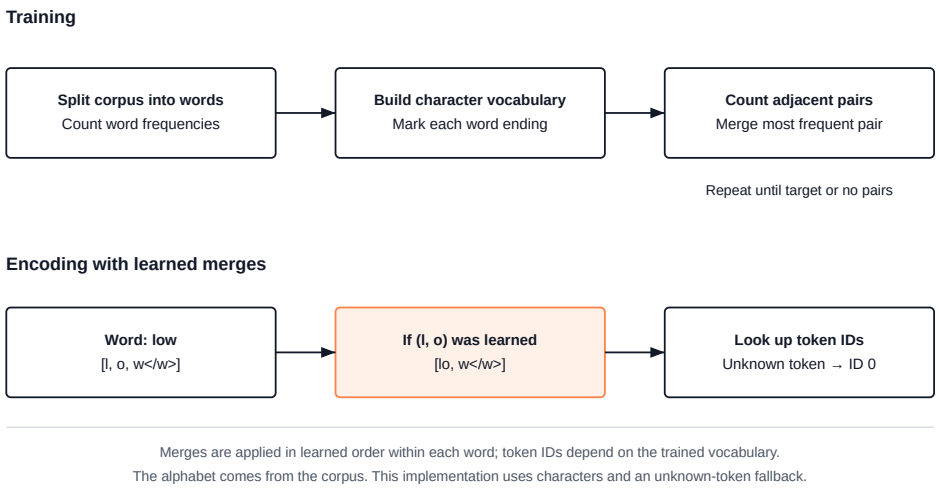


Figure 10.1: Training counts words and learns pair merges; encoding replays those merges within each word before looking up IDs. The visible notation `</w>` denotes the space stored at a word ending.

10.3 The Code

The tokenizer is a base class fixing the two-method interface, a character tokenizer that is The Problem's first extreme in code, and the BPE tokenizer, which is where the chapter lives. The BPE class holds a vocabulary in both directions (token to ID and ID to token) and the list of merges in the order they were learned. We call a merge's position in that list its **rank**; rank 0 is the first merge made.

SOURCE CODE MAPPING MOD 10

FILE: src/10_tokenization/10_tokenization.py
EXPORT: tinytorch.core.tokenization
SYMBOLS: BPETokenizer, train_bpe

Extracted & verified at build time ✓ tested

Source Code Mapping: Module 10 extracts Byte-Pair Encoding (BPE) vocabulary construction, token encoding, and decoding directly from the working repository.

10.3.1 The contract, and the character baseline

Production language processing frameworks like Hugging Face `tokenizers` and OpenAI `tiktoken` enforce a strict bidirectional contract between unstructured natural language strings and dense integer sequences. Before introducing statistical subword algorithms, character-level tokenizers illustrate this foundational contract in its simplest, lossless form. The language-model input pipeline begins with these integers. In Chapter 11, each ID selects a row of a lookup table, and the model's final layer produces one score per ID, so a tokenizer's job is a fixed two-way mapping between strings and IDs that must not change once the model starts training. The base class says only that much: `encode` and `decode`, each of which a subclass must supply.

```
class Tokenizer:
    """
    Base tokenizer class providing the interface for all tokenizers.
```

```
    """

    # Predefined symbolic tokens for common use cases
    TOK_UNKNOWN = '<UNK>'    # UNKNOWN
    # Words come from str.split(), so a space cannot collide with word content.
    # Diagrams write this boundary as </w> to make the otherwise invisible suffix clear.
    TOK_EOW = ' '            # END OF WORD

    def encode(self, text: str) -> List[int]:
        """
        Convert text to a list of token IDs.
        """
        raise NotImplementedError(
            f"encode() not implemented in base Tokenizer class\n"
            f"  ✗ Called encode() on abstract base class {self.__class__.__name__}\n"
            f"  🔔 Tokenizer is an interface - use a concrete implementation like CharTokenizer or BPETokenizer\n"
            f"  🔧 Example: tokenizer = CharTokenizer(['a', 'b', 'c']); tokenizer.encode('abc')"
        )

    def decode(self, tokens: List[int]) -> str:
        """
        Convert list of token IDs back to text.
        """
        raise NotImplementedError(
            f"decode() not implemented in base Tokenizer class\n"
            f"  ✗ Called decode() on abstract base class {self.__class__.__name__}\n"
            f"  🔔 Tokenizer is an interface - use a concrete implementation like CharTokenizer or BPETokenizer\n"
            f"  🔧 Example: tokenizer = CharTokenizer(['a', 'b', 'c']); tokenizer.decode([1, 2, 3])"
        )
```

The class attributes name the unknown token and the word boundary. `<UNK>` occupies ID 0. `TOK_EOW` is the string `" "`, a single space, appended to the last character of each word. Since `str.split()` removes whitespace before creating words, the boundary cannot collide with their contents. The notation `</w>` makes that invisible suffix visible on the page; the implementation does not replace literal `</w>` text.[46] The character tokenizer is the simplest thing that satisfies the contract, and it makes the trade in The Problem concrete: one ID per character, `<UNK>` at ID 0, and a `build_vocab` that sorts the characters it saw so that two runs on the same corpus agree.

[46] Different subword tokenizers use distinct conventions to preserve whitespace boundaries across merges: Sennrich et al. (2016) used trailing `</w>` (or `@@` continuation markers), Google's SentencePiece uses the Unicode meta-symbol ▁ (U+2581 Lower One Eighth Block), and OpenAI's GPT-2/tiktoken maps raw UTF-8 whitespace bytes into printable Unicode characters such as `Ġ` (U+0120).

```
class CharTokenizer(Tokenizer):
    """
    Character-level tokenizer that treats each character as a separate token.
```

```
    """

    def __init__(self, vocab: Optional[List[str]] = None):
        """
        Initialize character tokenizer.
        """
        if vocab is None:
            vocab = []

        # Add special unknown token; drop repeats so every token has exactly one ID
        self.vocab = list(dict.fromkeys([Tokenizer.TOK_UNKNOWN] + list(vocab)))
        self.vocab_size = len(self.vocab)

        # Create bidirectional mappings
        self.char_to_id = {char: idx for idx, char in enumerate(self.vocab)}
        self.id_to_char = {idx: char for idx, char in enumerate(self.vocab)}

        # Store unknown token ID
        self.unk_id = 0

    def build_vocab(self, corpus: List[str]) -> None:
        """
        Build vocabulary from a corpus of text.
        """
        # Collect all unique characters
        all_chars = set()
        for text in corpus:
            all_chars.update(text)

        # Sort for consistent ordering
        unique_chars = sorted(all_chars)

        # Rebuild vocabulary with <UNK> token first
        self.vocab = [Tokenizer.TOK_UNKNOWN] + unique_chars
        self.vocab_size = len(self.vocab)

        # Rebuild mappings
        self.char_to_id = {char: idx for idx, char in enumerate(self.vocab)}
        self.id_to_char = {idx: char for idx, char in enumerate(self.vocab)}

    def encode(self, text: str) -> List[int]:
        """
        Encode text to list of character IDs.
        """
        tokens = []
        for char in text:
```

```
            tokens.append(self.char_to_id.get(char, self.unk_id))
        return tokens

    def decode(self, tokens: List[int]) -> str:
        """
        Decode list of token IDs back to text.
        """
        chars = []
        for token_id in tokens:
            # Use unknown token for invalid IDs
            char = self.id_to_char.get(token_id,
↪   Tokenizer.TOK_UNKNOWN)
            chars.append(char)
        return ''.join(chars)
```

`encode` is a dictionary lookup per character with `<UNK>` as the default, and `decode` is the reverse lookup joined into a string. A known character survives the round trip, while an unknown character becomes the literal `<UNK>`. No merging occurs, so encoding preserves the character count rather than shortening the sequence. The BPE tokenizer starts from the same alphabet and earns its shorter sequences by merging.

10.3.2 Words and the end-of-word marker

Subword tokenizers in models like GPT-2 and SentencePiece must prevent merge operations from bleeding across word boundaries while preserving the ability to reconstruct whitespace during decoding. Boundary markers solve this ambiguity by tagging whether a token terminates a word or sits within an ongoing subword stem. Training has exactly one step, repeated, and before the first step the corpus has to be written as symbols. Module 10 works on words. Its `train` takes a list of text strings, splits each with `str.split()`, counts how often each word occurs, and turns each distinct word into a list of characters once; every pair count from then on is weighted by the word's frequency rather than recounted per occurrence. Two consequences follow. No merge can ever cross a space, because no pair does. And the tokenizer needs a way to tell the `w` that ends `low` from the `w` in the middle of `lower`, because when `decode` glues tokens back together it has to know where to put the spaces. That is the marker's job.

```
def __init__(self, vocab_size: int = 1000):
    """
    Initialize BPE tokenizer.
    """
    if isinstance(vocab_size, bool) or not isinstance(vocab_size, int)
↪       or vocab_size < 1:
        raise ValueError("vocab_size must be a positive integer")
    self.target_vocab_size = vocab_size
    self.vocab = []
    self.merges = []  # List of (pair, new_token) merges
    self.token_to_id = {}
    self.id_to_token = {}
```

```
@property
def vocab_size(self) -> int:
    """Actual vocabulary size, suitable for allocating an embedding
    ↪ table."""
    return len(self.vocab)
```

Initialization stores the requested target and empty learned state. The property above counts actual entries, so it starts at zero and changes when training replaces or extends `self.vocab`. Callers need not infer that count from the target.

```
def _get_word_tokens(self, word: str) -> List[str]:
    """
    Convert word to list of characters with end-of-word marker.
    """
    if not word:
        return []

    tokens = list(word)
    tokens[-1] += Tokenizer.TOK_EOW  # Mark end of word
    return tokens
```

`_get_word_tokens("low")` returns `['l', 'o', 'w </w>']`, written `l o w</w>` below. The last character carries the marker as part of the symbol, so `w</w>` and `w` are two different entries of the alphabet from the start, and every merge that reaches the end of a word produces a symbol that says so. Training adds both forms of every observed character, even if a form never occurs in the corpus: after seeing only `ab`, it can still represent `ba`, `a`, and `b`. The boundary nevertheless constrains larger tokens, as the worked trace shows: `low</w>` is a whole-word token that `lowest` can never reuse, because inside `lowest` the `w` has no marker.

10.3.3 Counting and merging pairs

At the algorithmic core of Byte-Pair Encoding is the iterative replacement of high-frequency adjacent symbol pairs with newly minted vocabulary IDs. Naive string replacement corrupts token boundaries that have already been formed, demanding an explicit symbol-list representation during both frequency counting and merging passes. Both halves of the step have a tempting shortcut that breaks. For counting, we could ask each candidate pair how often it occurs with a search over the corpus, but that is one full pass per candidate, even though one walk over the symbols can count all candidates. For merging, we could join each word back into a string and call `str.replace`. That is wrong, not slow. After the first merge the symbols are no longer single characters, and a string search cannot see where one symbol ends. Once `es` has been merged, `widest` is the five symbols `w i d es t</w>`; there is no `(s, t</w>)` pair in it, but the letters still read `widest`, and a `replace` on the joined string would happily glue the `s` inside `es` to the `t`. We count in a single pass over the word table with a `Counter`, weighting each pair by its word's frequency, and we merge by walking the list of symbols, where the boundaries are explicit.

```python
def _count_byte_pairs(word_tokens: Dict[str, List[str]], word_freq:
    Counter) -> Counter:
    """
    Count frequency of all adjacent token pairs across all words.
    """
    pair_counts = Counter()

    for word, freq in word_freq.items():
        tokens = word_tokens[word]
        # Count adjacent pairs
        for i in range(len(tokens) - 1):
            pair = (tokens[i], tokens[i + 1])
            pair_counts[pair] += freq

    return pair_counts
```

```python
def _merge_pair(word_tokens: Dict[str, List[str]], pair: Tuple[str,
    str]) -> str:
    """
    Merge one pair everywhere it occurs in all word token lists.
    """
    merged_token = pair[0] + pair[1]

    for word in word_tokens:
        tokens = word_tokens[word]
        new_tokens = []
        i = 0
        while i < len(tokens):
            if (i < len(tokens) - 1 and
                tokens[i] == pair[0] and
                tokens[i + 1] == pair[1]):
                # Merge pair
                new_tokens.append(merged_token)
                i += 2
            else:
                new_tokens.append(tokens[i])
                i += 1
        word_tokens[word] = new_tokens

    return merged_token
```

Notice that `_merge_pair` walks left to right and skips two positions after a match. On the symbols `a a a` with the pair `(a, a)`, that yields `aa a`, not `a aa` and not an overlapping double match. The left-to-right choice fixes how overlapping matches are resolved, but the same rule has to apply during training and encoding, or the two will disagree about how a word splits, and `_apply_merges` below uses the identical loop. The helper's name says "byte pairs" because BPE's name does; the symbols it counts here are characters.

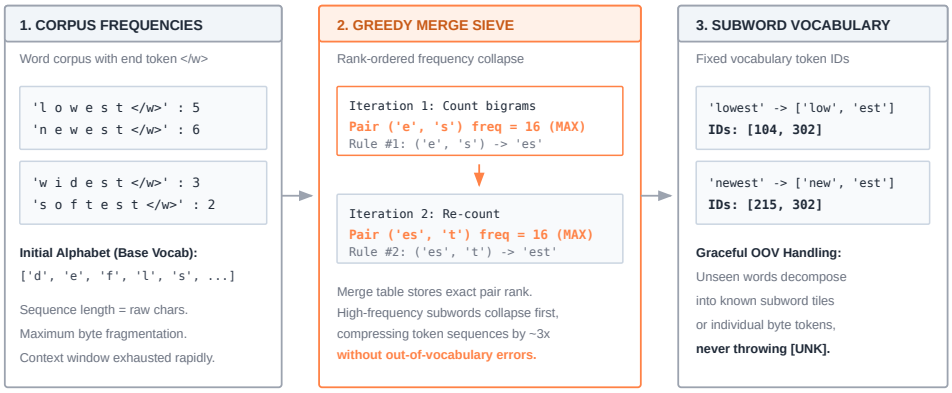


Figure 10.2: **BPE pair-frequency table and token collapse**: Training scans word frequencies to construct an adjacent digram frequency table. The most frequent pair (`('e', 's')` with count 9) is selected and merged into a new subword token `es`, collapsing the sequence length across the dictionary.

10.3.4 Training

In large language model pipelines such as LLaMA and GPT-6, tokenizer training is an offline preprocessing phase that freezes the discrete vocabulary before model architecture weights are allocated. This optimization loop balances sequence length compression against the memory footprint of downstream embedding and unembedding matrices. A language-model workflow normally trains its tokenizer before allocating the model. The requested `vocab_size` is stored as `target_vocab_size`; it is a training limit, not a promise about the resulting table. The read-only `vocab_size` property returns `len(self.vocab)`, the actual count that must size the model afterward. The lookup table in Chapter 11 has one row of D numbers per ID, and the output layer has one score per ID, so every entry we add here is a set of weights the model has to learn; at $D = 4{,}096$ in 32-bit floats that is 16 KB of table per entry, before the output layer. Some base entries are deliberately retained for words absent from the training corpus; larger merged entries earn their storage by shortening sequences.

```
def train(self, corpus: List[str], vocab_size: Optional[int] = None)
    -> None:
    """
    Train BPE on corpus to learn merge rules.
    """
    if vocab_size is not None:
        if isinstance(vocab_size, bool) or not isinstance(vocab_size,
            int) or vocab_size < 1:
            raise ValueError("vocab_size must be a positive integer")
        self.target_vocab_size = vocab_size

    # Count word frequencies and initialize character vocabulary.
    # Split each text on whitespace exactly as encode() does, so that
      the
```

```
    # symbols the trainer merges (with </w> on each word's last
    ↪ character)
    # are the symbols encode() will later look up.
    word_freq = Counter(word for text in corpus for word in
↪   text.split())
    vocab = set()
    word_tokens = {}

    for word in word_freq:
        tokens = self._get_word_tokens(word)
        word_tokens[word] = tokens
        # Every seen character must work both inside and at the end of
        ↪ a word.
        # Keeping both forms lets unseen words reuse the known
        ↪ alphabet.
        vocab.update(word)
        vocab.update(char + Tokenizer.TOK_EOW for char in word)

    self.vocab = sorted(vocab)
    if Tokenizer.TOK_UNKNOWN not in vocab:
        self.vocab = [Tokenizer.TOK_UNKNOWN] + self.vocab

    # Greedy merge loop: count pairs, merge best, repeat
    self.merges = []

    while len(self.vocab) < self.target_vocab_size:
        pair_counts = _count_byte_pairs(word_tokens, word_freq)
        if not pair_counts:
            break

        best_pair = pair_counts.most_common(1)[0][0]
        merged_token = _merge_pair(word_tokens, best_pair)
        if merged_token not in self.vocab:
            self.vocab.append(merged_token)
        self.merges.append(best_pair)

    self._build_mappings()
```

Before the loop, `train` builds a local word-frequency table and local lists of symbols. It replaces the tokenizer's old vocabulary and clears its old merges, so calling it again retrains rather than extending the previous state. Each pass counts pairs, picks the winner, and calls `_merge_pair`, which rewrites those local symbol lists in place. The merged string receives the next ID if it is new, and the pair is appended to `self.merges`. Finally, `_build_mappings` creates both lookup dictionaries. The caller's corpus strings remain unchanged; the tokenizer retains the vocabulary and rules, not the temporary word table.

Three things to notice. The loop runs until the vocabulary is full or no pair is left; it does not stop when the best pair occurs only once, so on a small corpus the last merges each glue together two symbols that meet in a single word and nowhere else, one vocabulary entry bought to shorten a single occurrence, with little opportunity for reuse. Hugging Face's BPE trainer exposes a `min_frequency` setting; adding one here would stop training

once the best pair becomes rare. When two pairs tie, `most_common` returns the one the `Counter` met first, which depends on corpus order; ties are common in small corpora and harmless, but they are why two BPE trainers given the same data can produce slightly different vocabularies. Both counting and rewriting scan the word table on every merge. With N initial symbols across distinct words and M merges, these scans give an $O(NM)$ upper bound; the lists shrink as training proceeds.

10.3.5 Encoding and decoding

Every inference request and training iteration in production LLM runtimes (such as vLLM or TensorRT-LLM) relies on the encoder to map raw text into input tensors and the decoder to reconstruct text from generated token logits. Ensuring that new inputs are segmented identically to training data requires the encoder to strictly replay learned merge ranks rather than making greedy subword choices. Text enters through `encode`; a list of predicted IDs can later return through `decode`. Decoding a complete list matters here: the final whitespace normalization means separately decoded tokens cannot simply be concatenated to reproduce whole-sequence decoding. These two methods are the ones that run on every request for the life of the model, and the encoder has one hard requirement. New text must split the way the training text split, because the model only knows the tokens it saw during training.

The tempting encoder is greedy: at each position, take the longest string that is in the vocabulary. That is a different segmentation rule and need not reproduce what the trainer saw. The merges were learned on words that already had the earlier merges applied; `st</w>` was learned from words containing `s` and `t</w>` as separate symbols after three earlier merges had run. A greedy encoder can grab a long vocabulary entry that the trainer would never have assembled at that position, and hand the model a token sequence it never saw in training. Replaying the merges in the order they were learned recreates the trainer's conditions exactly, so training text encodes exactly as it did during training, and the result is deterministic.

```
def _apply_merges(self, tokens: List[str]) -> List[str]:
    """
    Apply learned merge rules to token sequence.
    """
    if not self.merges:
        return tokens

    for merge_pair in self.merges:
        new_tokens = []
        i = 0
        while i < len(tokens):
            if (i < len(tokens) - 1 and
                tokens[i] == merge_pair[0] and
                tokens[i + 1] == merge_pair[1]):
                # Apply merge
                new_tokens.append(merge_pair[0] + merge_pair[1])
                i += 2
            else:
                new_tokens.append(tokens[i])
                i += 1
```

```
            tokens = new_tokens

        return tokens
```

```
def encode(self, text: str) -> List[int]:
    """
    Encode text using BPE.
    """
    if not self.vocab:
        return []

    # Simple word splitting (could be more sophisticated)
    words = text.split()
    all_tokens = []

    for word in words:
        # Get character-level tokens
        word_tokens = self._get_word_tokens(word)

        # Apply BPE merges
        merged_tokens = self._apply_merges(word_tokens)

        all_tokens.extend(merged_tokens)

    # Convert to IDs
    token_ids = []
    for token in all_tokens:
        token_ids.append(self.token_to_id.get(token, 0))  # 0 = <UNK>

    return token_ids
```

Core Invariant: Merge Rank Determinism

$$\text{Rank}(\text{pair}) = \text{Order of discovery}$$

Encoder must apply merges strictly in ascending rank order to guarantee deterministic segmentation matching training.

`_apply_merges` takes the rules in rank order and, for each one, walks the word with the same skip-two loop as `_merge_pair`. `encode` calls `str.split()`, then `_get_word_tokens` and `_apply_merges` for each word, then maps the resulting strings to IDs. Each helper constructs lists for this call; encoding does not alter the learned vocabulary or merge list. Every returned ID lies between zero and `vocab_size - 1` once the tokenizer has been trained.

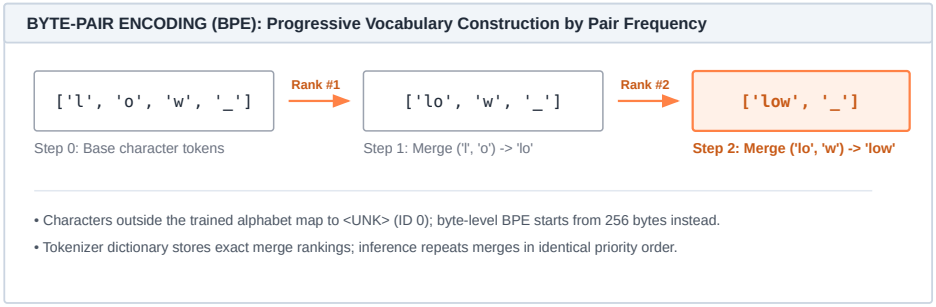


Figure 10.3: **Progressive BPE merge sequence**: Starting with base character tokens, adjacent pairs are iteratively merged according to their corpus frequency rank (`lo`, then `low`). Applying merges in strict rank order guarantees identical token segmentation during inference.

Every rule walks the current word whether or not it applies. Four learned rules therefore mean four scans per word, and adding rules increases that work even when a particular word cannot use them. An unseen character falls back to `<UNK>`. A known character in a new position does not: both its internal and word-final forms were reserved during training. The returned Python list has one entry per final token; the dataset and batching machinery of Chapter 5 can later collect compatible sequences for a model.

```
def decode(self, tokens: List[int]) -> str:
    """
    Decode token IDs back to text.
    """
    if not self.id_to_token:
        return ""

    # Convert IDs to tokens
    token_strings = []
    for token_id in tokens:
        token = self.id_to_token.get(token_id, Tokenizer.TOK_UNKNOWN)
        token_strings.append(token)

    # Join and clean up
    text = ''.join(token_strings)

    # Boundaries already are spaces, so literal text such as </w>
    ↪   stays intact.
    # Clean up extra spaces
    text = ' '.join(text.split())

    return text
```

`decode` looks each ID up and joins the token strings. It then uses `split()` and `join()` to normalize whitespace and remove the final boundary space. An invalid ID becomes literal `<UNK>`. Whitespace changes even when every character is known: `"low  lower\n"` and `"low lower"` encode identically because `str.split()` discards their separators. The round-trip contract is therefore normalized words over the observed character alpha-

bet. Literal marker text such as `</w>` survives when its characters are known; only actual spaces serve as boundaries. A byte alphabet addresses unseen characters, while preserving original whitespace requires changing the preprocessing and decoding rules.

10.4 A Worked Trace

We train on the five-word corpus `low`, `low`, `lower`, `newest`, `widest`. It contains ten distinct characters. Training reserves two forms per character, internal and word-final, plus `<UNK>`: twenty-one entries, IDs 0 through 20. Sorting places `d` at ID 1, `d</w>` at ID 2, `e` at ID 3, and so on through `w</w>` at ID 20. A target of 25 leaves room for four merges.

The local word table starts as follows; `low` has frequency two, and every other row has frequency one.

```
low:     l o w</w>
lower:   l o w e r</w>
newest:  n e w e s t</w>
widest:  w i d e s t</w>
```

Table 10.1: Four BPE merges after the twenty-one-entry base vocabulary. Counts include word frequency; tied pairs are selected in the order first encountered.

Rank	Most frequent pair	Count	New ID	Words changed
0	`(l, o)`	3	21	`low`, `lower`
1	`(lo, w</w>)`	2	22	`low`
2	`(w, e)`	2	23	`lower`, `newest`
3	`(s, t</w>)`	2	24	`newest`, `widest`

After rank 0, both `low` and `lower` start with the single symbol `lo`. Rank 1 completes `low`, but cannot merge `lo w` inside `lower`, where `w` has no boundary suffix. Rank 2 creates `we` in two words, and rank 3 creates `st</w>` in two words. The resulting table is:

```
low:     low</w>
lower:   lo we r</w>
newest:  n e we st</w>
widest:  w i d e st</w>
```

After rank 3 the vocabulary is full and the loop stops. Had we asked for more, it would have kept going; every remaining pair occurs once, and the module merges those too.

Now encode the unseen word `lowest`. Its symbols are `l o w e s t</w>`. `_apply_merges` replays the four rules in rank order, and each row below shows the word after that rule has walked it.

1. Rule 0, `(l, o)`: `lo w e s t</w>`.
2. Rule 1, `(lo, w</w>)`: no match, because the `w` in `lowest` carries no marker; `lo w e s t</w>`.
3. Rule 2, `(w, e)`: `lo we s t</w>`.

4. Rule 3, `(s, t</w>): lo we st</w>`.

Six characters became three IDs, `[21, 23, 24]`, and neither `lowest` nor `est` as a standalone word was in the training corpus. Notice what the marker did at rule 1. The whole-word token `low</w>` exists in the vocabulary, but `lowest` cannot use it, so the tokenizer splits the word as `lo we st</w>` rather than `low est`; removing the marker would change the candidate pairs, so a byte-level trainer would need its own trace before we could predict its split. The listing below runs the trace and then encodes a string with a non-ASCII letter to distinguish unseen characters from unseen word positions.

```
from tinytorch.core.tokenization import BPETokenizer

corpus = ["low", "low", "lower", "newest", "widest"]
tok = BPETokenizer(vocab_size=25)           # 21 base symbols + 4 merges
tok.train(corpus)

for rank, pair in enumerate(tok.merges):
    print(rank, pair, "->", pair[0] + pair[1], tok.token_to_id[pair[0] + pair[1]])

ids = tok.encode("lowest")
print(ids, [tok.id_to_token[i] for i in ids])   # [21, 23, 24] ['lo', 'we', 'st ']
assert tok.decode(ids) == "lowest"

ids = tok.encode("naïve widest")
print(ids, [tok.id_to_token[i] for i in ids])
# [9, 0, 0, 0, 4, 19, 5, 1, 3, 24]
# ['n', '<UNK>', '<UNK>', '<UNK>', 'e ', 'w', 'i', 'd', 'e', 'st ']
print(tok.decode(ids))                          # n<UNK><UNK><UNK>e widest
```

In lines 3 to 5, `BPETokenizer(vocab_size=25)` trains on the five-word corpus, establishing 21 base symbols and 4 learned merges. The loop in lines 7 to 8 prints the ranked merge rules corresponding to table 10.1. In lines 10 to 12, encoding the unseen word `'lowest'` applies the merge chain to yield token IDs `[21, 23, 24]` (`['lo', 'we', 'st ']`), which decode back losslessly to `'lowest'`.

The trace above lost three characters of `naïve` to `<UNK>`, and no further training on this corpus recovers them. TinyTorch's `BPETokenizer` seeds its base vocabulary from the characters it observes, so a character that never appears has no representation at all, and once an input collapses into `<UNK>` its information is gone. Modern LLM tokenizers, GPT-2 and LLaMA among them, avoid that case by seeding the base vocabulary with all 256 byte values (0x00 through 0xFF). Any UTF-8 string is then a sequence of known bytes even when no learned subword matches, so those tokenizers have no out-of-vocabulary case to handle. TinyTorch stops at the observed-character alphabet because the merge loop is the mechanism this module teaches, and byte seeding changes only which symbols that loop starts from.

The training target is not the output size. With `BPETokenizer(vocab_size=2)` and corpus `["ab"]`, the five base entries already exceed the target; no merge runs, `vocab_size` is 5, and `encode("ba")` returns `[3, 2]`. Conversely, a target of 100 on the five-word

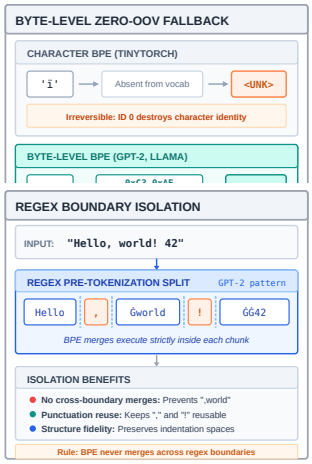
corpus stops at 34 entries after thirteen merges exhaust the available pairs. In both cases, allocate the embedding table in Chapter 11 from the actual `vocab_size`.

10.5 From TinyTorch to Production

The reference mechanism makes two production questions concrete. How can a tokenizer preserve text outside its training alphabet? How can it avoid scanning thousands of irrelevant rules for every word? Different tokenizers answer these questions differently, so it is useful to compare specific implementations rather than assume every deployed model uses one scheme.

The released GPT-2 encoder begins with UTF-8 bytes, remapped reversibly onto printable characters for its internal string operations. Its base alphabet covers all 256 byte values. It uses a regular expression (`'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+`) to partition text into lexical categories before BPE merges run. This boundary firewall ensures that contractions (such as `'ve`), numbers, and punctuation are never glued across words, preventing the tokenizer from polluting its merge table with combinations like `dog,` or `model?`. Within each chunk, it selects the lowest-ranked available pair and repeats; a cache stores earlier chunk results. Its decoder reconstructs the bytes before interpreting them as UTF-8. These choices preserve the distinction between `"low  lower\n"` and `"low lower"`, which TinyTorch intentionally discards. They also explain why a single token's bytes need not form a complete Unicode character: decoding an entire sequence is different from decoding each ID separately.

Hugging Face's BPE model interface exposes a vocabulary, ranked merges, and cache capacity, along with choices such as unknown-token handling and a word-ending suffix. Its trainer exposes the requested vocabulary size, minimum pair frequency, and initial alphabet. Those settings are part of the tokenizer's behavior, not interchangeable performance switches. A BPE model configured over observed characters does not acquire complete byte coverage merely because the library also supports byte-based preprocessing.



Regex pre-tokenization: Isolating words, punctuation, and whitespace into separate chunks prevents merges across boundaries, preserving punctuation reuse and indentation.

Table 10.2: Production alternatives address coverage and repeated work while retaining an explicit vocabulary and merge rules.

Mechanism	TinyTorch	A production alternative
Base alphabet	Both forms of each observed character, plus `<UNK>`	GPT-2 uses all byte values
Boundaries	Split on whitespace; append a space to each word	GPT-2 retains separators during pattern-based splitting
Encoding work	Scan each word once per learned rule	GPT-2 chooses among available pair ranks and caches chunks
Training limits	Target size, complete base alphabet, exhausted pairs	Hugging Face additionally exposes minimum frequency
Model interface	IDs range from 0 to actual `vocab_size - 1`	Vocabulary and preprocessing must match the model

The comparison suggests where to reduce repeated work. Our encoder does repeated work on repeated words; caching could remove it while leaving the IDs unchanged. A trainer can track which words contain a pair and update affected counts rather than scan every word again. Such changes need tests against the simple reference, because an altered tie-break or preprocessing rule can change the learned vocabulary even when the new loop is faster.

In production systems, tokenizing gigabytes of training data in Python with string loops is a major bottleneck. Modern libraries such as HuggingFace `tokenizers` are implemented in Rust: they memory-map the input text files, use lock-free trie structures or hash tables for fast merge rank lookups, and tokenize across all available CPU cores in parallel via Rayon. During autoregressive inference, the tokenizer also handles incremental streaming decoding without reparsing the entire historical token sequence.

Reserved tokens are the other difference worth naming. TinyTorch reserves one, `<UNK>` at ID 0. Production vocabularies usually reserve a few more, for jobs the merge rules cannot express: a padding token that fills shorter sequences so a batch forms a rectangle, beginning- and end-of-sequence tokens that mark document boundaries, and in chat models, tokens that separate turns. Each is a row of the embedding table and a column of the output layer like any other ID, so it costs parameters and must be learned. Each also needs a rule outside the tokenizer. The loss must skip padding positions, and a generation loop must know which ID means stop. Synthesis II shows the cost of leaving one out: concatenating sentences with no end-of-sequence token teaches the model to run from one sentence straight into the next.

The invariant that reaches the model is more important than either optimization. ID 22 denotes `low</w>` in our trace; another tokenizer can assign that same number to unrelated text. Keeping all IDs in range prevents an indexing error, but it does not detect a mismatched vocabulary. The alphabet, preprocessing, reserved tokens, ID mappings, and ordered merges must travel together with the model. Likewise, changing the vocabulary requires reconsidering the model's input and output dimensions. A model's context limit counts tokens rather than words, so the same amount of text can occupy different fractions of that limit under different segmentations.

10.6 Summary

Module 10 turns strings into stable integer sequences without tensors or gradients. `train` splits the corpus into words, counts frequencies, reserves internal and word-final forms of every observed character, and repeatedly merges the most frequent adjacent pair. It mutates the tokenizer's vocabulary and ordered merge list while leaving the caller's text unchanged. The actual `vocab_size` counts the resulting entries, which can be above or below the requested target.

`encode` uses that fixed state: split, mark word endings, replay merges, then look up IDs. `decode` joins their strings and normalizes whitespace. Known characters survive in new words and new word positions; unseen characters become `<UNK>`. The alphabet determines coverage, while the learned rules determine segmentation. A byte alphabet and whitespace-preserving preprocessing address separate limitations of this reference. Chapter 11 accepts the resulting IDs and builds the missing numerical operation: selecting a learned vector for each ID, then representing where that token occurs in the sequence.

10.7 Further Reading

- **Sennrich, Haddow, and Birch** (2016). "Neural machine translation of rare words with subword units." *ACL*. Byte-pair encoding adapted to text, the algorithm this chapter implements.
- **Radford et al.** (2019). "Language models are unsupervised multitask learners." *OpenAI*. GPT-2, including the byte-level base alphabet in the production comparison above.
- **Kudo and Richardson** (2018). "SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing." *EMNLP*. Tokenizing raw text, whitespace included, without a language-specific splitting step.

🔥 Chapter 11

Embeddings and Positional Encoding: Turning Token IDs into Vectors

Discrete token IDs are arbitrary categorical integers with no intrinsic geometric relationship; token 1542 is not mathematically "closer" to token 1543 than to token 50000. Feeding raw integers into linear layers incorrectly imposes ordinal magnitude on categorical labels. To enable semantic reasoning, discrete token IDs must be mapped into continuous, high-dimensional vector spaces where geometric distance reflects semantic similarity, complemented by positional encodings that supply critical sequence order.

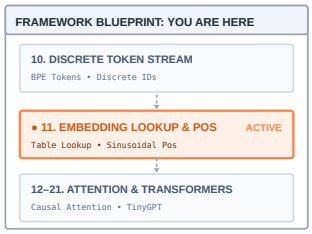
In PyTorch, looking up token embeddings and injecting positional information is achieved in a few expressive lines:

```python
import torch
import torch.nn as nn

vocab_size, d_model = 1000, 64
token_embed = nn.Embedding(vocab_size, d_model)
pos_embed = nn.Embedding(512, d_model)  # Maximum context length 512

token_ids = torch.tensor([[42, 108, 999]])  # 1 sequence of 3 tokens
positions = torch.arange(3).unsqueeze(0)

# Combine token semantic features with positional coordinates
x = token_embed(token_ids) + pos_embed(positions)
print("Representation tensor shape:", x.shape)  # (1, 3, 64)
```



Framework Blueprint: Module 11 projects discrete token integers into dense continuous geometry and encodes sequential ordering.

Underneath that simple addition, the runtime manages several low-level memory and gradient dynamics:

1. **Direct table gathering over one-hot matrix multiplication:** Rather than materializing massive, sparse one-hot matrices containing millions of zeros and executing bloated matrix multiplies, the embedding layer performs an efficient row-gather operation directly into physical DRAM.
2. **Positional coordinate injection:** Because attention mechanisms treat inputs as an unordered set, the runtime adds positional vectors directly into token representations, ensuring that the model can distinguish *"dog bites man"* from *"man bites dog"*.
3. **Scatter-add gradient accumulation:** In the backward pass, when identical token IDs appear multiple times within a batch, their gradients must be accumulated

into shared table rows using an unbuffered scatter-add (`np.add.at`), preventing subsequent occurrences from overwriting earlier derivatives.

This chapter constructs TinyTorch's embedding subsystem, converting discrete token sequences into continuous representation tensors (B, S, D) ready for self-attention in Chapter 12. Before building the embedding table, we must inspect why feeding raw integers fails and why one-hot matrices are computationally prohibitive.

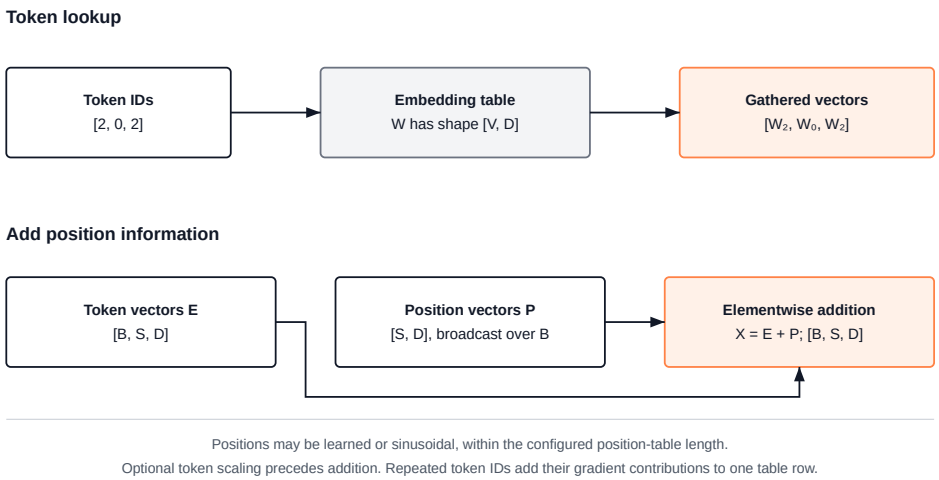


Figure 11.1: The embedding pipeline. A token ID selects one row of the embedding table; the row for that token's position is added; the sum is the vector the rest of the network sees.

11.1 The Problem

↩ **Jump back:** Chapter 10 *Discrete token ID integers serve as row indices into the embedding weight table.*

Suppose we skip the table and feed the integer straight into a `Linear` layer. The layer computes $w \cdot 1542$ for the token `"dog"` and $w \cdot 1543$ for whatever token happens to sit next to it in the vocabulary, and $w \cdot 50000$ for a token many thousands of slots away. The network is being told that adjacent IDs have similar magnitudes and that larger IDs should scale the same feature more strongly. Token IDs are labels, not quantities; the order the tokenizer assigned them carries no meaning, and any arithmetic on the raw integer invents a meaning that is not there.

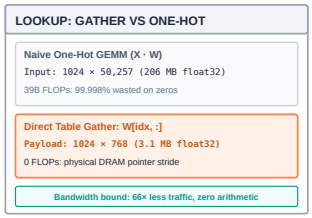
The textbook fix is the one-hot vector that Chapter 4 used for targets. Represent token i as a vector of length V with a 1 in slot i and zeros elsewhere, then multiply by a weight matrix. Every token is now equally far from every other, which is the right starting point, and a `Linear` layer can learn from it. The cost is the problem. With a vocabulary of $V = 50{,}257$ and a modest batch of 1,024 tokens, the one-hot input is $1{,}024 \times 50{,}257 \approx 51$ million floats, about 206 MB, of which exactly 1,024 are nonzero. The matrix multiply that follows, against a $50{,}257 \times 768$ weight matrix, performs about 39 billion multiplications, and only one in every 50,257 of them is by something other than zero.

The second problem appears once the integers are gone. Take the two sentences *"the dog bit the man"* and *"the man bit the dog"*. Their token vectors are the same five rows in a

different order. Any operation that treats its inputs as a set, such as summing them or averaging them, produces identical output for both sentences.

```python
import numpy as np

rows = {"the": [1.0, 0.0], "dog": [0.0, 1.0],
        "bit": [1.0, 1.0], "man": [0.5, 0.5]}
s1 = ["the", "dog", "bit", "the", "man"]
s2 = ["the", "man", "bit", "the", "dog"]
sum1 = sum(np.array(rows[w]) for w in s1)
sum2 = sum(np.array(rows[w]) for w in s2)
print(sum1, sum2, np.array_equal(sum1, sum2))
# [3.5 2.5] [3.5 2.5] True
```



Gather vs. One-Hot GEMM: Pointer-indexed table gathering eliminates sparse intermediate allocations and wasted zero arithmetic.

Unmasked self-attention preserves the rows rather than summing them away, but without positional information it follows any permutation of those rows. Swapping two tokens swaps their outputs; it does not tell the mechanism which came first. Chapter 12 will compute interactions between rows, so each row needs to carry the association between a token and its position.

Pause & Predict: The Parameter Weight of the Vocabulary

In a language model with vocabulary size $V = 100{,}000$ and hidden dimension $D = 4{,}096$, how many float32 parameters live strictly in the embedding table before a single Transformer layer is evaluated? If the batch size is $B = 4$ and sequence length is $S = 512$, what percentage of rows in the table are actually accessed in one forward pass?

11.2 The Idea

11.2.1 A table, not a multiply

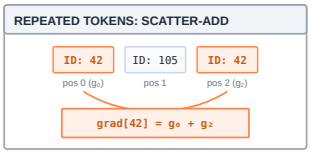
↪ **Jump ahead:** Chapter 12 *Continuous sequence representation tensors (B, S, D) feed scaled dot-product attention.*

In all modern sequence models—including BERT, GPT, and LLaMA—the input layer maps discrete token IDs into continuous semantic vectors. To eliminate the massive memory allocation and wasted arithmetic of multiplying one-hot vectors by a weight matrix, runtimes implement embeddings as a direct memory lookup (row-gather) into an embedding table. Take a three-row table `W = [[1, 2], [3, 4], [5, 6]]`. Token 2 selects `[5, 6]`; the sequence `[2, 0, 2]` selects `[[5, 6], [1, 2], [5, 6]]`. A one-hot row `[0, 0, 1]` multiplied by `W` produces the same `[5, 6]`, because it discards the first two rows and keeps the third. Repeating a token repeats its current vector; the lookup does not change the table.

Write the one-hot vector for token i as $\mathbf{e}_i$, a row vector of length V, and the weight matrix as W with shape $V \times D$, where D is the width of the vectors the rest of the network will use. The product picks out one row:

$$\mathbf{x}_i = \mathbf{e}_i W = W[i, :]$$

Every term in the sum is zero except the one where the 1 sits, so the product *is* row i. Nobody needs to build the one-hot vector or run the multiply. An `Embedding` layer stores W and, given an ID, copies row i out of it. The gather itself performs no arithmetic on the row values; it moves D floats from one place to another. An indexed read of this kind, which pulls selected rows out of a larger array, is called a **gather**.



Scatter-add backward routing for repeated token IDs.

The rows are ordinary parameters. They start random, and gradient descent moves them. Because the forward pass only touched the rows that were looked up, only those rows receive a gradient. If a batch contains only three occurrences of token 42, row 42 receives the sum of three gradients, one from each place it was used; the other $V - 1$ rows receive zero. With other tokens present, their rows receive their own contributions. The backward pass is the mirror of the gather, a **scatter-add**: take each output gradient and add it into the row it came from.

Nothing in the table says that `"dog"` and `"cat"` should be near each other. That closeness emerges from training, because the loss pushes tokens that appear in similar contexts toward rows that produce similar predictions. The table only makes closeness *possible*, by giving the model D continuous numbers per token instead of one categorical label.

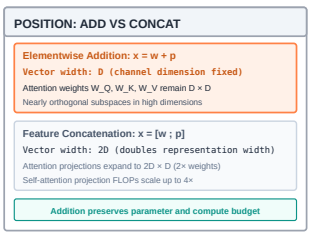
11.2.2 Position is a second table

Because self-attention operations are inherently permutation-equivariant, Transformer models like BERT and GPT-2 cannot distinguish word order without explicit coordinate hints. Adding a dedicated positional embedding table directly to token representations injects sequence order into the activations without increasing tensor dimensionality. The simplest way to tell the network where a token sits is a second table, P, with shape $S_{\text{max}} \times D$, one row per position up to some maximum length S_{max}. The vector for token i at position t is the sum of the two rows:

$$\mathbf{x}_{i,t} = W[i,:] + P[t,:]$$

Now `"dog"` at position 1 and `"dog"` at position 4 differ by $P[1,:] - P[4,:]$. The individual rows contain different token-position pairs for the two sentences. Summing the rows immediately would still lose order: the sum of token rows and the sum of position rows are unchanged by the swap. Attention must use the combined rows before any such reduction. The addition broadcasts across a batch, since every sequence in the batch uses the same position rows.

There are two ways to fill P. The first is to learn it, exactly like W. The second, from Vaswani et al. (2017), is to fill it once with a sine and cosine formula and never train it. For an even embedding width, that formula makes the row for position $t + \Delta$ a fixed rotation of the row for position t, whatever t is, and that one property is what the production schemes at the end of this chapter build on. The formula itself appears with the code that fills the table, where the reason for each piece of it is easier to see.



Position Injection: Elementwise addition preserves representation width D, avoiding parameter explosion in downstream attention projections.

11.3 The Code

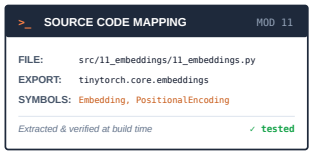
11.3.1 The embedding table

Consider the same $V = 50{,}257$, $D = 768$ table and batch of 1,024 token IDs used in The Problem. The one-hot route allocates about 206 MB for its input and performs

about 39 billion multiplications. A gather instead selects 1,024 rows of width 768. These are analytical counts, not measured timings; they identify the work avoided without assuming a particular machine or matrix-multiplication library.

For a contiguous float32 table, row i starts $i \times D \times 4$ bytes into the buffer (Chapter 1), following standard row-major contiguous strides $(D, 1)$. Selecting it copies D values, or 3,072 bytes at this width. The batch therefore contains about 3.1 MB of selected values. This is the payload size, not the total memory traffic of TinyTorch: NumPy creates the indexed result, and the Tensor constructor copies the result into its own storage. The lookup is constant-time with respect to vocabulary size for a fixed width, but moving one row still costs $O(D)$.

Module 11 follows the pattern of every layer since Chapter 2: an operation that does the work on arrays, and a layer that owns the table and calls it. The operation is the interesting half, because its backward routes each output gradient to the row that supplied its values. Three things to watch for: the one-line gather, the way the indices travel, and the scatter-add.



Source Code Mapping: Module 11 extracts embedding table lookup and sinusoidal positional encodings directly from the working repository.

```python
class EmbeddingFunction(Function):
    """
    The embedding lookup operation: forward gathers rows, backward scatters gradients back.
    """

    def forward(self, weight: np.ndarray) -> np.ndarray:
        """Gather one row per token id, i.e. weight[self.indices] (self.indices is an int array)."""
        return weight[self.indices]

    def backward(self, grad_output: np.ndarray) -> Tuple[Optional[np.ndarray]]:
        """
        Compute gradient for embedding lookup.
        """
        weight, = self.inputs
        grad_weight = None

        if weight.requires_grad:
            # Initialize gradient with zeros
            grad_weight = np.zeros_like(weight.data)

            # Scatter gradients back to embedding weights
            # np.add.at accumulates gradients for repeated indices
            indices_flat = np.asarray(self.indices).flatten()
            grad_output_reshaped = grad_output.reshape(-1, grad_output.shape[-1])

            np.add.at(grad_weight, indices_flat, grad_output_reshaped)

        return (grad_weight,)
```

`weight[self.indices]` is NumPy's fancy indexing, and it does exactly what the math says. The indices are not an input to the operation, they are a parameter of it, passed as `indices=` when the layer calls `apply` and stored on the node; the table is the only input,

so it is the only tensor the graph records, which is right, because nothing differentiates with respect to a token ID. `indices` can be a flat array of S IDs or a $B \times S$ batch; the result has one extra trailing dimension of size D either way.

The backward is where the operation earns its class. Here is the trap. The obvious way to send gradients back to the table is `grad_weight[indices] += grad_output`, and for the sequence `[2, 0, 3]` it works. For `[2, 0, 2]` it silently gives the wrong number. NumPy evaluates that line as a read, an add, and a write, so the two gradients for row 2 are both added to the *original* row and the second write overwrites the first. Row 2 ends up with one gradient instead of two, and nothing warns you. Chapter 6's rule that a tensor used twice receives two contributions has to hold for table rows as well, which is why the method uses `np.add.at`.[47] That is the scatter-add. For every flattened index it adds the matching gradient row into `grad_weight`, and repeated indices accumulate instead of racing. Rows that were never looked up stay zero. Notice what the method allocates, though. `grad_weight` is a full $V \times D$ array, 154 MB at the example dimensions, of which a batch of 1,024 tokens fills at most 1,024 rows. We pay to zero it and pay again when the optimizer reads it. The production bridge below shows what PyTorch's `sparse=True` does about that.

Why is `grad_weight[indices] += grad_output` one of the most insidious bugs in autograd implementations? In Python and NumPy, `a[idx] += b` is syntactic sugar for `a[idx] = a[idx] + b`. When the same token ID appears multiple times in a single batch (e.g. `[2, 0, 2]`), NumPy gathers `a[[2, 0, 2]]` (two copies of the original row 2), adds the gradients, then writes row 2 twice; the second write replaces the first, so one gradient is lost. The backward pass executes without throwing any error or warning, but silently drops gradient signal for repeated tokens. Only unbuffered or atomic accumulation (`np.add.at` or CUDA atomic adds) guarantees correct mathematical accumulation.

[47] Formally, if the forward operation is index-gather $Y = \text{gather}(W, \text{indices})$, its vector-Jacobian product is scatter-add $\nabla_W L = \text{scatter_add}(\mathbf{0}, \text{indices}, \nabla_Y L)$. In PyTorch ATen, this dispatches to the C++ kernel `at::native::embedding_dense_backward` or `scatter_add_`, which uses hardware atomic addition (`atomicAdd` in CUDA) to prevent simultaneous writes when identical token IDs occur within a micro-batch.

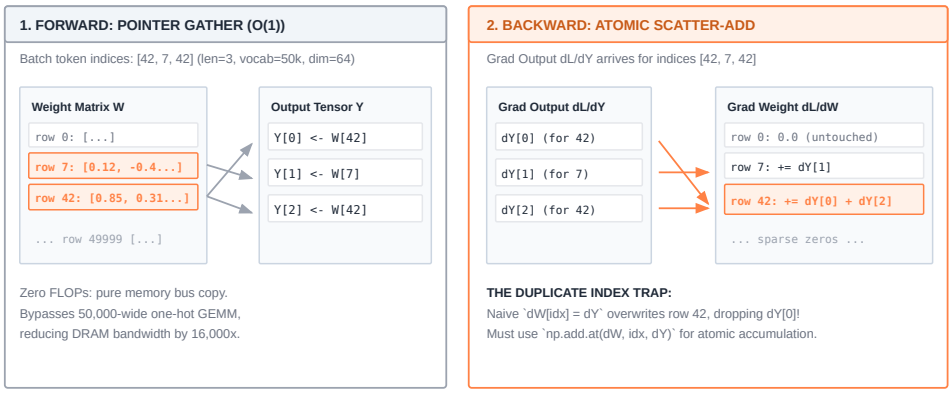


Figure 11.2: **Embedding table memory mechanics: forward gather and backward scatter-add**: In the forward pass, token IDs index directly into table rows, gathering vectors without constructing one-hot matrices. In the backward pass, repeated token IDs in the same sequence require atomic accumulation (`np.add.at`) into gradient rows, avoiding the silent write-collision data loss of naive += indexing.

The layer holds the table, initialized with the Xavier-uniform bound $\sqrt{6/(V+D)}$ used in the source, checks the IDs, and calls the operation.

```python
class Embedding:
    """
    Learnable embedding layer that maps token indices to dense vectors.
    """

    def __init__(self, vocab_size: int, embed_dim: int):
        """
        Initialize embedding layer with Xavier-uniform weights.
        """
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim

        # Xavier initialization for better gradient flow.
        # requires_grad=True is not decoration: apply() only records the
        # lookup when the weight requires grad, so without this flag the
        # backward you wrote above is never reached and the table never learns.
        limit = math.sqrt(6.0 / (vocab_size + embed_dim))
        self.weight = Tensor(
            rng.uniform(-limit, limit, (vocab_size, embed_dim)),
            requires_grad=True
        )

    def forward(self, indices: Tensor) -> Tensor:
        """
        Forward pass: lookup embeddings for given indices.
        """
        # Handle input validation
        # Tensor stores float32, but token IDs must still be finite integers.
        if not np.all(np.isfinite(indices.data)) or np.any(indices.data != np.floor(indices.data)):
            raise ValueError("Embedding token IDs must be finite integers")
        if np.any(indices.data >= self.vocab_size) or np.any(indices.data < 0):
            min_idx = int(np.min(indices.data))
            max_idx = int(np.max(indices.data))
            raise ValueError(
                f"Embedding index out of range for vocabulary size {self.vocab_size}\n"
                f"  ✗ Found indices: min={min_idx}, max={max_idx} (valid range: 0 to {self.vocab_size - 1})\n"
                f"  💡 Token IDs must be within the vocabulary. IDs >= vocab_size reference non-existent tokens\n"
                f"  🔧 Check your tokenizer output, or increase vocab_size to at least {max_idx + 1}"
            )

```

```
        # Perform embedding lookup through the operation (advanced
        ↪  indexing inside
        # EmbeddingFunction.forward, equivalent to one-hot
        ↪  multiplication but much
        # more efficient). Module 06's apply() records it for backward.
        return EmbeddingFunction.apply(self.weight,
        ↪  indices=indices.data.astype(int))

    def __call__(self, indices: Tensor) -> Tensor:
        """Allows the embedding to be called like a function."""
        return self.forward(indices)

    def parameters(self) -> List[Tensor]:
        """Return trainable parameters."""
        return [self.weight]

    def __repr__(self):
        ...
```

The IDs arrive as a `Tensor`, because that is what the loader of Chapter 5 hands over. Its storage is float32, so `forward` first checks that every value is finite and integer-valued, then checks the range `0 <= ID < vocab_size`, and only then converts the array to integer indices. A value such as `1.5` raises `ValueError`; it is not rounded down to token 1. Negative values are rejected too, preventing NumPy from interpreting `-1` as the last vocabulary row. The table is created with `requires_grad=True` because `apply` records an operation only when an input requires a gradient. Without that flag the layer would return the same forward values while losing the path that teaches the table.

The state boundary is small. `Embedding.weight` persists across calls; the integer index array belongs to the lookup node for one call. `Embedding.forward` validates the IDs, then `EmbeddingFunction.apply(weight, indices=...)` constructs that node, gathers the rows, and returns a Tensor whose backward path leads to `weight`. Forward changes neither the IDs nor the table. Backward accumulates into `weight.grad`; the optimizer of Chapter 7 is the component that later changes `weight.data`. The returned data does not share storage with the table, even though its gradient remains connected to it.

11.3.2 The sinusoidal table

Introduced in the seminal Transformer architecture (Vaswani et al.) and foundational to modern relative schemes like RoPE, sinusoidal positional encodings offer a deterministic mathematical alternative to learned position tables. By projecting token positions across a geometric progression of frequencies, fixed sinusoidal encodings eliminate position parameter storage while allowing attention heads to compute relative token distances. The two occurrences of token 2 still have identical vectors after the gather. A position row makes them distinguishable before Chapter 12 introduces interactions between tokens. Chapter 13 will place this composition at the front of its transformer model.

A learned `PositionalEncoding` owns $S_{\max} \times D$ parameters. For $S_{\max} = 1{,}024$ and $D = 768$, that is 786,432 values. With positions numbered from zero, row t only receives a contribution from a sequence containing position t; in ordinary full-sequence training that requires at least $t + 1$ tokens. Later rows may therefore see less data. A requested

position beyond `max_seq_len - 1` has no row at all. This limit comes from the allocated table, even if every training sequence was shorter.

A fixed formula offers a different trade-off. It can compute a row for a new position without training another parameter, although the row still needs computation and storage. The sinusoidal construction uses pairs of sine and cosine values. For position t and dimension pair $(2k, 2k+1)$:

$$P[t, 2k] = \sin(\omega_k t), \qquad P[t, 2k+1] = \cos(\omega_k t), \qquad \omega_k = \frac{1}{10000^{2k/D}}$$

Here ω_k is the angular frequency of pair k, and the frequencies fall off geometrically. Pair 0 has $\omega_0 = 1$ and completes a cycle every $2\pi \approx 6$ positions; for large even D, the last pair approaches $\omega = 1/10000$ and changes much more slowly over a few thousand positions. Think of a clock with $D/2$ hands, each turning slower than the one before. The fast hands distinguish neighboring positions, the slow hands distinguish distant ones, and the full row combines all these phases. The period of one pair is not a period of the whole table, and the formula alone does not guarantee that a trained model will generalize to longer contexts.

The table is a plain function of two integers. It builds the frequencies once, writing $10000^{-2k/D}$ as $\exp(-2k \ln 10000/D)$, then fills even columns with sines and odd columns with cosines, with a guard for an odd width so the last column is not left empty.

```python
def _compute_sinusoidal_table(max_len: int, embed_dim: int) ->
    np.ndarray:
    """
    Compute the raw sinusoidal positional encoding table as a numpy
    array.
    """
    # Create position indices [0, 1, 2, ..., max_len-1]
    position = np.arange(max_len, dtype=np.float32)[:, np.newaxis]  #
    (max_len, 1)

    # Create dimension indices for calculating frequencies
    div_term = np.exp(
        np.arange(0, embed_dim, 2, dtype=np.float32) *
        -(math.log(10000.0) / embed_dim)
    )  # (embed_dim//2,)

    # Initialize the positional encoding matrix
    pe = np.zeros((max_len, embed_dim), dtype=np.float32)

    # Apply sine to even indices (0, 2, 4, ...)
    pe[:, 0::2] = np.sin(position * div_term)

    # Apply cosine to odd indices (1, 3, 5, ...)
    if embed_dim % 2 == 1:
        # Handle odd embed_dim by only filling available positions
        pe[:, 1::2] = np.cos(position * div_term[:-1])
    else:
        pe[:, 1::2] = np.cos(position * div_term)
```

```
26
27      return pe
```

```
1  def create_sinusoidal_embeddings(max_seq_len: int, embed_dim: int) ->
   ↪  Tensor:
2      """
3      Create sinusoidal positional encodings as used in "Attention Is
   ↪  All You Need".
4      """
5      pe = _compute_sinusoidal_table(max_seq_len, embed_dim)
6      return Tensor(pe)
```

`position * div_term` multiplies a column of positions by a row of frequencies, so each entry is one position times one frequency. NumPy broadcasting does the work of a double loop. Row 0 is always `[0, 1, 0, 1, ...]`, because $\sin 0 = 0$ and $\cos 0 = 1$; that is a quick sanity check when something looks wrong. `create_sinusoidal_embeddings` wraps the array in a `Tensor` without `requires_grad`, so the table does not require gradients and `EmbeddingLayer.parameters()` excludes it. The implementation still allocates only `max_seq_len` rows; evaluating the formula at a larger position would require extending that allocation. Both fixed and learned tables occupy $4S_{\max}D$ bytes of float32 value storage here, but only learned positions need parameter gradients and optimizer state.

Two properties of the formula are worth keeping, because the production schemes lean on both. First, for even D, each sine and cosine pair has length exactly one, so every row of P has length $\sqrt{D/2}$, and adding it shifts a token row by a vector of the same length at every position, in a position-dependent direction. Second, also for even D, the row for position $t+\Delta$ is a fixed linear function of the row for position t, one that depends on the offset Δ and not on t. Within each pair, the angle-addition identities for sine and cosine give

$$\begin{bmatrix} P[t+\Delta, 2k] \\ P[t+\Delta, 2k+1] \end{bmatrix} = \begin{bmatrix} \cos(\omega_k \Delta) & \sin(\omega_k \Delta) \\ -\sin(\omega_k \Delta) & \cos(\omega_k \Delta) \end{bmatrix} \begin{bmatrix} P[t, 2k] \\ P[t, 2k+1] \end{bmatrix}$$

With the displayed sine-first coordinate order, the 2×2 matrix is a rotation by $-\omega_k \Delta$ under the usual counterclockwise-positive convention. A layer that wants to ask "what is Δ positions back from me" can therefore learn a single fixed matrix instead of a separate rule for every absolute position. That is the property The Idea promised, and the production bridge returns to rotation-based position information after the additive mechanism is complete.

11.3.3 The positional layer

For a batch of two sequences with two tokens each and $D = 2$, the token vectors have shape `(2, 2, 2)`. A position slice of shape `(2, 2)` supplies just four values. Reshaping it to `(1, 2, 2)` tells addition to reuse those same values for both sequences. An explicit `np.tile` would first duplicate them into eight values. At larger sizes that duplication becomes costly: tiling positions for eight sequences of 2,048 tokens at $D = 4{,}096$ would materialize about 268 MB before the addition even starts.

NumPy broadcasting avoids materializing that batch-sized position operand. It does not make the whole TinyTorch path allocation-free. `Tensor` slicing and reshaping return wrappers with their own copied storage, and addition allocates the output. What is avoided is the explicit $B \times S \times D$ tiled position table, not every copy of the smaller $S \times D$ slice.

The backward is where the broadcast has to be undone. Row t of the table was added to position t of every sequence in the batch, so it was used B times, and Chapter 6's rule is that a value used B times collects B gradient contributions. The output gradient therefore has to be summed over the batch axis before it can land in the table, and the slice that picked `seq_len` rows out of the table has to be undone as well, so that the gradient lands in the right rows and the rest stay zero. Module 11 writes none of that. Its learned `PositionalEncoding` slices the table with `Tensor` indexing, reshapes with `Tensor.reshape`, and adds with `+`, three operations Chapter 6 already knows how to reverse: `Add` reduces the broadcast batch axis with `_reduce_broadcast_grad`, `Reshape` restores the (S, D) shape, and `Slice` scatters the rows back into a zero table. The layer's job is only to use tensor operations and stay out of the way.

```
class PositionalEncoding:
    """
    Learnable positional encoding layer.
    """

    def __init__(self, max_seq_len: int, embed_dim: int):
        """
        Initialize learnable positional encoding.
        """
        self.max_seq_len = max_seq_len
        self.embed_dim = embed_dim

        # Initialize position embedding matrix
        # This scale depends on embed_dim, not vocabulary size. For large
        # vocabularies it can exceed the token initialization scale; optional
        # token scaling in EmbeddingLayer changes their relative magnitudes.
        # Learned positions are returned by parameters(), so the optimizer will
        # try to update them -- they have to carry gradients for that to mean
        # anything. (Sinusoidal encodings are the opposite: fixed by design.)
        limit = math.sqrt(2.0 / embed_dim)
        self.position_embeddings = Tensor(
            rng.uniform(-limit, limit, (max_seq_len, embed_dim)),
            requires_grad=True
        )

    def forward(self, x: Tensor, start_pos: int = 0) -> Tensor:
        """
        Add positional encodings to input embeddings.
```

```
        """
        if len(x.shape) == 2:
            raise ValueError(
                f"Expected 3D input (batch, seq, embed), got 2D: {x.shape}\n"
                f"  ✗ Missing batch dimension\n"
                f"  🔔 PositionalEncoding expects batched embeddings, not single sequences\n"
                f"  🔧 Add batch dim: x.reshape(1, {x.shape[0]}, {x.shape[1]})"
            )
        elif len(x.shape) != 3:
            raise ValueError(
                f"Expected 3D input (batch, seq, embed), got {len(x.shape)}D: {x.shape}\n"
                f"  ✗ Input must have exactly 3 dimensions\n"
                f"  🔔 PositionalEncoding expects shape (batch_size, sequence_length, embedding_dim)"
            )

        batch_size, seq_len, embed_dim = x.shape
        if not isinstance(start_pos, (int, np.integer)) or start_pos < 0:
            raise ValueError("start_pos must be a nonnegative integer")
        if start_pos + seq_len > self.max_seq_len:
            raise ValueError(
                f"Sequence runs past the maximum: positions {start_pos}..{start_pos + seq_len - 1} with max_seq_len={self.max_seq_len}\n"
                f"  ✗ Input has {seq_len} positions starting at {start_pos}, but only {self.max_seq_len} are available\n"
                f"  🔔 Learned positional encodings have a fixed maximum length set at initialization\n"
                f"  🔧 Either truncate input to {self.max_seq_len} tokens, or create a new PositionalEncoding(max_seq_len={start_pos + seq_len}, ...)"
            )

        if embed_dim != self.embed_dim:
            raise ValueError(
                f"Embedding dimension mismatch: input has {embed_dim}, expected {self.embed_dim}\n"
                f"  ✗ PositionalEncoding was created with embed_dim={self.embed_dim}, but input has embed_dim={embed_dim}\n"
                f"  🔔 Token embeddings and positional encodings must have the same dimension to be added together\n"
```

```
                f"  🔧 Ensure your Embedding layer uses embed_dim={self.embed_dim}, or create PositionalEncoding(embed_dim={embed_dim}, ...)"
            )

        # Slice position embeddings for this sequence length using Tensor slicing
        pos_embeddings = self.position_embeddings[start_pos:start_pos + seq_len]  # (seq_len, embed_dim)

        # Reshape to add batch dimension: (1, seq_len, embed_dim).
        # Use Tensor.reshape, not Tensor(pos_embeddings.data[np.newaxis]).
        # Reading .data and re-wrapping it builds a brand new leaf tensor, which
        # cuts these positions out of the graph -- the forward output looks
        # identical and the gradient silently never reaches the parameter.
        pos_embeddings_batched = pos_embeddings.reshape(1, seq_len, embed_dim)

        # Add positional information
        result = x + pos_embeddings_batched

        return result

    def __call__(self, x: Tensor, start_pos: int = 0) -> Tensor:
        """Allows the positional encoding to be called like a function."""
        return self.forward(x, start_pos)

    def parameters(self) -> List[Tensor]:
        """Return trainable parameters."""
        return [self.position_embeddings]

    def __repr__(self):
        ...
```

Before those three operations, `PositionalEncoding.forward` requires a 3D `(batch, seq, embed)` input with the configured embedding width. It also checks that `start_pos` is a nonnegative integer and `start_pos + seq_len <= max_seq_len`; an invalid range raises `ValueError` before slicing. These checks establish an unambiguous contract even where NumPy could broadcast an unintended shape. The argument `start_pos=1`, for example, selects rows 1 and 2 for a two-token call. Chapter 19 will use that offset when processing only a new suffix of a sequence.

The reshape must remain a Tensor operation. Rewrapping `pos_embeddings.data` in a fresh Tensor would produce the same forward values but create a new leaf, cutting the learned table out of the graph (violating the autograd tape connectivity established in Chapter 6). Preserving a gradient connection is distinct from sharing array storage.

The current `reshape` copies values through its wrapper and still records how gradients return to the original parameter.

11.3.4 Putting the two tables together

Production Transformer architectures unify token semantics and sequence geometry at their input stage, synthesizing discrete token sequences into a contiguous (B, S, D) activation tensor. Wrapping token lookup and positional injection into a single composite layer establishes a clean, modular front-end contract that feeds directly into downstream self-attention blocks. The last step is wiring the tables together, and this is where Chapter 13 picks up. `EmbeddingLayer` owns a token table and one of three position choices: a learned `PositionalEncoding`, the fixed sinusoidal table, or nothing. Its `forward` is written in a separate cell and attached to the class.

```
class EmbeddingLayer:
    """
    Complete embedding system combining token and positional embeddings.
    """

    def __init__(
        self,
        vocab_size: int,
        embed_dim: int,
        max_seq_len: int = 512,
        pos_encoding: str = 'learned',
        scale_embeddings: bool = False
    ):
        """
        Initialize complete embedding system.
        """
        # The contract table above says these must be positive, so enforce it here.
        # A zero or negative width otherwise builds a (V, 0) table that fails much
        # later, inside an unrelated broadcast.
        for field, value in (("vocab_size", vocab_size), ("embed_dim", embed_dim),
                             ("max_seq_len", max_seq_len)):
            if not isinstance(value, (int, np.integer)) or value <= 0:
                raise ValueError(f"{field} must be a positive integer, got {value!r}")

        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        self.max_seq_len = max_seq_len
        self.pos_encoding_type = pos_encoding
        self.scale_embeddings = scale_embeddings

        # Token embedding layer
        self.token_embedding = Embedding(vocab_size, embed_dim)

```

```
        # Positional encoding
        if pos_encoding == 'learned':
            self.pos_encoding = PositionalEncoding(max_seq_len,
            ↪  embed_dim)
        elif pos_encoding == 'sinusoidal':
            # Create fixed sinusoidal encodings (no parameters)
            self.pos_encoding =
            ↪  create_sinusoidal_embeddings(max_seq_len, embed_dim)
        elif pos_encoding is None:
            self.pos_encoding = None
        else:
            raise ValueError(
                f"Unknown positional encoding type:
                ↪  '{pos_encoding}'\n"
                f"  ✗ pos_encoding must be 'learned', 'sinusoidal',
                ↪  or None\n"
                f"  🔔 'learned' = trainable position embeddings
                ↪  (task-specific but fixed max length)\n"
                f"     'sinusoidal' = mathematical sin/cos patterns
                ↪  (no parameters, can extrapolate)\n"
                f"     None = no positional encoding (order-agnostic
                ↪  model)\n"
                f"  🔧 Use: EmbeddingLayer(...,
                ↪  pos_encoding='learned') or
                ↪  pos_encoding='sinusoidal'"
            )

    def __call__(self, tokens: Tensor, start_pos: int = 0) -> Tensor:
        """Allows the embedding layer to be called like a function."""
        return self.forward(tokens, start_pos)

    def parameters(self) -> List[Tensor]:
        """Return all trainable parameters."""
        params = self.token_embedding.parameters()
        if self.pos_encoding_type == 'learned':
            params.extend(self.pos_encoding.parameters())
        return params

    def __repr__(self):
        ...
```

```
def emblayer_forward(self, tokens: Tensor, start_pos: int = 0) ->
↪  Tensor:
    """
    Forward pass through complete embedding system.
    """
    # Validate start_pos once, for every strategy. Checking it only
    ↪  inside the
    # 'learned' and 'sinusoidal' branches would let pos_encoding=None
    ↪  silently
    # accept a negative or non-integer position.
```

```
    if not isinstance(start_pos, (int, np.integer)) or start_pos < 0:
        raise ValueError("start_pos must be a nonnegative integer")

    # Handle 1D input by adding batch dimension
    if len(tokens.shape) == 1:
        # NOTE: Tensor reshape preserves gradients
        tokens = tokens.reshape(1, -1)
        squeeze_batch = True
    else:
        squeeze_batch = False

    # Get token embeddings
    token_embeds = self.token_embedding.forward(tokens)  # (batch, seq, embed)

    # Scale embeddings if requested (transformer convention)
    if self.scale_embeddings:
        scale_factor = math.sqrt(self.embed_dim)
        token_embeds = token_embeds * scale_factor  # Use Tensor multiplication to preserve gradients

    # Add positional encoding
    if self.pos_encoding_type == 'learned':
        # Use learnable positional encoding
        output = self.pos_encoding.forward(token_embeds, start_pos)
    elif self.pos_encoding_type == 'sinusoidal':
        # Use fixed sinusoidal encoding (not learnable)
        batch_size, seq_len, embed_dim = token_embeds.shape
        if start_pos + seq_len > self.max_seq_len:
            raise ValueError("Sequence runs past the sinusoidal position table")
        pos_embeddings = self.pos_encoding[start_pos:start_pos + seq_len]  # Slice using Tensor slicing

        # Reshape to add batch dimension
        # Deliberately a fresh constant tensor: sinusoidal encodings are fixed,
        # so they take no gradient. The addition below still carries gradients
        # back through token_embeds, which is the part that learns.
        pos_data = pos_embeddings.data[np.newaxis, :, :]
        pos_embeddings_batched = Tensor(pos_data)

        output = token_embeds + pos_embeddings_batched
    else:
        # No positional encoding
        output = token_embeds

    # Remove batch dimension if it was added
    if squeeze_batch:
        # Tensor slicing from Module 01
        output = output[0]
```

```
54
55      return output
```

Calling `layer(tokens, start_pos)` enters `EmbeddingLayer.__call__`, then the attached `emblayer_forward`. A one-dimensional `(S,)` input is temporarily reshaped to `(1, S)`; a batched `(B, S)` input keeps its shape. The token lookup returns `(B, S, D)`. Optional scaling multiplies those token values by $\sqrt{D}$ before position addition; the default is `False`.[48] The selected position strategy then returns the same shape, and an input that lacked a batch axis has that temporary axis removed. Thus the caller gets `(S, D)` or `(B, S, D)` to match its input.

`parameters()` always returns the token table and includes the position table only when it is learned. The sinusoidal branch deliberately wraps its selected position values in a fresh constant Tensor; it has no position parameter to train, while addition still carries gradients into the token branch. With `pos_encoding=None`, the layer skips position addition entirely. Chapter 13 constructs this layer with learned positions and unscaled token embeddings. Its initializer makes no promise that token magnitudes dominate position magnitudes: the token bound depends on $V + D$, while the learned-position bound depends only on D.

[48] In the original Transformer (Vaswani et al., 2017), token embeddings have expected norm 1.0 (variance $1/D$), whereas sinusoidal rows have norm $\sqrt{D/2}$. Multiplying token embeddings by $\sqrt{D}$ scales their norm to match, preventing positional signals from dominating semantic representations.

11.4 A Worked Trace

A vocabulary of four tokens and a width of $D = 4$ is small enough to check every number by hand. Fix the table

$$W = \begin{bmatrix} +0.50 & -0.20 & +0.10 & +0.80 \\ -0.10 & +0.40 & -0.30 & +0.20 \\ +1.20 & +0.10 & -0.50 & -0.40 \\ +0.30 & -0.90 & +0.70 & +0.00 \end{bmatrix}$$

and feed it the sequence `[2, 0, 3]` with sinusoidal positions. With $D = 4$ there are two frequency pairs, $\omega_0 = 1/10000^0 = 1$ and $\omega_1 = 1/10000^{2/4} = 0.01$, so position t produces the row $[\sin t, \cos t, \sin 0.01t, \cos 0.01t]$. table 11.1 runs the three positions through both tables.

Table 11.1: **Gather and position addition**: The sequence `[2, 0, 3]` with $D = 4$, one row per dimension d. Values are rounded to two decimal places.

t	ID	d	$W[\text{ID}, d]$	$P[t, d]$	Sum
0	2	0	+1.20	0.00	+1.20
0	2	1	+0.10	1.00	+1.10
0	2	2	−0.50	0.00	−0.50
0	2	3	−0.40	1.00	+0.60
1	0	0	+0.50	0.84	+1.34
1	0	1	−0.20	0.54	+0.34
1	0	2	+0.10	0.01	+0.11

t	ID	d	$W[\text{ID}, d]$	$P[t, d]$	Sum
1	0	3	$+0.80$	1.00	$+1.80$
2	3	0	$+0.30$	0.91	$+1.21$
2	3	1	-0.90	-0.42	-1.32
2	3	2	$+0.70$	0.02	$+0.72$
2	3	3	$+0.00$	1.00	$+1.00$

The fast pair (dimensions 0 and 1) already differs noticeably between positions 0, 1, and 2; the slow pair (dimensions 2 and 3) has barely moved from $[0, 1]$. That is the clock with two hands, one turning a hundred times faster than the other.

For the backward pass, take the loss to be the sum of every entry in the output, so the gradient arriving at each of the three output rows is all ones. In the script below, lines 6 to 15 configure `EmbeddingLayer` with the 4×4 trace weights. Line 17 feeds the sequence `Tensor([[2, 0, 3]])`, producing output `h` of shape `(1, 3, 4)` verified in lines 18 to 21 against table 11.1. In line 24, `h.sum().backward()` propagates through the gather. Scatter-add routes row 0's gradient to $W[2, :]$, row 1's to $W[0, :]$, and row 2's to $W[3, :]$. Lines 25 to 28 confirm that `W_grad` holds ones in rows 0, 2, and 3 and zeros in row 1, while `layer.pos_encoding.grad` is `None` (sinusoidal encodings have no learned parameters). In lines 30 to 34, clearing gradients and running repeated token sequence `[[2, 0, 2]]` verifies that row 2 accumulates to twos (`[[1]*4, [0]*4, [2]*4, [0]*4]`), which naive in-place addition `grad_weight[indices] += ...` fails to compute correctly.

```
1  import numpy as np
2  from tinytorch.core.tensor import Tensor
3  from tinytorch.core.embeddings import EmbeddingLayer
4  import tinytorch.core.autograd
5
6  W_data = np.array([[+0.50, -0.20, +0.10, +0.80],
7                     [-0.10, +0.40, -0.30, +0.20],
8                     [+1.20, +0.10, -0.50, -0.40],
9                     [+0.30, -0.90, +0.70, +0.00]], dtype=np.float32)
10
11 layer = EmbeddingLayer(
12     vocab_size=4, embed_dim=4, max_seq_len=8,
   ↪   pos_encoding='sinusoidal'
13 )
14 # Replace the random initialization with the trace table.
15 layer.token_embedding.weight.data[:] = W_data
16
17 h = layer(Tensor([[2, 0, 3]]))  # one sequence in a batch of one
18 print(h.shape, np.round(h.data[0], 2))  # (1, 3, 4) and the Sum column
19 assert np.allclose(h.data[0, 0], [1.20, 1.10, -0.50, 0.60], atol=1e-2)
20 assert np.allclose(h.data[0, 1], [1.34, 0.34, 0.11, 1.80], atol=1e-2)
21 assert np.allclose(h.data[0, 2], [1.21, -1.32, 0.72, 1.00], atol=1e-2)
22
23 # Summing all entries makes the upstream gradient all ones.
24 h.sum().backward()
25 W_grad = layer.token_embedding.weight.grad
```

```
print(W_grad)  # rows 0, 2, 3 are ones; row 1 is zeros
assert np.array_equal(W_grad, [[1]*4, [0]*4, [1]*4, [1]*4])
assert layer.pos_encoding.grad is None

layer.token_embedding.weight.grad = None
layer(Tensor([[2, 0, 2]])).sum().backward()
assert np.array_equal(
    layer.token_embedding.weight.grad, [[1]*4, [0]*4, [2]*4, [0]*4]
)
```

11.4.1 Two routes back to learned tables

The sinusoidal case has no position gradient. To see the extra route for learned positions, use a two-sequence batch `[[2, 0], [2, 1]]`, width 2, and `start_pos=1`. Give token rows 0, 1, and 2 the values `[1, 2]`, `[3, 4]`, and `[5, 6]`. Position rows 1 and 2 are `[10, 20]` and `[30, 40]`. The first sequence becomes `[[15, 26], [31, 42]]`; the second becomes `[[15, 26], [33, 44]]`.

For the loss `output.sum()`, all four output rows receive `[1, 1]`. The token branch groups these gradients by ID, so token row 2 receives `[2, 2]`, rows 0 and 1 each receive `[1, 1]`, and row 3 stays zero. The position branch groups them by sequence position. Adding the two batch contributions reduces a `(2, 2, 2)` gradient to `(1, 2, 2)` with values `[[[2, 2], [2, 2]]]`. Reshape removes the leading axis, and slice-backward places those two rows into position rows 1 and 2. Position rows 0 and 3 stay zero. In the listing below, lines 1 to 5 initialize both parameter tables. Line 7 executes the forward pass with `start_pos=1`, verified in lines 8 to 10. In line 11, `output.sum().backward()` routes gradients along both branches simultaneously: line 12 validates `weight.grad` while line 13 validates `positions.grad`.

```
learned = EmbeddingLayer(4, 2, max_seq_len=4, pos_encoding='learned')
weight, positions = learned.parameters()
weight.data[:] = [[1, 2], [3, 4], [5, 6], [7, 8]]
positions.data[:] = [[0, 0], [10, 20], [30, 40], [50, 60]]
before = [parameter.data.copy() for parameter in learned.parameters()]

output = learned(Tensor([[2, 0], [2, 1]]), start_pos=1)
assert output.shape == (2, 2, 2)
assert np.array_equal(output.data, [[[15, 26], [31, 42]],
                                    [[15, 26], [33, 44]]])
output.sum().backward()
assert np.array_equal(weight.grad, [[1, 1], [1, 1], [2, 2], [0, 0]])
assert np.array_equal(positions.grad, [[0, 0], [2, 2], [2, 2], [0,
    0]])
for parameter, original in zip(learned.parameters(), before):
    assert np.array_equal(parameter.data, original)
```

The final assertion in lines 14 to 15 separates differentiation from learning. Forward read both tables, and backward filled their `.grad` arrays, but neither changed their values. Passing those parameters to an optimizer and calling `step()` is what would change the tables. Clearing gradients before the next training step prevents contributions from unrelated batches from accumulating together.

The destinations are compared in figure 11.3. Token rows group contributions by identity; position rows group them by the position used in the call.

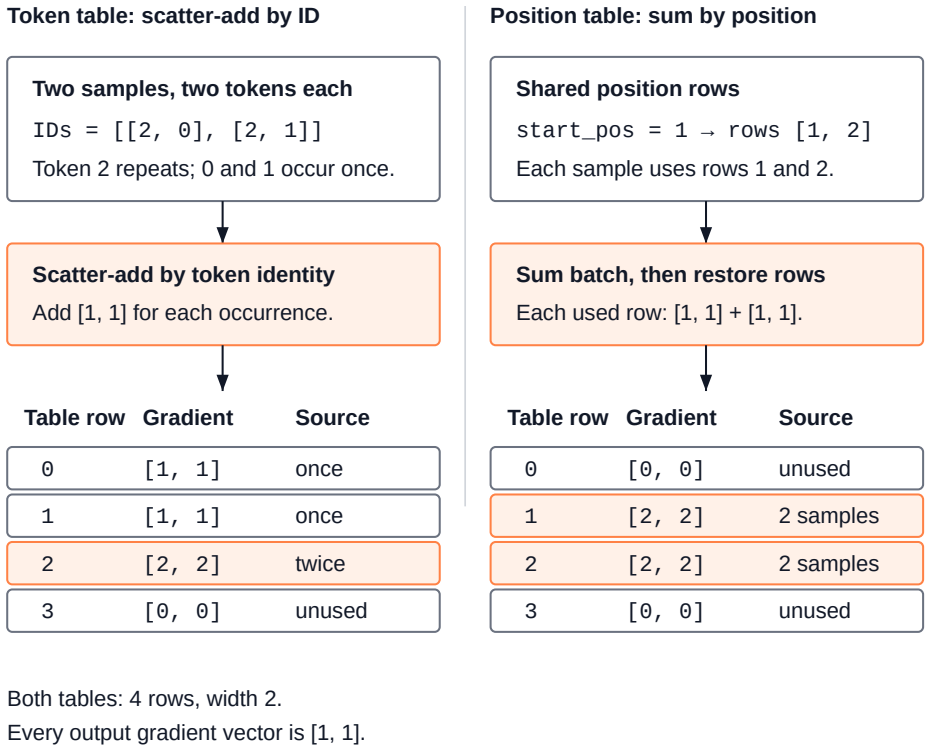


Figure 11.3: **Two gradient routes through an embedding layer**: Token-table gradients scatter-add by token identity; learned-position gradients sum across samples and return to the position rows used in the forward pass.

11.5 From TinyTorch to Production

PyTorch's `torch.nn.Embedding` exposes the same $V \times D$ weight table and gather, with options TinyTorch does not implement. `padding_idx` excludes a designated row from lookup gradients and initializes it to zero by default; a caller may explicitly assign another value. `max_norm` may renormalize selected rows in place during forward, changing the read-only state contract of our lookup. `sparse=True` returns a sparse weight gradient, so optimizer compatibility must be checked. These contracts are documented in the PyTorch embedding reference.

A dense gradient allocates values for every row. With $V = 50{,}257$ and $D = 768$, its float32 values occupy about 154 MB even when a batch uses only 1,024 IDs. The selected rows contain at most about 3.1 MB of values; a sparse representation must also store their indices and handle repeated IDs. TinyTorch keeps the full zero-filled array so that scatter-add and the existing optimizers remain easy to inspect. A zero gradient from this lookup also does not guarantee an unchanged row after every optimizer step: weight decay or momentum from earlier steps may still move it.

For $B = 8$, $S = 2{,}048$, $D = 4{,}096$, and a hypothetical 2-byte weight format, selected values occupy $BSD \times 2 = 134{,}217{,}728$ bytes. A kernel must at least read the source values and write its output; if both transfers reach the same memory interface, that is about 268 MB before indices or other work. At an assumed sustained 2 TB/s, dividing those bytes by the rate gives about 134 μs. This is a bandwidth model, not a TinyTorch timing or a hardware guarantee. TinyTorch uses float32 storage and creates additional arrays, while caching, repeated IDs, and allocation costs can change actual traffic. Downstream in Chapter 15, we will see that embedding table lookups operate firmly in the memory-bandwidth-bound regime of the Roofline model, with near-zero arithmetic intensity.

At industrial scale with vocabularies of $256{,}000$ tokens (such as Gemma) and hidden dimensions of $4{,}096$, the embedding table alone contains over 1 billion parameters (> 4 GB in FP32). Generating a dense $V \times D$ gradient tensor per micro-batch wastes gigabytes of memory on zeros. Production frameworks solve this with `sparse=True` (returning a `torch.sparse_coo_tensor` holding only active rows) or dedicated distributed embedding tables (such as TorchRec or Megatron-LM tensor parallel vocabulary sharding), communicating only the gathered rows across GPU nodes.

A different way to reduce persistent storage is **weight tying**, which uses the input table again to compute output scores as $\mathbf{h}W^T$. The original GPT-2 implementation gathers from `wte` at the input, adds a separately learned `wpe` table, and multiplies the final hidden rows by `wte` transposed at the output. Sharing that parameter saves a second $V \times D$ table. It also gives the table a second gradient path; the claim that only looked-up rows receive gradients applies to the gather operation, not to every possible use of a shared weight.

Position schemes change where the position signal enters the model. The comparison in table 11.2 separates the source implementation from a later alternative.

Table 11.2: **Position information enters at different points**: Fixed formulas avoid learning a row for every position, but do not guarantee successful inference at unseen sequence lengths.

Scheme	Where it acts	Parameters	Length limit	Used by
Sinusoidal (Vaswani et al., 2017)	Added to the token vector	None	Allocated table in TinyTorch	Original Transformer
Learned table	Added to the token vector	$S_{\max} \times D$	Allocated table	TinyTorch GPT, GPT-2
Rotary (RoPE)	Rotates query and key vectors inside attention	None for the rotation	Formula can extend; model limits remain	RoFormer, LLaMA, Mistral, Gemma
ALiBi (Press et al., 2022)	Static slope penalty added to attention logits	None	Linear penalty extrapolates	MPT, BLOOM

Rotary position embedding (RoPE) rotates pairs of coordinates in query and key vectors, two projections of a token vector that Chapter 12 will introduce. In the 2D

complex plane, each coordinate pair is rotated by angle multiples $m\theta$ and $n\theta$; their inner product retains token content while absolute positions cancel into their relative distance $(n - m)$. This uses the same sine/cosine angle-addition identities studied here, but changes the attention computation rather than adding a second table to the input. Evaluating a rotation at a larger position does not by itself guarantee that a model trained on shorter sequences will work there. TinyTorch's `EmbeddingLayer` implements learned and sinusoidal addition, not RoPE.

Core Invariant: RoPE Shift Invariance

$$\langle R_m q, R_n k \rangle = q^T R_{n-m} k$$

Rotates 2D coordinate subspaces by $m\theta$; attention dot product depends strictly on relative offset $(n - m)$.

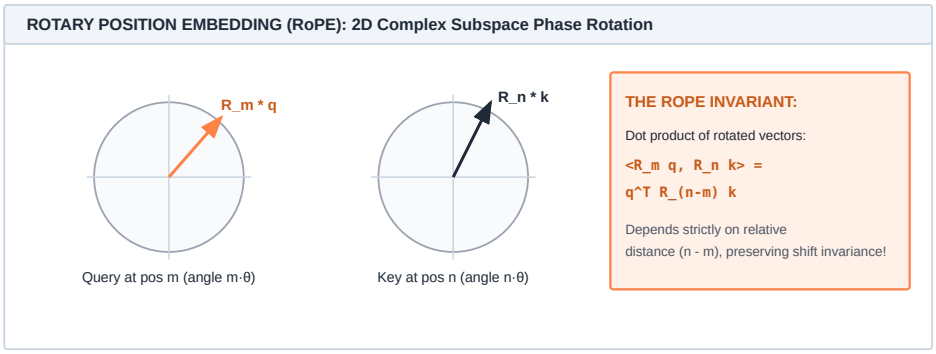


Figure 11.4: **Rotary Position Embedding (RoPE) 2D subspace rotation**: Instead of adding static position vectors to inputs, RoPE rotates 2D coordinate slices of query (m) and key (n) vectors by angle multiples $m\theta$ and $n\theta$. The resulting attention inner product depends on position only through the offset $(n - m)$.

Three practical checks follow from the reference. Keep token IDs paired with the tokenizer that assigned them. Include the token table and any learned position table in the optimizer's parameter list. Check position capacity before a call: both position strategies in `EmbeddingLayer` raise `ValueError` when the requested interval exceeds their allocated table. A sinusoidal table can be regenerated at a greater length without learning new rows, but that storage change and the model's ability to use longer inputs are separate questions.

11.6 Summary

Module 11 turns IDs into vectors by selecting rows from a persistent token table, optionally scaling them, and adding rows selected by sequence position. A `(B, S)` input returns `(B, S, D)` values. The gather node keeps the integer IDs so backward can sum all occurrences into the right token rows; learned positions instead collect batch contributions through addition, reshape, and slice. Forward and backward preserve parameter values, while backward accumulates parameter gradients for the optimizer. Fixed sinusoidal positions occupy storage but are excluded from `parameters()`.

The contract passed to Chapter 12 is a sequence of D-wide rows that already carry token and position information. Changing another token in the sequence does not yet change a row. Attention will add that missing interaction, using the rows to decide which other rows to read.

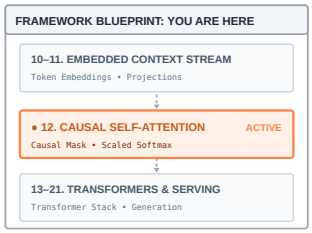
11.7 Further Reading

- **Mikolov et al.** (2013). "Efficient estimation of word representations in vector space." *arXiv*. Learned word vectors whose geometry reflects usage, trained apart from any downstream model.
- **Vaswani et al.** (2017). "Attention is all you need." *NeurIPS*. The sinusoidal position table this chapter builds, and the learned alternative.

🔥 Chapter 12

Attention Mechanisms: Scaled Dot-Product and Causal Masking

Fixed-window convolutions and sequential recurrent networks struggle to model long-range context and dynamic token interactions. A word's meaning in natural language depends heavily on distant context: in *"the server crashed because it overheated"*, resolving what *"it"* refers to requires the model to route information dynamically across the sequence. **Scaled dot-product attention** solves this by allowing every token to dynamically query, weigh, and aggregate representations from all other tokens in parallel.



Framework Blueprint: Module 12 implements scaled dot-product attention and multi-head projection with lower-triangular causal masking.

In modern PyTorch, scaled dot-product attention is invoked in a single functional call:

```
import torch
import torch.nn.functional as F

# Batch of 1, 4 heads, sequence length 8, head dimension 16
q = torch.randn(1, 4, 8, 16)
k = torch.randn(1, 4, 8, 16)
v = torch.randn(1, 4, 8, 16)

# Scaled dot-product attention with causal autoregressive masking
out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
print("Attention output shape:", out.shape)  # (1, 4, 8, 16)
```

Underneath that compact functional call, the runtime coordinates multiple linear algebra stages:

1. **The quadratic all-to-all similarity matrix (QK^T):** The engine multiplies query and key projections across all heads simultaneously, materializing an $(S \times S)$ score matrix per head. This creates the characteristic $O(S^2)$ quadratic memory and compute footprint that defines transformer scaling (Chapter 15).
2. **Variance scaling ($1/\sqrt{d_k}$):** As the head dimension d_k grows, dot products scale proportionally in variance. Scaling by $1/\sqrt{d_k}$ keeps logits in a moderate range, preventing Softmax from saturating into near-zero gradients.
3. **Causal autoregressive masking:** In generative decoders, future tokens must remain invisible. The runtime injects an upper-triangular mask of $-\infty$ prior to Softmax, forcing future attention probabilities to exactly zero without branching logic.

This chapter builds TinyTorch's multi-head causal attention engine, laying the core communication mechanism that powers the Transformer architecture in Chapter 13. Before building the multi-head projection pipeline, we must inspect why naive row averaging fails to capture contextual relationships.

12.1 The Problem

↩ **Jump back:** Chapter 11 *Input sequence vectors project into query, key, and value subspaces.*

After embedding lookup a sequence is a matrix X with one row per token and D columns. The `Linear` and activation layers in a token-wise network process those rows independently. The convolution from Chapter 9 mixed neighboring spatial positions through a fixed window, but this sequence computation needs weights that can change with the contents of the entire input.

The obvious fix is to let every token see a summary of the whole sequence. The simplest summary is the average of all rows, and it fails immediately.

```
X = [[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]]      # three tokens, two features each
avg = [sum(col) / len(X) for col in zip(*X)]  # one summary for the whole sequence
print([avg for _ in X])
# three identical rows of [0.666..., 0.666...]; every token receives the same vector
```

Averaging gives every token the identical vector, so a pronoun receives as much of *because* as of *machine*, and every output receives the same mixture of positional signals. A learned weight for each pair of positions is no better. The noun that *it* refers to can change from sentence to sentence; the right weights depend on what the tokens say, not on where they sit. A fixed table also imposes a maximum sequence length and does not adapt its weights to the input.

The operation we need has three properties. The mixing weights must be computed from the token contents themselves. They must form a proper set of proportions, nonnegative and summing to one, so a token can commit most of its attention to one source. The whole computation must also be expressible as matrix arithmetic so that all tokens compute their weights at once instead of one after another. Generation adds a fourth requirement, which we meet in Synthesis II: a token that will be predicted from its predecessors must not be allowed to read from its successors while training.

↪ **Jump ahead:** Chapter 19 *Evaluating attention across expanding prefixes incurs $O(S^2)$ recomputation; the KV cache eliminates it.*

Pause & Predict: The Attention Variance Explosion

Let query vector $q \in \mathbb{R}^{d_k}$ and key vector $k \in \mathbb{R}^{d_k}$ have elements drawn independently from zero-mean, unit-variance distributions. What is the expected mean and variance of their dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$? If $d_k = 128$, what happens to the softmax probabilities if we omit the $1/\sqrt{d_k}$ scaling factor?

12.2 The Idea

12.2.1 Queries, keys, and values

Modern sequence and vision architectures (such as GPT-6, LLaMA, and Vision Transformers) discard static receptive fields in favor of dynamic, content-dependent information routing. By decoupling a token's match criteria from its transmitted payload through distinct query, key, and value representations, attention mechanisms transform fixed

sequences into dynamic routing graphs. Before introducing three new names, consider one row choosing between two sources. Let its selection vector be $[0, 2]$ and the sources advertise $[1, 1]$ and $[1, 0]$. The dot products are 2 and 0, so the first source is preferred. Dividing by $\sqrt{2}$ and applying softmax gives weights of approximately $[0.8044, 0.1956]$. If the sources carry $[2, 1]$ and $[0, 3]$, the retrieved mixture is $[1.6089, 1.3911]$. The vectors used to choose a source and the content retrieved from it have different jobs.

The selection vector is the **query**, each advertisement is a **key**, and the retrieved content is a **value**. Attention is a lookup in which every token can play all three roles. From each token's row x_t we compute three vectors with three learned weight matrices; the projections also include biases, omitted here to keep the notation short:

- a query $q_t = x_t W_q$, what this token is looking for ("I am a pronoun; where is my noun?");
- a key $k_t = x_t W_k$, what this token advertises about itself ("I am a singular noun");
- a value $v_t = x_t W_v$, the content this token hands over if someone selects it.

The score between query i and key j is their dot product $q_i \cdot k_j$, which depends on both their lengths and their alignment; perpendicular vectors have score zero. Stacking the queries into a matrix Q and the keys into K (each of shape $S \times d_k$, where S is the sequence length and d_k the width of a query) gives all S^2 scores in one product, QK^T. A softmax across each row turns the scores for query i into proportions that sum to one, the normalization introduced in Chapter 2, applied across the candidate keys. The output for token i is the weighted sum of the values, so the whole mechanism is

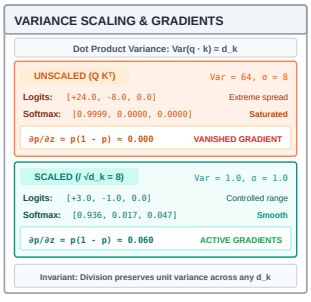
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

where V is the $S \times d_v$ matrix of values and M is a mathematical mask of zeros and negative infinities, and the division by $\sqrt{d_k}$ is the scaling that gives the mechanism its name.[49] Every quantity here is a matrix of shape $S \times S$ or $S \times d$, and every operation is a matrix multiply, an elementwise scale, an elementwise add, or a row-wise softmax. That is why attention runs in parallel across all tokens where earlier sequence models had to walk the sequence one step at a time. The three weight matrices are what make it learnable; the network is free to decide what each token should ask and advertise.

12.2.2 Why divide by $\sqrt{d_k}$

In deep Transformer architectures with wide attention heads (such as $d_k = 64$ or 128), unscaled inner products cause dot-product magnitudes to grow aggressively with projection dimension. This variance explosion pushes downstream softmax activations into extreme saturation regions where gradients vanish, crippling model training. Suppose the entries of q and k are independent with mean zero and variance one. Their dot product is a sum of d_k products, each with mean zero and variance one, so by the same variance argument Chapter 3 made for a layer's fan-in ($1/\sqrt{D_{\text{in}}}$), the sum has variance d_k and standard deviation $\sqrt{d_k}$. The spread of the scores grows with the width of the head, and a softmax fed scores spread too far apart collapses toward a nearly one-hot vector with very small derivatives. Dividing every score by $\sqrt{d_k}$ divides the variance by d_k and restores unit variance whatever the head width. The numbers for what goes wrong without it are beside the line of code that does the division.

[49] For independent zero-mean unit-variance components $q_i, k_i \sim \mathcal{N}(0, 1)$, $\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$. Dividing by $\sqrt{d_k}$ scales variance by $1/d_k$, restoring unit variance for arbitrary head width.



Variance scaling preserves unit variance for any head width, preventing softmax logits from saturating into near-zero gradients.

12.2.3 Causal masking

Autoregressive generative models such as GPT and LLaMA rely on causal masking to train efficiently in parallel on full sequences without leaking future ground-truth tokens to earlier positions. By replacing future attention scores with negative infinity before the softmax normalization, frameworks enforce strict temporal causality entirely through vectorized matrix operations. During generation the model predicts token $t+1$ from tokens 1 through t. If it were allowed to read token $t+1$ while training, it would learn to copy rather than predict, and at generation time there would be nothing to copy. The rule is that query i may attend to key j only when $j \leq i$, which the formula represents by adding a mask M before softmax. The implementation instead replaces forbidden scores directly with negative infinity:

$$M_{i,j} = \begin{cases} 0 & j \leq i \quad \text{(past and present, allowed)} \\ -\infty & j > i \quad \text{(future, forbidden)} \end{cases}$$

For four tokens the mask is zero on and below the diagonal and $-\infty$ above it:

$$M = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Because $e^{-\infty} = 0$, a masked position contributes nothing to the softmax denominator and receives exactly zero weight.[50] TinyTorch accepts a binary mask, with `1` for an allowed key and `0` for a forbidden key. Each query must retain at least one allowed key; a row with none has no probability distribution to normalize and raises `ValueError`.

[50] In 32-bit floating point, using $-\infty$ directly (`float("-inf")`) is mathematically exact since $e^{-\infty} = 0.0$ under IEEE 754. In 16-bit floating point (`float16` or `bfloat16`), production frameworks often clamp masked values to large finite negative numbers like -10^4 (or `torch.finfo(dtype).min`) to prevent downstream NaN anomalies during intermediate kernel reductions.

12.2.4 Several heads at once

Production language and vision models rely on Multi-Head Attention to attend simultaneously to multiple distinct representation subspaces and relational patterns. By partitioning the embedding dimension into parallel heads rather than replicating full-width computations, the architecture enriches representational capacity without increasing computational FLOPs. One attention pattern is one answer to "who looks at whom." A pronoun wants to find its noun; a verb wants its subject; a closing bracket wants its opening one. Multi-head attention runs H of these patterns side by side. It splits the D model dimensions into H slices of width $d_k = D/H$, lets each head attend using only its slice of Q, K, and V, concatenates the H outputs back into a D-wide row, and mixes them with one more learned matrix W_o. The model gets H independent routing decisions per token instead of one, and the code section shows why the slicing, rather than H full-width copies, is what makes that affordable.

In figure 12.1, B counts sequences in the batch, H counts heads, S counts tokens, and d is the width of each head. Each batch item and head runs the same score-to-mixture path independently.

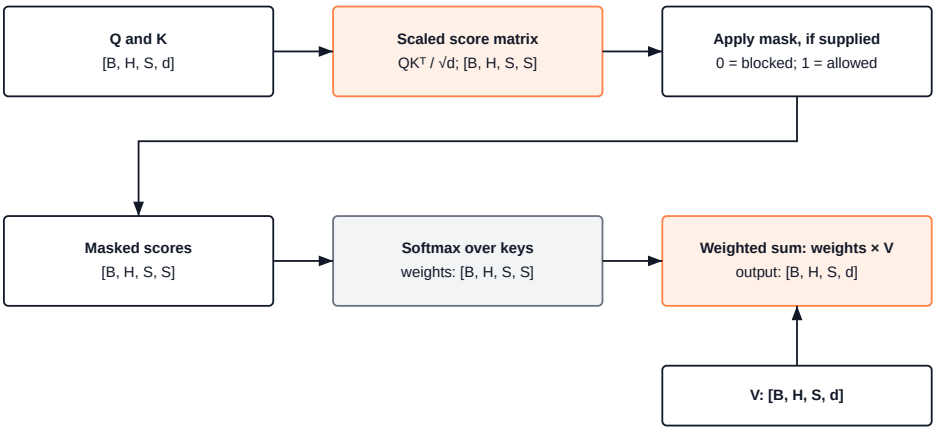


Figure 12.1: Attention as routing. Each query scores every key with a dot product, the scores are scaled by $1/\sqrt{d_k}$, future positions are masked out, and the resulting weights mix the values.

12.3 The Code

A caller enters through `MultiHeadAttention.forward(x, mask)` with shape `(B, S, D)`. The layer projects the rows, splits their feature axis into heads, calls `scaled_dot_product_attention`, merges the results, and applies its output projection. The inner function has five stages: scores, scale, mask, softmax, and value mixing. The following helpers explain those stages before returning to the outer layer. All differentiable operations come from earlier modules, so the backward rules from Chapter 6 supply the gradients.

SOURCE CODE MAPPING MOD 12

FILE: src/12_attention/12_attention.py
EXPORT: tinytorch.core.attention
SYMBOLS: MultiHeadAttention, scaled_dot

Extracted & verified at build time ✓ tested

Source Code Mapping: Module 12 extracts scaled dot-product attention, causal mask application, and MultiHeadAttention directly from the working repository.

12.3.1 The score matrix

In hardware-accelerated deep learning runtimes, the attention score matrix QK^T materializes all-to-all sequence dependencies in a single high-throughput batched GEMM. This formulation eliminates the sequential latency bottleneck of recurrent models, allowing modern accelerators to evaluate pairwise token correlations entirely in parallel across context windows. For the two-token example, QK^T produces a 2×2 matrix. Entry `(i, j)` records how query `i` scores key `j`; row `i` therefore lists all the sources that output row `i` can read. At sequence length S, this becomes S^2 entries per head. A recurrent network[51] instead carries one state forward a token at a time. Its next state depends on the previous state, whereas attention scores for a supplied sequence can be computed together.

Nested Python loops could compute the same dot products, but a matrix product states the complete operation in one call. This lets NumPy use its matrix-multiplication implementation and avoids a Python call for every scalar product. TinyTorch still per-

[51] **Recurrent network** (RNN): a sequence model that keeps a single hidden state vector and updates it once per token, $h_t = \tanh(W_h h_{t-1} + W_x x_t)$, so the state at position t depends on the state at $t-1$. This recurrence introduces a dependency between adjacent positions, so their state updates cannot all run independently.

forms all the score arithmetic; expressing it as a matrix operation changes how the work is executed.

```python
def _compute_attention_scores(Q: Tensor, K: Tensor) -> Tensor:
    """Compute raw attention scores via Q @ K^T.
    """
    K_t = K.transpose(-2, -1)
    return Q.matmul(K_t)
```

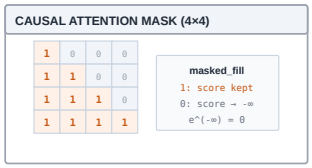
The helper contains two operations from Chapter 1. `transpose(-2, -1)` swaps the last two axes and preserves the leading axes. For a single head, `(S, d_k)` becomes `(d_k, S)`, making `Q.matmul(K_t)` a product with shape `(S, S)`. For a batch of heads, `(B, H, S, d_k)` becomes `(B, H, d_k, S)`, and the product has shape `(B, H, S, S)`. This batched path uses `np.matmul`. The returned transpose owns its buffer in TinyTorch; it does not expose a shared view of `K`.

12.3.2 Softmax by rows

Each row of the score matrix belongs to one query, so each row needs its own softmax over keys. Direct exponentiation can overflow for large scores. Module 12 reuses `Softmax()(scores, dim=-1)`, which subtracts the maximum of each row before exponentiating (Chapter 2). For a row containing an allowed finite score, the largest shifted entry is zero, its exponential is one, and the denominator is positive. Forbidden entries remain negative infinity and contribute zero. The mask validation is what rules out the undefined all-negative-infinity case.

The implementation makes separate NumPy passes to find the maximum, shift the scores, exponentiate, sum, and divide. These passes operate on a fully materialized score tensor. The production bridge will return to the memory cost of keeping and revisiting that tensor.

12.3.3 The causal mask



Causal mask: the module takes 1 (allowed) and 0 (blocked); blocked scores are replaced with $-\infty$ before softmax.

Synthesis II trains a next-token predictor, and the loss at position t scores a prediction of token $t+1$. That only works if position t's output was computed without reading token $t+1$. Chapter 19 leans on the same guarantee from the other side. It generates one token at a time and reuses the keys and values of earlier positions, which is valid only because an earlier position's output never depended on anything after it. The naive way to keep the future out is to run the model once per prefix, S forward passes for a sequence of S tokens, a thousand passes for 1,024 tokens to get what one masked pass gives.

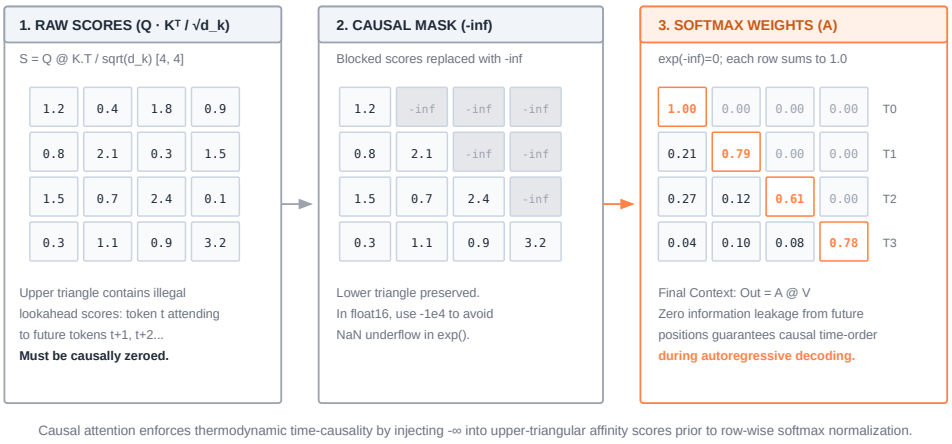
Setting a forbidden score to zero does not exclude it. Softmax over `[0.7071, 0.0]` gives approximately `[0.67, 0.33]`, so the forbidden key still receives a third of the weight. Subtracting a large finite constant is also insufficient for an unrestricted score range. A forbidden score of 10^{10} remains large after subtracting 10^{9}. Replacing the score with negative infinity enforces exclusion independently of its previous finite value.

```python
def _apply_mask(scores: Tensor, mask: Tensor) -> Tensor:
    """Apply a binary attention mask by replacing blocked scores with
    ↪   -infinity.
    """
```

```
    # Both guards below sweep the whole broadcast (B, H, S, S) array, so they cost
    # O(B*H*S^2) on EVERY forward call. That is the price of catching a mis-built mask
    # at the point of use. Production kernels validate the mask once where it is
    # constructed and keep the attention inner loop free of checks.
    if np.any((mask.data != 0) & (mask.data != 1)):
        raise ValueError("Attention mask must contain only 0 (blocked) or 1 (allowed)")
    allowed = np.broadcast_to(mask.data, scores.shape)
    if np.any(np.sum(allowed, axis=-1) == 0):
        raise ValueError("Each query must have at least one allowed key")
    return scores.masked_fill(allowed == 0, MASK_VALUE)
```

The mask arrives as ones and zeros, with the same convention as `np.tril(np.ones((S, S)))`. `_apply_mask` first rejects nonbinary entries, broadcasts the mask to the score shape, and checks that each query has an allowed key. `scores.masked_fill(allowed == 0, MASK_VALUE)` then returns a new tensor with blocked scores replaced by `float("-inf")`.[52] It preserves the input scores and records the operation for backward. The rule from Chapter 6 passes gradients at allowed positions and zeros them at forbidden positions. The mask itself is a fixed choice of connectivity, not a trainable parameter.

[52] In 32-bit floating point, `-inf` evaluates reliably. In half-precision formats like FP16, `-inf` can produce `NaN` values during exponent subtraction if all row entries are masked. Production engines frequently substitute large finite negatives such as `-1e4` or `-1e9` to avoid precision overflow.



Causal attention enforces thermodynamic time-causality by injecting -∞ into upper-triangular affinity scores prior to row-wise softmax normalization.

Figure 12.2: **The causal attention engine**: Queries and keys interact via scaled dot products $QK^T/\sqrt{d_k}$ to form an $S \times S$ score matrix. An upper-triangular mask sets future positions to $-\infty$ so row-wise softmax normalizes attention weights exclusively over past and present tokens ($j \leq i$), strictly preserving autoregressive causality during value mixing.

12.3.4 Scores, scale, mask, softmax, mix

The inner attention function puts the helpers in execution order. Its caller supplies already projected Q, K, and V; it returns both the mixed values and the weights used to mix them. Before composing the stages, the remaining helper supplies the scale factor.

The variance argument predicts a score standard deviation of 8 when $d_k = 64$. We can see the consequence without a random experiment. Softmax over `[24.0, -8.0, 0.0]` assigns approximately 3.8×10^{-11} to the last entry and almost all the mass to the first. Dividing those scores by 8 gives `[3.0, -1.0, 0.0]`, whose weights are approximately `[0.936, 0.017, 0.047]`. The diagonal softmax derivative is $p_i(1-p_i)$, so moving away from zero and one leaves more room for the scores to change during learning. Scaling limits saturation; it does not guarantee a particular distribution after training.

```python
def _scale_scores(scores: Tensor, d_k: int) -> Tensor:
    """Scale attention scores by 1/sqrt(d_k).
    """
    scale_factor = 1.0 / math.sqrt(d_k)
    return scores * scale_factor
```

```python
def scaled_dot_product_attention(Q: Tensor, K: Tensor, V: Tensor,
    mask: Optional[Tensor] = None) -> Tuple[Tensor, Tensor]:
    """Complete scaled dot-product attention.
    """
    scores = _compute_attention_scores(Q, K)
    scores = _scale_scores(scores, Q.shape[-1])
    if mask is not None:
        scores = _apply_mask(scores, mask)
    softmax = Softmax()
    attention_weights = softmax(scores, dim=-1)
    output = attention_weights.matmul(V)
    return output, attention_weights
```

`d_k` is read from the last axis of `Q`, so the function works at both the full feature width and the per-head width. The returned weights make the routing visible, while the output retains two gradient routes: through the final matrix product to `V`, and through softmax, masking, scaling, and scores to `Q` and `K`. Rewrapping `output.data` in a fresh tensor would preserve its numbers while breaking those routes. This forward call does not modify its input tensors or update parameters; backward accumulates gradients, and an optimizer performs the later parameter update.

12.3.5 The projections

The formula takes Q, K, and V as given, but the sequence only provides X. The naive move is to use X for all three. Then the score between tokens i and j is $x_i \cdot x_j$, which equals $x_j \cdot x_i$, so the raw score in each direction is identical, although row-wise softmax can still assign different weights because its denominators differ, and one vector has to serve as the question, the advertisement, and the content at once. Using the input directly would give this attention calculation no projection parameters of its own. Three learned matrices give the network three different linear views of the same token, and a fourth, W_o, mixes the heads back together. That is four $D \times D$ matrices, 2.36 million weights per layer at $D = 768$, the $4D^2$ term in Chapter 13's parameter count, plus four bias vectors that Module 12 gets for free by building each projection as a Chapter 3 `Linear`.

The projection weights must also begin at a controlled scale. Under the independent unit-variance input model, multiplying by a $D \times D$ matrix of unit-variance weights would give each projected coordinate variance D. Module 12 instead inherits `Linear`'s weight

standard deviation $1/\sqrt{D_{\text{in}}}$ from Chapter 3. For these square projections, the variance estimate becomes $D \times (1/D) = 1$. This explains why the projection initialization and the attention score scaling solve different parts of the same problem; neither replaces the other.

```
def __init__(self, embed_dim: int, num_heads: int):
    """
    Initialize multi-head attention.
    """
    if not isinstance(embed_dim, (int, np.integer)) or embed_dim <= 0:
        raise ValueError("embed_dim must be a positive integer")
    if not isinstance(num_heads, (int, np.integer)) or num_heads <= 0:
        raise ValueError("num_heads must be a positive integer")
    if embed_dim % num_heads != 0:
        raise ValueError(
            f"Multi-head attention dimension mismatch\n"
            f"  ❌ embed_dim={embed_dim} is not divisible by num_heads={num_heads} (remainder={embed_dim % num_heads})\n"
            f"  💡 Multi-head attention splits embed_dim equally among heads, so embed_dim must be a multiple of num_heads\n"
            f"  🔧 Try: embed_dim={num_heads * (embed_dim // num_heads + 1)} (next valid size) or num_heads=1"
        )

    self.embed_dim = embed_dim
    self.num_heads = num_heads
    self.head_dim = embed_dim // num_heads

    # Linear projections for queries, keys, values
    self.q_proj = Linear(embed_dim, embed_dim)
    self.k_proj = Linear(embed_dim, embed_dim)
    self.v_proj = Linear(embed_dim, embed_dim)

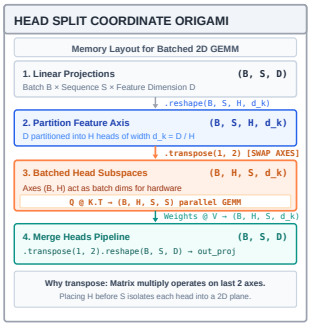
    # Output projection to mix information across heads
    self.out_proj = Linear(embed_dim, embed_dim)
```

```
def parameters(self) -> List[Tensor]:
    """
    Return all trainable parameters.
    """
    params = []
    params.extend(self.q_proj.parameters())
    params.extend(self.k_proj.parameters())
    params.extend(self.v_proj.parameters())
    params.extend(self.out_proj.parameters())
    return params
```

The constructor requires positive integer dimensions and rejects an `embed_dim` not divisible by `num_heads`. Its persistent state consists of four `Linear` layers. `parameters()` returns their eight parameter tensors, weight then bias for `q_proj`, `k_proj`, `v_proj`, and

`out_proj`, without copying those parameters. Passing this list to an optimizer therefore updates the same tensors used on the next forward call.

12.3.6 The heads



Coordinate origami swaps head and sequence dimensions to place independent 2D matrix planes into the batch grid for hardware GEMM execution.

The Idea said a pronoun, a verb, and a closing bracket each want a different routing pattern, and a width-768 layer can provide twelve such patterns. The naive way to get twelve patterns is twelve full-width attention layers side by side, each with its own four 768×768 matrices. That is 28.3 million weights per layer instead of 2.36 million, and twelve times the score arithmetic, $12 \times S^2 \times D$ multiply-adds. The mechanism that makes twelve heads cost what one head costs is the slice. Project once at full width, cut each of Q, K, and V into twelve strips of 64 columns, run attention on each strip, and concatenate. The score work is $H \times S^2 \times d_k = S^2 \times D$, the same as a single head, and each head uses a distinct output-column slice of the projection matrices.

```
def _split_heads(self, x: Tensor, batch_size: int, seq_len: int) ->
↪   Tensor:
    """Reshape to separate attention heads for parallel processing.
    """
    x = x.reshape(batch_size, seq_len, self.num_heads, self.head_dim)
    return x.transpose(1, 2)
```

```
def _merge_heads(self, x: Tensor, batch_size: int, seq_len: int) ->
↪   Tensor:
    """Merge attention heads back into single embedding dimension.
    """
    x = x.transpose(1, 2)
    return x.reshape(batch_size, seq_len, self.embed_dim)
```

```
def forward(self, x: Tensor, mask: Optional[Tensor] = None) -> Tensor:
    """
    Forward pass through multi-head attention.
    """
    # Step 1: Extract dimensions and validate
    batch_size, seq_len, embed_dim = x.shape
    if embed_dim != self.embed_dim:
        raise ValueError(
            f"MultiHeadAttention input dimension mismatch\n"
            f"  ❌ Expected embed_dim={self.embed_dim}, got
            ↪   {embed_dim} from input shape {x.shape}\n"
            f"  💡 The last dimension of input must match embed_dim
            ↪   from initialization
            ↪   (MultiHeadAttention({self.embed_dim},
            ↪   {self.num_heads}))\n"
            f"  🔧 Try: x.reshape({x.shape[0]}, {x.shape[1]},
            ↪   {self.embed_dim}) or create new
            ↪   MultiHeadAttention({embed_dim}, num_heads)"
        )

    # Step 2: Project to Q, K, V
```

```
    Q = self.q_proj.forward(x)
    K = self.k_proj.forward(x)
    V = self.v_proj.forward(x)

    # Step 3: Split into heads
    Q = self._split_heads(Q, batch_size, seq_len)
    K = self._split_heads(K, batch_size, seq_len)
    V = self._split_heads(V, batch_size, seq_len)

    # Step 4: Apply attention -- reshape mask (batch, seq_q, seq_k) to
    # (batch, 1, seq_q, seq_k) for broadcasting across heads.
    mask_reshaped = mask
    if mask is not None and len(mask.shape) == 3:
        batch_size_mask, seq_q_mask, seq_k_mask = mask.shape
        mask_reshaped = mask.reshape(batch_size_mask, 1, seq_q_mask,
↪   seq_k_mask)

    attended, _ = scaled_dot_product_attention(Q, K, V,
↪   mask=mask_reshaped)

    # Step 5: Merge heads back together
    concat_output = self._merge_heads(attended, batch_size, seq_len)

    # Step 6: Apply output projection
    output = self.out_proj.forward(concat_output)

    return output
```

The projections run once at full width. `_split_heads` reshapes `(B, S, D)` to `(B, S, H, d_k)` and swaps the middle axes, so the attention function receives `(B, H, S, d_k)`. There is no Python loop over heads. A supplied three-dimensional mask `(B, S, S)` becomes `(B, 1, S, S)` and broadcasts across heads; a shared `(S, S)` mask can broadcast directly. The constructor's divisibility check ensures that every feature belongs to exactly one head.

After attention, `_merge_heads` reverses the axis swap and combines the head and feature axes. The output projection then mixes information across heads and returns `(B, S, D)`. Split and merge preserve the association between a token and its features, but each `reshape` and `transpose` returns a tensor with its own storage in TinyTorch. Forward creates these temporary activations without changing the input or the four projection layers.

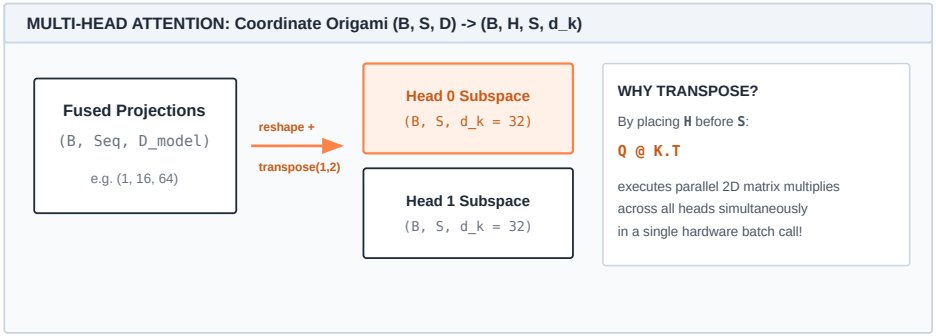


Figure 12.3: **Multi-head coordinate origami**: Slicing the full hidden embedding (B, S, D) into (B, H, S, d_k) via reshape and transposition isolates head subspaces into independent 2D planes, enabling hardware parallel matrix multiplication across all heads in a single batch call.

12.4 A Worked Trace

Two tokens with $d_k = 2$ are enough to see every step. Take

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}, \quad \mathbf{M} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

so that $\sqrt{d_k} = \sqrt{2} \approx 1.4142$. Row 0 belongs to token 0 and may only see itself; row 1 belongs to token 1 and may see both. The mask is written the way Module 12 takes it, ones where looking is allowed, and `_apply_mask` replaces each score whose mask entry is zero with $-\infty$.

Table 12.1: Scaled dot-product attention with a causal mask on a two-token sequence, one row per token.

Step	Operation	Token 0 (row 0)	Token 1 (row 1)
1	Scores QK^T	$[1.0, 1.0]$	$[2.0, 0.0]$
2	Divide by $\sqrt{2}$	$[0.7071, 0.7071]$	$[1.4142, 0.0]$
3	Replace blocked scores	$[0.7071, -\infty]$	$[1.4142, 0.0]$
4	Softmax per row	$[1.0, 0.0]$	$[0.80443, 0.19557]$
5	Weights $\cdot V$	$[2.0, 1.0]$	$[1.6089, 1.3911]$

The first score in row 1 is $[0, 2] \cdot [1, 1] = 2$. After scaling, its exponential is $e^{1.4142} \approx 4.11325$, compared with $e^0 = 1$ for the other key. Dividing by their sum, 5.11325, produces the row's weights. Their value mixture is $0.80443[2, 1] + 0.19557[0, 3] = [1.6089, 1.3911]$. This is the same selection introduced in The Idea, now placed inside a complete two-query call.

Token 0's second score was 0.7071 before masking, a perfectly respectable value, and the mask removed it entirely; the forbidden key receives exactly zero weight. Token 1 puts 80% of its weight on token 0 because its query $[0, 2]$ aligns with key $[1, 1]$ and is orthogonal to key $[1, 0]$. The following assertions in the first listing reproduce table 12.1 step-by-step from the module. In lines 7 to 10, `Q`, `K`, `V`, and lower-triangular mask `M` are defined. Line 11 computes raw dot-product scores (QK^T), scaled by $1/\sqrt{2}$ in line 12. Line 13 applies the causal mask, replacing forbidden entries with $-\infty$ (verified by `isneginf` in line 16). Line 14 invokes `scaled_dot_product_attention(Q, K, V, mask=M)`, and lines 18 to 19 confirm that attention weights `w` ($[1, 0]$ and $[0.80443, 0.19557]$) and output values `out` ($[2, 1]$ and $[1.6089, 1.3911]$) agree with the manual trace.

```
1  import numpy as np
2  from tinytorch.core.tensor import Tensor
3  from tinytorch.core.attention import scaled_dot_product_attention
4  from tinytorch.core.attention import _compute_attention_scores
5  from tinytorch.core.attention import _scale_scores, _apply_mask
6
7  Q = Tensor([[1.0, 0.0], [0.0, 2.0]])
8  K = Tensor([[1.0, 1.0], [1.0, 0.0]])
9  V = Tensor([[2.0, 1.0], [0.0, 3.0]])
10 M = Tensor(np.tril(np.ones((2, 2))))
11 raw = _compute_attention_scores(Q, K)
12 scaled = _scale_scores(raw, 2)
13 masked = _apply_mask(scaled, M)
14 out, w = scaled_dot_product_attention(Q, K, V, mask=M)
15 assert np.allclose(raw.data, [[1.0, 1.0], [2.0, 0.0]])
16 assert np.isneginf(masked.data[0, 1])
17 assert np.isfinite(scaled.data).all()  # masking preserved its input
18 assert np.allclose(w.data, [[1, 0], [0.80443, 0.19557]], atol=1e-5)
19 assert np.allclose(out.data, [[2, 1], [1.6089, 1.3911]], atol=1e-4)
20 print(out.data)
```

The layer adds learned projections and head axes to this picture. In the second snippet below, lines 3 to 5 instantiate `MultiHeadAttention(embed_dim=4, num_heads=2)`, an input sequence `X` of shape `(1, 3, 4)`, and a lower-triangular causal mask `(1, 3, 3)`. Line 6 evaluates `Y = mha(X, mask)`, preserving input dimensions `(1, 3, 4)` in line 7. Lines 9 to 11 trace the internal projection and head-splitting step `_split_heads`, reorganizing the tensors to shape `(1, 2, 3, 2)` (batch, head, token, strip). Line 12 runs attention over all heads simultaneously. Lines 13 to 15 isolate head 0's 3×3 attention weight matrix `w0`, verifying that its rows sum to 1.0 (line 14) and all values above the diagonal are strictly 0.0 (line 15).

```
1  from tinytorch.core.attention import MultiHeadAttention
2
3  mha = MultiHeadAttention(embed_dim=4, num_heads=2)
4  X = Tensor(np.eye(3, 4)[None])                    # (1, 3, 4): three
   ↪   one-hot tokens
5  mask = Tensor(np.tril(np.ones((1, 3, 3))))        # (1, 3, 3)
6  Y = mha(X, mask)
```

```
print(Y.shape)                                    # (1, 3, 4): same shape as X

Qh = mha._split_heads(mha.q_proj(X), 1, 3)        # (1, 2, 3, 2): batch, head, token, strip
Kh = mha._split_heads(mha.k_proj(X), 1, 3)
Vh = mha._split_heads(mha.v_proj(X), 1, 3)
_, w = scaled_dot_product_attention(Qh, Kh, Vh, mask=mask.reshape(1, 1, 3, 3))
w0 = w.data[0, 0]                                 # head 0's (3, 3) weights
print(w0.sum(axis=1))                             # approximately [1, 1, 1]
print(np.allclose(np.triu(w0, 1), 0.0))           # True: nothing above the diagonal
```

For this causal mask and finite unmasked scores, the weights sum to one and the upper triangle is zero. These checks establish that the routing weights are valid. A stronger behavioral check asks whether changing a future token can alter an earlier output. The earlier query, its allowed keys, and its allowed values are unchanged, so the earlier output must remain unchanged too.

```
X_changed = Tensor(X.data.copy())
X_changed.data[0, 2] = [3.0, -2.0, 1.0, 4.0]  # change only the final token
Y_changed = mha(X_changed, mask)
assert np.allclose(Y.data[:, :2], Y_changed.data[:, :2], atol=1e-6)
assert np.allclose(w.data.sum(axis=-1), 1.0)
assert np.allclose(np.triu(w.data, 1), 0.0)
```

The same argument will apply block by block in Chapter 13 because its remaining operations act on each token separately. Causality therefore becomes a property of the complete model rather than only of one weight matrix.

12.5 From TinyTorch to Production

The reference keeps the complete score matrix because it makes the computation easy to inspect. Its storage grows quadratically with sequence length. This is a consequence of having one score for every query-key pair, even though the final output has only one feature vector per query.

Consider $H = 12$ heads of width $d_k = 64$, a batch of $B = 8$ sequences, and four bytes per element. These are array-size estimates, not measured peak memory.

- At $S = 2{,}048$, the three projected tensors occupy $3BHSd_k \times 4 = 150{,}994{,}944$ bytes, approximately 151 MB.
- The score matrix alone occupies $BHS^2 \times 4 = 1{,}610{,}612{,}736$ bytes, approximately 1.61 GB. A separately stored weight matrix has the same size.
- Quadrupling the context to $S = 8{,}192$ quadruples projection storage to approximately 604 MB but multiplies score storage by sixteen, to approximately 25.8 GB.

These counts exclude parameters, gradients, other activations, and intermediate copies. They establish how the arrays scale, without claiming that their sum is the complete memory requirement.

FlashAttention changes how attention uses memory. It partitions the calculation into blocks that fit in small, fast on-chip storage, reducing transfers to and from the GPU's larger main memory. For each query it maintains a running maximum and normalization sum, allowing later key blocks to join the same softmax calculation. The output can therefore be accumulated without storing a complete $S \times S$ score matrix. This is exact attention in its mathematical formulation, with possible floating-point differences from a different evaluation order. Tiling reduces intermediate storage and memory traffic; dense attention still scores all allowed query-key pairs.

In modern GPU architectures, bandwidth to high-bandwidth memory (HBM) is several times lower than to on-chip SRAM (the FlashAttention paper cites about 1.5 to 2.0 TB/s HBM vs. about 19 TB/s SRAM on an NVIDIA A100). Standard attention materializes the $S \times S$ score matrix in HBM, reads it back for softmax, writes the probabilities to HBM, and reads them again to multiply by V. FlashAttention tiles Q, K, and V into small SRAM blocks, computes attention using the online softmax algorithm (tracking running max m_i and normalizer ℓ_i), and accumulates output vectors entirely within SRAM registers without ever writing the quadratic intermediate matrix to DRAM (as profiled in Chapter 15 and accelerated via tiled loops in Chapter 18, with an end-to-end blueprint developed in Chapter 23).

The PyTorch scaled-dot-product attention interface can select optimized or math implementations according to the input and available backends. Its boolean mask uses `True` for allowed entries, while a floating-point mask is an additive score bias. TinyTorch instead accepts only a binary allow-mask even though its Tensor storage is float32. PyTorch's function also returns only the mixed output; TinyTorch returns `(output, weights)` to make the mechanism inspectable. These interface differences matter when translating a test. A production causal option specifies the connectivity without requiring the caller to construct a square mask, but it does not by itself guarantee a particular backend.

Table 12.2: The same attention calculation can use different intermediate storage.

	Module 12	FlashAttention-style execution
Scores	Full `(B, H, S, S)` array	Blocks used while accumulating outputs
Softmax	Separate NumPy passes across each row	Running maximum and normalization sum
Causal connectivity	Binary mask; blocked scores replaced with $-\infty$	Enforced inside the block computation
Returned weights	Full probability tensor	Not required to produce the mixed output
Intermediate score storage	Quadratic in S	No full square score tensor

The comparison in table 12.2 concerns execution and storage. It does not add FlashAttention to this NumPy implementation. For either approach, valid probability rows sum to one and a causal output cannot depend on later input tokens. Comparisons should

allow for floating-point rounding and should use probabilities before any production attention dropout, since dropout changes individual row sums.

A separate opportunity appears during generation. Under causal attention, adding a token does not change the keys and values already computed for earlier tokens. Chapter 19 will introduce the state and lifetime needed to reuse them. That cache removes repeated projection work but still leaves the new query to attend over its growing prefix. The attention computation and the storage decision remain distinct mechanisms.

Standard Multi-Head Attention (MHA) pairs each query head with its own dedicated key and value heads ($H_q = H_k = H_v$). While computationally expressive, storing separate KV vectors per head for long contexts causes the KV cache to overwhelm accelerator DRAM during generation. **Multi-Query Attention** (MQA; Shazeer, 2019) collapses all heads to a single shared key/value projection ($H_k = H_v = 1$), reducing KV cache memory by $H\times$ at the cost of modest model capacity. Modern frontier models (LLaMA 3, Mistral, Gemma) adopt the optimal middle ground: **Grouped-Query Attention** (GQA; Ainslie et al., 2023), grouping H_q query heads into G key/value partitions ($H_k = H_v = G$, typically $G = 8$), cutting KV memory traffic with quality close to multi-head attention. Chapter 19 explores this cache lifecycle in depth.

12.6 Summary

Module 12 computes scores, divides by the square root of the query width, replaces blocked scores with negative infinity, normalizes across keys, and mixes the values. `MultiHeadAttention` supplies four learned projections around that function. Its head split and merge preserve token order, and its output has the same `(B, S, D)` shape as its input. Forward preserves the input and parameter values; the graph records the operations needed for backward.

For a valid binary mask and finite unmasked scores, each weight row sums to one and forbidden keys receive zero weight. An all-blocked row is rejected. These are the local checks behind the stronger causal contract: changing a future input cannot change an earlier output. Chapter 13 will combine this token-mixing layer with per-token transformations, normalization, and residual connections to build a complete predictor.

12.7 Further Reading

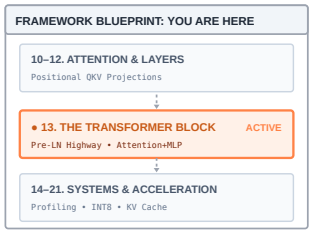
- **Bahdanau, Cho, and Bengio** (2015). "Neural machine translation by jointly learning to align and translate." *ICLR*. Attention as learned, data-dependent alignment, before transformers.
- **Vaswani et al.** (2017). "Attention is all you need." *NeurIPS*. Scaled dot-product and multi-head attention in the form this chapter implements.
- **Shazeer** (2019). "Fast transformer decoding: One write-head is all you need." Multi-Query Attention (MQA), sharing key-value projections across heads to reduce memory bandwidth during inference.
- **Dao et al.** (2022). "FlashAttention: fast and memory-efficient exact attention with IO-awareness." *NeurIPS*. Computing the exact attention result without storing the full score matrix this chapter keeps.

- **Ainslie et al.** (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." *EMNLP*. Grouped-Query Attention as the modern standard balancing memory bandwidth and model capacity.

🔥 Chapter 13

The Transformer: Assembling GPT from Attention Blocks

Stacking attention layers directly leads to gradient instability and signal degradation as network depth increases. To build scalable, multi-layer foundation models, attention must be organized into a robust, repeating architectural block. The **Transformer decoder block** stabilizes deep sequence processing by combining causal multi-head attention with position-wise feed-forward networks (MLPs), Pre-Layer Normalization (Pre-LN), and residual skip connections.



Framework Blueprint: Module 13 synthesizes causal self-attention, feedforward MLPs, and Pre-LN residual highways into the complete TinyGPT model.

In PyTorch, a full transformer block is declared and composed in standard modules:

```
import torch
import torch.nn as nn

# Modern decoder-only Transformer block (Pre-LN with causal mask)
layer = nn.TransformerEncoderLayer(
    d_model=64, nhead=4, dim_feedforward=256, norm_first=True,
    batch_first=True
)

x = torch.randn(2, 8, 64)       # Batch of 2, sequence length 8,
    hidden dim 64
mask = nn.Transformer.generate_square_subsequent_mask(8)
out = layer(x, is_causal=True, src_mask=mask)

print("Decoder block output shape:", out.shape)  # torch.Size([2, 8,
    64])
```

Underneath that composable layer block, the runtime balances multiple structural and gradient invariants:

1. **The Pre-LN residual highway:** By applying layer normalization to the inputs of each sub-layer rather than their outputs (Pre-LN), the main residual stream remains unnormalized. This creates an unhindered identity highway for gradients during backpropagation.
2. **Balanced compute and parameter allocation:** The block balances parameter capacity between token communication (multi-head attention, $4D^2$ parameters) and channel-wise feature transformation (the two-layer feed-forward MLP with a $4\times$ expansion, $8D^2$ parameters).
3. **End-to-end logit projection:** Stacking N identical blocks and projecting the final hidden states through a vocabulary linear head produces next-token logit distributions of shape (B, S, V).

This chapter synthesizes the components of Part II into TinyGPT, TinyTorch's flagship generative language model, setting the stage for text generation in Synthesis II. Before assembling the model, we must inspect what happens when deep attention networks lack residual pathways and layer normalization.

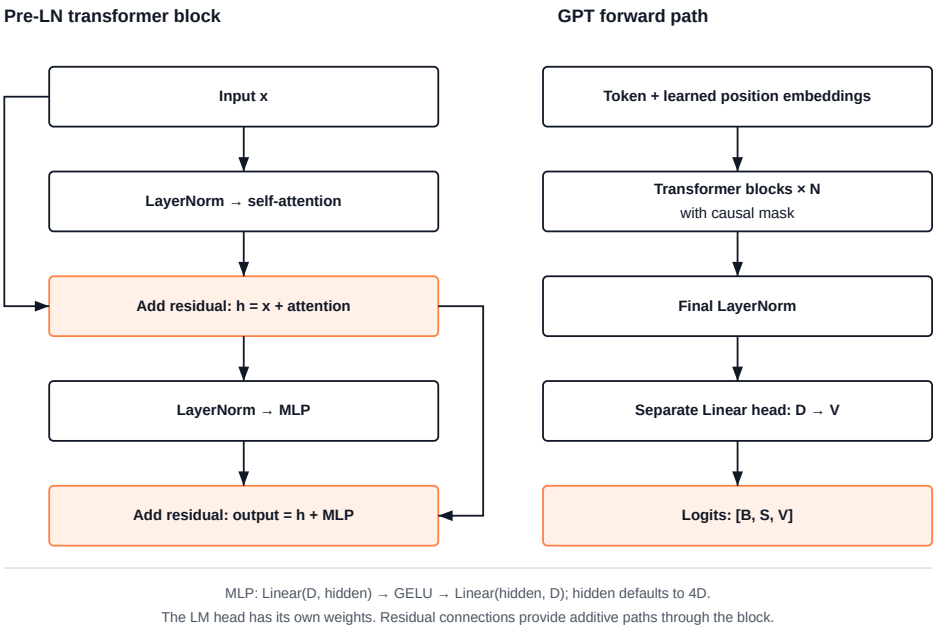


Figure 13.1: **Composing a transformer**: Each block adds two learned updates to the residual stream. The complete GPT stacks these blocks before a final normalization and a separate vocabulary projection.

Module 13 exports this model as `GPT`, with `TinyGPT` retained as an alias; the class itself lives in the hand-maintained `tinytorch/models/transformer.py`. Its caller supplies token IDs of shape (B, S) and receives logits of shape (B, S, V), one row of next-token scores per position. The training and sampling loop in Synthesis II will use that contract.

13.1 The Problem

↩ **Jump back:** Chapter 12 *Causal multi-head self-attention forms the core token-mixing sub-layer of the Transformer block.*

Two things go wrong if you simply stack attention layers.

The first is the lack of a dedicated transformation of each token's features after mixing. Attention is already nonlinear because its softmax weights depend on the input. Its output, however, is organized as a weighted mixture of projected values. A separate MLP lets every token transform that mixture through a nonlinear hidden layer without constructing another matrix of interactions between positions. It reuses the linear-activation-linear pattern from Part I at every position.

The second problem is depth. In a simplified stack whose layers each multiply the signal's scale by a gain, a gain of 1.2 across 24 layers gives $1.2^{24} \approx 79$; a gain of 0.8 gives $0.8^{24} \approx 0.005$. The chain rule similarly composes derivatives on the way back. The stacked sigmoids in Chapter 2 illustrated this gradient problem. The variance-based initialization in Chapter 3 controls a fresh linear layer's scale, but training changes its

weights. The following linear experiment isolates the difference between replacing a signal and adding a normalized update.

Pause & Predict: The Residual Highway Gradient Propagation

Consider a deep network of $N = 24$ layers without residual connections: $x_{l+1} = f_l(x_l)$. Its end-to-end gradient expands via the chain rule as a product of 24 Jacobians: $\frac{\partial x_{24}}{\partial x_0} = \prod_{l=0}^{23} J_l$. If the average singular value of each Jacobian is 0.9, what is the gradient magnitude reaching the first layer? What happens when we add residual skip connections $x_{l+1} = x_l + f_l(x_l)$ so each term becomes $(I + J_l)$?

Listing 13.1 Replacement and residual updates: A seeded linear experiment contrasts multiplying a signal with adding an update computed from normalized inputs.

```
import math, random
rng = random.Random(0)
D, L = 64, 24

def rms(v):
    """Root-mean-square scale of a vector."""
    return math.sqrt(sum(x * x for x in v) / len(v))

def apply(W, v):
    """Multiply a D x D matrix by a length-D vector."""
    return [sum(w * x for w, x in zip(row, v)) for row in W]

x = [rng.gauss(0.0, 1.0) for _ in range(D)]
plain, stream = list(x), list(x)
for layer in range(1, L + 1):
    W = [[rng.gauss(0.0, 1.0) * 1.2 / math.sqrt(D)
          for _ in range(D)] for _ in range(D)]
    plain = apply(W, plain)
    normalized = [s / rms(stream) for s in stream]
    stream = [s + u for s, u in zip(stream, apply(W, normalized))]
    if layer in (1, 6, 12, 24):
        print(f"layer {layer:2d}   plain rms = {rms(plain):6.2f}"
              f"   stream rms = {rms(stream):5.2f}")
```

```
layer  1   plain rms =   1.18   stream rms =  1.58
layer  6   plain rms =   2.66   stream rms =  3.10
layer 12   plain rms =   9.15   stream rms =  3.65
layer 24   plain rms =  83.96   stream rms =  5.88
```

The seeded experiment ends with an RMS of 83.96 for `plain`, about 76 times its initial RMS of 1.10. The `stream` column grows too, but reaches only 5.88 in this run because each layer computes its update from a normalized input instead of multiplying the entire running total. This experiment is an analogy for the block's data flow, not its implementation: it uses RMS scaling without mean subtraction and a random linear update instead of attention and an MLP. The useful pattern is to normalize the branch input, compute an update, and add that update to the stream.

13.2 The Idea

13.2.1 The residual stream

↪ **Jump ahead:** Synthesis II
Autoregressive text generation and next-token probability distribution sampling with TinyGPT.

Originally introduced in deep vision models like ResNet, residual skip connections form the uninterrupted central communication backbone of modern Transformer architectures like GPT and LLaMA. By providing an identity shortcut alongside each learned transformation, residual connections ensure that gradients can propagate backward through hundreds of layers without vanishing. A **residual connection** passes a layer's input around the layer and adds it to the layer's output:

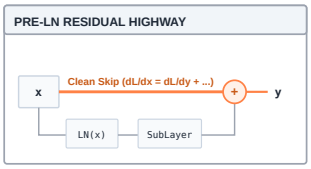
$$x_{l+1} = x_l + f_l(x_l)$$

Here x_l is the vector a token carries into layer l, f_l is whatever the layer computes, and x_{l+1} is what the token carries out. The layer's job changes from "produce the next representation" to "produce a correction to the current one."

The gradient shows why this helps with depth. Differentiating the equation gives

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_{l+1}} \left(I + \frac{\partial f_l}{\partial x_l} \right)$$

where $\mathcal{L}$ is the loss and I is the identity matrix. The identity term means the gradient arriving at layer $l+1$ reaches layer l unchanged, with the layer's own contribution added on. Across N residual sub-layers the chain rule still multiplies N Jacobians, the matrices of output derivatives with respect to input features, but each has the form $I + J_l$. Expanding that product reveals a direct identity contribution as well as paths through the learned updates. This route makes gradient propagation easier; it does not guarantee that gradients cannot grow, shrink, or cancel. In code this needs nothing new. The autograd mechanism from Chapter 6 already sums the gradients of a tensor that is used on two paths, and x_l is used on two paths: once into f_l, once straight into the addition.



Pre-LN residual gradient highway ($I + J_l$).

A useful picture is to think of the residual path as a **stream**, a D-dimensional scratchpad that each token carries through the whole network. Each sub-layer reads the stream, computes a delta, and writes the delta back by addition. An update can cancel an earlier value by adding its negative, so the stream is not an immutable record. The useful guarantee is structural: the old vector reaches the addition directly instead of having to pass through the learned transformation. A 12-block GPT has 24 sub-layers reading and writing the same stream.

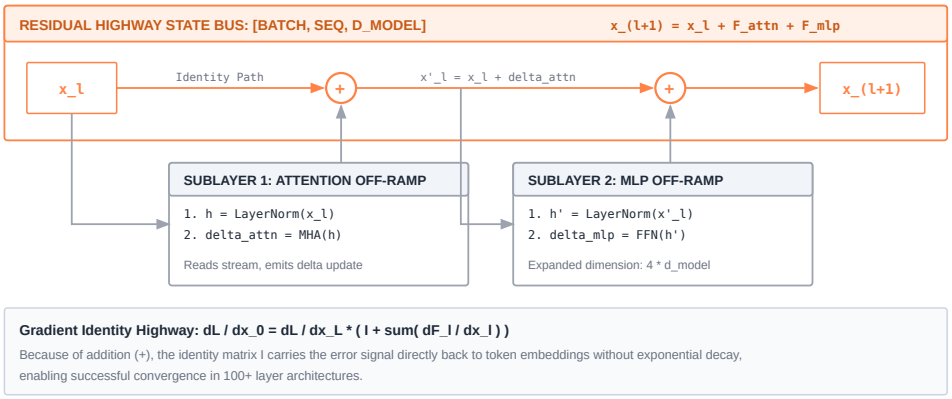


Figure 13.2: **The residual stream communication highway**: Token embeddings enter a central communication bus that flows directly from input to final projection. Each transformer block taps into the bus, extracts standardized features via LayerNorm, computes attention and MLP deltas, and adds them back onto the highway ($x_{l+1} = x_l + f_l(x_l)$). The identity term in $I + J_l$ gives gradients a direct backward route through every block.

13.2.2 Layer normalization

While batch normalization dominates convolutional computer vision, sequence models like Transformers rely on LayerNorm to stabilize activation dynamics independently of batch size. Because it standardizes statistics across feature channels for each token in isolation, it eliminates inter-sample batch dependencies and preserves autoregressive causality. Residual additions preserve a direct route through the stack, but they do not bound its scale. Each sub-layer still benefits from inputs at a known scale, for the same reason the initialization in Chapter 3 scales each weight matrix. **Layer normalization** standardizes each token's vector across its D features. For one token with features $x_1, \ldots, x_D$,

$$\mu = \frac{1}{D}\sum_{i=1}^{D} x_i, \qquad \sigma^2 = \frac{1}{D}\sum_{i=1}^{D}(x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y_i = \gamma_i \hat{x}_i + \beta_i$$

where μ and σ^2 are the mean and variance of that one token's features, ϵ is a small constant (typically 10^{-5}) that keeps the division safe when the variance is near zero, and γ and β are learned vectors of length D that let the network scale and shift the result. They start at $\gamma = 1$ and $\beta = 0$, so a fresh LayerNorm is pure standardization. Training can learn a different output scale and offset, but these shared parameters cannot recover the particular mean removed from each row. Everything here is computed from one token's own D numbers; why that matters is the first thing The Code section takes up.

13.2.3 Where the normalization goes

Modern generative language models (including GPT-2, GPT-3, and LLaMA) departed from the original Transformer specification by shifting the placement of normalization layers. Relocating normalization from the residual path to the sub-layer inputs (Pre-LN)

creates a pristine identity gradient highway, eliminating the need for delicate learning-rate warmup schedules and enabling stable scaling to hundreds of layers. The original transformer normalized the stream itself after each addition, an arrangement now called **Post-LN**:

$$x_{l+1} = \text{LN}(x_l + f_l(x_l))$$

TinyTorch places normalization at the input of the sub-layer, an arrangement called **Pre-LN**:

$$x_{l+1} = x_l + f_l(\text{LN}(x_l))$$

In Post-LN, each residual addition feeds a normalization, so even the direct route through a block includes normalization derivatives. A stack of N blocks has two such normalizations per block. In Pre-LN, each branch computes a normalized version of its input while the bypass carries the unnormalized stream to the addition. Only the final LayerNorm, just before the vocabulary projection, normalizes the stream leaving the stack. The reference listings use this Pre-LN arrangement.

Why did many decoder models move from the original Post-LN arrangement (Vaswani et al.) to Pre-LN (GPT-2, LLaMA, TinyGPT)? In Post-LN ($x_{l+1} = \text{LN}(x_l + f_l(x_l))$), the residual connection passes *through* the normalization layer. Computing the gradient requires chaining through LN′ at every single block. For deep networks ($N \geq 24$), this degrades gradient magnitudes, requiring delicate learning rate warmup schedules to prevent divergence. In Pre-LN ($x_{l+1} = x_l + f_l(\text{LN}(x_l))$), the residual bypass is an uninterrupted identity path $I + J_l$, allowing backpropagation gradients to flow from the loss directly to early token embeddings without passing through a normalization derivative on the bypass.

13.2.4 The MLP

In all Transformer architectures, multi-head attention acts purely as a routing mechanism that re-mixes existing features across sequence positions. To synthesize new representations and store associative knowledge, every block pairs attention with a position-wise Feed-Forward Network (MLP) that transforms features along the channel dimension. The second sub-layer supplies the per-token feature transformation missing from attention alone. The **MLP** (multi-layer perceptron, the transformer literature's name for a two-layer feed-forward network) widens each token's vector from D to $4D$, applies GELU from Chapter 2, and projects back to D:[53]

$$\text{MLP}(x) = \text{GELU}(xW_1 + b_1)\,W_2 + b_2, \qquad W_1 \in \mathbb{R}^{D\times 4D},\ W_2 \in \mathbb{R}^{4D\times D}$$

Here x is one token's row vector, as in the attention projection $q_t = x_t W_q$. The same W_1, W_2 apply to every position, and no position sees any other. The division of labor inside a block is therefore clean. Attention decides which tokens' information to gather; the MLP decides what to do with what was gathered, one token at a time. The feed-forward sub-layer has the same linear-activation-linear shape as the perceptron built in Synthesis I and Chapter 3, here with GELU and a 4D hidden width, applied independently to every token.

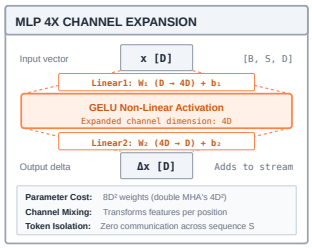
[53] The $4\times$ hidden expansion ($d_{\text{ff}} = 4D$) originated in Vaswani et al. (2017) as an empirical heuristic balancing non-linear representation capacity against parameter budget. It allocates $2 \times D \times 4D = 8D^2$ weights to the feed-forward network, exactly double the $4D^2$ projection weights of Multi-Head Attention. Modern architectures using 3-matrix gated activations like SwiGLU (e.g. LLaMA) scale the expansion to approximately $\frac{8}{3}D$ so that $3 \times D \times \frac{8}{3}D = 8D^2$, maintaining the exact same parameter and FLOP allocation.

13.2.5 Assembling the model

By stacking identical Transformer blocks and terminating with a vocabulary projection head, decoder architectures like GPT-2 and LLaMA construct unified generative language models. This end-to-end composition processes raw token sequences through continuous residual representations to predict next-token probability distributions across the entire vocabulary. With the block defined, the whole model is a short list. Tokens $t_1, \ldots, t_S$ enter as embedding rows plus position vectors from Chapter 11, pass through N blocks, get normalized once more, and project onto the vocabulary:

$$
\begin{aligned}
h_0 &= E_{\text{tok}}[t] + P, \\
h_l &= \text{Block}_l(h_{l-1}) \quad \text{for } l = 1, \ldots, N, \\
\text{logits} &= \text{LN}_{\text{final}}(h_N)\, W_{\text{head}}
\end{aligned}
$$

Here E_{tok} is the $V \times D$ embedding table, P the position vectors, and W_{head} a $D \times V$ matrix that turns a D-dimensional hidden state into one score per vocabulary entry, the logits used by Chapter 4. The output has one row of V scores per position, and the `GPT` listing explains why one row per position, rather than one row per pass, is the whole point.

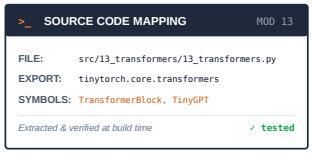


MLP Channel Expansion: Position-wise feedforward network widens features from D to 4D for non-linear separation before returning to stream dimension D.

13.3 The Code

A block receives a tensor of shape (B, S, D) and returns the same shape. Its attention layer uses the head split and causal mask from Chapter 12; its feature transformation reuses `Linear` and `GELU`. The missing operation is normalization of one token's row before either branch reads it.

Consider `[1, 2, 3, 4]`. Subtracting its mean of 2.5 and dividing by its standard deviation gives approximately `[-1.3416, -0.4472, 0.4472, 1.3416]`. Appending another token or putting another sequence in the batch must not change that result. Pooling statistics across rows would violate this requirement and could even let a future token influence an earlier prediction. `LayerNorm.forward` therefore validates the final dimension and calls `LayerNormFunction.apply(x, gamma, beta, eps=eps)`. As in Chapter 6, `apply` creates an operation for this call, passes NumPy arrays into its `forward`, and records the Tensor inputs needed for backward.



Source Code Mapping: Module 13 extracts LayerNorm, MLP expansion, TransformerBlock, and the flagship TinyGPT model directly from the working repository.

```python
def forward(self, x: np.ndarray, gamma: np.ndarray, beta: np.ndarray)
    -> np.ndarray:
    """
    Apply layer normalization to a NumPy array.
    """
    # Compute statistics across last dimension (features)
    mean_data = np.mean(x, axis=-1, keepdims=True)
    # Compute variance: E[(x - μ)²]
    diff = x - mean_data
    variance = np.mean(diff * diff, axis=-1, keepdims=True)
    # Normalize: (x - mean) / sqrt(variance + eps)
    self.std_data = np.sqrt(variance + self.eps)
    self.normalized_data = diff / self.std_data
```

```
    # Apply learnable transformation: gamma * normalized + beta
    return gamma * self.normalized_data + beta
```

```
class LayerNorm:
    """
    Layer Normalization for transformer blocks.
    """

    def __init__(self, normalized_shape: int, eps: float = 1e-5):
        """
        Initialize LayerNorm with learnable parameters.
        """
        if not isinstance(normalized_shape, (int, np.integer)) or
        ↪   normalized_shape <= 0:
            raise ValueError("TinyTorch LayerNorm normalizes one
            ↪   positive final dimension")
        self.normalized_shape = normalized_shape
        self.eps = eps

        # Learnable parameters: scale and shift
        self.gamma = Tensor(np.ones(normalized_shape),
        ↪   requires_grad=True)  # Scale parameter
        self.beta = Tensor(np.zeros(normalized_shape),
        ↪   requires_grad=True)  # Shift parameter

    def forward(self, x: Tensor) -> Tensor:
        """
        Apply layer normalization.
        """
        if not x.shape or x.shape[-1] != self.normalized_shape:
            raise ValueError(f"LayerNorm expected final dimension
            ↪   {self.normalized_shape}, got {x.shape}")
        return LayerNormFunction.apply(x, self.gamma, self.beta,
        ↪   eps=self.eps)

    def __call__(self, x: Tensor) -> Tensor:
        """Allows the layer norm to be called like a function."""
        return self.forward(x)

    def parameters(self) -> List[Tensor]:
        """Return learnable parameters."""
        return [self.gamma, self.beta]
```

Every statistic in `LayerNormFunction.forward` uses the last axis with `keepdims=True`. For an input (B, S, D), the mean and standard deviation have shape $(B, S, 1)$; subtraction and division broadcast each statistic across its own row. `gamma` and `beta` each have shape $(D,)$ and broadcast across batch and sequence positions. The layer owns these $2D$ trainable values. A constant row becomes zero after centering and therefore returns `beta`, with `eps` keeping the division defined.

The operation stores `normalized_data` and `std_data` for this particular call. They belong to the operation recorded on the graph, not to shared running statistics in the

`LayerNorm` object. Calling the same layer again creates another operation with its own saved arrays. Its returned Tensor has new storage; the input stream remains unchanged. This separation lets two branches normalize their inputs without overwriting values another graph path still needs.

On the backward pass, `grad_output` has the output's (B, S, D) shape. The gradient of `beta` sums these contributions over batch and sequence, because every row used the same shift. The gradient of `gamma` first multiplies by the saved normalized values and then makes the same reductions. For the input gradient, `grad_norm = grad_output * gamma.data` applies the learned scale *before* the row means are subtracted. Moving `gamma` outside those reductions would be wrong once different features learn different scales. The two mean terms account for the input's effect on its own mean and variance; dividing by the saved standard deviation completes the return to the input's units.

```
def backward(self, grad_output: np.ndarray) ->
    Tuple[Optional[np.ndarray], Optional[np.ndarray],
    Optional[np.ndarray]]:
    """Compute gradients for LayerNorm (x, gamma, beta)."""
    x, gamma, beta = self.inputs

    grad_x = grad_gamma = grad_beta = None
    normalized = self.normalized_data
    std_data = self.std_data

    # Gradient for beta: sum over all dims except last
    if beta.requires_grad:
        # Sum over batch and sequence dimensions
        grad_beta = grad_output.copy()
        while grad_beta.ndim > 1:
            grad_beta = grad_beta.sum(axis=0)

    # Gradient for gamma: sum of (grad_output * normalized) over
        batch/seq dims
    if gamma.requires_grad:
        grad_gamma = (grad_output * normalized).copy()
        while grad_gamma.ndim > 1:
            grad_gamma = grad_gamma.sum(axis=0)

    # Gradient for x: full LayerNorm backward formula
    if x.requires_grad:
        # grad flowing through gamma: grad_output * gamma
        grad_norm = grad_output * gamma.data

        mean_grad = np.mean(grad_norm, axis=-1, keepdims=True)
        mean_grad_norm = np.mean(grad_norm * normalized, axis=-1,
    keepdims=True)
        grad_x = (1.0 / std_data) * (grad_norm - mean_grad -
    normalized * mean_grad_norm)

    return (grad_x, grad_gamma, grad_beta)
```

The returned tuple follows the input order `(x, gamma, beta)`. Autograd sends each gradient to the matching Tensor and accumulates contributions if that Tensor appears

elsewhere in the graph. An input that does not require a gradient can still train `gamma` and `beta`; the operation checks each input independently.

`MLP` transforms the features attention has gathered, independently at each position. Its `linear1` widens the last axis from D to $4D$, `gelu` changes the hidden features nonlinearly, and `linear2` returns the last axis to D so the result can be added to the stream. No operation in this path mixes sequence positions. The default hidden width allocates $8D^2$ matrix weights to the MLP, compared with attention's $4D^2$ projection weights; changing it is a capacity choice rather than a requirement of residual addition.

```python
class MLP:
    """
    Multi-Layer Perceptron (Feed-Forward Network) for transformer blocks.
    """

    def __init__(self, embed_dim: int, hidden_dim: Optional[int] = None, dropout_prob: float = 0.0):
        """
        Initialize MLP with two linear layers.
        """
        if hidden_dim is None:
            hidden_dim = 4 * embed_dim  # Standard 4x expansion

        self.embed_dim = embed_dim
        self.hidden_dim = hidden_dim
        # Keep this teaching model deterministic; do not silently ignore dropout.
        if dropout_prob != 0:
            raise ValueError("TinyTorch MLP supports dropout_prob=0 only")
        self.dropout_prob = dropout_prob

        # Two-layer feed-forward network
        self.linear1 = Linear(embed_dim, hidden_dim)
        self.gelu = GELU()  # Use GELU activation from activations module
        self.linear2 = Linear(hidden_dim, embed_dim)

    def forward(self, x: Tensor) -> Tensor:
        """
        Forward pass through MLP.
        """
        # First linear layer with expansion
        hidden = self.linear1.forward(x)

        # GELU activation (YOUR activation from Module 02!)
        hidden = self.gelu.forward(hidden)

        # Second linear layer back to original size
        output = self.linear2.forward(hidden)

        return output
```

```
39
40      def __call__(self, x: Tensor) -> Tensor:
41          """Allows the MLP to be called like a function."""
42          return self.forward(x)
43
44      def parameters(self) -> List[Tensor]:
45          """Return all learnable parameters."""
46          params = []
47          params.extend(self.linear1.parameters())
48          params.extend(self.linear2.parameters())
49          return params
```

`linear1.forward(x)` multiplies each row by its stored $(D, 4D)$ weight matrix and adds its $(4D,)$ bias. `gelu.forward(hidden)` records an elementwise operation on that widened tensor; this reference uses the sigmoid approximation from Chapter 2. `linear2.forward(hidden)` contracts the feature axis back to D. The position and batch axes never enter a reduction, so changing one token cannot affect another through this path. `MLP.parameters()` returns both linear layers' weights and biases; GELU contributes no parameters. The constructor rejects nonzero `dropout_prob` because this compact block omits dropout.

The return to width D is what makes `TransformerBlock.forward` possible. `ln1.forward(x)` computes a normalized branch input, and `attention.forward(normed1, mask)` returns an update with exactly the stream's shape. In `x = x + attention_out`, Python rebinds the local name `x` to a new Tensor; it does not modify the caller's Tensor. `ln2` must read this updated stream, not the original argument, so the MLP can act on the attention result. The final addition returns another Tensor of shape (B, S, D).

```
1  class TransformerBlock:
2      """
3      Complete Transformer Block with self-attention, MLP, and residual
   ↪   connections.
4      """
5
6      def __init__(self, embed_dim: int, num_heads: int, mlp_ratio:
   ↪   Optional[float] = None, *,
7                   ff_dim: Optional[int] = None, dropout_prob: float =
   ↪   0.0):
8          """
9          Initialize a complete transformer block.
10         """
11         self.embed_dim = embed_dim
12         self.num_heads = num_heads
13
14         # Multi-head self-attention
15         self.attention = MultiHeadAttention(embed_dim, num_heads)
16
17         # Layer normalizations (pre-norm architecture)
18         self.ln1 = LayerNorm(embed_dim)  # Before attention
19         self.ln2 = LayerNorm(embed_dim)  # Before MLP
20
```

```
        # Feed-forward network. mlp_ratio and ff_dim are two spellings of the same
        # quantity, so accepting both at once would silently honor one and drop the other.
        if ff_dim is not None and mlp_ratio is not None:
            raise ValueError(
                "Give the MLP width once. Pass mlp_ratio or ff_dim, not both "
                f"(got mlp_ratio={mlp_ratio}, ff_dim={ff_dim})."
            )
        if ff_dim is not None:
            hidden_dim = ff_dim
        else:
            hidden_dim = int(embed_dim * (4 if mlp_ratio is None else mlp_ratio))
        self.mlp = MLP(embed_dim, hidden_dim, dropout_prob)

    def forward(self, x: Tensor, mask: Optional[Tensor] = None) -> Tensor:
        """
        Forward pass through transformer block.
        """
        # First sub-layer: Multi-head self-attention with residual connection
        # Pre-norm: LayerNorm before attention
        normed1 = self.ln1.forward(x)
        # Self-attention: query, key, value are all the same (normed1)
        attention_out = self.attention.forward(normed1, mask)

        # Residual connection
        x = x + attention_out

        # Second sub-layer: MLP with residual connection
        # Pre-norm: LayerNorm before MLP
        normed2 = self.ln2.forward(x)
        mlp_out = self.mlp.forward(normed2)

        # Residual connection
        output = x + mlp_out

        return output

    def __call__(self, x: Tensor, mask: Optional[Tensor] = None) -> Tensor:
        """Allows the transformer block to be called like a function."""
        return self.forward(x, mask)

    def parameters(self) -> List[Tensor]:
        """Return all learnable parameters."""
        params = []
        params.extend(self.attention.parameters())
```

```
        params.extend(self.ln1.parameters())
        params.extend(self.ln2.parameters())
        params.extend(self.mlp.parameters())
        return params
```

Both additions record their two inputs on the graph. During backward, the incoming gradient reaches the residual input directly and also travels through the learned branch; autograd adds the contributions where those paths meet. Neither the forward additions nor `parameters()` update a weight. The block's parameter list collects attention, both normalizations, and the MLP so that an optimizer can update the entire composition after backward.

For the input `[3, 7, 1]`, the corresponding training targets might be `[7, 1, 5]`. `GPT.forward` can compute all three predictions in one pass because the mask forbids position 0 from reading token 7 or 1, and position 1 from reading token 1. The next-token targets come from the training data; `GPT` returns scores and does not construct labels or compute a loss.

The constructor accepts its building blocks as optional keyword arguments (`block_cls`, `norm_cls`, `mask_fn`, and the sampling and generation functions), and each one left out defaults to the class or function exported from this module. That lets this module's notebook build a `GPT` from your own `TransformerBlock`, `LayerNorm`, and mask and test them before they are exported. The constructor then creates an `EmbeddingLayer` with learned position vectors. Each iteration calls `block_cls(embed_dim, num_heads)` to construct a new block and appends it to `self.blocks`, so depth adds independent parameters rather than reusing one block's weights. In `forward`, the token shape supplies `seq_len`; the embedding layer returns (B, S, D), adding position rows beginning at `start_pos`. `forward` calls `self._mask_fn(seq_len)`, by default `create_causal_mask`, which returns a lower-triangular matrix of ones with zeros above the diagonal and a leading singleton axis. For $S = 3$ its rows are `[1, 0, 0]`, `[1, 1, 0]`, and `[1, 1, 1]`. Attention broadcasts this $(1, S, S)$ mask across batches and heads, following its binary mask contract.

```
class GPT:
    """
    Complete GPT (Generative Pre-trained Transformer) model.
    """

    def __init__(
        self,
        vocab_size: int,
        embed_dim: int,
        num_layers: int,
        num_heads: int,
        max_seq_len: int = 1024,
        *,
        block_cls: Optional[type] = None,
        norm_cls: Optional[type] = None,
        mask_fn: Optional[Callable] = None,
        sample_fn: Optional[Callable] = None,
        generate_fn: Optional[Callable] = None,
```

```
    ):
        block_cls = block_cls or _exported("TransformerBlock")
        norm_cls = norm_cls or _exported("LayerNorm")
        self._mask_fn = mask_fn or _exported("create_causal_mask")
        self._sample_fn = sample_fn or _exported("sample_next_token")
        self._generate_fn = generate_fn or _exported("generate")

        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        self.num_layers = num_layers
        self.num_heads = num_heads
        self.max_seq_len = max_seq_len

        # Embedding layer (Token + Learned Positional)
        self.embedding_layer = EmbeddingLayer(vocab_size, embed_dim,
        ↪   max_seq_len)

        # Stack of transformer blocks
        self.blocks = [
            block_cls(embed_dim, num_heads) for _ in range(num_layers)
        ]

        # Final layer normalization
        self.ln_f = norm_cls(embed_dim)

        # Language modeling head (projects to vocabulary)
        self.lm_head = Linear(embed_dim, vocab_size, bias=False)

    def forward(self, tokens: Tensor, start_pos: int = 0) -> Tensor:
        """Forward pass through GPT model."""
        batch_size, seq_len = tokens.shape

        # Token + positional embeddings
        x = self.embedding_layer.forward(tokens, start_pos)

        # Causal mask for autoregressive routing
        mask = self._mask_fn(seq_len)

        # Transformer blocks
        for block in self.blocks:
            x = block.forward(x, mask)

        # Final LayerNorm & LM Head projection
        x = self.ln_f.forward(x)
        logits = self.lm_head.forward(x)

        return logits

    def __call__(self, tokens: Tensor, start_pos: int = 0) -> Tensor:
        return self.forward(tokens, start_pos)

    def _create_causal_mask(self, seq_len: int) -> Tensor:
```

```
        return self._mask_fn(seq_len)

    def _sample_next_token(self, logits: np.ndarray, temperature: float
    ↪   = 1.0) -> int:
        ...

    def generate(
        self,
        prompt_tokens: Tensor,
        max_new_tokens: int = 50,
        temperature: float = 1.0,
    ) -> Tensor:
        """Autoregressively generate tokens using the decoupled
        ↪   generate utility."""
        ...

    def parameters(self) -> List[Tensor]:
        """Return all learnable parameters."""
        params = []
        params.extend(self.embedding_layer.parameters())
        for block in self.blocks:
            params.extend(block.parameters())
        params.extend(self.ln_f.parameters())
        params.extend(self.lm_head.parameters())
        return params
```

The loop passes the same mask to every block and carries the returned stream into the next. After the last block, `ln_f` normalizes the stream for the output projection. `lm_head` is a bias-free `Linear(D, V)` with its own weight matrix; it turns every final row into V logits without changing batch or position axes. No softmax is needed here because the loss accepts logits, and the generation loop will choose how to turn the last row into a token.

The model retains its parameters between calls, but an ordinary `forward` retains no usable attention history for a later call. `start_pos` selects positional rows only. Passing one token with `start_pos=2` gives it position 2 but does not supply tokens 0 and 1 to attention. The cache in Chapter 19 will add that missing state. Until then, generation must pass the whole growing prefix each time. Learned position capacity also bounds a call: `start_pos + S` cannot exceed `max_seq_len`.

13.4 A Worked Trace

Take the first of two tokens, whose stream vector is $\mathbf{x} = [1, 2, 3, 4]$ with $D = 4$, and follow it through one block with $\gamma = 1$, $\beta = 0$. The first LayerNorm is fully hand-checkable:

$$\mu = \frac{1+2+3+4}{4} = 2.5,$$
$$\sigma^2 = \frac{2.25+0.25+0.25+2.25}{4} = 1.25,$$
$$\sqrt{\sigma^2+\epsilon} \approx 1.1180$$

$$\hat{\mathbf{x}} = \frac{[1, 2, 3, 4] - 2.5}{1.1180} = [-1.3416, -0.4472, +0.4472, +1.3416]$$

We can make the *actual* attention and MLP hand-checkable by choosing their weights. Use two heads, set the query and key projections to zero, and set the value and output projections to identity matrices. Before masking every attention score is zero. The causal mask leaves token 0 with attention weights `[1, 0]` and token 1 with `[0.5, 0.5]` in each head. Choose the second input row as `[4, 3, 2, 1]`, whose normalized values are the negatives of the first row's. Attention therefore returns the first normalized row at position 0 and an all-zero row at position 1.

For this trace only, set `ff_dim=4` rather than the default 16. Identity MLP matrices and zero biases then make its output exactly GELU of `ln2`'s result. These choices keep every real helper and graph operation in the path; no sub-layer result is substituted by hand. We can follow the first token in table 13.1 while the second token makes the attention average visible.

Table 13.1: **A real block with controlled weights**: The first token traverses both branches and residual additions. The second token has the opposite normalized input, so its causal attention average cancels.

Stage	Computes	Input	Output
Stream in			$[1.000, 2.000, 3.000, 4.000]$
1. `ln1`	$\mathrm{LN}(\mathbf{x})$	$[1.000, 2.000, 3.000, 4.000]$	$[-1.342, -0.447, +0.447, +1.342]$
2. `attention`	weighted normalized values	$[-1.342, -0.447, +0.447, +1.342]$	$[-1.342, -0.447, +0.447, +1.342]$
3. add	$\mathbf{x}_1 = \mathbf{x} + \mathrm{Attn}$	$[1.000, 2.000, 3.000, 4.000]$	$[-0.342, 1.553, 3.447, 5.342]$
4. `ln2`	$\mathrm{LN}(\mathbf{x}_1)$	$[-0.342, 1.553, 3.447, 5.342]$	$[-1.342, -0.447, +0.447, +1.342]$
5. `mlp`	identity, GELU, identity	$[-1.342, -0.447, +0.447, +1.342]$	$[-0.124, -0.142, +0.305, +1.217]$
6. add	$\mathbf{x}_2 = \mathbf{x}_1 + \mathrm{MLP}$	$[-0.342, 1.553, 3.447, 5.342]$	$[-0.466, 1.410, 3.752, 6.559]$

In listing 13.2, line 6 instantiates `TransformerBlock(4, 2, ff_dim=4)` and lines 7 to 13 zero the biases while configuring projection and feedforward weights as identity matrices. Line 15 supplies a two-token input `x` alongside the causal mask in line 17. Lines 18 to 22 trace the internal sequence step-by-step: `n1 = block.ln1(x)`, `attention = block.attention(n1, mask)`, first residual `x1 = x + attention`, second norm `n2 = block.ln2(x1)`, and `mlp = block.mlp(n2)`. In line 23, `result = block(x, mask)` runs the full composite forward method. Lines 24 to 26 verify that `result` matches the explicit residual sum `x1 + mlp`, that `x` is unmodified, and that token 1's attention update cancels to zero as predicted. Line 27 prints the final block outputs (lines 28 to 31).

Listing 13.2 Tracing the complete block: Controlled parameters expose the causal average and both residual additions without replacing any forward method.

```
import numpy as np
import tinytorch.core.autograd
from tinytorch.core.tensor import Tensor
from tinytorch.core.transformers import TransformerBlock,
    create_causal_mask

block = TransformerBlock(4, 2, ff_dim=4)
for param in block.parameters():
    param.data.fill(0)
for norm in (block.ln1, block.ln2):
    norm.gamma.data.fill(1)
for layer in (block.attention.v_proj, block.attention.out_proj,
              block.mlp.linear1, block.mlp.linear2):
    layer.weight.data[:] = np.eye(4)

x = Tensor([[[1., 2., 3., 4.], [4., 3., 2., 1.]]])
before = x.data.copy()
mask = create_causal_mask(2)
n1 = block.ln1(x)
attention = block.attention(n1, mask)
x1 = x + attention
n2 = block.ln2(x1)
mlp = block.mlp(n2)
result = block(x, mask)
np.testing.assert_allclose(result.data, (x1 + mlp).data)
np.testing.assert_array_equal(x.data, before)
np.testing.assert_allclose(attention.data[0, 1], 0, atol=1e-6)
print(np.round(result.data[0], 4))
```

```
[[-0.4657  1.4104  3.752   6.5592]
 [ 5.2175  3.3048  1.8576  0.8759]]
```

The parameter assignments above create the trace's starting state. The subsequent forward calls leave those weights and the caller's `x` unchanged; each residual addition returns a new Tensor. Token 1 receives no attention update in this chosen example, yet its MLP still changes its features. Attention mixing and the per-token transformation have distinct jobs even when one branch's output is zero.

A full GPT adds embeddings and vocabulary scores around that block mechanism. The second half of the trace uses the default MLP width and follows shapes through two blocks. A `GPT` with $D = 8$, two heads, two blocks, and a vocabulary of 16 tokens, given one sequence of three token IDs, moves the following tensors; the leading batch axis of 1 is left off.

Table 13.2: **Shape boundaries in GPT**: A two-block model preserves stream width until its final vocabulary projection.

Stage	Tensor	Shape (rows × columns)
Input	token IDs	3 integers
`embedding_layer`	stream h_0	3×8
Block 1, `ln1` output	normalized copy	3×8
Block 1, `attention`, per head	Q, K, V	3×4 each, 2 heads
Block 1, `attention`, per head	scores $QK^\top/\sqrt{4}$ + mask	3×3
Block 1, `mlp` hidden	after `linear1` and `gelu`	3×32
Block 1 output, Block 2 output	stream h_1, h_2	3×8
`ln_f`, then `lm_head`	normalized stream, then logits	3×8, then 3×16

The full model must preserve causality across every block. In listing 13.3, line 8 initializes `model = GPT(vocab_size=16, embed_dim=8, num_layers=2, num_heads=2, max_seq_len=8)` and line 9 feeds a three-token sequence `[[3, 7, 1]]`. Line 10 confirms output shape `(1, 3, 16)`, and line 11 verifies the exact parameter count of 2,080. In lines 14 to 15, appending token 5 to form sequence `[[3, 7, 1, 5]]` verifies prefix invariance: `assert np.allclose(logits.data, longer.data[:, :3])` passes, proving future tokens cannot leak backward across either block. In lines 18 to 25, a next-token cross-entropy loss at the final sequence position backpropagates through the entire depth of both blocks; lines 20 to 25 confirm that token embeddings and the active position vectors (indices 0 to 2) receive nonzero gradients, while unvisited position slots (index 3+) remain strictly zero.

Listing 13.3 Checking the GPT contract: Appending a suffix preserves prefix logits, and a last-position loss reaches token and positional embeddings.

```
from tinytorch.core.transformers import GPT
from tinytorch.core.losses import CrossEntropyLoss

def count_parameters(model):
    """Total scalars across every tensor in parameters()."""
    return sum(int(np.prod(p.shape)) for p in model.parameters())

model = GPT(vocab_size=16, embed_dim=8, num_layers=2, num_heads=2,
    max_seq_len=8)
logits = model(Tensor([[3, 7, 1]]))
print(logits.shape)                          # (1, 3, 16)
print(count_parameters(model))               # 2080

# Appending a token must preserve every earlier logit row.
longer = model(Tensor([[3, 7, 1, 5]]))
assert np.allclose(logits.data, longer.data[:, :3], atol=1e-5)

# The loss at the last position reaches parameters on both ends of the
    stack.
loss = CrossEntropyLoss()(logits[:, -1, :], Tensor([5]))
loss.backward()
assert np.isfinite(model.embedding_layer.token_embedding.weight.grad).
    all()
assert np.any(model.embedding_layer.token_embedding.weight.grad != 0)
assert np.any(model.lm_head.weight.grad != 0)
positions = model.embedding_layer.pos_encoding.position_embeddings
assert np.all(np.max(np.abs(positions.grad[:3]), axis=1) > 0)
np.testing.assert_array_equal(positions.grad[3:], 0)
```

Of the 2,080 parameters, 1,744 sit in the two blocks (872 each, of which 552 are the MLP's, 288 attention's including its four biases, and 32 the two LayerNorms'). The matrix weights alone have the two-to-one MLP-to-attention ratio derived beside the `MLP` listing. The rest are the embeddings (128 in the token table and 64 in the learned position table), the final LayerNorm's 16, and the head's 128. The stream keeps shape 3×8 from the embeddings to the final normalization. Everything that changes shape (the per-head split, the 3×3 score matrix, the $4\times$ hidden layer) lives inside a sub-layer and is folded back before the addition. The final row, at zero-based position 2, contains 16 scores for a fourth token the model has not seen.

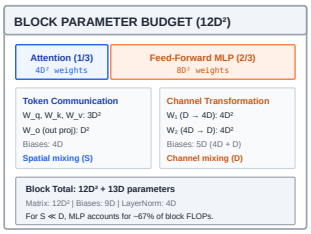
13.5 From TinyTorch to Production

The released GPT-2 implementation uses the same Pre-LN attention/addition and MLP/addition sequence, learned positions, and a final normalization. Its MLP uses a different GELU approximation, based on a hyperbolic tangent. More consequently, the final projection multiplies the final hidden states by the transpose of the token

embedding table (defined in Chapter 11) instead of owning another matrix. This is **weight tying**. Input lookup and output scoring then train one shared table through two paths, so the optimizer must receive that Tensor only once.

The memory consequence follows directly from the shapes. Untied, the token table and head each own VD weights. Tying removes one VD allocation but constrains both uses to share their learned values. For the trace model, it removes 128 parameters and changes the total from 2,080 to 1,952. At $V = 50{,}257$ and $D = 768$, the same change saves $38{,}597{,}376$ parameters. These are parameter counts, not measured speedups; output scoring still multiplies every final hidden row by a vocabulary-sized matrix.

The default TinyTorch block owns $12D^2$ matrix weights, $9D$ biases, and $4D$ normalization parameters, for $12D^2 + 13D$ scalars. With N blocks, vocabulary size V, and position capacity $S_{\max}$, its untied GPT owns $N(12D^2 + 13D) + 2VD + S_{\max}D + 2D$ parameters. Substituting $D = 8$, $N = 2$, $V = 16$, and $S_{\max} = 8$ gives $1{,}744 + 256 + 64 + 16 = 2{,}080$. The count depends on allocated position capacity, even when a call uses only three positions. Changing `ff_dim` or removing biases changes the formula.



Block Parameter Budget: Matrix weights split 1:2 between multi-head attention (4D squared) and feedforward MLP (8D squared), totaling 12D squared scalars.

Normalization placement is another design choice with a training consequence. Xiong and colleagues analyzed large gradients near the output at initialization in Post-LN transformers and demonstrated Pre-LN training without learning-rate warmup in their experiments. Warmup means gradually increasing the learning rate during early updates. This evidence motivates the reference's placement; it does not guarantee stable training for every depth, initialization, or learning rate. The residual-gradient equation explains the structural difference, while a training run tests whether the chosen scales work together.

The Llama 2 reference preserves the two residual additions but changes their branches. RMSNorm divides each row by its root-mean-square scale without subtracting its mean, and learns a featurewise scale without a shift. A gated MLP uses SwiGLU,[54] and rotary positions alter queries and keys inside attention rather than adding a position table to the stream. These are alternatives to the components traced above; TinyTorch's Module 13 implements none of those replacements.

[54] **SwiGLU**: an MLP whose hidden layer multiplies two linear projections elementwise, with one passed through the Swish activation. The gate lets one projection control the contribution of the other. Using three matrices rather than two changes the parameter count, so a narrower hidden layer can offset that extra matrix.

Table 13.3: **Alternative components, shared residual structure**: The implementations differ in normalization, positions, activation, and parameter sharing while preserving two additions per block.

Design choice	Module 13 `GPT`	GPT-2 released code	Llama 2 reference
Normalization	LayerNorm, Pre-LN	LayerNorm, Pre-LN	RMSNorm, Pre-LN
Position	learned table, added	learned table, added	RoPE, inside attention
MLP	$D \to 4D \to D$, sigmoid GELU	$D \to 4D \to D$, tanh GELU	gated MLP, SwiGLU
Head	separate $D \times V$	tied to embedding	separate $D \times V$
Dropout	none	absent from released inference path	none

The implementations in table 13.3 need not schedule those operations alike. TinyTorch executes separate NumPy operations for centering, squaring, reduction, scaling, and addition; its intermediate arrays occupy ordinary host memory. A production implementation may combine operations into a compiled kernel to avoid some intermediate

reads and writes. The code here establishes the arithmetic and data dependencies, not a GPU memory-traffic count or a universal fusion speedup. Chapter 15 will distinguish operation counts and storage estimates from elapsed measurements.

When a transformer exceeds the memory capacity of a single GPU, Megatron-LM splits individual weight matrices across GPUs without requiring pipeline stalls. In the Multi-Head Attention block, W_q, W_k, W_v are column-parallel (split across heads), while W_o is row-parallel, requiring only a single `AllReduce` communication at the block output. Similarly, in the MLP block, W_1 is column-parallel ($D \to 4D/K$) and W_2 is row-parallel ($4D/K \to D$), requiring exactly one `AllReduce` before the residual addition. This symmetric decomposition minimizes inter-GPU communication to just two collective calls per transformer block.

Generation adds a separate cost. Module 13 appends one sampled token at a time, but each ordinary forward call recomputes the full prefix. The cache introduced in Chapter 19 will reuse earlier key and value projections; it will still need attention over the available prefix. That optimization changes retained state between calls, whereas the block traced here is a complete computation from its supplied sequence.

13.6 Summary

A GPT forward call begins with (B, S) token IDs. The embedding layer adds learned position rows, each block normalizes a branch input and adds attention and MLP updates, and the final normalization and head return (B, S, V) logits. The mask prevents every block from reading future positions, while normalization and the MLP act within each position. Together those properties explain why appending a suffix preserves earlier predictions.

Persistent state belongs to the parameter tensors. Forward operations create new activations and, when recording gradients, per-call operation records; they do not overwrite the caller's stream or update weights. `parameters()` exposes the complete ownership tree to the optimizer, and backward follows the recorded residual branches into that tree. Synthesis II uses these logits to construct a next-token loss and to sample from the final row. It must supply targets during training and the whole prefix during ordinary generation.

You have now constructed the flagship generative architecture of TinyTorch: `TinyGPT` (Modules 01 to 13). You can execute the milestone suite via the Tito companion CLI:

```
# Required Parts 1 and 2, in order: TinyGPT on Shakespeare, then sequence routing
tito milestone run 05

# Part 2 alone: prove multi-head attention on sequence routing tasks
tito milestone run 05 --part 2

# Optional Part 3: Train TinyCopilot on Python code with AST compiler gating
tito milestone run 05 --part 3

# Optional Part 4: Conversational Q&A and the Overfitting Detective
tito milestone run 05 --part 4
```

This milestone challenges multi-head attention on sequence routing tasks (reversal, copying, and a prefix-selected mix of the two), trains TinyGPT and gates it on training loss, held-out loss against a bigram baseline, and a causality probe before sampling from it, and scores greedy TinyCopilot completions for unseen function headers with `ast.parse` (the gate is at least 1 of 7; correct trainings parsed 1 to 6). Milestone 05 completes once Parts 1 and 2 have both passed; Parts 3 and 4 are optional extensions. Turn to Synthesis II to walk through the complete language modeling loop and next-token generation.

13.7 Further Reading

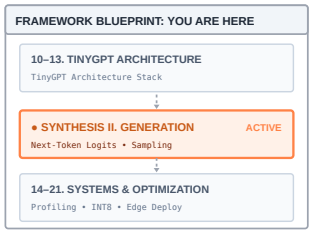
- **Xiong et al.** (2020). "On layer normalization in the transformer architecture." *ICML*. Post-LN and Pre-LN compared, and why the Pre-LN arrangement this chapter uses trains without warmup.
- **Radford et al.** (2019). "Language models are unsupervised multitask learners." *OpenAI*. The GPT-2 block that the chapter's `GPT` follows.
- **Kaplan et al.** (2020). "Scaling laws for neural language models." *arXiv*. How loss falls with parameters, data, and compute for this family of models.
- **Shoeybi et al.** (2019). "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism." *arXiv*. Tensor parallelism across multi-head attention and feed-forward linear layers.

🔥 Chapter 14

Synthesis II: Convolution, Attention, and Generated Text

14.0.1 The History and the Problem

In the mid-2010s, deep learning had conquered spatial perception with Convolutional Neural Networks (CNNs), but natural language processing remained bottlenecked. Sequence modeling relied heavily on Recurrent Neural Networks (RNNs) and LSTMs. This was a critical limitation: processing words sequentially meant that training could not be parallelized across GPUs, and models struggled to "remember" context from the beginning of a long paragraph.



Framework Blueprint: Synthesis II runs both Part II architectures against a task and a baseline, then closes the generative loop that Part III optimizes.

14.0.2 The Monumental Breakthrough

What needed to get unlocked was a mechanism that could look at an entire sequence at once and determine which words mattered to each other, regardless of their distance. In 2017, the introduction of the Transformer architecture ("Attention Is All You Need") provided this monumental breakthrough. It replaced sequential bottlenecks with self-attention mechanisms that processed data in parallel.

14.0.3 What Was Solved

Transformers fundamentally solved the context window and parallelization problem, kicking off the generative AI revolution. By closing the autoregressive loop, these architectures became capable of generating fluent, context-aware text one token at a time. Part II built these two foundational architectures, namely convolutional networks for spatial perception and transformers for dynamic sequence reasoning. Synthesis II integrates Modules 09 through 13 (`tinytorch/src/09_convolutions` through `tinytorch/src/13_transformers`). In the codebase, this synthesis chapter unites Milestone 04 (Convolutional Networks) and Milestone 05 (TinyGPT and the Transformer Era). This synthesis puts each architecture to work against concrete benchmarks, measuring what structural inductive bias buys, and closing the autoregressive loop that turns language models into generative systems.

In modern deep learning, autoregressive text generation is steered through stochastic sampling controls:

```
import torch
import torch.nn.functional as F

# Next-token autoregressive sampling with temperature and Top-k
    filtering
```

```python
def sample_next_token(logits, temperature=0.8, top_k=10):
    scaled_logits = logits / temperature
    top_values, _ = torch.topk(scaled_logits, top_k)
    scaled_logits[scaled_logits < top_values[:, [-1]]] = -float("inf")
    probs = F.softmax(scaled_logits, dim=-1)
    return torch.multinomial(probs, num_samples=1)
```

Underneath that sampling function, the runtime closes the full architecture loop:

1. **Temperature-scaled logit shaping:** Dividing raw next-token logits by temperature T flattens ($T > 1$) or sharpens ($T < 1$) the probability distribution before Softmax, controlling the balance between creative diversity and deterministic fidelity.
2. **Top-k probability truncation:** Masking out the long tail of low-probability vocabulary tokens with $-\infty$ prevents catastrophic non-sequiturs during open-ended generation.
3. **Closing the end-to-end generative cycle:** The sampling loop feeds newly generated token IDs back into the input sequence, creating the autoregressive inference workload that Part III will profile, compress, and accelerate.

This synthesis runs across three core regimes: evaluating convolutional networks against dense MLPs on TinyDigits (Milestone 04), testing attention on sequence reasoning tasks (Milestone 05, Part 2), and generating natural language and code with TinyGPT and TinyCopilot (Milestone 05, Parts 1 and 3).

14.1 Convolution on TinyDigits

↩ **Jump back:** Chapter 9 *The direct loop implementation this network runs, and the im2col lowering it does not use.*

Synthesis I's `DigitMLP` flattened each 8×8 digit into 64 numbers and learned 2,410 parameters. The CNN script (Milestone 04, `tinytorch/milestones/04_1998_cnn/01_lecun_tinydigits.py`, runnable via `tito milestone run cnn`) keeps the image an image. One `Conv2d` slides eight 3×3 filters over it, `MaxPool2d` halves each 6×6 feature map to 3×3, and a single `Linear` layer reads the 72 values that remain.

```python
def forward(self, x):
    # Conv + ReLU + Pool
    out = self.conv1.forward(x)
    out = self.relu1.forward(out)
    out = self.pool1.forward(out)

    # Flatten: (batch, 8, 3, 3) → (batch, 72)
    batch_size = out.shape[0]
    # Preserve the graph so gradients reach the convolutional filters.
    out = out.reshape(batch_size, -1)

    # Final classification
    out = self.fc.forward(out)
    return out
```

The flatten uses `Tensor.reshape`, so the pooled maps stay on the autograd graph and the loss can reach the filters. Wrapping `out.data` in a new `Tensor` would train the classifier and leave all eight filters at their initial values. Accuracy alone would not expose

that, because a classifier reading random filters still reaches about 81 percent on these digits, so the milestone also requires a nonzero filter gradient and filters that move at least 25 percent of their initial norm, and exits with status 1 when they do not.

The filters hold $8 \times 3 \times 3 + 8 = 80$ parameters and the classifier $72 \times 10 + 10 = 730$, which makes 810, a third of the MLP's 2,410. Parameters are not work, though. Every filter weight is applied at all 36 positions of its 6×6 output, so one image costs $8 \times 36 \times 9 = 2{,}592$ multiply-accumulates in the convolution and 720 in the classifier, 3,312 in all. The MLP spends $64 \times 32 + 32 \times 10 = 2{,}368$. Weight sharing moves the budget from stored weights to reused ones.

Table 14.1: Held-out accuracy on the 200 TinyDigits test images for both networks trained the same way, with SGD at learning rate 0.01 and batches of 32. Each cell is the range over five runs that differ only in batch shuffling; one test image is half a percentage point.

Epochs	CNN, 810 parameters	MLP, 2,410 parameters
20	73.0 to 75.5%	81.0 to 82.5%
50	86.0 to 87.0%	88.5 to 89.0%
100	89.0 to 90.5%	90.5 to 91.0%

The course materials compare the CNN's 50-epoch result with the MLP milestone's 20-epoch result, which is how they come to announce a win for convolution. At equal budgets there is no win on these images. The MLP leads at every row of table 14.1, and both networks approach 90 percent. An 8×8 digit has only 64 pixels, so a dense first layer can afford to connect every pixel to every hidden unit, and nothing about the input forces the network to share weights.

Resolution changes that. A dense layer's weight count grows with the number of input values, while a convolutional layer's depends only on filter size and channel counts. For a 32×32 color image, 3,072 input values, the MLP's 32-unit hidden layer would need $3{,}072 \times 32 = 98{,}304$ weights. The same eight 3×3 filters over three input channels need $8 \times 3 \times 3 \times 3 = 216$. That is the constraint LeNet was designed around, and it is the regime of the milestone's optional CIFAR-10 script. On 8×8 digits the constraint does not bind yet. The parameter saving is real and the accuracy gain is not.

The price shows up on the clock. In benchmark runs on a modern workstation, one training epoch over the 1,000 training images takes 1.48 seconds for the CNN and 0.017 seconds for the MLP, about 87 times longer for 1.4 times the multiply-accumulates. Arithmetic does not explain a gap that size. Chapter 9's `_convolve_loops` nests seven Python loops, so each of the 2,592 multiply-accumulates in the convolution is a scalar Python operation, and the backward pass loops again. The MLP's 2,368 run inside two NumPy `matmul` calls per batch. The im2col lowering in Chapter 9 turns the loops into one matrix multiply, and this measurement is the reason to want it.

14.2 Attention on Structured Sequences

This section examines the sequence routing experiment of **Milestone 05: The Transformer Era (2017)** (`02_vaswani_attention.py`, runnable via `tito milestone run 05 --part 2`), which isolates the **basic transformer mechanism**: proving that multi-head attention

learns sequence routing and content-dependent permutations without recurrent state. It does not train on prose. Instead, it builds an `AttentionTransformer` from the Chapter 11 through Chapter 13 pieces: token embeddings, a learned positional table, and two Post-LN blocks with four heads, a ReLU feed-forward layer, and no causal mask, since its tasks need every position to see every other. With a 29-token vocabulary (26 letters, padding, and two task markers) and 64-dimensional embeddings, it has 104,157 parameters.

↩ **Jump back:** Chapter 12 *Scaled dot-product attention and the multi-head projections these tasks exercise.*

It puts that model through three tests on six-letter sequences. Reversal maps `PYTHON` to `NOHTYP`. Copying maps `TENSOR` to itself. The mixed task prefixes each sequence with a marker, `[R]` or `[C]`, and the model must reverse or copy depending on which marker it sees. The first two tests share one model, trained on reversal and then on copying; the mixed test starts from fresh weights. A sequence counts as correct only if every position is. The pass thresholds are 95 percent on reversal, 95 on copying, and 90 on the mixed task, and training stops as soon as held-out accuracy crosses the threshold.

The training step, quoted from the script, flattens the model's `[B, S, V]` logits to `B·S` classification rows before the loss, counts completely correct sequences beside the loss, and takes its batches from Chapter 5's `DataLoader`.

```
def train_epoch(model, dataloader, optimizer, loss_fn):
    """Train for one epoch using YOUR DataLoader."""
    total_loss = 0.0
    correct_sequences = 0
    total_samples = 0

    # Use YOUR DataLoader for batched training!
    for input_batch, target_batch in dataloader:
        batch_size = input_batch.shape[0]

        logits = model(input_batch)

        # Reshape for loss computation
        logits_2d = logits.reshape(-1, model.vocab_size)
        target_1d = target_batch.reshape(-1)
        loss = loss_fn(logits_2d, target_1d)

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        total_loss += loss.data * batch_size

        # Check sequence accuracy per sample in batch
        pred = np.argmax(logits.data, axis=-1)
        for i in range(batch_size):
            if np.array_equal(pred[i], target_batch.data[i]):
                correct_sequences += 1
        total_samples += batch_size

    return total_loss / total_samples, (correct_sequences /
    ↪   total_samples) * 100
```

Table 14.2: Results of the attention sequence routing tasks over six runs that differ only in batch shuffling. On a standard laptop each epoch takes 0.13 to 0.17 seconds.

Task	Held-out sequences	Epochs to threshold	Sequence accuracy
Reversal	200	2	97.0 to 99.0%
Copying	200	2	100%
Mixed, by prefix	300	4	99.0 to 99.7%

Two epochs is fast, and the reason is structural. Reversal and copying are fixed permutations of positions. With a learned positional table and no mask, an attention head can learn queries and keys that match output position i to input position $5 - i$, or to i itself, and the value vectors carry the letter across. Copying is easier still, because the residual path already carries each letter straight through. The mixed task is the real test of attention. The permutation now depends on the content of position zero, so every position must read the marker and route accordingly. Starting from fresh weights, like reversal, it needs twice as many epochs, and it is the behavior that a fixed, input-independent mixing of positions cannot express.

The script builds its optimizer list from each component's `parameters()`, including the learned positional table. That table receives gradients and is updated, unlike Chapter 11's intentionally fixed sinusoidal table. Checking this registration boundary matters even when a score passes, because a model can learn through its other weights while an omitted component never changes.

Scoring uses `np.argmax`, and here the argmax is exactly right. Reversal has one correct output, so the most probable token is the answer and sampling would only add errors. Deterministic sequence routing proves the mechanics of attention, but production language modeling requires open-ended generation. That challenge defines our flagship workload: **TinyGPT** (`01_tinygpt_shakespeare.py`, runnable via `tito milestone run 05`). TinyGPT recreates the **Generative LLM Revolution**: the foundational autoregressive transformer architecture behind GPT-3 and ChatGPT. It is our performance-oriented workload designed for open-ended generation, where many continuations are acceptable, always taking the argmax fails, and autoregressive inference creates the quadratic prefix recomputation bottleneck that Part III and Synthesis III will systematically optimize with Key-Value caching, quantization, and kernel acceleration.

14.3 The Problem

Take a trained `TinyGPT`, feed it the prompt "TinyTorch is", and do the obvious thing: pick the highest-scoring token, append it, repeat.

```
# The obvious decoder: always take the argmax
for _ in range(20):
    logits = model.forward(Tensor(np.array([token_ids])))   # [1, S, V]
    next_id = int(np.argmax(logits.data[0, -1]))            # last row, best score
```

```
    token_ids.append(next_id)
```

↩ **Jump back:** Chapter 13 *Full-prefix forward passes through TinyGPT's stacked Pre-LN Transformer decoder blocks.*

This is **greedy decoding**: always take the highest-scoring continuation. It is reproducible, but it can repeat a pattern. If a model ranks "of" highest after "the" and "the" highest after "of," this loop keeps selecting the same pair for as long as those rankings persist. Random sampling offers other continuations; it cannot compensate for a model that has learned poor probabilities.

Sampling from the full softmax distribution creates a different trade-off. Suppose five tokens receive probabilities $[0.50, 0.30, 0.10, 0.06, 0.04]$. The final three tokens each look unlikely, yet together they are selected 20 percent of the time. If we want only the first two candidates, we must remove the other three and renormalize the retained probabilities to $[0.625, 0.375]$. This motivates filtering, independent of vocabulary size. Whether removing those candidates improves the text depends on the model and task.

Behind these decoding choices sits a question this book has not yet answered. `TinyGPT` outputs a row of V logits at every position. A classifier in Synthesis I had one label per image. What is the label for position 7 of a sentence?

Pause & Predict: The Sampling Temperature Continuum

Let a model's raw logits over three candidate tokens be $z = [4.0, 2.0, 1.0]$. What are the normalized softmax probabilities when temperature $T = 1.0$? What happens to the distribution in the limit as $T \to 0^+$? And as $T \to \infty$? Why does zero temperature yield purely deterministic generation while infinite temperature ignores model knowledge entirely?

14.4 The Idea

14.4.1 The next token is the label

↪ **Jump ahead:** Chapter 19 *Eliminating the redundant $O(S^2)$ key-value recomputation during autoregressive generation.*

In autoregressive language model training (such as GPT and LLaMA), models learn representations via self-supervised teacher forcing, where every sequence position serves simultaneously as context and ground-truth target. For input IDs $[7, 3, 9, 4]$, take targets $[3, 9, 4, 12]$. At position zero, the model sees 7 and predicts 3; at position one, it sees 7 and 3 and predicts 9. The final row sees all four input tokens and predicts 12. The causal mask lets us compute these four predictions in one pass without allowing a row to read its target.

This next-token rule gives a sequence probability by multiplying conditional probabilities:

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1})$$

Here x_t is the token at position t and $P(x_t \mid x_1, \ldots, x_{t-1})$ is the model's probability for it given everything before it. The causal mask from Chapter 12 guarantees that the logits at position t were computed from tokens 1 through t only, so the row of logits at position t is a prediction of x_{t+1}. The training target is therefore the input sequence shifted left by one token. A window of S tokens gives S separate predictions in one

forward pass, and the loss is the cross-entropy of Chapter 4 averaged over all $B \times S$ positions in the batch.

14.4.2 The generation loop

Generation reuses the same forward pass with no labels. Encode the prompt into token IDs. Run the model. Take the last row of logits, the prediction for the token after the prompt. Choose a token from it. Append that token to the sequence and run the model again on the longer sequence. The loop below stops after a fixed number of tokens. Stopping on an end-of-sequence token would require reserving and training such a token, then testing for its ID in the loop; this small corpus does neither. That is the whole model of generation; what it costs per iteration is a question for the code. Notice that at step t, this naive loop recomputes self-attention over all preceding $t - 1$ tokens even though their keys and values never change. In Chapter 19, we will eliminate this $O(S^2)$ prefix recomputation wall by caching previously materialized key and value tensors in DRAM, so each step projects only the new token, although that token still attends over the whole growing prefix.

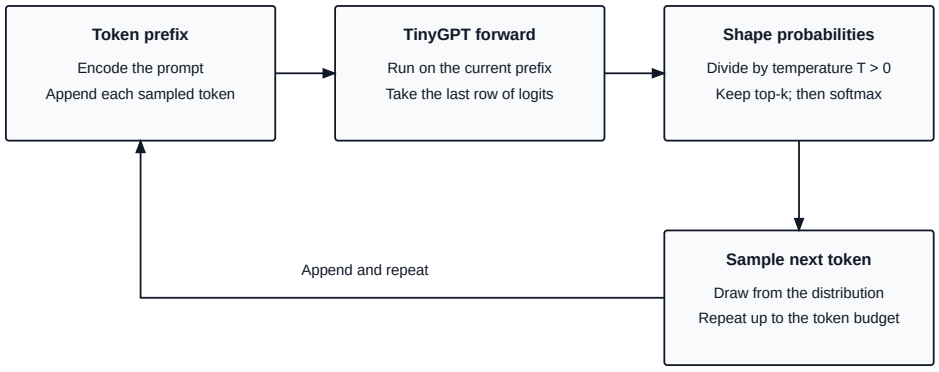


Figure 14.1: The positive-temperature generation loop. Each pass recomputes the retained token prefix, then samples from its last logit row and appends the selected ID.

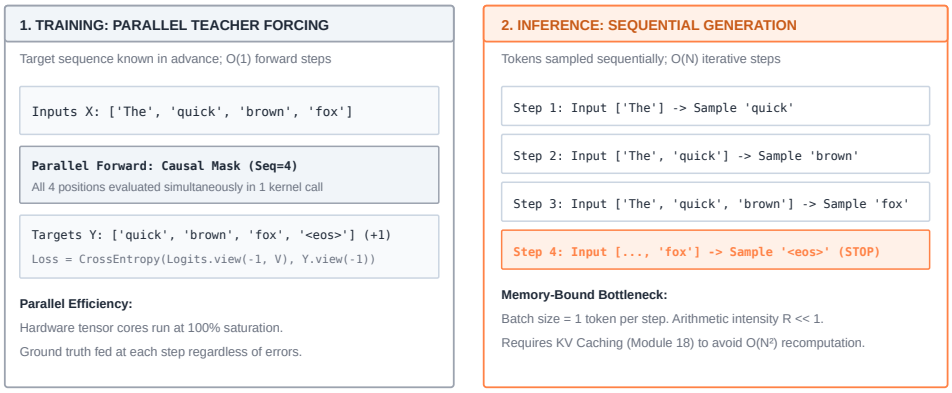
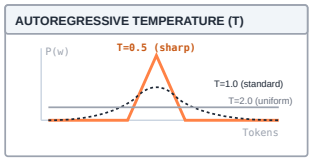


Figure 14.2: **Teacher forcing vs. autoregressive text generation**: In training, teacher forcing presents the full sequence simultaneously, using causal masking so position t predicts target $t+1$ ($O(1)$ parallel passes). In autoregressive inference, generation proceeds iteratively token-by-token, taking the last logit row, sampling with temperature/top-k, and appending to the prompt prefix ($O(T)$ sequential passes).

14.4.3 Temperature



Temperature scaling reshaping logit probabilities ($T \to 0$ vs. $T > 1$).

In modern generative AI inference (from consumer chat endpoints to code completion), **temperature** serves as the primary operational dial governing output entropy and creativity vs. determinism. The choice inside the loop controls how strongly we prefer the largest scores. Let $\mathbf{z} \in \mathbb{R}^V$ be the last row of logits and $T > 0$ a scalar called the **temperature**. Dividing the logits by T before the softmax gives

$$P_i = \frac{\exp(z_i/T)}{\sum_{j=1}^{V} \exp(z_j/T)}$$

where P_i is the probability of token i. At $T = 1$ this is the ordinary softmax and the model's distribution is used as trained. Below 1 the gaps between logits grow, so the largest logit takes more of the mass and the output becomes more predictable. Above 1 the gaps shrink and the mass spreads out.

The limits are worth holding in mind. As $T \to 0$, every gap $z_i - z^*$ between a token and the maximum logit z^* is divided by a vanishing number, so every non-maximum exponent goes to $-\infty$ and the distribution collapses onto the argmax. With a unique maximum, greedy decoding is the zero-temperature limit. Tied maxima share probability in the positive-temperature limit; our explicit zero branch instead selects the first maximum. As $T \to \infty$, every $z_i/T \to 0$, all exponents become equal, and $P_i \to 1/V$, a uniform draw over the vocabulary.

14.4.4 Top-k

Standard in production LLM inference pipelines, **top-k filtering** bounds decoding instability by strictly pruning the improbable long tail of vocabulary candidates before sampling. While temperature reshapes the distribution, it never removes the tail completely; top-k filtering does. Keep the k largest logits and set every other logit to $-\infty$:

$$z_i' = \begin{cases} z_i & \text{if } z_i \text{ is among the } k \text{ largest} \\ -\infty & \text{otherwise} \end{cases}$$

Since $\exp(-\infty) = 0$, the discarded tokens have exactly zero probability after the softmax, and the remaining k probabilities are renormalized to sum to one.[55] With $k = 2$ on the five-token example above, the 20 percent tail vanishes. For distinct logits in exact arithmetic, temperature and top-k commute. Dividing by a positive T does not change which k logits are largest, so applying either first gives the same distribution.

[55] In addition to fixed-count top-k, production systems frequently use **nucleus sampling** (top-p; Holtzman et al., 2020), which dynamically retains the smallest set of tokens whose cumulative probability reaches threshold p (e.g., $p = 0.9$). Unlike top-k, top-p automatically expands its candidate pool when predictions are uncertain (flat distribution) and contracts to a single token when confident (peaked distribution).

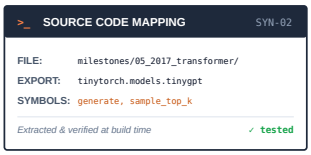
14.5 The Code

14.5.1 Picking one token

Every token a language model writes passes through one decision: given V scores, which token comes next? A chat server makes that decision for every token of every reply, so it belongs in a function of its own, with the knobs as arguments. The Problem showed the choice this function makes explicit: select the maximum, sample from every candidate, or sample from a restricted set.

There are two quieter ways to get it wrong, and both are arithmetic rather than statistics. The first is overflow. Logits come out of a real model in float32, and `exp` overflows float32 past about 88.7. A logit of 10 is ordinary. Divide it by a temperature of 0.1 and it is 100, `exp(100)` is `inf`, and the whole probability vector comes back `nan`. The second is the cost of finding the top k. The obvious way is to sort all V logits and take the last k. A full sort orders every candidate even though the filter only needs the value at the cutoff. That extra ordering work repeats for every generated token.

So the sampler has three jobs. Reshape the distribution with temperature and top-k as the previous section defined them. Find the k-th largest logit without sorting. And take the softmax in a form that cannot overflow, which is the shift-by-the-maximum trick Chapter 4 already built.



Source Code Mapping: Synthesis II combines Modules 09 to 13 to extract the autoregressive token generation loop, temperature scaling, and top-k sampling directly from the working repository.

```python
import numpy as np

def sample_next_token(logits, temperature=1.0, top_k=None, rng=None):
    """Pick one token ID from a vector of logits."""
    rng = rng or np.random.default_rng()
    z = np.asarray(logits, dtype=np.float64)
    if z.ndim != 1 or len(z) == 0 or not np.isfinite(z).all():
        raise ValueError("logits must be a nonempty, finite vector")
    if not np.isfinite(temperature) or temperature < 0:
        raise ValueError("temperature must be finite and nonnegative")
    if top_k is not None and (not isinstance(top_k, (int, np.integer))
        or top_k <= 0):
        raise ValueError("top_k must be a positive integer or None")
    if temperature == 0:
        return int(np.argmax(z))

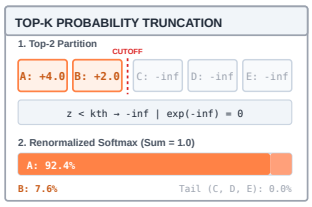
    if top_k is not None and top_k < len(z):
```

```
        kth_largest = np.partition(z, -top_k)[-top_k]    # the k-th
    biggest value
        z = np.where(z < kth_largest, -np.inf, z)         # drop
    everything below it

    with np.errstate(over="ignore", under="ignore"):
        z = (z - np.max(z)) / temperature
        exp_z = np.exp(z)
    probs = exp_z / exp_z.sum()
    return int(rng.choice(len(probs), p=probs))
```

The sampler separates the discrete zero-temperature case from the probability calculation. At zero it returns `argmax` exactly; at a positive temperature it subtracts the maximum before division, so the largest scaled logit is zero. Negative values may underflow after exponentiation, but no positive exponent can overflow. The vector must contain finite scores and the temperature must be finite and nonnegative. These are caller-visible conditions rather than silent substitutions.

Why can an implementation never directly evaluate $\mathbf{z}/T$ when $T = 0$? Mathematically, $T \to 0$ collapses the Boltzmann distribution onto the argmax (greedy decoding). In IEEE 754 floating-point arithmetic, however, dividing a finite logit by `0.0` produces `inf` or `nan`, causing the subsequent softmax subtraction and exponential to fail catastrophically. A robust generation harness must explicitly branch on `temperature == 0` and return `int(np.argmax(logits))` directly, bypassing the floating-point softmax division entirely.



Top-k Truncation: Logits below the rank k threshold are clamped to $-\infty$, zeroing tail exponent probabilities and renormalizing survivors to sum to 1.0.

`np.partition(z, -top_k)[-top_k]` finds the cutoff without fully sorting the vocabulary. `np.where` discards every score below that cutoff before the shifted softmax and random draw. Ties at the cutoff survive, so this implementation may retain more than k tokens; with distinct logits it retains exactly k. The module's own sampler uses the same temperature rule without the top-k filter.

```
def _sample_next_token(self, logits: np.ndarray, temperature: float =
    1.0) -> int:
    return self._sample_fn(logits, temperature, rng=rng)
```

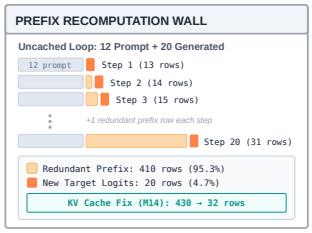
The module helper also validates temperature, returns `argmax` at zero, and centers the logits in float64 before dividing at positive temperatures. Its local `np.errstate` permits very negative scaled values to become $-\infty$, contributing zero probability. It flattens the logits with `reshape(-1)`, so it samples for one sequence at a time. The book helper adds top-k, explicit logit-vector validation, and a caller-supplied random generator.

14.5.2 The loop

The loop is the part a user sees. Type a prompt into a chat window, and what comes back, one token at a time, is this function running on a server. `TinyGPT.forward` returns logits for every position of its input, so the obvious loop is short: run the model on everything so far, pick a token from the last row, append it, repeat. Two versions of that loop look right and are not. One decodes the growing sequence to a string each step and re-encodes it for the next forward pass. BPE merges depend on their neighbors (Chapter 10), so the tokenizer can split the text differently from how the model produced it, and the model ends up conditioning on tokens it never emitted. The other feeds the whole history

without limit. The positional table from Chapter 11 has `max_seq_len` rows, and a context past that capacity is rejected.

The loop is also more expensive than it looks, and keeping the exact arithmetic in front of the reader clarifies the computational cost before Chapter 19 removes much of the repeated work. Every iteration runs the full context through the model to read one row of logits. With a 12-token prompt and 20 new tokens, the contexts are 12, 13, and so on up to 31 tokens long, so 430 rows go through the model to produce 20 tokens, about 21 rows of work per token. After the first iteration, all but one row per iteration recomputes something the previous iteration already computed. We accept that here. The fix is to keep integer IDs rather than strings, crop the context to the model's window, and, in Chapter 19, cache what the past rows produced.



Prefix Recomputation Wall: Without KV caching, generating 20 tokens evaluates 430 rows, where most rows are prefix recomputation.

Run the following book-code blocks in order after the sampler definition; together they import the framework, define the helpers, train a model, and generate text.

```
from tinytorch.core.tensor import Tensor
from tinytorch.core.tokenization import BPETokenizer
from tinytorch.core.transformers import TinyGPT
from tinytorch.core.losses import CrossEntropyLoss
from tinytorch.core.optimizers import AdamW
import tinytorch.core.autograd

def generate_text(model, tokenizer, prompt, max_new_tokens=50,
                  temperature=0.8, top_k=40, max_context=32,
    rng=None):
    """Autoregressively extend a prompt, one token per forward
        pass."""
    token_ids = tokenizer.encode(prompt)
    if not token_ids:
        raise ValueError("prompt must encode to at least one token")
    if not 0 < max_context <= model.max_seq_len:
        raise ValueError("max_context must fit the model's position
            table")

    for _ in range(max_new_tokens):
        context = token_ids[-max_context:]                    # at most
    this many position rows will be read
        logits = model.forward(Tensor(np.array([context])))  # [1, S,
    V]
        last_row = logits.data[0, -1, :]                      #
    prediction for the next token
        token_ids.append(sample_next_token(last_row, temperature,
    top_k, rng))

    return tokenizer.decode(token_ids)
```

Notice what the loop does not do. It never looks at any logit row except the last one, even though the model computed all S rows; that waste is the 430-row number above. It re-encodes nothing; token IDs accumulate as integers and are decoded once at the end. And it crops the context to `max_context`, which must be positive and no larger than the model's `max_seq_len`. Cropping forgets tokens outside the window and restarts the retained context at position zero. That is an explicit bounded-context policy, not the

same computation as attention over an arbitrarily long history. The module's generation method has the same forward/sample/append structure, but rejects requests that would exceed its position table rather than cropping them.

```
def generate(
    self,
    prompt_tokens: Tensor,
    max_new_tokens: int = 50,
    temperature: float = 1.0,
) -> Tensor:
    """Autoregressively generate tokens using the decoupled generate
    ↪ utility."""
    return self._generate_fn(
        self,
        prompt_tokens,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
        max_seq_len=self.max_seq_len,
        rng=rng,
    )
```

It takes and returns a `Tensor` of token IDs, so the caller does the encoding and decoding. It feeds the whole history every step with no crop, so every context actually passed to `forward` must fit `max_seq_len`. The method conservatively requires the prompt plus all requested new tokens to fit `max_seq_len`, including the final sampled token. Before entering the loop it rejects an empty prompt, an invalid token count, or a request beyond that capacity. Each sampling call checks the temperature. Each iteration also concatenates a fresh ID array; caching attention state in Chapter 19 addresses the repeated model computation, a separate cost from managing this ID history. `generate_text` is this loop with the crop, the top-k, and the tokenizer at both ends.

14.5.3 Building the training pairs

Synthesis I trained a classifier, and the habit a classifier teaches is one example per input: a window of tokens in, one label out. Carry that habit over to a language model and it throws away almost everything the forward pass produced. `TinyGPT.forward` returns a row of logits at every one of the S positions, and the causal mask has already made each row an honest prediction of the token after it. Score only the last row, and a 32-token window yields one training signal for a full forward pass. Score all of them, and the same pass yields 32. The transformer forward pass is unchanged; the loss now scores 32 positions instead of one. This is what the old name **teacher forcing**[56] means: during training the model reads the true previous tokens rather than its own guesses, and every position is graded at once.

The label for position t is the token at position $t + 1$, so the target sequence is the input sequence shifted left by one. That is all `make_windows` has to build. Cut the token stream into chunks of $S + 1$, hand out the first S tokens as the input and the last S as the target, and every window carries its own labels.

```
def make_windows(token_ids, seq_len):
```

[56] **Teacher forcing**: training a sequence model by feeding it the true tokens up to position t and grading its prediction for $t + 1$, rather than feeding it its own earlier predictions. With a causal mask, every position in a window can be trained this way in one forward pass.

```
    """Slice one token stream into (input, target) pairs shifted by one."""
    if not isinstance(seq_len, int) or seq_len <= 0:
        raise ValueError("seq_len must be a positive integer")
    if len(token_ids) <= seq_len:
        raise ValueError("stream needs at least seq_len + 1 tokens")
    inputs, targets = [], []
    for start in range(len(token_ids) - seq_len):
        chunk = token_ids[start : start + seq_len + 1]   # seq_len + 1 tokens
        inputs.append(chunk[:-1])                        # first seq_len
        targets.append(chunk[1:])                        # last seq_len
    return np.array(inputs, dtype=np.int64), np.array(targets, dtype=np.int64)
```

Each chunk is one token longer than the window so that the input and target overlap on all but their ends; table 14.4 in the worked trace shows it on six tokens. The windows start at every position, so consecutive windows overlap on all but one token. For a stream of 1,000 tokens and `seq_len=32`, that is nearly a thousand windows and about 62,000 stored integers for a corpus of 1,000, a 62-fold blow-up. On the five sentences below it is nothing. A larger loader can retain one token stream and copy only the windows needed for the current batch. Sampling starting positions then changes the selection policy without requiring every overlapping window to exist in memory at once.

14.5.4 The training run

Now the Part I loop gets a language model in the middle. Every piece is already built: `BPETokenizer` (Chapter 10) turns the corpus into one token stream, `make_windows` turns the stream into pairs, `TinyGPT` (Chapter 13) turns a batch of windows into $[B, S, V]$ logits, and `CrossEntropyLoss` (Chapter 4) and `AdamW` (Chapter 7) do what they did in Synthesis I. This final block uses the imports and helpers defined above.

One seam does not fit. Chapter 4's `CrossEntropyLoss` takes a matrix of $[N, C]$ logits and a vector of N integer labels, and the model hands back a three-dimensional block. The obvious repair is a Python loop over positions that calls the loss once per position and adds the results. With $B = 16$ and $S = 32$ that is 512 loss calls per step, each building its own softmax and its own slice of the autograd graph, for a quantity that is one mean over 512 numbers. The better repair uses the fact from the previous subsection: every position is its own prediction problem, so there is no reason the loss should know which positions came from which window. Reshape $[B, S, V]$ to $[B \cdot S, V]$ and $[B, S]$ to $[B \cdot S]$, and one loss call scores all 512 predictions. TinyTorch's `Tensor.reshape` copies into a new owned buffer, as Chapter 1 explained, but preserves the operation in the autograd graph. The benefit here is one loss call with the right batch interpretation, not a claim of zero data movement.

```
corpus = [
    "TinyTorch is an educational framework for machine learning systems.",
    "A machine learning framework is built from first principles.",
```

```
    "Tensors own NumPy arrays and track their shapes.",
    "Autograd walks the computation graph in reverse to compute gradients.",
    "Attention lets tokens exchange information across a sequence.",
]

tokenizer = BPETokenizer(vocab_size=320)          # the corpus's characters, then merges while pairs remain
tokenizer.train(corpus)
stream = []
for line in corpus:
    stream += tokenizer.encode(line)              # append IDs; word-final tokens carry boundaries

SEQ_LEN, BATCH = 32, 16
inputs, targets = make_windows(stream, SEQ_LEN)

model = TinyGPT(vocab_size=len(tokenizer.vocab), embed_dim=64,
                num_heads=4, num_layers=2, max_seq_len=SEQ_LEN)
loss_fn = CrossEntropyLoss()
opt = AdamW(model.parameters(), lr=3e-3, weight_decay=0.01)

initial_loss = None
for epoch in range(200):
    for start in range(0, len(inputs), BATCH):
        x = Tensor(inputs[start:start + BATCH])
        y = Tensor(targets[start:start + BATCH])
        logits = model.forward(x)                                  # [B, S, V]
        B, S, V = logits.data.shape
        loss = loss_fn(logits.reshape(B * S, V), y.reshape(B * S))  # every position is an example
        if initial_loss is None:
            initial_loss = float(loss.data)
        loss.backward()
        opt.step()
        opt.zero_grad()

print("vocabulary, tokens, windows:", len(tokenizer.vocab), len(stream), len(inputs))
print("first and final loss:", initial_loss, float(loss.data))
print(generate_text(model, tokenizer, "TinyTorch is", max_new_tokens=20,
                    max_context=SEQ_LEN, rng=np.random.default_rng(0)))
```

The one line that is new relative to Synthesis I is the reshape, the same one the attention script makes. A classifier hands the loss B rows and B labels; a language model hands it $B \cdot S$ of each. Nothing else in the loop changes, and that is the point: the engine of Part I did not know it was training a classifier, and it does not need to know it is training a language model now. The important boundary is between the model's sequence-shaped output and the loss's list of independent classification rows.

Two ways to write that reshape look the same and are not. `logits.reshape(B * S, V)` is a `Tensor` method from Chapter 1, so the reshaped logits stay on the autograd graph and the loss reaches the weights. `Tensor(logits.data.reshape(B * S, V))` wraps the raw array in a fresh `Tensor` with no history, so `loss.backward()` finds nothing behind the loss, the optimizer moves nothing, and the loss prints the same number every epoch. Writing the second form leaves the loss flat near its initial value (about $\ln V$ or above), which is the signature failure to diagnose when a language model refuses to learn.

A useful reference loss is $\ln V$, the cross-entropy of a uniform prediction assigning every token probability $1/V$. A randomly initialized model need not predict uniformly, so its first loss need not equal this value. The printed vocabulary size uses the tokenizer's actual vocabulary, not its requested target of 320. The window builder rejects a stream too short to supply even one input/target pair.

This inline experiment verifies software composition. A falling loss shows that gradients reach the model's parameters through the composed forward and backward graph; it does not by itself establish that every module is correct. For book compilation speed, this inline example trains on a minimal five-sentence toy corpus, which naturally yields memorized continuations rather than general language generation. The companion CLI runs a larger workload: `tito milestone run 05` trains a character-level TinyGPT on the first 35,000 characters of TinyShakespeare for 12 epochs. It holds out the 4,000 characters that follow, never training on them. Before training, it checks that your `CrossEntropyLoss` matches a NumPy computation on one batch and that a repeated token gets different predictions at different positions. Every loss the gate reads is computed in NumPy from the model's logits, not taken from your loss function. The gate requires a training loss below 1.20, measured after training, a best held-out loss at least 0.35 below a bigram model counted from the training text, and a causality probe: changing the tokens after a cut must leave every prediction before the cut unchanged. A model without working attention stalls near the bigram loss, and a model without the causal mask reads the next token, drives its training loss toward zero, and fails the probe. At these settings the samples show character statistics and Shakespeare-like formatting, not coherent English; more data, capacity, and steps are what move a model toward readable text. There is no document-separator token in the toy corpus: concatenating sentences directly teaches a transition from the end of one sentence to the start of the next. For a production experiment, reserve separate text for evaluation and decide explicitly how document boundaries should be represented.

14.5.5 From Shakespeare to TinyCopilot: Generative Code and Compiler Gating

Training an autoregressive model on natural prose like Shakespeare demonstrates statistical language structure: meter, vocabulary, and dialogue cadence. However, natural language is forgiving. A typo, an extra space, or an unexpected comma rarely prevents human comprehension.

Code generation represents a critical evolution in sequence modeling. Programming languages possess strict context-free grammars. In Python, indentation defines block scoping, and keywords such as `def`, `class`, `return`, and `if` demand exact downstream syntax. If an attention head fails to balance an opening delimiter or track variable scope across positions, the generated code fails to parse.

This rigidity introduces an objective evaluation tool that natural language lacks: the **Abstract Syntax Tree (AST) compiler gate**. Rather than relying solely on subjective inspection or character cross-entropy loss, generated completions can be passed directly

to Python's built-in `ast.parse()`. A code completion model is evaluated on its **AST Validity Rate (%)**: does the model's completion parse as Python?

Students explore this directly in Milestone 05 Part 3 with **TinyCopilot** (`tinytorch/milestones/05_201` runnable via `tito milestone run 05 --part 3`). The model trains on curated idiomatic Python algorithms (`TinyPy`) and completes function definitions such as `def square(x):`, `def clamp(x, lo, hi):`, and `class Counter:`, names that never appear in its training text, validating each completion against the AST parser exactly as generated, except a final line cut off by the token budget. The score uses the greedy completion for each prompt, so it is a property of the trained network rather than of a random draw; the sampled completion is printed for display only. Part 3 passes when training loss falls below 1.0, the causality probe holds, and at least one of the seven unseen prompts parses. Finally, Part 4 (`04_tinygpt_chat.py`, runnable via `tito milestone run 05 --part 4`) tests conversational question answering on TinyTorch concepts and introduces the **Overfitting Detective**, examining how an overparameterized transformer easily achieves near-zero training loss on a compact corpus while diverging on held-out validation queries.

14.6 A Worked Trace

Take a five-token vocabulary with raw logits $\mathbf{z} = [4.0, 2.0, 1.0, 0.0, -2.0]$ and push it through the sampler at several settings. Each column of table 14.3 is one call to `sample_next_token` stopped just before the random draw.

Table 14.3: Probabilities the sampler draws from for one logit vector under four settings.

Token	Logit z_i	$T = 1.0$	$T = 0.5$	$T = 2.0$	Top-$k = 2$, $T = 0.8$
A ("the")	$+4.0$	82.92%	97.93%	56.30%	92.41%
B ("a")	$+2.0$	11.22%	1.79%	20.71%	7.59%
C ("this")	$+1.0$	4.13%	0.24%	12.56%	0 (filtered)
D ("of")	0.0	1.52%	0.03%	7.62%	0 (filtered)
E ("cat")	-2.0	0.21%	0.00%	2.80%	0 (filtered)
Entropy (bits)		0.88	0.16	1.74	0.39

Reading across a row shows what each knob does. Cooling to $T = 0.5$ doubles every logit gap, and token A goes from 83 percent to 98 percent. Heating to $T = 2.0$ halves the gaps and the tail token E, which had a 1-in-500 chance, now has a 1-in-36 chance. Top-$k = 2$ sends C, D, and E to exactly zero and splits their mass between A and B in proportion to $\exp(z_A/T)$ and $\exp(z_B/T)$. The entropy row summarizes how spread out each distribution is: $H = -\sum_i p_i \log_2 p_i$, with zero-probability terms contributing zero. A deterministic choice has zero entropy. At $T = 0.5$ the 0.16 bits are close to deterministic; at $T = 2.0$ the 1.74 bits approach the $\log_2 5 \approx 2.32$ bits of a uniform draw.

The following calculation in the listing below reconstructs the probability columns before the random draw. In line 4, `z = np.array([4.0, 2.0, 1.0, 0.0, -2.0])` defines the raw logit vector. In `shaped_probs` (lines 6 to 10), lines 7 to 8 clamp sub-top-k

logits to $-\infty$ via `np.partition`, and lines 9 to 10 perform stabilized softmax division scaled by `temperature`. The loop in lines 12 to 16 prints formatted probability percentages matching table 14.3. Lines 18 to 19 confirm via `np.isclose` that cooled sampling ($T = 0.5$) concentrates 97.93% mass on token A, while top-$k = 2, T = 0.8$ concentrates 92.41% on token A.

```
import numpy as np
np.set_printoptions(suppress=True)

z = np.array([4.0, 2.0, 1.0, 0.0, -2.0])

def shaped_probs(z, temperature=1.0, top_k=None):
    if top_k is not None:
        z = np.where(z < np.partition(z, -top_k)[-top_k], -np.inf, z)
    e = np.exp((z - np.max(z)) / temperature)
    return e / e.sum()

for label, p in [("T=1.0", shaped_probs(z, 1.0)),
                 ("T=0.5", shaped_probs(z, 0.5)),
                 ("T=2.0", shaped_probs(z, 2.0)),
                 ("top-k=2, T=0.8", shaped_probs(z, 0.8, top_k=2))]:
    print(f"{label:16s}", np.round(p * 100, 2))

assert np.isclose(shaped_probs(z, 0.5)[0] * 100, 97.93, atol=0.01)
assert np.isclose(shaped_probs(z, 0.8, top_k=2)[0] * 100, 92.41,
    atol=0.01)
```

The training side has a trace too, and it is even smaller. Slice the token stream $[7, 3, 9, 4, 12, 5]$ with `make_windows(stream, seq_len=4)`:

Table 14.4: Input and target windows for a six-token stream with `seq_len=4`.

Window	Input	Target
0	$[7, 3, 9, 4]$	$[3, 9, 4, 12]$
1	$[3, 9, 4, 12]$	$[9, 4, 12, 5]$

Position 2 of window 0 holds token 9, and its target is 4, the token that follows 9 in the stream. The causal mask ensures that the logits at that position were computed from $[7, 3, 9]$ alone, so the model is being asked the only fair question: given these three tokens, what comes next?

14.7 From TinyTorch to Production

The loop in this synthesis chapter recomputes the entire retained context at each step. A server can instead retain attention keys and values from earlier steps, the mechanism developed in Chapter 19. Only after introducing that cache can later model calls process a single new token while still attending to the past. The comparison in table 14.5 describes this cached execution, not `generate_text` as written. It counts one multiply and one

add as two floating-point operations (FLOPs), and assumes each matrix weight is read once per forward call.

During **prefill**, the prompt's S tokens go through the model in one pass, and every weight matrix is multiplied against S rows at once. Each weight loaded from memory participates in $2S$ floating-point operations, so there is more opportunity to reuse a loaded weight. During cached **decode**, the projections for new content process a single row while attention reads stored keys and values. At batch size one, each matrix weight then performs little arithmetic per use. The current loop processes many rows again, so its arithmetic and memory costs differ. Whether a measured run is limited by bandwidth or arithmetic also depends on sequence length, batch size, and implementation; Chapter 15 introduces measurement before Chapter 19 changes the execution.

Table 14.5: The two phases of generation and why they stress different parts of the hardware.

	Prefill (the prompt)	Decode (each new token)
Tokens per forward pass	All S prompt tokens at once	One
Shape of each weight multiply	$[S \times D] \cdot [D \times D]$	$[1 \times D] \cdot [D \times D]$
Arithmetic per weight loaded	$2S$ FLOPs per weight	2 FLOPs per weight
Potential pressure	Arithmetic across prompt rows	Weight and cache reads for each new row

Consider a hypothetical cached decoder with 7 billion parameters stored at 2 bytes each. Suppose every generated token requires reading those weights once through a memory interface that can deliver 2 TB/s, using decimal units.

- Weight bytes per token: $7 \times 10^9 \times 2 = 14$ GB.
- Minimum time for those reads: $14/2000 = 0.007$ seconds, or 7 ms.
- Corresponding ceiling from weight reads alone: $1/0.007 \approx 143$ tokens/s.

This is a conditional estimate, not a TinyTorch measurement. It excludes attention-cache traffic, intermediate arrays, scheduling, and arithmetic. Batching requests can reuse a weight read across several rows; retaining weights in faster memory changes the traffic assumption. Chapter 15 develops tools for checking such assumptions against a measured workload.

In industrial serving engines such as vLLM and TensorRT-LLM, memory bandwidth bottlenecks during autoregressive decoding are mitigated through two primary strategies:

1. **Continuous Batching**: Decoupling sequence arrivals and dynamically packing tokens from independent requests into unified matrix multiplications, amortizing weight read bandwidth across concurrent users.
2. **Speculative Decoding**: Running a lightweight draft model (e.g., 1B parameters) ahead to propose candidate sequences of K tokens, followed by a single forward pass of the target model (e.g., 70B parameters) to verify all K candidates in parallel. Because the target model evaluates K tokens concurrently, its execution shifts toward the compute-bound regime of prompt prefill, achieving $2\times$ to $3\times$ decode speedups without altering output distributions.

The useful distinction is between work we repeat and work every new token still requires. A cache can reuse earlier attention keys and values, but the new token still needs its own projections, attention calculation, and vocabulary scores. Cropping changes which history is available; caching is intended to preserve the result while reusing prior state. We keep the uncached loop here as the reference against which Chapter 19 can check that claim.

A serving system often reports the delay until the first output token separately from the delay between later tokens. The first includes processing the prompt; later steps extend an existing sequence. Neither delay identifies a bottleneck by itself. To decide whether fewer bytes, less arithmetic, or a different execution schedule will help, we must first measure the computation we actually run. That is Chapter 15's task.

14.8 Summary

This synthesis chapter ran both Part II architectures against a task and a baseline, then turned the `TinyGPT` of Chapter 13 into a system that trains on plain text and writes text back, using the same next-token objective that generative pretraining uses. On 8×8 digits, convolution came close to a dense network with a third of the parameters but trailed it at every budget, and cost about 87 times the training time because its loops run in Python; its advantage grows with input size. Attention learned fixed permutations in two epochs and a content-dependent choice between them in four, which is the routing a fixed mixing of positions cannot do. Generation is the case with no single right answer.

For text, training needs no labels beyond the text itself, because the target at every position is the next token, and one forward pass over a window of S tokens yields S training examples that the causal mask keeps honest. Generation is a loop that runs the model, reads the last row of logits, samples one token, and appends it. Inside that loop, temperature divides the logits to sharpen or flatten the distribution, and top-k sets the tail to exactly zero; greedy decoding is the zero-temperature limit and can produce repetitive continuations. The invariant of this implementation is that each sampled token is appended as an ID and every subsequent forward pass recomputes the retained context. Its cost grows with the number of context rows; cached one-row decoding is a later implementation. Chapter 15 measures where the time goes before choosing an optimization.

14.9 Further Reading

- **LeCun et al.** (1998). "Gradient-based learning applied to document recognition." *Proceedings of the IEEE*. The convolutional network the first part of this synthesis scales down.
- **Vaswani et al.** (2017). "Attention is all you need." *NeurIPS*. The architecture the attention tasks exercise.
- **Radford et al.** (2019). "Language models are unsupervised multitask learners." *OpenAI Technical Report*. GPT-2 and generative pretraining.
- **Brown et al.** (2020). "Language models are few-shot learners." *NeurIPS*. GPT-3 and emergent capabilities from scale.

- **Holtzman et al.** (2020). "The curious case of neural text degeneration." *ICLR*. Why greedy decoding repeats itself and how sampling choices change generated text.

PART III

Systems & Acceleration

🔥 Chapter 15

Profiling: The Roofline Model

In machine learning systems, optimizing without measurement is guesswork. As models scale from millions to billions of parameters, understanding where execution time and memory are spent is essential. A practitioner must be able to inspect a model architecture and quantitatively predict its resource requirements: how many FLOPs does a forward pass consume, how much memory is dedicated to parameters vs. activations, and what hardware limits govern execution speed?

In PyTorch, runtime execution and operator bottlenecks are inspected with the built-in profiler:

```
import torch
from torch.profiler import profile, ProfilerActivity

x = torch.randn(1024, 1024)

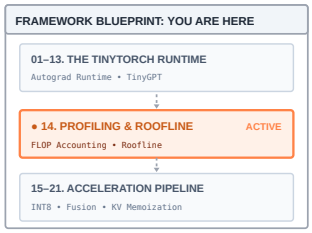
# Profile operator execution time and tensor allocations
with profile(activities=[ProfilerActivity.CPU], record_shapes=True) as
    prof:
    y = torch.relu(x @ x)

print(prof.key_averages().table(sort_by="cpu_time_total",
    row_limit=5))
```

Underneath that profiling report, the systems profiler tracks the fundamental physical reality of computation:

1. **Operator runtime attribution:** The profiler hooks into dispatcher call-sites to measure the wall-clock execution time and kernel launch overheads of individual layers and functions.
2. **Memory traffic accounting:** The engine tracks buffer allocations, peak memory residency, and memory deallocations, distinguishing persistent parameter storage from transient activation buffers.
3. **Operational intensity and hardware bounds:** By comparing floating-point arithmetic (FLOPs) against memory moved across the bus (bytes), a practitioner can place operations on the **Roofline Model** and ask whether an operator is **memory-bandwidth bound** or **compute-bound**; `torch.profiler` supplies timings, shapes, and optional FLOP estimates, not a bus-traffic measurement.

This chapter establishes TinyTorch's profiling harness, providing the analytical and empirical diagnostic foundations that guide all performance optimizations in Part III. Before constructing these profiling tools, the first step is to examine why naive arithmetic counting fails to predict execution time.



Framework Blueprint: Module 14 grounds execution in physics, computing operational intensity and plotting operations on the Roofline model.

↩ **Jump back:** Chapter 13
Accounting for TinyGPT's parameter budget across multi-head attention and feed-forward sub-layers.

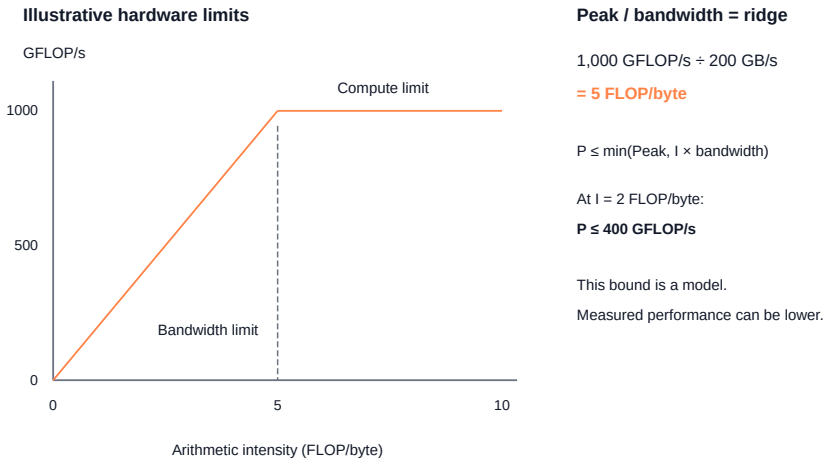


Figure 15.1: The roofline model. Performance rises with arithmetic intensity along the bandwidth ceiling until it reaches the compute ceiling; the corner where they meet is the ridge point.

15.1 The Problem

The natural way to estimate how long a computation takes is to count its arithmetic. A Linear layer with n inputs and m outputs, applied to R rows, does $2Rnm$ floating-point operations, or FLOPs, counting one for each multiply and one for each add.[57] Doubling the rows doubles the FLOPs, which suggests doubling the time.

Try it. Take one weight matrix the size of a real model's layer and push two rows through it, then sixteen. The second call does eight times the arithmetic.

[57] In computing and machine learning systems literature, one **Multiply-Accumulate** (MAC) or Fused Multiply-Add (FMA, $a \cdot b + c$) counts as two floating-point operations: one multiplication and one addition (1 MAC = 2 FLOPs). Hardware accelerators execute FMAs in a single clock cycle, so hardware peak FLOPS ratings are calculated as $2 \times$ FMA units $\times$ frequency.

```python
import time
import numpy as np

rng = np.random.default_rng(0)
# One Linear weight occupies 67,108,864 bytes.
W = rng.standard_normal((4096, 4096), dtype=np.float32)

def time_ms(rows):
    x = rng.standard_normal((rows, 4096), dtype=np.float32)
    for _ in range(5):                          # warmup, explained below
        x @ W
    t0 = time.perf_counter()
    for _ in range(20):
        x @ W
    return (time.perf_counter() - t0) / 20 * 1000.0

print(f"2 rows:  {time_ms(2):.2f} ms")
print(f"16 rows: {time_ms(16):.2f} ms")
```

This experiment deliberately has no expected pair of timings. The second call does eight times as much arithmetic, but it need not take eight times as long. Both calls reuse the same weight matrix, and the numerical library can choose different matrix-multiply routines for different shapes. Record the measurements on the machine that will run the workload.

A nearly flat pair would be evidence against predicting time from FLOPs alone. One possible explanation is that fetching the weights costs more than the arithmetic, allowing additional rows to reuse data already fetched. Another is a change in the library's execution path. The timer alone does not distinguish them. Counting both operations and bytes gives a model against which to test those explanations.

Pause & Predict: Arithmetic Intensity & The Memory Wall

Suppose an accelerator has a peak compute throughput of $P_{\text{peak}} = 100$ TFLOP/s and a memory bandwidth of $B_{\text{peak}} = 1{,}000$ GB/s. What is the hardware ridge point I_{ridge}? If an autoregressive token decode step has an arithmetic intensity of $I = 0.5$ FLOP/B, what is the maximum achievable compute throughput? What percentage of the processor's peak compute capacity sits idle waiting for memory transfers?

15.2 The Idea

15.2.1 Two costs, two rates

↪ **Jump ahead:** Chapter 16 *Using INT8 quantization to break memory-bandwidth bottlenecks identified by the Roofline model.*

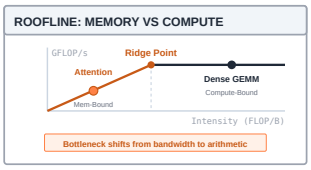
In production runtimes and hardware profilers like NVIDIA Nsight and PyTorch Profiler, diagnosing why an execution kernel is slow begins by decomposing execution into two independent physical limits: processor math throughput and bus data transfer. Because modern accelerators execute math orders of magnitude faster than off-chip DRAM can deliver data, an operator's execution time is governed by which resource starves first. Every operation a model performs has two costs. It does some number of floating-point operations, call it F. It also moves some number of bytes, Q, between memory and the processor: weights read in, activations read in, results written out. The machine, in turn, has two rates. It can perform at most P_{peak} FLOPs per second, and it can move at most B_{peak} bytes per second between memory and the processor, its memory bandwidth.

The arithmetic alone needs at least F/P_{peak} seconds. Moving the bytes needs at least Q/B_{peak} seconds. Both must finish, so elapsed time cannot be less than the larger of those lower bounds. The idealized model allows arithmetic and transfers to overlap; incomplete overlap or extra work makes execution slower:

$$T \geq \max\left(\frac{F}{P_{\text{peak}}}, \frac{Q}{B_{\text{peak}}}\right)$$

For a large weight matrix and a small number of rows, the weight reads can dominate Q. Increasing the rows then raises the arithmetic term much faster than the transfer term. If transfer remains the larger term, the model predicts little change in the lower bound. This is a conditional explanation for flat timings, not proof that the experiment reached the bound.

15.2.2 Arithmetic intensity and the roofline



The Roofline model: operational intensity separates the memory-bound bandwidth slope from the compute-bound peak ceiling.

In production performance analysis, tools like NVIDIA Nsight Compute and Intel Advisor evaluate kernel efficiency by plotting operations onto the **Roofline model**. By comparing floating-point work against the memory traffic required by an operator, this diagnostic framework reveals whether execution is bottlenecked by the memory bus or by compute pipelines. Their ratio summarizes the balance between these costs. **Arithmetic intensity** is the FLOPs an operation performs per byte it moves:

$$I = \frac{F}{Q} \quad \text{(FLOPs per byte)}$$

An operation with high intensity does a lot of work on each byte it fetches; an operation with low intensity fetches a byte, touches it once or twice, and throws it away. Rewrite the time bound as achieved performance, F/T, and it becomes the **roofline**:

$$\frac{F}{T} \le \min\left(P_{\text{peak}},\ I \times B_{\text{peak}}\right)$$

Plotted against I, as in figure 15.1, the right-hand side is a slanted line that rises with intensity until it hits a flat ceiling at P_{peak}. The corner is the **ridge point**, the intensity at which the two ceilings meet:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{peak}}}$$

Left of the ridge, the model's lower ceiling comes from memory bandwidth; this is its **memory-bound** region. Reducing transferred bytes or increasing available bandwidth raises that ceiling. Right of the ridge, peak arithmetic throughput supplies the lower ceiling; this is its **compute-bound** region. Fewer FLOPs or faster arithmetic can then reduce the time bound. Real operations can run below both ceilings because of costs such as Python calls, allocation, or incomplete use of the processor.

The illustrative machine in figure 15.1 has a peak of 1,000 GFLOP/s and bandwidth of 200 GB/s. Here a GFLOP is a billion floating-point operations, and a GB is a billion bytes. Its ridge is $1000/200 = 5$ FLOPs per byte. An operation at 2 FLOPs per byte has a ceiling of $2 \times 200 = 400$ GFLOP/s. These are chosen numbers that make the relationship visible, not specifications for the reader's computer.[58]

15.2.3 Why a Linear layer's intensity grows with its row count

In modern LLM serving engines such as vLLM and TensorRT-LLM, batching requests is the primary mechanism used to transform memory-bandwidth-bound matrix-vector operations into compute-bound matrix multiplications. By sharing the cost of streaming massive weight matrices from DRAM across multiple input tokens simultaneously, serving systems dramatically improve hardware utilization. Apply the definition to the layer that dominates every model in this book. A Linear layer with n inputs, m outputs, and R rows does $F = 2Rnm$ FLOPs. It reads $4nm$ bytes of weights (four per float32), reads $4Rn$ bytes of input, and writes $4Rm$ bytes of output. When R is small compared to n and m, the weights dominate the bytes:

$$I \approx \frac{2Rnm}{4nm} = \frac{R}{2}$$

[58] Real hardware accelerators exhibit dramatically different ridge points: an illustrative laptop CPU with 1.5 TFLOP/s of FP32 arithmetic and 400 GB/s of memory bandwidth, roughly the class of an Apple M3 Max, has $I_{\text{ridge}} \approx 3.75$ FLOP/B (the same chip's GPU, with several times the arithmetic, has a far higher ridge), an Intel Xeon 8480+ has $I_{\text{ridge}} \approx 25$ FLOP/B (7.6 TFLOP/s/307 GB/s), and an NVIDIA H100 SXM5 GPU has $I_{\text{ridge}} \approx 295$ FLOP/B (989 TFLOP/s FP16/3,350 GB/s HBM3). Higher ridge points demand higher batch sizes to escape the memory-bound regime.

Under this traffic model, a Linear layer's intensity is roughly half its row count. The approximation assumes float32 weights, one transfer of each weight, and small input and output arrays relative to the weight matrix. One row gives about 0.5 FLOPs per byte; sixty-four rows give about 32. Rows can come from several examples in a batch or several positions in a sequence. As established in Chapter 1, memory layout and contiguous strides govern transfer efficiency. At $I \approx 0.5$ FLOP/B, the model places the operation far left of any realistic ridge, so its ceiling comes from memory bandwidth. Chapter 18 examines how fewer intermediate arrays could reduce traffic, and Chapter 19 shows how caching removes repeated projections while attention still reads the growing prefix.

This explains why the generation algorithm matters. Chapter 13's ordinary `GPT.generate` recomputes the supplied prefix, so a prefix of length S sends S rows through each projection. Processing only the newest position requires the cache introduced in Chapter 19, and even then attention must read earlier keys and values. An `S = 1` forward pass here is a one-token input, not a cached decoding step with a long history.

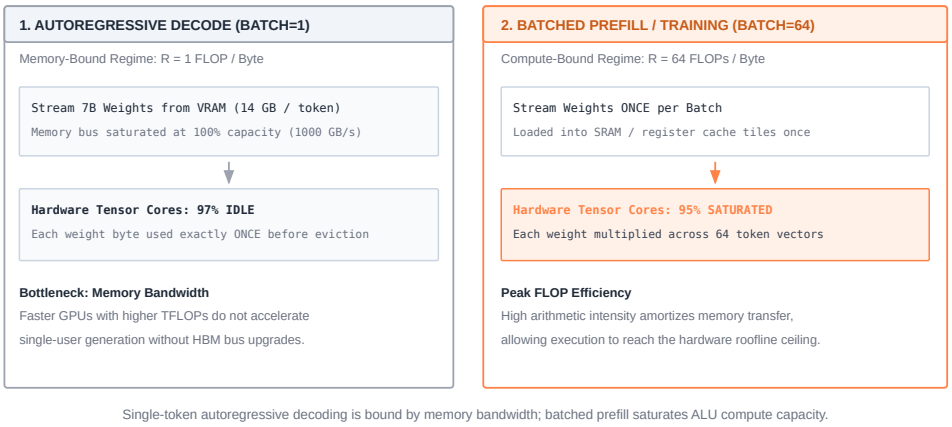


Figure 15.2: **Weight streaming and operational intensity across batch sizes**: At batch size $B = 1$ (single-token inference), weights are streamed from DRAM over the memory bus and touched only once, resulting in low arithmetic intensity ($I \approx 0.5$ FLOPs/byte) and severe memory bandwidth starvation. At batch size $B = 64$, each loaded weight can serve all 64 rows, raising modeled arithmetic intensity to about 32 FLOPs/byte, which lies right of the ridge on many CPUs but left of it on accelerators with higher ridges.

Consider the reference `TinyGPT` configuration ($d_{\text{model}} = 128$, $n_{\text{layers}} = 4$, context S, vocabulary $V = 1000$). Each Transformer block contains $4d^2$ projection parameters in attention (W_Q, W_K, W_V, W_O) and $8d^2$ parameters in the two-layer MLP (W_1, W_2).

Per block, the projection weights total $12d^2 = 196{,}608$ floats, or about 786 KB in FP32; across $L = 4$ blocks, $12Ld^2 = 786{,}432$ floats, or about 3.15 MB. Per block, these weights dominate activation traffic when batch size is 1:

- **Autoregressive Decode** ($S = 1$): A single new token processes each weight matrix once. Per block, arithmetic is $2 \times 1 \times 196,608 \approx 3.93 \times 10^5$ FLOPs, and memory traffic is dominated by reading that block's 786 KB of weights.

$$I_{\text{decode}} \approx \frac{3.93 \times 10^5 \text{ FLOPs}}{7.86 \times 10^5 \text{ Bytes}} \approx 0.50 \text{ FLOP/B}$$

At 0.50 FLOP/B, execution sits deeply in the memory-bound slope of the roofline.

- **Prompt Prefill** ($S = 16$): The entire prompt of 16 tokens passes through simultaneously. Per block, arithmetic scales linearly to $2 \times 16 \times 196,608 \approx 6.29 \times 10^6$ FLOPs, while that block's weight reads remain 786 KB:

$$I_{\text{prefill}} \approx \frac{6.29 \times 10^6 \text{ FLOPs}}{7.86 \times 10^5 \text{ Bytes}} \approx 8.0 \text{ FLOP/B}$$

Arithmetic intensity increases $16\times$, pushing execution toward or past the ridge point of typical desktop processors.

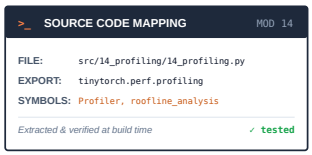
15.2.4 Measuring instead of assuming

While static analytical models predict execution bounds under idealized assumptions, production profilers in PyTorch and JAX rely on empirical execution traces to capture hardware realities like cache hierarchy effects and runtime launch overheads. Measuring wall-clock latency alongside memory allocations is critical to verify whether an operator actually achieves the bandwidth or compute ceilings predicted by theory. The roofline needs an operation count F, a traffic estimate Q, and applicable machine ceilings. Layer shapes supply the first. For an isolated matrix multiply, counting each input and weight read once and each output write once gives a useful traffic model. For a whole model, parameter storage plus boundary activations does not suffice: intermediate arrays add transfers, embedding lookups touch selected rows, and caches, smaller and faster storage levels that retain recently used data, may serve repeated reads without accessing main memory.

A timer supplies elapsed time T, from which F/T is an achieved compute rate. Dividing the modeled Q by T gives a rate under that traffic assumption. A row-count sweep can reveal patterns consistent with bandwidth or compute limits, but it does not automatically measure either hardware peak. The worked trace keeps the observed times and the assumed roofline separate.

15.3 The Code

Begin at `Profiler.profile_forward_pass(model, input_tensor)`, the entry point that joins the counters to execution. It reads the parameter count and a per-sample FLOP estimate, probes memory using a separate dummy input, then times repeated calls on the supplied input. The returned dictionary mixes those sources, so reading it requires keeping their meanings separate. The following sections trace each helper before returning to the entry point to verify how it combines a per-sample count with a whole-batch time.



Source Code Mapping: Module 14 extracts analytical FLOP accounting, parameter memory quantification, and roofline profiling directly from the working repository.

15.3.1 Counting parameters and bytes

In deployment frameworks and model registries like Hugging Face and PyTorch, static parameter accounting is the first diagnostic performed to verify that a network's weights will fit within an accelerator's VRAM budget. Inspecting tensor metadata across the module hierarchy establishes this static resident memory footprint before allocating transient activation buffers or key-value caches. Parameter storage determines whether a model can fit in memory before any input is processed. For a `Linear(3, 2)` like the last

layer of Chapter 3's parameter ledger, eight float32 parameters occupy $8 \times 4 = 32$ bytes. Counting the arrays also avoids omissions when a model contains biases, normalization parameters, or lookup tables in addition to weight matrices.

The architecture provides a useful independent check, but the framework can count the arrays it already owns. Each parameter is a `Tensor` around a NumPy array, and `.data.size` gives its element count. The model's `parameters()` method collects those tensors, so the main path is a sum over that list.

```
def count_parameters(self, model) -> int:
    """
    Count total trainable parameters in a model.
    """
    if hasattr(model, 'parameters'):
        return sum(p.data.size for p in model.parameters())
    if hasattr(model, 'weight'):
        return _count_layer_parameters(model)
    return 0
```

A model with `parameters()` takes the first branch, which sums the sizes of the tensors that method returns. This includes both `Sequential` and `GPT`; their own parameter methods collect the child layers. Without that method, an object with a `weight` takes the fallback helper, which counts its weight and optional bias. An object exposing neither contributes zero. The profiler relies on the model to provide the parameter list; it does not discover arbitrary arrays hidden inside a custom object.

```
def _calculate_parameter_memory(self, model) -> float:
    """
    Calculate memory used by model parameters in megabytes.
    """
    param_count = self.count_parameters(model)
    return (param_count * BYTES_PER_FLOAT32) / MB_TO_BYTES
```

The byte calculation assumes float32 storage and multiplies the element count by `BYTES_PER_FLOAT32`, which is four. That remains the physical dtype of TinyTorch's simulated quantized tensors in Chapter 16; their separate analytical INT8 report describes a proposed packed representation. A container that really stores several dtypes would instead sum each array's `nbytes`. The module divides by `MB_TO_BYTES = 1_000_000`, so its `memory_mb` fields use decimal megabytes: TinyGPT's 4,328,448 parameter bytes print as 4.33. Parameter storage can approximate traffic for a dense layer whose weights must all be fetched, but it does not count how many times a pass reads each byte or whether a cache serves those reads.

15.3.2 Counting FLOPs

Production profiling utilities like DeepSpeed's FLOP profiler, PyTorch's `FlopCounterMode`, and `fvcore` inspect computational graph shapes to quantify the algorithmic work required by a forward pass independently of hardware runtime. This symbolic accounting enables engineers to calculate operational intensity and estimate compute costs across varying batch sizes and context lengths before executing expensive training or inference runs. Operation counts answer a different question from parameter counts. That `Linear(3, 2)` has six weights whether the input contains one row or four, but its

matrix-product work grows from twelve FLOPs to forty-eight. A count computed before execution makes that distinction visible without paying for a timed forward pass.

Counting each multiplication as it executes would add work to the operation under study. Instead, the profiler reads shapes. A Linear matrix product costs $2 \times \text{rows} \times \text{in} \times \text{out}$ under the multiply-plus-add convention. This is a conventional estimate: it omits the bias addition and treats every dot-product term as one multiplication plus one addition. In a transformer block, the two attention matrix products add $4BS^2D$ FLOPs for batch B, sequence length S, and embedding width D. The GPT counter omits embeddings, normalization, activations, softmax, and residual additions; these can still consume time and memory.

The module counts per sample, with the outer batch dimension excluded. For Linear input `(B, n)`, one sample is one row. For `(B, S, n)`, one sample contains S rows, so `_count_linear_flops` includes that sequence-length factor. Only the outer batch factor remains for the caller.

```
def _count_linear_flops(model, input_shape: Tuple[int, ...]) -> int:
    """
    Count FLOPs for a Linear layer forward pass.
    """
    in_features = input_shape[-1]
    out_features = model.weight.shape[1] if hasattr(model, 'weight')
    ↪ else 1
    positions = int(np.prod(input_shape[1:-1])) if len(input_shape) >
    ↪ 2 else 1
    return positions * in_features * out_features * 2
```

The input width comes from the input shape and the output width from the weight, which is stored as `[in, out]` since Chapter 3. `count_flops` dispatches on what it is handed.

```
def count_flops(self, model, input_shape: Tuple[int, ...]) -> int:
    """
    Count per-sample FLOPs for one forward pass.
    """
    model_name = model.__class__.__name__

    if model_name == 'Linear':
        return _count_linear_flops(model, input_shape)
    elif model_name == 'Conv2d':
        return _count_conv_flops(model, input_shape)
    elif model_name == 'Sequential' or hasattr(model, 'layers'):
        return self._count_sequential_flops(model, input_shape)
    elif hasattr(model, 'blocks') and hasattr(model, 'embed_dim'):
        return self._count_transformer_flops(model, input_shape)
    else:
        sample_shape = input_shape[1:] if len(input_shape) > 1 else
    ↪ input_shape
        return int(np.prod(sample_shape))
```

A flat `Sequential` sums its layers in order and propagates the shape after each Linear, convolution, or pooling layer. The shape update matters: a convolution after pooling must be counted on the smaller image. Pooling and other fallback layers receive an

estimate of one operation per input element, not an exact operation count. A GPT is recognized by `blocks` and `embed_dim`, and uses a separate sequence-level rule.

```
def _count_transformer_flops(self, model, input_shape: Tuple[int, ...])
    -> int:
    """
    Count FLOPs for one sequence through a GPT-style model (Module
    13).
    """
    seq_len = input_shape[-1]
    embed_dim = model.embed_dim
    flops = 0
    for block in model.blocks:
        attn, mlp = block.attention, block.mlp
        for layer in (attn.q_proj, attn.k_proj, attn.v_proj,
            attn.out_proj, mlp.linear1, mlp.linear2):
            flops += seq_len * _count_linear_flops(layer, (1,
    layer.in_features))
        flops += 4 * seq_len * seq_len * embed_dim
    flops += seq_len * _count_linear_flops(model.lm_head, (1,
    embed_dim))
    return flops
```

Each block holds six Linear layers: four attention projections and two MLP projections. The helper obtains a one-row count for each and multiplies by S, then adds $4S^2D$ for attention. The output head contributes one more Linear count. Four blocks therefore contain twenty-four projections plus the head, and the worked trace checks their sum.

Omitting an operation from this FLOP estimate does not make it free. An activation that reads one array and produces another still allocates and moves data. TinyTorch also implements compound activations with several NumPy operations. This is why a shape counter and a timer complement each other, and why later fusion experiments must measure the whole changed computation.

15.3.3 Timing a forward pass

A single call supplies one sample of elapsed time. It can include one-time initialization or contend with another process for the processor. `measure_latency` first runs **warmup** calls whose times it discards, then records repeated calls individually. Warmup targets steady execution; it deliberately does not measure the first-call cost.

```
def measure_latency(self, model, input_tensor, warmup: int = 10,
    iterations: int = 100) -> float:
    """
    Measure model inference latency with statistical rigor.
    """
    if iterations < 1 or warmup < 0:
        raise ValueError("iterations must be positive and warmup
            nonnegative")
    # Warmup runs to stabilize performance
    for _ in range(warmup):
        _ = model.forward(input_tensor)
```

```
    # Measurement runs
    times = []
    for _ in range(iterations):
        start_time = time.perf_counter()
        _ = model.forward(input_tensor)
        end_time = time.perf_counter()
        times.append((end_time - start_time) * 1000)  # Convert to
    milliseconds

    # Calculate statistics - use median for robustness
    times = np.array(times)
    median_latency = np.median(times)

    return float(median_latency)
```

The method rejects zero measurement iterations and negative warmup counts. Each timed iteration reads `time.perf_counter()` immediately before and after `model.forward(input_tensor)` and converts the difference from seconds to milliseconds. The clock is intended for elapsed-time intervals. Storing each difference allows the method to return their median, or **p50**, rather than letting one unusually slow call determine the result. For times `[1, 1, 1, 1, 9]` milliseconds, the median is 1 while the mean is 2.6.

A **p99** reports the 99th percentile of the measured times and describes their slow tail. A wide gap between p50 and p99 is a reason to investigate variability, not evidence for one particular cause. Module 14 returns only p50. It also calls the model repeatedly without resetting its state or switching its mode, so comparisons must use the same model configuration. A stateful generation loop needs an explicit reset protocol before this timer can make its runs comparable.

15.3.4 Memory during a pass

A forward pass can allocate many temporaries even when its parameter arrays already exist. `measure_memory` asks how much allocation occurs while a dummy forward pass runs. This is a different question from how many bytes cross the memory bus. An array reused many times contributes once to live storage and many times to traffic; an embedding table may occupy memory while a lookup touches only one row.

```
def measure_memory(self, model, input_shape: Tuple[int, ...]) ->
    Dict[str, float]:
    """
    Estimate the live footprint and trace allocations during a forward
    pass.
    """
    # Own the tracing session only if the caller has not already
        started it.
    owns_trace = not tracemalloc.is_tracing()
    if owns_trace:
        tracemalloc.start()
    try:
```

```
        baseline_memory = tracemalloc.get_traced_memory()[0]
        parameter_memory_mb = self._calculate_parameter_memory(model)
        dummy_input = self._dummy_input(model, input_shape)
        output = model.forward(dummy_input)
        # Count the actual input and output buffers, not two input-sized
        # buffers. A view or identity output does not allocate another one.
        activation_bytes = dummy_input.data.nbytes
        if not np.shares_memory(dummy_input.data, output.data):
            activation_bytes += output.data.nbytes
        activation_memory_mb = activation_bytes / MB_TO_BYTES
        _, peak_memory = tracemalloc.get_traced_memory()
        # Parameters were already live at baseline. Add them to the NEW
        # allocations, even when they were tracked by a caller's session:
        # baseline subtraction removes those existing bytes first.
        peak_memory_mb = parameter_memory_mb + max(0, peak_memory - baseline_memory) / MB_TO_BYTES
    finally:
        if owns_trace:
            tracemalloc.stop()

    useful_memory = parameter_memory_mb + activation_memory_mb
    # Keep a lower bound from known live buffers even if tracing misses one.
    peak_memory_mb = max(peak_memory_mb, useful_memory)
    return {
        'parameter_memory_mb': parameter_memory_mb,
        'activation_memory_mb': activation_memory_mb,
        'peak_memory_mb': peak_memory_mb,
        'memory_efficiency': _calculate_memory_efficiency(useful_memory, peak_memory_mb)
    }
```

Read the ownership rule first. The method starts `tracemalloc` only if no caller-owned tracing session exists. It records the current allocated-byte baseline, counts parameter storage, creates a dummy input, and calls `forward`. `_dummy_input` produces token IDs for a model exposing `vocab_size`, otherwise floating-point activations. The separate dummy pass has the supplied input's shape, but not its values.

The activation estimate uses the actual input and output array sizes. If the output shares the input buffer, `np.shares_memory` prevents counting that buffer twice. The traced peak minus the baseline estimates newly allocated bytes. Existing parameter storage is then added, because those weights remain live during the pass. Subtracting the baseline first also avoids counting already-traced parameters twice when a caller started tracing earlier.

For example, 16 MB of existing parameters and a new 4 MB output require at least 20 MB of live storage, even if the input is tiny. Taking the larger of 16 and 4 would miss their simultaneous lifetime. The method instead adds existing parameters to new allocations,

then uses `max` to keep the result at least as large as the known parameters plus boundary activations. All these MB values use the profiler's binary scaling.

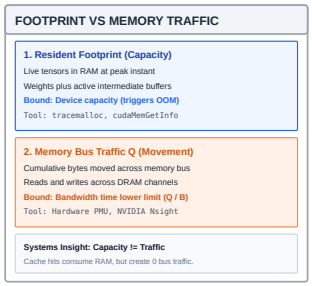
The `finally` block stops only a tracing session this call started, including when `forward` raises. A caller's earlier allocation peak can still inflate the result because the method preserves that session's history. The report is an estimate of model-related live storage, not total process memory or a count of bytes transferred from main memory. `memory_efficiency` divides the estimated useful storage by that peak; it is not hardware utilization.

15.3.5 The roofline and the profiler

The entry point must keep the origins of its numbers visible. FLOPs come from shapes, latency comes from execution, and peak memory combines counted storage with traced allocations. The next helpers turn them into rates and a classification, but arithmetic on a measurement does not remove its assumptions.

```
def _compute_derived_metrics(flops: int, latency_ms: float,
                             peak_memory_mb: float) -> Dict[str, float]:
    """
    Turn raw counts and timings into comparable rates.
    """
    latency_seconds = latency_ms / 1000.0
    gflops_per_second = (flops / 1e9) / max(latency_seconds, 1e-6)
    memory_bandwidth = peak_memory_mb / max(latency_seconds, 1e-6)
    theoretical_peak_gflops = 100.0
    computational_efficiency = min(gflops_per_second /
    ↪ theoretical_peak_gflops, 1.0)

    return {
        'gflops_per_second': gflops_per_second,
        'memory_bandwidth_mbs': memory_bandwidth,
        'computational_efficiency': computational_efficiency
    }
```



Capacity vs. Traffic: Resident footprint dictates peak memory residency and out-of-memory limits, while memory bus traffic governs transfer latency along the roofline slope.

The first rate divides estimated batch FLOPs by measured batch time, reporting computational throughput in GFLOP/s. The second divides reported peak resident storage by that time and labels the result `memory_bandwidth_mbs`. Here we must dissect a critical systems distinction: this metric measures **resident allocation velocity**, not physical DRAM memory bus traffic. Physical hardware bus traffic (captured via hardware performance monitoring units like Linux `perf` or NVIDIA Nsight) tracks actual 64-byte cache lines crossing between processor caches and physical DRAM pins. If an operation reuses data residing entirely inside L1/L2 caches, physical DRAM traffic is virtually zero despite high memory operations. Conversely, if successive unfused operators flush and re-read arrays from DRAM, physical bus traffic can far exceed resident peak footprint. The helper also divides achieved GFLOP/s by an assumed nominal ceiling of 100 GFLOP/s, clamping the result to one. That `computational_efficiency` field provides an illustrative baseline rather than a measured percentage of a specific machine's hardware capacity.

```
def _analyze_bottleneck(gflops_per_second: float,
                        memory_bandwidth_mbs: float) -> Dict[str, Any]:
```

```
    """
    Illustrate a heuristic memory/compute classification.
    """
    is_memory_bound = memory_bandwidth_mbs > gflops_per_second * 100
    return {
        'is_memory_bound': is_memory_bound,
        'is_compute_bound': not is_memory_bound,
        'bottleneck': 'memory' if is_memory_bound else 'compute'
    }
```

The classifier compares `memory_bandwidth_mbs` with `100 * gflops_per_second`. Both rates divide by the same elapsed time, so elapsed time cancels algebraically from the comparison. The classification label is therefore determined strictly by footprint relative to counted operations. Because the footprint field uses decimal megabytes (10^6 bytes per MB), the threshold corresponds to 10 FLOPs per resident footprint byte. Neither metric queries a measured machine ceiling or hardware bus counter directly, so `memory` and `compute` serve as heuristic architectural indicators. `profile_forward_pass` returns them alongside the underlying raw measurements.

```
def profile_forward_pass(self, model, input_tensor) -> Dict[str, Any]:
    """
    Comprehensive profiling of a model's forward pass.
    """
    param_count = self.count_parameters(model)
    flops = self.count_flops(model, input_tensor.shape)
    memory_stats = self.measure_memory(model, input_tensor.shape)
    latency_ms = self.measure_latency(model, input_tensor, warmup=5,
↪   iterations=20)

    # count_flops is per sample; measure_latency times the whole batch.
↪   Dividing
    # one by the other without this factor understates throughput by
↪   the batch
    # size, which is why every GFLOP/s and bottleneck label used to
↪   look
    # memory-bound no matter what the model did.
    batch_size = input_tensor.shape[0] if len(input_tensor.shape) > 1
↪   else 1
    batch_flops = flops * batch_size

    derived = _compute_derived_metrics(batch_flops, latency_ms,
↪   memory_stats['peak_memory_mb'])
    bottleneck = _analyze_bottleneck(derived['gflops_per_second'],

↪   derived['memory_bandwidth_mbs'])

    report = {
        # 'flops' stays per sample, matching count_flops. 'batch_flops'
↪       is the
        # figure the throughput below was computed from.
```

```
        'parameters': param_count, 'flops': flops, 'batch_flops':
        ↪   batch_flops,
        'latency_ms': latency_ms,
        **memory_stats, **derived, **bottleneck
    }
    # Keep the report on the instance so one profiler reused across
    ↪   several
    # models leaves a record the caller can compare afterwards.
    self.measurements[model.__class__.__name__] = report
    return report
```

The key join occurs at `batch_flops = flops * batch_size`. `count_flops` reports one sample, whereas `measure_latency` times the whole input tensor. Omitting the multiplier makes a larger batch appear artificially inefficient. For `Linear(3, 2)` and an input of shape `(4, 3)`, the parameter count is eight including bias, the per-sample matrix-product count is twelve, and `batch_flops` is forty-eight. A GPT sample is a whole sequence, so its sequence-length cost is already inside `flops`; only the outer batch dimension is multiplied here. The entry point also chooses five warmup calls and twenty timed calls rather than the timer's defaults.

The returned fields retain those distinctions instead of collapsing them into one score. `profile_backward_pass`, despite its name, estimates backward cost from the forward report; it does not execute `backward()`. The roofline calculation below requires explicit compute and bandwidth ceilings. The following book-only class holds those assumptions and applies the bound from The Idea.

```
class Roofline:
    """A roofline with explicitly supplied compute and bandwidth
    ↪   ceilings."""
    def __init__(self, peak_gflops, bandwidth_gbs):
        self.peak_gflops = peak_gflops
        self.bandwidth_gbs = bandwidth_gbs
        self.ridge = peak_gflops / bandwidth_gbs   # FLOPs per byte

    def bound_ms(self, flops, nbytes):
        """Lower time bound under the supplied ceilings and traffic
        ↪   model."""
        compute_ms = flops / (self.peak_gflops * 1e9) * 1e3
        memory_ms = nbytes / (self.bandwidth_gbs * 1e9) * 1e3
        return max(compute_ms, memory_ms)

    def regime(self, intensity):
        return "memory-bound" if intensity < self.ridge else
        ↪   "compute-bound"
```

`bound_ms` is the inequality from The Idea in milliseconds; `regime` compares intensity with the assumed ridge. Changing either ceiling changes the prediction without changing the program. A measured time above this bound can reflect overhead, extra traffic, or incomplete use of the machine. A time below it indicates that the chosen ceilings or traffic assumptions did not apply. The bound is a reference for checking a model, not a substitute for the timer.

15.4 A Worked Trace

The trace first checks deterministic counts against TinyGPT. It then times an isolated matrix multiply and compares its modeled work with an explicitly hypothetical roofline. Keeping these steps separate prevents a correct parameter count from being mistaken for a measurement of memory traffic.

15.4.1 Counting TinyGPT

Take the `TinyGPT` that Synthesis II trained, which is Chapter 13's NumPy `GPT` under the name the milestone gave it, at `GPT(vocab_size=1000, embed_dim=128, num_layers=4, num_heads=4, max_seq_len=256)`. Its parameters live in two lookup tables, four identical blocks, a final LayerNorm, and the head, and each entry is a product of shapes plus a bias where the layer has one.

Table 15.1: Parameter count of a four-block TinyGPT with $D = 128$, a 1,000-token vocabulary, and a 256-position table, counted as Chapter 13 builds it, with a learned position table, biases on the four projections, and none on the head.

Component	Shape arithmetic	Parameters
Token embedding	1000×128	128,000
Position table	256×128	32,768
Per block: two LayerNorms	$2 \times (128 + 128)$	512
Per block: Q, K, V, output projections	$4 \times (128 \times 128 + 128)$	66,048
Per block: MLP expand and contract	$(128{\times}512{+}512){+}(512{\times}128{+}128)$	131,712
Four blocks	$4 \times 198{,}272$	793,088
Final LayerNorm	$128 + 128$	256
Output head	128×1000, no bias	128,000
Total		**1,082,112**

At four bytes per parameter that is 4,328,448 bytes, which the profiler reports as 4.13 MB. This is storage, not Q: a lookup need not read its whole table, while temporary activations can be read and written several times.

FLOPs depend on the row count. Each block has six Linear layers whose in-times-out products sum to $4 \times 128^2 + 2 \times 128 \times 512 = 196{,}608$, so one row through one block costs $2 \times 196{,}608 = 393{,}216$ FLOPs in Linear layers, plus the attention products. One row through the head costs $2 \times 128 \times 1000 = 256{,}000$. Compare a one-token input ($B = 1, S = 1$) with a 64-token prompt ($B = 1, S = 64$). Both are ordinary uncached forward passes.

Table 15.2: FLOP counts and boundary-storage accounting for TinyGPT at two sequence lengths. The last row is a footprint ratio, not measured arithmetic intensity.

Quantity	One token, $S = 1$	Prompt, $S = 64$
Linear FLOPs, four blocks	1,572,864	100,663,296
Attention products, $4 \times 4 \times S^2 \times 128$	2,048	8,388,608
Head FLOPs	256,000	16,384,000
Total FLOPs	**1,830,912**	**125,435,904**
Storage bytes (parameters + input + logits)	4,332,452	4,584,704
FLOPs per storage byte	**0.42**	**27.4**

The longer input increases counted arithmetic by about 68.5 times, while this storage total increases by about 5.8 percent. The table deliberately excludes intermediate activations. Its last row checks the ratio of two known counts; it cannot place the model on a main-memory roofline. That would require a traffic estimate accounting for lookup rows, intermediate arrays, and reuse. The arithmetic in the listing below verifies the table without executing a forward pass. In line 1, dimensions `D, V, L, P = 128, 1000, 4, 256` define the TinyGPT configuration. Lines 2 to 6 express the analytical parameter formulas, verified by `assert params == 1_082_112` in line 7. The loop in lines 9 to 14 evaluates total FLOPs and storage footprint across prompt lengths $S = 1$ and $S = 64$, printing the ratios 0.42 and 27.42 matching table 15.2.

The same counts say when attention's quadratic term takes over. Per block, the two attention products cost $4S^2D$ FLOPs, the four projections $8SD^2$, and the MLP $16SD^2$. The products overtake the projections when $S > 2D$, the MLP when $S > 4D$, and all six Linear layers together when $S > 6D$. For TinyGPT's $D = 128$, those thresholds are 256, 512, and 768 tokens, and its 256-position table stops the model at the first one, so Linear layers dominate every configuration it can run. At $S = 64$ the attention products are under 8 percent of the block arithmetic in table 15.2. A model with $D = 4096$ reaches the all-Linear threshold at 24,576 tokens, so whether attention dominates depends on the context the model actually runs at, not on the architecture alone.

```
D, V, L, P = 128, 1000, 4, 256
per_block_linear = 2 * (4 * D * D + 2 * D * 4 * D)      # six Linear
↪   layers, one row
head = 2 * D * V
block_params = (4 * D + 4 * (D * D + D)
                + D * 4 * D + 4 * D + 4 * D * D + D)
params = V * D + P * D + L * block_params + 2 * D + D * V
assert params == 1_082_112

for S in (1, 64):
    flops = L * S * per_block_linear + L * 4 * S * S * D + S * head
    # Float32 parameters, token IDs, and output logits.
    nbytes = 4 * params + 4 * S + 4 * S * V
    print(f"S={S:3d}  FLOPs={flops:>12,}  bytes={nbytes:>10,}  "
          f"F/storage={flops / nbytes:5.2f}")
```

```
S=  1  FLOPs=   1,830,912  bytes= 4,332,452  F/storage= 0.42
S= 64  FLOPs= 125,435,904  bytes= 4,584,704  F/storage=27.36
```

Now the module's counters on the model itself. `count_flops` takes the input shape and, for a GPT, counts one sequence of that length.

```
from tinytorch.core.transformers import GPT
from tinytorch.perf.profiling import Profiler

model = GPT(vocab_size=1000, embed_dim=128, num_layers=4,
            num_heads=4, max_seq_len=256)
profiler = Profiler()
print(profiler.count_parameters(model))
print(profiler.count_flops(model, (1, 1)), profiler.count_flops(model, (1,
    64)))
```

```
1082112
1830912 125435904
```

The counters return the same three numbers as the hand calculation. The input term is four bytes per token because the reference `Tensor` stores float32 arrays, including token IDs. GPT returns `(B, S, V)` logits, so a 64-token input produces $64 \times 1000 \times 4 = 256{,}000$ output bytes. The memory probe counts those actual boundary arrays; its peak can be larger because it also traces intermediate allocations.

15.4.2 Comparing a measured sweep with a model

The second half returns to the 4096×4096 weight matrix from The Problem. Its float32 storage is 67,108,864 bytes. For the traffic model, assume the multiplication reads each weight and input once and writes each output once. This gives $Q = 4(N^2 + 2RN)$ for a square weight of width N and R rows. The assumption describes an ideal transfer count at a chosen memory boundary; it does not count NumPy's internal traffic.

The timer expects a `forward` method and an input object. This wrapper deliberately returns the NumPy result directly to isolate the matrix multiplication rather than add a Tensor construction. Input and weight allocation happen before timing. The sweep prints actual local times alongside achieved FLOP/s and a transfer-rate estimate under the stated Q model.



Linear Intensity Scaling: For small row counts, weight bytes dominate activation traffic, yielding arithmetic intensity of approximately R/2. Execution stays flat on the memory bus until crossing the ridge into compute saturation.

```
import numpy as np
from tinytorch.core.tensor import Tensor

rng = np.random.default_rng(0)
N = 4096
W = rng.standard_normal((N, N), dtype=np.float32)

class MatMul:
    def forward(self, x):
        return np.matmul(x.data, W)

for rows in (1, 2, 8, 16, 64, 256, 1024):
    X = Tensor(rng.standard_normal((rows, N), dtype=np.float32))
    ms = profiler.measure_latency(MatMul(), X, warmup=5, iterations=50)
    flops = 2 * rows * N * N
```

```
    nbytes = W.nbytes + X.data.nbytes + rows * N * 4
    seconds = ms / 1000
    print(f"rows={rows:4d}  I_model={flops / nbytes:6.1f}  "
          f"p50={ms:7.3f} ms  GFLOP/s={flops / 1e9 / seconds:7.1f}  "
          f"modeled GB/s={nbytes / 1e9 / seconds:6.1f}")
```

A flat region in measured time suggests that additional arithmetic is reusing a cost already paid. A region where FLOP/s levels off suggests a limit on arithmetic throughput. These patterns guide investigation, but neither establishes the machine's peak by itself. Cache reuse, changes in the library routine, and other processes can change the curve. Record the matrix size, dtype, library configuration, and timing protocol with any result intended for comparison.

For a calculation independent of a particular machine, use the illustrative ceilings from figure 15.1: 1,000 GFLOP/s and 200 GB/s. The resulting ridge is 5 FLOPs per byte. Table 15.3 shows the two time terms and their maximum; every entry is calculated, not measured.

Table 15.3: Calculated roofline bounds for a 4096×4096 float32 multiplication, using hypothetical 1,000 GFLOP/s compute and 200 GB/s bandwidth ceilings. Intensity is in FLOPs per modeled byte. These are lower bounds under the stated assumptions, not benchmark results.

Rows	Modeled intensity	Compute term (ms)	Transfer term (ms)	Bound (ms)
1	0.50	0.034	0.336	0.336
2	1.00	0.067	0.336	0.336
8	3.98	0.268	0.337	0.337
16	7.94	0.537	0.338	0.537
64	31.03	2.147	0.346	2.147
256	113.78	8.590	0.377	8.590
1,024	341.33	34.360	0.503	34.360

At eight rows, the transfer term of 0.337 ms exceeds the compute term of 0.268 ms. At sixteen rows, arithmetic becomes the larger term. The corresponding intensities, 3.98 and 7.94, lie on opposite sides of the assumed ridge of 5. The following checks reproduce those two rows using the book-only class.

```
roof = Roofline(peak_gflops=1000, bandwidth_gbs=200)
assert roof.ridge == 5
for rows in (8, 16):
    flops = 2 * rows * N * N
    nbytes = 4 * (N * N + 2 * rows * N)
    intensity = flops / nbytes
    print(rows, roof.regime(intensity),
          f"{roof.bound_ms(flops, nbytes):.3f} ms")
```

```
8 memory-bound 0.337 ms
16 compute-bound 0.537 ms
```

This calculation explains how row count changes the limiting term. It does not predict the reader's timings unless both ceilings and the transfer model apply to that machine and routine. A measured time above the bound leaves several possible causes; a time below it is a reason to revise the assumptions. The useful next experiment changes one factor, such as matrix size or row count, and checks whether the explanation still fits.

15.4.3 Reading a complete forward report

The TinyGPT counter checks did not execute the model. `profile_forward_pass` does: once for its dummy memory probe, five times for warmup, and twenty times for timing. The following call measures a 64-token input and checks the report's accounting without promising any particular latency or allocation peak.

```
ids = Tensor(rng.integers(0, 1000, size=(1, 64)))
report = profiler.profile_forward_pass(model, ids)
assert report['parameters'] == 1_082_112
assert report['flops'] == 125_435_904
assert report['batch_flops'] == report['flops']  # batch size is one
seconds = max(report['latency_ms'] / 1000, 1e-6)
expected = report['batch_flops'] / 1e9 / seconds
assert np.isclose(report['gflops_per_second'], expected)
print(f"p50: {report['latency_ms']:.3f} ms")
print(f"peak footprint estimate: {report['peak_memory_mb']:.3f} MB")
print(f"heuristic label: {report['bottleneck']}")
```

The assertions pin down what can be predicted from shapes and the report's own convention. The printed values require execution. In particular, the heuristic label uses a storage-over-work ratio; it is not the result of placing TinyGPT on the hypothetical roofline above. A defensible optimization experiment preserves this distinction when comparing the original model with a changed one.

15.5 From TinyTorch to Production

TinyTorch exposes the accounting in ordinary Python. Production profiling adds finer observation points so that a slow forward pass can be traced to particular operations. PyTorch's `torch.profiler` can record CPU and accelerator activities, input shapes, and tensor allocation information.[59] Its optional FLOP estimate uses formulas for selected operations, including matrix multiplication and two-dimensional convolution; even a production report requires checking which operations it counts. These capabilities are described in the PyTorch profiler documentation.

Timing requires another distinction on a GPU. The CPU can queue an operation and return before the GPU finishes it. Timing only that call measures submission, not completion. An accurate device measurement uses timing events on the GPU or waits for queued work to finish before reading the host clock. PyTorch documents both methods in its CUDA timing guidance. TinyTorch's NumPy timer measures the synchronous CPU forward call used in this module.

[59] In modern PyTorch 2.x, the profiler often reveals that memory-bound pointwise and reduction operators dominate runtime. PyTorch Inductor addresses this by delegating operator fusion directly to OpenAI Triton, compiling fused GPU kernels that keep intermediate tensors in on-chip SRAM instead of issuing repeated DRAM roundtrips.

Allocation tracking still does not reveal memory traffic. NVIDIA's Nsight Compute collects hardware metrics for GPU kernels, the individual device computations, and offers roofline analysis using compute and memory information. Its profiling guide distinguishes traffic at different cache and memory levels. A weight read served from cache does not have the same cost as fetching that weight from main memory. The chosen memory boundary therefore belongs in the definition of Q.

The comparison in table 15.4 separates the three kinds of evidence. A framework trace locates operations; an allocation report describes storage lifetime; hardware traffic metrics test transfer assumptions. These tools answer related questions, but their byte fields are not interchangeable.

Table 15.4: Measurement sources in the reference profiler and production tools. Storage, traffic, and time remain separate quantities.

Quantity	TinyTorch `Profiler`	Production observation
FLOPs	Shape formulas plus fallback estimates	Operator formulas with documented coverage
Storage	Parameter count, boundary arrays, traced allocation peak	Framework allocation and lifetime records
Traffic	No hardware traffic counter	Hardware metrics at a specified memory level
Time	Host clock around synchronous `forward`	Host/device events and operation timelines
Bottleneck	Fixed footprint-to-work heuristic	Workload evidence compared with applicable ceilings

TinyTorch uses analytical FLOP accounting and Python's `time.perf_counter` to estimate compute rates and memory bottlenecks on the host CPU. In production GPU environments, profiling requires visibility into hardware execution pipelines:

1. **PyTorch Profiler** (`torch.profiler`): Records operator execution timelines, kernel dispatch latency, memory allocation events, and exports execution traces viewable in Chrome (`chrome://tracing`) or Perfetto.
2. **NVIDIA Nsight Systems** (`nsys`): Traces system-wide execution, correlating host CPU threads, CUDA runtime calls, PCIe memory transfers, and GPU kernel execution on a unified microsecond timeline to detect CPU starvation and pipeline bubbles.
3. **NVIDIA Nsight Compute** (`ncu`): Profiles individual CUDA kernels at the hardware instruction level, measuring streaming multiprocessor (SM) occupancy, warp stall reasons, Tensor Core utilization, and cache hit rates against hardware roofline limits.

The transferable habit is to make each optimization claim match its evidence. Reducing parameter bytes establishes a storage change. Reducing modeled transfers establishes a prediction under stated assumptions. A repeatable timing improvement on the same workload establishes an execution result. Subsequent chapters require all three distinctions because simulated low-precision arithmetic can still execute in float32.

15.6 Summary

`profile_forward_pass` joins shape-based counts, a separate memory probe, and repeated timed calls. Its central accounting invariant is that FLOPs per sample must be multiplied by batch size before division by whole-batch time. Its memory probe adds existing parameters to new traced allocations, preserves a caller's tracing session, and reports a footprint estimate. Neither that footprint nor the heuristic bottleneck label measures memory traffic.

The roofline adds an explicit model of transferred bytes and machine ceilings. Its bound, $T \geq \max(F/P_{\text{peak}}, Q/B_{\text{peak}})$, explains why more arithmetic need not increase time proportionally. For a float32 Linear layer with few rows and one transfer per weight, intensity is approximately half the row count. That gives a reason to investigate weight traffic, while the timer determines whether a proposed change helps the actual workload. Chapter 16 follows one such proposal: represent weights with integer codes, check reconstruction error, and distinguish modeled packed storage from the NumPy arrays that execute the reference.

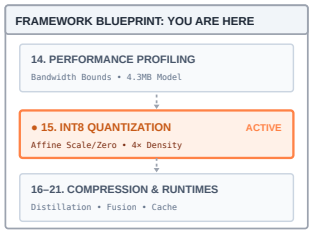
15.7 Further Reading

- **Williams, Waterman, and Patterson** (2009). "Roofline: an insightful visual performance model for multicore architectures." *CACM*. The roofline model this chapter applies.
- **Hennessy and Patterson** (2017). *Computer Architecture: A Quantitative Approach*, 6th edition, Morgan Kaufmann. Memory hierarchies and the discipline of measuring before optimizing.

🔥 Chapter 16

INT8 Quantization: Compressing Floats into 8-Bit Integers

Standard neural network training uses 32-bit floating-point (FP32) arithmetic, but serving large models on edge devices and production servers is heavily constrained by memory capacity, memory bandwidth, and power consumption. Post-training quantization (PTQ) addresses these limits by representing 32-bit floating-point weights and activations as 8-bit integers (INT8). Packed one byte per value, INT8 weights need a quarter of the FP32 storage. Whether they also run faster depends on hardware with integer matrix units, and whether accuracy survives depends on how well the integer grid covers each tensor's values.



Framework Blueprint: Module 15 maps 32-bit floats onto 8-bit integer codes with a scale and zero point, simulating INT8 in float32 and modeling the packed storage a real runtime would use.

In PyTorch, quantizing a tensor to 8-bit integers is achieved with explicit scale and zero-point parameters:

```
import torch

x = torch.randn(4, 4)

# Quantize continuous float32 into 8-bit signed integers
scale, zero_point = 0.05, 0
q_x = torch.quantize_per_tensor(x, scale=scale, zero_point=zero_point,
    dtype=torch.qint8)
dequant = q_x.dequantize()

print("Original FP32:\n", x[0, :2])
print("Quantized INT8 codes:\n", q_x.int_repr()[0, :2])
print("Reconstructed FP32:\n", dequant[0, :2])
```

Underneath that transformation, the runtime executes an affine mapping between continuous real numbers and discrete hardware bytes:

1. **Affine coordinate mapping:** The engine maps real values onto discrete integers through $q = \text{round}(x/S) + Z$ and clamps values to $[-128, 127]$, balancing dynamic range against rounding precision.
2. **Fourfold memory bandwidth reduction:** Packing numbers into single-byte representations reduces memory footprint by $4\times$, quadrupling cache line occupancy and slashing the memory-bus traffic diagnosed in Chapter 15.
3. **Integer widening in arithmetic units:** While storage uses 8-bit integers, matrix units widen intermediate products into 32-bit accumulators (INT32) to prevent numerical overflow during dot products, before rescaling back to floating point.

This chapter constructs TinyTorch's quantization engine. It simulates INT8 by keeping integer codes in float32 arrays and multiplying in float32, measuring numerical fidelity and modeling the packed storage an industrial runtime uses. Before constructing the quantization formulas, the first step is to examine the memory traffic bottleneck that makes 32-bit floats so costly at inference.

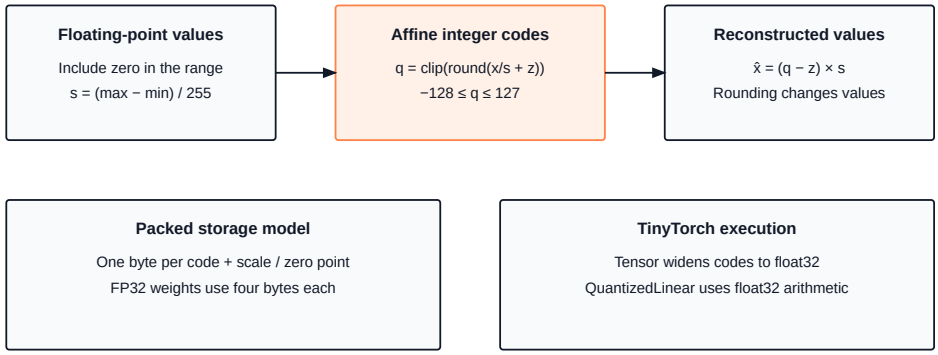


Figure 16.1: **Quantization and storage**: A scale and zero point map floats to integer codes and reconstruct rounded values. Packed storage uses one byte per code; TinyTorch wraps codes in float32 arrays.

16.1 The Problem

↩ **Jump back:** Chapter 15 *Applying operational intensity to determine when weight memory traffic dominates FP32 inference.*

A 32-bit float costs 4 bytes. That is the whole problem, and a napkin shows how much it hurts. A single square linear layer with 4096 inputs and 4096 outputs holds $4096 \times 4096 = 16{,}777{,}216$ weights, which is 67 MB in FP32. Generating one token multiplies one activation vector by that matrix, so the processor must stream all 67 MB through memory to do $2 \times 4096 \times 4096 \approx 33.5$ million arithmetic operations. That is 0.5 operations per byte, the arithmetic intensity defined in Chapter 15, which makes weight bandwidth a plausible bottleneck when the weights must be fetched from main memory. At a hypothetical sustained memory bandwidth of 100 GB/s, those bytes alone imply a 0.67 ms lower bound per token per layer. Faster arithmetic cannot lower this traffic bound. Packing the weights addresses the byte count; batching and reuse address how often those bytes must cross memory.

- Weights: 16,777,216. FP32 storage: 67.1 MB. INT8 storage: 16.8 MB.
- If a kernel reads packed codes directly, weight bytes streamed per generated token drop by $4\times$; at the assumed 100 GB/s the corresponding lower bound drops from 671 μs to 168 μs.
- Arithmetic operations per token are unchanged at 33.5 million. Counting weight bytes alone, the intensity rises from 0.5 to 2.0 operations per byte. Whether memory still limits execution depends on the machine and kernel.

The first attempt fails on four small weights. Casting truncates each value toward zero:

```
weights = [0.0123, -0.0871, 0.0456, -0.0012]
print([int(w) for w in weights])   # every weight becomes 0
```

The integers -128 through 127 are perfectly good for counting, but the weights do not live on that grid. Quantization is the trick of moving them onto it.

Pause & Predict: Dynamic Range vs. Quantization Resolution

Suppose an activation vector has 99.9% of its values clustered tightly in $[-2.0, 2.0]$, but a single activation outlier spikes at $x = 127.0$. If we apply asymmetric INT8 quantization across $[-2.0, 127.0]$ (255 discrete bins), what is the resulting step scale S? What happens to the resolution of the non-outlier features? How does this explain why naive INT8 quantization degrades large language models?

16.2 The Idea

16.2.1 One scale and one zero point

↪ **Jump ahead:** Chapter 17 *combining weight quantization with knowledge distillation for compound model compression.*

In production quantization runtimes like TensorRT and PyTorch AO (`torch.ao.quantization`), affine quantization maps continuous floating-point activations onto discrete 8-bit integers using an explicit scale factor and zero-point offset. This parameterization preserves representation fidelity for skewed or one-sided distributions—such as post-ReLU activations that are strictly non-negative—without discarding half the available integer grid. Consider `[0.0, 0.5, 1.0]`. Its endpoints are one unit apart. Dividing that interval into 255 steps gives a step width of $1/255$. Code -128 represents zero, code 127 represents one, and the midpoint rounds to code 0, which reconstructs as $128/255 \approx 0.50196$. The middle value changes slightly; it no longer disappears. The step width and the code assigned to zero are the two numbers needed to repeat this mapping for another tensor.

Take a tensor whose values lie between x_{min} and x_{max}. Stretch that interval so that it includes 0.0 (if the values are all positive, as the output of a ReLU is, the interval grows down to zero), and then spread it evenly over the 256 codes a signed byte can hold. The width of one step is the **scale**:

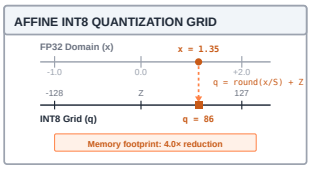
$$S = \frac{x_{\text{max}} - x_{\text{min}}}{255}$$

Divide every value by S and the tensor spans 255 steps. Shifting it so that x_{min} lands on -128 takes one integer offset, the **zero point** Z, which is the code that represents 0.0:

$$Z = \text{round}\left(-128 - \frac{x_{\text{min}}}{S}\right), \qquad q = \text{clamp}\left(\text{round}\left(\frac{x}{S} + Z\right),\ -128,\ 127\right)$$

Here x is the original float and q is its 8-bit code. Rounding the zero point can put an endpoint code just outside the byte, so the clamp is necessary even for the observed range. A grid chosen from different data may require much larger clipping. Stretching the interval to include zero keeps Z inside the byte, but it can enlarge the step substantially for a narrow interval far from zero. The reconstruction bound must use that enlarged interval. To get a float back, undo the shift and the division:

$$\hat{x} = (q - Z) \times S$$



Affine INT8 quantization: continuous FP32 values project onto 256 discrete bins with scale S and zero-point Z.

The hat marks that $\hat{x}$ is a reconstruction, not the original. For values in the range used to construct this grid, reconstruction differs by at most half a step in exact arithmetic:

$$|x - \hat{x}| \leq \frac{S}{2} = \frac{x_{\max} - x_{\min}}{510}$$

The bound describes individual values inside the observed range, with an additional floating-point rounding error in the implementation. It does not bound the model's prediction error, nor does it apply to later inputs outside a frozen calibration range. Within its scope, it is about 0.2% of the zero-inclusive range, regardless of the tensor's size. The bound is also the warning. It is relative to the range, not to x. A weight of 0.001 in a tensor whose values span $[-1, 1]$ is rounded to the grid step $S \approx 0.0078$, and comes back as 0. Small values in a tensor with one large outlier are where quantization loses information.

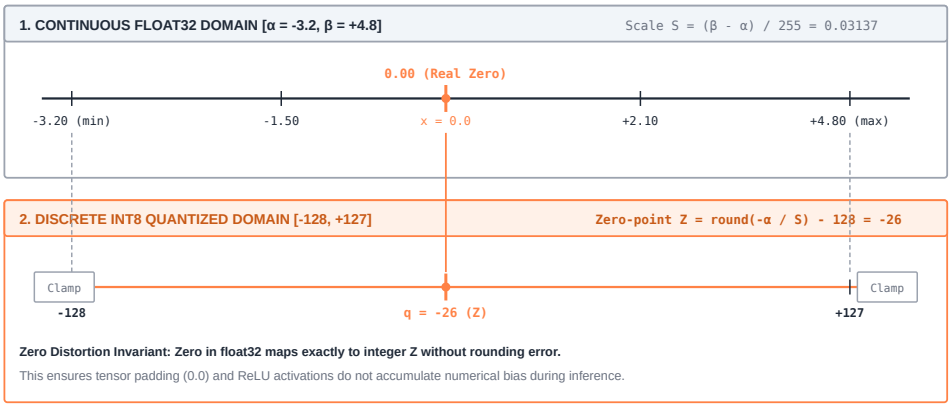
A signal-to-noise ratio turns the per-value bound into one number for the whole tensor. If rounding errors are spread evenly across $[-S/2, S/2]$, their root-mean-square value is $S/\sqrt{12}$. A signal that fills the range uniformly has root-mean-square value $(x_{\max} - x_{\min})/\sqrt{12}$, and the ratio of the two, expressed in decibels, is

$$\text{SNR} = 20 \log_{10} \frac{x_{\max} - x_{\min}}{S} = 20 \log_{10} (2^b - 1) \approx 6.02\, b \text{ dB}$$

for a grid of b bits. Eight bits give $20 \log_{10} 255 \approx 48.1$ dB, and each added bit adds about 6 dB, because it doubles the number of steps. The figure is a ceiling, not a prediction. Real weight tensors cluster near zero rather than filling their range, so their signal power falls below the uniform case while the noise, which is set by the step, does not; one outlier that stretches the range lowers the ratio for every other value. How much ratio a given model can tolerate is a property of that model, and only an accuracy measurement settles it.

This mapping is called **asymmetric** because the grid does not have to be centered on zero; it is the form Module 15 uses. The **symmetric** special case takes $x_{\min} = -x_{\max}$, so that $Z = 0$ and the formulas lose a term; it wastes codes when a tensor is all positive, which is why the general form earns its extra integer.[60] A tensor of all one value has no range at all, and the module handles it with a separate branch that encodes every element as code 0 and lets Z and S carry the constant.

[60] **Hardware efficiency of symmetric quantization**: In symmetric quantization, $Z = 0$, which halves the algebraic expansion of integer matrix multiplication. Expanding the general asymmetric product yields $(Q_x - Z_x)(Q_W - Z_W)^T = Q_x Q_W^T - Z_x \sum Q_W^T - Z_W \sum Q_x^T + Z_x Z_W K$. For symmetric weights ($Z_W = 0$), two terms vanish entirely: $Q_x Q_W^T - Z_x \sum Q_W^T$. Because weights are static during inference, the column reduction vector $\sum Q_W^T$ can be precomputed once at compilation time, leaving hardware tensor cores to execute a pure fused multiply-add on raw integer codes with a single runtime vector correction.



Affine uniform quantization maps continuous FP32 values to discrete INT8 bins while strictly preserving exact zero representation.

Figure 16.2: **Affine quantization grid and zero-point alignment**: Continuous float32 values across dynamic range $[x_{\min}, x_{\max}]$ are discretized into 256 uniform integer bins ($q \in [-128, 127]$). Scale S specifies step size, zero-point Z aligns real value 0.0 to an exact integer code, and clamping prevents out-of-range overflow, bounding reconstruction error to $\leq S/2$.

16.2.2 A scale shared by the whole tensor

In hardware accelerators with dedicated integer pipelines—such as NVIDIA Tensor Cores and Intel VNNI—matrix multiplies execute directly on low-precision integer codes, deferring floating-point math until after reduction. Sharing a scale across an entire tensor allows the runtime to factor scalar arithmetic out of the inner loop, executing high-throughput integer dot products while rescaling the accumulated result only once. A shared scale also explains how a kernel could multiply the integer codes. TinyTorch's layer first reconstructs float32 operands; the following identity enables checking that path against a separate integer calculation. Write the reconstructed activation row and weight matrix in terms of their codes, $\hat{x} = S_x(Q_x - Z_x)$ and $\hat{W} = S_W(Q_W - Z_W)$. Their product factors:

Core Invariant: Symmetric Zero-Point Cancellation

$Z_W = 0 \implies (Q_x - Z_x)Q_W^T = Q_x Q$

Weight symmetry removes the input-dependent $Z_W \sum Q_x$ correction; the remaining $Z_x \sum Q_W$ term depends only on the weights and can be precomputed.

$$\hat{y} = \hat{x}\hat{W}^T = (S_x S_W)\left((Q_x - Z_x)(Q_W - Z_W)^T\right)$$

For this equation, the weight matrix has one output per row; the source Linear stores its transpose with shape `[in_features, out_features]`. Everything inside the last parentheses is integer arithmetic: subtract each zero point from its codes, which is still an integer, then take the dot products. The two scales are ordinary floats, and their product can rescale each accumulated output once. A single scale per tensor makes this factorization possible. The stored codes fit in a byte, but subtracting zero points and summing products require wider integers. The book-only integer function below will make those intermediate types explicit. A kernel that reads packed INT8 weights would move a quarter of the weight bytes. On the memory-bound slope of the Roofline model (Chapter 15), that bounds the possible gain at $4\times$ without changing F. TinyTorch does not realize this reduction: its codes live in float32 arrays and the multiply runs in float32. Production integer units such as AVX-512 VNNI and GPU Tensor Cores are outside this module.

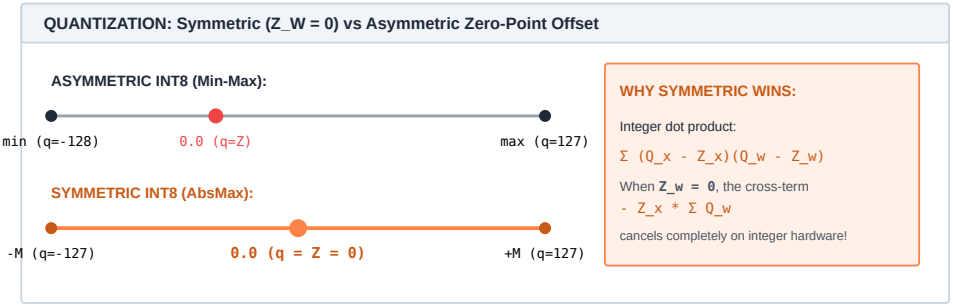


Figure 16.3: **Symmetric vs. asymmetric quantization grids**: Asymmetric quantization maps arbitrary $[x_{\min}, x_{\max}]$ to $[-128, 127]$ with an integer offset Z. Symmetric quantization centers the grid on zero ($Z_W = 0, \pm M$), sacrificing minor dynamic range on skewed distributions to eliminate runtime zero-point subtraction overhead on hardware DP4A / Tensor Cores.

Everything in this chapter happens after training, with no gradient in sight. It is **post-training quantization**, a transformation of a finished model. Weight-only conversion needs the weights; static activation quantization also needs representative calibration inputs.

16.3 The Code

16.3.1 Quantize and dequantize

>_ SOURCE CODE MAPPING MOD 15
FILE: src/15_quantization/15_quantization.py
EXPORT: tinytorch.perf.quantization
SYMBOLS: quantize_int8, QuantizedLinear
Extracted & verified at build time ✓ tested

Source Code Mapping: Module 15 extracts affine INT8 quantization, dynamic range calibration, and integer matrix multiplication directly from the working repository.

In deep learning deployment frameworks like TensorRT and bitsandbytes, quantization and dequantization kernels form the foundational boundary operations bridging continuous floating-point models and low-bit representations. These routines convert high-precision parameter arrays into discrete byte-sized integer codes to shrink storage footprint and bandwidth demands, while defining the inverse mapping to reconstruct floating-point values. The weights form the primary starting point because they represent the dominant memory footprint to compress. The 4096 by 4096 layer from The Problem holds 16.8 million floats. A packed representation needs one integer code per value and enough information to reconstruct the float grid. Everything else in the chapter sits on this pair of functions, and Chapter 17 will change the number of matrix entries after this chapter changes their precision.

The Problem already showed the naive version. Cast each weight to an integer and every one of them, being smaller than 1 in magnitude, becomes 0. A packed integer layer would occupy 16.8 MB for its weights and would compute nothing but its bias. The grid must distinguish these small floats despite having only 256 codes. The scale is the answer. Measure the zero-inclusive range, divide by its grid step, add the zero point, and round. The weights now spread across the whole byte, and each one comes back to within $S/2$ of where it started.

`quantize_int8` does the measuring, dividing, shifting, rounding, and clamping.

```python
def quantize_int8(tensor: Tensor) -> Tuple[Tensor, float, int]:
    """
    Quantize FP32 tensor to INT8 using asymmetric (min-max) quantization.
    """
    data = tensor.data
    if data.size == 0 or not np.all(np.isfinite(data)):
        raise ValueError("Quantization requires nonempty finite values")

    # Step 1: Find dynamic range
    min_val = float(np.min(data))
    max_val = float(np.max(data))

    # Step 2: Handle edge case (constant tensor).
    # All elements have the same value c, so there is no range to map. We encode
    # every element as q=0 and choose zero_point/scale so dequantization recovers
    # the constant via (0 - zero_point) * scale = c.
    if max_val == min_val:
        c = min_val
        if c == 0:
            scale = 1.0
            zero_point = 0
        else:
            # Encode the magnitude in the scale, preserving both fractional
            # and large constants with a zero point that fits in one byte.
            zero_point = -1 if c > 0 else 1
            scale = abs(c)
        quantized_data = np.zeros_like(data, dtype=np.int8)
        return Tensor(quantized_data), scale, zero_point

    # Step 3: Nudge the range to include zero before computing scale.
    # If min_val and max_val share a sign (every post-ReLU activation, for
    # instance), the zero_point implied by the raw range falls outside
    # [-128, 127]. Clamping it there destroys the affine mapping and silently
    # corrupts every value.
    # Widening the range so it straddles zero is what PyTorch and TFLite do,
    # and it costs at most one quantization level of precision.
    min_val = min(min_val, 0.0)
    max_val = max(max_val, 0.0)

    # Step 4: Calculate scale
    # Map [min_val, max_val] to [INT8_MIN_VALUE, INT8_MAX_VALUE] (INT8 range)
    scale = (max_val - min_val) / INT8_LEVELS
```

```
    # Step 5: Calculate zero_point (the code that min_val maps to is -128)
    zero_point = int(np.round(INT8_MIN_VALUE - min_val / scale))

    # zero_point is now guaranteed inside the INT8 range by construction;
    # clamp defensively rather than as a correctness crutch.
    zero_point = int(np.clip(zero_point, INT8_MIN_VALUE, INT8_MAX_VALUE))

    # Step 6: Apply quantization formula q = (x / scale) + zero_point, clamp to
    # the INT8 range, and cast to int8. Tensor() will widen the int8 array back
    # to float32 (see the NOTE in the docstring), so the codes are exact but the
    # array is not one byte per element.
    quantized_data = np.round(data.astype(np.float64) / scale + zero_point)
    quantized_data = np.clip(quantized_data, INT8_MIN_VALUE, INT8_MAX_VALUE).astype(np.int8)

    return Tensor(quantized_data), scale, zero_point
```

Steps 3 through 6 are the formulas from The Idea, in order. The initial validation rejects empty arrays and nonfinite values, since neither supplies a usable finite range. Step 2 is the case the formulas cannot handle, a tensor with no range, which the module encodes as all zeros and lets the scale and zero point reconstruct. Step 3 is the one that is easy to skip and wrong to skip: without it, an all-positive tensor puts Z below -128, the clamp on the next line silently moves it, and every value comes back shifted. The comments say so because a student who removes the line sees no error, only worse accuracy. The constant branch has a different invariant. For a positive constant `c`, it chooses `scale = abs(c)`, `zero_point = -1`, and all-zero codes, so reconstruction gives `(0 - (-1)) * abs(c) = c`. A negative constant uses zero point `+1`; zero itself uses scale one and zero point zero. Putting the magnitude in the scale preserves fractional constants such as `0.25`. Testing exact equality of the extrema also matters: a tiny nonconstant interval still needs a grid, not a branch that discards its variation. The general branch uses float64 intermediates for division, so a positive scale too small for float32 does not become zero before rounding. Reconstruction uses the same wider intermediate and clips to the finite float32 endpoints before returning a new Tensor. These choices preserve the finite range supported by the original Tensor; they do not change its storage type.

```
def dequantize_int8(q_tensor: Tensor, scale: float, zero_point: int) -> Tensor:
    """
    Dequantize INT8 tensor back to FP32.
    """
    if scale <= 0 or not np.isfinite(scale):
```

```
        raise ValueError(f"Scale must be positive and finite, got {scale}")
    if not (-128 <= zero_point <= 127):
        raise ValueError(f"Zero point must be in [-128, 127], got {zero_point}")

    # Apply inverse quantization formula
    # This is the correct inverse of: quantized = (value / scale) + zero_point
    # Therefore: value = (quantized - zero_point) * scale
    # Compute on a wider grid: a valid float32 endpoint can round just beyond
    # float32's range, while a subnormal scale may not fit in float32 at all.
    dequantized_data = (q_tensor.data.astype(np.float64) - zero_point) * scale
    fp32_limit = np.finfo(np.float32).max
    return Tensor(np.clip(dequantized_data, -fp32_limit, fp32_limit))
```

Run the pair on three numbers.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.perf.quantization import quantize_int8, dequantize_int8, QuantizedLinear

x = Tensor(np.array([0.5, -0.3, 0.1], dtype=np.float32))
q, scale, zero_point = quantize_int8(x)
print(q.data, scale, zero_point)           # [127. -128.    0.] 0.003137... -32
print(dequantize_int8(q, scale, zero_point).data)   # [0.4988 -0.3012 0.1004]
```

The range is $[-0.3, 0.5]$, so the scale is $0.8/255 = 0.003137$ and the zero point is $\text{round}(-128 + 0.3/0.003137) = -32$. The code for 0.1 is $\text{round}(31.875 - 32) = 0$, which is the zero point doing its job: 0.1 is not zero, but it sits 32 steps above the value represented by the zero point, and the grid records that. Every reconstruction is within $S/2 \approx 0.0016$ of its original, as the bound promised. One thing to notice about the printout is that the codes print as `127.` with a decimal point. `quantize_int8` does produce an `int8` array, but it hands the array to `Tensor`, and Chapter 1's `Tensor` stores its arrays as float32. So in this NumPy framework the codes occupy four bytes each the moment they are wrapped, and the memory saving is accounted for by `memory_usage` below rather than realized in RAM. The byte saving needs a container that preserves the integer dtype. Neither function mutates its input; each returns a new Tensor, along with grid metadata in the quantizer's case.

The constant case can be checked independently of the general rounding bound:

```
for value in (0.0, 0.25, -0.5):
    original = Tensor([value, value])
    codes, scale, zero = quantize_int8(original)
```

```
    restored = dequantize_int8(codes, scale, zero)
    assert np.array_equal(restored.data, original.data)
    assert np.all(codes.data == 0)
    print(value, scale, zero)
# 0.0 1.0 0
# 0.25 0.25 -1
# -0.5 0.5 1
```

`calibrate` widens the observed input range to include zero before checking equality, so its constant branch applies only to all-zero samples. A constant positive calibration set uses the ordinary grid from zero to that value. Both routes must preserve small nonzero magnitudes; replacing their equality test with a broad absolute tolerance would erase valid inputs.

16.3.2 The quantized layer

Production inference engines encapsulate low-precision execution into specialized modules—such as PyTorch's `nn.quantized.Linear` or TensorRT's quantized plugin layers—that bind pre-quantized weights with runtime activation scaling. This modular abstraction decouples offline weight compression from online activation calibration, enabling seamless graph rewrites that replace memory-hungry FP32 layers with integer kernels. A packed integer layer would keep its weight codes in one byte each and consume them directly. TinyTorch keeps float32 arrays for compatibility with the Tensor implemented earlier, so `QuantizedLinear` cannot provide that storage reduction. Reading the class is still useful because it separates three moments that an inference implementation must handle: choosing the weight grid at construction, choosing an input grid during calibration, and applying those grids during each forward pass.

A reconstruction buffer has a cost. If a hypothetical packed layer reads 16.8 MB of codes, writes 67.1 MB of reconstructed weights, and then reads that entire buffer for the multiply, those three array operations involve about 151 MB. This is an accounting example, not measured memory-bus traffic; caches and buffer reuse affect actual transfers. TinyTorch also stores the codes in float32, so this packed-code count does not describe its allocation. A deployment kernel must preserve the storage advantage while arranging the multiply. The production bridge returns to that choice.

That leaves the activations, which have a scale of their own in the product and no obvious place to get it from. For weights, the range is known the moment training ends; look at the matrix and take its minimum and maximum. Activations depend on the input, so their range is not known until the input arrives. There are two honest options. **Dynamic quantization** computes S_x and Z_x from each input as it comes, which measures the current range but still rounds values and costs a pass over the activations before every multiply. **Static quantization** runs a few representative inputs through the model offline, records the range the layer sees, and freezes that as the layer's input grid. This offline step is called **calibration**, and the recorded range is only as good as the samples were representative.

`QuantizedLinear` is built from an existing `Linear`. It quantizes the weights once, at construction, and the bias too, each with its own scale and zero point. These code arrays are independent of the original parameters. The wrapper also keeps an `original_layer` reference for memory accounting; changing the original weights later does not recompute its codes. Initially `input_scale` and `input_zero_point` are `None`, so inputs remain

unrounded until `calibrate` sets both fields. The following four excerpts are methods of this same class.

```
def __init__(self, linear_layer: Linear):
    """
    Create quantized version of existing linear layer.
    """
    self.original_layer = linear_layer

    # Quantize weights
    self.q_weight, self.weight_scale, self.weight_zero_point =
    ↪   quantize_int8(linear_layer.weight)

    # Quantize bias if it exists
    if linear_layer.bias is not None:
        self.q_bias, self.bias_scale, self.bias_zero_point =
        ↪   quantize_int8(linear_layer.bias)
    else:
        self.q_bias = None
        self.bias_scale = None
        self.bias_zero_point = None

    # Store input quantization parameters (set during calibration)
    self.input_scale = None
    self.input_zero_point = None
```

`calibrate` receives a list of input Tensors. Flattening them combines their observed values into one range, so every feature shares one input scale. The extrema are widened to include zero. The all-zero case uses scale one and zero point zero; otherwise the same affine-grid formulas determine the two fields. This method returns no converted samples. Its lasting effect is the new input grid.

```
def calibrate(self, sample_inputs: List[Tensor]):
    """
    Calibrate input quantization parameters using sample data.
    """
    # Collect all input values
    all_values = []
    for inp in sample_inputs:
        all_values.extend(inp.data.flatten())

    all_values = np.array(all_values)
    if all_values.size == 0 or not np.all(np.isfinite(all_values)):
        raise ValueError("Calibration requires nonempty finite
        ↪   samples")

    # Calculate input quantization parameters, widening the range to
    # straddle zero exactly as quantize_int8 does (post-ReLU inputs
    ↪   are
    # all non-negative, and a zero_point outside the INT8 range would
    # otherwise be clamped and corrupt every value)
    min_val = min(float(np.min(all_values)), 0.0)
```

```
    max_val = max(float(np.max(all_values)), 0.0)

    if max_val == min_val:
        self.input_scale = 1.0
        self.input_zero_point = 0
    else:
        self.input_scale = (max_val - min_val) / INT8_LEVELS
        self.input_zero_point = int(np.round(INT8_MIN_VALUE - min_val
        ↪   / self.input_scale))
        self.input_zero_point = int(np.clip(self.input_zero_point,
        ↪   INT8_MIN_VALUE, INT8_MAX_VALUE))
```

That stored state selects the branch at the start of `forward`. The method can now expose rounding and saturation using a grid measured on different inputs.

```
def forward(self, x: Tensor) -> Tensor:
    """
    Forward pass with quantized computation.
    """
    # For educational purposes, we dequantize and compute in FP32
    # Production systems use specialized INT8 GEMM operations

    # Round the input onto the calibrated grid and back, so the
    ↪   activations
    # carry the same rounding (and saturation) they would on INT8
    ↪   hardware
    if self.input_scale is not None:
        if not np.all(np.isfinite(x.data)):
            raise ValueError("QuantizedLinear forward requires finite
            ↪   input values")
        q_x = np.clip(np.round(x.data.astype(np.float64) /
↪   self.input_scale + self.input_zero_point),
                      INT8_MIN_VALUE, INT8_MAX_VALUE)
        x = dequantize_int8(Tensor(q_x), self.input_scale,
↪   self.input_zero_point)

    # Dequantize weights
    weight_fp32 = dequantize_int8(self.q_weight, self.weight_scale,
↪   self.weight_zero_point)

    # Perform computation (same as original layer)
    result = x.matmul(weight_fp32)

    # Add bias if it exists
    if self.q_bias is not None:
        bias_fp32 = dequantize_int8(self.q_bias, self.bias_scale,
↪   self.bias_zero_point)
        result = Tensor(result.data + bias_fp32.data)

    return result
```

Read `forward` in call order. If an input grid exists, it rounds and clamps `x` using that grid, then calls `dequantize_int8` to create a float32 reconstruction. It independently reconstructs the stored weight codes, calls `x.matmul(weight_fp32)`, and adds a reconstructed bias if present. For input shape `[batch, in_features]` and weight shape `[in_features, out_features]`, the returned Tensor has shape `[batch, out_features]`. None of these steps changes the stored codes or recalibrates the grid. Inputs outside the calibrated range saturate at the edge of the byte; the drift trace below shows how this changes the output.

`memory_usage` estimates one byte per weight and bias code plus eight metadata bytes per quantized array: a float32 scale and a four-byte zero point. A layer with a bias needs two such pairs.

```
def memory_usage(self) -> Dict[str, float]:
    """
    Model packed INT8 bytes, including metadata; not actual NumPy
    storage.
    """
    # Original FP32 usage
    original_weight_bytes = self.original_layer.weight.data.size *
    BYTES_PER_FLOAT32
    original_bias_bytes = 0
    if self.original_layer.bias is not None:
        original_bias_bytes = self.original_layer.bias.data.size *
    BYTES_PER_FLOAT32

    # Quantized INT8 usage
    quantized_weight_bytes = self.q_weight.data.size * BYTES_PER_INT8
    quantized_bias_bytes = 0
    if self.q_bias is not None:
        quantized_bias_bytes = self.q_bias.data.size * BYTES_PER_INT8

    # Overhead for the quantization parameters (a scale and a zero
    point):
    # a few bytes, negligible next to the arrays
    # Each quantized array needs its own float32 scale and integer zero
    point.
    overhead_bytes = BYTES_PER_FLOAT32 * 2 * (1 + int(self.q_bias is
    not None))

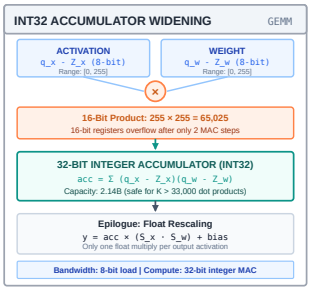
    quantized_total = quantized_weight_bytes + quantized_bias_bytes +
    overhead_bytes
    original_total = original_weight_bytes + original_bias_bytes

    return {
        'original_bytes': original_total,
        'quantized_bytes': quantized_total,
        'compression_ratio': original_total / quantized_total if
        quantized_total > 0 else 1.0
    }
```

The report counts a hypothetical packed parameter format. It excludes Python objects, the actual float32 code arrays, the retained `original_layer`, and temporary reconstruc-

tion buffers. The input-grid fields are also outside this parameter-storage count. These boundaries matter most for small layers, where metadata can exceed the code bytes.

The integer identity can be checked against the same layer without changing Module 15. It quantizes the input on the fly, subtracts both zero points, takes the dot products in 32-bit integers, and rescales once per output.



Integer MAC datapath: 8-bit operand products require a 32-bit integer accumulator (INT32) to prevent numerical overflow across reduction dimension K, followed by a single floating-point rescale epilogue.

```
def int8_forward(layer: QuantizedLinear, x: Tensor) -> np.ndarray:
    """The integer path: codes in, one rescale out. Not in Module 15."""
    q_x, x_scale, x_zero = quantize_int8(x)
    lhs = q_x.data.astype(np.int32) - x_zero  # [rows, in]
    rhs = layer.q_weight.data.astype(np.int32) - layer.weight_zero_point  # [in, out]
    acc = lhs @ rhs  # int32 dot products
    y = acc * (x_scale * layer.weight_scale)  # one float multiply per output
    if layer.q_bias is not None:
        y = y + dequantize_int8(layer.q_bias, layer.bias_scale, layer.bias_zero_point).data
    return y
```

One detail matters for the accumulator. After the zero points are subtracted each operand spans up to 255, so one product can reach $255 \times 255 = 65{,}025$, and a dot product over K inputs sums K of them. A 32-bit accumulator holds up to $2{,}147{,}483{,}647$, enough for K up to about $33{,}000$ inputs at the worst case; a 16-bit accumulator cannot hold even one worst-case product. This book function uses 32-bit accumulation, as does the integer-kernel design it models. NumPy's `int32` does it here, and the trace checks that the sums stay far from the edge. This function is an arithmetic demonstration, not a packed or optimized kernel: its NumPy operands are int32 arrays, and it allocates them on every call.

16.3.3 Quantizing a whole model

In end-to-end model optimization pipelines like PyTorch's FX Graph Mode Quantization or ONNX Runtime, converting an entire network requires traversing the computational graph and swapping operators in dependency order. Because downstream layers operate on the discretized outputs of upstream operations, static quantization must sequentially propagate representative inputs through converted prefixes to record authentic activation distributions. A two-layer model introduces a dependency that a single layer does not have. In `Sequential(Linear(4, 8), ReLU(), Linear(8, 2))`, a calibration input of shape `[1, 4]` must pass through the first Linear and ReLU before it can calibrate the second Linear, whose input shape is `[1, 8]`. Passing raw samples to both layers would measure the wrong distribution and even the wrong feature count.

`quantize_model` walks the container in order. It collects a layer's calibration inputs through the prefix already converted, constructs a `QuantizedLinear`, and assigns the replacement to `model.layers[i]`. For a nested container, it collects that container's input samples and recurses. The call returns `None`; the supplied model now owns the replacements. Copy the model first when a separate baseline is needed. A standalone

Linear has no containing slot to replace, so the function rejects it and directs callers to `QuantizedLinear(linear)`.

```
def quantize_model(model, calibration_data: Optional[List[Tensor]] = None) -> None:
    """
    Quantize all Linear layers in a model in-place, including nested containers.
    """
    if hasattr(model, 'layers'):
        for i, layer in enumerate(model.layers):
            if isinstance(layer, Linear) or hasattr(layer, 'layers'):
                # Collect calibration inputs if data provided
                cal_inputs = None
                if calibration_data is not None:
                    cal_inputs = _collect_layer_inputs(model, i, calibration_data)

                # Replace with quantized version
                if isinstance(layer, Linear):
                    model.layers[i] = _quantize_single_layer(layer, cal_inputs)
                else:
                    quantize_model(layer, cal_inputs)

    elif isinstance(model, Linear):
        raise ValueError(
            f"Cannot quantize single Linear layer in-place\n"
            f"  ✗ quantize_model() modifies models in-place, but a single layer has no container to modify\n"
            f"  💡 In-place modification requires a container (like Sequential) that holds layer references\n"
            f"  🔧 Use QuantizedLinear directly: quantized_layer = QuantizedLinear(your_linear_layer)"
        )

    else:
        raise ValueError(
            f"Unsupported model type for quantization: {type(model).__name__}\n"
            f"  ✗ quantize_model() expects a model with .layers attribute (like Sequential)\n"
            f"  💡 The function iterates through model.layers to find and replace Linear layers\n"
            f"  🔧 Wrap your layers in Sequential: model = Sequential(layer1, activation, layer2)"
        )
```

The ordering is in the loop itself. Layer `i` is replaced before the loop reaches layer `i + 1`, so when the helper below runs calibration data forward through layers `0` to `i - 1`, it runs through the quantized versions, and layer `i` is calibrated on the activations it will actually receive.

```python
def _collect_layer_inputs(model, layer_index: int, calibration_data:
    List[Tensor], max_samples: int = 10) -> List[Tensor]:
    """
    Forward calibration data through preceding layers to collect
    inputs for a specific layer.
    """
    sample_inputs = []
    for data in calibration_data[:max_samples]:
        x = data
        for j in range(layer_index):
            layer = model.layers[j]
            # Calibration describes inference, so disable Dropout
                (also inside
            # nested Sequential containers) using the Module 03 mode
                flag.
            if 'training' in
                inspect.signature(layer.forward).parameters:
                x = layer.forward(x, training=False)
            else:
                x = layer.forward(x)
        sample_inputs.append(x)
    return sample_inputs
```

The helper calls preceding layers in inference mode when their `forward` accepts `training`. This disables Chapter 3's Dropout, including Dropout inside a nested Sequential. Otherwise the frozen grid would describe randomly masked training activations. `_quantize_single_layer` then constructs the wrapper and, when samples were supplied, calls `calibrate`.

The `max_samples` cap selects at most ten items from the supplied list; an item may itself be a batch. `calibrate` scans their flattened values and saves only the scale and zero point, not the samples. Extra items past the cap do not influence this grid. An empty or nonfinite calibration set raises an error. Representative coverage matters more than a universal sample count, so a held-out comparison must check for missed ranges.

The wrapper named `Quantizer.quantize_model` has a different contract from the standalone function shown here. It returns parameter artifacts and modeled packed-storage statistics without replacing layers or calibrating activations. Its report includes a scale and zero point for each parameter array. Calling the wrapper is therefore insufficient to change model execution.

16.4 A Worked Trace

A 2 by 2 weight matrix and a 2-element input are enough to check every step by hand.

$$W = \begin{bmatrix} 1.27 & -0.638 \\ 0.253 & -1.019 \end{bmatrix}, \qquad x = \begin{bmatrix} 0.5 & -0.3 \end{bmatrix}$$

The weights range from -1.019 to 1.27, so $S_W = 2.289/255 = 0.008976$ and $Z_W = \text{round}(-128 + 1.019/0.008976) = \text{round}(-14.48) = -14$.

Dividing by the scale, adding the zero point, and rounding gives $1.27/0.008976-14 = 127.48 \to 127$, $-0.638/0.008976 - 14 = -85.08 \to -85$, $0.253/0.008976 - 14 = 14.18 \to 14$, and $-1.019/0.008976 - 14 = -127.52 \to -128$.

The input ranges from -0.3 to 0.5, so $S_x = 0.8/255 = 0.003137$ and $Z_x = \text{round}(-128 + 0.3/0.003137) = -32$. Then $0.5/0.003137 - 32 = 127.4 \to 127$ and $-0.3/0.003137 - 32 = -127.6 \to -128$.

$$Q_W = \begin{bmatrix} 127 & -85 \\ 14 & -128 \end{bmatrix}, \qquad Q_x = \begin{bmatrix} 127 & -128 \end{bmatrix}$$

Subtracting the zero points gives the integer operands, $Q_x - Z_x = [159, -96]$ and, row by row, $Q_W - Z_W = [141, -71]$ and $[28, -114]$. The accumulators and the rescale by $S_x S_W = 2.816 \times 10^{-5}$ give the two outputs in table 16.1, alongside the exact float result.

Table 16.1: One INT8 linear layer traced against exact FP32 arithmetic.

Output	Exact FP32	Accumulator	Rescaled INT8 result	Error
y_0	$0.5(1.27) + (-0.3)(-0.638) = 0.8264$	$159(141) + (-96)(-71) = 29{,}235$	$29{,}235 \times 2.816 \times 10^{-5} = 0.8233$	-0.0031
y_1	$0.5(0.253) + (-0.3)(-1.019) = 0.4322$	$159(28) + (-96)(-114) = 15{,}396$	$15{,}396 \times 2.816 \times 10^{-5} = 0.4336$	$+0.0014$

The first output is off by 0.003, about a third of a percent, and the trace shows where it came from. The reconstructions are $\hat{x} = [0.4988, -0.3012]$ and, for the first row of W, $[1.2657, -0.6373]$; using the unrounded grid values in the product gives approximately 0.8233. Each is within its $S/2$ bound, 0.0016 for the input and 0.0045 for the weights, and on this row their errors pushed the same direction. A packed copy of the four weights would take 4 code bytes plus 8 metadata bytes, rather than 16 FP32 bytes. The two-element bias adds its own code bytes and metadata. For the complete tiny layer, `memory_usage()` therefore reports 24 original bytes and 22 modeled bytes, only $1.09\times$ compression. Metadata becomes relatively small for large arrays; `memory_usage` on the 4096 by 4096 layer from The Problem reports 67,125,248 bytes against 16,781,328, a ratio of 4.0.

The class should first agree with the four weights traced by hand. In the listing below, line 4 defines the 2×2 weight matrix, loaded into `Linear(2, 2)` in lines 5 to 7 (transposed to match input-by-output storage convention). Line 9 wraps it in `layer = QuantizedLinear(linear)`. In lines 10 to 12, `layer.q_weight.data.T`, `layer.weight_scale`, and `layer.weight_zero_point` print the exact quantized integer codes ($[[127, -85], [14, -128]]$), scale factor (0.008976), and zero-point offset (-14) derived above.

Line 14 supplies the two-element test vector $x = [0.5, -0.3]$. Line 15 computes the baseline FP32 reference `[[0.8264, 0.4322]]`. In line 16, `layer.forward(x)` executes the dequantize-in-float path, yielding `[[0.82404, 0.43267]]`. In line 17,

`int8_forward(layer, x)` executes the pure INT8 integer accumulation path, producing `[[0.8233, 0.4336]]` matching table 16.1.

```
from tinytorch.core.layers import Linear
from tinytorch.perf.quantization import QuantizedLinear

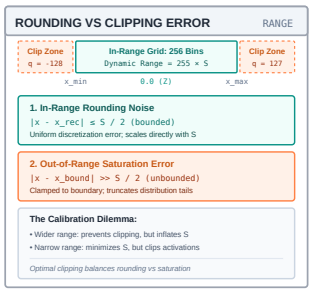
W = np.array([[1.27, -0.638], [0.253, -1.019]], dtype=np.float32)   # rows are outputs
linear = Linear(2, 2)
linear.weight.data[:] = W.T
linear.bias.data[:] = 0.0

layer = QuantizedLinear(linear)
print(layer.q_weight.data.T, layer.weight_scale, layer.weight_zero_point)
# [[ 127.  -85.]
#  [  14. -128.]] 0.008976... -14

x = Tensor(np.array([[0.5, -0.3]], dtype=np.float32))
print(x.data @ W.T)                  # [[0.8264 0.4322]]  exact
print(layer.forward(x).data)         # [[0.82404 0.43267]]  weights rounded, float multiply
print(int8_forward(layer, x))        # [[0.8233 0.4336]]  integer path: both operands rounded
```

The module's uncalibrated `forward` (line 16) rounds only the weights, so it lands closer to the exact answer on this example than `int8_forward` (line 17), which rounds both operands. The two functions model different inference paths. After `calibrate`, the module also rounds its inputs, but onto the stored grid rather than a grid measured from each new input. Comparisons must state which of these paths they used; otherwise an apparent improvement can come from changing the experiment.

The same layer illustrates this calibration state change. The samples `[[0.4, -0.2]]` and `[[0.1, 0.5]]` have a combined range of $[-0.2, 0.5]$. `calibrate` stores $S_x = 0.7/255 \approx 0.0027451$ and $Z_x = -55$. It leaves the weight and bias codes unchanged. The later input `[[2.0, -1.5]]` is outside this recorded range in both coordinates.



Quantization error decomposition: In-range discretization error is strictly bounded by half a grid step ($S/2$), whereas out-of-range values saturate at boundary codes, producing severe clipping distortion.

```
layer.calibrate([Tensor([[0.4, -0.2]]), Tensor([[0.1, 0.5]])])
x_drift = Tensor([[2.0, -1.5]])
scale, zero = layer.input_scale, layer.input_zero_point
raw_codes = np.round(x_drift.data.astype(np.float64) / scale + zero)
codes = np.clip(raw_codes, -128, 127)
print(raw_codes, codes)              # [[ 674. -601.]] [[ 127. -128.]]
print(dequantize_int8(Tensor(codes), scale, zero).data)
# [[ 0.49961 -0.20039]]
print(layer.forward(x_drift).data)   # [[0.76006 0.33064]]
print(x_drift.data @ W.T)            # [[3.497   2.0345]]
```

The large discrepancy occurs before matrix multiplication. Clamping makes `2.0` behave like approximately `0.49961` and `-1.5` like `-0.20039`. The half-step bound for values inside the calibration interval cannot protect these out-of-range inputs. `forward`

keeps the same scale afterward; it does not widen the range in response to this call. Calling `calibrate` again would replace the grid, while constructing a fresh wrapper would restore the uncalibrated path.

A calibration comparison must hold the weights fixed. The following experiment builds one model and copies it for both paths, so the only changed condition is whether the input grid was calibrated. The explicit generator initializes the weight arrays as well as the inputs; the constructor's separate random state cannot change the experiment.

```
import copy
from tinytorch.core.layers import Sequential
from tinytorch.core.activations import ReLU
from tinytorch.perf.quantization import quantize_model

rng = np.random.default_rng(0)
inputs = Tensor(rng.standard_normal((16, 4)).astype(np.float32))
calibration = [Tensor(rng.standard_normal((1, 4)).astype(np.float32))
    for _ in range(8)]
original = Sequential(Linear(4, 8), ReLU(), Linear(8, 2))
for parameter in original.parameters():
    parameter.data[:] = rng.standard_normal(parameter.shape) * 0.1
reference = original(inputs).data.copy()
for name, data in (("weights only", None), ("calibrated inputs",
    calibration)):
    model = copy.deepcopy(original)
    quantize_model(model, data)
    error = np.abs(model(inputs).data - reference).max()
    print(name, f"{error:.7f}")
# weights only 0.0006070
# calibrated inputs 0.0019063
```

The first run isolates weight rounding. The second includes rounding and possible saturation of the inputs to each layer. `_collect_layer_inputs` runs the calibration samples through the already converted prefix, so the second layer observes the output of the quantized first layer. Neither path promises a universal error ordering. To explain a larger calibrated error, inspect which inference activations fall outside the recorded ranges instead of attributing every difference to weight precision.

16.5 From TinyTorch to Production

PyTorch 2.9 provides a direct comparison through its eager-mode `torch.ao.quantization` API. The following version-specific example selects dynamic quantization for `Linear` layers. PyTorch's documentation directs new development toward torchao; the older API remains useful here because its conversion closely matches the layer replacement traced in this chapter. PyTorch 2.9 quantization documentation.

```
import torch
import torch.ao.quantization as tq

model_fp32 = torch.nn.Sequential(torch.nn.Linear(4096, 4096))
```

```
model_int8 = tq.quantize_dynamic(model_fp32, {torch.nn.Linear},
    dtype=torch.qint8)
```

The default in this exact call is per-tensor weight quantization. In the PyTorch 2.9 source, a set of layer types with `dtype=torch.qint8` selects `default_dynamic_qconfig`, whose weight observer uses `torch.per_tensor_symmetric`. Per-channel weights are a separate configuration, `per_channel_dynamic_qconfig`, which assigns a scale per output channel. That option limits the influence of an outlier to its own channel; it is not the default selected by the example. Conversion selection, weight observer, and configuration definitions make the distinction explicit.

The storage and execution changes remain the important contrast with TinyTorch. The converted layer passes packed quantized weights to a quantized backend rather than dequantizing a full float32 weight array before every multiply, as its forward method shows. Static quantization adds an offline calibration pass to choose activation grids. As in `QuantizedLinear.calibrate`, those grids describe the data observed during calibration; they cannot guarantee that future inputs stay within the same range.

An integer dot product is only one deployment design. A weight-only kernel can keep activations in floating point and decode small groups of packed weights immediately before using them. The important distinction from TinyTorch is where expansion happens. Expanding a full weight matrix into a separate float32 array creates a large temporary buffer; consuming small decoded groups within the multiply can avoid that buffer. The benefit still depends on the kernel and workload, and cannot be inferred from the code width alone.

A finer grouping also changes accuracy. Giving each output channel or small weight group its own scale prevents one outlier from coarsening the grid for the entire tensor,[61] at the cost of more metadata and rescaling work. With 4-bit codes there are only 16 levels, so this choice becomes more consequential. Training with rounding simulated in the forward pass is another option, called quantization-aware training.[62] These are extensions to the grid mechanism, not paths implemented by Module 15.

TinyTorch simulates affine quantization by mapping floats to integer steps and dequantizing back to float32 before matrix multiplication. In production acceleration runtimes (e.g., NVIDIA TensorRT, vLLM, and CUTLASS):

1. **Native Integer Execution**: Accelerators execute integer matrix multiplications ($D = A_{\text{int8}} \times B_{\text{int8}}$) directly on hardware Tensor Cores using the `mma.sync.aligned.m16n8k32.row.col` PTX instruction. Intermediate products accumulate in 32-bit integer registers (`int32`) before a single epilogue instruction applies floating-point scale factor multiplication and bias addition.
2. **FP8 and Microscaling**: Modern architectures (NVIDIA Hopper H100, Ada Lovelace, and Blackwell) extend low-precision execution to 8-bit floating point (FP8): `E4M3` (1 sign bit, 4 exponent bits, 3 mantissa bits) provides higher precision ($[-448, 448]$) for forward-pass GEMM activations and weights, while `E5M2` (1 sign bit, 5 exponent bits, 2 mantissa bits) matches FP16's wide dynamic range ($[-57344, 57344]$) to prevent underflow during backward-pass gradient accumulation. Microscaling formats (e.g., MXFP4) provide up to $4\times$ higher throughput than FP16 without requiring complex zero-point tracking.

[61] **Outlier features in large language models**: When Transformer parameter counts scale past 6.7B, emergent outlier features appear in a small subset ($< 0.1\%$) of hidden state channels, exhibiting magnitudes $20\times$ to $100\times$ larger than normal activations across all tokens (Dettmers et al., 2022, *LLM.int8()*). Per-tensor or per-token quantization causes severe perplexity degradation because scaling for the outlier compresses all other channels into zero or one bit of resolution. This behavior is strongly coupled to normalization layers (Chapter 13), where LayerNorm standardizes variance across dimensions but leaves specific feature coordinates free to concentrate scale. Production runtimes resolve this via mixed-precision decomposition: outlier coordinate channels are routed to standard FP16 GEMM, while the remaining $> 99.9\%$ are safely executed on INT8 tensor cores.

[62] **Straight-through estimator**: the rule that lets gradients flow through a rounding step during training. Rounding has zero derivative almost everywhere, so backpropagation would stop at it; the estimator pretends the derivative is one, so the optimizer can update the underlying floating-point weights despite the discrete forward operation. The backward rule is an approximation to guide training, not the derivative of rounding.

Table 16.2: Storage and execution are separate choices.

	TinyTorch `QuantizedLinear`	PyTorch 2.9 dynamic Linear	Weight-only design
Weight storage	INT8-range codes in float32 arrays	Packed quantized weights	Packed codes with group metadata
Input grid	Optional fixed calibration grid	Chosen during inference	Activations stay floating point
Multiply	Reconstruct full weight array, then float32 matmul	Quantized backend operation	Decode weight groups as needed
Evidence of benefit	Rounding error and modeled bytes	Requires workload measurements	Requires workload measurements

The comparison in table 16.2 keeps two contracts separate. The grid defines which values can be reconstructed and what rounding or saturation changes. The execution format defines which bytes are allocated and which arithmetic the kernel performs. TinyTorch provides a readable reference for the first contract. Its modeled storage report must not substitute for a measurement of the second.

Three diagnostic principles guide empirical quantization. When a quantized model loses accuracy, find the tensor whose largest magnitude is far from its typical magnitude, because that is where the grid is too coarse, and ask whether per-row or per-group scales fix it before reaching for training. When a quantized model is not faster, check whether the kernel actually consumes integers or dequantizes first, and whether the workload was memory-bound to begin with; smaller stored weights alone do not establish a speedup for a multiply limited by arithmetic. Check how far inference activations wander outside the calibration range before freezing that grid.

16.6 Summary

`quantize_int8` returns integer-range codes plus a scale and zero point without changing the original tensor. `dequantize_int8` reconstructs their float32 values. For a value within the observed range, the reconstruction error is bounded by half a grid step, apart from floating-point rounding. The bound concerns that value, not the network's output, and it does not protect inputs outside a frozen calibration range.

`QuantizedLinear` snapshots weight and bias codes at construction. Calibration changes only its input-grid state; forward reconstructs the operands, multiplies in float32, and returns a new output. `quantize_model` replaces layers in container order, using inference-mode activations from the already converted prefix to calibrate each replacement. The separate integer trace explains how scales can be factored out of dot products, but it is not the module's execution path. Packed code bytes and metadata are modeled; the Tensor arrays still occupy float32 storage. Chapter 17 changes the other factor in the storage calculation, the number of entries a layer needs, rather than the precision of each entry.

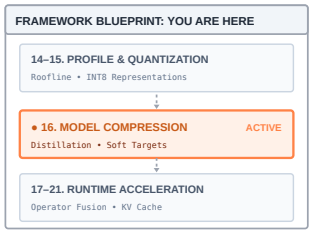
16.7 Further Reading

- **Jacob et al.** (2018). "Quantization and training of neural networks for efficient integer-arithmetic-only inference." *CVPR*. The affine scheme with a scale and zero point that this chapter implements, and the integer arithmetic it enables.
- **Nagel et al.** (2021). "A white paper on neural network quantization." *arXiv*. Post-training quantization in practice, including per-channel scales and outliers.
- **Gholami et al.** (2021). "A survey of quantization methods for efficient neural network inference." *arXiv*. The wider design space, from uniform grids to mixed precision.

🔥 Chapter 17

Model Compression: Pruning, Low-Rank SVD, and Distillation

Quantization shrinks each stored number; compression reduces how many numbers a model keeps or computes with. Pruning zeroes the weights or whole neurons that matter least, low-rank factorization replaces a weight matrix with two thin ones, and knowledge distillation trains a small student to match a large teacher's full output distribution, including the probabilities it assigns to wrong classes.



Framework Blueprint: Module 16 transfers teacher dark knowledge to a compact student network via softened probability distributions.

In PyTorch, applying magnitude-based pruning is an effortless one-liner:

```
import torch
import torch.nn.utils.prune as prune

layer = torch.nn.Linear(128, 128)

# Prune 50% of connections with smallest magnitude
prune.l1_unstructured(layer, name="weight", amount=0.5)

print("Sparsity ratio:", (layer.weight == 0).float().mean().item())  # 0.5
print("Tensor storage shape:", layer.weight.shape)                   # (128, 128)
```

Underneath that simple call, the systems reality of compression reveals critical nuances:

1. **The dense array illusion:** A pruned tensor with 50% zeros in a dense float32 array still occupies exactly the same memory footprint and performs the same number of arithmetic multiplications. Dense GEMM kernels do not check for zeros before multiplying.
2. **Mask indirection overhead:** Pruning frameworks maintain auxiliary binary masks alongside original weights, temporarily *increasing* memory consumption until weights are pruned permanently.
3. **Structured vs. low-rank dimension reduction:** To achieve actual speedups without specialized sparse hardware, compression must either prune whole structured channels or factorize weight matrices via low-rank SVD ($W \approx UV$), physically reducing matrix dimensions and arithmetic counts.

This chapter implements TinyTorch's compression tools (pruning, SVD factorization, and knowledge distillation) and measures what each one changes. Before building these techniques, the first step is to inspect the systems trap of dense zeroed weights.

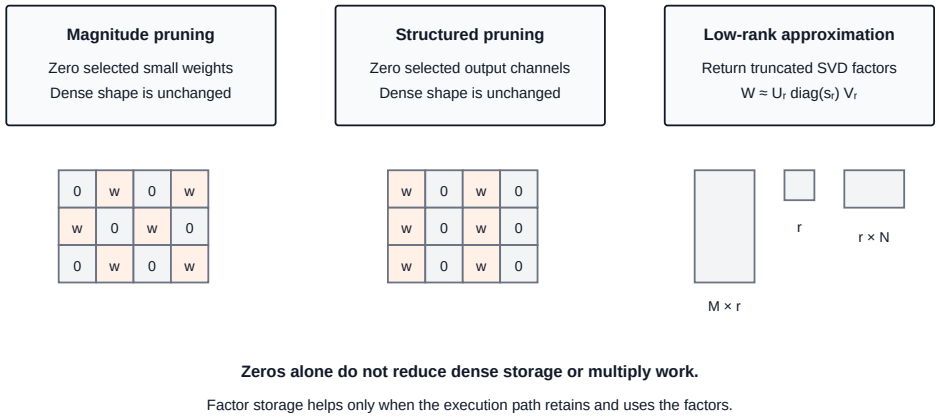


Figure 17.1: **Compression representations**: Both pruning functions preserve dense array dimensions. Low-rank approximation returns three factors; a caller must retain and use them to benefit from their smaller storage.

17.1 The Problem

↩ **Jump back:** Chapter 4 *Temperature-scaled Kullback-Leibler (KL) divergence transfers dark knowledge from teacher to student.*

Start with weights `[[1, 0.1, 3], [-2, 0.2, 1]]`. Removing the three smallest magnitudes zeros `0.1`, `0.2`, and the first `1`; a stable ranking resolves the tie between the two entries of magnitude one. The matrix remains 2×3, and `x @ W` still computes three outputs. A small magnitude is a candidate for removal, but its effect also depends on the input and the weights that follow it.

Do that to a 4096×4096 layer and then look at what the computer sees. The array still has 16.8 million cells. In FP32 it still occupies 67.1 MB, and a packed INT8 representation would still occupy 16.8 MB. A zero takes the same number of bytes as any other value. The forward pass $Y = XW$ still performs $2 \times 4096 \times 4096 \approx 33.5$ million floating-point operations per token (counting a multiply and an addition separately), because the multiply loop does not check for zeros before it multiplies. The dense operation count and allocation are unchanged; elapsed time remains a measurement for Chapter 15's profiler.

Listing 17.1 Zeroing leaves dense storage unchanged: The assertions check the state contract described in the surrounding text.

```python
import numpy as np

W = np.array([[1.0, 0.1, 3.0], [-2.0, 0.2, 1.0]], dtype=np.float32)
before = W.nbytes
order = np.argsort(np.abs(W).ravel(), kind="stable")
W.flat[order[:3]] = 0
assert np.array_equal(W, [[0, 0, 3], [-2, 0, 1]])
assert W.shape == (2, 3) and W.nbytes == before == 24
```

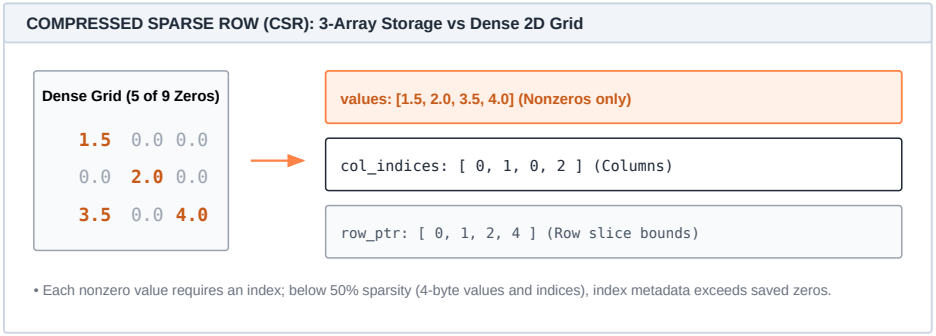
[63] **Compressed sparse row (CSR)**: a format storing nonzeros with column indices and row start pointers. It saves memory only when fewer than half the entries are nonzero, because each stored value carries an index.

A sparse format such as compressed sparse row[63] stores nonzero values with their

positions. At 80 percent sparsity, the large matrix needs about 3.4 million values and as many column indices. With four bytes for each value and index, this is about 26.8 MB, plus the row-position array. The value-only estimate of a $5\times$ reduction becomes roughly $2.5\times$ after indexing. A sparse multiplication must interpret these positions, so its runtime depends on the pattern, batch size, and implementation. TinyTorch's dense NumPy multiplication does not consume such a format. Deleting whole hidden units provides another route: rebuild smaller dense arrays that the existing `Linear` can use.

Pause & Predict: The Breakeven SVD Rank

For a linear layer weight matrix $W \in \mathbb{R}^{M\times N}$, low-rank factorization replaces W with $W_A \in \mathbb{R}^{M\times r}$ and $W_B \in \mathbb{R}^{r\times N}$. This changes the parameter count from $M \times N$ to $r(M+N)$. For a square projection matrix where $M = N = 4096$, what is the maximum rank r that yields any memory reduction? What rank r provides a $2\times$ reduction in parameters? If you choose rank $r = 512$, what percentage of weight parameters are eliminated?



Core Invariant: CSR Index Metadata Breakeven

$\text{Memory}_{\text{CSR}} = \text{NNZ}\times(\text{sizeof}(V)+\text{sizeof}($

Because each nonzero requires an explicit column index, CSR with equal-width values and indices saves memory only when sparsity exceeds 50% (plus the row pointers); narrower values, such as FP16 with 32-bit indices, move the breakeven toward 67%.

Figure 17.2: **Compressed Sparse Row (CSR) 3-array representation**: Storing sparse matrices requires three coordinated buffers: nonzero `values`, corresponding `col_indices`, and `row_ptr` slicing bounds. Below 50% sparsity with equal-width values and indices, column index metadata outweighs the memory saved by omitting zero elements.

17.2 The Idea

The three weight transformations make different commitments. A mask identifies values to remove, neuron deletion changes neighboring shapes together, and factorization changes the multiplication itself.

↪ **Jump ahead:** Chapter 21 *Integrating distilled student models into the final production optimization pipeline.*

17.2.1 Unstructured pruning: zero the small weights

In sparse deep learning libraries and model compression pipelines (such as PyTorch's pruning utilities and SparseGPT), unstructured pruning identifies and zeroes out individual weights with minimal impact on model outputs. While this fine-grained sparsity drastically reduces nonzero parameters for compressed storage, standard GEMM execution runtimes cannot translate scattered zeros into latency speedups without specialized

hardware support. Magnitude pruning picks a threshold θ and replaces every weight with

$$W_{\text{sparse}} = W \odot M, \qquad M_{i,j} = \begin{cases} 1 & |W_{i,j}| \geq \theta \\ 0 & |W_{i,j}| < \theta \end{cases}$$

where $\odot$ multiplies element by element and M is the binary mask. A percentile threshold can miss the intended count when magnitudes tie. The module instead ranks entries and selects the requested number, with a stable tie order. On its own it does not change the shape, so the dense allocation and arithmetic count are unchanged. Without specialized hardware support (such as NVIDIA Ampere 2:4 structured sparse Tensor Cores), unstructured zero values still traverse the memory bus and consume ALU cycles, failing to shift the Roofline curve (Chapter 15). Modern LLM pruning algorithms address accuracy rather than this hardware limitation: Wanda (Sun et al., 2023) scores weights by $|W_{ij}| \cdot \|X_j\|_2$, and SparseGPT (Frantar and Alistarh, 2023) uses an approximate second-order reconstruction, both preserving perplexity at high sparsity. The module still implements plain magnitude pruning because it makes the storage and arithmetic contract easy to check.

17.2.2 Structured pruning: delete whole neurons

To achieve immediate inference speedups on standard hardware without relying on sparse runtime engines, production optimizers like Torch-Pruning employ structured pruning to eliminate entire channels, attention heads, or hidden neurons. By physically slicing out structural dimensions rather than masking individual entries, this technique yields smaller dense matrices that directly reduce memory traffic and FLOP counts in off-the-shelf BLAS libraries. Chapter 3's `Linear` stores its weight as inputs by outputs and computes $y = xW + b$, so a hidden neuron in a two-layer network is one column of the first weight matrix, one entry of its bias, and one row of the second weight matrix. Delete all three together and the network still computes a valid function, just with one fewer hidden unit. If W_1 is $D \times H$ and W_2 is $H \times K$, removing neuron j turns them into $D \times (H-1)$ and $(H-1) \times K$. Both matrices are now smaller dense arrays, and the ordinary matrix multiply from Chapter 3 runs on them unchanged.

The question is which neurons to delete. The simplest score is the norm of the neuron's incoming weights, the length of column j of W_1 measured either as a sum of absolute values (L1) or as the root of a sum of squares (L2); Module 16 uses L2, the deletion function in The Code uses L1, and on the worked trace they agree. For bounded inputs, a column of small weights contributes little to the preactivation, but its bias can still produce a large activation. The score ignores that bias and does not establish that removal is harmless. Ranking columns by that norm and keeping the top fraction is the whole algorithm. It is a crude score, because it ignores how much the next layer relies on that neuron, and the worked trace below shows exactly when it misjudges.

17.2.3 Low-rank factorization: replace the matrix with two thin ones

In model compression and efficient parameter adaptation—ranging from SVD layer decomposition to modern low-rank adaptation (LoRA)—low-rank factorization decomposes large projection matrices into pairs of narrow matrices. By projecting activations through a low-dimensional bottleneck subspace, this architecture reduces both parameter storage and floating-point operations while preserving the principal transformation

dynamics. For a concrete example, the 3×3 matrix filled with $4/3$ can be represented by a column $[1, 1, 1]^T$ times a row $[4/3, 4/3, 4/3]$. Six stored numbers reproduce nine entries because all columns point in the same direction. Its rank is one, the number of independent column directions. Low-rank factorization seeks a few such directions that reproduce a more general matrix approximately.

Every $M \times N$ matrix can be written as a sum of simple pieces. The **singular value decomposition** says

$$W = U\Sigma V^T = \sum_{i=1}^{\min(M,N)} \sigma_i \, u_i v_i^T$$

where each u_i is a column of U (length M), each v_i is a column of V (length N), the columns of U are mutually perpendicular unit vectors and so are the columns of V, and Σ is a diagonal matrix holding the **singular values** $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$. Each term $u_i v_i^T$ is an outer product, a column times a row, which is an $M \times N$ matrix of rank one. The SVD sorts the pieces so the largest come first.

Keeping only the first r terms gives the rank-r approximation

$$W \approx W_r = \sum_{i=1}^{r} \sigma_i \, u_i v_i^T = W_A W_B,$$
$$W_A = U_r \sqrt{\Sigma_r} \in \mathbb{R}^{M \times r}, \qquad W_B = \sqrt{\Sigma_r} V_r^T \in \mathbb{R}^{r \times N}.$$

where U_r holds the first r columns of U, V_r the first r columns of V, and $\sqrt{\Sigma_r}$ splits each σ_i evenly across the two factors. The split is a convention that keeps the two factors at similar scale. For a `Linear` weight stored as inputs by outputs, M is the input width and N the output width, and the layer becomes two linear layers in sequence, one with weight W_A that maps M inputs to r, then one with weight W_B that maps r to N outputs. Chapter 23 describes a related but different use of thin matrices, Low-Rank Adaptation (LoRA), which freezes the base weights and trains a separately initialized low-rank update rather than an SVD of W.

Two facts make this useful. First, the Eckart-Young theorem (1936) says W_r is the best rank-r approximation there is, and its error is exactly the part you dropped:

$$\|W - W_r\|_F = \sqrt{\sum_{i=r+1}^{\min(M,N)} \sigma_i^2}$$

where $\|\cdot\|_F$ is the Frobenius norm.[64] You can read the error off the singular values before deciding on r. Second, the parameter count changes from $M \times N$ to $r \times (M + N)$, and so does the multiply count, because computing xW_A and then multiplying the result by W_B costs $2r(M + N)$ operations instead of $2MN$.[65]

[64] **Frobenius norm**: the square root of the sum of the squares of every entry, which is the ordinary vector length applied to a matrix as if it were one long vector. For the 3×3 matrix in the worked trace, the Frobenius norm equals $\sqrt{18} \approx 4.24$, so a reconstruction error of 1.41 is one third of the norm but only $2/18 \approx 11\%$ of the matrix energy (the squared norm).

[65] **Activation traffic in low-rank factorization**: While factorizing $W \in \mathbb{R}^{M \times N}$ into $W_A \in \mathbb{R}^{M \times r}$ and $W_B \in \mathbb{R}^{r \times N}$ strictly reduces parameter storage and multiply-accumulate FLOPs for $r < \frac{MN}{M+N}$, it introduces an intermediate hidden activation tensor $h = xW_A \in \mathbb{R}^{B \times r}$ for batch size B. In unfused execution runtimes, h must be written to DRAM or L2 cache after the first GEMM and reloaded for the second GEMM. If B is large, the memory bandwidth consumed by streaming intermediate activations can offset the weight traffic savings, making operator fusion of xW_AW_B or kernel chaining essential.

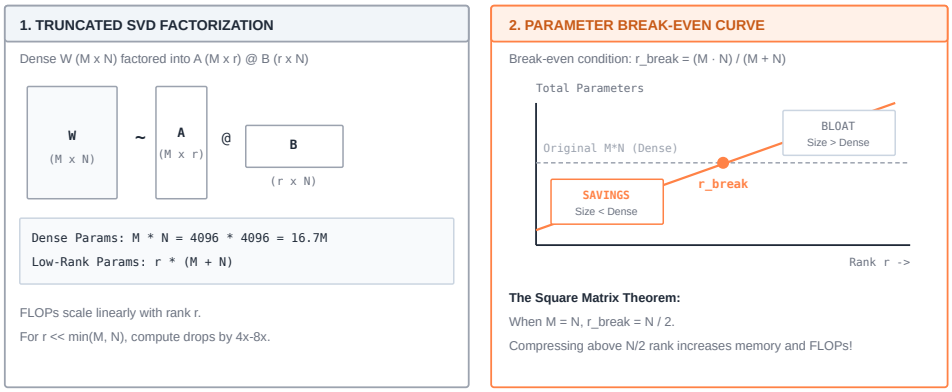
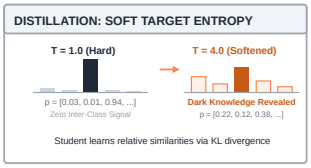


Figure 17.3: **Low-rank SVD factorization and breakeven rank**: A weight matrix $W \in \mathbb{R}^{M\times N}$ is factorized into two thin matrices $W_A \in \mathbb{R}^{M\times r}$ and $W_B \in \mathbb{R}^{r\times N}$. The scree plot (left) reveals exponential decay of singular values, indicating low effective rank. Stored parameters drop from $M \times N$ to $r(M + N)$, achieving real storage and FLOP savings whenever chosen rank $r < \frac{MN}{M+N}$ (right).

17.2.4 Distillation: teaching the small model from the large one



Soft target entropy: higher temperature $T = 4$ uncovers dark knowledge in non-target class distributions.

In production model deployment pipelines—such as compressing foundation models into edge-ready variants like DistilBERT or mobile vision backbones—knowledge distillation recovers accuracy lost during structural pruning or architecture downsizing. By supervising a compact student model with the softened probability distributions of a high-capacity teacher, the training process transfers dark knowledge about inter-class similarities that one-hot labels discard. Pruning and factorization both approximate the original computation and can reduce accuracy. The usual repair is to train the smaller model for a while afterward, and one way to guide that training is **knowledge distillation** (Hinton, Vinyals, and Dean, 2015). Instead of training the small *student* only on the true labels, you also train it to match the full output distribution of the large *teacher*.

The loss has two parts. The hard part is the ordinary cross-entropy from Chapter 4 between the student's prediction and the true label. The soft part compares the student's and teacher's softmax outputs after both sets of logits are divided by a **temperature** $T > 1$, which flattens the distributions so the small probabilities become visible. The comparison is the Kullback-Leibler divergence, $\sum_k p_k \log(p_k/q_k)$, which is zero when the two distributions agree and grows as they separate. A weight α mixes the two:

$$\mathcal{L} = \alpha\,\mathrm{KL}\left(p^{(T)}_{\text{teacher}} \,\|\, p^{(T)}_{\text{student}}\right) + (1-\alpha)\,\mathrm{CE}(p_{\text{student}}, y)$$

Knowledge distillation trains a compact student model, but the training run must host two networks simultaneously:

- **Teacher State**: The full-scale teacher model (e.g., 12-layer GPT, 120M parameters) runs in inference mode. While gradients and optimizer moment buffers are omitted, its weights ($120\text{M} \times 4\text{ B} = 480$ MB) and forward activations must reside in memory.
- **Student State**: The compact student model (e.g., 4-layer GPT, 15M parameters) requires weights (60 MB), gradients (60 MB), AdamW moment buffers (120 MB),

and activations whose size depends on batch and sequence length (for example, an assumed 120 MB).

- **Logit Buffers**: Both teacher and student produce vocab-sized logit arrays (B, S, V). For $B = 16, S = 1024, V = 32000$, each logit tensor spans $16 \times 1024 \times 32000 \times 4$ B ≈ 2.1 GB.

Dual forward passes and high-dimensional KL divergence evaluations require substantial temporary memory headroom before the compressed student can be deployed standalone.

Module 16 defaults to $T = 3$ and $\alpha = 0.7$. Both terms average over examples after summing over classes, so duplicating a batch leaves the objective unchanged. The soft term here omits the optional temperature-squared multiplier; changing temperature therefore changes both the target distribution and its gradient scale.[66] The training loop still uses Chapter 8's forward, backward, and update sequence, with the teacher held fixed.

17.3 The Code

Everything below works on Chapter 3's `Linear` and `Sequential` and Chapter 6's `Tensor`. `Linear` keeps its weight as an `(in_features, out_features)` array and computes `x @ weight + bias`, and every shape comment follows that layout.

17.3.1 Measuring and creating unstructured sparsity

Every compression claim starts with a number. A paper reports that a model is 80 percent sparse, a pruning pass has to prove it removed what it was asked to, and the 2:4 recipe in the production bridge begins by asking which two weights in every group of four are smallest. All of that needs one function that counts zeros and one that makes them.

The obvious count divides zeros by all parameters, biases included. Chapter 3's `Linear` initializes its biases to zero, so on a fresh two-layer model with 64 inputs, 128 hidden units, and 10 outputs, that count reports 1.4 percent sparsity, 138 zeros out of 9,610 parameters, before anything has been pruned. That number describes all-parameter sparsity, but it does not measure the weight sparsity this chapter is trying to change. The obvious pruning pass has a subtler flaw. It thresholds each layer on its own, which forces exactly 80 percent out of every layer whether that layer's weights are uniformly tiny or uniformly large.

The module counts weight matrices only and ranks their entries together, so that a layer whose weights are small gives up more of them than a layer whose weights are large.

```
def measure_sparsity(model) -> float:
    """
    Calculate the percentage of zero weights in a model.
    """
    total_params = 0
    zero_params = 0

    for param in model.parameters():
        # Only count weight matrices (2D), not biases (1D)
```

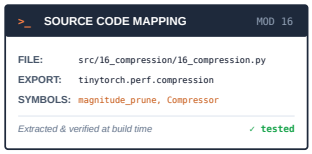
[66] **The T^2 scaling factor in knowledge distillation**: In the high-temperature limit ($T \to \infty$), the Taylor expansion of softened softmax probabilities yields $p_i \approx 1/K + (z_i - \bar{z})/(KT)$. Differentiating the KL divergence between teacher and student softened distributions with respect to student logit z_i produces $\partial \mathcal{L}_{\text{KL}}/\partial z_i \approx (1/(KT^2))(z_i - v_i)$, where v_i is the teacher logit. Because the gradient magnitude decays quadratically $(1/T^2)$ as logits are flattened, Hinton et al. (2015) multiply the soft loss term by T^2 to keep the gradient scale and the balance between hard cross-entropy and soft distillation invariant to temperature adjustments. TinyTorch intentionally omits T^2 to preserve code transparency, requiring users to adjust α or learning rate when modifying T.

```
        # Biases are often initialized to zero, which would skew
        ↪ sparsity
        if len(param.shape) > 1:
            total_params += param.size
            zero_params += np.sum(param.data == 0)

    if total_params == 0:
        return 0.0

    return (zero_params / total_params) * 100.0
```

The `len(param.shape) > 1` test is the whole bias exclusion, and the result is a percentage, not a fraction; on the fresh model above it reports `0.0`. Pruning gathers the same matrices, selects the smallest `int(sparsity * count)` entries globally, and zeros those entries in place.



Source Code Mapping: Module 16 extracts knowledge distillation loss, magnitude pruning, and low-rank approximation directly from the working repository.

```
def magnitude_prune(model, sparsity: float = 0.9):
    """
    Remove weights with smallest magnitudes to achieve target
    ↪ sparsity.
    """
    if not np.isfinite(sparsity) or not 0 <= sparsity <= 1:
        raise ValueError("sparsity must be between 0 and 1")
    weight_params = [p for p in model.parameters() if p.ndim > 1]
    if not weight_params:
        return model

    # Rank globally, including existing zeros. Selecting by index also
    ↪ handles
    # tied magnitudes and the endpoints 0% and 100% exactly.
    magnitudes = np.concatenate([np.abs(p.data).ravel() for p in
    ↪ weight_params])
    prune_count = int(sparsity * magnitudes.size)
    prune_mask = np.zeros(magnitudes.size, dtype=bool)
    prune_mask[np.argsort(magnitudes, kind="stable")[:prune_count]] =
    ↪ True
    offset = 0
    for param in weight_params:
        mask = prune_mask[offset:offset +
    ↪ param.size].reshape(param.shape)
        param.data[mask] = 0
        offset += param.size
    return model
```

The stable global sort gives a deterministic rule for ties. `prune_mask` has one position per weight across the model; the loop slices that flat mask back into each parameter's original shape using `offset`. Assigning through `param.data[mask]` changes the existing arrays, and the return value is the same model object. Biases are untouched. Existing zeros participate in the ranking, so a request below the current sparsity does not restore weights. These are the invariants a caller can check without measuring accuracy or speed.

The mask is temporary: an ordinary optimizer update can make a zeroed weight nonzero again because this function installs no persistent training mask.

17.3.2 Structured pruning: deleting neurons across a layer pair

In the two-input, three-hidden-unit pair used below, removing the middle hidden unit means removing column 1 of the first weight and row 1 of the second. Zeroing only the first column and its bias suppresses that unit under ReLU, but the arrays still have their original shapes. This is the distinction between Module 16's pruning operation and the book's deletion extension.

Zeroing their weights is `magnitude_prune` again, with the zeros arranged in whole columns instead of scattered. That is what the module's `structured_prune` does, and it does it on purpose, so that `measure_sparsity` can see its work and so that the next step, the shape change, is a separate decision.

```
def structured_prune(model, prune_ratio: float = 0.5):
    """
    Remove entire channels/neurons based on L2 norm importance.
    """
    if not np.isfinite(prune_ratio) or not 0 <= prune_ratio <= 1:
        raise ValueError("prune_ratio must be between 0 and 1")
    # Prune the hidden Linear layers. The last Linear is the head: its output
    # channels are the classes, so zeroing them removes classes, not neurons.
    # A model with a single Linear has nothing else to prune and is pruned as is.
    def collect_linears(layer):
        # Sequential containers may contain other Sequential containers. Walk
        # their execution order so the terminal classifier is identified once.
        if isinstance(layer, Linear):
            return [layer]
        return [linear for child in getattr(layer, 'layers', [])
                for linear in collect_linears(child)]

    linears = collect_linears(model)
    hidden = linears[:-1] if len(linears) > 1 else linears

    for layer in hidden:
        weight = layer.weight.data

        # Calculate L2 norm for each output channel (column)
        channel_norms = np.linalg.norm(weight, axis=0)

        # Find channels to prune (lowest importance)
        num_channels = weight.shape[1]
        num_to_prune = int(num_channels * prune_ratio)

        if num_to_prune > 0:
            # Get indices of channels to prune (smallest norms)
```

```
            prune_indices = np.argsort(channel_norms,
    kind="stable")[:num_to_prune]

            # Zero out entire channels
            weight[:, prune_indices] = 0

            # Also zero corresponding bias elements if bias exists
            if layer.bias is not None:
                layer.bias.data[prune_indices] = 0

    return model
```

The routine walks nested `Sequential` containers in execution order to collect their `Linear` layers. It then excludes the final linear layer when there is more than one, preserving the classifier head. A standalone or single-layer linear model is the explicit exception and is pruned directly. The separate `compress_model` convenience path validates its configuration before mutation and rejects low-rank replacement because rebuilding layers is not implemented there.

For each selected layer, `np.linalg.norm(weight, axis=0)` reduces over input rows and leaves one L2 score per output column. `int(num_channels * prune_ratio)` sets the removal count, and stable `argsort` picks the lowest scores, including a deterministic order for ties. The assignments zero those columns and the matching bias entries. No outgoing rows are removed. With ReLU next, the zeroed preactivation becomes zero; with a sigmoid it becomes 0.5, so the same mask would not silence the unit.

The shape change must preserve how the two layers connect. A hidden unit occupies one column of W_1, one entry of b_1, and one row of W_2. The book function below takes both layers, retains the same ordered neuron indices in all three places, and constructs a new pair. On a $4096 \to 16384 \to 4096$ pair, retaining 12,288 hidden units changes the weights to 4096×12288 and 12288×4096. Their total weight count drops by 33,554,432, or about 134 MB in float32, without a sparse decoder.

Listing 17.2 Deleting a neuron across adjacent layers: Matching survivor indices keep the two new layers connected without sharing parameters with the original pair.

```
from tinytorch.core.tensor import Tensor
from tinytorch.core.layers import Linear, Sequential

def prune_neurons(first: Linear, second: Linear, keep: int):
    """Delete hidden neurons between two Linear layers; return the new pair. Not in Module 16."""
    if first.out_features != second.in_features:
        raise ValueError("Hidden dimensions must match")
    if not isinstance(keep, (int, np.integer)) or not 1 <= keep <= first.out_features:
        raise ValueError("keep must be an integer within the hidden width")
    w1, w2 = first.weight.data, second.weight.data      # [D, H] and [H, K]
    scores = np.sum(np.abs(w1), axis=0)                 # L1 norm of each column
    survivors = np.sort(np.argsort(scores, kind="stable")[-keep:])  # top-`keep`, in order

    new_first = Linear(first.in_features, keep, bias=first.bias is not None)
    new_first.weight = Tensor(w1[:, survivors].copy(), requires_grad=True)
    if first.bias is not None:
        new_first.bias = Tensor(first.bias.data[survivors].copy(), requires_grad=True)
    new_second = Linear(keep, second.out_features, bias=second.bias is not None)
    new_second.weight = Tensor(w2[survivors, :].copy(), requires_grad=True)
    if second.bias is not None:
        new_second.bias = Tensor(second.bias.data.copy(), requires_grad=True)
    return new_first, new_second
```

`np.sort` on the survivor indices keeps the neurons in their original order. Advanced indexing and the Tensor constructor create new storage; copying the output bias as well ensures that training the returned pair cannot mutate the original pair. The module scores by L2 and this function by L1, and both select the same middle neuron in the worked trace. Construct a new optimizer for the returned parameters before recovery training, since an existing optimizer still holds the original objects.

17.3.3 Low-rank factorization

Column scores can miss redundancy between columns. The all-$4/3$ matrix introduced earlier has no zero column, yet one direction describes it exactly. The same possibility motivates examining the singular values of a larger trained projection. The reconstruction

error determines which directions can be discarded, while storage arithmetic determines whether the retained factors are smaller than the original matrix.

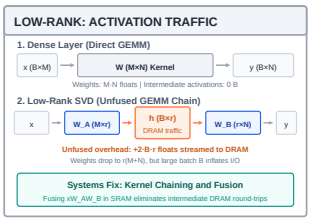
Factorizing $W \in \mathbb{R}^{M\times N}$ into $W_A \in \mathbb{R}^{M\times r}$ and $W_B \in \mathbb{R}^{r\times N}$ saves parameters only when

$$r \times (M+N) < M \times N \quad \Longleftrightarrow \quad r < \frac{MN}{M+N}.$$

For a square 4096×4096 layer the break-even rank is $4096^2/8192 = 2048$, half the full rank. At $r = 256$ the count drops from 16,777,216 to $256 \times 8192 = 2{,}097{,}152$, an $8\times$ reduction in weight entries and matrix-multiplication arithmetic. These counts describe the two folded factors, excluding the unchanged bias and the new intermediate activation. Whether the accuracy survives depends on how fast the singular values of that particular layer decay, which is a property of the trained weights, not of the method.

The naive way to choose r is trial. Pick a rank, factorize, retrain, measure, and repeat, with a training run per guess. Eckart-Young removes the guessing from the reconstruction side. One SVD returns every singular value, and the error at every r reads off the tail of that list before a single factor is built. Whether the accuracy survives is still a question for fine-tuning, but you go into it knowing how much of the matrix you dropped.

The module's function does the SVD and the truncation and stops there.



Intermediate activation traffic: factorizing a layer saves weight parameters but writes and reloads intermediate tensor h, requiring operator fusion when batch size B is large.

```python
def low_rank_approximate(
    weight_matrix: np.ndarray, rank_ratio: float = 0.5
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Approximate weight matrix using low-rank decomposition (SVD).
    """
    if not np.isfinite(rank_ratio) or not 0 < rank_ratio <= 1:
        raise ValueError("rank_ratio must be in (0, 1]")
    m, n = weight_matrix.shape

    # Perform SVD. NumPy's third return value is already V^T, not V.
    U, S, Vt = np.linalg.svd(weight_matrix, full_matrices=False)

    # Determine target rank
    max_rank = min(m, n)
    target_rank = max(1, int(rank_ratio * max_rank))

    # Enforce the break-even rank r* from the Foundations section.
    ↪   Above it the
    # two factors plus the singular values hold MORE numbers than the
    ↪   dense
    # matrix they replace, so the factorization is a decompression.
    ↪   Refuse it
    # instead of returning a larger "approximation" that looks like a
    ↪   win.
    break_even_rank = (m * n) / (m + n + 1)
    if target_rank > break_even_rank:
        raise ValueError(
            f"rank_ratio={rank_ratio} gives rank {target_rank} for a
            ↪   {m}x{n} matrix, "
```

```
            f"above the break-even rank r* = {break_even_rank:.1f}.
            ↪  The factors would "
            f"hold {target_rank * (m + n + 1):,} parameters against {m
            ↪  * n:,} dense. "
            f"Use rank_ratio <= {break_even_rank / max_rank:.3f}."
        )

    # Copy the slices so they release the full SVD backing arrays.
    ↪  Views
    # would keep the uncompressed factors alive despite their smaller
    ↪  shapes.
    U_truncated = U[:, :target_rank].copy()
    S_truncated = S[:target_rank].copy()
    Vt_truncated = Vt[:target_rank, :].copy()

    return U_truncated, S_truncated, Vt_truncated
```

The function validates `rank_ratio` in $(0, 1]$, computes `max(1, int(rank_ratio * min(M, N)))`, and raises an error if that rank exceeds the three-factor break-even rank $MN/(M + N + 1)$. This fraction refers to the maximum possible rank, not the matrix's measured numerical rank. A ratio of 0.5 on a 3×3 matrix therefore keeps one term. NumPy returns singular values in descending order and its third result is already V^T, so the function selects columns of U, entries of `S`, and rows of V^T. Each slice is copied. A view would keep the full SVD backing array alive and defeat the storage reduction.

The returned arrays have shapes (M, r), $(r,)$, and (r, N) and own $r(M + N + 1)$ values. This three-factor representation saves storage when $r < MN/(M + N + 1)$. It does not modify the input matrix or any layer. The book extension below folds the singular values into two factors, reducing that count to $r(M + N)$, and places the original bias after the second multiplication. Reconstructing the full matrix first would lose the storage benefit.

Listing 17.3 Building a forward path from SVD factors: Two new linear layers consume the folded factors and preserve the output bias.

```
from tinytorch.perf.compression import low_rank_approximate

def factorize_linear(layer: Linear, rank_ratio: float):
    """Replace one Linear(N -> M) with Linear(N -> r) then Linear(r ->
    ↪   M). Not in Module 16."""
    U, S, Vt = low_rank_approximate(layer.weight.data, rank_ratio)   #
    ↪   weight is [N, M]
    root = np.sqrt(S)
    inner = Linear(layer.in_features, len(S), bias=False)
    inner.weight = Tensor(U * root, requires_grad=True)              #
    ↪   [N, r], column i scaled by sqrt(sigma_i)
    outer = Linear(len(S), layer.out_features, bias=layer.bias is not
    ↪   None)
    outer.weight = Tensor(root[:, None] * Vt, requires_grad=True)    #
    ↪   [r, M], row i scaled by sqrt(sigma_i)
    if layer.bias is not None:
        outer.bias = Tensor(layer.bias.data.copy(),
    ↪   requires_grad=True)
    return Sequential(inner, outer)
```

Because Chapter 3's weight is inputs by outputs, `inner` maps the input width to r and `outer` maps r to the output width. Ranks above the break-even point are refused: `low_rank_approximate` raises an error rather than return factors larger than the dense matrix, so `rank_ratio=1.0` is not a way to reproduce the original layer. At an accepted rank, the omitted singular directions can change the output. Both weights and the copied output bias belong to the returned model. The original layer remains available for comparison, and the caller must replace it explicitly in the enclosing model.

17.3.4 The distillation loss

Each approximation above can alter the model's predictions. The pruned pair in the worked trace moves its output by 0.1 on a single input; a 4096×4096 layer cut to rank 256 loses whatever sits in singular values 257 through 4096. Recovery training can reduce that error, and the question is what objective to train on.

Fine-tuning the small model on the labels alone hands it one number per example, the index of the correct class. The large model knows more than that. Show it a picture and its logits might be $[6, 2, 0]$ for cat, dog, and car, which softmax turns into 0.98, 0.018, and 0.002. The ranking of the wrong answers, dog far ahead of car, is information the label throws away, and at $T = 1$ it is nearly invisible anyway. Divide the logits by $T = 3$ first and the same three become 0.715, 0.188, and 0.097, a distribution a student can actually be trained to match.

The loss has a soft term and a hard term. The soft term is the KL divergence from the teacher's tempered softmax to the student's, the hard term is Chapter 4's cross-entropy, and α mixes them. The module packages the two models and the two constants in a class whose only real method is the loss.

```
class KnowledgeDistillation:
    """
    Knowledge distillation for model compression.
    """

    def __init__(self, teacher_model, student_model, temperature=3.0,
    ↪   alpha=0.7):
        """
        Initialize knowledge distillation.
        """
        if not np.isfinite(temperature) or temperature <= 0:
            raise ValueError("temperature must be finite and positive")
        if not np.isfinite(alpha) or not 0 <= alpha <= 1:
            raise ValueError("alpha must be between 0 and 1")
        self.teacher_model = teacher_model
        self.student_model = student_model
        self.temperature = temperature
        self.alpha = alpha

    def distillation_loss(self, student_logits: Tensor,
    ↪   teacher_logits: Tensor, true_labels) -> Tensor:
        """
        Calculate a differentiable, batch-mean distillation loss.
        """
        if student_logits.ndim != 2 or student_logits.shape !=
        ↪   teacher_logits.shape:
            raise ValueError("Student and teacher logits must have
            ↪   matching (batch, classes) shapes")
        batch_size, num_classes = student_logits.shape
        if batch_size == 0 or num_classes == 0:
            raise ValueError("Distillation needs at least one sample
            ↪   and one class")
        labels = true_labels.data if isinstance(true_labels, Tensor)
↪   else np.asarray(true_labels)
        if labels.shape == (batch_size,):
            if (not np.all(np.isfinite(labels)) or np.any(labels !=
            ↪   np.floor(labels))
                    or np.any(labels < 0) or np.any(labels >=
                    ↪   num_classes)):
                raise ValueError("Class labels must be integer IDs in
                ↪   [0, num_classes)")
            labels = labels.astype(np.intp)
        elif labels.shape == student_logits.shape:
            if (not np.all(np.isfinite(labels)) or np.any(labels < 0)
                    or not np.allclose(labels.sum(axis=1), 1.0)):
                raise ValueError("Target probabilities must be
                ↪   nonnegative and sum to one per sample")
        else:
            raise ValueError("Labels must have shape (batch,) or
            ↪   (batch, classes)")

```

```
        # Only the student side is differentiable. The teacher's log-probabilities
        # are constants even when its forward pass was recorded by autograd.
        student_log_probs = log_softmax(student_logits / self.temperature)
        teacher_log_probs = log_softmax(Tensor(teacher_logits.data) / self.temperature)
        teacher_probs = Tensor(np.exp(teacher_log_probs.data))
        soft_loss = (teacher_probs * (teacher_log_probs - student_log_probs)).sum(axis=-1).mean()

        hard_log_probs = log_softmax(student_logits)
        if labels.ndim == 1:
            hard_loss = hard_log_probs[np.arange(batch_size), labels].mean() * -1.0
        else:
            hard_loss = (Tensor(labels) * hard_log_probs).sum(axis=-1).mean() * -1.0

        return self.alpha * soft_loss + (1 - self.alpha) * hard_loss

    # The three helpers below are NumPy-only and sit outside the differentiable
    # path. They exist so the 📊 analysis cell can print the soft and hard terms
    # separately at several temperatures; distillation_loss itself uses
    # log_softmax from Module 04, which already provides stable log-probabilities.
    # Keeping them here means core.compression ships two small reimplementations
    # of arithmetic Module 04 owns, which is the price of a self-contained table.
    def _softmax(self, logits):
        """Compute softmax with numerical stability (analysis only, not autograd)."""
        exp_logits = np.exp(logits - np.max(logits, axis=-1, keepdims=True))
        return exp_logits / np.sum(exp_logits, axis=-1, keepdims=True)

    def _kl_divergence(self, p, q):
        """Compute KL divergence between distributions."""
        return np.mean(np.sum(p * np.log((p + 1e-8) / (q + 1e-8)), axis=-1))

    def _cross_entropy(self, predictions, labels):
        """Compute a NumPy batch-mean cross-entropy for the analysis table."""
        # Simple implementation for integer labels
        if labels.ndim == 1:
```

```
75              return -np.mean(np.log(predictions[np.arange(len(labels)),
                ↪   labels] + 1e-8))
76          else:
77              return -np.mean(np.sum(labels * np.log(predictions +
                ↪   1e-8), axis=1))
```

Follow `distillation_loss` from its arguments to its return value. The two logit tensors must have matching `(batch, classes)` shapes. A label vector may arrive as a NumPy integer array or as a `Tensor`, whose storage converts even class IDs to float32. The method checks that each ID is finite, integral, and in range before converting it to an indexing dtype. A matrix of target probabilities takes the other branch, which checks nonnegative entries and a unit row sum. These checks distinguish a malformed target from a model that merely predicts the wrong class.

The next lines establish which model learns. `log_softmax(student_logits / self.temperature)` uses the operations from Chapter 4 and Chapter 6 and retains a path back to the student. `Tensor(teacher_logits.data)` constructs a new leaf without gradient tracking, so the teacher distribution becomes a constant even if its forward pass recorded a graph. Multiplying that constant distribution by `teacher_log_probs - student_log_probs` implements KL in the teacher-to-student direction. The class sum produces one loss per example; the final mean produces one scalar for the batch. The hard term follows the same reduction convention at temperature one. Taking `.data` from the student at this point would leave the numerical loss looking reasonable while disconnecting the computation that trains it.

The method returns a scalar `Tensor`, ready for `backward()`. Its stable `log_softmax` path does not use the NumPy `_kl_divergence` and `_cross_entropy` helpers beneath it; those helpers support the module's analysis table. With identical teacher and student logits, the soft term is zero and only the hard term remains. Duplicating every example leaves both terms unchanged. The trace below checks the gradient, teacher immutability, and batch-duplication invariant in a training step.

17.4 A Worked Trace

17.4.1 Following one distillation update

Use one input, two output classes, and explicitly chosen weights so that the trace has no random initialization hidden in it. In listing 17.4, lines 7 to 10 construct `teacher` and `student` linear layers with weights `[[2.0, 0.0]]` and `[[0.0, 0.0]]`. Constructing the optimizer in line 11 before the forward pass enables gradient tracking on the student's parameters. Line 12 initializes `KnowledgeDistillation(teacher, student, temperature=2.0, alpha=0.5)`. In lines 13 to 15, input `x` evaluates `teacher_logits` while `teacher_before` takes a reference copy.

Lines 17 to 19 evaluate the combined distillation loss. In line 20, `loss.backward()` executes backpropagation. Lines 21 to 23 confirm that `student.weight.grad` receives the analytical gradient `[[-0.307765, 0.307765]]` while `teacher.weight.grad` remains `None` (the teacher is not trained). In line 24, `optimizer.step()` updates the student; lines 25 to 27 verify that student loss decreased while teacher parameters remained

strictly frozen. Lines 29 to 31 confirm that duplicating the input maintains consistent sample-mean loss.

Listing 17.4 A distillation update with fixed teacher state: The assertions check the state contract described in the surrounding text.

```
1  import numpy as np
2  from tinytorch.core.tensor import Tensor
3  from tinytorch.core.layers import Linear
4  from tinytorch.core.optimizers import SGD
5  from tinytorch.perf.compression import KnowledgeDistillation
6
7  teacher = Linear(1, 2, bias=False)
8  student = Linear(1, 2, bias=False)
9  teacher.weight.data[:] = [[2.0, 0.0]]
10 student.weight.data[:] = [[0.0, 0.0]]
11 optimizer = SGD(student.parameters(), lr=0.1)
12 kd = KnowledgeDistillation(teacher, student, temperature=2.0,
   ↪  alpha=0.5)
13 x, labels = Tensor([[1.0]]), Tensor([0])
14 teacher_logits = teacher(x)
15 teacher_before = teacher.weight.data.copy()
16
17 optimizer.zero_grad()
18 loss = kd.distillation_loss(student(x), teacher_logits, labels)
19 before = float(loss.data)
20 loss.backward()
21 print(np.round(student.weight.grad, 6))  # [[-0.307765  0.307765]]
22 assert np.allclose(student.weight.grad, [[-0.307765, 0.307765]],
   ↪  atol=1e-6)
23 assert teacher.weight.grad is None
24 optimizer.step()
25 after = kd.distillation_loss(student(x), teacher_logits, labels)
26 assert float(after.data) < before
27 assert np.array_equal(teacher.weight.data, teacher_before)
28
29 pair = Tensor([[1.0], [1.0]])
30 duplicated = kd.distillation_loss(student(pair), teacher(pair),
   ↪  Tensor([0, 0]))
31 assert np.isclose(duplicated.data, after.data)
```

The teacher's softened probability for class zero is approximately `0.731059`; the student's is `0.5`. The soft contribution to the first logit's gradient is `0.5 * (0.5 - 0.731059) / 2`, and the hard contribution is `0.5 * (0.5 - 1)`. Their sum is `-0.307765`, the value printed by `backward()` in line 21. Subtracting that gradient raises the first student weight and lowers the second, while the teacher stays unchanged. The division by temperature comes from differentiating the softened logits; the batch mean would supply another division for a larger batch. No temperature-squared multiplier cancels either effect in this implementation.

17.4.2 Deleting one neuron

Take a two-layer network with two inputs, three hidden neurons, and one output, with ReLU between the layers. In Chapter 3's layout the first weight is 2×3 with one column per hidden neuron, and the second is 3×1 with one row per hidden neuron:

$$W_1 = \begin{bmatrix} 1.0 & 0.1 & 3.0 \\ -2.0 & 0.2 & 1.0 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0.5 & 0.0 & -1.0 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 2.0 \\ 0.5 \\ -1.0 \end{bmatrix}, \quad b_2 = 0.1$$

The L1 column norms of W_1 are 3.0, 0.3, and 4.0, and the L2 norms are 2.24, 0.22, and 3.16; by either score the middle neuron loses. For the input $x = [1.0, 0.5]$, the hidden activations before pruning are $h = \text{ReLU}(xW_1 + b_1) = \text{ReLU}([0.5, 0.2, 2.5]) = [0.5, 0.2, 2.5]$ and the output is $y = 2.0(0.5) + 0.5(0.2) - 1.0(2.5) + 0.1 = -1.3$. After pruning, $h' = [0.5, 2.5]$ and $y' = 1.0 - 2.5 + 0.1 = -1.4$. The layers shrank from 2×3 and 3×1 to 2×2 and 2×1, and the output moved by 0.1.

That 0.1 is the deleted neuron's activation (0.2) times its outgoing weight (0.5). Had the outgoing weight been 4.0 instead, the same deletion would have moved the output by 0.8 while the column score stayed at 0.3. The incoming-weight score is blind to what the next layer does with the neuron, which is why production pruning methods score with gradients or with the effect on the loss rather than with weight magnitude alone.

Both functions agree with the hand arithmetic, and running them side by side shows exactly what separates them. `prune_neurons` returns a narrower pair; `structured_prune` zeros the same neuron in place and leaves the shapes alone. The zeroed neuron has a zero bias and a zero column, so its activation is $\text{ReLU}(0) = 0$ and it contributes nothing, which is why the two outputs match.

Figure 17.4 follows the same hidden unit through both weight matrices. The numerical output alone cannot distinguish the zeroed model from the physically smaller one.

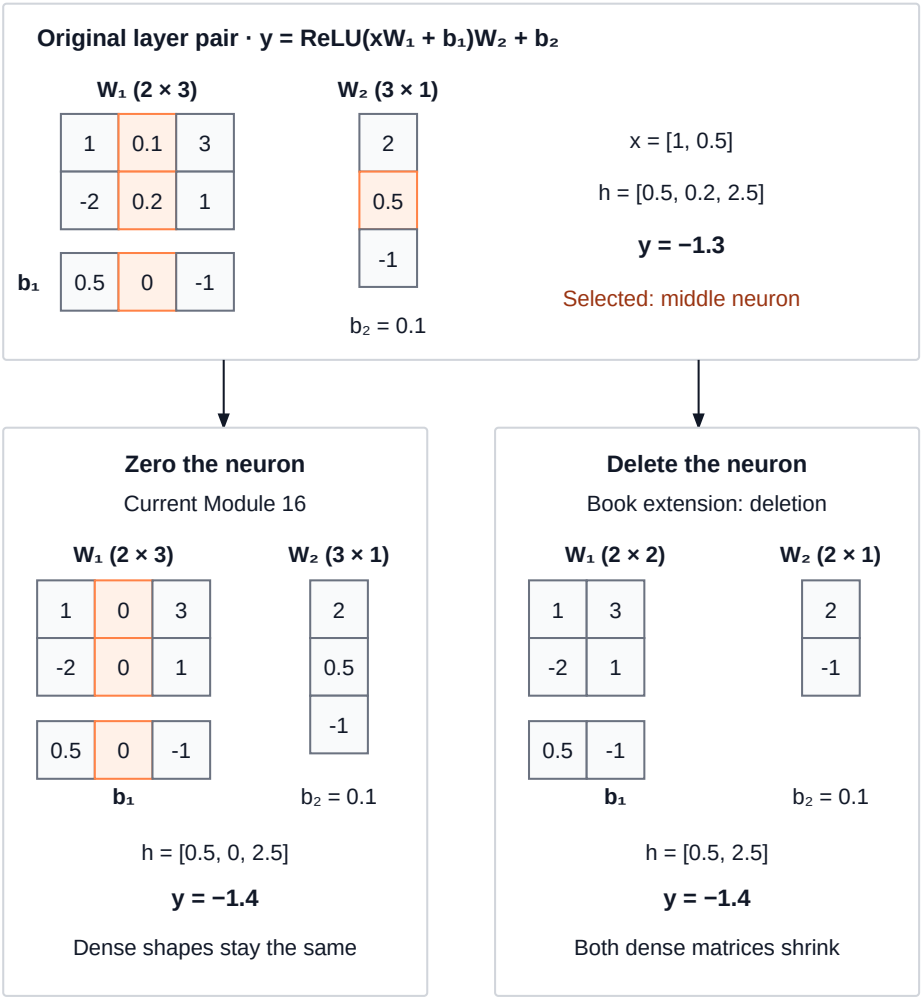


Figure 17.4: **Zeroing vs. deleting a hidden neuron**: Both changes move the output from −1.3 to −1.4 for the same input. Module 16 preserves dense shapes; the book extension deletes the matching first-layer column, bias entry, and second-layer row.

Listing 17.5 Comparing zeroing with physical deletion: The assertions check the state contract described in the surrounding text.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.layers import Linear, Sequential
from tinytorch.core.activations import ReLU
from tinytorch.perf.compression import structured_prune

first, second = Linear(2, 3), Linear(3, 1)
first.weight.data[:] = [[1.0, 0.1, 3.0], [-2.0, 0.2, 1.0]]
first.bias.data[:] = [0.5, 0.0, -1.0]
second.weight.data[:] = [[2.0], [0.5], [-1.0]]
second.bias.data[:] = [0.1]
x = Tensor(np.array([[1.0, 0.5]], dtype=np.float32))
net = Sequential(first, ReLU(), second)
print(net.forward(x).data)                                          #
    [[-1.3]]
assert np.allclose(net(x).data, [[-1.3]])

small_first, small_second = prune_neurons(first, second, keep=2)
small = Sequential(small_first, ReLU(), small_second)
print(small.forward(x).data, small_first.weight.data.shape)     #
    [[-1.4]] (2, 2)
assert np.allclose(small(x).data, [[-1.4]])

structured_prune(net, prune_ratio=1 / 3)
print(first.weight.data)                                            # middle
    column zeroed
print(net.forward(x).data, first.weight.data.shape)                 #
    [[-1.4]] (2, 3)
assert np.allclose(net(x).data, small(x).data)
```

17.4.3 Rank-1 approximation of a 3×3 matrix

Consider the matrix

$$W = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

Its singular values are $\sigma_1 = 4, \sigma_2 = 1, \sigma_3 = 1$. You can check the first one by hand. W times the vector $[1, 1, 1]$ gives $[4, 4, 4]$, so $u_1 = v_1 = [1, 1, 1]/\sqrt{3}$ with $\sigma_1 = 4$. Keeping only that term,

$$W_1 = \sigma_1 u_1 v_1^T = \frac{4}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \qquad W - W_1 = \frac{1}{3} \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$$

The Frobenius norm of the residual is $\sqrt{3 \cdot (2/3)^2 + 6 \cdot (1/3)^2} = \sqrt{2} \approx 1.41$, and Eckart-Young predicts $\sqrt{\sigma_2^2 + \sigma_3^2} = \sqrt{2}$. The two factors are $W_A = \sqrt{4}\, u_1 \approx [1.155, 1.155, 1.155]^T$ and $W_B = \sqrt{4}\, v_1^T$, the same three numbers as a row. An SVD routine may flip the signs of both u_1 and v_1; their outer product remains unchanged, so the test must compare the reconstructed matrix rather than assume a sign. Each of the three ranks tells a different story about storage, summarized in table 17.1.

Table 17.1: **Storage of three-factor SVD truncations**: Module 16 retains the singular values separately. The original matrix stores nine values; only rank one saves storage in this example, and it is the only rank `low_rank_approximate` returns. The rank-2 and rank-3 rows come from truncating NumPy's SVD directly.

Rank r	Stored values	Factor shapes	Frobenius error	Relative size
3 (refused; NumPy SVD)	21	$(3,3), (3,), (3,3)$	0	$2.33\times$
2 (refused; NumPy SVD)	14	$(3,2), (2,), (2,3)$	1	$1.56\times$
1 (`low_rank_approximate`)	7	$(3,1), (1,), (1,3)$	$\sqrt{2}$	$0.78\times$

Rank 2 is larger than the original matrix, as predicted by the three-factor bound $r < 9/7$. Folding `s` into two factors saves another r entries, giving six entries at rank one and twelve at rank two. Keeping the dense original is cheaper than either full-rank representation. Reconstruction error and storage are separate criteria even on this small example.

The following check measures both the reconstruction and the storage returned by the module at rank one, then asks the module for ranks two and three. The rank ratio is $r/3$ because the matrix has full rank 3. The error assertion checks the Eckart-Young prediction; the size and ownership assertions check the returned factors. Above the break-even rank the module refuses, so the listing truncates NumPy's SVD directly to show what those ranks would have cost.

Listing 17.6 Checking SVD error and owned storage: The assertions check the state contract described in the surrounding text.

```
from tinytorch.perf.compression import low_rank_approximate

W = np.array([[2.0, 1.0, 1.0],
              [1.0, 2.0, 1.0],
              [1.0, 1.0, 2.0]])
U_all, all_sigma, Vt_all = np.linalg.svd(W)
print("singular values:", np.round(all_sigma, 3))          # [4. 1.
↪   1.]

U, S, Vt = low_rank_approximate(W, 1 / 3)                  # rank 1 <
↪   r* = 9/7
error = np.linalg.norm(W - (U * S) @ Vt)
print(f"rank 1: kept {np.round(S, 3)}, error {error:.3f}")  # kept
↪   [4.], error 1.414
assert np.isclose(error, np.sqrt(np.sum(all_sigma[1:] ** 2)))   #
↪   Eckart-Young
assert U.size + S.size + Vt.size == 7                      # fewer
↪   than 9 dense values
assert all(f.flags.owndata for f in (U, S, Vt))

for r in (2, 3):                                           # above r*
↪   = 9/7
    try:
        low_rank_approximate(W, r / 3)
        refused = False
    except ValueError:
        refused = True
    W_r = (U_all[:, :r] * all_sigma[:r]) @ Vt_all[:r]       # NumPy
↪   truncation, outside the module
    print(f"rank {r}: refused={refused}, NumPy error {np.linalg.norm(W
↪   - W_r):.3f}, {7 * r} values")
    assert refused
```

Rank 1 gives error 1.414. Ranks 2 and 3 exceed the break-even rank 9/7, so `low_rank_approximate` refuses them; computed directly with NumPy's SVD, they give 1.000 and a reconstruction within floating-point tolerance, while storing 14 and 21 values against the dense nine. The ownership assertion checks a separate invariant: small shapes really correspond to small independent factor buffers, rather than views keeping the full SVD allocation alive.

17.5 From TinyTorch to Production

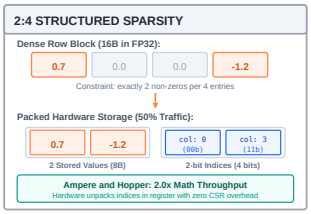
The reference pruning functions make one in-place edit. PyTorch's pruning utility instead maintains a mask during subsequent forward passes. Its pruning tutorial describes `weight_orig`, a `weight_mask` buffer, and a forward hook that multiplies them. Removing this reparameterization with `prune.remove` leaves the current masked values in the dense

parameter. The distinction matters during recovery training: the active mask can continue suppressing selected connections, whereas TinyTorch's one-time zero assignment allows them to regrow. Neither operation by itself rebuilds narrower matrices.

A sparse execution path can exploit zeros without changing the logical matrix dimensions. For example, PyTorch documents semi-structured sparse tensors with a restricted pattern, a packed value representation, and metadata identifying retained positions. A 2:4 pattern retains two values in each group of four. Metadata adds storage beyond those values, and supported devices, dtypes, shapes, and operations constrain its use. The relevant comparison is therefore a specific packed representation and its supported multiplication, not a dense matrix that happens to contain zeros. Module 16 implements neither this format nor its kernel.

Low-rank adaptation uses thin-matrix arithmetic for a different purpose. LoRA (Hu et al., 2021) freezes pretrained weights and trains a low-rank update $W + AB$, with $A \in \mathbb{R}^{M \times r}$ and $B \in \mathbb{R}^{r \times N}$. The update adds $r(M + N)$ trainable parameters alongside the base MN weights. In contrast, `factorize_linear` replaces the base matrix directly; an adapter does not shrink the base allocation. TinyTorch computes an SVD approximation of existing weights rather than an adapter.

The teacher-student objective transfers directly. Hinton, Vinyals, and Dean (2015) soften teacher predictions to guide a student, scaling gradients by T^2. TinyTorch intentionally omits that multiplier. In the local trace, increasing temperature changes both target entropy and relative gradient scale. A production pipeline must specify reductions and scalings explicitly; the label "distillation" alone does not guarantee matching losses.



2:4 structured sparsity: storing two nonzero weights per four-element block with two-bit indices achieves 2.0x math throughput on Tensor Cores.

Unstructured pruning zeroes arbitrary weights, producing irregular sparsity that standard GEMM hardware cannot accelerate without indexing overhead. Modern production runtimes resolve this through hardware-aligned sparsity:

1. **NVIDIA 2:4 Structured Sparsity**: Starting with Ampere and Hopper, Tensor Cores support native 2:4 structured sparsity. For every four values along a row, at least two must be zero. Accelerators store two values per group plus 2-bit position metadata, reducing weight storage by somewhat less than half and offering up to twice the peak math throughput on supported shapes; accuracy after pruning must still be measured.
2. **Channel and Block Pruning**: Serving frameworks (such as TensorRT and ONNX Runtime) favor structured pruning, removing entire attention heads (Chapter 12), MLP intermediate channels, or transformer blocks. Preserving contiguous memory strides lets standard dense BLAS kernels consume the smaller matrices without specialized sparse hardware; the latency change still has to be measured.

Table 17.2 separates allocation changes from the required execution path. A proposed optimization needs both a representation that saves work or storage and a kernel that consumes that representation.

Table 17.2: **Representation and execution**: Pruning, factorization, and adaptation change different parts of the state. Smaller stored factors do not establish a latency improvement by themselves.

Method	Stored state	Execution requirement
TinyTorch pruning	Original dense arrays, with zeros	Existing dense multiplication; no sparse work skipped

Method	Stored state	Execution requirement
Packed sparsity	Retained values plus position metadata	A compatible sparse multiplication
Neuron deletion	Smaller adjacent weight matrices	Dense multiplication with reduced hidden width
Folded SVD factors	Two weights totaling $r(M + N)$ values	Two successive dense multiplications
LoRA update	Base W plus $r(M + N)$ update values	Base computation plus update, or a merged dense weight

17.6 Summary

The compression call chain starts by deciding what state changes. `magnitude_prune` ranks weights globally and zeros selected entries in the existing arrays. `structured_prune` selects output columns, including their biases, while preserving the terminal head of a multilayer model. Both retain dense dimensions. The book's `prune_neurons` extension makes the separate shape change across adjacent layers, and its new parameters require a new optimizer.

`low_rank_approximate` returns three independently owned arrays and leaves the input untouched. Their singular values predict matrix reconstruction error, while their sizes determine storage savings. `factorize_linear` folds those values into two new layers so that the forward pass uses the factors. Distillation then provides a scalar loss whose gradient reaches the student and stops at the teacher; accuracy recovery must still be evaluated.

The invariant to carry into Chapter 18 is that an optimization claim must name the representation, its consumer, and the comparison being measured. Reducing weights addresses only part of a forward pass. Chapter 18 examines the intermediate activations between operations and whether changing their computation order or lifetime can reduce work without altering the intended result.

17.7 Further Reading

- **Han, Pool, Tran, and Dally** (2015). "Learning both weights and connections for efficient neural networks." *NeurIPS*. Magnitude pruning followed by retraining.
- **Li et al.** (2017). "Pruning filters for efficient ConvNets." *ICLR*. Structured pruning by the norm of each filter's weights, which removes work that zeroed individual weights do not.
- **Denton et al.** (2014). "Exploiting linear structure within convolutional networks for efficient evaluation." *NeurIPS*. Low-rank approximation of trained layers.
- **Hinton, Vinyals, and Dean** (2015). "Distilling the knowledge in a neural network." *arXiv*. Temperature-softened targets and the T^2 factor this chapter's loss omits.

🔥 Chapter 18

Acceleration: Kernel Fusion and Cache Tiling

Elementwise operations such as addition, scaling, and activation functions do little arithmetic per value. In an eager runtime that executes one operation at a time, each operation writes a full intermediate array that the next one reads back, so a chain of them can spend more time moving arrays through memory than computing on them.

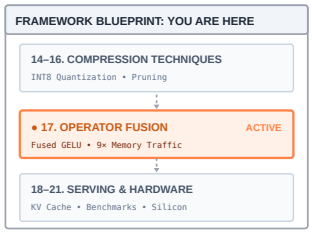
In modern PyTorch, compiler backends (like `torch.compile`) automate hardware acceleration by fusing operators into a single compiled kernel:

```
import torch

# Eager execution: 2 separate kernel launches, 1 intermediate DRAM buffer
def eager_op(x, bias):
    return torch.relu(x + bias)

# Compiled execution: Inductor fuses add + relu into a single pass
compiled_op = torch.compile(eager_op)

x = torch.randn(2048, 2048)
bias = torch.randn(2048)
out = compiled_op(x, bias)
```



Framework Blueprint: Module 17 compares GELU expressions with different intermediate lifetimes, a tiled matrix multiply, and im2col convolution, and models the traffic a compiled fused kernel would avoid.

Underneath that simple compilation decorator, the runtime executes two fundamental hardware optimizations:

1. **Operator fusion:** In eager execution, `x + bias` writes an entire 16 MB float32 tensor to DRAM, only for `relu` to immediately read those bytes back. Fusing the two operations keeps values in processor registers, eliminating the round-trip memory traffic entirely.
2. **Cache tiling for data reuse:** For matrix multiplication, partitioning large matrices into small blocks (for example 32×32 for a 32 KB budget) that fit within high-speed L1/L2 cache prevents cache lines from being prematurely evicted to main memory.
3. **Stride-conscious memory access:** Aligning iteration loops with contiguous row-major memory strides prevents the catastrophic cache thrashing caused by strided column-major traversals.

This chapter writes a GELU expressed with fewer temporary Tensors and a tiled matrix multiply in NumPy, then times both. Before building these accelerated patterns, the first step is to inspect the latency penalty that unfused memory traffic and unaligned strides impose on hardware.

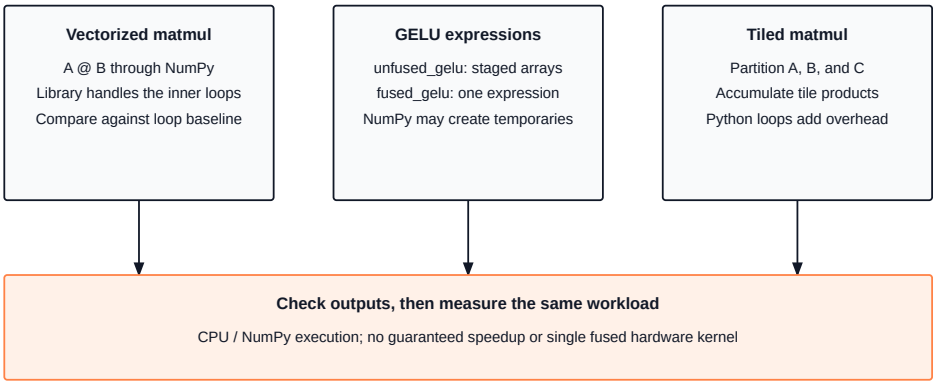


Figure 18.1: **Three NumPy execution paths**: Vectorized multiplication delegates to a library, the GELU helpers retain different intermediate arrays, and tiled multiplication explicitly accumulates block products. Every comparison checks values before measuring time.

18.1 The Problem

↩ **Jump back:** Chapter 2 *The tanh GELU form introduced in Module 02, re-expressed here to compare intermediate-array lifetimes.*

In eager execution, deep learning frameworks evaluate expressions statement by statement. When evaluating a compound expression, such as an activation function or a sequence of linear layers, the runtime allocates a fresh buffer for each intermediate result, writes it to main memory, and reads it back for the subsequent operation. For elementwise operations with low arithmetic intensity, memory bus bandwidth rather than floating-point compute capacity becomes the primary bottleneck. Moving transient values back and forth across the memory bus consumes substantial latency and energy even when the underlying mathematical operations are trivial.

Memory access patterns provide a second concrete performance bottleneck. A two-row array `[[1, 2, 3], [4, 5, 6]]` stores its rows consecutively in ordinary C order, using the contiguous strides derived in Chapter 1 ($S_0 = 3, S_1 = 1$). Walking a row visits adjacent floats at unit stride; walking a column advances three floats between entries, stepping across row strides. Both traversals sum to 21. Listing 18.1 enlarges this pattern so that access order can affect timing.

The script reports timings for the machine running it; neither traversal has a fixed speedup. Their mathematical sums agree, but changing the reduction order can change the last few floating-point digits. The check compares both answers with `A.sum(dtype=np.float64)` before timing. Its tolerance suits this seeded example; other input magnitudes and sizes may need a different bound.

A cache stores recently accessed data near the processor. Main memory transfers chunks called cache lines, so adjacent floats can share one transfer. For example, a hypothetical 64-byte line holds sixteen float32 values. A row walk can use all sixteen; a column walk may use only one before moving to another line.[67] Whether a later access reuses that line depends on the array size and cache behavior. The stride explains a possible cost; it does not measure transferred bytes.

[67] **Hardware prefetchers and TLB thrashing in strided traversals**: Modern processor memory controllers incorporate hardware stream prefetchers that detect sequential cache line access patterns and proactively stream adjacent lines from DRAM into L2 and L1 caches ahead of execution demand. In a row-major traversal, contiguous access maximizes prefetcher accuracy. In a column-major traversal of a 2048×2048 float32 matrix, each consecutive step jumps by 8,192 bytes (two full 4 KB virtual memory pages). This strided access defeats stream prefetchers and exhausts the Translation Lookaside Buffer (TLB), inducing frequent hardware page table walks for virtual-to-physical address translation.

Listing 18.1 Access-order experiment: The same array is traversed by rows and columns.

```python
import numpy as np
import time

rng = np.random.default_rng(17)
A = rng.standard_normal((2048, 2048)).astype(np.float32)  # about 17 MB

def sum_by_rows(A):
    total = 0.0
    for i in range(A.shape[0]):
        total += A[i, :].sum()      # adjacent values
    return total

def sum_by_cols(A):
    total = 0.0
    for j in range(A.shape[1]):
        total += A[:, j].sum()      # values separated by a row stride
    return total

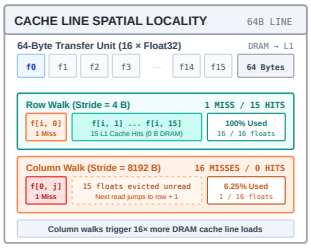
reference = A.sum(dtype=np.float64)
for f in (sum_by_rows, sum_by_cols):
    np.testing.assert_allclose(f(A), reference, rtol=1e-5, atol=1e-2)
    f(A)                            # warm up once
    t0 = time.perf_counter()
    f(A)
    print(f"{f.__name__}: {(time.perf_counter() - t0) * 1e3:.0f} ms")
```

Now look at what a framework does when you write a GELU. Chapter 2 introduced the tanh form,

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1+\tanh\left(\sqrt{2/\pi}\,\left(x+0.044715\,x^3\right)\right)\right),$$

and `unfused_gelu` evaluates it as eight steps: cube, scale, add, scale, tanh, add one, multiply by x, and halve. Each step computes a NumPy array and copies it into a `Tensor`. The seven named intermediates stay alive until return. At three inputs this is easy to inspect; at millions of inputs, retaining those full arrays becomes a substantial allocation cost.

Matrix multiplication presents a different opportunity. Each input element contributes to several outputs. For $C = AB$, a row of A is needed for every output column, and a column of B for every output row. Grouping nearby output calculations can make those reuses occur close together in time. Analysis requires tracking both the accumulation into C and which input blocks each step reads.



Cache line spatial locality: 64-byte hardware transfers load 16 float32 values. Row walks use all 16 floats per line (1 miss, 15 hits), while column walks stride across rows and use only 1 float per line (16 misses).

Pause & Predict: Hardware Cache Line Footprint

On a processor architecture with 64-byte cache lines, how many compulsory cache misses occur when summing a 2048×2048 float32 matrix by contiguous rows? How many cache misses occur when summing the exact same matrix by columns if the cache cannot hold all 2048 rows simultaneously? What ratio of cache misses separates the two traversals?

18.2 The Idea

18.2.1 One boundary, counted in crossings

In modern hardware architectures, the primary performance bottleneck is the memory wall: transferring data between off-chip DRAM and the processor die consumes an order of magnitude more energy and latency than executing arithmetic instructions. High-performance systems software and GPU runtimes maximize efficiency by treating the chip boundary as a costly threshold, prioritizing reuse within registers and on-chip SRAM caches. Consider a tile product that repeatedly uses the same small blocks. Keeping those blocks in a cache can avoid fetching them from main memory for every arithmetic operation. Registers hold individual working values; caches hold recently accessed chunks; main memory holds the larger arrays. Table 18.1 describes their roles without assuming a particular processor.

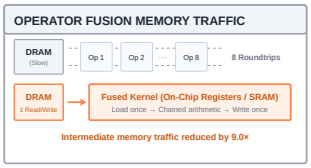
↪ **Jump ahead:** Chapter 23 *How modern compilers (C++, Triton, and PyTorch Inductor) automate tiling and fusion at hardware scale.*

Table 18.1: The memory levels relevant to allocation and reuse. NumPy code chooses arrays and operations; hardware manages which cache lines remain resident.

Level	What it holds	Role in this chapter
Registers	Values currently used by instructions	Possible home for compiled scalar intermediates
Caches	Recently accessed memory chunks	Possible reuse of matrix blocks
Main memory	Arrays and other program data	Backing storage for NumPy allocations

The analytical model counts requested data separately from measured time. A logical read of an array is an operation's need for its values; it may be served from a cache. An allocated array occupies storage, but allocation alone does not reveal how many bytes crossed a hardware boundary. Chapter 15's arithmetic intensity divides floating-point work by a stated byte count. Here every such count needs its assumptions attached.

18.2.2 Intermediate arrays and their lifetime



Kernel fusion memory ledger: 8 unfused DRAM round-trips (360 B) collapse into 1 streaming fused pass (40 B).

In deep learning compilers like PyTorch Inductor (`torch.compile`), Triton, and XLA, operator fusion serves as the frontline defense against the memory bandwidth saturation caused by eager execution. When evaluating multi-step mathematical expressions without fusion, frameworks allocate and stream transient intermediate arrays to and from main memory, incurring severe DRAM traffic penalties. At $x = 1$, the first three GELU steps produce 1, 0.044715, and 1.044715. In `unfused_gelu`, each value occupies the corresponding position in a different array. Naming `temp1`, `temp2`, and `temp3` keeps their `Tensor` objects reachable even after the next step has consumed their values. `Tensor` construction copies the incoming NumPy array, so these are separate owned buffers.

The compact expression avoids retaining seven named `Tensor` wrapper objects, but it does not eliminate intermediate memory traffic. In Python and NumPy, writing a compound one-liner like `0.5 * x * (1.0 + np.tanh(...))` still allocates an anonymous, temporary NumPy array buffer for every binary operator and function call. Each sub-expression executes a distinct compiled C loop that reads operands from DRAM and writes intermediate arrays back to memory. In contrast, true kernel fusion (as performed

by compiler runtimes like PyTorch `torch.compile`/Inductor, Triton, or XLA) inspects the computational graph and generates a single machine-code loop: each input element x_i is loaded into a processor register once, evaluated through the entire mathematical chain entirely within register space, and written back to DRAM only once as the final result. This distinction isolates allocation lifetimes during eager execution while demonstrating why optimizing compilers are indispensable for memory-bound operators.

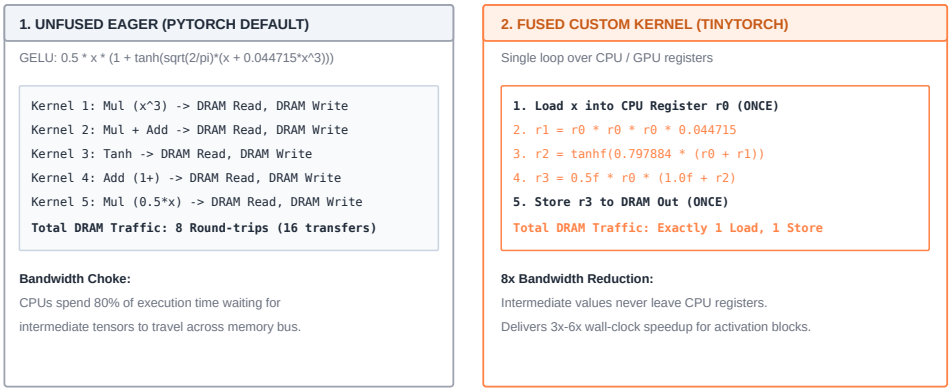
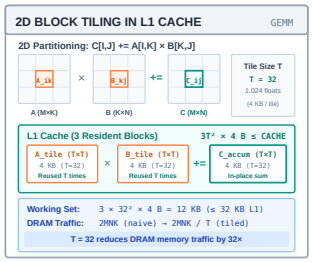


Figure 18.2: **Memory traffic in unfused vs. fused kernel execution**: In unfused execution, each elementwise operation launches a separate kernel that reads operands from DRAM and writes temporary buffers back to DRAM, saturating memory bandwidth and thrashing CPU caches. In a fused kernel, the input is loaded once into fast processor registers/L1 cache, all intermediate arithmetic evaluates in registers, and only the final result is written to memory.

18.2.3 Tiling: reuse before eviction

High-performance linear algebra libraries—such as NVIDIA cuBLAS, CUTLASS, and Intel oneMKL—rely on cache tiling to transform memory-bound matrix multiplications into compute-bound throughput. By partitioning large matrix operands into compact 2D sub-blocks that fit snugly within fast L1/L2 caches or GPU shared memory, hardware units reuse loaded data elements repeatedly before they are evicted back to high-latency DRAM. A matrix multiply has the opposite structure. Every output element needs a whole row of A and a whole column of B, so the data cannot be consumed in one streaming pass. It can, however, be reused. $C_{ij} = \sum_k A_{ik}B_{kj}$ uses A_{ik} in the computation of every C_{ij} for all N values of j, and B_{kj} in every C_{ij} for all M values of i. The question is whether those reuses happen while the value is still in fast memory or after it has been evicted and must be fetched again.



2D cache block tiling: Partitioning matrix multiplication into $T \times T$ sub-blocks keeps three tiles (A, B, and C) in on-chip L1 cache ($3T^2 \times 4$ B $\leq$ cache size). Each float is reused T times, cutting modeled input loads by $T\times$.

Tiling answers by changing the loop order. Instead of computing C one element at a time, compute it one $T \times T$ block at a time. To produce block (I, J) of C, walk along k in steps of T: fetch the $T \times T$ block of A at (I, K) and the $T \times T$ block of B at (K, J), multiply them, and add the product into the block of C. While those two input blocks sit in fast memory, every one of their T^2 elements is used T times.

For a simplified input-load model, assume each block product loads its two input blocks once and reuses them internally. Ignore cache reuse between block products and traffic for the output accumulator. When all dimensions divide evenly by T, the scalar loop requests $2MNK$ input elements, while the blocked model requests

$(M/T)(N/T)(K/T) \times 2T^2 = 2MNK/T$. The factor of T belongs to this model. A real processor may reuse data in either loop, so this is not a measured memory-traffic reduction.

The tile cannot grow without limit. Three blocks must fit in fast memory at once, a block of A, a block of B, and the block of C being accumulated, so the constraint is

$$3\,T^2 \times 4\,\text{bytes} \leq \text{cache size}.$$

For a hypothetical cache budget of 32,768 bytes, the inequality gives $T \leq 52$. A tile of 32 fits this three-block budget; a tile of 64 requires 49,152 bytes. The estimate excludes other live data, library workspace, and cache-line effects. Choosing a tile therefore starts with a capacity estimate and ends with measurement, rather than a promise that a particular size is fastest.[68]

[68] **Hierarchical multi-level tiling and GotoBLAS**: High-performance implementations (such as OpenBLAS, BLIS, and NVIDIA CUTLASS) do not stop at single-level cache tiling. As established by Kazushige Goto and Robert van de Geijn (2008), GEMM is structured as a hierarchy of nested tiles matching hardware memory levels: an outer macro-kernel packs $M_c \times K_c$ panels of A and $K_c \times N_c$ panels of B into the L2/L3 cache, while an inner micro-kernel streams tiny $m_r \times n_r$ blocks directly into CPU SIMD or GPU register files. This eliminates cache evictions during the inner dot products and achieves over 90% of theoretical peak hardware arithmetic throughput.

18.2.4 Why libraries route through matrix multiply

Deep learning engines like PyTorch, cuDNN, and TensorRT systematically lower diverse operations—including 2D convolutions via `im2col` and multi-head attention projections—into standard General Matrix Multiplies (GEMM). Because dense matrix-matrix multiplication possesses an operational intensity that scales with matrix dimension, transforming neural network workloads into GEMM kernels allows hardware accelerators to reach near-peak arithmetic throughput. The BLAS interface that NumPy calls groups its routines into three levels by the shapes they combine, and the levels differ in how much arithmetic each moved value can support. Count float32 values and assume each operand is read once and each result written once.

- **Level 1, vector with vector**, such as AXPY ($y \leftarrow \alpha x + y$) on n elements: $2n$ FLOPs over $3n$ values, or $12n$ bytes, which is $1/6$ FLOP per byte whatever n is.
- **Level 2, matrix with vector**, such as GEMV ($y \leftarrow Ax$) with an $n \times n$ matrix: $2n^2$ FLOPs over about n^2 values, or $4n^2$ bytes, which is about $1/2$ FLOP per byte.
- **Level 3, matrix with matrix**, GEMM, with $n \times n$ operands: $2n^3$ FLOPs over $3n^2$ values, or $12n^2$ bytes, which is $n/6$ FLOPs per byte and grows with the matrix.

Only Level 3 has an intensity that grows with problem size. That growth is what tiling exploits, and it is the reason a library can bring matrix multiplication close to a machine's arithmetic ceiling while vector routines stay bound by memory. The levels map directly onto this book's workloads. A Linear layer applied to a batch of rows is a GEMM. The same layer applied to one row is a GEMV, which is the cached decode step in table 14.5: each weight is read to perform two floating-point operations.

18.3 The Code

18.3.1 The two GELUs

In modern Transformer architectures like GPT and BERT, the Gaussian Error Linear Unit (GELU) serves as the standard feed-forward activation function, commonly optimized in production engines via fused CUDA or Triton kernels. Because its mathematical approximation chains multiple transcendental and polynomial operations, measuring how GELU is structured reveals the stark performance difference between retaining intermediate tensor buffers and evaluating arithmetic in a single pass. The entry point

receives a `Tensor` and reads `x.data`. Every intermediate has the same shape as that input because the operations act element by element. `unfused_gelu` first computes `x.data**3`, constructs a new `Tensor` from that array, and binds it to `temp1`. Each following line reads either that temporary's `.data` or the original input. The last line wraps the halved result and returns it; no input is mutated.

```
def unfused_gelu(x: Tensor) -> Tensor:
    """
    Deliberately unfused GELU implementation for performance
    comparison.
    """
    # Unfused version - creates many intermediate arrays
    sqrt_2_over_pi = np.sqrt(2.0 / np.pi)

    # Each operation creates a temporary array (simulating kernel
    launches)
    temp1 = Tensor(x.data**3)  # x³
    temp2 = Tensor(0.044715 * temp1.data)  # 0.044715 * x³
    temp3 = Tensor(x.data + temp2.data)  # x + 0.044715 * x³
    temp4 = Tensor(sqrt_2_over_pi * temp3.data)  # √(2/π) * (...)
    temp5 = Tensor(np.tanh(temp4.data))  # tanh(...)
    temp6 = Tensor(1.0 + temp5.data)  # 1 + tanh(...)
    temp7 = Tensor(x.data * temp6.data)  # x * (1 + tanh(...))
    result = Tensor(0.5 * temp7.data)  # 0.5 * x * (...)

    return result
```

The local names keep seven intermediate `Tensor` buffers alive while the output is constructed. Each constructor copies its input into owned float32 storage, so counting just the mathematical operations misses these copies. After return, the local intermediate objects can be released. The caller receives a fresh output with the input's shape. Compare that lifetime with the compact implementation.

```
def fused_gelu(x: Tensor) -> Tensor:
    """
    Compact GELU expression that avoids retaining intermediate Tensor
    copies.
    """
    # Mathematical constant for GELU approximation
    sqrt_2_over_pi = np.sqrt(2.0 / np.pi)

    # Fused GELU computation - all operations in single expression
    # By computing the full expression in a single line, we avoid
    creating intermediate
    # Tensor objects. NumPy still allocates temporary arrays
    internally, so real
    # kernel fusion requires compiled frameworks like XLA or
    torch.compile.
    result_data = 0.5 * x.data * (
        1.0 + np.tanh(sqrt_2_over_pi * (x.data + 0.044715 *
        x.data**3))
```

SOURCE CODE MAPPING MOD 17

FILE: src/17_acceleration/17_acceleration.py
EXPORT: tinytorch.perf.acceleration
SYMBOLS: vectorized_matmul, fused_gelu

Extracted & verified at build time ✓ tested

Source Code Mapping: Module 17 extracts operator fusion, memory traffic accounting, and tiled matrix multiplication directly from the working repository.

```
    )

    return Tensor(result_data)
```

The constant `sqrt_2_over_pi` sets the coefficient in the tanh approximation. The expression computes the same formula without wrapping each intermediate in a `Tensor`; only the final `Tensor(result_data)` makes the returned object. NumPy still creates temporary arrays while evaluating the expression. Both functions preserve the input, and neither records the differentiable operations from Chapter 6.

Listing 18.2 GELU timing comparison: Compare the two allocation strategies on the same input.

```
import time
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.perf.acceleration import fused_gelu, unfused_gelu

def time_ms(f, x, reps=5):
    f(x)                                          # warm up
    best = float("inf")
    for _ in range(reps):
        t0 = time.perf_counter()
        f(x)
        best = min(best, time.perf_counter() - t0)
    return best * 1e3

x = Tensor(rng.standard_normal((1024, 1024)).astype(np.float32))
print(f"unfused: {time_ms(unfused_gelu, x):.0f} ms")
print(f"fused:   {time_ms(fused_gelu, x):.0f} ms")
assert np.allclose(unfused_gelu(x).data, fused_gelu(x).data,
    atol=1e-6)
```

In listing 18.2, the names `fused_gelu` and `unfused_gelu` describe how the computation is expressed, not a promise about this timing result. The first constructs one output `Tensor`; the second constructs eight and retains its named intermediates until the function returns. Both evaluate NumPy array operations eagerly, and both can allocate internal temporaries. Their measured difference includes allocation and object-lifetime effects. The operation ledger below separates these measurements from a compiled-kernel model.

There is also a boundary with Chapter 6. Both helpers read `x.data` and wrap the NumPy result in a fresh `Tensor`, so neither records a backward path to `x`. `tiled_matmul` below has the same boundary. They are forward-computation experiments, not drop-in replacements for differentiable operations in a training model. A training implementation would need a `Function` with a backward rule as well as an efficient forward kernel.

18.3.2 Tiled matrix multiply

The matrix product in a `Linear` layer needs the whole reduction over its shared dimension. For A of shape (M, K) and B of shape (K, N), the caller expects a fresh

(M, N) output. `vectorized_matmul` validates the input ranks and inner dimensions, calls `np.matmul(a.data, b.data)`, and wraps its result. It also supports broadcastable batch axes; `tiled_matmul` deliberately accepts only two-dimensional inputs.

Take $A = [[1, 2, 3], [4, 5, 6]]$ and $B = [[1, 0], [0, 1], [1, 1]]$. With a tile size of 2, the first two terms of each inner product contribute `[[1,2],[4,5]]`. The final term contributes `[[3,3],[6,6]]`. The result must include both contributions. This small example exposes the state that the tiled implementation must maintain.

The function allocates `C` as zeros before the loops. Each input slice is a NumPy view into `a.data` or `b.data`; the code reads those views and adds the block product into a slice of `C`. The inputs remain unchanged. Returning `Tensor(C)` copies the accumulator into the output's owned storage.

```
def tiled_matmul(a: Tensor, b: Tensor, tile_size: int = 64) -> Tensor:
    """
    Cache-aware matrix multiplication using tiling (also called blocking).
    """
    # Input validation
    if len(a.shape) != 2 or len(b.shape) != 2:
        raise ValueError(
            f"Tiled matrix multiplication requires 2D tensors\n"
            f"  ❌ Got shapes {a.shape} and {b.shape} ({len(a.shape)}D and {len(b.shape)}D tensors)\n"
            f"  🔔 Tiling partitions a matrix into 2D blocks, so there must be exactly two axes\n"
            f"  🔧 Add dimensions with reshape: tensor.reshape(1, -1) for a row vector or tensor.reshape(-1, 1) for a column"
        )

    if a.shape[-1] != b.shape[-2]:
        raise ValueError(
            f"Tiled matrix multiplication shape mismatch: {a.shape} @ {b.shape}\n"
            f"  ❌ Inner dimensions don't match: a.shape[-1]={a.shape[-1]} vs b.shape[-2]={b.shape[-2]}\n"
            f"  🔔 Each tile of A's columns must align with tiles of B's rows for block multiplication\n"
            f"  🔧 Reshape to align: b.reshape({a.shape[-1]}, -1) or transpose if dimensions are swapped"
        )

    if not isinstance(tile_size, (int, np.integer)) or tile_size < 1:
        raise ValueError(
            f"Tile size must be at least 1, got {tile_size}\n"
            f"  🔔 The tile is the block edge length, so it has to be a positive number of rows/columns"
        )

    A, B = a.data, b.data
```

```
    M, K = A.shape
    N = B.shape[1]
    C = np.zeros((M, N), dtype=A.dtype)

    # Three loops over TILES. The inner block product is one NumPy matmul on
    # data small enough to stay in cache for the whole (i, j) tile.
    for i0 in range(0, M, tile_size):
        i1 = min(i0 + tile_size, M)
        for j0 in range(0, N, tile_size):
            j1 = min(j0 + tile_size, N)
            for k0 in range(0, K, tile_size):
                k1 = min(k0 + tile_size, K)
                C[i0:i1, j0:j1] += A[i0:i1, k0:k1] @ B[k0:k1, j0:j1]

    return Tensor(C)
```

The validation checks establish the contract: two matrices, matching inner dimensions, and a positive integer tile size. The mechanism is the three loops at the end. They walk over blocks of rows of C, blocks of columns of C, and blocks of the shared dimension, and the body multiplies one block of A by one block of B and accumulates into one block of C. The `min` calls let the last tile along each edge be smaller than `tile_size` when the matrix dimension is not a multiple of it, which tile sizes 2 and 4 in listing 18.3 exercise. The invariant belongs to the innermost loop: before each `k0` iteration, the output slice contains the contributions from all earlier slices of the shared dimension. `+=` adds the next contribution. Replacing it with `=` would keep only the last block product, and tests where the shared dimension fits in one tile would miss that error. After the loop, every part of the inner product has contributed exactly once.

Listing 18.3 Partial-tile check: Small integer-valued matrices permit exact result checks.

```
from tinytorch.perf.acceleration import tiled_matmul, vectorized_matmul

A = Tensor([[1, 2, 3], [4, 5, 6]])
B = Tensor([[1, 0], [0, 1], [1, 1]])
expected = np.array([[4, 5], [10, 11]], dtype=np.float32)
for tile_size in (1, 2, 4):
    np.testing.assert_array_equal(tiled_matmul(A, B, tile_size).data, expected)
np.testing.assert_array_equal(vectorized_matmul(A, B).data, expected)
```

The multiply within each block uses NumPy's `@`. NumPy documents that `matmul` uses an optimized BLAS library when possible; BLAS is a collection of compiled linear-algebra routines. The Python loops expose block selection while the library performs each block's arithmetic. NumPy matmul documentation.

This extra Python loop structure can cost more than one whole-matrix library call. The block size also changes summation order, so larger floating-point examples need numerical tolerances rather than bitwise equality. Measure against `vectorized_matmul`

using the same inputs. A faster result is useful only after it satisfies the same numerical contract.

18.3.3 Convolution as one matrix multiply

Chapter 9 ended with a trade it could only describe. Module 09's `Conv2d` computes every multiply-add as a separate Python operation, and the chapter modeled im2col as the way out: copy each patch into a row, flatten each filter into a column, and let one matrix multiply do every dot product. Module 17 builds that path into the package, and it is the one place in the module where acceleration reaches a layer a model actually uses.

↩ **Jump back:** Chapter 9 *The seven nested loops of Conv2d and the high-performance im2col lowering promise.*

The chapter model in Chapter 9 filled the patch matrix one output position at a time, which is still a Python loop over $NH_{\text{out}}W_{\text{out}}$ positions. `im2col` turns the loop around. For kernel element (i, j), the input values it touches at every output position form one strided slice of the padded input, so the function copies K^2 slices instead of $NH_{\text{out}}W_{\text{out}}$ patches. For a 3×3 kernel that is nine copies whatever the image size.

```
def im2col(x: Tensor, kernel_size: int, stride: int = 1, padding: int = 0) -> Tensor:
    """
    Unroll every convolution patch of a batch of images into one row of a matrix.
    """
    if len(x.shape) != 4:
        raise ValueError(
            f"im2col requires a 4D tensor (batch, channels, height, width)\n"
            f"  ✗ Got shape {x.shape} ({len(x.shape)}D)\n"
            f"  💡 Convolution slides over the last two axes of every channel of every image, so all four axes must be present\n"
            f"  🔧 Add a batch axis for a single image: x.reshape(1, *x.shape)"
        )

    N, C, H, W = x.shape
    if kernel_size < 1 or stride < 1 or padding < 0:
        raise ValueError(
            f"Invalid im2col parameters: kernel_size={kernel_size}, stride={stride}, padding={padding}\n"
            f"  💡 kernel_size and stride must be at least 1, and padding cannot be negative"
        )
    if kernel_size > H + 2 * padding or kernel_size > W + 2 * padding:
        raise ValueError(
            f"Kernel does not fit the padded input\n"
            f"  ✗ kernel_size={kernel_size} but the padded input is {H + 2 * padding}×{W + 2 * padding}\n"
            f"  💡 Every patch must lie inside the padded image, so the kernel can be no larger than it\n"
            f"  🔧 Use a smaller kernel or more padding"
        )
```

```
    padded = np.pad(x.data, ((0, 0), (0, 0), (padding, padding),
    (padding, padding)),
                    mode='constant', constant_values=0)
    out_h = (H + 2 * padding - kernel_size) // stride + 1
    out_w = (W + 2 * padding - kernel_size) // stride + 1

    # One strided slice per kernel offset: slice (i, j) holds, for
       every output
    # position at once, the input value that kernel element (i, j)
       multiplies.
    cols = np.empty((N, C, kernel_size, kernel_size, out_h, out_w),
    dtype=padded.dtype)
    for i in range(kernel_size):
        for j in range(kernel_size):
            cols[:, :, i, j, :, :] = padded[:, :,
                                            i:i + stride *
    out_h:stride,
                                            j:j + stride *
    out_w:stride]

    # Positions first (n, oh, ow), then the patch in (c, ki, kj) order
       to match
    # weight.reshape(out_channels, -1).
    cols = cols.transpose(0, 4, 5, 1, 2, 3).reshape(N * out_h * out_w,
    C * kernel_size * kernel_size)
    return Tensor(cols)
```

The six-axis buffer `cols` has shape $(N, C, K, K, H_{\text{out}}, W_{\text{out}})$. After the loop, `cols[n, c, i, j, oh, ow]` holds the input value that kernel element (i, j) of channel c multiplies at output position (oh, ow) of image n. The transpose moves the three position axes to the front and the reshape flattens the rest, in the order (channel, kernel row, kernel column). That order is the one contract that matters: `weight.reshape(out_ch, -1)` flattens a filter in exactly that order, and a patch matrix with its columns permuted still multiplies without error while pairing the wrong pixels with the wrong weights. The unit test checks it with a two-channel input whose channels are numbered apart.

```
def im2col_conv2d(x: Tensor, weight: Tensor, bias: Optional[Tensor] =
None,
                  stride: int = 1, padding: int = 0) -> Tensor:
    """
    2D convolution computed as one matrix multiply over an im2col
    patch matrix.
    """
    if len(weight.shape) != 4 or weight.shape[2] != weight.shape[3]:
        raise ValueError(
            f"im2col_conv2d requires a 4D weight with a square
            kernel\n"
            f"  ✗ Got weight shape {weight.shape}\n"
            f"  💡 Conv2d stores its filters as (out_channels,
            in_channels, k, k)\n"
```

```
            f"  🔧 Pass conv.weight from a Conv2d layer"
        )
    if len(x.shape) != 4 or x.shape[1] != weight.shape[1]:
        raise ValueError(
            f"Input channels do not match the filters\n"
            f"  ❌ Input shape {x.shape}, weight shape {weight.shape}\n"
            f"  🔔 Each filter has one slice per input channel, so x.shape[1] must equal weight.shape[1]\n"
            f"  🔧 Check that the input is (N, C, H, W) with C = {weight.shape[1]}"
        )

    N, _, H, W = x.shape
    out_ch, _, k, _ = weight.shape
    out_h = (H + 2 * padding - k) // stride + 1
    out_w = (W + 2 * padding - k) // stride + 1

    cols = im2col(x, kernel_size=k, stride=stride, padding=padding)
    w_matrix = Tensor(weight.data.reshape(out_ch, -1).T)

    out = vectorized_matmul(cols, w_matrix).data          # (N*out_h*out_w, out_ch)
    if bias is not None:
        out = out + bias.data

    return Tensor(out.reshape(N, out_h, out_w, out_ch).transpose(0, 3, 1, 2))
```

`im2col_conv2d` is the convolution in three steps. It builds the patch matrix, multiplies it by the transposed filter matrix with the `vectorized_matmul` from earlier in the module, and moves the channel axis from last to second. It reads `.data` and wraps the result in a new `Tensor`, like the other helpers in this chapter, so it computes Chapter 9's forward pass but records no backward path.

Listing 18.4 The loops are the reference: The matrix-multiply convolution must reproduce Module 09's output before its timing means anything.

```
from tinytorch.core.spatial import Conv2d
from tinytorch.perf.acceleration import im2col_conv2d

conv = Conv2d(3, 16, kernel_size=3, padding=1)
x = Tensor(rng.standard_normal((2, 3, 16, 16)).astype(np.float32))
np.testing.assert_allclose(im2col_conv2d(x, conv.weight, conv.bias, padding=1).data,
                           conv(x).data, atol=1e-5)
```

The module's analysis cell times both versions on two layers. In one run on an Apple M5 Max laptop (NumPy 2.5 with Apple's Accelerate BLAS), a $2 \times 3 \rightarrow 16$ layer on 16×16 images takes 33.3 ms through the loops and 0.043 ms through im2col, and a

$2 \times 16 \to 32$ layer takes 334 ms against 0.096 ms, about 770 and 3,500 times faster. Repeated runs move these times by several percent, and other machines will give other ratios. The patch matrix is 9.0 times the size of the input in both cases, as the storage count in Chapter 9 predicted. Neither version does less arithmetic; the difference is whether the multiply-adds run in the interpreter or inside one library call.[69]

[69] **Implicit GEMM in production convolution libraries**: While explicit `im2col` lowers convolution to standard Level 3 GEMM, materializing the patch matrix X_{col} in DRAM incurs a severe K^2 memory footprint penalty ($9\times$ expansion for 3×3 kernels). Production acceleration libraries (such as NVIDIA cuDNN and oneDNN) circumvent this via *implicit GEMM*: instead of constructing X_{col} in memory, GPU thread blocks compute patch index arithmetic on the fly inside register address calculations. Each thread loads the appropriate image pixel directly into shared memory or registers as needed by the matrix multiply, gaining GEMM computational efficiency without allocating any intermediate buffer.

18.3.4 Training through im2col

A convolution that only runs forward can evaluate a CNN but not train one. In matrix form the forward pass is $Y = X_{\text{col}}W + b$, with X_{col} the patch matrix and W the flattened filters. Writing G for the output gradient in the same row layout, the matrix-multiply rule from Chapter 6 gives $\partial W = X_{\text{col}}^{\top} G$ and $\partial X_{\text{col}} = G W^{\top}$, two matrix multiplies of the same size as the forward pass, and ∂b is a column sum of G. The one step that is not a matrix multiply turns ∂X_{col} into a gradient for the image. Every pixel was copied into several rows, so its gradient is the sum of the gradients of its copies. That is col2im.

```
def col2im(cols: Tensor, x_shape: Tuple[int, int, int, int],
    kernel_size: int,
           stride: int = 1, padding: int = 0) -> Tensor:
    """
    Add every row of a patch-gradient matrix back into the image it
    came from.
    """
    N, C, H, W = x_shape
    out_h = (H + 2 * padding - kernel_size) // stride + 1
    out_w = (W + 2 * padding - kernel_size) // stride + 1
    expected = (N * out_h * out_w, C * kernel_size * kernel_size)
    if tuple(cols.shape) != expected:
        raise ValueError(
            f"Patch matrix does not match the image it should come
            from\n"
            f"  ✗ Got cols shape {cols.shape}, expected {expected}
            for x_shape={x_shape}, "
            f"kernel_size={kernel_size}, stride={stride},
            padding={padding}\n"
            f"  🔔 col2im must use the same x_shape, kernel_size,
            stride, and padding as the im2col it reverses\n"
            f"  🔧 Pass the parameters the forward pass used"
        )

    # Undo im2col's layout: rows back to (n, oh, ow), columns back to
      (c, i, j)
    cols6 = cols.data.reshape(N, out_h, out_w, C, kernel_size,
    kernel_size).transpose(0, 3, 4, 5, 1, 2)

    padded = np.zeros((N, C, H + 2 * padding, W + 2 * padding),
    dtype=cols6.dtype)
    for i in range(kernel_size):
        for j in range(kernel_size):
            # Accumulate: a pixel covered by several patches collects
              all their gradients
```

```
            padded[:, :,
                   i:i + stride * out_h:stride,
                   j:j + stride * out_w:stride] += cols6[:, :, i, j,
    :, :]

    return Tensor(padded[:, :, padding:padding + H, padding:padding +
        W])
```

`col2im` walks the same kernel offsets and strided slices as `im2col`, with the copy turned into +=. Within one offset the slice touches each pixel at most once, so the in-place add is safe; across offsets, the adds are what combine overlapping patches. Assignment instead of addition would keep only the last patch's gradient for each pixel, and the unit test catches it by sending a gradient of one from every patch entry and checking that the 3×3 example comes back as the coverage counts `[[1,2,1],[2,4,2],[1,2,1]]`. A second test checks the defining property of a transpose, $\langle \text{im2col}(x), c \rangle = \langle x, \text{col2im}(c) \rangle$, for random x and c with stride and padding.[70]

`Im2colConv2dFunction` puts the three gradients into the same `Function` shape as Module 09's `Conv2dFunction`. Its forward pass saves the patch matrix and the flattened filters; its backward pass uses them.

```
def backward(self, grad_output: np.ndarray) -> Tuple[np.ndarray, ...]:
    """
    Turn the output gradient into gradients for the input, weight, and
    bias.
    """
    x, weight = self.inputs[0], self.inputs[1]
    out_ch, _, k, _ = weight.shape

    G = np.asarray(grad_output).transpose(0, 2, 3, 1).reshape(-1,
    out_ch)

    grad_weight = (self.cols.T @ G).T.reshape(weight.shape)
    grad_cols = G @ self.w_matrix.T
    grad_x = col2im(Tensor(grad_cols), x.shape, k, self.stride,
    self.padding).data

    if len(self.inputs) > 2:
        return grad_x, grad_weight, G.sum(axis=0)
    return grad_x, grad_weight
```

Keeping `self.cols` alive until backward holds the K^2-times-larger buffer for every convolution in a model until its gradient is computed. Rebuilding it from the saved input would trade that memory for a second `im2col`. The unit test compares all three gradients with Module 09's loops on the same weights, and the module test checks that one SGD step moves the weights the same way on both paths. In the same run, a full training step on the $2 \times 16 \to 32$ layer takes 1,116 ms through the loops and 0.23 ms through `Im2colConv2dFunction`, about 4,900 times faster.

---

[70] **The adjoint relationship of im2col and col2im**: In functional analysis, the inner product identity $\langle Ax, y \rangle = \langle x, A^* y \rangle$ defines the adjoint operator A^*. Because `im2col` is a linear transformation mapping input pixel arrays to 2D patch matrices, `col2im` is its exact linear adjoint (matrix transpose). In the forward pass, one pixel fans out to multiple overlapping patches; in the backward pass, the adjoint operator accumulates cotangents from all overlapping patches back into that single pixel coordinate.

18.4 A Worked Trace

18.4.1 GELU values and an operation ledger

Run the unfused GELU on five inputs, $x = [-2, -1, 0, 1, 2]$, and record every intermediate. Values are rounded to five places. The final columns count logical whole-array reads and writes for the mathematical operations only, excluding constructor copies, internal temporaries, and cache behavior. Five float32 values occupy 20 bytes.

Table 18.2: Intermediate values for three of the five inputs. The last row gives a hypothetical compiled single-pass baseline; neither TinyTorch helper implements that baseline.

Step	Expression	$x = -1$	$x = 1$	$x = 2$	Reads	Writes
1	$t_1 = x^3$	-1.00000	1.00000	8.00000	1	1
2	$t_2 = 0.044715\, t_1$	-0.04472	0.04472	0.35772	1	1
3	$t_3 = x + t_2$	-1.04472	1.04472	2.35772	2	1
4	$t_4 = \sqrt{2/\pi}\, t_3$	-0.83356	0.83356	1.88119	1	1
5	$t_5 = \tanh(t_4)$	-0.68238	0.68238	0.95460	1	1
6	$t_6 = 1 + t_5$	0.31762	1.68238	1.95460	1	1
7	$t_7 = x\, t_6$	-0.31762	1.68238	3.90920	2	1
8	$y = 0.5\, t_7$	-0.15881	0.84119	1.95460	1	1
	Operation total				**10**	**8**
	Ideal compiled total				**1**	**1**

At $x = 1$, the result is approximately 0.84119, matching the tanh GELU computation. Table 18.2 accounts for eighteen logical array passes, or 360 bytes, before Tensor copies. A hypothetical implementation that reads each input once and writes each output once accounts for 40 bytes. The ratio of nine compares two models, not the measured traffic of the two NumPy helpers, as analyzed in the Napkin Math below.

Track the byte traffic for an elementwise sequence over an N-element float32 array ($N = 10^6$, array footprint 4 MB):

- **Unfused Sequence (8 distinct operations)**:
  - Step 1: $x^3 \implies$ 1 read + 1 write = 8 MB
  - Step 2: $0.044715 \cdot t_1 \implies$ 1 read + 1 write = 8 MB
  - Step 3: $x + t_2 \implies$ 2 reads + 1 write = 12 MB
  - Step 4: $\sqrt{2/\pi} \cdot t_3 \implies$ 1 read + 1 write = 8 MB
  - Step 5: $\tanh(t_4) \implies$ 1 read + 1 write = 8 MB
  - Step 6: $1 + t_5 \implies$ 1 read + 1 write = 8 MB
  - Step 7: $x \cdot t_6 \implies$ 2 reads + 1 write = 12 MB
  - Step 8: $0.5 \cdot t_7 \implies$ 1 read + 1 write = 8 MB
  - Total unfused DRAM traffic: 10 reads + 8 writes = 18 passes × 4 MB = 72 MB.
- **Fused Kernel (1 streaming pass)**:
  - Load x_i once from memory into an on-chip register file (1 read = 4 MB).

- Evaluate all intermediate powers, products, and tanh within local register banks (zero DRAM traffic).
- Write final output y_i once back to main memory (1 write = 4 MB).
- Total fused DRAM traffic: 1 read + 1 write = 2 passes × 4 MB = 8 MB.

Under this logical-pass model, the ratio is 72 MB/8 MB = 9.0×. Caches can serve some unfused reads, so physical traffic and the resulting speedup must be measured; for a truly bandwidth-bound chain, the model bounds the gain at about 9×.

Listing 18.5 reproduces the values and checks the output against `fused_gelu`. In lines 1 to 3, constants and the 5-element test vector `x` are declared. Lines 5 to 12 step through the 8 operations sequentially, recording intermediate arrays and tracking logical array reads and writes. Lines 14 to 15 print each stage's values corresponding to table 18.2. In lines 17 to 21, total memory traffic is computed (360 bytes unfused vs. 40 bytes ideal compiled). Line 22 asserts the modeled 9.0× ratio, and line 23 confirms agreement within `np.allclose` tolerance between the step-by-step trace `y` and `fused_gelu(Tensor(x)).data`.

Listing 18.5 GELU intermediate ledger: Record values and modeled reads and writes.

```
SQRT_2_OVER_PI = np.float32(np.sqrt(2.0 / np.pi))
COEFF = np.float32(0.044715)
x = np.array([-2.0, -1.0, 0.0, 1.0, 2.0], dtype=np.float32)
steps = []
t1 = x ** 3;                    steps.append(("t1", t1, 1, 1))
t2 = COEFF * t1;                steps.append(("t2", t2, 1, 1))
t3 = x + t2;                    steps.append(("t3", t3, 2, 1))
t4 = SQRT_2_OVER_PI * t3;       steps.append(("t4", t4, 1, 1))
t5 = np.tanh(t4);               steps.append(("t5", t5, 1, 1))
t6 = 1.0 + t5;                  steps.append(("t6", t6, 1, 1))
t7 = x * t6;                    steps.append(("t7", t7, 2, 1))
y = 0.5 * t7;                   steps.append(("y",  y,  1, 1))

for name, val, r, w in steps:
    print(f"{name:2s} {np.round(val, 5)}  reads={r} writes={w}")

array_bytes = x.nbytes                                  # 20 bytes
unfused = sum(r + w for _, _, r, w in steps) * array_bytes
ideal_compiled = 2 * array_bytes
print(f"operation ledger: {unfused} B, ideal compiled:
↪   {ideal_compiled} B, "
      f"ratio {unfused / ideal_compiled:.1f}x")
assert unfused / ideal_compiled == 9.0
assert np.allclose(y, fused_gelu(Tensor(x)).data)
```

18.4.2 Tiling: which blocks are loaded

Return to the $(2, 3)$ by $(3, 2)$ product from The Code with `tile_size=2`. There is one output tile, but two iterations along the shared dimension. Initially `C=[[0,0],[0,0]]`. At `k0=0`, the product of the $(2, 2)$ slices adds `[[1,2],[4,5]]`. At `k0=2`, `min(k0+2,3)` gives 3, so the final slices have shapes $(2, 1)$ and $(1, 2)$. Their product adds `[[3,3],[6,6]]`, leaving `C=[[4,5],[10,11]]`.

Figure 18.3 shows why both contributions belong to the same output tile.

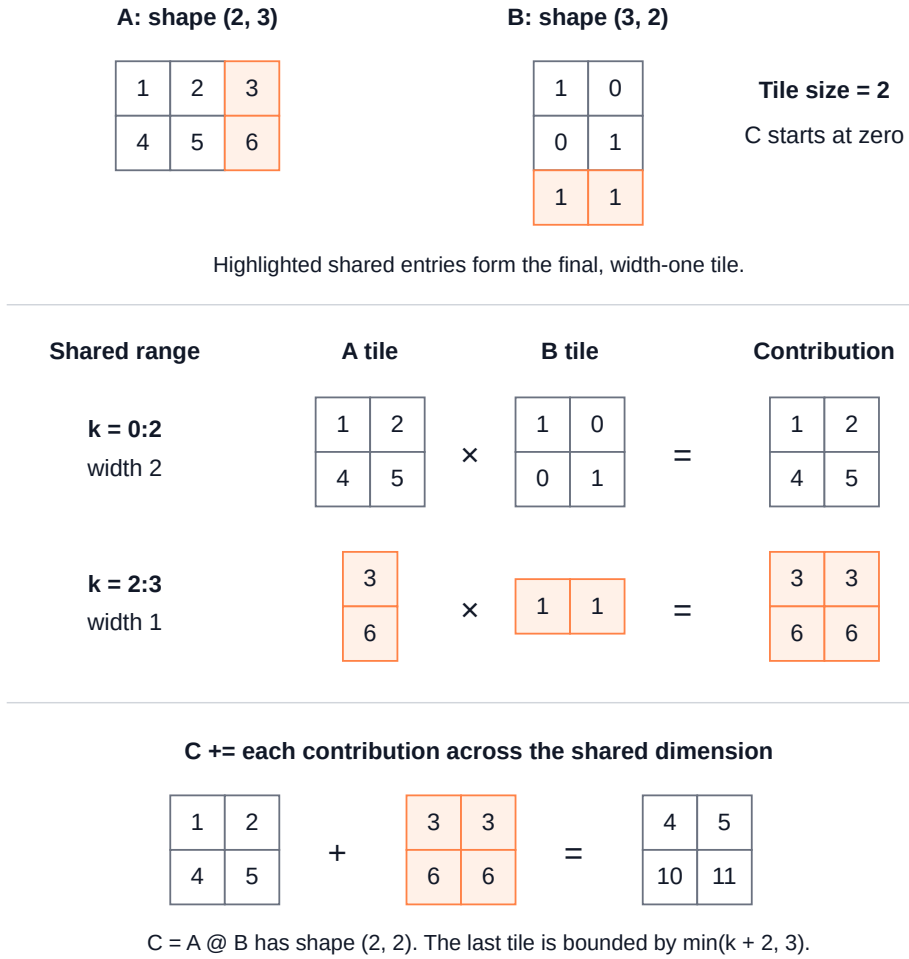


Figure 18.3: **Accumulation across the shared dimension**: With tile size two, the first shared tile contributes [[1,2],[4,5]], and the final one-element tile contributes [[3,3],[6,6]]. Adding both produces AB = [[4,5],[10,11]].

The `+=` is essential. Assignment would discard the first contribution and return `[[3,3],[6,6]]`. An input whose shared dimension fits in one tile would hide that mistake. Listing 18.6 checks both contributions independently before comparing their sum with the source function.

Listing 18.6 Two contributions to one tile: Check both products before accumulating them.

```
first = A.data[:, :2] @ B.data[:2, :]
last = A.data[:, 2:3] @ B.data[2:3, :]
np.testing.assert_array_equal(first, [[1, 2], [4, 5]])
np.testing.assert_array_equal(last, [[3, 3], [6, 6]])
np.testing.assert_array_equal(first + last, tiled_matmul(A, B, 2).data)
```

The input blocks requested can be counted without simulating hardware cache internals. Listing 18.7 sums block sizes under the simplified load model from The Idea. Partial tiles use their actual dimensions.

Listing 18.7 Input-load model: Count block requests without measuring cache traffic.

```
def count_tile_loads(M, N, K, T):
    loads = 0
    for i0 in range(0, M, T):
        for j0 in range(0, N, T):
            for k0 in range(0, K, T):
                di = min(T, M - i0); dj = min(T, N - j0); dk = min(T,
↪   K - k0)
                loads += di * dk + dk * dj          # one block of A,
↪   one of B
    return loads

def count_naive_loads(M, N, K):
    return M * N * 2 * K                            # a row and a
    ↪   column per output

for (n, T) in [(4, 2), (4, 4), (1024, 32), (1024, 64)]:
    naive, tiled = count_naive_loads(n, n, n), count_tile_loads(n, n,
↪   n, T)
    print(f"n={n:5d} T={T:3d}  naive={naive:>12,}  tiled={tiled:>12,}
    ↪   "
          f"ratio={naive / tiled:.0f}")
assert count_naive_loads(4, 4, 4) == 128 and count_tile_loads(4, 4, 4,
↪   2) == 64
```

At $n = 1024$ and $T = 64$, this model counts 2,147,483,648 scalar input requests against 33,554,432 blocked input requests. Multiplying by four gives about 8.6 GB against 134 MB of logical input data. The model excludes output reads and writes, and real cache traffic can differ. Both algorithms still compute the same 1,073,741,824 multiply-add pairs.

18.4.3 im2col: which pixels land in which row

Take one 3×3 image holding the values 1 through 9 and a 2×2 kernel with stride 1. There are four output positions, and at kernel offset $(0, 0)$ the strided slice is the top-left 2×2 block `[[1,2],[4,5]]`, one value per position. Offset $(0, 1)$ gives `[[2,3],[5,6]]`,

offset $(1, 0)$ gives `[[4,5],[7,8]]`, and offset $(1, 1)$ gives `[[5,6],[8,9]]`. Reading the four slices at position $(0, 0)$ in offset order gives its patch `[1,2,4,5]`; the other positions give `[2,3,5,6]`, `[4,5,7,8]`, and `[5,6,8,9]`. The center value 5 appears in all four rows, which is the storage cost in miniature: nine input values became sixteen matrix entries.

18.5 From TinyTorch to Production

A compiled fused kernel can compute all eight GELU steps on one element before advancing to the next. A kernel is a compiled numerical routine; fusion combines operations that would otherwise run separately. Registers can hold the scalar intermediates, so those intermediates need not become full arrays. This is the hypothetical two-pass baseline in the worked trace, whereas the TinyTorch helpers remain eager NumPy computations.

PyTorch's performance guide describes combining pointwise operations through TorchInductor and `torch.compile`.[71] Such a compiler examines a computation larger than one individual array operation, which lets it choose where to retain intermediates. Whether it fuses a particular chain depends on the supported operations and backend, as examined in the production bridge below. PyTorch performance tuning guide.

In standard eager execution (both PyTorch eager mode and TinyTorch), each operator dispatches an isolated C++ or CUDA kernel. Intermediate tensors are written to high-bandwidth device memory (HBM/DRAM) and immediately read back by the next kernel, consuming memory bandwidth and incurring kernel launch overhead.

PyTorch 2.0 addresses this with `torch.compile` and **TorchInductor**:

- **Graph Tracing**: TorchDynamo captures Python bytecode and extracts a clean computational graph of PyTorch operations without user code modification.
- **Pattern Matching and Fusion**: TorchInductor inspects operator chains (such as linear + bias + GELU, or multi-step elementwise activations) and fuses them into a single loop nest.
- **Code Generation**: Inductor generates C++ OpenMP kernels (with SIMD vectorization for CPU) or OpenAI **Triton** kernels (for GPU).
- **Register Storage**: Intermediate results (x^3, intermediate products, hyperbolic tangents) can stay in processor registers or L1 cache, so only the input read and output write reach DRAM.

[71] **Pointwise vs. reduction kernel fusion**: Compiler backends (such as PyTorch Inductor and OpenAI Triton) classify fusion into two primary categories: pointwise fusion and reduction fusion. Pointwise operations (such as Add, Mul, ReLU, and GELU) require no communication between distinct tensor coordinates, making them embarrassingly parallel and straightforward to fuse into a single loop over processor registers. Reduction operations (such as LayerNorm, Softmax, or GEMM), however, require aggregating values across coordinates. Fusing a reduction requires inter-thread communication through GPU shared memory, tree-based reduction sweeps, or warp-level shuffle instructions (`__shfl_down_sync`).

Table 18.3: The production alternative changes who schedules the operations and where intermediates can live. It does not make NumPy expression syntax a fusion guarantee.

	TinyTorch helpers	Compiled fused alternative
Execution	NumPy evaluates successive array operations	One routine may evaluate a supported chain
Intermediate storage	Temporary arrays; extra retained Tensor copies in `unfused_gelu`	Scalar intermediates may stay in registers
Multiplication	Explicit Python blocks or one library call	Tuned library or compiled blocking strategy

Validation	Compare outputs, shapes, and timing	The same checks, plus compiler-specific constraints

The alternatives in table 18.3 preserve the need to validate outputs. The load models establish a performance hypothesis, but hardware limits dictate realized speedups. A roofline ridge occurs at peak arithmetic throughput divided by memory bandwidth: a 10 TFLOP/s processor with 100 GB/s bandwidth yields a ridge of 100 FLOPs per byte. Workloads operating below that ridge remain memory bound regardless of nominal intensity.

Tiling and fusion also incur overheads: larger blocks demand working storage, fused chains increase register pressure, and Python dispatch can offset cache savings. The core methodology remains invariant: establish numerical equivalence within a stated tolerance, quantify modeled memory reductions, and benchmark identical workloads on target hardware.

18.6 Summary

`unfused_gelu` allocates seven intermediate Tensors, whereas `fused_gelu` returns the final Tensor from an eagerly evaluated expression without autograd overhead. Both preserve inputs and return identical shapes. The memory ledger reveals theoretical traffic reductions without implying that eager NumPy reaches compiled kernel throughput.

`tiled_matmul` accumulates block products across shared-dimension tiles into a preallocated buffer, maintaining the invariant that every block contributes exactly once. Timing against `vectorized_matmul` highlights interpreter loop overhead relative to BLAS routines.

`im2col` lowers 2D convolutions into matrix multiplication by extracting K^2 strided slices into a patch matrix. Inverting this mapping via `col2im` provides exact backward gradients through scatter-add accumulation, enabling end-to-end training via BLAS kernels.

Chapter 19 advances from within-operator reuse to cross-step caching. During autoregressive generation, previous keys and values remain invariant as tokens arrive, shifting the memory bottleneck from recomputation to cache management.

18.7 Further Reading

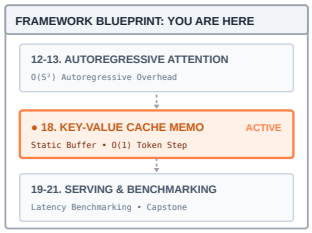
- **Goto and van de Geijn** (2008). "Anatomy of high-performance matrix multiplication." *ACM TOMS*. The blocking that turns Level 3 reuse into speed on real caches.
- **Chen et al.** (2018). "TVM: an automated end-to-end optimizing compiler for deep learning." *OSDI*. A compiler that performs the operator fusion this chapter expresses by hand.
- **Jouppi et al.** (2017). "In-datacenter performance analysis of a tensor processing unit." *ISCA*. The definitive look at how specialized hardware structures like systolic arrays accelerate foundational matrix multiplication loops.
- **Lawson et al.** (1979). "Basic Linear Algebra Subprograms for Fortran usage." *ACM TOMS*. The original BLAS interface behind the three levels above.

🔥 Chapter 19

The KV Cache: Remembering Keys and Values During Generation

Autoregressive text generation produces output one token at a time: each generated token is appended to the context, and the extended sequence is passed back to the model. In a naive implementation, every step reruns the model over the whole prefix, recomputing the keys and values of tokens whose keys and values have not changed. Over S generated tokens, the projection work grows as $O(S^2)$ and the attention scores as $O(S^3)$ (recalling the causal attention projection and scoring mechanisms established in Chapter 12).

In modern transformer inference libraries, this recomputation is avoided by enabling Key-Value caching:



Framework Blueprint: Module 18 eliminates O(S²) autoregressive recomputation through static Key-Value cache buffer management.

```python
import torch
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("gpt2")

# Autoregressive forward pass with KV caching enabled
input_ids = torch.tensor([[50256]])  # Initial prompt token
outputs = model(input_ids, use_cache=True)

# Inspect cached Key and Value states across all decoder blocks
past_key_values = outputs.past_key_values
print("Cached layers count:", len(past_key_values))
print("Key cache shape per layer:", past_key_values[0][0].shape)  # (B, H, S, d_k)
```

Underneath that simple flag, the runtime manages a dedicated memory buffer system:

1. **Static buffer pre-allocation:** The runtime pre-allocates contiguous memory buffers for Key and Value projections up to the maximum context window, avoiding expensive dynamic memory reallocations at every generated token.
2. **Single-token projection and moving write cursor:** At each decode step, only the newest single token is projected through W_Q, W_K, W_V. The new key and value vectors are written into the cache at the current sequence index cursor, leaving previous keys and values untouched.
3. **Asymptotic projection reduction:** Caching cuts per-step projection cost from $O(t)$ rows to a single row, dropping cumulative sequence projection work from $O(S^2)$ to $O(S)$; attention still reads the growing prefix.

This chapter constructs TinyTorch's KV cache, checks that cached and full-prefix attention agree within floating-point tolerance, and then measures generation time for

both. Before building the cache, inspecting the arithmetic waste of uncached generation reveals why naive autoregression scales poorly.

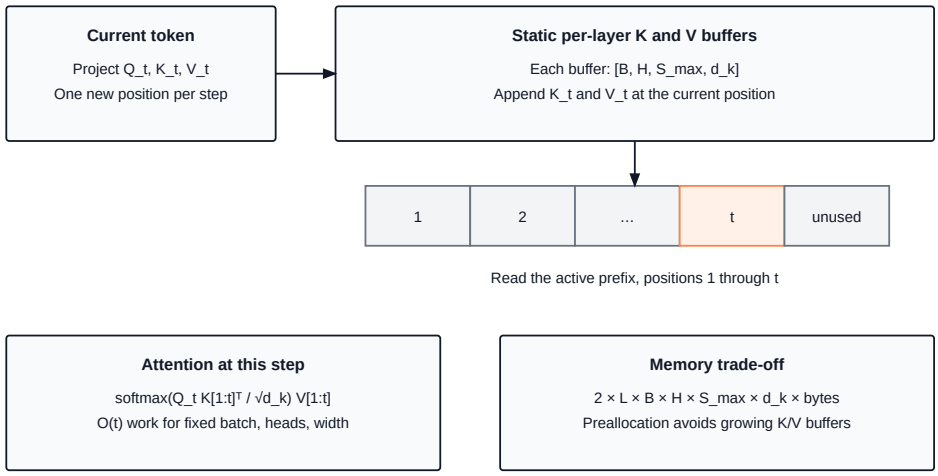


Figure 19.1: **The KV cache**: Each step writes one new key and value per head and layer, then attends over the growing prefix. Reserved capacity stays fixed while the number of keys read increases.

19.1 The Problem

↩ **Jump back:** Chapter 12 *Causal attention score matrices and key/value projection mechanics.*

Synthesis II's generation loop keeps a growing list of token ids. At each step it hands the whole list to `model.forward`, reads the logits at the last position, samples one token, appends it, and goes around again. Nothing in that loop remembers the previous step. Inside every attention layer, the forward pass projects all t tokens into queries, keys, and values, forms the $t \times t$ score matrix, and produces t output rows, of which the loop uses exactly one.

Counting rows makes the waste concrete. Token x_1 is pushed through W_K and W_V at step 1, again at step 2, and again at every step after that. Across S successive prefixes of lengths 1 through S, the number of rows projected is $1+2+\cdots+S = S(S+1)/2$, when S would have been enough. Listing 19.1 checks the triangle by counting each prefix.

Listing 19.1 Repeated projection count: Each prefix reprojects its earlier rows.

```
def naive_generation_cost(num_steps):
    """Rows pushed through W_k and W_v when every step re-feeds the
    ↪   whole sequence."""
    total = 0
    for t in range(1, num_steps + 1):
        rows_this_step = t              # tokens x_1 .. x_t are all
↪   re-projected
        total += rows_this_step
    return total

for S in (10, 100, 500):
    print(f"S={S:4d}: {naive_generation_cost(S):8,d} rows projected, "
          f"{naive_generation_cost(S) / S:6.1f}x more than the {S}
          ↪   needed")
# S=  10:       55 rows projected,     5.5x more than the 10 needed
# S= 100:    5,050 rows projected,    50.5x more than the 100 needed
# S= 500:  125,250 rows projected,   250.5x more than the 500 needed
```

Attention adds a second source of repeated work. The score matrix at step t has t^2 entries, so summed over a generation it grows as S^3, and the $t-1$ output rows that get discarded each step were each a full pass through the value matrix. Step 500 does roughly 500 times the projection work of step 1 and 250,000 times the score work, which is the recomputation Synthesis II counted. Every step also allocates fresh tensors one row larger than the last, so the allocator is busy exactly when the arithmetic is busiest.

The remedy is the oldest trick in programming. If a computation is repeated with the same inputs, store the result and look it up. The name for that trick is **memoization**, and the only question is which results are safe to store.

↪ **Jump ahead:** Chapter 20 *Measuring KV cache latency and memory footprint under strict monotonic timing controls.*

19.2 The Idea

19.2.1 Keys and values of past tokens never change

In production serving engines like vLLM and TensorRT-LLM, efficient autoregressive decoding relies on the mathematical immutability of historical token representations. Because causal masking prevents later tokens from influencing earlier activations, the key and value vectors generated at past positions remain invariant for the remainder of sequence generation. Consider token i at some attention layer, with input row $\mathbf{x}_i$ and the layer's projection matrices W_K and W_V. Its key and value are

$$\mathbf{k}_i = \mathbf{x}_i W_K, \qquad \mathbf{v}_i = \mathbf{x}_i W_V.$$

Three facts together make $\mathbf{k}_i$ and $\mathbf{v}_i$ constants for the rest of the generation. The weights are frozen at inference time, so W_K and W_V are the same at every step. The projection is a function of $\mathbf{x}_i$ and the weights alone, with no dependence on any other token. Finally, $\mathbf{x}_i$ itself does not change when later tokens arrive, because of the causal mask. The mask

in every layer below this one forbids position i from attending to positions after i, so the row that reaches this layer for token i is a function of tokens 1 through i only, and those tokens are already fixed. The third fact is the one the whole scheme rests on. Without a causal mask, appending token t would change every earlier row at every layer, and there would be nothing safe to cache.[72]

[72] **Caching Rotary Position Embeddings (RoPE)**: In architectures employing Rotary Position Embeddings (such as LLaMA, Mistral, and Gemma), position information is embedded by rotating queries and keys by position-dependent angles: $\tilde{\mathbf{q}}_m = R_{\Theta,m}\mathbf{q}_m$ and $\tilde{\mathbf{k}}_n = R_{\Theta,n}\mathbf{k}_n$. Because the attention dot product depends strictly on relative displacement $(m-n)$ via $(R_{\Theta,m}\mathbf{q})^T(R_{\Theta,n}\mathbf{k}) = \mathbf{q}^T R_{\Theta,n-m}\mathbf{k}$, keys are rotated at their absolute write slot n = seq_pos *before* insertion into the KV cache. Once written, cached keys never need to be re-rotated when future tokens are generated.

19.2.2 Only the newest query is needed

Modern inference runtimes exploit the asymmetry of autoregressive generation by collapsing full matrix-matrix attention into a single vector-matrix operator. Because language modeling only requires predicting the immediate next token, the execution graph discards historical query rows and evaluates attention exclusively for the current time step. The loop reads the logits at the last position and discards the rest, so the only required attention output at step t is the final row. That row is

$$\mathbf{o}_t = \text{softmax}\left(\frac{\mathbf{q}_t K_{\le t}^T}{\sqrt{d_k}}\right) V_{\le t},$$

where $\mathbf{q}_t = \mathbf{x}_t W_Q$ is the new token's query, $K_{\le t}$ and $V_{\le t}$ are the $t \times d_k$ matrices whose rows are the cached keys and values plus the new one, and d_k is the key width as in Chapter 12. No causal mask is needed because the row only contains keys at positions up to t; there is nothing in the future to hide. A separate mask can still exclude an earlier key, for example a padding position. The cached implementation honors such masks. This is Chapter 12's formula restricted to one query row, and it is a vector-matrix product rather than a matrix-matrix product.

19.2.3 The per-step recurrence

In production serving loops, this incremental state update constitutes the core decode execution cycle executed across every transformer layer. By formulating generation as an append-and-score recurrence rather than a full-sequence re-evaluation, the engine replaces quadratic graph recomputation with a deterministic per-token sequence of operations. Putting the two facts together, one generation step at one attention layer does four things.

Core Invariant: Cursor Write-Head Advance

$\text{cache}[:, :, \text{seq_pos}, :] = k_{\text{new}}, \quad \text{seq_pos} \mathrel{+}= 1$

Preallocating max capacity turns each step's $O(t)$ buffer reallocation into an $O(1)$ slot write; reading the prefix still copies $O(t)$ rows.

1. Project only the new token: $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$, three products of a $1 \times D$ row with a $D \times d_k$ matrix.
2. Append $\mathbf{k}_t$ and $\mathbf{v}_t$ to the cached $K_{<t}$ and $V_{<t}$ to form $K_{\le t}$ and $V_{\le t}$.
3. Score $\mathbf{q}_t$ against all t cached keys, scale by $1/\sqrt{d_k}$, and take one softmax over the resulting t numbers.
4. Mix the t cached values with those weights to produce $\mathbf{o}_t$.

Nothing in steps 1 through 4 touches an old key or value except to read it. The cache is append-only, and a single integer tracks the filled prefix shared by the layers. The stored key and value arrays carry the rest of its state.

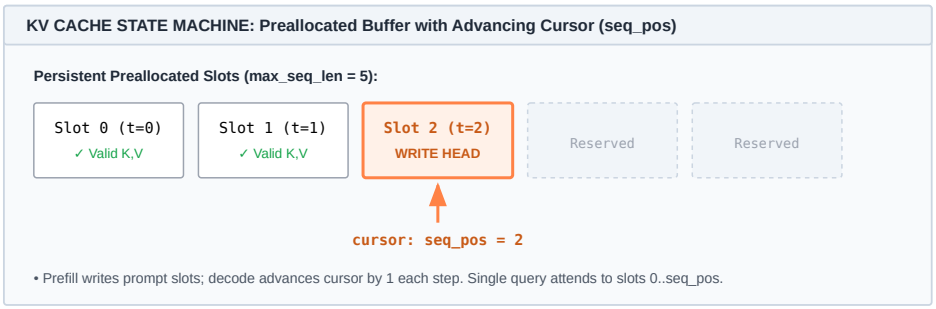
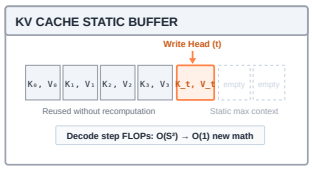


Figure 19.2: **KV cache advancing write cursor**: Rather than reallocating expanding key/value tensors each generation step, a fixed buffer of shape $(B, H, S_{\max}, d_k)$ is preallocated. Each token generation step writes the newly projected key and value into the single slot indexed by `seq_pos` and advances the cursor, so the persistent buffers are never reallocated; reads still copy the prefix.

19.2.4 What it costs and what it buys

The KV cache represents the quintessential systems trade-off in deep learning infrastructure: exchanging memory footprint for arithmetic throughput. While eliminating redundant projection FLOPs accelerates generation, the accumulating cache shifts the execution profile into a memory-bandwidth-bound regime and dictates the serving concurrency of GPU clusters. Per step, the projection work drops from t rows to one and the score work from t^2 entries to t. Summed over a generation of S tokens, the projections go from $O(S^2)$ to $O(S)$ and the scores from $O(S^3)$ to $O(S^2)$. The step still grows with t through the score row, so caching does not make decode constant-time. The two columns of table 19.1 count work for the same final output row. How much of the step this growing read consumes depends on the context length and model width.



KV Cache Buffer Reuse: Keys and values for previous tokens are retained in preallocated memory, converting O(S²) autoregressive projection FLOPs into an O(1) write per decode step.

Table 19.1: **Per-step work**: Work at generation step t for one attention layer, with and without the cache.

Per-step quantity	Naive loop	With the cache
Rows projected through W_Q, W_K, W_V	t	1
Score entries computed	t^2	t
Output rows computed	t	1
Persistent KV storage	rebuilt during each forward	fixed buffers; temporary prefix copies remain

The price is memory. Every token adds one key row and one value row per head per layer, and they stay for the life of the sequence. For a hypothetical model with 32 layers, 32 heads of width 128, and 16-bit values, one token costs $2 \times 32 \times 32 \times 128 \times 2$ bytes, which is 524,288 bytes per token. A 4,096-token context costs 2,147,483,648 bytes, about 2.15 GB, per sequence before the weights are counted. Serve 64 such sequences at once and the cache alone requires about 137 GB. Memory has been traded for compute, and whether the trade pays depends on the recomputation avoided, the bandwidth available,

and the cache capacity, as budgeted in the Napkin Math below. The cache saves repeated projections while retaining keys and values that each later query must read, which is the thread the production bridge picks up.

For autoregressive generation, calculate the KV cache memory footprint across context lengths and concurrency:

- **Per-Token Buffer Formula**:

$$\text{Memory per token} = 2\ (\text{key and value}) \times L \times D \times b$$

  where L is the number of transformer layers, D is model embedding dimension ($H \times d_k$), and b is byte width per float.

- **TinyGPT** ($L = 4, D = 128$, **FP32** $b = 4$):

$$2 \times 4 \times 128 \times 4\ \text{B} = 4{,}096\ \text{bytes per token} = 4\ \text{KB/token}.$$

  For its full $S = 256$ positions: 256×4 KB $= 1.0$ MB per sequence.

- **Production LLaMA-3-8B** ($L = 32, H_{kv} = 8, d_k = 128, D_{kv} = 1024$, **BF16** $b = 2$):

$$2 \times 32 \times 1024 \times 2\ \text{B} = 131{,}072\ \text{bytes per token} \approx 131\ \text{KB/token}.$$

  For $S = 8{,}192$ context length: $8{,}192 \times 131{,}072$ B $= 1.07$ GB per sequence. Serving a batch of 64 concurrent requests requires 64×1.07 GB $= 68.7$ GB dedicated exclusively to the KV cache, far exceeding the model weights (16 GB).

- **Decode Generation Latency Equation**: Because single-token generation is memory-bandwidth bound (Chapter 15), per-token decode latency is governed by memory streaming:

$$\text{Latency per token} \approx \frac{\text{Model Weight Bytes} + \text{Active KV Cache Bytes}}{\text{Memory Bus Bandwidth}}.$$

  On an accelerator with 1,000 GB/s memory bandwidth running LLaMA-3-8B (16 GB weights) with a 4 GB active batch KV cache, each token requires transferring 20 GB of data, yielding a physical decode speed limit of 20 ms/token (50 tokens/s).

Pause & Predict: KV Cache Memory Exhaustion

Suppose an inference server has 16 GB of free GPU VRAM available for KV cache allocations. It serves a model with $L = 32$ layers, $H_{kv} = 32$ key/value heads, head dimension $d_k = 128$, and 16-bit precision ($b = 2$ bytes). How many total tokens can fit across all concurrent KV caches before the server runs out of memory (OOM)? If each user request has an average context length of $S = 4{,}096$ tokens, how many concurrent users can the server support simultaneously?

19.3 The Code

19.3.1 A buffer sized up front, with one cursor

High-throughput inference engines avoid dynamic heap reallocations during generation by preallocating fixed-size tensor buffers up to the model's maximum sequence length.

By managing valid token positions through a monotonically advancing write cursor, the runtime converts expensive per-token allocations and tensor copies into zero-overhead contiguous slice writes. Every attention layer in TinyGPT needs somewhere to keep the keys and values of every token so far, and it needs them for as long as the sequence lives. In the 32-layer example that means 32 layers, each with one key buffer and one value buffer, and every generation step adds one row to all 64 of them. The container must retain those rows across model calls.

The obvious container is a list that grows, or in NumPy a `concatenate` of the old rows with the new one. Concatenate copies. At step t it allocates a fresh t-row array, copies the $t - 1$ rows already there, and frees the array from the step before. Over a generation of S tokens the copying alone moves $S(S + 1)/2$ rows, the same triangular growth analyzed in The Problem, only now measured in bytes copied rather than rows projected. The memory allocator runs once per layer per step, exactly when arithmetic throughput should peak, replacing redundant compute with allocation churn.

A write can cost the same at step 500 as at step 1 if the maximum sequence length is known before allocation. Allocate `max_seq_len` rows per layer once, keep one integer that says how many are filled, and make each step a single write at that cursor. The write copies one new row into its reserved slot. Reading the history starts with a NumPy slice up to the cursor, but `get` wraps that slice in a new `Tensor`. Chapter 1's constructor copies its input, so this implementation copies the prefix on read. Preallocation fixes persistent storage; it does not eliminate all temporary allocations.

```
class KVCache:
    """
    Efficient key-value cache for autoregressive generation.
    """

    def __init__(self, batch_size: int, max_seq_len: int, num_layers:
    ↪   int,
                 num_heads: int, head_dim: int):
        """
        Initialize KV cache for efficient generation.
        """
        for name, value in (("batch_size", batch_size),
        ↪   ("max_seq_len", max_seq_len),
                            ("num_layers", num_layers), ("num_heads",
↪   num_heads),
                            ("head_dim", head_dim)):
            if not isinstance(value, (int, np.integer)) or
            ↪   isinstance(value, bool) or value <= 0:
                raise ValueError(f"{name} must be a positive integer")
        self.batch_size = batch_size
        self.max_seq_len = max_seq_len
        self.num_layers = num_layers
        self.num_heads = num_heads
        self.head_dim = head_dim

        # Current sequence position (how many tokens are cached)
        self.seq_pos = 0
        self._generation_active = False
```

```
        # Cache storage: list of (key_cache, value_cache) tuples per layer
        self.caches = []

        for layer_idx in range(num_layers):
            # Pre-allocate cache tensors with maximum size
            # Shape: (batch_size, num_heads, max_seq_len, head_dim)
            key_cache = Tensor(np.zeros((batch_size, num_heads, max_seq_len, head_dim)))
            value_cache = Tensor(np.zeros((batch_size, num_heads, max_seq_len, head_dim)))

            self.caches.append((key_cache, value_cache))

    def update(self, layer_idx: int, key: Tensor, value: Tensor) -> None:
        """
        Update cache with new key-value pairs for given layer.
        """
        if not isinstance(layer_idx, (int, np.integer)) or not 0 <= layer_idx < self.num_layers:
            raise ValueError(
                f"Invalid layer index for cache update\n"
                f"  ✗ layer_idx={layer_idx} is out of range [0, {self.num_layers - 1}]\n"
                f"  🔔 KVCache was initialized with num_layers={self.num_layers}, so valid indices are 0 to {self.num_layers - 1}\n"
                f"  🔧 Check your transformer block loop: for layer_idx in range({self.num_layers})"
            )

        if self.seq_pos >= self.max_seq_len:
            raise ValueError(
                f"KV cache is full - cannot add more tokens\n"
                f"  ✗ Current position {self.seq_pos} has reached max_seq_len={self.max_seq_len}\n"
                f"  🔔 The cache was pre-allocated for {self.max_seq_len} tokens maximum. Autoregressive generation cannot exceed this limit.\n"
                f"  🔧 Either: (1) call cache.reset() to start a new sequence, or (2) create a larger cache with max_seq_len > {self.max_seq_len}"
            )

        expected_shape = (self.batch_size, self.num_heads, 1, self.head_dim)
        if key.shape != expected_shape or value.shape != expected_shape:
            raise ValueError(f"Cache update requires key and value shape {expected_shape}")
```

```
60
61          # Get cache for this layer
62          key_cache, value_cache = self.caches[layer_idx]
63
64          # Update cache at current position (efficient O(1) write)
65          # Note: We use .data here because caching is inference-only
            ↪   (no gradients needed)
66          # This avoids gradient tracking overhead during generation
67          key_cache.data[:, :, self.seq_pos:self.seq_pos+1, :] =
    ↪   key.data
68          value_cache.data[:, :, self.seq_pos:self.seq_pos+1, :] =
    ↪   value.data
69
70          # Note: seq_pos is advanced externally via advance() after all
            ↪   layers process
71
72      def get(self, layer_idx: int) -> Tuple[Tensor, Tensor]:
73          """
74          Retrieve cached key-value pairs for attention computation.
75          """
76          if not isinstance(layer_idx, (int, np.integer)) or not 0 <=
            ↪   layer_idx < self.num_layers:
77              raise ValueError(
78                  f"Invalid layer index for cache retrieval\n"
79                  f"  ✗ layer_idx={layer_idx} is out of range [0,
                    ↪   {self.num_layers - 1}]\n"
80                  f"  🔄 KVCache was initialized with
                    ↪   num_layers={self.num_layers}, so valid indices are
                    ↪   0 to {self.num_layers - 1}\n"
81                  f"  🔧 Check your transformer block loop: for
                    ↪   layer_idx in range({self.num_layers})"
82              )
83
84          # Get cache for this layer
85          key_cache, value_cache = self.caches[layer_idx]
86
87          # Return only the valid portion (up to current sequence
            ↪   position)
88          # seq_pos tracks where to write next, so we have seq_pos valid
            ↪   tokens
89          valid_len = self.seq_pos
90
91          # Note: Creating new Tensors from .data (no gradient tracking)
92          # This is correct for inference-only caching
93          cached_keys = Tensor(key_cache.data[:, :, :valid_len, :])
94          cached_values = Tensor(value_cache.data[:, :, :valid_len, :])
95
96          return cached_keys, cached_values
97
98      @contextmanager
99      def generation(self) -> Iterator["KVCache"]:
100         """Use cached attention only inside this inference scope.
```

```
        """
        previous = self._generation_active
        self._generation_active = True
        try:
            yield self
        finally:
            self._generation_active = previous

    def advance(self) -> None:
        """
        Advance sequence position after processing current token.
        """
        if self.seq_pos >= self.max_seq_len:
            raise ValueError("Cannot advance: KV cache is full")
        self.seq_pos += 1

    def reset(self) -> None:
        """
        Reset cache for new generation sequence.
        """
        self.seq_pos = 0

        # Zero out caches for clean state (helps with debugging)
        for layer_idx in range(self.num_layers):
            key_cache, value_cache = self.caches[layer_idx]
            key_cache.data.fill(0.0)
            value_cache.data.fill(0.0)

    def get_memory_usage(self) -> Dict[str, Union[int, float]]:
        """
        Calculate memory usage of the cache system.
        """
        # Calculate size of one cache tensor
        cache_size = self.batch_size * self.num_heads *
    self.max_seq_len * self.head_dim

        # Each layer has key_cache + value_cache
        total_cache_tensors = self.num_layers * 2
        total_elements = cache_size * total_cache_tensors
        total_bytes = total_elements * _BYTES_PER_FLOAT32
        total_mb = total_bytes / _BYTES_PER_MB

        return {
            'total_mb': total_mb,
            'per_layer_mb': total_mb / self.num_layers,
            'cache_tensors': total_cache_tensors,
            'total_elements': total_elements
        }
```

Three details carry the design. The cursor `seq_pos` is shared by all layers, and `update` does not move it; every layer writes at the same slot for the same token, and `advance` moves the cursor once after the whole model has processed that token. Splitting the

write from the advance is what lets one cache serve a stack of layers. `get` returns the rows before the cursor, which is the history and not the slot `update` just filled; the step below appends the row it has in hand, so the newest token attends to itself without the cache having to know which layer is asking. The bounds check in `update` refuses to write past `max_seq_len` rather than growing the buffer, which is the whole point of sizing it up front.

Each layer's key buffer and value buffer are separate `Tensor` objects of shape `(batch_size, num_heads, max_seq_len, head_dim)`, so the write is a slice assignment along the sequence axis and the read is a slice of the same array. `reset` zeroes the buffers as well as the cursor, a pass over memory that the cursor alone would make unnecessary; the module pays it so that a stale row can never be mistaken for a live one while debugging.

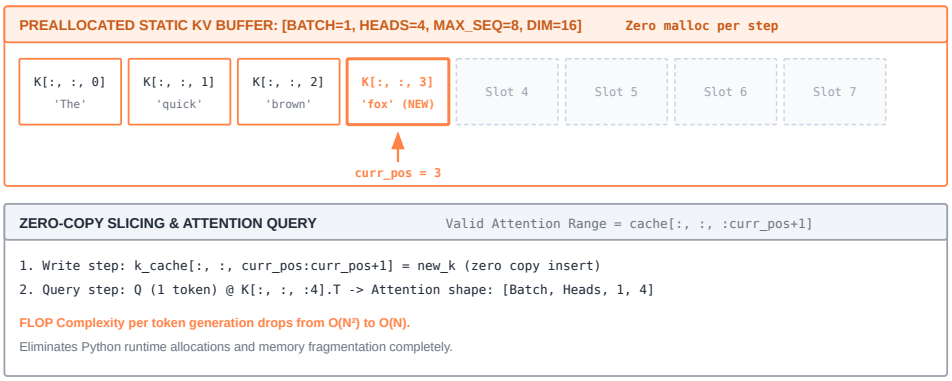
For batch size one, four layers of four heads of width 32 over 256 positions reserve $2 \times 4 \times 4 \times 256 \times 32 \times 4 = 1{,}048{,}576$ bytes. `get_memory_usage` divides by 10^6 and reports `total_mb` of about 1.05. That number measures reserved key/value storage only; temporary prefix copies and model parameters are separate.

>_ SOURCE CODE MAPPING MOD 18

FILE: src/18_memoization/18_memoization.py
EXPORT: tinytorch.perf.memoization
SYMBOLS: KVCache, CachedAttention

Extracted & verified at build time ✓ tested

Source Code Mapping: Module 18 extracts pre-allocated Key-Value caches, cursor write indexing, and cached attention directly from the working repository.



KV caching replaces quadratic recomputation with static ring-buffer pointers, reducing incremental token generation to linear O(N) cost.

Figure 19.3: **The KV cache buffer state machine**: Sizing key and value buffers up front for `max_seq_len` eliminates $O(S^2)$ memory reallocation during autoregressive generation. A write cursor `seq_pos` tracks the filled history. For each new token, keys and values are written into slot t, the past prefix is retrieved without causal masking, and the pointer advances once per sequence step.

19.3.2 One query row against the cached keys

At every step the generation loop reads one row of logits, the last one, and throws the rest away. Chapter 12's `MultiHeadAttention.forward` does not know that. Hand it the t tokens seen so far and it projects all t of them, forms the $t \times t$ score matrix, masks the upper triangle, and returns t output rows. At step 500 that is 1,500 row projections, 250,000 scores, and 499 output rows computed and thrown away so that one row can be read.

The Idea showed that the retained output row depends only on $\mathbf{q}_t$ and the cached $K_{\leq t}$ and $V_{\leq t}$. The cached step projects one row instead of t, writes its key and value into the cache, reads the history back, and scores that single query against the t keys. The score matrix collapses to a score row, the t softmaxes to one softmax over t numbers, and causal masking becomes unnecessary because every key in the prefix sits at or before

position t. An explicit mask can still remove keys from that prefix. The function takes the block's own attention layer, so it uses the layer's projections and head count and owns no weights of its own.

```
def _cached_generation_step(x: Tensor, attention: Any, cache_obj: "KVCache",
                            layer_idx: int, mask: Optional[Tensor] = None) -> Tensor:
    """
    Execute a single cached generation step for one new token.
    """
    batch_size = x.shape[0]
    num_heads = attention.num_heads
    head_dim = attention.head_dim

    # Step 1: Project new token to Q, K, V
    Q_new = attention.q_proj.forward(x)  # (batch, 1, embed_dim)
    K_new = attention.k_proj.forward(x)
    V_new = attention.v_proj.forward(x)

    # Step 2: Split into heads (batch, num_heads, 1, head_dim),
      reusing Module 12's helper
    Q_heads = attention._split_heads(Q_new, batch_size, 1)
    K_heads = attention._split_heads(K_new, batch_size, 1)
    V_heads = attention._split_heads(V_new, batch_size, 1)

    # Step 3: Update cache with new K, V
    cache_obj.update(layer_idx, K_heads, V_heads)

    # Step 4: Retrieve ALL cached K, V (includes history + new token)
    # cache_obj.get() only returns tokens already made visible by
      advance()
    # (by design -- see KVCache.get()), which does NOT yet include the
      token
    # just written by update() above. Append this step's own K/V,
      which we
    # already have in hand, so the new token attends to itself too.
    K_history, V_history = cache_obj.get(layer_idx)
    K_all = Tensor(np.concatenate([K_history.data, K_heads.data],
    axis=2))
    V_all = Tensor(np.concatenate([V_history.data, V_heads.data],
    axis=2))

    # Step 5: Compute attention using new Q with all cached K, V
    # This repeats Module 12's scale/mask/softmax/matmul on raw NumPy
      rather than
    # calling scaled_dot_product_attention, deliberately: that path
      builds autograd
    # Function nodes for every step, and a decode step needs no
      gradients. The
    # masking sentinel is imported from Module 12 rather than written
      out again.
```

```
    K_transposed = np.transpose(K_all.data, (0, 1, 3, 2))
    scores = np.matmul(Q_heads.data, K_transposed) / np.sqrt(head_dim)

    # A single query can still mask cached keys (for example, padding).
    # Match Module 12: binary masks, with at least one allowed key per query.
    if mask is not None:
        allowed = mask.data
        if np.any((allowed != 0) & (allowed != 1)):
            raise ValueError("Attention mask must contain only 0 (blocked) or 1 (allowed)")
        if allowed.ndim == 3:
            allowed = allowed[:, None, :, :]
        allowed = np.broadcast_to(allowed, scores.shape)
        if np.any(~np.any(allowed != 0, axis=-1)):
            raise ValueError("Attention mask must allow at least one key per query")
        scores = np.where(allowed != 0, scores, MASK_VALUE)

    # Stable softmax
    scores_max = np.max(scores, axis=-1, keepdims=True)
    exp_scores = np.exp(scores - scores_max)
    attention_weights = exp_scores / np.sum(exp_scores, axis=-1, keepdims=True)

    # Apply attention to values
    attention_output = np.matmul(attention_weights, V_all.data)

    # Step 6: Merge heads back (Module 12's helper) and project to output
    concat_output = attention._merge_heads(Tensor(attention_output), batch_size, 1)

    return attention.out_proj.forward(concat_output)
```

Read the six steps against the recurrence in The Idea. Step 1 projects the one new token through the layer's three projections, the vector-matrix shape Chapter 15 showed is memory bound. Step 2 splits the row across heads, so that every head has its own $1 \times d_k$ query, key, and value. Steps 3 and 4 are the exchange with the cache: write the new key and value at the cursor, read the history, and append the new row to it, so `K_all` holds tokens 1 through t with the newest last. Step 5 computes scores of shape `(batch, heads, 1, t)`, applies an optional binary mask, and normalizes along the last axis. A mask value of zero excludes a key; a query with no allowed key raises an error because its softmax would be undefined. The helper uses raw NumPy arrays for this inference-only calculation, so the cache does not preserve attention gradients. The surrounding model can still record other operations unless the caller enters `no_grad()`. Step 6 stitches the heads back together and applies the output projection, exactly as the full layer would.

The ordering of steps 3 and 4 is where this code goes wrong when written from scratch. The boundary is `seq_pos`: `get` excludes the current slot until `advance`. Forgetting the concatenation therefore removes self-attention; advancing before the concatenation

includes the new row twice. Reading the history before the write would still work if the new row were appended explicitly. The invariant is which positions appear exactly once, rather than write-before-read alone.

That function has to be called in place of the layer's `forward`, and only during generation. The module does that without editing the attention class at all. A `CachedAttention` stands in front of each block's attention layer, holds the original, and checks both the input shape and whether the caller has entered `cache.generation()`.

```
class CachedAttention:
    """
    Stand-in for one block's attention layer while a KV cache is
    enabled.
    """

    def __init__(self, attention: Any, cache_obj: KVCache, layer_idx:
        int) -> None:
        self.attention = attention      # the original
            MultiHeadAttention layer
        self.cache = cache_obj
        self.layer_idx = layer_idx

    def parameters(self) -> List[Tensor]:
        """The stand-in owns no weights of its own."""
        return self.attention.parameters()

    def forward(self, x: Tensor, mask: Optional[Tensor] = None) ->
        Tensor:
        """
        Route one call to the right path.
        """
        if not self.cache._generation_active or x.shape[1] > 1:
            return self.attention.forward(x, mask)
        return _cached_generation_step(x, self.attention, self.cache,
            self.layer_idx, mask)

    def __call__(self, x: Tensor, mask: Optional[Tensor] = None) ->
        Tensor:
        return self.forward(x, mask)
```

Outside `cache.generation()`, every call goes to the original attention layer, including a one-token call. Inside the scope, a one-token call uses `_cached_generation_step`; a longer sequence still uses the original layer and leaves the cache untouched. Sequence length alone cannot express intent. A one-token input might be an independent request or a training example, neither of which should read the previous request's history.

The scope changes one flag, `_generation_active`. Entering it saves the flag's previous value and sets it to true. The `finally` clause restores that value when control leaves, including when a forward pass raises. This is the same control-flow guarantee as closing a file when a `with` block ends. It does not reset the cursor, clear the buffers, or disable gradient recording; those are separate responsibilities. The stand-in also returns the original attention layer's parameters, so installing it does not create or replace weights.

19.3.3 Prefill and decode through the same step

Serving frameworks manage two fundamentally distinct runtime phases for every language model request: a prompt processing phase that populates state, followed by an iterative autoregressive decoding phase. Unifying these phases within a single execution abstraction ensures consistent buffer management while preventing cross-request cache contamination in multi-turn serving. A request arrives as a prompt of P tokens, and only then does the model produce one token at a time. Synthesis II named the two phases prefill and decode. The cache is what makes decode cheap, but prefill has to fill it first, and the request after this one must not see this one's tokens.

The tempting way to build that is two paths, the full-sequence attention of Chapter 12 for the prompt and the cached step for decode, with the keys and values handed from one to the other. Two paths means two places for the ordering bug to hide and a hand-off that has to agree about where the cursor is. The failure here is quiet. Forget to reset between requests and the new prompt's first token attends over the old prompt's keys, and no exception reports it; the model simply answers a question nobody asked.

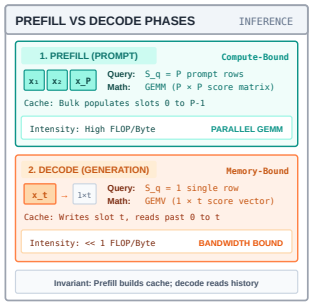
The module therefore runs both phases inside one generation scope, with one token per call, `advance` once per token, and `reset` once at the top. Attaching the cache is a loop over the blocks.

```python
def enable_kv_cache(model: Any) -> KVCache:
    """
    Enable KV caching for a transformer model without editing its layers.
    """
    # Step 1: Validate model and create cache
    cache, head_dim = _create_cache_storage(model)

    # Step 2: Put a stand-in in front of each block's attention layer
    for layer_idx, block in enumerate(model.blocks):
        if isinstance(block.attention, CachedAttention):  # enabled twice: start from the original
            block.attention = block.attention.attention
        block.attention = CachedAttention(block.attention, cache, layer_idx)

    # Step 3: Print confirmation
    print("⚡ KV Cache enabled for model!")
    print(f"   Architecture: {model.num_layers} layers × {model.num_heads} heads × {head_dim}D")
    print(f"   Memory: {cache.get_memory_usage()['total_mb']:.2f} MB")
    print("   Cache stored in: model._kv_cache")
    print()
    print("🔌 To disable: call disable_kv_cache(model)")
    print()

    return cache
```



Prefill vs. Decode Phases: Prefill processes prompt tokens in parallel (S_q = P) saturating compute via GEMM, whereas decode generates single tokens (S_q = 1) bound by memory bandwidth.

The helper it calls first, `_create_cache_storage`, checks that the model has the five attributes a GPT-style model exposes (`embed_dim`, `num_layers`, `num_heads`, `max_seq_len`, and `blocks`), sizes a `KVCache` from them, and stores it on the model as `_kv_cache`. The loop then puts a stand-in in front of every block's attention, unwrapping first if the cache

was enabled twice, so that each stand-in holds the original layer and not another stand-in. Attaching these wrappers makes cached execution available but does not activate it. `_cached_generate` enters the scope for its two phases; an ordinary `GPT.generate` remains the full-prefix baseline even while wrappers are installed.

```
def _cached_generate(model: Any, prompt_tokens: List[int],
    max_new_tokens: int,
                     temperature: float, cache: KVCache) -> List[int]:
    """
    Run autoregressive generation using the KV cache.
    """
    if not isinstance(max_new_tokens, (int, np.integer)) or
        max_new_tokens < 0:
        raise ValueError("max_new_tokens must be a nonnegative
            integer")
    if not np.isfinite(temperature) or temperature < 0:
        raise ValueError("temperature must be finite and nonnegative")
    if len(prompt_tokens) == 0:
        raise ValueError("prompt_tokens must contain at least one
            token")
    # The last sampled token is returned, not fed back into the model,
        so the run
    # needs len(prompt) + max_new_tokens - 1 slots, one fewer than it
        generates.
    if max_new_tokens > 0 and len(prompt_tokens) + max_new_tokens - 1
        > cache.max_seq_len:
        raise ValueError("Prompt and generation exceed cache
            capacity")
    generated = []
    cache.reset()    # a fresh sequence: the cursor back to slot 0, no
    rows from the last request

    if max_new_tokens == 0:
        return generated

    with cache.generation():
        # Phase 1: PREFILL - process prompt tokens one at a time to
            populate cache
        # Each token goes through the CachedAttention stand-ins, which
            write its
        # K,V into the cache. start_pos=cache.seq_pos gives the token
            its true
        # position, so a single-token forward computes exactly what
            the full
        # sequence would have computed for that position.
        for token in prompt_tokens:
            token_tensor = Tensor(np.array([[token]]))  # (1, 1)
            logits = model.forward(token_tensor,
    start_pos=cache.seq_pos)
            cache.advance()

        # Get logits for last prompt token (predicts next token)
```

```
        last_logits = logits.data[0, -1, :]  # (vocab_size,)

        # Phase 2: GENERATE - one token at a time using cache
        for step in range(max_new_tokens):
            # Zero temperature means greedy decoding, including tied logits.
            if temperature == 0:
                next_token = int(np.argmax(last_logits))
            else:
                scaled_logits = last_logits / temperature
                exp_logits = np.exp(scaled_logits - np.max(scaled_logits))
                probs = exp_logits / np.sum(exp_logits)
                next_token = int(rng.choice(len(probs), p=probs))
            generated.append(next_token)
            if step == max_new_tokens - 1:
                break

            # Feed single token through model (cache handles history)
            token_tensor = Tensor(np.array([[next_token]]))  # (1, 1)
            logits = model.forward(token_tensor, start_pos=cache.seq_pos)  # (1, 1, vocab_size)
            cache.advance()

            last_logits = logits.data[0, -1, :]

    return generated
```

The loop owns the request lifetime. `reset` first clears the previous request, and `with cache.generation():` activates cached routing until the loop exits. Two further details keep the positions consistent. Every call feeds exactly one token, with `start_pos=cache.seq_pos`, which is the argument Chapter 11 gave the position table and Chapter 13 threaded through `GPT.forward` so that a single token knows where in the sequence it sits; without it every cached step would be embedded at position zero. The caller invokes `advance` exactly once per token, after the whole stack of blocks has processed it, so every layer writes the same slot.

For a nonempty prompt, the final prefill logits predict the first generated token. The loop then samples that token and processes it to obtain the next logits. The final requested token is returned without another forward pass because its logits would be unused. For prompt length P and positive output length N, the cursor ends at $P+N-1$. The capacity check therefore requires only the $P + N - 1$ cached positions to fit in `max_seq_len`, one position looser than `GPT.generate`, which limits the returned length $P + N$. These are different counts: the last returned token occupies the output list but has no cached key or value. A zero-token request returns an empty list after resetting the cache. Prefilling one token at a time costs P sequential steps; the bridge shows what production does instead. After the scope exits, ordinary forwards stop reading the cache even though its buffers remain attached. Taking the wrappers off entirely is the loop in reverse.

```python
def disable_kv_cache(model: Any) -> None:
    """
    Disable KV caching and restore original attention behavior.
    """
    if not getattr(model, '_cache_enabled', False):
        print("⚠️  KV cache not enabled on this model")
        return

    # Take the stand-ins out; each one still holds the original layer
    for block in model.blocks:
        if isinstance(block.attention, CachedAttention):
            block.attention = block.attention.attention

    # Clean up
    model._cache_enabled = False
    if hasattr(model, '_kv_cache'):
        delattr(model, '_kv_cache')

    print("✓ KV cache disabled, original attention restored")
```

Each stand-in still holds the layer it replaced, so unwrapping is an assignment, and the model afterward is the model before, which the trace checks.

19.4 A Worked Trace

Take $d_k = 2$, one head, one layer, and weights chosen so every product can be read off by eye: $W_Q = W_K = I$ and $W_V = 2I$, with the output projection I and every bias zero, so each token's query and key are the token itself, its value is the token doubled, and the attention output passes through unchanged. The prompt is two tokens, $\mathbf{x}_1 = [1, 0]$ and $\mathbf{x}_2 = [0, 1]$, and a third input row is fixed at $\mathbf{x}_3 = [1, 1]$ to stand in for decode. These are attention-layer inputs, not token IDs sampled by a full language model. The scale is $1/\sqrt{2} \approx 0.7071$. Table 19.2 follows the new row from projection through the output at each position.

Table 19.2: **Three cached steps**: Each attention input is fixed so the arithmetic can be checked. Each step writes one slot and scores every slot up to and including it.

Step	Input $\mathbf{x}_t$	$\mathbf{k}_t, \mathbf{v}_t$ written	Slot	Rows scored	Weights	Output $\mathbf{o}_t$
Prefill, $t = 1$	$[1, 0]$	$[1, 0]$, $[2, 0]$	0	1	$[1.0]$	$[2.0, 0.0]$
Prefill, $t = 2$	$[0, 1]$	$[0, 1]$, $[0, 2]$	1	2	$[0.3302, 0.6698]$	$[0.6605, 1.3395]$
Decode, $t = 3$	$[1, 1]$	$[1, 1]$, $[2, 2]$	2	3	$[0.2483, 0.2483, 0.5035]$	$[1.5035, 1.5035]$

The decode step is worth doing by hand. After the write, the cache holds

$$K_{\le 3} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V_{\le 3} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 2 & 2 \end{bmatrix}.$$

The query $\mathbf{q}_3 = [1, 1]$ scores $[1, 1, 2]$ against the three keys, and dividing by $\sqrt{2}$ gives $[0.7071, 0.7071, 1.4142]$. Subtracting the largest scaled score before exponentiating gives $[e^{-0.7071}, e^{-0.7071}, 1] \approx [0.4931, 0.4931, 1]$. Their sum is about 1.9861, so the weights are $[0.2483, 0.2483, 0.5035]$. Mixing the values, $0.2483\,[2, 0] + 0.2483\,[0, 2] + 0.5035\,[2, 2] = [1.5035, 1.5035]$. The new token puts just over half its attention on itself because its key is the sum of the other two and scores highest against its own query.

Run the module on the same numbers and it reproduces the table line for line. In listing 19.2, lines 7 to 12 configure `MultiHeadAttention(embed_dim=2, num_heads=1)` with identity projections and zero biases. Line 13 instantiates a static `KVCache` buffer for maximum sequence length 8. Lines 15 to 16 define the three test tokens and wrap the layer in `cached_attention = CachedAttention(attn, cache, layer_idx=0)`. Inside `cache.generation()` (lines 18 to 24), lines 19 to 24 feed tokens one-by-one: `cached_attention` computes single-query attention against the accumulated prefix, prints the state matching table 19.2 (lines 25 to 27), and advances the write pointer (`cache.advance()`).

Lines 29 to 31 compare the concatenated step-by-step cached outputs against full parallel causal attention (`full = attn(Tensor([tokens]), create_causal_mask(3))`), verifying equivalence within 10^{-5} tolerance. Lines 33 to 34 confirm that outside the generation context, the layer defaults to uncached execution. Finally, lines 35 to 36 verify that `cache.reset()` clears `seq_pos` back to zero.

Listing 19.2 A scoped attention trace: Every cached row matches full causal attention.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.attention import MultiHeadAttention
from tinytorch.core.transformers import create_causal_mask
from tinytorch.perf.memoization import KVCache, CachedAttention

attn = MultiHeadAttention(embed_dim=2, num_heads=1)
I = np.eye(2, dtype=np.float32)
for name, w in (("q_proj", I), ("k_proj", I), ("v_proj", 2 * I),
    ("out_proj", I)):
    layer = getattr(attn, name)
    layer.weight.data[:] = w
    layer.bias.data[:] = 0
cache = KVCache(batch_size=1, max_seq_len=8, num_layers=1,
    num_heads=1, head_dim=2)

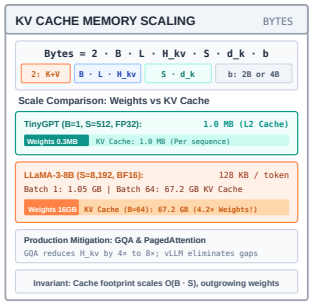
tokens = [[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]]
cached_attention = CachedAttention(attn, cache, layer_idx=0)
cached_rows = []
with cache.generation():
    for t, x in enumerate(tokens, start=1):
        out = cached_attention(Tensor([[x]]))
        history, _ = cache.get(0)
        cached_rows.append(out.data.copy())
        print(t, cache.seq_pos, history.shape[2], out.data[0,
            0].round(4))
        cache.advance()
# 1 0 0 [2. 0.]
# 2 1 1 [0.6605 1.3395]
# 3 2 2 [1.5035 1.5035]

full = attn(Tensor([tokens]), create_causal_mask(3))
np.testing.assert_allclose(
    np.concatenate(cached_rows, axis=1), full.data, rtol=1e-5,
    atol=1e-6)
# The completed cache does not affect an ordinary one-token call.
ordinary = cached_attention(Tensor([[tokens[0]]]))
np.testing.assert_allclose(ordinary.data,
    attn(Tensor([[tokens[0]]])).data)
cache.reset()
assert cache.seq_pos == 0 and cache.get(0)[0].shape[2] == 0
print(full.data[0, -1])            # [1.5035 1.5035]: same as the
    cached step
```

The history row counts are the cache's visible prefix. At step 3, `seq_pos=2` and `get` returns shape `(1, 1, 2, 2)`; concatenating the current key produces `(1, 1, 3, 2)`. The query has shape `(1, 1, 1, 2)`, so its score row has shape `(1, 1, 1, 3)`. After `advance`, the cursor is 3 and all three stored rows become history. Comparing every row provides the definitive correctness invariant. Feed all three tokens at once to the full layer

with the causal mask, and the final row of its 3×2 output is the same $[1.5035, 1.5035]$; every row of it, in fact, matches the corresponding cached step. The cached path and the full-sequence path compute the same function; if they ever disagree on the last row, the implementation must be checked for stale slots, a missed `advance`, incorrect positions, or a duplicated current row.

The wrapper lifetime can also be checked at model level. With prompt `[3, 7]` and two requested output tokens, prefill writes slots 0 and 1. Its last logits predict the first output token. Processing that token writes slot 2 and predicts the second, which is returned without another forward. The cursor is therefore 3 when the scope exits. The next ordinary forward must behave independently of those three cached positions, which listing 19.3 checks.



KV Cache Memory Scaling: Footprint scales as $2 \cdot B \cdot L \cdot H_{kv} \cdot S \cdot d_k \cdot b$ bytes. At long context and high concurrency, KV cache size outgrows model weights, motivating grouped-query attention and paged allocation.

Listing 19.3 Request lifetime: Repeated generation resets state while ordinary forwards remain independent.

```
from tinytorch.core.autograd import no_grad
from tinytorch.core.transformers import GPT
from tinytorch.perf.memoization import (
    enable_kv_cache, disable_kv_cache, _cached_generate)

model = GPT(vocab_size=12, embed_dim=16, num_layers=2,
            num_heads=2, max_seq_len=8)
prompt = [3, 7]
with no_grad():
    expected = model.generate(Tensor([prompt]), 2, temperature=0)
    ordinary = model(Tensor([[5]])).data.copy()
    cache = enable_kv_cache(model)
    try:
        generated = _cached_generate(model, prompt, 2, 0, cache)
        assert generated == expected.data[0, -2:].astype(int).tolist()
        assert cache.seq_pos == 3
        np.testing.assert_allclose(model(Tensor([[5]])).data,
    ↪   ordinary)
        repeated = _cached_generate(model, prompt, 2, 0, cache)
        assert repeated == generated and cache.seq_pos == 3
    finally:
        disable_kv_cache(model)
```

Zero temperature chooses the largest logit and makes the comparison independent of sampling randomness. Repeating the request checks that `reset` begins again at slot 0. The ordinary forward between requests checks a different boundary: leaving the scope restores ordinary routing without erasing the cache. In a manual replay, the caller must supply both `with cache.generation():` and `start_pos=cache.seq_pos`, then advance once after the complete model call. Resetting a cursor without entering the scope would compute each token independently and would fail the equivalence check.

Correctness and timing need separate evidence. The generation-speedup script in Synthesis III replays a fixed sequence through both paths, compares the logits at every position, then measures repeated replays after warmup. Holding the token sequence fixed avoids timing two different sampled workloads. No numerical speedup follows

from the three-row trace alone. Longer prefixes increase the work saved, but prefix copies, matrix multiplication overhead, and the model's remaining layers determine the elapsed-time ratio. Chapter 20 develops the measurement method.

19.5 From TinyTorch to Production

Production serving preserves the per-layer keys and values while changing their allocation and access. Hugging Face's cache strategy documentation distinguishes a growing `DynamicCache` from a preallocated `StaticCache`. Fixed capacity can support compilation with stable shapes, at the cost of reserving unused positions. TinyTorch preallocates its persistent buffers too, but constructs copies of the growing prefix for attention; its temporary array shapes still change. Fixed storage alone does not establish a fixed-shape compiled decode path.

A production implementation can also prefill several prompt positions together. It computes their projections as a matrix operation and applies a causal mask so that each prompt row sees only its predecessors and itself. Decode then uses one new query against the stored prefix. This separates the time needed to process the prompt from the time for each later output token. TinyTorch's token-by-token prefill exposes the same position invariant with one loop, at the cost of serial calls.

The memory budget follows directly from the reference layout. For L layers, H key/value heads, head width d_k, and b bytes per value, each cached position requires $2LHd_kb$ bytes per sequence. Grouped-query attention lets multiple query heads share a key/value head, reducing H in this expression.[73] Quantized caches reduce b by approximating stored values. A sliding window instead limits how many positions remain visible. TinyTorch implements none of these storage changes; its cache retains float32 rows up to its fixed capacity and raises an error when full.

A different allocation strategy can preserve all the rows while reserving less unused space. PagedAttention, introduced by Kwon and colleagues, divides a request's keys and values into blocks that need not be contiguous in physical memory.[74] A block table maps a logical position to its allocated block. New blocks arrive as the sequence grows, and completed requests release blocks for reuse. This changes where attention finds the keys, rather than which keys the query can see, as analyzed in the production bridge below. Prefix sharing can additionally reuse blocks containing identical earlier context.

In naive serving engines, allocating contiguous memory buffers for `max_seq_len` up front wastes up to 60 to 80% of high-bandwidth GPU memory due to three forms of fragmentation:

- **Internal Fragmentation**: Unused reserved slots when requests finish before reaching `max_seq_len`.
- **External Fragmentation**: Memory gaps between requests of varying length.
- **Reservation Fragmentation**: Reserving peak sequence capacity before tokens are generated.

The **PagedAttention** algorithm (Kwon et al., 2023, powering **vLLM**) adopts operating system virtual memory principles:[75]

- **Non-Contiguous KV Blocks**: Keys and values are partitioned into fixed-size physical blocks (typically 16 or 32 tokens).

[73] **Grouped-Query Attention (GQA) and Multi-Query Attention (MQA)**: Standard Multi-Head Attention (MHA) maintains an equal number of query, key, and value heads ($H_q = H_{kv}$). Multi-Query Attention (MQA, Shazeer, 2019) collapses key and value projections into a single head ($H_{kv} = 1$) shared across all query heads, slashing KV cache memory by $H_q\times$. Grouped-Query Attention (GQA, Ainslie et al., 2023, adopted by LLaMA-2/3 and Mistral) strikes a balance by grouping query heads into G groups (e.g., 32 query heads divided into 8 KV heads, $H_{kv} = 8$), reducing KV cache memory traffic by $4\times$ with minimal degradation in model reasoning capability.

[74] **Virtual memory**: the operating system mechanism that lets a program address memory as if it were one contiguous range while the physical pages behind it are scattered wherever the allocator found room, with a page table translating one to the other. PagedAttention applies the same indirection to KV rows so that a 4,000-token sequence can occupy 250 scattered 16-token blocks instead of one reserved 4,096-row buffer.

[75] **Continuous batching and iteration-level scheduling**: Traditional inference engines employ static batching, locking a batch of requests together until the slowest request finishes, which leads to GPU compute starvation due to request length variance. Orca (Yu et al., 2022) and vLLM introduce continuous batching (iteration-level scheduling). Because the KV cache maintains independent physical page chains for each sequence, completed sequences can be evicted immediately after any decode step, allowing newly arrived prompts to enter the running batch at the next iteration without stalling active generations.

- **Block Tables**: A dynamic page table maps logical token positions $(0, \ldots, t)$ to non-contiguous physical GPU memory pages.
- **Zero-Waste Allocation**: Memory blocks are allocated on demand as tokens are generated, reducing memory waste to under 4%.
- **Copy-on-Write Prefix Sharing**: Parallel beams in beam search or shared system prompts reference the same physical KV blocks, enabling instantaneous prefix reuse.

Table 19.3 separates these alternatives from the reference. Paging can preserve the full-context computation, subject to the same floating-point tolerance. Quantization introduces approximation, and a sliding window changes the attended context; each requires its own quality check. Optimized decode kernels like FlashDecoding parallelize along the cached sequence dimension to saturate memory bandwidth.[76] Increased latency at long context is a reason to measure growing cache reads, not proof that they are the only bottleneck. The invariant remains that every required position occurs once, belongs to this request, and carries the same key and value that an uncached causal forward would compute.

Table 19.3: **Serving alternatives**: What changes between TinyTorch's cache and a production serving engine. Storage-preserving alternatives retain the recurrence; approximation or eviction needs additional checks.

	This chapter	Production serving
Cache layout	$(1, H, S_{\text{max}}, d_k)$ float32 tensors per layer	$(B, H, S_{\text{max}}, d_k)$ tensors per layer, or blocks addressed through a table
Prefill	One token at a time through the cached step	Prompt positions processed together with causal attention
Memory per token	$2LHd_k$ float32 values	$2LH_{kv}d_k$ values; $H_{kv} < H$ under grouped-query attention, possibly quantized
Capacity	Fixed `max_seq_len`, error when full	Paged on demand, shared prefixes, sliding windows
Decode kernel	One NumPy matmul of the query row against the cached keys, per head	Optimized kernels read contiguous or paged keys and values

[76] **FlashDecoding and sequence-split parallelism**: While standard FlashAttention parallelizes along batch and query sequence dimensions, autoregressive decode features a single query token ($S_q = 1$) attending to thousands of cached keys ($S_{\text{kv}} \gg 1$), underutilizing GPU streaming multiprocessors. FlashDecoding (Dao et al., 2023) resolves this bottleneck by splitting the cached KV sequence length across independent GPU thread blocks. Each block computes local softmax running maxima and partial attention outputs in SRAM; a lightweight secondary reduction kernel combines these partial states via logsumexp rescaling, achieving near-optimal memory bandwidth saturation during decode.

19.6 Summary

A cached step projects one new token, writes its key and value at `seq_pos`, joins the copied history with the current row, and computes one attention output. Every layer uses the same slot before the model-level caller advances once. The causal dependency of earlier rows makes their keys and values reusable, while the growing score row keeps attention work proportional to context length.

The lifetime has two boundaries. `reset` starts a new sequence; `cache.generation()` selects cached execution and restores ordinary forwards when its block exits. `_cached_generate` owns both boundaries, including the position offset and the decision not to forward the last sampled token. The trace verifies all cached outputs against the

full causal computation, then checks that a later ordinary request sees no stale history. Chapter 20 takes this correctness check as the starting point for measuring elapsed time with warmups, repeated samples, and reported variation.

19.7 Further Reading

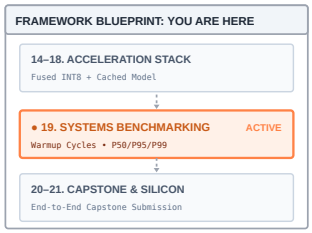
- **Pope et al.** (2023). "Efficiently scaling transformer inference." *MLSys*. Prefill, decode, and the memory the key-value cache occupies at scale.
- **Kwon et al.** (2023). "Efficient memory management for large language model serving with PagedAttention." *SOSP*. The paged cache in the production comparison above.

🔥 Chapter 20

Benchmarking: Measuring Reliable Speedups

Performance claims in machine learning systems, such as *"this optimization provides a 3× speedup"*, cannot be checked without a stated, reproducible benchmarking method. Naive timing using basic wall-clock timers produces misleading results contaminated by Python interpreter overhead, cache cold starts, asynchronous execution queues, and operating system noise.

In production engineering, a reliable benchmark protocol requires explicit synchronization and statistical aggregation:



Framework Blueprint: Module 19 establishes experimental rigor: discarding cold-start caches, measuring percentiles, and auditing throughput.

```python
import torch
import time

x = torch.randn(2048, 2048)

# 1. Warm-up iterations: prime caches and trigger JIT/library compilation
for _ in range(5):
    _ = x @ x

# 2. Synchronized monotonic measurement
times = []
for _ in range(20):
    t0 = time.perf_counter()
    _ = x @ x
    times.append((time.perf_counter() - t0) * 1000)  # Convert to ms

print(f"Median: {torch.tensor(times).median():.2f} ms")
```

Underneath that careful loop, a valid systems benchmark enforces several physical and statistical controls:

1. **Warmup isolation:** Untimed initial iterations prime instruction and data caches, trigger one-time allocator pools, and allow dynamically loaded BLAS routines to configure threads, preventing cold-start outliers from skewing steady-state performance.
2. **High-resolution monotonic timing:** Using hardware monotonic counters (`time.perf_counter()`) avoids the clock slews and negative durations that plague calendar wall-clock timers (`time.time()`).

3. **Distributional reporting over single-number averages:** Measuring variance across multiple runs reveals tail latency spikes (P90, P99) caused by operating system context switching, memory garbage collection, and thermal throttling.

This chapter builds TinyTorch's benchmarking suite, establishing the rigorous empirical measurement standards required for the Capstone evaluation in Chapter 21. Before building the benchmarking harness, inspecting common timing pitfalls reveals how naive timing misleads.

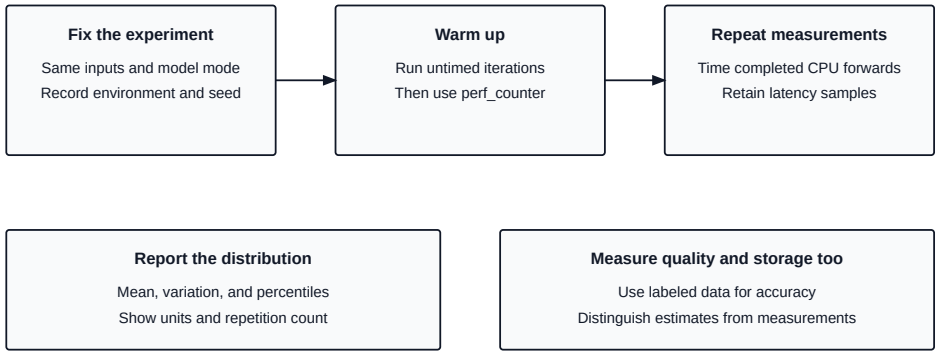


Figure 20.1: Fix the workload and model mode, warm up, and retain repeated timings. Report the distribution alongside quality checks, keeping storage estimates distinct from measured allocations.

20.1 The Problem

↩ **Jump back:** Chapter 8 *Training loop timing: early epoch steps include allocator initialization and worker process startup.*
↩ **Jump back:** Chapter 19 *Benchmarking cached vs. uncached autoregressive generation under strict warmup controls.*

The script below is how most people time a model the first time, and it is wrong in three separate ways. The model is a stand-in that sleeps instead of multiplying, so the intended delays are explicit and can execute without any other TinyTorch class.

```
import time

class ToyModel:
    """Stands in for a real model: slow once, then fast, with a periodic stall."""
    def __init__(self):
        self.calls = 0

    def forward(self, x):
        self.calls += 1
        if self.calls == 1:
            time.sleep(0.040)       # one-time setup: allocate, load, compile
        if self.calls % 25 == 0:
            time.sleep(0.010)       # something else grabbed the CPU
        time.sleep(0.005)           # the steady-state work
```

```
        return x

model = ToyModel()
t0 = time.perf_counter()
model.forward(None)
t1 = time.perf_counter()
print(f"latency: {(t1 - t0) * 1000:.1f} ms")
```

The first call requests 40 ms of setup sleep plus 5 ms of steady work, totaling 45 ms before timer and scheduling overhead. That is the first flaw. A real model may also pay first-call costs, such as initializing a numerical library or bringing data into a processor cache. In the toy, comparing the 45 ms cold call with a 5 ms warm call creates an apparent 9x speedup without changing the model at all.

The second flaw is that one measurement is one sample. Even after warmup, the same forward pass does not take the same time twice. The operating system schedules other work, the caches hold different contents, the clock frequency drifts. In any 100 consecutive calls after the first, the toy requests 5 ms of sleep on 96 calls and 15 ms on four calls; observed durations also include scheduling delays. A single timing that happens to catch a stall reports a model three times slower than it is; a single timing that misses every stall reports a model that never stalls.

The third flaw appears once measurements are repeated and averaged. A simple calculation makes this concrete. If 99 requests take 5.0 ms and one stalls for 500 ms, the mean is 9.95 ms, a number no single request ever experienced. The mean is pulled up by the single outlier and reveals neither what a typical request costs nor how severe the worst stalls are. Latency is not symmetric noise around a center; it has a floor and a long right tail, and the arithmetic mean hides both.

↪ **Jump ahead:** Chapter 21 *The multi-axis scorecard: evaluating accuracy retention, compression ratio, and runtime speedup.*

The timer here surrounds completed NumPy work on the CPU. A device that accepts work into a queue needs a different timing boundary: the host call may finish before the computation does. The production bridge returns to that distinction after the reference loop is clear.

20.2 The Idea

Treating a latency measurement as a sample from a probability distribution requires designing the benchmark so that the sampled distribution matches the operational target. That single reframing produces every rule in the chapter.

20.2.1 Warmup

Standard machine learning benchmarking suites, including MLPerf Inference and PyTorch's benchmark utilities, mandate dedicated warmup iterations before recording any performance metrics. Untimed warm iterations prime CPU instruction caches, trigger memory pool allocations, and settle processor dynamic frequency scaling (DVFS) so measurements reflect steady-state execution rather than one-time runtime overheads. For a sustained inference comparison, the distribution of interest is steady state, the cost once the model is loaded and running. An application concerned with startup must measure cold latency separately. Cold runs come from a different distribution, one that includes setup, and mixing the two corrupts every summary statistic. The fix is to run the model

several times and discard those initial timings, so that the retained list is drawn purely from the steady-state distribution.[77]

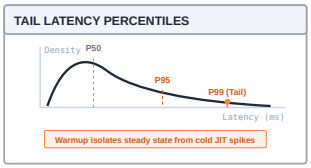
[77] **DVFS and thermal settling**: Modern processors use Dynamic Voltage and Frequency Scaling (DVFS) to balance power and thermals. Under standard operating system balanced governors, processors boot from low idle frequencies (such as 800 MHz) and take tens of milliseconds to ramp to peak turbo clocks. Conversely, sustained execution heats the die and triggers thermal throttling that lowers clocks by 15% to 30%. Rigorous systems benchmarks lock hardware governors to performance mode and run continuous thermal-soak iterations prior to recording measurements.

20.2.2 Repetition and the shape of the distribution

In production serving environments governed by strict Service Level Objectives (SLOs), performance is characterized by complete latency distributions rather than isolated average timings. Systems noise from operating system scheduling, memory allocator stalls, and shared bus contention produces a heavy right-skewed tail that single-number arithmetic means obscure. After warmup, run the model N times and keep every timing $t_1, \ldots, t_N$. Two summaries are familiar. The **mean** $\bar{t}$ is the sum divided by N, and the **standard deviation** s measures how far a typical timing sits from the mean:

$$\bar{t} = \frac{1}{N}\sum_{i=1}^{N} t_i, \qquad s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(t_i - \bar{t})^2}$$

The mean and standard deviation remain valid summaries of asymmetric data, but neither reveals directly where the slowest requests lie. Latency often has that asymmetry. Nothing can run faster than the floor, and occasional stalls push a few timings far above it, so the histogram has a sharp left edge and a long right tail. For such a distribution the useful summaries come from sorting.



Latency distribution anatomy: tail percentiles P_{95} and P_{99} capture system stalls that the mean obscures.

A **percentile** is a position in the sorted list. Sort the N timings in ascending order, and the p-th percentile P_p is the value at a fractional index p percent of the way through that sorted list. Three percentiles carry most of the meaning:

- P_{50}, the median, is what a typical request costs. Half the runs were faster, half slower. Moving a few already-large observations farther upward leaves the median unchanged, making it useful for typical request cost.
- P_{95} and P_{99} are the **tail latency**, the cost seen by the slowest 5 and 1 percent of requests. A promise that 99 percent of requests finish within 20 ms is a bound on P_{99}. It concerns a fraction of requests, not a fixed group of users.[78]

[78] **SLA** (service-level agreement): a written promise about a service's behavior, such as "99 percent of requests complete within 20 ms." A benchmark that reports only the mean cannot determine whether that percentile promise is met.

When p percent of the way through the list falls between two entries, TinyTorch interpolates linearly between them, the same convention NumPy uses by default. With seven sorted timings, P_{50} sits at position $0.5 \times 6 = 3$, exactly the fourth entry, while P_{95} sits at position 5.7, seven tenths of the way from the sixth entry to the seventh. The worked trace does this arithmetic by hand.

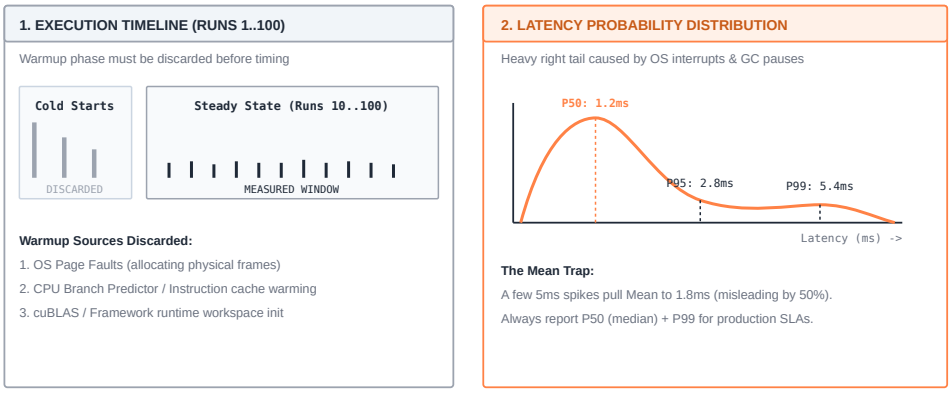


Figure 20.2: **The anatomy of latency distributions and tail percentiles**: Discarding initial cold-start runs (affected by OS page faults, library dynamic linking, and cache warming) isolates steady-state performance. The resulting latency distribution is characteristically right-skewed; reporting median P_{50} alongside tail percentiles P_{95} and P_{99} reveals unpredictable system stalls that mean latency alone conceals.

20.2.3 How many runs

Rigorous benchmarking standards, such as those established by MLPerf, enforce minimum trial counts to ensure statistical confidence when characterizing high percentiles. Estimating tail metrics like P_{99} or $P_{99.9}$ from too few iterations yields erratic results that fail to capture rare hardware and operating system hiccups. A percentile is only as trustworthy as the number of samples behind it. P_{99} describes an event that happens once per hundred runs, so twenty runs can easily miss that upper tail entirely. An interpolated percentile still exists, but provides little evidence about a rare event. The Napkin Math below turns that intuition into a concrete sample count.

A run exceeds the true 99th percentile with probability $p = 0.01$. The chance that N runs contain no such event is $(1-p)^N$, so the chance of seeing at least one is $1-(1-p)^N$. Asking for a 95 percent chance of catching at least one tail event gives

$$1 - 0.99^N \geq 0.95 \quad \Longrightarrow \quad N \geq \frac{\ln 0.05}{\ln 0.99} \approx \frac{-2.996}{-0.01005} \approx 298.07.$$

Round up to **299 runs** to meet the inequality. This calculation concerns observing at least one tail event, assuming independent samples. It does not establish a precise percentile estimate: one event gives little evidence about the tail's shape. `MLPerf.run_standard_benchmark` defaults to 100 runs, while `Benchmark` defaults to 10 measured runs. Both are classroom starting points; repeat the experiment and collect more samples before interpreting its P_{99} as a service guarantee.[79]

20.2.4 Throughput

While interactive consumer applications demand low latency, production serving infrastructure and training pipelines optimize primarily for system throughput to maximize hardware utilization and operational economics. Benchmarking harnesses measure this dimension to understand how batching saturates memory bandwidth and matrix cores under sustained load. Latency is the cost of one request. **Throughput** is how many

[79] **Python garbage collection pauses and allocator churn**: Python's automatic garbage collector tracks cyclic object references. When generational garbage collection triggers during a timed iteration, it pauses execution to traverse heap object references, introducing multi-millisecond tail latency spikes. High-precision benchmarking harnesses explicitly invoke `gc.collect()` before timing and call `gc.disable()` during active timing loops, re-enabling collection only between iterations to prevent allocator maintenance from contaminating P_{95} and P_{99} latency metrics.

samples the model finishes per second, which is what matters when requests arrive in batches. Over the measured window it is total samples over total time, with B the batch size and the timings in milliseconds:

$$\text{Throughput} = \frac{N \cdot B}{\sum_{i=1}^{N} t_i} \times 1000 \quad \text{samples per second}$$

A batch of 32 that takes 40 ms has a worse latency than a batch of 1 that takes 5 ms and four times the throughput: 800 rather than 200 samples per second. The two numbers answer different questions. The module's MLPerf-style path assumes one example per input and reports `1000 / mean_latency_ms`; for a batched harness experiment, compute samples per second using the batch size explicitly.

The formula is a special case of **Little's law**, which relates three averages for any system in steady state: the number of items in the system L equals their arrival rate λ times the time each spends inside, W (Little, 1961). Rearranged, throughput is $\lambda = L/W$. A harness that runs one batch at a time holds $L = B$ samples and keeps each for one batch latency, so its throughput is $B/\bar{t}$, the formula above. The law needs no assumption about how requests arrive, which makes it a check on any reported pair of numbers. A server that claims 800 samples per second at 40 ms per request must be holding about 32 requests at once. Raising throughput at a fixed latency therefore means holding more work concurrently, through larger batches or more parallel streams, and when latency grows with batch size, as it does here, the gain is less than the batch size suggests.

20.2.5 Fairness

In systems evaluation, valid comparative claims—such as benchmarking an optimized kernel against an unfused baseline—require strict isolation of experimental variables. Confounding factors like disparate batch dimensions, differing numerical precisions, or mismatched hardware frequency states invalidate empirical speedup ratios. One more rule follows from the same model. A comparison is fair only when both sides are measured under the same conditions, which means the same input, the same machine, the same warmup and run counts, and the same statistic on each side. Comparing a median against a mean, or a warm run against a cold one, yields a ratio that reflects the measurement flaw rather than true model performance.

Pause & Predict: The Mean vs. Median Latency Trap

Suppose you record 20 benchmark runs of a model forward pass. Exactly 19 iterations finish in 1.0 ms each, but iteration 14 experiences an operating system context switch and takes 21.0 ms. What is the mean latency across the 20 runs? What is the median (P_{50})? If a client requires an SLA of $P_{99} \leq 5.0$ ms, does this service pass or fail?

20.3 The Code

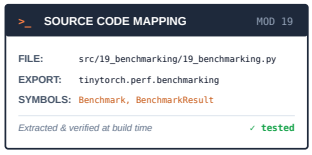
Read the core path first: `Benchmark.run_latency_benchmark` produces a list of durations, and `BenchmarkResult` summarizes that list. Compare one unchanged baseline with one candidate before introducing multiple candidates or a statistical test. The mod-

ule's suite combines several metrics and plots comparisons; it cannot repair mismatched workloads or establish statistical significance from a plot.

The implementation consists of four modular components. A percentile function handles sorting and linear interpolation. `BenchmarkResult` holds every timing from one run and computes statistical summaries. `Benchmark` executes the warmup-then-measure loop. Finally, the `MLPerf` class evaluates result latencies alongside task accuracy to render a compliance verdict. The latency harness constructs a TinyTorch `Tensor` and passes it to the model's `forward` method. The MLPerf-style loop also accepts `predict` or an arbitrary callable, but every route must return predictions that the accuracy phase can interpret.

20.3.1 Percentiles by hand

The median and tail comparisons in this chapter are positions in a sorted list. A bound on individual decode-step latency needs a percentile of those steps; a bound on the whole request instead needs whole-request timings. A report that the KV cache made the typical step faster needs P_{50} before and after. So the first thing to understand is the function that reads those positions.



Source Code Mapping: Module 19 extracts benchmark timing harnesses, latency percentile metrics, and the classroom MLPerf-style check directly from the working repository.

The naive way is to sort the timings and index with the truncated position, `s[int(0.95 * n)]`. On the seven timings of the worked trace that returns 18.40 ms for P_{95}, the stall itself, where NumPy reports 14.44 ms. Neither number is wrong in the abstract, but different conventions can produce substantially different values on a small sample. A comparison must name and use one convention consistently. NumPy treats the position as fractional and interpolates linearly between the two entries it falls between. The module's `BenchmarkResult.percentile` calls `np.percentile` for exactly that reason; here is the same rule written out, as book code, so that the arithmetic in the trace has something to point at.

```
def percentile(values, p):
    """The value p percent of the way through the sorted list (NumPy's
    ↪   linear rule). Book code."""
    s = sorted(values)
    pos = (p / 100) * (len(s) - 1)    # fractional index into the
  ↪   sorted list
    lo = int(pos)                     # the entry at or below that
  ↪   position
    hi = min(lo + 1, len(s) - 1)      # the entry above it (clamped at
  ↪   the end)
    return s[lo] + (s[hi] - s[lo]) * (pos - lo)
```

Notice that `pos` runs from 0 to `len(s) - 1`, so P_0 is the minimum and P_{100} is the maximum, and that when `pos` lands on a whole number the interpolation term vanishes and the function returns a timing that actually happened. This is the linear interpolation rule used by the module. The book helper assumes a nonempty list and $0 \leq p \leq 100$.

20.3.2 The result container

Chapter 21 measures a baseline and an optimized model and asks how the median moved and how the tail moved. Both answers come from the same list of timings, and so does the verdict at the end of this chapter. A report that someone else will read has to be able to show that list.

Chapter 15's `Profiler.measure_latency` returns the median of its internal timings. That compact result answers a typical-cost question, but it cannot recover a tail percentile afterward. Module 19 therefore calls the profiler with one measured iteration at a time and retains those individual results. Seven durations, including a stall, remain seven observations rather than one median. `BenchmarkResult` keeps that raw list and computes summaries from it.

```
@dataclass
class BenchmarkResult:
    """
    Container for benchmark measurements with statistical analysis.
    """
    metric_name: str
    values: List[float]
    metadata: Dict[str, Any] = field(default_factory=dict)

    def __post_init__(self):
        """Compute statistics after initialization."""
        if not self.values:
            raise ValueError(
                f"Empty values list for BenchmarkResult\n"
                f"  ✗ Cannot compute statistics: values=[] (0 measurements)\n"
                f"  🔔 BenchmarkResult needs data to compute mean, std, percentiles\n"
                f"  🔧 Add measurements: BenchmarkResult('{self.metric_name}', [1.2, 1.3, 1.1])"
            )

        self.mean = statistics.mean(self.values)
        self.std = statistics.stdev(self.values) if len(self.values) > 1 else 0.0
        self.median = statistics.median(self.values)
        self.min_val = min(self.values)
        self.max_val = max(self.values)
        self.count = len(self.values)

        # 95% Student-t confidence interval for the mean. The critical value
        # depends on n through the degrees of freedom, so it cannot be the
        # constant 1.96: at n = 10 that constant reports an interval 13% too
        # narrow. One sample has no degrees of freedom and therefore no
        # interval at all, so the bounds are None rather than the mean itself.
        # A zero-width 95% interval printed beside a result is a false claim.
        if self.count > 1:
```

```
            margin_error = t_critical_95(self.count - 1) * (self.std /
    ↪ np.sqrt(self.count))
            self.ci_lower = self.mean - margin_error
            self.ci_upper = self.mean + margin_error
        else:
            self.ci_lower = self.ci_upper = None

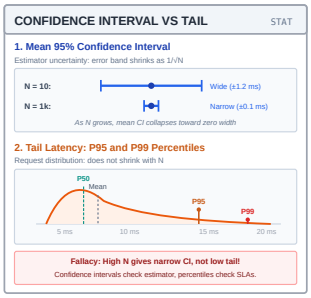
    def percentile(self, p: float) -> float:
        """The value p percent of the way through the sorted
        ↪ measurements (NumPy's linear rule)."""
        return float(np.percentile(self.values, p))

    def to_dict(self) -> Dict[str, Any]:
        """Convert to dictionary for serialization."""
        ...

    def __str__(self) -> str:
        return f"{self.metric_name}: {self.mean:.4f} ± {self.std:.4f}
        ↪ (n={self.count})"
```

`BenchmarkResult(name, values)` invokes `__post_init__`. That method rejects an empty list and stores the count, mean, sample standard deviation, median, and extrema. `percentile(p)` computes its answer on demand. Treat `values` as a finished record: the constructor retains the supplied list rather than copying it. Appending later would leave the stored mean and count unchanged while changing the next percentile result.

The confidence interval uses $t_{0.975,\,N-1}\,s/\sqrt{N}$ on either side of the mean, a Student-t critical value rather than the normal 1.96 (at $N = 10$, 2.262 against 1.96). It describes uncertainty in the estimated mean; it is not a range containing 95 percent of requests. With a strongly skewed sample it can still be poorly calibrated, and with a single sample there is no interval at all: `ci_lower` and `ci_upper` are `None`. The raw list is what permits a reader to assess these limitations or compute another summary.



Estimator uncertainty vs. tail latency: increasing sample size N narrows the confidence interval of the mean ($1/\sqrt{N}$), but does not reduce the tail latency experienced by slow requests.

20.3.3 The harness

The harness supplies the repeated observations needed to compare models. For a Chapter 19 decoder, its usefulness depends on the caller holding the prefix and generation lengths fixed. It automates repetition; the caller still decides which workload to repeat and how many observations the claim needs.

The cold timing in the opening included 40 ms of setup. `Benchmark` separates setup from the measurements it returns. Its constructor defaults to five warmup runs and ten measured runs; the example below explicitly requests ten and one hundred. No fixed warmup count guarantees stabilization, and ten timings provide little evidence about rare stalls. The constructor records those counts, the model references, and basic platform metadata.

```
class Benchmark:
    """
    Professional benchmarking system for ML models and operations.
    """

```

```
    def __init__(self, models: List[Any], datasets: List[Any],
                 warmup_runs: int = DEFAULT_WARMUP_RUNS,
    measurement_runs: int = DEFAULT_MEASUREMENT_RUNS):
        """
        Initialize benchmark with models and datasets.
        """
        if warmup_runs < 0 or measurement_runs <= 0:
            raise ValueError("warmup_runs must be nonnegative and
            ↪   measurement_runs positive")
        self.models = list(models)
        # Keep display names when possible, but never let copied names
        ↪   overwrite results.
        self.model_names = []
        for i, model in enumerate(self.models):
            stem = str(getattr(model, 'name', None) or f'model_{i}')
            name = stem
            suffix = 1
            while name in self.model_names:
                name = f'{stem}_{suffix}'
                suffix += 1
            self.model_names.append(name)
        self.datasets = datasets
        self.warmup_runs = warmup_runs
        self.measurement_runs = measurement_runs
        self.results = {}

        # Use Profiler from Module 14 for measurements
        self.profiler = Profiler()

        # System information for metadata (using Python standard
        ↪   library)
        self.system_info = {
            'platform': platform.platform(),
            'processor': platform.processor(),
            'python_version': platform.python_version(),
            'cpu_count': os.cpu_count() or 1,  # os.cpu_count() can
            ↪   return None
        }
```

Task accuracy requires a model-provided `evaluate(dataset)` result. Synthetic accuracy probes are available only with explicit `simulate=True`, are labeled as simulated, and cannot support a deployment recommendation. A failed probe raises instead of receiving a plausible default score. The memory benchmark reports the profiler's traced allocation peak even for small runs; it does not substitute a parameter-byte estimate for that measurement. Neither quantity is total process memory or memory traffic.

The suite assigns each model a unique result key once. Unnamed models become `model_0`, `model_1`, and so forth; colliding names receive suffixes. Every metric reuses those keys, so a copied name cannot overwrite another model's measurement. `BenchmarkSuite.run_full_benchmark(input_shape=...)` and `analyze_optimization_techniques(..., input_shape=...)` pass the same requested shape to latency and memory measurement. For `Linear(10, 2)`, a representa-

tive batch might have shape `(2, 10)`; the default `(1, 28, 28)` is not inferred from the dataset and will not fit that layer.

The `system_info` is the beginning of the reproducibility habit the bridge insists on; it travels with every result as metadata. The measuring loop reuses Chapter 15's timer for the warmup and then times every run on its own.

```
def benchmark_run_latency_benchmark(self, input_shape: Tuple[int, ...] = (1, 28, 28)) -> Dict[str, BenchmarkResult]:
    """
    Benchmark model inference latency using Profiler.
    """
    results = {}

    for i, model in enumerate(self.models):
        model_name = self.model_names[i]

        # Create input tensor for profiling
        input_tensor = Tensor(rng.standard_normal(input_shape).astype(np.float32))

        # Warm up through the Profiler (that one timing is discarded), then
        # record every measured run on its own so BenchmarkResult sees the tail
        self.profiler.measure_latency(model, input_tensor, warmup=self.warmup_runs, iterations=1)
        latencies = []
        for _ in range(self.measurement_runs):
            latencies.append(self.profiler.measure_latency(model, input_tensor, warmup=0, iterations=1))

        results[model_name] = BenchmarkResult(
            f"{model_name}_latency_ms",
            latencies,
            metadata={'input_shape': input_shape, **self.system_info}
        )

    return results
```

Follow one model through the loop. It creates one random input tensor outside the timed region, then calls the profiler with the requested warmup count and one additional measured iteration. That duration is discarded too. Each following profiler call uses zero warmup and one iteration, so the returned list contains exactly `measurement_runs` durations in milliseconds. With five warmups and ten retained measurements, the model receives sixteen forwards: five warmups, one discarded timing, and ten retained timings.

The input is reused within that model's measurements, but a new random tensor is drawn for the next model. Equal shapes therefore do not guarantee identical values across candidates. The harness also retains the model objects: a forward that updates a counter or cache keeps that state into the next call. It neither resets a KV cache nor selects evaluation mode for the caller. Set the desired mode and define the state boundary first. Repeated cached decoding must reproduce the same prefix and generation lengths, using

Chapter 19's cache lifetime rules; otherwise later timings measure progressively longer requests.

The toy makes that state visible through `calls`. The following snippet inspects actual durations under this protocol:

```
from tinytorch.perf.benchmarking import Benchmark

toy = ToyModel()
toy.name = "toy"                                    # the harness labels results by this
bench = Benchmark(models=[toy], datasets=[], warmup_runs=10, measurement_runs=100)
result = bench.run_latency_benchmark(input_shape=(1, 4))["toy"]
print(result)
print(f"p50 {result.percentile(50):.2f}  p95 {result.percentile(95):.2f}  p99 {result.percentile(99):.2f}  max {result.max_val:.2f}")
```

The toy receives 111 calls: ten warmups, one discarded timing, and one hundred retained measurements. Calls 25, 50, 75, and 100 stall, so four retained observations include an extra 10 ms. If sleep durations exactly matched their requests, the measured list would contain 96 values of 5 ms and four of 15 ms: mean 5.4 ms, median 5 ms, $P_{95} = 5$ ms, and $P_{99} = 15$ ms. These are predictions from the toy's schedule, not claimed measurements of a laptop. Real timings include overhead and additional delays. Compare the observed list with that pattern before interpreting a changed percentile.

20.3.4 The verdict

A benchmark ends in a decision, and Chapter 21 needs two kinds. The first is absolute. A model going into a service is asked to meet a latency bound, and the bound is about the slow requests, so it is a bound on a tail percentile such as P_{99}. The module's `MLPerf` check below gates on P_{90} instead, because it times one example per call and MLPerf Inference's single-stream scenario reports 90th-percentile latency, while its server scenario uses the 99th. The book helper `check_compliance` further down keeps P_{99} as the stricter service bound. The second is relative. Quantization in Chapter 16, pruning in Chapter 17, and fusion in Chapter 18 each change the numbers a model produces, some by rounding and some by removing weights outright, and a speedup is worth reporting only if the model still works afterward.

The naive verdict compares means and never asks about accuracy. It passes a model that averages 5 ms and stalls to 30 ms on every hundredth request, which is the request the user remembers. And it declares an INT8 model twice as fast without asking whether the rounding cost it accuracy, which would make it a different model rather than a faster one. The fix is two bars that both have to clear, latency and accuracy, and a fixed protocol around them so that two submissions can be compared. The module's classroom `MLPerf` class holds four task configurations inspired by MLPerf Tiny, each with an input shape, an accuracy target, and a latency bound. Passing this local check is not an official MLPerf result. Its timing loop uses a context manager that reads the clock on the way in and the way out.

```
@contextmanager
def precise_timer():
    """
    High-precision timing context manager for benchmarking.
    """
    class Timer:
        def __init__(self):
            self.elapsed = 0.0
            self.start_time = None

    timer = Timer()
    timer.start_time = time.perf_counter()

    try:
        yield timer
    finally:
        timer.elapsed = time.perf_counter() - timer.start_time
```

```
def _mlperf_run_latency_test(self, model: Any, test_inputs: List[Any],
                                    benchmark_name: str, num_runs: int)
    -> Tuple[List[float], List[Any]]:
    """
    Run latency measurement phase with warmup.
    """
    # Warmup phase (10% of runs)
    warmup_runs = max(1, num_runs // 10)
    print(f"   Warming up ({warmup_runs} runs)...")
    for i in range(warmup_runs):
        if hasattr(model, 'forward'):
            model.forward(test_inputs[i])
        elif hasattr(model, 'predict'):
            model.predict(test_inputs[i])
        elif callable(model):
            model(test_inputs[i])

    # Measurement phase
    print(f"   Measuring performance ({num_runs} runs)...")
    latencies = []
    predictions = []

    for i, test_input in enumerate(test_inputs):
        with precise_timer() as timer:
            try:
                if hasattr(model, 'forward'):
                    output = model.forward(test_input)
                elif hasattr(model, 'predict'):
                    output = model.predict(test_input)
                elif callable(model):
                    output = model(test_input)
                else:
```

```
                    raise TypeError(f"{type(model).__name__} has no
                    ↪   forward(), predict(), or __call__ to
                    ↪   benchmark")
            except Exception as exc:
                raise RuntimeError(f"{benchmark_name}: model failed on
                ↪   input {i}: {exc}") from exc

            predictions.append(output)

        latencies.append(timer.elapsed * 1000)  # Convert to ms

    return latencies, predictions
```

This loop warms up on the first `max(1, num_runs // 10)` inputs, then times one forward per input and retains the outputs for accuracy scoring. Retaining a prediction also retains whatever storage that output owns until scoring finishes; this is an evaluation loop, not a peak-memory measurement. The standard benchmark then builds the inputs, runs both phases, and applies the two bars.

```
def mlperf_run_standard_benchmark(self, model: Any, benchmark_name:
↪   str,
                                  num_runs: int = 100,
                                  test_inputs: Optional[List[Any]] = None,
                                  labels: Optional[np.ndarray] = None) ->
↪   Dict[str, Any]:
    """
    Run a standardized MLPerf benchmark.
    """
    if benchmark_name not in self.benchmarks:
        available = list(self.benchmarks.keys())
        raise ValueError(
            f"Unknown MLPerf benchmark: '{benchmark_name}'\n"
            f"  ❌ '{benchmark_name}' is not a registered benchmark\n"
            f"  🔔 MLPerf defines standard edge ML benchmarks for
            ↪   reproducible comparison\n"
            f"  🔧 Choose from: {available}"
        )

    config = self.benchmarks[benchmark_name]
    print(f"🧪 Running MLPerf {benchmark_name} benchmark...")
    print(f"   Target: {config['target_accuracy']:.1%} accuracy (MLPerf
    ↪   Tiny), "
          f"p90 < {config['max_latency_ms']}ms (TinyTorch classroom
          ↪   ceiling)")

    # Use the caller's test set, or generate standardized test inputs
    # (as Tensors for TinyTorch model compatibility)
    input_shape = config['input_shape']
    if test_inputs is not None:
        test_inputs = list(test_inputs)
        num_runs = len(test_inputs)
```

```
    else:
        # Seeded from self.random_seed, not a hardcoded constant: the same
        # inputs on every run of this benchmark, and a different set only when
        # the caller asks for a different seed. A hardcoded seed here would make
        # the constructor's random_seed argument decorative.
        input_rng = np.random.default_rng(self.random_seed)
        test_inputs = []
        for _ in range(num_runs):
            if len(input_shape) == 2:  # Audio/sequence data (keyword_spotting, anomaly_detection)
                arr = input_rng.standard_normal(input_shape).astype(np.float32)
            else:  # Image data (visual_wake_words, image_classification) - use CHW for Conv2d
                arr = input_rng.integers(0, 256, input_shape).astype(np.float32) / 255.0
                if arr.ndim == 4 and arr.shape[-1] == 3:  # (B,H,W,C) -> (B,C,H,W)
                    arr = np.transpose(arr, (0, 3, 1, 2))
            test_inputs.append(Tensor(arr))

    # Run latency and accuracy tests using helpers
    if num_runs <= 0:
        raise ValueError("Benchmark needs at least one input")
    if labels is not None and np.asarray(labels).shape != (num_runs,):
        raise ValueError("labels must contain one class index per test input")
    latencies, predictions = self._run_latency_test(model, test_inputs, benchmark_name, num_runs)
    accuracy = self._run_accuracy_test(model, predictions, benchmark_name, num_runs, labels)

    # Compile results. The gate is p90, MLPerf Inference's single-stream metric,
    # because every timed call here processes exactly one example. The mean and
    # p99 are reported alongside it but neither decides the verdict.
    mean_latency = float(np.mean(latencies))
    p90_latency = float(np.percentile(latencies, 90))
    p99_latency = float(np.percentile(latencies, 99))
    accuracy_met = bool(accuracy >= config['target_accuracy'])
    latency_met = bool(p90_latency <= config['max_latency_ms'])

    results = {
        'synthetic_labels': labels is None,
        'official_mlperf': False,
        'benchmark_name': benchmark_name,
        'model_name': getattr(model, 'name', 'unknown_model'),
        'accuracy': float(accuracy),
```

```
        'mean_latency_ms': mean_latency,
        'std_latency_ms': float(np.std(latencies)),
        'p50_latency_ms': float(np.percentile(latencies, 50)),
        'p90_latency_ms': p90_latency,
        'p99_latency_ms': p99_latency,
        # The OBSERVED maximum, not a ceiling. The ceiling is target_latency_ms.
        'max_observed_latency_ms': float(np.max(latencies)),
        # One example per timed call, so 1/latency is a real rate. Not 'fps':
        # keyword spotting scores audio windows and there are no frames here.
        'inferences_per_second': float(1000 / mean_latency),
        'gating_statistic': 'p90_latency_ms',
        'target_accuracy': float(config['target_accuracy']),
        'target_latency_ms': float(config['max_latency_ms']),
        'accuracy_met': accuracy_met,
        'latency_met': latency_met,
        'compliant': accuracy_met and latency_met and labels is not None,
        'num_runs': int(num_runs),
        'random_seed': int(self.random_seed)
    }

    print(f"   Results: {accuracy:.1%} accuracy, {p90_latency:.1f}ms p90 latency (gated), "
          f"{mean_latency:.1f}ms mean")
    print(f"   Compliance: {'✓ PASS' if results['compliant'] else '✗ FAIL'}")

    return results
```

The returned dictionary reports the mean and several percentiles, but the latency decision reads P_{90}, the single-stream statistic; P_{99} is reported beside it and does not decide the verdict. Accuracy comes from the following helper.

```
def _mlperf_run_accuracy_test(self, model: Any, predictions: List[Any],
                                    benchmark_name: str, num_runs: int,
                                    labels: Optional[np.ndarray] = None) -> float:
    """
    Calculate accuracy from predictions against the reference labels.
    """
    # anomaly_detection sits in the binary branch because this harness has one
    # scoring path. Real MLPerf Tiny scores it by AUC over an autoencoder's
    # reconstruction error, which needs the full score distribution rather than a
```

```
    # thresholded label, so the simplification is TinyTorch's and not the standard's.
    binary = benchmark_name in ['keyword_spotting', 'visual_wake_words', 'anomaly_detection']
    num_classes = 2 if binary else 10
    if num_runs <= 0 or len(predictions) != num_runs:
        raise ValueError("Accuracy needs one prediction per test input")
    rng = np.random.default_rng(self.random_seed)
    true_labels = np.asarray(labels) if labels is not None else rng.integers(0, num_classes, num_runs)
    if (true_labels.shape != (num_runs,)
            or not np.issubdtype(true_labels.dtype, np.integer)
            or np.any(true_labels < 0) or np.any(true_labels >= num_classes)):
        raise ValueError("labels must contain one integer class index in the task's range per input")

    predicted_labels = []
    for pred in predictions:
        raw = np.asarray(pred.data if isinstance(pred, Tensor) else pred)
        # Each timed input represents one example: accept a score vector or a
        # singleton batch, never flatten several examples into one prediction.
        if raw.ndim == 2 and raw.shape[0] == 1:
            raw = raw[0]
        if raw.ndim == 0 and binary:
            raw = raw.reshape(1)
        allowed_widths = (1, 2) if binary else (10,)
        if raw.ndim != 1 or raw.size not in allowed_widths:
            raise ValueError(f"Prediction must contain {allowed_widths} class scores for one example")
        if not np.issubdtype(raw.dtype, np.number) or np.iscomplexobj(raw) or not np.all(np.isfinite(raw)):
            raise ValueError("Prediction class scores must be finite real numbers")
        raw = _extract_pred_array(raw)
        if binary and raw.size == 1:
            if not 0 <= raw[0] <= 1:
                raise ValueError("A single binary score must be a probability in [0, 1]")
            predicted_labels.append(int(raw[0] > 0.5))
        else:
            predicted_labels.append(int(np.argmax(raw)))

    return float(np.mean(true_labels == predicted_labels))
```

Read its checks before its final average. For a binary task, each output must contain two finite real class scores, or one probability in $[0, 1]$. For image classification it must contain exactly ten finite scores. A vector or a singleton batch such as `(1, 2)` is accepted;

a batch containing several examples is not silently flattened. The helper rejects NaNs and invalid shapes before choosing a class. Labels must be integer class indices, one per input, within the task's class range.

For scores `[[0.2, 0.8]]`, removing the singleton batch leaves `[0.2, 0.8]`; `argmax` selects class 1. Comparing that prediction with label 1 contributes one correct answer. A single binary probability uses a threshold of 0.5 instead. Agreement count divided by input count is the accuracy. This path makes invalid numerical output an error rather than a plausible class-zero prediction.

Without supplied labels, the class draws synthetic labels from its seed and sets `synthetic_labels=True`. Such a run always has `compliant=False`, even if a small sample happens to exceed the accuracy threshold. Supplying labels enables the local decision, but cannot prove their provenance: the caller must supply independent ground truth. Here, as book code, are the two numerical bars on an already-validated `BenchmarkResult`, plus a relative comparison. These helpers assume finite measured values, positive latency denominators, and task accuracies in $[0, 1]$; they do not validate a dataset or replace the module's input checks.

```
def check_compliance(result, accuracy, max_latency_ms, min_accuracy):
    """A fast model that is wrong does not pass. Both bars, or
    ↪   neither. Book code."""
    latency_ok = result.percentile(99) <= max_latency_ms
    accuracy_ok = accuracy >= min_accuracy
    return {
        "p99_ms": round(result.percentile(99), 2),
        "latency_ok": latency_ok,
        "accuracy_ok": accuracy_ok,
        "compliant": latency_ok and accuracy_ok,
    }

def compare(baseline, optimized, baseline_acc, optimized_acc,
↪   retain=0.99):
    """Speedup at the median and at the tail, valid only if accuracy
    ↪   held. Book code."""
    return {
        "speedup_p50": round(baseline.percentile(50) /
        ↪   optimized.percentile(50), 2),
        "speedup_p99": round(baseline.percentile(99) /
        ↪   optimized.percentile(99), 2),
        "keeps_accuracy": optimized_acc >= retain * baseline_acc,
    }
```

Notice that `check_compliance` reads P_{99} and never the mean, so a model that is fast on average but stalls too often fails. `compare` reports the speedup twice, at the median and at the tail, because an optimization can improve one without the other. Removing work can shorten typical execution while unrelated scheduling stalls remain. The relative retention threshold is a policy chosen for this book. The capstone owns its separate classroom event policy through `OlympicEvent` and `qualifies_event`; those rules do not alter the measurements produced here. Official benchmark requirements depend on the task and scenario; neither this helper nor the module implements their complete protocol.

20.4 A Worked Trace

The trace uses ten invented durations for one model to make the bookkeeping visible. Interpret them as a harness run with two warmups, one discarded timing, and seven retained measurements. The sixth retained measurement is a stall.

Table 20.1: Ten runs of one model under the naive script and under the TinyTorch protocol.

Run	Latency (ms)	Naive script	TinyTorch protocol	What happened
1	48.20	included	discarded (warmup)	cold start: allocation, first touch of memory
2	14.50	included	discarded (warmup)	caches still filling
3	7.10	included	discarded timing	first profiler call's timing
4	5.12	included	measured ($i = 1$)	steady state
5	5.08	included	measured ($i = 2$)	steady state
6	4.95	included	measured ($i = 3$)	steady state
7	5.20	included	measured ($i = 4$)	steady state
8	5.10	included	measured ($i = 5$)	steady state
9	18.40	included	measured ($i = 6$)	stall: the OS ran something else
10	5.05	included	measured ($i = 7$)	steady state

An unfiltered repeated loop averages all ten, and the sum $48.20 + 14.50 + 7.10 + 5.12 + 5.08 + 4.95 + 5.20 + 5.10 + 18.40 + 5.05 = 118.70$ divided by 10 gives 11.87 ms, more than twice the steady-state cost, with the cold start alone contributing 4.3 ms of the excess.

The protocol keeps runs 4 through 10, sorted: $[4.95, 5.05, 5.08, 5.10, 5.12, 5.20, 18.40]$, with $N = 7$ and positions 0 through 6, as listed in table 20.1.

- **Mean**: $48.90/7 = 6.99$ ms, still inflated by the stall.
- P_{50}: position $0.5 \times 6 = 3$, the fourth entry, 5.10 ms. This is the steady-state cost.
- P_{95}: position $0.95 \times 6 = 5.7$, so $5.20 + 0.7 \times (18.40 - 5.20) = 5.20 + 9.24 = 14.44$ ms.
- P_{99}: position $0.99 \times 6 = 5.94$, so $5.20 + 0.94 \times 13.20 = 17.61$ ms.
- **Throughput**: $7/0.0489$ s $= 143.1$ samples per second.

Constructing the container checks that the hand arithmetic agrees with the implementation. In line 3, `raw` lists the ten recorded durations from table 20.1. Line 5 builds

`naive = BenchmarkResult("naive", raw)`, which naively includes cold-start and setup artifacts, printing a distorted mean of 11.87 ms in line 8. In line 6, `protocol = BenchmarkResult("protocol", raw[3:])` discards the two warmups and initial profiler timing. Lines 9 to 12 print the true steady-state metrics: median $P_{50} = 5.10$ ms, $P_{95} = 14.44$ ms, $P_{99} = 17.61$ ms, and throughput of 143.1 samples/sec. Lines 13 to 15 assert agreement with the manual mean, P_{50}, and P_{99} to within 0.01 ms.

```
from tinytorch.perf.benchmarking import BenchmarkResult

raw = [48.20, 14.50, 7.10, 5.12, 5.08, 4.95, 5.20, 5.10, 18.40, 5.05]

naive = BenchmarkResult("naive", raw)              # all ten runs
protocol = BenchmarkResult("protocol", raw[3:])    # setup observations discarded

print(f"naive mean:   {naive.mean:.2f} ms")
print(f"protocol p50: {protocol.percentile(50):.2f} ms  (median attribute: {protocol.median:.2f})")
print(f"protocol p95: {protocol.percentile(95):.2f} ms")
print(f"protocol p99: {protocol.percentile(99):.2f} ms")
print(f"throughput:   {protocol.count / (sum(protocol.values) / 1000):.1f} samples/s")
assert abs(naive.mean - 11.87) < 0.01
assert abs(protocol.percentile(50) - 5.10) < 0.01
assert abs(protocol.percentile(99) - 17.61) < 0.01
```

For these seven observations, the median is 5.10 ms (line 9) and the interpolated P_{99} is 17.61 ms (line 11). Seven observations cannot establish what one future request in a hundred will experience. The trace checks the percentile arithmetic; a repeated experiment must supply the evidence for a tail-latency claim.

The accuracy path needs a separate trace, with labels fixed independently of the model. Use a small synthetic task: label an input 1 when the sum of its first two entries is positive, otherwise 0. Pad the remaining audio-shaped input with zeros. The four chosen pairs sum to $-3, 3, -1, 1$, so the labels are `[0, 1, 0, 1]`.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.perf.benchmarking import MLPerf

class SumClassifier:
    def forward(self, x):
        total = float(x.data[0, :2].sum())
        return Tensor([[-total, total]])  # shape (1, 2)

pairs = [(-2, -1), (2, 1), (-2, 1), (2, -1)]
inputs = []
for pair in pairs:
    data = np.zeros((1, 16000), dtype=np.float32)
    data[0, :2] = pair
    inputs.append(Tensor(data))
labels = np.array([0, 1, 0, 1], dtype=np.int64)
```

```
report = MLPerf().run_standard_benchmark(
    SumClassifier(), "keyword_spotting", test_inputs=inputs,
    labels=labels
)
assert report["num_runs"] == 4
assert report["accuracy"] == 1.0
assert report["synthetic_labels"] is False
assert report["compliant"] == report["latency_met"]
```

The first input produces scores `[3, -3]` and selects class 0; the second produces `[-3, 3]` and selects class 1. All four predictions agree with the independent rule, so accuracy is 1.0. The run count becomes four because supplied inputs determine it, regardless of the default argument. Latency remains a machine measurement and is not assigned a fixed expected value.

This checks scoring and orchestration on a known rule, not keyword-spotting quality. The field `synthetic_labels=False` means labels were supplied; it does not certify that the data represents real audio. Replacing one output with `[NaN, NaN]` must raise `ValueError` before a report is returned. Replacing the input and label lists with a held-out task dataset is what turns a pipeline check into a task evaluation.

20.5 From TinyTorch to Production

A CUDA operation can return control to the CPU while its work is still queued on the GPU. To time completed computation with a CPU clock, synchronize before the start reading and after the model call, before the end reading. Alternatively, record CUDA events[80] around the device work and wait for completion before reading elapsed time, as highlighted in the Checkpoint below. These are different timing boundaries from TinyTorch's completed CPU forwards. PyTorch's CUDA semantics documentation explains this distinction; its benchmark timer supplies warmups, replication, and accelerator synchronization.[81]

A common benchmarking trap occurs when measuring accelerator code using standard CPU timers:

```
# FAULTY BENCHMARK: Measures CPU kernel launch dispatch, NOT GPU
    execution!
t0 = time.perf_counter()
output = model(inputs)  # Enqueues kernels asynchronously to device
    stream
t1 = time.perf_counter()
print(f"Latency: {(t1 - t0) * 1000:.2f} ms")  # Phantom speedup:
    reports 0.05 ms!
```

Because device driver kernel launches return asynchronously to the host, `t1` records when the command was enqueued, not when computation completed.

The Fix: Synchronize host execution before and after the timed region:

```
# CORRECT BENCHMARK:
torch.cuda.synchronize()
```

[80] **CUDA event**: a marker recorded in a GPU stream, the ordered queue of device work. With timing enabled, two completed events provide elapsed time between their positions in that stream. Work from other streams or gaps between submitted operations can still affect the interval.

[81] **CUDA Graphs and kernel launch overhead**: Even when GPU execution is asynchronous, the CPU overhead of launching individual CUDA kernels (typically 5 to 10 microseconds per kernel dispatch) can dominate execution time for small models or shallow layers. CUDA Graphs capture an entire sequence of kernel launches and memory dependencies into an execution graph. Subsequent benchmark iterations replay the pre-instantiated graph with a single CPU launch call, eliminating host driver overhead and isolating pure silicon execution latency.

```
t0 = time.perf_counter()
output = model(inputs)
torch.cuda.synchronize()
t1 = time.perf_counter()
```

Alternatively, record timestamps on device streams using `torch.cuda.Event` with `enable_timing=True`.[82]

A more complete experiment also varies and records its conditions. Repeating the same setup at different times can reveal interference that one long run hides. Interleaving baseline and candidate measurements can reduce the effect of a slow change in machine load. Neither a warmup nor a fixed random seed guarantees a stable machine, as detailed in the Systems Perspective below, so preserve the raw observations and report the comparison procedure.

In real production and edge benchmarking environments, unpredictable performance jitter often stems from dynamic OS and power management rather than algorithmic code:

- **CPU Frequency Scaling**: Dynamic voltage and frequency scaling (DVFS) governors like Linux `powersave` or `ondemand` ramp clock frequencies dynamically. Rigorous server benchmarking pins governors to constant frequencies:

```
sudo cpupower frequency-set --governor performance
```

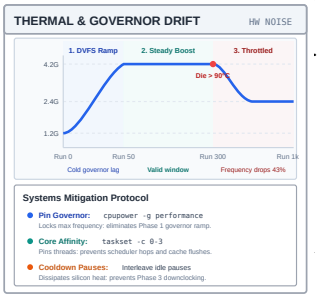
- **Thermal Throttling**: Sustained compute heats processor silicon, causing hardware safety governors to throttle clock multipliers midway through benchmark loops. Monitor die temperature and allow inter-run cooling pauses.
- **Core Pinning and Thread Affinity**: Preemptive OS scheduler migrations move threads across cores, invalidating L1/L2 caches. Pin benchmark processes to isolated cores using `taskset` or `numactl`:

```
taskset -c 0-3 python3 benchmark_inference.py
```

More samples support uncertainty analysis. A bootstrap[83] can show how a statistic varies across resampled observations, but it cannot invent rare stalls absent from the measured list. The statistic itself must match the question. Python's `timeit` documentation recommends considering the minimum as a lower bound on small-snippet execution cost under interference. An application latency claim instead needs the delays its users experience, including the tail.

MLPerf formalizes workloads, quality checks, and measurement scenarios for comparisons across systems.[84] TinyTorch borrows the original four task categories from MLPerf Tiny, which targets small, resource-constrained devices.[85] It does not reproduce the official datasets, quality metrics, or complete protocols. The Tiny benchmark description defines that suite's scope, while the Inference submission guide describes the checks accompanying an official inference submission. A TinyTorch `compliant=True` remains a local classroom verdict.

[82] **CUDA events vs. host synchronization**: Calling `torch.cuda.synchronize()` on the CPU blocks the host thread until the GPU has completely drained all submitted command queues. While necessary before reading CPU clocks, doing so inside a per-layer loop artificially flushes the GPU hardware execution pipeline.



Hardware governor and thermal drift: untuned CPUs ramp frequencies late and throttle when hot, distorting sustained speedup claims.

[83] **Bootstrap**: sample with replacement from the measured observations, recompute the statistic, and repeat to examine its spread. Its usefulness depends on whether those observations and the resampling scheme represent the experiment; resampling cannot recover unobserved tail events.

[84] Vijay Janapa Reddi et al., "MLPerf Inference Benchmark", *International Symposium on Computer Architecture (ISCA)*, 2020.

[85] Colby Banbury et al., "MLPerf Tiny Benchmark", *NeurIPS Datasets and Benchmarks Track*, 2021.

Table 20.2: What changes between the classroom harness and production measurement, and what does not.

Concern	TinyTorch benchmark classes	PyTorch/MLPerf practice
Clock	`time.perf_counter` on the CPU, read through `Profiler` or `precise_timer`	CUDA events, or `synchronize()` before reading the CPU clock
Warmup	`Benchmark`: 5 warmups plus 1 discarded timing; `MLPerf`: at least 1 warmup, otherwise a tenth of the runs	Warmup and replication configured for the workload
Repetition	`Benchmark`: 10; `MLPerf`: 100, unless supplied inputs set the count	Counts chosen for measurement uncertainty or prescribed by benchmark rules
Statistic reported	mean, std, median, min, max, an approximate interval on the mean, percentiles on demand	Summary and scoring rule depend on the experiment
Accuracy floor	`MLPerf`: local task target; `Benchmark`: caller-supplied evaluation	Task-specific quality target with prescribed validation data
Reproducibility	Per-model input reused; counts and basic platform metadata recorded; caller controls mode and state	Full hardware and software configuration logged with every result

The comparison in table 20.2 preserves the chapter's central contract: a report must explain what completed, what was retained, and which statistic was compared. A production experiment adds workload-specific execution boundaries and validation, rather than making a classroom sample authoritative by collecting more of it. The reporting discipline extends beyond ML; Hoefler and Belli discuss reproducible performance experiments and the evidence needed to interpret their results.[86]

[86] Torsten Hoefler and Roberto Belli, "Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results", *SC*, 2015.

20.6 Summary

The latency harness creates an input, warms the model, discards one additional timing, and retains one duration per measured forward. `BenchmarkResult` keeps those observations and computes summaries; the mean describes average cost, the median describes typical cost, and a tail percentile describes a high position in the sorted sample. A small sample can demonstrate that arithmetic without establishing a rare-event guarantee.

The MLPerf-style path adds predictions and labels to the timing loop. It validates their shapes and values, counts agreements, and checks both accuracy and P_{90} against local thresholds. Synthetic labels cannot pass that local check, and supplied labels still need independent provenance. Chapter 21 uses these measurement habits to compare combinations of optimizations under its own event policy. Preserve the baseline, control

input shape and model state, check quality on the same data, and compare the same named statistic before combining another mechanism.

20.7 Further Reading

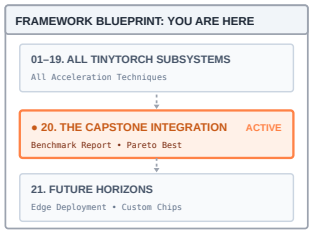
- **Georges, Buytaert, and Eeckhout** (2007). "Statistically rigorous Java performance evaluation." *OOPSLA*. Warmup, repeated runs, and confidence intervals for timing in a managed runtime.
- **Mattson et al.** (2020). "MLPerf training benchmark." *MLSys*. What a fair machine learning benchmark must fix: the task, the quality target, and the rules.
- **Reddi et al.** (2020). "MLPerf inference benchmark." *ISCA*. Latency percentiles and serving scenarios for inference.
- **Banbury et al.** (2021). "MLPerf Tiny benchmark." *NeurIPS*. Standardizing benchmarks for deeply embedded systems and microcontrollers, where memory constraints dictate optimization choices.
- **Little** (1961). "A proof for the queuing formula $L = \lambda W$." *Operations Research*. The law relating concurrency, throughput, and latency used above.

🔥 Chapter 21

The Capstone: From an Optimization to a Checkable Report

Real-world machine learning systems engineering is never about applying a single optimization in isolation; it is an exercise in multi-objective trade-offs. An optimization that doubles inference speed is unacceptable if it drops accuracy below the task's floor, and a compression technique that reduces storage can still lose if it adds latency. The systems engineer must synthesize multiple optimizations into a coherent, balanced production pipeline.

In production serving engines, optimizing an end-to-end generative pipeline composes multiple acceleration levers into a single loop:



Framework Blueprint: Module 20 synthesizes training, profiling, quantization, and caching into a verified production optimization report.

```python
import torch

# The end-to-end optimized generation pipeline:
# Tokenization -> Embedding -> Quantized Transformer Blocks -> KV-Cached Autoregression
def generate_tokens(model, prompt_ids, max_new_tokens=20):
    input_ids = prompt_ids
    past_key_values = None
    generated = []
    for _ in range(max_new_tokens):
        # Forward pass using cached past key-values and quantized weights
        outputs = model(input_ids, past_key_values=past_key_values, use_cache=True)
        next_token = outputs.logits[:, -1, :].argmax(dim=-1, keepdim=True)
        generated.append(next_token)
        input_ids = next_token  # Feed only the single newest token
        past_key_values = outputs.past_key_values
    return torch.cat(generated, dim=-1)
```

Underneath that end-to-end generation loop, the systems engineer must navigate complex trade-offs and non-linear interactions:

1. **The Amdahl's Law bottleneck:** Individual optimizations (INT8 quantization, kernel fusion, and KV caching) do not multiply independently. Once the KV cache eliminates redundant attention projections, remaining runtime is dominated by MLP blocks, and optimizing memory traffic buys smaller incremental gains.

2. **Quality regression bounds:** Compressing parameters or applying low-rank SVD perturbs floating-point numerical outputs. Any speedup claim is invalid unless candidate model accuracy is strictly evaluated against the baseline ground truth.
3. **The Pareto trade-off surface:** A systems deployment scorecard must balance accuracy, persistent memory footprint, and latency percentiles (P50, P99), identifying configurations that lie on the optimal trade-off frontier.

This chapter combines Part III's measurements and models into one checkable report, tracing each claimed gain to the architectural lever that produced it. Before synthesizing candidate models, one must inspect the reporting traps that arise when isolated speedups are combined naively.

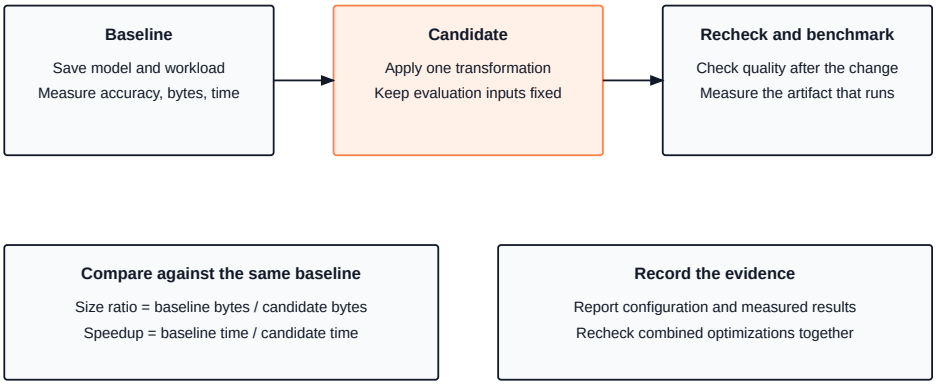


Figure 21.1: **An optimization needs a comparison**: Preserve the baseline and workload, transform a candidate, recheck quality, and measure its actual arrays and execution before recording the comparison.

21.1 The Problem

↩ **Jump back:** Chapter 8 *Model evaluation, validation loss floors, and test accuracy metrics.*
↩ **Jump back:** Chapter 16 *Synthesizing INT8 quantization, distillation, and KV caching into a coherent serving pipeline.*

Consider a baseline that gets four of four labels right. Zeroing an output weight makes the candidate predict class zero for every input, so only two answers remain correct. The storage ratio is still 1.0 because a zero occupies the same float32 slot as any other value. If the timing improves, that result must travel with the accuracy loss and unchanged storage count. The report in figure 21.1 keeps those quantities separate.

Combining transformations creates another reporting trap. Suppose four separate experiments against one baseline produce speedups of 2.0x, 1.5x, 1.3x, and 4.2x. Their product is 16.4x, but these ratios do not reveal whether the transformations remove the same work. They cannot establish the speed of the combined model.

The napkin version of the failure is Amdahl's law.[87] Say matrix multiplies take 70 percent of a decode step and everything else takes 30 percent. Speed the multiplies up by 10x and the step takes $0.30 + 0.70/10 = 0.37$ of its old time, a 2.7x speedup. Speed them up infinitely and the step still takes 0.30 of its old time. A lever that shrinks one term of a sum can never buy more than that term's share of the total, and once it has bought most of it, the next lever on the same term buys almost nothing.

[87] **Amdahl's law**: if a fraction f of the total time is sped up by a factor k, the overall speedup is $1/((1-f)+f/k)$. The limit as $k \to \infty$ is $1/(1-f)$, the ceiling set by the part that was left alone.

The product can also underestimate a combination. Suppose a 100-millisecond task spends 40 milliseconds in each of two routines and 20 milliseconds elsewhere. Making either routine four times faster gives a solo time of 70 milliseconds, or a 1.43x speedup. Improving both gives 40 milliseconds, or 2.5x, greater than the product of the two solo ratios, about 2.04x. The missing information is which work each transformation changes.

21.2 The Idea

21.2.1 Three terms in a decode step

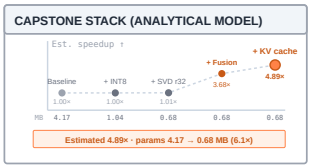
↪ **Jump ahead:** Synthesis III *The MLPerf Optimization Olympics: deploying optimized candidate models against strict competitive scoring gates.*

Analytical performance engineering in systems like TensorRT-LLM and vLLM decomposes end-to-end token latency into distinct physical and runtime bottlenecks. Rather than treating execution as an opaque black box, systems architects isolate memory streaming, compute throughput, and host dispatch latency to predict where acceleration efforts will yield diminishing returns. Chapter 15 bounded the time of one operation by the larger of two costs, the bytes it must move divided by memory bandwidth and the arithmetic it must do divided by peak throughput. It then noted a third floor for small operations. A separate operation also incurs dispatch overhead: the work of invoking the routine, preparing its arguments, and returning control. The size of that overhead depends on the runtime. Many small operations can spend more time in dispatch than in moving operands or computing results. Those three costs are the cost model of this chapter:

$$T_{\text{step}} = \max\left(\frac{Q}{B_{\text{peak}}}, \frac{F}{P_{\text{peak}}}\right) + n_{\text{ops}} \cdot c$$

Here Q is the bytes streamed in the step, F the arithmetic operations, B_{peak} and P_{peak} the machine's two ceilings from Chapter 15, n_{ops} the number of separate operations the step launches, and c the fixed cost of one launch. The maximum is Chapter 15's roofline; the added term is the launch floor. For this estimate, adding a fixed dispatch cost to a roofline bound establishes the composite model. That is a modeling assumption, not an execution law: asynchronous host dispatch can overlap device work, and a real trace may require per-operation costs rather than one aggregate maximum.

Each of the four levers acts on one of the three quantities, and that is the whole reason the model has three terms rather than one. INT8 quantization (Chapter 16) shrinks Q, low-rank factorization (Chapter 17) shrinks matrix storage and arithmetic but adds intermediate arrays and products, kernel fusion (Chapter 18) shrinks n_{ops} here, although a fused implementation can also reduce intermediate-memory traffic, and the KV cache (Chapter 19) changes how F grows with the number of tokens already generated. The Code says how much, lever by lever, next to the line that implements each one.



Estimated cumulative speedup and parameter memory as the four levers stack, from the analytical model in table 21.2.

21.2.2 When speedups multiply, and when they do not

In production systems optimization, evaluating compound speedups requires navigating Amdahl's Law and the non-linear interactions of stacked compiler passes. Naively multiplying isolated component speedups overstates end-to-end gains because each optimization diminishes the relative runtime share of the subsystem it accelerates. Ratios measured along a sequence always multiply. If baseline time is 100 milliseconds, the first candidate takes 70, and the second takes 40, then $(100/70)(70/40) = 100/40$. The

intermediate time cancels. These are marginal ratios, each comparing a candidate with the immediately preceding configuration.

Solo ratios use a different denominator. Both compare with the unchanged baseline, so their product has no such cancellation. When two transformations act on the same dominant term, their solo factors can happen to predict the combination. Once another cost becomes significant, the estimate changes. Reducing dispatch overhead, for example, can expose byte transfers that barely mattered in the original total. The practical task is to measure the marginal change against the program that will actually precede it.

That last observation is the practical content of the chapter. The marginal speedup a lever appears to deliver depends on the order in which the levers are applied, while this model's total for the full stack does not. Its configuration edits commute; real transformations can change one another's inputs, so reversing quantization and factorization need not produce the same weights. Only the model, or a measurement of the complete stack, gives a number that survives, and the rest of the chapter builds both.

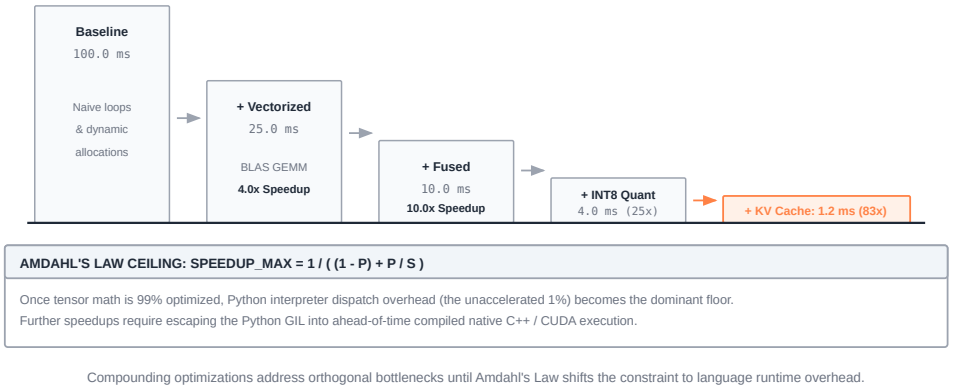


Figure 21.2: **Compound optimization stacking and Amdahl's law**: Stacking multiple optimization techniques (SVD low-rank compression, INT8 quantization, KV caching, and kernel fusion) yields substantial cumulative speedups. However, each subsequent optimization exhibits diminishing marginal returns as Amdahl's law dictates that unaccelerated serial components (framework dispatch, Python overhead, memory allocation) eventually dominate execution time.

21.3 The Code

Control enters `BenchmarkReport.benchmark_model` with a model, an input Tensor, and integer class labels. The method gathers measurements into `self.metrics`; `generate_submission` combines two reports without running either model again. Validation and event eligibility read that record. Following this path first establishes what a reported improvement means before estimating which transformation might produce one.

21.3.1 From model outputs to report state

For inputs of shape `(N, features)`, a classifier must return finite real scores of shape `(N, classes)`. `benchmark_model` first counts parameters and calls `measure_memory`, then runs the full test batch for accuracy. It rejects malformed scores, NaN or infinite values, and labels that are not integer indices into the class axis. These checks precede `argmax`: an array of NaNs can otherwise select class zero and falsely reward a broken model when the labels happen to be zero.

```
def benchmark_model(self, model, X_test: Tensor, y_test, num_runs: int = 100) -> Dict[str, Any]:
    """
    Benchmark model performance comprehensively.
    """
    if len(X_test.shape) == 0 or X_test.shape[0] == 0:
        raise ValueError("X_test must contain at least one sample")
    y_test = np.asarray(y_test)
    if y_test.shape != (X_test.shape[0],):
        raise ValueError("y_test must contain one class index per sample")
    if num_runs <= 0:
        raise ValueError("num_runs must be positive")
    # Count parameters and stored size (see measure_memory)
    param_count = (
        model.count_parameters()
        if hasattr(model, "count_parameters")
        else sum(p.data.size for p in model.parameters())
    )
    model_size_mb = self.measure_memory(model)

    # Validate classification outputs before argmax: NaN scores can otherwise
    # silently select class zero and make a diverged model look accurate.
    predictions = model.forward(X_test)
    scores = np.asarray(predictions.data)
    if scores.ndim != 2 or scores.shape[0] != X_test.shape[0] or scores.shape[1] == 0:
        raise ValueError("Predictions must have shape (samples, classes) with at least one class")
    if not np.issubdtype(scores.dtype, np.number) or np.iscomplexobj(scores) or not np.all(np.isfinite(scores)):
        raise ValueError("Prediction scores must be finite real numbers")
    if not np.issubdtype(y_test.dtype, np.integer):
        raise ValueError("y_test must contain integer class indices")
    if np.any(y_test < 0) or np.any(y_test >= scores.shape[1]):
        raise ValueError("y_test class indices must be within the prediction classes")
    pred_labels = np.argmax(scores, axis=1)
    accuracy = np.mean(pred_labels == y_test)

```

```
    # Latency: untimed warmup, then num_runs single-sample calls timed one by one
    # Why multiple runs? See "Variance" section in Foundations
    latencies = self.measure_latency(model, X_test, num_runs)

    avg_latency = np.mean(latencies)
    std_latency = np.std(latencies)
    median_latency = np.median(latencies)
    # Tail percentiles come from Module 19's BenchmarkResult, which already holds
    # the sample and its percentile rule. The tail is what an SLA is written
    # against, so a report that stores only a center cannot express one.
    latency_dist = BenchmarkResult('latency_ms', latencies)
    p95_latency = latency_dist.percentile(95)
    p99_latency = latency_dist.percentile(99)

    # Throughput: the same warmup-then-time shape you implement in measure_latency,
    # applied to the WHOLE batch in one forward call. It is given rather than left as
    # a second exercise because only the denominator differs: samples per second here,
    # milliseconds per sample there. Read the two loops side by side.
    # This is deliberately NOT 1000 / latency; batching raises samples/second without
    # making any one sample faster (Foundations, "Latency vs. Throughput")
    batch_size = X_test.shape[0]
    for _ in range(min(5, num_runs)):
        _ = model.forward(X_test)
    batch_seconds = []
    for _ in range(num_runs):
        with precise_timer() as timer:
            _ = model.forward(X_test)
        batch_seconds.append(timer.elapsed)
    # Median batch time, floored so a batch that measures 0.0 s cannot divide by zero
    throughput = batch_size / max(np.median(batch_seconds), 1e-9)

    # Store metrics (all as Python native types for JSON serialization)
    self.metrics = {
        'parameter_count': int(param_count),
        'model_size_mb': float(model_size_mb),
        'accuracy': float(accuracy),
        'latency_ms_mean': float(avg_latency),
        'latency_ms_std': float(std_latency),
        'latency_ms_median': float(median_latency),
        'latency_ms_p95': float(p95_latency),
        'latency_ms_p99': float(p99_latency),
```

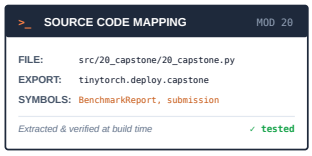
```
        'throughput_samples_per_sec': float(throughput)
    }

    print(f"\n📊 Benchmark Results for {self.model_name}:")
    print(f"  Parameters: {param_count:,}")
    print(f"  Size: {model_size_mb:.5f} MB ({model_size_mb * 1_000_000:,.0f} bytes)")
    print(f"  Accuracy: {accuracy*100:.1f}%")
    print(f"  Latency: {avg_latency:.2f}ms ± {std_latency:.2f}ms (median {median_latency:.2f}ms)")
    print(f"  Tail latency: p95 {p95_latency:.2f}ms, p99 {p99_latency:.2f}ms")
    print(f"  Throughput: {throughput:,.0f} samples/sec (batch of {batch_size})")

    return self.metrics
```

After accuracy, `measure_latency` selects the first sample, warms up with up to five untimed calls, and collects `num_runs` durations, 100 by default. The method then times full-batch forwards in a separate loop. The median batch duration determines throughput; the mean, standard deviation, and median of single-sample durations describe latency. Only after these measurements succeed does it replace `self.metrics` with nine Python numeric values and return that same dictionary. The caller controls model mode and mutable state; the harness does not reset a cache or disable training behavior automatically.

The two timing boundaries answer different questions. A faster batch does not establish lower single-request latency, and the first sample alone does not represent every possible input length. The report discards the raw durations after summarizing them. It builds a Chapter 20 `BenchmarkResult` to store P_{95} and P_{99} beside the mean, standard deviation, and median, but a study that needs a different statistic, or a tail estimate from more than `num_runs` samples, must keep its own observations.



Source Code Mapping: Module 20 extracts end-to-end benchmark reporting, optimization comparison, and competition submission generation directly from the working repository.

```
def generate_submission(
    baseline_report: BenchmarkReport,
    optimized_report: Optional[BenchmarkReport] = None,
    student_name: Optional[str] = None,
    techniques_applied: Optional[List[str]] = None
) -> Dict[str, Any]:
    """
    Generate a standardized benchmark submission.
    """
    submission = {
        'tinytorch_version': '0.1.0',
        'submission_type': 'capstone_benchmark',
        'timestamp': baseline_report.timestamp,
        'system_info': baseline_report.system_info,
        'baseline': {
            'model_name': baseline_report.model_name,
            'metrics': baseline_report.metrics
        }
```

```
    }

    # Add student name if provided
    if student_name:
        submission['student_name'] = student_name

    # Add optimization results if provided
    if optimized_report:
        submission['optimized'] = {
            'model_name': optimized_report.model_name,
            'metrics': optimized_report.metrics,
            'techniques_applied': techniques_applied or []
        }

        # Calculate improvement metrics
        # Compare medians when both reports carry one (Module 19: the median is
        # the honest center of a skewed latency distribution); a report that
        # recorded only a mean still compares
        key = 'latency_ms_median' if all('latency_ms_median' in r.metrics for r in (baseline_report, optimized_report)) else 'latency_ms_mean'
        baseline_latency = baseline_report.metrics[key]
        optimized_latency = optimized_report.metrics[key]
        baseline_size = baseline_report.metrics['model_size_mb']
        optimized_size = optimized_report.metrics['model_size_mb']

        submission['improvements'] = {
            # Floor both denominators: an empty model measures 0.0 ms, and a model
            # pruned to sparsity 1.0 stores 0 bytes
            'speedup': float(baseline_latency / max(optimized_latency, 1e-6)),
            'compression_ratio': float(baseline_size / max(optimized_size, 1e-9)),
            'accuracy_delta': float(
                optimized_report.metrics['accuracy'] - baseline_report.metrics['accuracy']
            )
        }

    return submission
```

Storage has an equally specific boundary. `measure_memory` sums each parameter array's `nbytes`, unless the model supplies `size_bytes()`. This counts stored parameters, not peak process memory or temporary activations. Zeros still occupy space. Module 20's simulated quantization example supplies a byte counter that includes both rounded float32 code arrays and retained original weights. Packing integer codes would be a different storage representation.

`generate_submission` reads the reports and builds a dictionary with baseline and optional optimized sections. It compares median latency when both reports contain a

median, otherwise it compares means. The size ratio is baseline storage divided by candidate storage; the accuracy delta is candidate accuracy minus baseline accuracy. These separate values preserve a trade-off that a single score would hide. System information and the timestamp come from the baseline report's construction. They do not establish that the optimized run used the same machine or data, so the experiment must control those conditions.

The submission keeps references to the report dictionaries; it is not an immutable snapshot. Editing a metric afterward can change the submission while leaving its derived ratios stale. `validate_submission_schema` checks required metrics in both sections, checks optional median, spread, tail-percentile, and throughput fields when supplied (requiring median $\leq P_{95} \leq P_{99}$), and recomputes supplied improvements for consistency. It cannot rerun the models or establish data provenance. `save_submission` only serializes, so the caller should validate the completed record before saving it.

Event eligibility is another explicit check. `qualifies_event(report.metrics, OlympicEvent.LATENCY_SPRINT)` applies the selected event's eligibility rule to an already measured report. The helper validates accuracy, median latency, and stored size before checking the selected event. It leaves the report and submission schema unchanged, and `generate_submission` does not call it automatically. The latency and memory events require absolute accuracy of at least 0.85; the extreme-push event requires 0.80. These floors are not relative to a baseline. The accuracy event instead requires median latency below 100 milliseconds and stored size below 10 MB (the report's `model_size_mb` key holds bytes divided by 10^6). All-around has no extra eligibility floor and no combined ranking formula. This keeps measurement, file validation, and classroom policy visible as separate decisions.

21.3.2 The machine and the model as a handful of numbers

Hardware-software co-design frameworks and production sizing calculators (such as Megatron-LM or DeepSpeed resource planners) abstract complex accelerators and model topologies into a compact set of first-principles parameters. Parameterizing memory bandwidth, floating-point throughput, and kernel dispatch latency enables engineers to project optimization headroom analytically before committing to costly implementation efforts. A report establishes what happened to one candidate. To choose the next candidate, an analytical model estimates which part of a forward pass consumes the most work. Four on/off transformations give sixteen combinations; a small analytical model allows inspecting their interactions before implementation. Its result remains a hypothesis to test on the chosen workload.

The machine below assumes 200 GB/s of bandwidth, 1000 GFLOP/s of arithmetic throughput, and 5 microseconds of dispatch per modeled operation. These are illustrative inputs, not measurements of this run or promises about NumPy. The model dimensions match the transformer used in Chapter 15. The four switches represent hypothetical packed weights, factorized matrices, grouped operations, and reused keys and values.

Listing 21.1 Analytical inputs: Separate stored parameters from the dimensions of a forward call.

```python
from dataclasses import dataclass, replace

@dataclass
class Machine:
    bandwidth_gbs: float = 200.0  # assumed bandwidth, GB/s
    peak_gflops: float = 1000.0   # assumed throughput, GFLOP/s
    launch_us: float = 5.0        # fixed cost per operation

@dataclass
class Config:
    D: int = 128                # embedding width
    L: int = 4                  # transformer blocks
    V: int = 1000               # vocabulary
    bytes_per_weight: int = 4   # 4 for FP32, 1 for INT8
    mlp_rank: int = 0           # 0: dense; r: rank r
    fused: bool = False         # fold elementwise chains
    kv_cache: bool = False      # cache keys and values

def mlp_params(cfg):
    if cfg.mlp_rank == 0:
        return 8 * cfg.D * cfg.D  # D x 4D expand, 4D x D contract
    return 10 * cfg.mlp_rank * cfg.D  # 2 factor pairs, 2*r*(D+4D)

def matmul_params(cfg):  # decode step matrix weights
    return cfg.L * (4 * cfg.D * cfg.D + mlp_params(cfg)) + cfg.V *
    ↪   cfg.D

def stored_params(cfg):  # stored matrix and embedding parameters
    return matmul_params(cfg) + cfg.V * cfg.D
```

`mlp_params` applies Chapter 17's factorization count to both MLP matrices. At rank $D/4$, their combined count falls from $8D^2$ to $2.5D^2$. `stored_params` adds the whole token embedding table because every row occupies storage. Traffic is different. A forward over `rows` token positions selects only `rows * D` embedding entries, while the dense projections read their weight matrices. The estimate below charges each matrix once per forward, assuming its weights are reused across the rows in that call. Biases, normalization parameters, and position vectors are outside this matrix subtotal; table 15.1 gives the fuller stored count.

21.3.3 The cost of one decode step

One iteration of generation predicts the next token from a prefix. Its cost changes as the prefix grows, so total time is a sum over iterations, not one measured step multiplied by the response length. Without a cache, a prefix of length `n` sends `n` rows through each projection. With a cache, only the new row is projected, although attention still reads `n` keys and values.

The model accounts for matrix weights, selected embedding rows, linear-layer input and output arrays, and cache reads and writes. Its byte ledger omits attention-score and elementwise intermediates, allocator behavior, and hardware cache effects. It is a partial logical-transfer estimate for comparing these mechanisms, not a traffic counter or a prediction interval. The arithmetic similarly counts matrix products and attention, rather than every scalar operation.

Dispatch needs another assumption. The operation grouping in table 21.1 supplies illustrative unfused and fused counts. These are hypothetical execution groups, not a count of NumPy calls made by `GPT`; Module 17 does not fuse an entire transformer. Factorizing each MLP matrix introduces an additional product, which the code charges as two extra operations per block.

Table 21.1: **Assumed operation groups**: A hypothetical grouping of dense transformer work before and after fusion. These counts configure the analytical model; they are not a measured TinyTorch execution trace.

Piece of one block	Separate operations, unfused	After fusion
Two LayerNorms (mean, subtract, square, mean, add ϵ, square root, divide, scale, shift, each)	18	2
Attention (three projections with biases, scores, scale, mask, softmax as five steps, weighting, output projection with bias)	17	5
MLP (expand with bias, GELU as eight elementwise steps, contract with bias)	12	2
Two residual adds	2	0
Per block	**49**	**9**
Outside the blocks (embedding lookup, position add, final LayerNorm, bias-free head)	12	3

The KV cache is the one lever that does not simply scale a term, so it gets a branch of its own. Without the cache, the step after t tokens re-projects all $t + 1$ tokens through every layer and recomputes attention over all pairs, so F grows linearly with t in the projections and quadratically in attention. With it, the step projects one token and reads the cached keys and values for the other t, so F stays flat apart from a small linear attention term while Q picks up a linear term for the cache reads. That branch is the only place the step's position t enters the code.

Listing 21.2 A partial cost ledger: Count selected rows, linear arrays, and cache transfers before estimating step time.

```python
OPS_PER_BLOCK = {False: 49, True: 9}    # unfused, fused
OPS_OUTSIDE   = {False: 12, True: 3}    # embedding, LayerNorm, head

def step_ms(cfg, m, t):  # napkin time for token after t tokens
    n = t + 1
    if cfg.kv_cache:
        rows = 1
        attn_flops = 4 * n * cfg.D * cfg.L
        kv_bytes = 8 * n * cfg.D * cfg.L    # read cached K and V (FP32)
    else:
        rows = n
        attn_flops = 4 * n * n * cfg.D * cfg.L
        kv_bytes = 0
    flops = 2 * matmul_params(cfg) * rows + attn_flops
    weight_bytes = matmul_params(cfg) * cfg.bytes_per_weight
    embedding_bytes = rows * cfg.D * cfg.bytes_per_weight
    linear_entries = cfg.L * (18 * cfg.D + 4 * cfg.mlp_rank)
    linear_entries += cfg.D + cfg.V  # output head
    activation_bytes = 4 * rows * linear_entries
    cache_writes = 8 * cfg.D * cfg.L if cfg.kv_cache else 0
    nbytes = (weight_bytes + embedding_bytes + activation_bytes
              + kv_bytes + cache_writes)
    ops = cfg.L * OPS_PER_BLOCK[cfg.fused] + OPS_OUTSIDE[cfg.fused]
    if cfg.mlp_rank:
        ops += 2 * cfg.L  # two extra MLP factor products

    memory_ms = nbytes / (m.bandwidth_gbs * 1e9) * 1e3
    compute_ms = flops / (m.peak_gflops * 1e9) * 1e3
    launch_ms = ops * m.launch_us / 1e3
    return max(memory_ms, compute_ms) + launch_ms

def generation_ms(cfg, m, n_tokens):
    return sum(step_ms(cfg, m, t) for t in range(n_tokens))
```

The cache branch removes a factor of n from projection arithmetic and a factor of n from attention arithmetic. It also reduces embedding and linear activation traffic to one row. In exchange it adds $8nDL$ bytes of cache reads and $8DL$ bytes of cache writes to Q. That is the trade Chapter 19 made, arithmetic for bandwidth, and it only pays once n is large enough that the recomputation was the dominant term. `generation_ms` sums the steps, so a generation of 128 tokens is 128 calls with t running from 0 to 127. A profiler on a real serving engine reports the same three terms under the names memory-bound, compute-bound, and launch-bound, and the bridge comes back to that vocabulary.

21.3.4 The levers and the waterfall

Performance engineering teams at scale track end-to-end acceleration through optimization waterfalls that attribute runtime gains to individual architectural levers. By contrast-

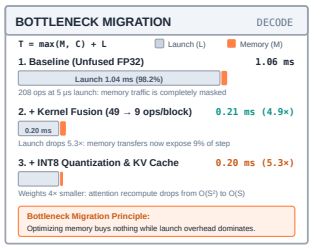
ing isolated component speedups with cumulative stacked deployments, an optimization waterfall exposes the precise point where hardware bottlenecks shift and returns begin to diminish. The waterfall places solo and sequential comparisons beside each other. A solo ratio isolates a transformation against the original baseline. A marginal ratio compares with the immediately preceding configuration. Keeping both columns prevents a change in the denominator from being mistaken for a change in the transformation.

Here is the version that survives. Each lever is a one-line edit to the configuration, so a stack is just the levers applied in sequence to one `Config`. INT8 sets `bytes_per_weight` to 1, which divides the weight part of Q by four and leaves F alone. SVD sets `mlp_rank` to $D/4$, replacing the MLP's $8D^2$ weights with $2.5D^2$. Its matrix arithmetic falls with that count, while the extra factor outputs and products add activation traffic and dispatch. Fusion flips the operation count from the left column of table 21.1 to the right. The KV cache flips the branch in `step_ms`. `waterfall` then prints all three views at once, each lever alone, the product of the solo numbers, and the cumulative total as the levers stack in a stated order, so that the product and the stack disagree on the same page.

Listing 21.3 Marginal and cumulative ratios: Apply configuration edits in an explicit order without changing the baseline.

```
LEVERS = {
    "INT8":     lambda c: replace(c, bytes_per_weight=1),
    "SVD":      lambda c: replace(c, mlp_rank=c.D // 4),
    "fusion":   lambda c: replace(c, fused=True),
    "KV cache": lambda c: replace(c, kv_cache=True),
}

def waterfall(cfg, m, n_tokens, order=LEVERS):
    """Each lever alone against the baseline, then stacked in order."""
    base = generation_ms(cfg, m, n_tokens)
    print(f"baseline {base:9.2f} ms")
    product = 1.0
    for name, lever in order.items():
        alone = base / generation_ms(lever(cfg), m, n_tokens)
        product *= alone
        print(f"  {name:9s} alone {alone:5.2f}x")
    print(f"  product of the solo speedups {product:5.2f}x")
    prev, cur = base, cfg
    for name, lever in order.items():
        cur = lever(cur)
        now = generation_ms(cur, m, n_tokens)
        print(f"  +{name:8s} {now:9.2f} ms  marginal {prev / now:5.2f}x"
              f"  cumulative {base / now:5.2f}x")
        prev = now
```



Decode Bottleneck Migration: In an unfused baseline, kernel launch overhead accounts for 98 percent of the first step's time and 89 percent of the 128-token total. Reducing memory traffic via INT8 buys virtually nothing until kernel fusion eliminates 81 percent of dispatches.

Two properties make the comparison reproducible. `replace` returns a new configuration and leaves the old one alone, so a lever is a pure function and a stack is function composition; nothing about applying INT8 depends on whether fusion came first, except the number that comes out. `order` is an argument rather than a constant so that the

trace can run the same four levers forward and reversed, and show the cumulative total agreeing while every marginal factor moves. The table records a cumulative comparison: each row retains the preceding changes and adds one more. Stating the order makes each marginal ratio interpretable.[88]

[88] **Non-commutative optimization stacking order**: In real production compilers (such as TensorRT and PyTorch Inductor), optimization passes are non-commutative: the sequence in which optimizations are applied alters both numerical accuracy and execution speed. Quantizing weights to INT8 prior to structured pruning restricts magnitude scoring resolution to 256 discrete bins, increasing channel ranking error. Conversely, performing SVD low-rank factorization prior to quantization introduces intermediate activations that may require distinct calibration dynamic ranges. The canonical lowering pipeline applies algebraic fusion and pruning first, followed by quantization calibration, and locks memory buffers via CUDA Graphs last.

21.4 A Worked Trace

The first trace follows four labeled inputs into a submission. The second checks the analytical model's arithmetic. The last tests a stateful transformation on a fixed token sequence. They establish different claims, so their results stay separate.

21.4.1 Four predictions become a submission

Set both matrices of a two-input, two-hidden-unit, two-output `SimpleMLP` to the identity and both biases to zero. The four input rows are positive, so ReLU leaves them unchanged and both linear layers preserve their values. The scores are `[[2, 1], [1, 2], [3, 1], [1, 3]]`; their class indices are `[0, 1, 0, 1]`, exactly the labels. Each layer owns four weights and two biases, so the model stores twelve float32 entries, or 48 bytes.

The candidate is a deep copy. Zeroing its second output weight changes the scores to `[[2, 0], [1, 0], [3, 0], [1, 0]]`. It now predicts `[0, 0, 0, 0]` and gets two of four labels right. The array shapes and byte counts are unchanged. In listing 21.4 below, lines 9 to 14 set up the four-sample input `X`, labels `y`, and the identity baseline model. Lines 15 and 16 construct the candidate model with one weight zeroed (`candidate.fc2.weight.data[1, 1] = 0`). Lines 18 to 21 run five benchmark iterations on both models. Lines 22 and 23 combine the two reports into a submission via `generate_submission`. The assertions in lines 24 to 29 verify that baseline accuracy is 1.0, candidate accuracy dropped to 0.5, parameter count remains 12, and accuracy delta is −0.5.

Listing 21.4 Four predictions and a report: A weight change preserves array storage but lowers accuracy.

```
import copy
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.olympics import (
    SimpleMLP, BenchmarkReport, generate_submission,
    validate_submission_schema, qualifies_event, OlympicEvent,
)

X = Tensor([[2, 1], [1, 2], [3, 1], [1, 3]])
y = np.array([0, 1, 0, 1], dtype=np.int64)
model = SimpleMLP(input_size=2, hidden_size=2, output_size=2)
for layer in (model.fc1, model.fc2):
    layer.weight.data[:] = np.eye(2)
    layer.bias.data[:] = 0
candidate = copy.deepcopy(model)
candidate.fc2.weight.data[1, 1] = 0

baseline = BenchmarkReport("identity_mlp")
optimized = BenchmarkReport("zeroed_output_weight")
baseline.benchmark_model(model, X, y, num_runs=5)
optimized.benchmark_model(candidate, X, y, num_runs=5)
submission = generate_submission(baseline, optimized,

    techniques_applied=["zero_one_weight"])
assert baseline.metrics["accuracy"] == 1.0
assert optimized.metrics["accuracy"] == 0.5
assert baseline.metrics["parameter_count"] == 12
assert baseline.metrics["model_size_mb"] * 1e6 == 48
assert submission["improvements"]["compression_ratio"] == 1.0
assert submission["improvements"]["accuracy_delta"] == -0.5
assert validate_submission_schema(submission)
assert not qualifies_event(optimized.metrics,
    OlympicEvent.LATENCY_SPRINT)
```

In lines 20 and 21, `benchmark_model` receives `(4, 2)` scores, reduces the class axis with `argmax`, and counts label agreements over four rows. Its returned metrics describe each model separately. In lines 22 and 23, `generate_submission` records compression ratio 1.0 and accuracy delta -0.5, while the speedup uses the two measured median latencies. Five repeats keep this trace short; they do not establish a stable performance result.

The final two assertions in lines 30 and 31 answer different questions. In line 30, the file passes validation because its schema fields and derived comparisons agree. In line 31, the candidate fails the latency event because its accuracy is below 0.85, regardless of its speed. A correct reporting pipeline can reject an optimization; that is the intended result here. After deciding which candidate to retain, `save_submission(submission, path)` records the already validated dictionary.

21.4.2 The analytical baseline, by hand

Take $D = 128$, $L = 4$, and $V = 1000$. The subtotal contains 914,432 matrix weights and 128,000 token-embedding entries. Those are stored counts. At the first step, `rows = 1`, so the transfer ledger charges $914{,}432 \times 4 = 3{,}657{,}728$ matrix bytes, 512 selected embedding bytes, and 41,376 linear input/output bytes. Its total is 3,699,616 bytes, or 0.0185 milliseconds at the assumed 200 GB/s. Matrix and attention arithmetic contribute 1,830,912 operations, or 0.00183 milliseconds at the assumed 1000 GFLOP/s. The maximum is the byte term. The $4 \times 49 + 12 = 208$ assumed operation groups add 1.040 milliseconds of dispatch, giving `step_ms(Config(), Machine(), 0) = 1.0585` milliseconds.

At the final step, the uncached prefix has 128 rows. The same matrices serve those rows, but selected embeddings contribute 65,536 bytes and linear inputs/outputs contribute 5,296,128 bytes. The total transfer ledger is 9,019,392 bytes, or 0.0451 milliseconds. Arithmetic grows to 267,649,024 operations, or 0.2676 milliseconds, and becomes the larger term. The step estimate is 1.3076 milliseconds. Summing the 128 different steps gives 149.76 milliseconds. The changing row count is why multiplying the first step by 128 would miss part of the cost.

Pause & Predict: Amdahl's Law and the Diminishing Returns of Stacking

Suppose in a baseline generative decode step taking 10.0 ms, attention projections consume 6.0 ms (60% of total latency) and MLP layers consume 4.0 ms (40%). If an optimal KV cache speeds up attention projections by $10\times$ (slashing attention latency to 0.6 ms), what is the new total step latency? What is the overall model speedup? Why did a $10\times$ component speedup produce only a $2.17\times$ end-to-end speedup?

21.4.3 The waterfall

Running `waterfall(Config(), Machine(), 128)` produces table 21.2. Every value is an analytical estimate from the displayed assumptions, not a measured TinyTorch speedup.

Table 21.2: **Estimated composition costs**: Four transformations applied to the analytical model in the stated order. The product of solo speedups is 4.00x; the full-stack estimate is 4.89x.

Stage	Total for 128 tokens	Marginal speedup	Cumulative speedup	Lever alone vs. baseline
Baseline FP32, dense, unfused, no cache	149.76 ms		1.00x	
+ INT8 weights	149.67 ms	1.00x	1.00x	1.00x
+ MLP factorized at rank 32	148.84 ms	1.01x	1.01x	1.01x
+ fusion	40.68 ms	3.66x	3.68x	3.60x
+ KV cache	30.63 ms	1.33x	4.89x	1.10x

Most of the initial estimate is the assumed dispatch term. Reducing weight bytes alone barely changes it: when an initial baseline is severely dispatch-bound ($n_{\text{ops}} \cdot c \gg$

compute, memory), reducing memory traffic via INT8 quantization yields an apparent $1.00\times$ speedup, buying at most $1.03\times$ in this model. Fusion lowers that term, which makes the remaining row-dependent work more consequential and lets KV caching deliver a larger marginal gain. Caching then removes repeated projections and reduces the linear activation traffic to one row, while adding context-dependent reads of stored keys and values.

The order exposes a vital systems cost that a static parameter count would miss. Reversing `LEVERS` yields marginal factors of 1.10x for caching, 4.93x for fusion, 0.87x for factorization, and 1.03x for INT8, arriving at the identical final 30.63 milliseconds. Why does low-rank SVD factorization become an active slowdown (0.87x) at that stage? Because factorizing a weight matrix $\mathbf{W} \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ into rank-r factors $\mathbf{AB}$ splits one single matrix multiplication ($\mathbf{xW}$) into two sequential matrix multiplications ($(\mathbf{xA})\mathbf{B}$). In autoregressive token generation where the input is a single token ($B = 1, S = 1$), the arithmetic work per layer is tiny. Splitting each MLP matrix adds two operations per block (39 to 47 per step after fusion) and an intermediate rank-r activation. When dispatch latency outweighs arithmetic time, replacing one larger matrix multiplication with two smaller ones degrades wall-clock latency despite reducing total theoretical FLOPs. Because the configuration edits commute, the final configuration and total agree in both orders, as summarized in the Napkin Math scorecard below. A real pair of transformations may change one another's weights or supported execution path, so this equality is a property of the model, not a general guarantee.



Stacking Order and Dispatch Penalty: Multiplying solo speedups predicts 4.00x, while sequential stacking reaches 4.89x. In reverse order, SVD factorizing causes an active 0.87x slowdown because splitting the MLP matrices adds two dispatches per block (39 to 47 per fused step) on single tokens.

Track the compounding gains and theoretical limits of stacking all four optimization levers on TinyGPT ($L = 4, D = 128, V = 1000$ for $S = 128$ tokens):

- **Baseline (FP32, Dense, Unfused, Uncached)**:
  - Memory footprint: 4.17 MB of matrix and embedding parameters (1,042,432 entries).
  - Latency ledger: 149.8 ms ($O(S^2)$ projections + 208 kernel launches/step).
- **+ INT8 Quantization**:
  - Parameter memory: 4.17 MB $\rightarrow$ 1.04 MB ($4.0\times$ compression).
  - Execution speedup: Marginal $1.00\times$ (memory traffic drops, but dispatch dominates).
- **+ Low-Rank SVD Factorization (rank 32)**:
  - Parameter memory: 1.04 MB $\rightarrow$ 0.68 MB ($6.1\times$ total compression).
  - Matrix arithmetic: Reduces MLP FLOPs by 68%, marginal speedup $1.01\times$.
- **+ Kernel Fusion (pointwise + LayerNorm)**:
  - Launch overhead: 49 ops/block $\rightarrow$ 9 ops/block ($5.4\times$ launch reduction).
  - Latency: Drops from 148.8 ms $\rightarrow$ 40.7 ms (marginal $3.66\times$ speedup).
- **+ KV Caching**:
  - Projection scaling: $O(S^2) \rightarrow O(S)$ token evaluation.
  - Latency: Drops from 40.7 ms $\rightarrow$ 30.6 ms (marginal $1.33\times$ speedup).
- **Compounded Total**:
  - Combined speedup: 149.8 ms$/30.6$ ms $= 4.89\times$.
  - Footprint reduction: 4.17 MB $\rightarrow$ 0.68 MB ($6.1\times$ reduction).

Notice the discrepancy: multiplying solo speedups predicts $1.00\times1.01\times3.60\times1.10 = 4.00\times$, whereas sequential compounding achieves $4.89\times$ because fusion uncovers the

row-dependent arithmetic and activation traffic that the KV cache removes, savings that launch latency previously masked.

21.4.4 Measuring one lever

A cache changes execution order and state, so correctness comes before timing. The verification uses a fixed sequence of token IDs and compares each cached output with the last row of an uncached forward over the same prefix. This is a teacher-forced workload: the next input is fixed rather than sampled from the logits. Both paths therefore perform the same task even if small floating-point differences would lead their samplers to choose different tokens.

Listing 21.5 Two paths for the same prefixes: Define the uncached and cached request paths and their timing helper.

```
import time
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.core.transformers import GPT
from tinytorch.perf.benchmarking import BenchmarkResult
from tinytorch.perf.memoization import enable_kv_cache,
    disable_kv_cache

model = GPT(vocab_size=1000, embed_dim=128, num_layers=4,
            num_heads=4, max_seq_len=256)
tokens = np.random.default_rng(7).integers(0, 1000, size=(1, 64))

def uncached_prefixes():
    rows = []
    for stop in range(1, tokens.shape[1] + 1):
        logits = model.forward(Tensor(tokens[:, :stop]))
        rows.append(logits.data[:, -1:, :])
    return np.concatenate(rows, axis=1)

def cached_prefixes(cache):
    cache.reset()
    rows = []
    with cache.generation():
        for pos in range(tokens.shape[1]):
            logits = model.forward(Tensor(tokens[:, pos:pos + 1]),
                                   start_pos=cache.seq_pos)
            cache.advance()
            rows.append(logits.data)
    return np.concatenate(rows, axis=1)

def timed(fn, warmup=3, repeats=15):
    for _ in range(warmup):
        fn()
    durations = []
    for _ in range(repeats):
        start = time.perf_counter()
        fn()
        durations.append((time.perf_counter() - start) * 1000)
    return BenchmarkResult("fixed_prefixes_ms", durations)
```

The driver below checks both request paths before comparing their timings.

Listing 21.6 Check before timing: Reuse the prefix functions to verify equivalent outputs and isolate cache state before measuring the two paths.

```
reference = uncached_prefixes()
cache = enable_kv_cache(model)
try:
    candidate = cached_prefixes(cache)
    np.testing.assert_allclose(candidate, reference, rtol=1e-4,
    atol=1e-5)
    assert cache.seq_pos == tokens.shape[1]
    # A second request must start from an empty cache and reproduce
      the first.
    np.testing.assert_allclose(cached_prefixes(cache), candidate,
                               rtol=1e-4, atol=1e-5)
    baseline = timed(uncached_prefixes)
    optimized = timed(lambda: cached_prefixes(cache))
    print({"speedup_p50": baseline.median / optimized.median,
           "max_logit_error": float(np.max(np.abs(candidate -
             reference))),
           "cache_mb": cache.get_memory_usage()["total_mb"],
           "tokens": tokens.shape[1], "runs": optimized.count})
finally:
    disable_kv_cache(model)
```

The `cache.generation()` scope activates cached attention only for the loop and restores the previous mode on exit. The baseline runs outside that scope, so its forwards use ordinary full-prefix attention even while the wrappers remain attached. It does not reset or advance the position; those calls belong to the request driver. The outer `try/finally` removes the cache wrappers even if an assertion fails. The reset check guards the boundary between requests; the position-by-position comparison guards the boundary between layers and tokens. Neither check can be replaced by asserting that generation returned the requested number of token IDs.

The report deliberately names `max_logit_error` rather than filling an `accuracy` field with 1.0. Agreement with the reference tests the cache transformation, not whether this untrained model predicts useful text. A task-quality report needs held-out targets and a specified metric. `generate_submission` expects classification accuracy, so this generation experiment reports its own metrics rather than manufacturing an accuracy value to fit that schema.

Timing begins only after the equivalence checks pass. Its ratio describes 64 fixed prefixes, including the same output collection on both paths; it is neither per-token service latency nor sampled generation throughput. The cache reserves 1,048,576 bytes for this model and still allocates temporary prefixes, as Chapter 19 showed. A complete footprint report should separate these reserved buffers from parameter storage and transient allocations. Repeat the experiment across lengths before extrapolating the speedup to a longer response. Fifteen runs provide an initial median comparison, not a reliable service-tail estimate.

21.5 From TinyTorch to Production

A production comparison must establish the combined execution path and its costs. The order in which the levers pay off depends on the model, the batch size, the sequence length, and the chip, so serving systems measure the full stack under the traffic they expect and tune from there. The cost model in this chapter is what an engineer sketches before that measurement to decide which lever to try first; it is not a substitute for the measurement.

A production runtime can change the execution path behind the same mathematical model. Fusion groups operations so intermediate values need not pass through separate arrays. Graph replay records a fixed dependency structure and reuses it, reducing repeated work in dispatch.[89] Packed weight formats can reduce stored bytes and transfers, but dequantization and kernel support determine the resulting time. None of those benefits follows merely from changing `bytes_per_weight` or `fused` in the analytical model. The executable candidate and its workload must establish the gain.

Moving from classroom Python evaluation to industrial production deployment requires automated optimization graphs and specialized inference engines:

- **Compilation Graphs (TensorRT/OpenVINO)**: Convert PyTorch models into static computation graphs. TensorRT performs vertical fusion (Conv + Bias + ReLU), horizontal fusion (combining parallel Multi-Head Attention projections), and INT8 calibration with hardware Tensor Core kernels.
- **Inference Serving (Triton Inference Server)**: Coordinates multi-model orchestration, dynamic batching (accumulating concurrent incoming requests into mini-batches on the fly), and concurrent model execution across multiple GPUs.
- **CUDA Graphs Integration**: Captures the entire forward execution sequence into an immutable graph launch, reducing per-token CPU-to-GPU dispatch overhead from 10 to 20 μs down to sub-microsecond driver triggers.
- **Deployment CI Quality Gates**: Enforces strict regression suites: a candidate container must verify bitwise equivalence, bounded perplexity delta ($\Delta\text{PPL} \leq 0.1$), and strict P99 latency SLAs before traffic routing.

Production serving stacks automate this optimization progression (see Systems Perspective above). What carries over unchanged is the arithmetic of stacking. Time is a sum of terms, each lever shrinks one, and the stack's speedup is what the sum says, not the product of solo measurements. Production engineers speak of a workload being launch-bound, memory-bound, or compute-bound, and each of those words names which term is largest at the moment and therefore which lever is worth applying next. That vocabulary is this chapter's cost model with the numbers filled in by a profiler.

The model suggests a transformation by identifying the largest estimated term. After implementing that transformation, the profiler and correctness checks determine whether the expected work disappeared and whether another cost became dominant. The final report records the actual stack, its order, input dimensions, timing statistic, and quality result. Those conditions let a reader distinguish a different workload from a failed reproduction.

---

[89] **CUDA Graphs and dispatch elimination**: Standard PyTorch execution submits individual CUDA kernel launches over PCIe driver queues, incurring 5 to 15 μs CPU overhead per operation. CUDA Graphs record an entire sequence of kernel launches, memory copies, and stream dependencies into a single executable graph. Subsequent executions launch the entire graph with a single CPU instruction, slashing launch overhead to $< 1\,\mu$s. Because graph nodes reference fixed device memory addresses and grid dimensions, dynamic input shapes require bucketed padding or multiple pre-captured graphs.

21.6 Summary

Module 20 turns model execution into a comparison that another reader can check. It validates class scores and labels, measures accuracy, single-sample latency, batch throughput, and stored parameter bytes, then places the resulting values in a report. A submission compares two reports using the same latency statistic, a storage ratio, and an accuracy delta. Validation checks the record's consistency; event eligibility applies a separate rule. The four-row trace passes the first check and fails the second because the candidate loses half its correct predictions.

The analytical model explains why an optimization's marginal gain depends on the work that remains. Its byte ledger distinguishes stored embedding capacity from selected rows and distinguishes uncached prefix rows from a cached single row. The cache experiment then tests real outputs and request state before timing either path. Synthesis III carries this discipline to a trained model and a held-out workload. The artifact to preserve is the baseline, candidate, workload, and evidence together; a speedup detached from those cannot explain what improved.

With the optimization and benchmarking modules complete (Milestone 06 requires Modules 01 to 04, 06, 07, 09, and 11 to 19), the practitioner can enter the final systems arena:

```
tito milestone run 06
```

This milestone deploys the complete systems optimization stack (profiling bottleneck diagnosis, simulated INT8 quantization, weight pruning, and Key-Value cache generation speedup) against strict competitive scoring gates. Turn to Synthesis III to see how the milestone computes each model division's Pareto frontier from its measurements and marks a candidate Pareto only when that frontier contains it.

21.7 Further Reading

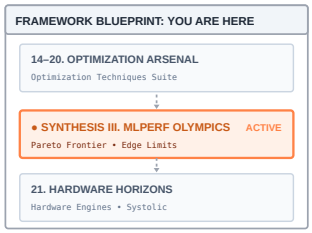
- **Reddi et al.** (2020). "MLPerf inference benchmark." *ISCA*. Accuracy floors that an optimized submission must still meet, the rule this chapter's events apply.
- **Banbury et al.** (2021). "MLPerf Tiny benchmark." *NeurIPS*. Emphasizes how constraints at the edge demand strict optimization discipline, tying perfectly into this capstone's multi-objective evaluation.
- **Sculley et al.** (2015). "Hidden technical debt in machine learning systems." *NeurIPS*. The costs a deployed system carries beyond its model.

🔥 Chapter 22

Synthesis III: The MLPerf Optimization Olympics

22.0.1 The History and the Problem

Following the Transformer breakthrough, model sizes exploded. From GPT-1 to GPT-3, parameter counts grew by orders of magnitude, vastly outpacing the growth of hardware memory and compute capabilities. This created a critical deployment crisis: models became so massive and computationally hungry that they were too slow for real-time inference and too large to fit on standard edge devices or even single data center GPUs.



Framework Blueprint: Synthesis III measures optimized candidates against one baseline on accuracy, storage, and latency, and computes which of them lie on the Pareto frontier.

22.0.2 The Monumental Breakthrough

What needed to get unlocked was a way to compress these models and accelerate their execution without destroying the intelligence they had learned. The monumental breakthroughs came from systems engineering: techniques like INT8 quantization shrank memory footprints by 4x, and KV-caching eliminated redundant matrix multiplications during autoregressive generation. In 2018, MLPerf was established to provide standardized workloads, quality targets, and measurement rules so that such optimizations could be compared fairly.

22.0.3 What Was Solved

This combination of algorithmic compression and hardware-aware execution solved the deployment bottleneck. Optimization became a rigorous science governed by a multi-objective Pareto scorecard: models could now run fast and fit into constrained memory, provided they cleared strict accuracy retention floors.

In competitive systems benchmarks like MLPerf, raw speedup numbers and compression ratios mean nothing if the candidate model compromises accuracy. A system optimization is only valid if it satisfies strict accuracy retention floors while delivering verifiable reductions in memory footprint and latency under concrete edge hardware constraints.

In production engineering, deployment qualification is governed by a multi-objective Pareto scorecard:

```
import torch

# Multi-objective Pareto evaluation scorecard
def evaluate_candidate(baseline_acc, candidate_acc, speedup,
    compression_ratio):
    accuracy_retention = candidate_acc / baseline_acc
```

```
    passes_floor = accuracy_retention >= 0.99  # Relative retention
    floor: keep 99% of baseline
    return {
        "retention": accuracy_retention,
        "speedup": speedup,
        "compression": compression_ratio,
        "qualified": passes_floor
    }
```

Underneath that evaluation gate, the systems engineer audits candidate models across three competing axes:

1. **Strict accuracy qualification floors:** Optimization techniques that introduce quantization noise or prune weights must retain at least 99% of baseline accuracy ($R \geq 0.99$); failing to clear this threshold disqualifies the candidate immediately regardless of speed.
2. **Physical memory budget enforcement:** Quantized weights and runtime KV-cache allocations are audited against target device memory limits to guarantee that models fit without swapping to disk or crashing with out-of-memory errors.
3. **Reproducible speedup certification:** Candidates are benchmarked under identical warm-up, iteration, and timer controls, mapping Pareto-optimal trade-offs across storage, latency, and quality.

This synthesis chapter concludes the TinyTorch systems journey with the **MLPerf Optimization Olympics** (Milestone 06 in the repository), challenging practitioners to deploy optimized models against strict multi-objective scoring gates, and closing with a measured KV-cache replay of TinyGPT whose logits must match uncached replay before either path is timed (on small workloads, the cached path can be slower). Before auditing submissions, common traps that invalidate optimization claims must be inspected.

Synthesis Scope: Synthesis III integrates Modules 14 through 20 (`tinytorch/src/14_profiling` through `tinytorch/src/20_capstone`) with the models of Part I and Part II; `tito` requires Modules 01 to 04, 06, 07, 09, and 11 to 19 to run the milestone. In the repository, this corresponds to **Milestone 06** (`milestones/06_2018_mlperf/`, runnable via `tito milestone run mlperf` or `tito milestone run 06`). It provides two canonical scripts: `01_optimization_olympics.py` evaluates `DigitMLP`, `SimpleCNN`, and a small GPT across profiling, INT8 quantization, pruning, and KV caching, computes each division's Pareto frontier from the measurements, and marks a candidate Pareto only when that frontier contains it, while `02_generation_speedup.py` checks logit equivalence of cached and uncached replay on a small `GPT` before timing both paths. The listings in this chapter provide a pure-Python model of that same MLPerf scorecard, small enough to check by hand.

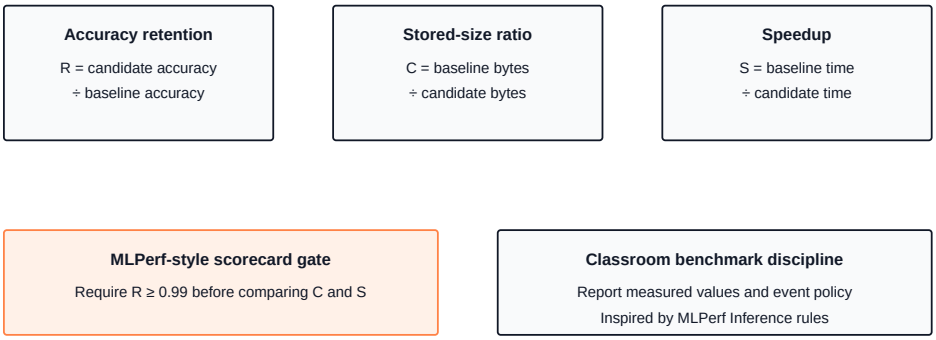


Figure 22.1: The MLPerf Optimization Olympics scorecard: one model, three axes, and a floor on accuracy that a submission must clear before its memory and latency numbers count.

22.1 The Problem

A benchmark that measures one thing rewards whatever is cheapest on that one thing. A model can appear fast simply because it ignores its input.

↩ **Jump back:** Chapter 16 *Uniform INT8 affine mapping, scale calibration, and clipping noise trade-offs.*
↩ **Jump back:** Chapter 17 *Structured pruning, SVD rank truncation, and distillation loss surfaces.*
↩ **Jump back:** Chapter 21 *Evaluating multi-objective trade-offs across accuracy retention, memory compression, and speedup.*

```python
class ZeroModel:
    """Answers every question with the same ten zeros."""
    def forward(self, x):
        return [0.0] * 10

def time_ms(model, x, runs=100):
    import time
    best = float("inf")
    for _ in range(runs):
        t0 = time.perf_counter()
        model.forward(x)
        best = min(best, (time.perf_counter() - t0) * 1000)
    return best

print("ZeroModel latency (ms):", round(time_ms(ZeroModel(), [0.0] *
    64), 5))
```

This model performs no learned computation. An `argmax` classifier breaks its ten-way tie by choosing class zero, giving 10% accuracy on ten balanced classes. Its parameter storage is zero bytes. The timing depends on the machine, but the failure is visible without a stopwatch: a latency contest alone has no reason to prefer a useful prediction. Quantization and pruning expose a less extreme version of the same trade-off. They alter weights, so their storage claims must be accompanied by a check of the resulting predictions. A hypothetical packed 2-bit representation needs one sixteenth of the FP32 weight-code bytes before metadata, but that count says nothing about its accuracy.

The opposite failure is quieter. Chapter 17 showed that zeroing half the entries of a weight matrix leaves the dense multiply doing exactly the same work, since a zero still gets multiplied and added. A student who prunes to 50% sparsity and reports "half the parameters removed" has said something true about the count and nothing about the bytes on disk or the milliseconds per call. Whether that pruning paid off depends on how the weights are stored and how the multiply is run, and neither is visible in the sparsity figure.

So the need is concrete. Take one model, apply the optimizations, and report accuracy, bytes, and latency together, against the unoptimized model, with a rule that disqualifies any candidate whose accuracy falls too far. That rule is the difference between a benchmark and a leaderboard for `ZeroModel`.

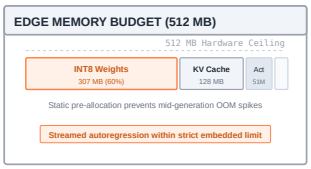
22.2 The Idea

22.2.1 Three ratios and a floor

↪ **Jump ahead:** Chapter 23 *Taking the framework below NumPy with compiled C++ and GPU kernels, measured the same way.*

Everything in the scorecard is a ratio of the optimized model against the baseline, so the units cancel. The ratios still depend on the workload, implementation, and machine; cancelling units does not make measurements portable. Let A be accuracy on a fixed test set, M the bytes needed to store the parameters, and t the median latency of one forward pass after warmup, with subscripts for the baseline and the candidate. Three numbers summarize a submission:

$$R = \frac{A_{\text{cand}}}{A_{\text{base}}}, \qquad C = \frac{M_{\text{base}}}{M_{\text{cand}}}, \qquad S = \frac{t_{\text{base}}}{t_{\text{cand}}}$$



Edge hardware memory budget: INT8 weights and KV cache fit comfortably within a 512 MB RAM embedded ceiling.

R is **accuracy retention**, the fraction of the baseline's accuracy the candidate keeps. C is the size ratio, how many times smaller the candidate is. S is the speedup. All three are better when larger, but only C and S are things to maximize. R is a gate. This chapter chooses a relative retention floor of 99%, so its rule reads $R \geq 0.99$. Against an 88%-accurate baseline, `ZeroModel` retains only $0.10/0.88 \approx 0.114$ and fails. The score function assumes a baseline with nonzero accuracy and positive storage and timing values. Those conditions make its divisions meaningful.

22.2.2 Events

The capstone module defines classroom events through `OlympicEvent` and checks eligibility with `qualifies_event`. A latency sprint requires accuracy of at least 85 percent; a memory challenge has the same floor. An accuracy contest requires latency strictly below 100 ms and stored size strictly below 10 in the report's `model_size_mb` units, each equal to 10^6 bytes. An "extreme push" lowers the accuracy floor to 80 percent. The all-around event accepts any valid measurements without adding an eligibility floor. These rules determine eligibility; the helper neither ranks candidates nor invents a combined all-around score. The floors are absolute here, rather than relative to a baseline, and they are classroom policy rather than official MLPerf requirements. Milestone 06 applies a third, separate set of pass gates that test your modules rather than rank candidates: the baseline must reach 80 percent test accuracy, and the INT8 candidate must stay within 3 percentage points of it.

The set of candidates that cannot improve on one axis without giving something up on another is called the **Pareto frontier**.[90] A benchmark with floors is a way of asking one question about that frontier at a time. Where on the frontier a deployment should sit depends on its quality requirements and resource budget, and the scorecard's job is to make the frontier visible so that the decision can be made with numbers.

[90] **Pareto frontier**: in a trade-off between several objectives, the set of options that no other option beats on every objective at once. A candidate off the frontier is simply worse than some other candidate on all axes; moving along the frontier trades one axis for another, and a benchmark cannot say which trade is right, only where the frontier is.

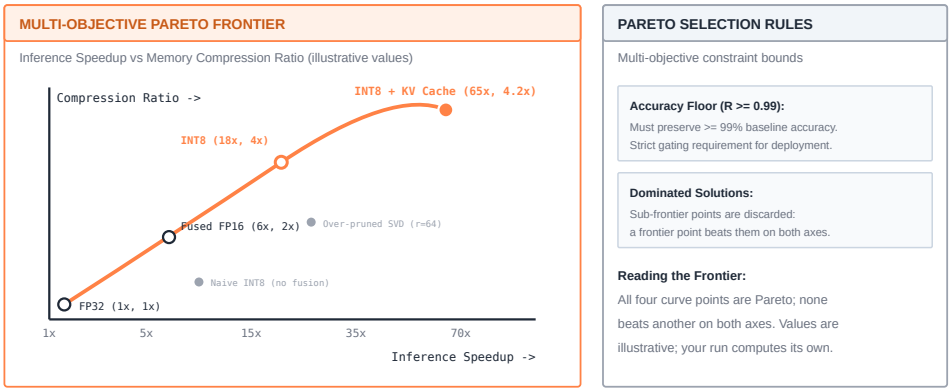


Figure 22.2: **The MLPerf Optimization Olympics Pareto frontier**: Evaluating optimization candidates across accuracy retention, memory compression, and inference speedup reveals the multi-objective trade-off curve. The chapter's relative retention floor ($R \geq 0.99$) disqualifies over-compressed or degraded models before the frontier of non-dominated candidates is drawn. The capstone events apply separate absolute accuracy floors (85%, or 80% for the extreme push). The plotted values are illustrative, not one measured run.

Pause & Predict: MLPerf Inference Scenarios and Operational Constraints

Consider evaluating a 7B parameter quantized model across two distinct deployment targets: (1) an interactive voice assistant on a smartphone, and (2) a high-volume batch indexing pipeline in a data center. Which MLPerf scenario (SingleStream vs. Offline) governs each deployment? Why does reporting a single average latency metric fail to characterize either system?

22.3 The Code

The course scripts execute the experiments. The chapter's smaller score function explains how to interpret their results. Keeping these roles separate allows tracing the experiment without mistaking an illustrative ledger for measured evidence.

22.3.1 Script 1: profile, quantize, prune, benchmark

The first script trains `DigitMLP` for forty epochs on TinyDigits, then walks through the optimization steps in order. Each step is a function that takes the model and one of Part III's classes.

Profiling comes first, and it has to. Every ratio on the scorecard compares a candidate against a baseline number, so before altering any weights, the unoptimized model requires

its parameter count, its bytes, its FLOPs, its latency, and its accuracy recorded where subsequent steps cannot overwrite them. The tempting shortcut is to skip the ceremony, wrap one forward pass in `time.time()`, and call that the baseline latency. Chapter 15 showed why that one call is the wrong number. The first call may pay initialization costs, and any single call may encounter scheduling noise. An unusually slow baseline would inflate every later speedup before any optimization had run. The profiler's answer is the one Chapter 15 built: warm up, repeat, and keep the median.

```
# Excerpt from milestones/06_2018_mlperf/01_optimization_olympics.py (step_1_profile, trimmed)
profiler = Profiler()
param_count = profiler.count_parameters(model)
param_bytes = sum(p.data.nbytes for p in model.parameters())
flops = profiler.count_flops(model, (1, 64))

sample_input = Tensor(X_test.data[:1])
latency_ms = profiler.measure_latency(model, sample_input, warmup=3, iterations=10)

outputs = model(X_test)
predictions = np.argmax(outputs.data, axis=1)
baseline_acc = np.mean(predictions == y_test) * 100
```

SOURCE CODE MAPPING SYN-03

FILE: milestones/06_2018_mlperf/
EXPORT: tinytorch.milestones.mlperf
SYMBOLS: optimization_olympics

Extracted & verified at build time ✓ tested

Source Code Mapping: Synthesis III combines Modules 14 to 20 to benchmark the combined impact of INT8 quantization, KV caching, and operator fusion under strict hardware ceilings.

The byte count sums the actual parameter arrays. It excludes Python object overhead, intermediate activations, and any serialized file header, so it should be labeled parameter-array storage. The profiler times one held-out input row; accuracy uses all held-out rows. These are different batch sizes because they answer different questions, but every candidate must use the same timing input shape and the same accuracy dataset as the baseline. The script stops unless the baseline reaches 80 percent test accuracy; a model whose optimizer never moved it would otherwise become the reference every candidate is measured against. It also stops if the profiler's own numbers are wrong: the parameter and FLOP counts must equal those derived from the layer shapes (2,410 parameters, and 4,736 FLOPs at $2 \cdot \text{in} \cdot \text{out}$ per `Linear`), and the latency must be a positive, finite measurement. During training, your `CrossEntropyLoss` on the first batch must match a NumPy computation, because training can converge on a wrong loss value while every reported loss is meaningless. Later steps apply the same discipline to the other tools: `BenchmarkResult` and `pareto_frontier` must reproduce hand-worked fixtures and consistent latency summaries before they score anything, and the GPT must show that its logits vary, that no position reads later tokens, and that earlier tokens matter before it is cached, timed, or quantized.

Quantization comes next. `Quantizer.quantize_model` produces an artifact containing each parameter's codes, scale, zero point, and original shape. It also models a packed representation with one byte per code and eight bytes of metadata per parameter array. This artifact does not execute a forward pass. The script must reconstruct an executable candidate before it can measure how rounding changes predictions.

```
# Excerpt from 01_optimization_olympics.py (step_2_quantize, trimmed)
quant_result = Quantizer.quantize_model(model)
```

```
worst_steps = check_int8_artifact(list(model.parameters()),
    quant_result, Quantizer)
# Counted from the stored code arrays, not param_bytes / reported
    ratio.
quant_size = int8_storage_bytes(quant_result)
measured_ratio = param_bytes / quant_size
```

`check_int8_artifact` stops the milestone unless every parameter became integer codes in $[-128, 127]$ that decode back to the original weights. `int8_storage_bytes` then counts one byte per stored code plus eight bytes of scale and zero point per parameter array, reading the arrays the quantizer returned rather than trusting the compression ratio it reports. `Quantizer.quantize_model` returns codes, scales, and zero points without changing `model`. The script then makes an independent copy of the trained `DigitMLP`, restores each parameter from those codes, and measures its predictions on the same test set. That second half of `step_2_quantize` is essential: rounding has to reach the forward pass before its effect on accuracy can be measured.

```
# Continued excerpt from step_2_quantize
import copy

quant_model = copy.deepcopy(model)
quant_params = [p for layer in quant_model.layers for p in
    layer.parameters()]
for idx, param in enumerate(quant_params):
    entry = quant_result['quantized_layers'][f'param_{idx}']
    restored = Quantizer.dequantize_tensor(
        entry['quantized'], entry['scale'], entry['zero_point'])
    param.data = restored.data.reshape(entry['original_shape'])
outputs_quant = quant_model(X_test)
quant_acc = np.mean(np.argmax(outputs_quant.data, axis=1) == y_test) *
    100
```

Follow one parameter through the loop: its index selects the corresponding quantized entry, its scale and zero point reconstruct rounded weights, and its original shape restores the matrix expected by `Linear`. The baseline remains untouched. The resulting accuracy measures quantization error under floating-point execution; it does not demonstrate an integer kernel or packed storage. The script requires this accuracy to stay within 3 points of the baseline. The byte count models one byte per code plus one float32 scale and one int32 zero point per parameter array; TinyTorch stores the code tensors and the reconstructed weights as float32 arrays.

Pruning changes a different property, the number of nonzero weights. `magnitude_prune` zeroes weights in place, and a zero has no memory of what it replaced. If the script pruned `model` itself, the FP32 baseline would be gone. A saved baseline accuracy would still supply the denominator of R, but the original model could no longer be rerun to check predictions or timing. Recomputing the baseline after pruning would silently compare the candidate with itself. So the script makes an independent deep copy first and prunes that copy. Both candidate copies run on the same held-out inputs.

```
# Excerpt from 01_optimization_olympics.py (step_3_prune, trimmed)
model_copy = copy.deepcopy(model)

sparsity_before = Compressor.measure_sparsity(model_copy)
Compressor.magnitude_prune(model_copy, sparsity=0.5)
sparsity_after = Compressor.measure_sparsity(model_copy)

outputs_pruned = model_copy(X_test)
pruned_acc = np.mean(np.argmax(outputs_pruned.data, axis=1) == y_test)
    * 100
```

`magnitude_prune` gathers every weight matrix (biases excluded), ranks their magnitudes globally, and zeroes the smallest half in place. `measure_sparsity` counts the zeros, the script requires the zero fraction to land within 2 points of the requested 50 percent, and the pruned accuracy is measured on the candidate. Its dense array bytes remain unchanged. A sparse representation would also need indices; the worked trace below models those separately. The script preserves the unmodified baseline for all comparisons.

The benchmark step owns the latency axis, and it asks a different question from the profiler's. The profiler wanted a typical number for the baseline. A latency distribution also reveals occasional slow calls that its median hides. The 95th percentile, for example, marks the value at or below which roughly 95% of recorded calls fall. A single timed call cannot answer either question; it picks one point on the latency distribution at random and reports it as if it were the whole. Chapter 20's `Benchmark` runs the forward pass repeatedly, keeps every timing, and reports a mean, a standard deviation, and a 95th percentile. The script uses `Benchmark`'s defaults, five warmups and ten measurements, and selects a sorted entry rather than using `BenchmarkResult.percentile`, so its P95 convention differs from Chapter 20's interpolated percentile. With ten samples, index `int(10 * 0.95) = 9` is the last sorted entry, so this "P95" is the maximum of the ten.

```
# Excerpt from 01_optimization_olympics.py (step_6_benchmark, trimmed)
benchmark = Benchmark(models=[model], datasets=[(X_test, y_test)])
latency_results = benchmark.run_latency_benchmark(input_shape=(1, 64))
bench_result = list(latency_results.values())[0]

sorted_vals = sorted(bench_result.values)
p95_latency = sorted_vals[min(int(len(sorted_vals) * 0.95),
    len(sorted_vals) - 1)]
```

The excerpt shows one invocation. The script calls this same helper for the baseline, reconstructed quantized candidate, and pruned candidate, so each row receives its own measured accuracy and latency distribution. The summary distinguishes actual array bytes from the modeled packed-code count. The ratios therefore compare executed candidates; they do not infer speed from sparsity or code width.

22.3.2 Script 2: generation with and without the cache

The second script measures a transformation that reuses earlier projection results. A language model generates one token at a time, and every new token attends over every token before it. The naive loop, the one Synthesis II ran, reruns the full forward pass over

the whole sequence at every step. For 16 tokens that means step t processes t positions and the loop processes $1+2+\cdots+16 = 136$ positions in all. At a 4,096-token context the same sum is over 8 million positions to produce 4,096 tokens, almost all of it recomputing keys and values that have not changed since the previous step. Chapter 19's cache keeps those keys and values, so each step processes one new token and reads the rest back, 16 positions for 16 tokens. The ratio $136/16 = 8.5$ counts projection rows removed, not end-to-end speedup or all attention work. The script itself replays 64 tokens (2,080 against 64 positions). A cached query still scores its growing prefix. The script checks equivalent execution paths before comparing their timings.

The script builds the actual two-block `GPT` from Chapter 13 with vocabulary 27, width 32, and two heads. It replays one fixed token sequence through both paths, rather than sampling different inputs for each timing. The core loop is extracted here from the script.

```
from contextlib import nullcontext

def replay_prefixes(model, tokens, cache=None):
    """Return the next-token logits at every position of one fixed sequence.

    The cache cursor advances once after all layers process a token. Resetting
    at entry makes each replay an independent request, including warmup runs.
    Cached forwards run inside an explicit generation scope; ordinary forwards
    resume when the scope exits, including after an exception.
    """
    from tinytorch.core.tensor import Tensor

    if cache is not None:
        cache.reset()
    logits = []
    with cache.generation() if cache is not None else nullcontext():
        for position in range(tokens.shape[1]):
            if cache is None:
                output = model(Tensor(tokens[:, :position + 1]))
            else:
                output = model(Tensor(tokens[:, position:position + 1]),
                               start_pos=cache.seq_pos)
                cache.advance()
            logits.append(output.data[:, -1, :].copy())
    return np.stack(logits, axis=1)
```

The `generation()` context enables cache reuse only for this request. `nullcontext()` gives the uncached path the same loop structure without changing execution mode. Resetting the cursor alone would not activate cached forward calls. When the context exits, ordinary forwards resume, including if an exception interrupts the loop. Each uncached call sees the full causal prefix. Each cached call sees one token at its correct position, traverses every transformer block, and advances the shared cursor once afterward. The returned array holds next-token logits at every position, so `compare_cached_replay`

checks all of them with `assert_allclose` before reporting a speedup. The timer includes the same complete replay on each path after warmup, with cache reset at the start of every cached replay. A `finally` block removes the wrapper even when a check fails. This is a controlled inference workload on fixed tokens (see Checkpoint below), not a claim about language quality or a guaranteed speedup on every machine.

When evaluating quantized models in multi-step generation, a common pitfall is verifying only single-step logit cosine similarity or top-1 agreement on a fixed prefix while ignoring multi-step autoregressive drift:

- **Compounding Rounding Error**: At step $t = 1$, an INT8 weight matrix shifts logits slightly, often retaining the reference argmax token. But at step $t = 50$, deviations in past hidden states fed back through the KV cache accumulate, eventually flipping a sampling decision.
- **Divergence Point**: Once a single token diverges between the FP32 reference and INT8 candidate, the two models traverse entirely different trajectories through sequence space. Measuring post-divergence logit similarity is meaningless.
- **Verification Rule**: Benchmark perplexity (PPL) on held-out evaluation text over full context windows, or assert bounded maximum absolute logit error ($\max |z_{\text{quant}} - z_{\text{ref}}| < \epsilon$) across the entire fixed prefix before testing free generation.

22.3.3 The scorecard

The script now has measurements for each candidate, but the final table leaves the acceptance decision to the reader. Anyone reading that table wants one verdict, and the naive way to produce one is a weighted sum: so much weight on accuracy, so much on size, so much on speed. Testing that approach against the FP32 baseline from the trace below, with half the weight on retention and a quarter each on the other two ratios, illustrates the failure mode. A careful INT8 candidate that keeps 99.4% of the accuracy, is 3.97x smaller, and runs at the same speed scores 1.7. A reckless one that keeps 68% of the accuracy, is 7.9x smaller, and runs 100x faster scores 27. A sum lets one axis buy another, and practitioners submitting to a benchmark will find the weighting that lets them buy the most. A floor cannot be bought. Above it, memory and latency compete on their own; below it, nothing else is looked at.

The scorecard takes a baseline and a candidate, each described by three numbers, and returns the three ratios and the book's relative accuracy verdict. Its event adapter imports the capstone policy from the exported TinyTorch package. It converts the trace's field names and byte count into the report format without restating the eligibility rules.

```
from tinytorch.olympics import OlympicEvent, qualifies_event

def score(baseline, candidate):
    """Three ratios, each candidate-vs-baseline, plus the chapter's
    ↪   retention floor."""
    retention = candidate["accuracy"] / baseline["accuracy"]
    return {
        "retention": retention,
        "size_ratio": baseline["bytes"] / candidate["bytes"],
        "speedup": baseline["p50_ms"] / candidate["p50_ms"],
        "retention_ok": retention >= 0.99,
    }
```

```
12
13 def qualifies(candidate, event):
14     metrics = {
15         "accuracy": candidate["accuracy"],
16         "latency_ms_median": candidate["p50_ms"],
17         "model_size_mb": candidate["bytes"] / 1e6,
18     }
19     return qualifies_event(metrics, event)
```

There is deliberately nothing clever here. The three ratios are three divisions, the floor is one comparison, and the event adapter delegates to the capstone helper. The value of the scorecard is not in the code but in what it refuses to do. It will not combine the ratios into a single number, but the function accepts whatever measurements the caller supplies. It cannot tell a measured value from an estimate; that distinction belongs in the report and the experiment that produced it. The production bridge returns to the protocol needed to make such evidence independently reproducible.

22.4 A Worked Trace

The trace has two parts. The first pushes one small weight matrix through two algorithms that change values, so that every number can be checked by hand. The second fills in the scorecard for `DigitMLP`.

22.4.1 One matrix, quantized and pruned

Script 1 measures the rounded model's accuracy, but its packed-storage ratio remains an analytical count. A small trace separates the arithmetic of rounding from the representation used to store its result. The comfortable assumption is that it does nothing, since the codes decode back into weights. For this input they decode back into nearby weights, each within half a quantization step of where it started, and the only way to see how wide that step is, and which weights pruning would then discard, is to push a few weights through both rules by hand. Take the six weights $w = [-1.0,\ 0.0,\ 2.0,\ 0.5,\ 1.5,\ -0.5]$, the example from Module 15's own docstring. In the first listing below, lines 1 to 7 define asymmetric min-max quantization `quantize_int8(values)`, and lines 9 and 10 define `dequantize_int8`. Line 12 initializes test weights `w`. Line 13 executes quantization, and line 14 prints scale factor 0.0118, zero point -43, and quantized integer codes `[-128, -43, 127, 0, 84, -86]`. Line 15 dequantizes the codes back into approximate floats.

```
1 def quantize_int8(values):
2     """Asymmetric min-max INT8: returns (codes, scale, zero_point)."""
3     lo, hi = min(min(values), 0.0), max(max(values), 0.0)
4     scale = (hi - lo) / 255
5     zero_point = round(-128 - lo / scale)
6     codes = [max(-128, min(127, round(v / scale + zero_point))) for v
  ↪   in values]
7     return codes, scale, zero_point
8
9 def dequantize_int8(codes, scale, zero_point):
```

```
    return [(c - zero_point) * scale for c in codes]

w = [-1.0, 0.0, 2.0, 0.5, 1.5, -0.5]
codes, scale, zp = quantize_int8(w)
print(round(scale, 4), zp, codes)
print([round(v, 4) for v in dequantize_int8(codes, scale, zp)])
```

The scale is $3/255 = 0.011765$ and the zero point is $\text{round}(-128 + 85) = -43$. For this nonconstant, zero-spanning input, the hand function follows Module 15's rule and the module agrees. In the second snippet below, line 5 runs the actual package implementation `tinytorch.perf.quantization.quantize_int8` on a Tensor, and line 6 confirms exact numerical agreement with the manual trace.

```
import numpy as np
from tinytorch.core.tensor import Tensor
from tinytorch.perf.quantization import quantize_int8

codes, scale, zp = quantize_int8(Tensor(np.array(w, dtype=np.float32)))
print(codes.data, round(scale, 6), zp)       # [-128. -43. 127. 0. 84.
↪   -86.] 0.011765 -43
```

table 22.1 lists each weight's code, its reconstruction, and its fate under 50% magnitude pruning, which selects the three smallest magnitudes by stable rank. For these distinct boundary magnitudes, a cutoff of 0.75 gives the same selection; the implementation uses ranks so ties cannot change the number selected.

Table 22.1: Six weights through min-max INT8 quantization (scale 0.011765, zero point -43) and through 50% magnitude pruning (threshold 0.75).

Weight	INT8 code	Restored	Abs. error	Kept?	Pruned
-1.0	-128	-1.0000	0	yes	-1.0
0.0	-43	0.0000	0	no	0.0
2.0	127	2.0000	0	yes	2.0
0.5	0	0.5059	0.0059	no	0.0
1.5	84	1.4941	0.0059	yes	1.5
-0.5	-86	-0.5059	0.0059	no	0.0

The largest reconstruction error is half a quantization step, 0.0059, for these values. Endpoint clipping can change that bound for other input ranges. The three weights that survive pruning are the three largest in magnitude, and the sparsity is exactly $3/6 = 50\%$. Nothing about either operation depends on the size of the matrix; `DigitMLP`'s 2,368 weights go through the same two rules.

22.4.2 The scorecard for `DigitMLP`

A benchmark is more useful when the expected output is calculated in advance, so the prediction comes before the ledger. `DigitMLP` has two linear layers, 64 inputs to 32 hidden units to 10 classes, and its parameter count follows from that shape: $64 \times 32 + 32 +$

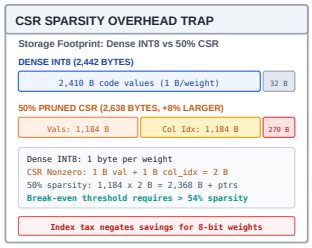
$32 \times 10 + 10 = 2{,}410$. At 4 bytes each that is $9{,}640$ bytes, and one forward pass costs $2 \times (64 \times 32 + 32 \times 10) = 4{,}736$ FLOPs, counting a multiply and an add as two. These are exact parameter and matrix-product counts for this architecture; the FLOP count omits bias additions and activations.

Napkin math for the rest. A hypothetical packed INT8 format keeps 1 byte per parameter plus a scale and a zero point for each of the four parameter tensors, so about $2{,}410 + 32 = 2{,}442$ bytes, a size ratio just under 4. Pruning half the weights to zero and storing them densely changes nothing, and storing them in the compressed sparse row format of Chapter 17 costs an index per surviving weight, which at 50% sparsity roughly cancels the saving. Both reconstructed quantization and dense pruning leave the matrix shapes and dense arithmetic unchanged. Their latency therefore has to be measured; neither representation claim supplies a speedup.

The ledger in the listing below is a separate arithmetic exercise. In lines 1 to 6, the analytical storage formulas compute dense FP32 (9,640 bytes), packed INT8 (2,442 bytes), and 50% sparse CSR representation (2,638 bytes). Lines 8 to 12 populate the candidate dictionary. The loop in lines 14 to 19 calls `score(base, cand)` to evaluate accuracy retention R, size compression ratio C, speedup S, and event qualification, printing the rows of table 22.2.

```
1  params = 64 * 32 + 32 + 32 * 10 + 10            # 2,410
2  weights = 64 * 32 + 32 * 10                      # 2,368, biases
   ↪  excluded
3  fp32_bytes = params * 4                          # 9,640
4  int8_bytes = params + 4 * (4 + 4)                # codes + (scale, zero
   ↪  point) x 4 tensors
5  nnz = weights // 2                               # 1,184 survivors at
   ↪  50% sparsity
6  csr50_bytes = nnz * 2 + (65 + 33) * 2 + 42 + 4 * 8   # value+index,
   ↪  row starts, biases, scales
7
8  ledger = {
9      "fp32":   {"accuracy": 0.880, "bytes": fp32_bytes,  "p50_ms":
       ↪  0.05},
10     "int8":   {"accuracy": 0.880, "bytes": int8_bytes,  "p50_ms":
       ↪  0.05},
11     "pruned": {"accuracy": 0.860, "bytes": csr50_bytes, "p50_ms":
       ↪  0.05},
12 }
13 base = ledger["fp32"]
14 for name, cand in ledger.items():
15     s = score(base, cand)
16     print(f"{name:7s} R={s['retention']:.3f} C={s['size_ratio']:.2f}x
       ↪  "
17           f"S={s['speedup']:.2f}x floor={s['retention_ok']} "
18           f"events={[e.value for e in OlympicEvent if qualifies(cand,
             ↪  e)]}")
```

The sparse byte count assumes INT8 values with 1-byte column indices (every column index is below 64, so one byte suffices), 2-byte row-start entries for the 65 and 33 row boundaries of the stored $(64, 32)$ and $(32, 10)$ weight matrices, 42 bias bytes, and the scales.[91] The loop prints the rows of table 22.2 from those counts.



CSR Sparsity Overhead: At 8-bit precision, adding a 1-byte column index per nonzero weight doubles per-element storage, making 50% pruned CSR (2,638 B) larger than dense INT8 (2,442 B) until sparsity exceeds about 54%.

[91] The storage overhead of Compressed Sparse Row (CSR) depends on matrix dimensions: column indices require $\lceil \log_2(N)/8 \rceil$ bytes per nonzero element (1 byte for $N \leq 256$, 2 bytes for $N \leq 65{,}536$), while row pointers require $\lceil \log_2(\text{nnz})/8 \rceil$ bytes per row. When weight matrices are wide (e.g., $N = 4096$ in modern LLMs), column indices take 2 bytes, meaning a 1-byte INT8 value carries a 2-byte index, tripling the per-weight storage, so CSR saves space only above about 67% sparsity unless block-sparse representations (e.g., 2:4 structured sparsity with 2-bit index nibbles) are used.

Table 22.2: An illustrative scorecard for `DigitMLP`. Bytes count the stated hypothetical formats; accuracies are chosen inputs. All three rows have assumed speedup $S = 1$ and pass all five classroom event floors. These rows do not report the scripts' measurements.

Candidate format	Bytes	Accuracy	R	C	Pass floor?
FP32 dense	9,640	88%	1.000	1.00	yes
INT8 packed	2,442	88%	1.000	3.95	yes
INT8 + 50% prune, CSR	2,638	86%	0.977	3.65	no

The packed INT8 ratio is $9{,}640/2{,}442 \approx 3.95$, including the metadata counted by `Quantizer.quantize_model`. The sparse row is larger because each surviving code needs an index and each matrix needs row boundaries. Zeroing half the weights did not halve these bytes. The illustrative pruned candidate also fails the chapter's relative retention floor while passing the module's absolute event floors. The policies ask different questions. Finally, the $1.00\times$ speedup is an assumption used to exercise `score`; an experiment must replace both timing inputs with observations, or mark speedup as unavailable.

22.5 From TinyTorch to Production

The classroom scorecard checks arithmetic on supplied measurements. An external benchmark must also control how those measurements were obtained. MLPerf Inference's rules specify workloads, quality requirements, execution scenarios, and reproducibility requirements. Its LoadGen harness issues queries and records completions.[92] The scenario determines the reported performance metric; there is no single universal latency number. TinyTorch's class named `MLPerf` does not implement that official protocol.

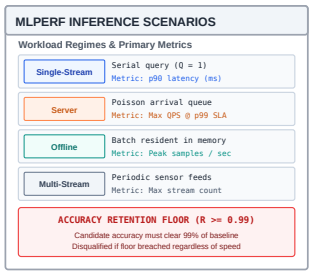
Deploying quantized transformer models on consumer hardware (phones, laptops, embedded edge boards) requires moving beyond Python runtimes to specialized C/C++ inference engines:

- **GGUF Format and llama.cpp**: Georgi Gerganov's `llama.cpp` uses the unified binary file format **GGUF** (successor to GGML). GGUF bundles model architecture hyperparameters, tokenizer vocabulary, and mixed-precision quantized tensor weights (e.g., Q4_K_M, Q8_0) into a single mmap-friendly file.
- **Zero-Copy Memory Mapping (`mmap`)**: Instead of allocating and loading gigabytes of weights into heap RAM on startup, edge runtimes map the GGUF file directly into virtual address space. Pages are faulted into memory on demand by the OS kernel, enabling near-instantaneous startup and memory sharing across processes.
- **Heterogeneous Offloading**: GGUF runtimes dynamically split layers between available compute backends, offloading N transformer layers to a discrete or integrated GPU (via Metal on Apple Silicon, CUDA, or Vulkan) while executing remaining layers on the CPU SIMD units (ARM NEON, x86 AVX-512).
- **ONNX Runtime (ORT)**: For cross-platform enterprise edge deployments, ONNX Runtime compiles computational graphs with hardware-specific Execution Providers (DirectML, OpenVINO, QNN for Qualcomm NPUs), fusing attention and layer normalization while executing quantized integer GEMM kernels.

[92] MLPerf Inference standardizes benchmarking across four distinct operational scenarios: (1) **Single-Stream**: queries arrive one at a time with query size 1, measuring 90th-percentile latency for interactive edge devices; (2) **Server**: queries arrive following a Poisson distribution, measuring the maximum supported query rate (QPS) subject to a strict 99th-percentile latency constraint; (3) **Offline**: all input queries are resident in memory upfront, measuring peak throughput (samples/second) where latency is unconstrained; and (4) **Multi-Stream**: each query carries several samples (for example, eight camera frames), and the metric is the 99th-percentile query latency.

- **Embedded Microcontrollers (Bare-Metal Arena)**: In bare-metal environments (ARM Cortex-M, RISC-V targeted by MLPerf Tiny), dynamic heap allocation (`malloc`/`new`) is strictly prohibited. Runtimes like TFLite Micro and TinyEngine mandate static memory arenas pre-allocated at compile time into fixed contiguous SRAM offsets to prevent fragmentation crashes.

Table 22.3 separates the local experiment from the additional controls around an official result.



MLPerf Inference Scenarios: Workload arrival patterns determine whether latency (Single-Stream), QPS under SLA (Server), or throughput (Offline) is benchmarked, all bounded by the non-negotiable R >= 0.99 accuracy retention floor.

Table 22.3: This synthesis chapter teaches a controlled comparison; an official benchmark also standardizes the workload and execution protocol.

Question	Synthesis III	MLPerf Inference
Who controls timing?	The script's repeated calls	LoadGen's query protocol
What quality rule applies?	Chapter retention floor or classroom event policy	Task-specific quality requirement
Which artifact runs?	Baseline and each executable candidate	Submitted system under test
What makes results comparable?	Same data, shapes, and local timing procedure	Defined benchmark and scenario rules

The practical questions are the same when reading either report. Which artifact ran? Were its quality and latency measured on the stated workload? Does the byte count describe resident arrays or a proposed file format? Where do metadata and sparse indices enter that count? A pruning experiment with unchanged latency can be a correct result: a dense kernel still visits the zero entries. The scorecard lets the reader distinguish that result from a failed measurement, bridging the gap to industrial edge deployment engines (see Systems Perspective above).

Before any compressed or accelerated model artifact is deployed into production pipelines, four verification gates must be certified:

1. **End-to-End Task Metric Parity:** Passing a generic synthetic accuracy test is insufficient. The quantized or pruned candidate must be evaluated on the full target test split (such as perplexity on validation text, top-1 accuracy on classification benchmarks, or exact-match scores) to confirm that degradation stays within the pre-agreed tolerance threshold.
2. **True Resident Footprint Accounting:** Parameter storage on disk does not equal resident memory usage at runtime. Total memory consumption must account for weight buffers, activation working memory, runtime context overhead, and dynamic Key-Value cache allocations at peak batch size.
3. **Warm-Up and Tail Latency Audit:** Single average execution times conceal catastrophic tail latencies caused by cache misses, memory paging, or thermal throttling. Runtimes must measure $p50$, $p90$, and $p99$ latencies under realistic concurrent load after adequate warm-up iterations.
4. **Logit and Numerical Equivalence Bounds:** Intermediate layer activations and output logits must match reference single-precision executions within expected floating-point rounding tolerances before measuring speedups.

22.6 Summary

This synthesis chapter built the scorecard that turns Part III's separate optimizations into a verdict. Three ratios, accuracy retention R, size ratio C, and speedup S, each compare the optimized model against the baseline; a floor on R decides whether the other two are looked at; and the capstone helper checks eligibility for each classroom event without combining the ratios into a ranking. The worked trace pushed six weights through min-max INT8 quantization and 50% magnitude pruning by hand, then counted a proposed packed representation of `DigitMLP` to show that its packed INT8 format would give $3.95\times$, that 50% pruning with sparse indices gives less, and that neither changes how many multiplies the dense forward pass performs. Reading the synthesis scripts closely exposed where their printed numbers are measurements and where they are assumptions, which is the skill the scorecard exists to teach. The invariant to carry forward is that an optimization has paid off only when the optimized artifact itself has been measured on all three axes, against the same baseline, with an accuracy floor it cleared. Chapter 23 applies the same discipline to kernels. It leaves NumPy for compiled C++ and GPU code and measures what each step actually gains.

22.7 Further Reading

- **Reddi et al.** (2020). "MLPerf inference benchmark." *ISCA*. The benchmark design the MLPerf Optimization Olympics borrows: fixed tasks, quality floors, and measured latency.
- **Mattson et al.** (2020). "MLPerf training benchmark." *MLSys*. The training-side counterpart and its rules for comparable results.
- **Banbury et al.** (2021). "MLPerf Tiny benchmark." *NeurIPS*. The extension of the MLPerf suite to microcontroller-class devices, emphasizing the extreme memory and latency constraints that edge deployments face.

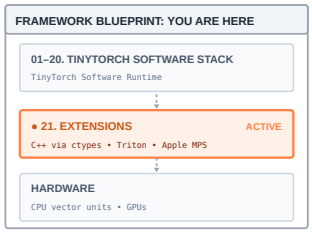
PART IV

Extensions & Accelerators

🔥 Chapter 23

Extending TinyTorch

23.1 The Core Framework and the Extensibility Dilemma



Framework Blueprint: the optional extensions connect the core 20-module TinyTorch stack to hardware kernels, compilation, and production memory optimization.

Across Modules 01 through 20 (Chapter 1 through Chapter 21), the preceding chapters constructed a complete, self-contained deep learning framework from bare silicon concepts up to modern generative AI:

- An n-dimensional array abstraction with arbitrary striding and slicing (Chapter 1)
- A reverse-mode automatic differentiation tape with topological backpropagation (Chapter 6)
- Composable parametric layers, activations, loss functions, and optimizers (Chapter 3, Chapter 2, Chapter 4, Chapter 7)
- Data ingestion pipelines, multi-headed self-attention, and autoregressive GPT transformers (Chapter 5, Chapter 12, Chapter 13)
- Execution profiling, uniform INT8 quantization, weight pruning, and MLPerf benchmarking (Chapter 15, Chapter 16, Chapter 17, Chapter 20)

Every single module in the core operates on a shared, unified execution contract. A `Tensor` encapsulates contiguous memory; a `Function` records forward and backward transformations on the autograd tape; a `Layer` manages trainable parameters; an `Optimizer` executes step updates.

Yet in modern deep learning systems, the core runtime is only the beginning. When neural networks scale to billions of parameters, or when production serving demands microsecond latencies, frameworks face an architectural dilemma:

1. **The Monolithic Framework Trap:** Shoveling every emerging systems optimization (low-rank adapters, activation checkpointing, custom GPU kernels, mixed-precision loss scaling, graph compilers) directly into the core framework causes it to balloon into a brittle monolith where bugs in experimental kernels destabilize the core autograd engine.
2. **The Extensibility Model:** Keep the engine core minimal, stable, and transparent, while exposing strict, first-class extension points. Every specialized optimization lives as an autonomous extension that plugs cleanly into the core contracts.

TinyTorch adopts the extensibility model. The 20 core modules constitute an immutable foundation. Specialized components, including vectorized C++ CPU kernels, Apple MPS dispatch, OpenAI Triton shaders, parameter-efficient fine-tuning, and graph fusion compilers, live in the `tinytorch.extensions` ecosystem.

23.2 Why Not Just Use PyTorch? The Minix and xv6 Lesson

A natural question arises: *Why study and build systems extensions in TinyTorch rather than jumping straight into authoring custom extensions for PyTorch or JAX?*

The answer lies in the classic pedagogical tradition of operating systems education: from John Lions' legendary commentary on Unix 6th Edition, to Andrew Tanenbaum's Minix, to MIT's xv6.

When Linus Torvalds wanted to understand how an operating system truly manages physical hardware, he did not begin by modifying a commercial monolith. He studied Minix, a compact microkernel operating system written by Tanenbaum specifically for teaching. Minix was small enough to be held in a single person's head, yet real enough to demonstrate preemptive scheduling, memory management, and system calls. That tactile understanding gave Linus the foundational mental model to author the initial Linux kernel.

Years later, when MIT redesigned its operating systems curriculum (6.828 / 6.1810), it chose xv6, a modern reimplementation of Unix 6th Edition in 10,000 lines of clean ANSI C. When teaching virtual memory and process scheduling, MIT does not ask students to modify the production Linux kernel. Linux contains over 30 million lines of C, thousands of hardware drivers, lock-less read-copy-update (RCU) primitives, and deep architecture-specific macros. Attempting to implement a simple copy-on-write page allocator in Linux forces a student to spend 90% of their time fighting macro definitions, build harnesses, and kernel ABI wrappers before manipulating a page table. In xv6, the abstraction boundaries are razor-sharp: a student can implement copy-on-write or a priority scheduler in 40 lines of clear C and inspect every page table manipulation directly in physical memory.

PyTorch is the Linux kernel of deep learning. It is an industrial engineering masterpiece, but extending it requires navigating the `TORCH_LIBRARY` dispatcher macros, pybind11 ABI translation layers, internal ATen tensor representations (`c10::TensorImpl`), CUDA stream abstractions, and massive multi-repository CMake build graphs. Writing a custom C++ operator in PyTorch requires dozens of lines of framework glue code just to deliver a tensor pointer to a loop.

TinyTorch is the Minix and xv6 of deep learning systems:

- A `Tensor` is directly backed by a contiguous memory buffer.
- A `Function` has exactly two methods: `forward` and `backward`.
- A `Layer` has exactly two methods: `forward` and `parameters()`.
- An `Optimizer` is a clean iterator over parameter tensors.

Implementing Low-Rank Adaptation (LoRA) in TinyTorch requires about 30 lines of Python to reveal how frozen base weights and trainable low-rank adapters interact with the autograd tape. Writing a SIMD GEMM kernel passes memory addresses across a simple C foreign function interface (FFI) without layers of framework obfuscation; the Apple GPU path, by contrast, borrows PyTorch's MPS backend. The FFI path exposes the unadorned reality of the hardware memory subsystem, CPU vector registers, and cache hierarchy.

23.3 The Four Native Extension Points

TinyTorch exposes four native extension points that correspond to the four fundamental boundaries of deep learning execution:

1. **The Autograd Boundary (`Function`):** Subclassing `Function` defines custom forward and backward semantics, intercepting the autograd tape to drop or recompute intermediate activations (as in activation checkpointing).

2. **The Architectural Boundary (`Layer`):** Subclassing `Layer` introduces custom architectural primitives, freezes pretrained weight matrices by toggling `requires_grad=False`, and exposes low-rank adapter parameters to the optimizer (as in LoRA).
3. **The Training Dynamics Boundary (`Optimizer`):** Interfacing with `optimizer.params` enables custom gradient transformations, loss scaling for mixed-precision arithmetic, gradient clipping, and advanced update rules (such as Lion or Muon).
4. **The Silicon Boundary (Backend FFI):** Extracting contiguous memory pointers from `Tensor.data` enables invoking native compiled C++ shared libraries via `ctypes`, launching GPU kernels in OpenAI Triton, or offloading computation to the Apple Silicon GPU through PyTorch's MPS backend (which requires `torch`).

↩ **Jump back:** Chapter 7 *Adam moment state tensors (m_t, v_t) and memory consumption during gradient descent.*
↩ **Jump back:** Chapter 6 *The autograd tape, topological graph traversal, and intermediate tensor lifetimes.*
↩ **Jump back:** Chapter 18 *Hardware cache lines, 2D loop tiling, and register residency.*

23.3.1 Repository Architecture: Core vs. Extensions

Industrial frameworks like PyTorch and JAX decouple their core computational engines from domain-specific libraries to maintain stability while enabling ecosystem velocity. Isolating stable autograd primitives from rapidly evolving kernel optimizations prevents experimental features from compromising foundational runtime guarantees. To preserve stability, TinyTorch enforces a strict structural separation between the core standard library and the extensions ecosystem:

```
tinytorch/
├── tinytorch/
│   ├── core/           # 20-module core engine
│   │   ├── tensor.py   # Tensor memory & autograd
│   │   ├── autograd.py # Topological tape
│   │   ├── layers.py   # Linear, Sequential, etc.
│   │   ├── losses.py   # MSE & CrossEntropy
│   │   ├── optimizers.py
│   │   ├── dataloader.py
│   │   ├── attention.py
│   │   └── transformers.py
│   ├── perf/           # Baseline profiling
│   │   ├── profiling.py
│   │   └── benchmarking.py
│   └── extensions/     # Contrib ecosystem
│       ├── template.py # Starter template
│       ├── lora.py     # Parameter-efficient adapters
│       ├── checkpoint.py
│       ├── loss_scaler.py
│       ├── compile.py  # Kernel fusion compiler
│       ├── simd_ops.py # Vectorized C++ kernels
```

```
│       ├── mps_ops.py  # Apple Silicon MPS
│       └── triton_gelu.py
└── tests/extensions/   # Parity test suite
```

The `tinytorch/core/` and `tinytorch/perf/` directories contain the standard library exported by Modules 01 through 19; Module 20 exports to `tinytorch/olympics.py`. The `tinytorch/extensions/` directory is the open workshop: every optimization lives as an isolated, importable Python or C++ module that plugs into the core boundaries without modifying core source files.

23.3.2 The Extension Contract and Starter Template

Extensible systems enforce formal architectural contracts to integrate external plug-ins without destabilizing core dispatch pipelines or causing opaque runtime crashes. Establishing explicit invariants for memory layout, autograd propagation, and device availability ensures modular components interoperate reliably across heterogeneous backends. To ensure that every extension functions harmoniously with the rest of the framework, all extensions adhere to the four-part **Extension Contract**:

1. **Systems Framing:** Every extension must clearly state the physical resource bottleneck it resolves (e.g., DRAM capacity, memory bandwidth, vector register utilization, or floating-point underflow).
2. **Tensor and NumPy Interoperability:** Functions must accept either a TinyTorch `Tensor` or a raw NumPy `ndarray`. When passed a `Tensor`, the function must return a `Tensor` (preserving the computation graph if differentiable). When passed a NumPy array, it must return a NumPy array.
3. **Autograd Transparency:** Operations that participate in backpropagation must explicitly define their mathematical derivatives via `Function.forward` and `Function.backward`.
4. **Graceful Fallback:** If a required compiler, runtime library, or physical GPU accelerator is unavailable, the extension must fall back cleanly to a pure-Python or NumPy reference implementation rather than crashing on import.

TinyTorch provides a canonical starter template in `tinytorch/extensions/template.py` that demonstrates all four properties across every native boundary:

1. **Autograd Function Boundary:** When implementing a custom autograd operator, subclass `Function` and implement the forward and backward transformations:

```python
class CustomScaledShift(Function):
    """Example autograd operation: y = alpha * x + beta."""

    def forward(self, x: np.ndarray) -> np.ndarray:
        alpha = getattr(self, "alpha", 2.0)
        beta = getattr(self, "beta", 1.0)
        return alpha * x + beta

    def backward(self, grad_output):
        # Unwrap grad_output to NumPy array
        is_t = isinstance(grad_output, Tensor)
        g = grad_output.data if is_t else grad_output
```

```
        alpha = getattr(self, "alpha", 2.0)
        # Derivative w.r.t x is alpha * grad
        dx = alpha * g
        return (dx,)
```

2. **Architectural Layer Boundary:** When authoring a custom neural network module with learnable parameters, subclass `Layer` and register its parameter tensors:

```
class CustomResidualBlock(Layer):
    """Example custom neural network module."""

    def __init__(self, dim: int):
        super().__init__()
        std = 1.0 / np.sqrt(dim)
        w_init = np.random.randn(dim, dim) * std
        self.weight = Tensor(
            w_init.astype(np.float32),
            requires_grad=True,
        )
        self.bias = Tensor(
            np.zeros(dim, dtype=np.float32),
            requires_grad=True,
        )

    def parameters(self):
        """Return list of learnable parameters."""
        return [self.weight, self.bias]

    def forward(self, x: Tensor) -> Tensor:
        # Residual branch: x + Linear(x)
        linear_out = x.matmul(self.weight)
        return x + linear_out + self.bias
```

3. **Training Dynamics Boundary:** When modifying parameter gradients prior to the optimizer step (such as threshold clipping or spectral projection), wrap parameter updates in an in-place transformation:

```
class CustomGradientTransform:
    """Example training dynamics hook: scales gradients or clips
    ↪ thresholds."""

    def __init__(self, clip_value: float = 1.0):
        self.clip_value = clip_value

    def transform_gradients(self, parameters: List[Tensor]) -> None:
        """In-place gradient transformation before
        ↪ optimizer.step()."""
        for p in parameters:
            if p.grad is not None:
                p.grad = np.clip(p.grad, -self.clip_value,
↪ self.clip_value)
```

4. **Silicon Kernel Boundary:** When accelerating operations via compiled C++, Triton, or GPU APIs, encapsulate native dispatch behind runtime feature detection with a transparent pure-NumPy fallback:

```python
def custom_accelerated_op(
    x: Union[Tensor, np.ndarray],
) -> Union[Tensor, np.ndarray]:
    """Example hardware-accelerated ReLU demonstrating safe fallback.
    """
    is_tensor = isinstance(x, Tensor)
    x_data = x.data if is_tensor else np.asarray(x)

    if has_custom_hardware():
        # Dispatch to compiled C++ shared library, Triton kernel, or
        ↪ GPU runtime
        out_data = _hardware_relu_kernel(x_data)
    else:
        # Transparent fallback to the CPU reference implementation
        ↪ (ReLU)
        out_data = np.maximum(0, x_data)

    return Tensor(out_data) if is_tensor else out_data
```

The 5-Step Extension Recipe

Adding a new capability to TinyTorch follows an explicit five-step protocol:

1. **Identify the Boundary:** Map the optimization to `Function` (math/memory), `Layer` (adapters/primitives), `Optimizer` (gradients/moments), or C-ABI `ctypes`/GPU runtime (silicon).
2. **Copy the Template:** Duplicate `tinytorch/extensions/template.py` to `tinytorch/extensions/<feature_name>.py`.
3. **Implement the Contract:** Satisfy the core invariants: handle both `Tensor` and `np.ndarray`, define backward passes for differentiable operations, and fall back to NumPy if hardware compilers or libraries are missing.
4. **Author Unit Tests:** Duplicate `tinytorch/tests/extensions/test_template.py` to `tinytorch/tests/extensions/test_<feature_name>.py` and verify numerical parity against an unoptimized NumPy reference (`np.allclose(out, ref, atol=1e-5)`).
5. **Register Public Symbols:** Export new classes and functional wrappers in `tinytorch/extensions/__init__.py`.

23.4 The Flat Extension Catalog

The sections below examine each primary extension in the TinyTorch ecosystem. In keeping with a rigorous systems focus, artificial groupings are avoided: whether an extension optimizes mathematical parameters, memory lifetimes, numerical precision, or physical silicon execution, each appears as a complete, self-contained systems case study.

Each extension establishes:

1. **The Systems Bottleneck:** The physical hardware constraint being hit.

2. **Architectural Strategy:** How the extension bypasses the constraint.
3. **The Implementation:** The actual source code.
4. **The Scorecard:** Quantitative metrics for the efficiency gain, labeled as measured on a stated machine or as analytical counts.

23.4.1 Extension 1: Parameter-Efficient Fine-Tuning (LoRA)

In modern generative AI deployments, Low-Rank Adaptation (LoRA) is the standard mechanism for fine-tuning foundation models without duplicating massive parameter sets for each downstream task. By freezing pretrained checkpoints and training only compact rank-decomposition matrices, systems can serve hundreds of task-specific adapters concurrently from a single shared base model.

The Systems Bottleneck: Fine-tuning large language models by updating all parameters encounters a severe memory wall. Modern adaptive optimizers such as Adam maintain two 32-bit floating-point state tensors per parameter: the first moment vector m_t (momentum) and the second raw moment vector v_t (variance). Each parameter requires 8 bytes of optimizer state, plus 4 bytes for the stored gradient, plus 4 bytes for the weight itself, totaling 16 bytes per parameter. For a 7-billion parameter model, storing Adam optimizer states alone consumes $7 \times 10^9 \times 8 = 56$ GB of memory, far exceeding consumer GPU memory before a single activation is allocated.

Architectural Strategy: Low-Rank Adaptation (LoRA) freezes the pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ by setting `requires_grad=False`. It decomposes the weight update ΔW into the product of two small, low-rank matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, where the rank $r \ll \min(d, k)$:

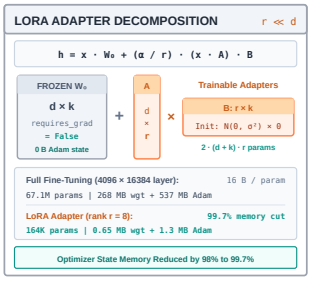
$$h = xW_0 + x\Delta W = xW_0 + xAB$$

Because W_0 is frozen, the autograd engine allocates gradients and Adam optimizer states solely for A and B, drastically reducing optimizer-state memory while restricting each update to rank r.

The Implementation: `LoRALinear` subclasses `Layer`, holds frozen weights, and trains only the low-rank factor matrices:

```python
class LoRALinear(Layer):
    """
    Low-Rank Adaptation (LoRA) for a Linear layer.
    Replaces a dense weight matrix with a frozen weight matrix + a low-rank adapter.
    """
    def __init__(self, in_features: int, out_features: int, rank: int = 4):
        super().__init__()
        # 1. The frozen pre-trained weights
        self.weight = Tensor(np.random.randn(in_features, out_features) / np.sqrt(in_features), requires_grad=False)
        self.bias = Tensor(np.zeros(out_features), requires_grad=False)

        # 2. The low-rank adapter matrices (these are trained!)
```



LoRA Rank Decomposition: The pretrained base weight W_0 is frozen ($requires_grad = False$), eliminating optimizer momentum and variance states. Training is restricted to low-rank matrices A and B, slashing Adam memory overhead by over 98 percent.

```
        # A maps from in_features down to the bottleneck rank
        self.A = Tensor(np.random.randn(in_features, rank) /
        ↪   np.sqrt(in_features), requires_grad=True)
        # B maps from the bottleneck rank back up to out_features
        self.B = Tensor(np.zeros((rank, out_features)),
        ↪   requires_grad=True)

    def forward(self, x: Tensor) -> Tensor:
        # Standard forward pass: x @ W + b
        base = x.matmul(self.weight) + self.bias

        # Adapter forward pass: (x @ A) @ B
        adapter = x.matmul(self.A).matmul(self.B)

        # The autograd tape automatically merges gradients where they
        ↪   sum!
        return base + adapter

    def parameters(self):
        """Only the adapter matrices train; the base weight and bias
        ↪   stay frozen.
        """
        return [self.A, self.B]
```

Pause & Predict: The Memory Economics of Freezing Base Weights

Suppose you fine-tune a 7-billion parameter language model in 16-bit precision using the Adam optimizer. Each parameter requires 2 bytes for the weight, 2 bytes for the gradient, and 8 bytes for Adam moment states (m_t and v_t in FP32), totaling 12 bytes of training state per parameter (84 GB). If you freeze all base weights and attach a LoRA adapter with rank $r = 16$ affecting 0.2% of the parameters (14 million parameters), how much memory do the frozen base weights require? How much memory do the LoRA parameters and their optimizer states require? What is the total reduction in training memory state?

The Analytical Scorecard: Consider a standard transformer feedforward projection layer ($d_{\text{in}} = 4096, d_{\text{out}} = 16384$, totaling 67.1 million parameters). table 23.1 details the parameter counts and memory footprints across adapter ranks compared to full fine-tuning.

Table 23.1: **Memory footprint of full fine-tuning vs. LoRA on a 4096 × 16384 projection layer.** Counted, not measured. Storing Adam optimizer states (8 bytes per parameter) dominates full fine-tuning; LoRA reduces trainable-weight and optimizer-state memory by more than 98%. The frozen base weight (268.4 MB) stays resident in every row.

Configuration	Trainable Parameters	Trainable Weight Memory	Adam State Memory	Memory Reduction
Full Fine-Tuning	67,108,864	268.4 MB	536.9 MB	Reference (0.0%)
LoRA ($r = 64$)	1,310,720	5.2 MB	10.5 MB	98.0%
LoRA ($r = 16$)	327,680	1.3 MB	2.6 MB	99.5%
LoRA ($r = 8$)	163,840	0.66 MB	1.31 MB	99.8%
LoRA ($r = 4$)	81,920	0.33 MB	0.66 MB	99.9%

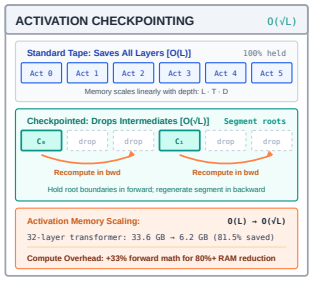
23.4.2 Extension 2: Activation Checkpointing

Pioneered in frontier training systems like Megatron-LM and DeepSpeed, activation checkpointing is an essential systems primitive for scaling context lengths and model depths on memory-constrained GPUs. By selectively discarding intermediate forward tensors and recomputing them on-demand during the backward pass, frameworks trade minor redundant arithmetic for substantial memory footprint reductions.

The Systems Bottleneck: In deep networks, the autograd tape saves intermediate tensor activations produced by every forward layer to evaluate gradient chain rules during backpropagation. For an L-layer transformer with sequence length T and hidden dimension D, activation memory scales linearly with depth: $O(L \cdot T \cdot D)$. On long sequences or deep architectures, activation memory quickly dwarfs parameter storage and causes out-of-memory (OOM) crashes during training.

Architectural Strategy: Activation Checkpointing (or gradient recomputation) discards intermediate forward activations from memory. The forward pass runs inside a `no_grad()` context, caching only the input boundary tensors between major layer blocks. During backpropagation, the forward pass for each block is recomputed dynamically from the cached boundary tensors. By dividing an L-layer network into $\sqrt{L}$ segments, activation memory scaling drops from $O(L)$ to $O(\sqrt{L})$, trading a 33% increase in compute for an 80% reduction in peak activation memory.

The Implementation: `checkpoint` wraps any forward callable inside a custom `Function`, detaching inputs during the backward pass to seed local recomputation:



Activation Checkpointing: Standard autograd holds all L layer activations in memory. Checkpointing stores only boundary tensors ($O(\sqrt{L})$), recomputing intermediate activations during backpropagation to reduce peak memory by up to 80 percent at a 33 percent compute overhead.

```
def checkpoint(function, *args):
    """
    Activation Checkpointing.
    Recomputes the forward pass during the backward pass to save
    memory.
    """
    class CheckpointFunction(Function):
```

```
        def forward(self, *numpy_arrays):
            # Run forward pass WITHOUT tracking gradients to save memory
            with no_grad():
                out_tensor = function(*self.inputs)
            # The returned value of forward() must be a numpy array
            return out_tensor.data if isinstance(out_tensor, Tensor) else out_tensor

        def backward(self, grad_output):
            # 1. Detach inputs so they act as graph leaves for the recomputation
            detached_inputs = []
            for inp in self.inputs:
                if isinstance(inp, Tensor):
                    x = Tensor(inp.data, requires_grad=inp.requires_grad)
                else:
                    x = inp
                detached_inputs.append(x)

            # 2. Recompute the forward pass WITH gradients enabled
            recomputed_out = function(*detached_inputs)

            # 3. Trigger the local backward pass from the recomputed output
            grad_tensor = Tensor(grad_output) if not isinstance(grad_output, Tensor) else grad_output
            recomputed_out.backward(grad_tensor)

            # 4. Return the gradients of the inputs (numpy arrays)
            grads = []
            for x in detached_inputs:
                if isinstance(x, Tensor) and x.grad is not None:
                    g = x.grad.data if isinstance(x.grad, Tensor) else x.grad
                    grads.append(g)
                else:
                    grads.append(None)
            return tuple(grads)

    return CheckpointFunction.apply(*args)
```

The Analytical Scorecard: table 23.2 models activation memory for transformers of increasing depth L ($T = 2048$, $D = 4096$) with and without activation checkpointing. It assumes 1.05 GB of saved activations per layer and a boundary tensor of $T \cdot D \cdot 4 = 33.6$ MB, so $\sqrt{L}$ segments hold $M = \sqrt{L}\, b + (L/\sqrt{L})\, A$.[93]

[93] **Activation checkpointing trade-offs**: In an L-layer neural network, standard autograd retains L activation tensors in memory, scaling as $\mathcal{O}(L)$. Checkpointing segments the model into k subgraphs of length $m = L/k$, storing only the boundary activations. Minimizing total peak memory $M(k) = k + L/k$ yields an optimal segment length $k^* = L^{1/2}$, cutting peak memory from $\mathcal{O}(L)$ to $\mathcal{O}(L^{1/2})$. The cost is exactly one extra forward pass per segment during backpropagation, adding a deterministic 33% computational overhead.

Table 23.2: **Modeled activation memory under activation checkpointing ($T = 2048, D = 4096$, float32).** Standard training scales linearly $O(L)$; checkpointing scales sublinearly $O(\sqrt{L})$ with uniform segments.

Layer Depth (L)	Standard Forward Memory	Checkpointed Memory	Compute Overhead	Peak Memory Saved
8 Layers	8.4 GB	3.06 GB	+33.3%	63.5%
16 Layers	16.8 GB	4.33 GB	+33.3%	74.2%
32 Layers	33.6 GB	6.13 GB	+33.3%	81.8%
64 Layers	67.2 GB	8.67 GB	+33.3%	87.1%

23.4.3 Extension 3: Mixed-Precision Loss Scaling

Automated mixed-precision engines, such as PyTorch AMP and NVIDIA Apex, rely on dynamic loss scaling to enable fast 16-bit training on hardware Tensor Cores. Because half-precision floating-point formats possess limited exponent ranges, multiplying the loss by a scaling factor shifts vanishingly small backward gradients away from the zero-underflow boundary.

The Systems Bottleneck: Training deep models in 16-bit floating-point (FP16) halves memory consumption and doubles compute throughput. However, IEEE 754 half-precision float16 allocates only 5 bits for the exponent, providing an exponent range of $[-14, 15]$. Its smallest positive normalized number is $2^{-14} \approx 6.1 \times 10^{-5}$, and its absolute minimum subnormal representation is $2^{-24} \approx 5.96 \times 10^{-8}$. Backpropagation gradients in deep neural networks often fall below 10^{-5}. In raw FP16, these small gradient values silently underflow to exact zero, extinguishing parameter updates and causing training to diverge.

Architectural Strategy: Loss scaling multiplies the scalar loss by a large constant factor S (e.g., $S = 65,536 = 2^{16}$) immediately before calling `.backward()`. By the linearity of differentiation:

$$\nabla_\theta(S \cdot \mathcal{L}) = S \cdot \nabla_\theta \mathcal{L}$$

Scaling shifts small gradient values up by 16 bits into the center of the normal FP16 exponent range, completely preventing underflow during backward tape traversal. Before the optimizer applies its parameter update, the gradients are unscaled by dividing by S:

$$\nabla_\theta \mathcal{L} = \frac{1}{S} \nabla_\theta (S \cdot \mathcal{L})$$

The Implementation: `LossScaler` manages the scaling lifecycle and directly unscales gradients across registered optimizer parameters:

```
class LossScaler:
    """
    Mixed Precision Loss Scaler.
    Multiplies the loss by a large constant before backward() to
    prevent float16 underflow,
    then unscales the gradients before the optimizer step.
```

```
    """
    def __init__(self, scale: float = 65536.0):
        self.scale = scale

    def backward(self, loss: Tensor):
        # Multiply the loss to protect gradients from underflow
        scaled_loss = loss * self.scale
        # TinyTorch's autograd engine pushes the scaled gradients back
        scaled_loss.backward()

    def unscale_(self, optimizer: Optimizer):
        # Traverse the registered parameters and divide the gradients
        ↪ back down
        for p in optimizer.params:
            if p.grad is not None:
                p.grad = p.grad / self.scale
```

The Numerical Scorecard: table 23.3 shows the IEEE 754 rounding behavior of small backpropagation gradients in unscaled FP16 vs. scaled FP16.

Table 23.3: **Numerical stability of gradient propagation under IEEE 754 float16.** Unscaled FP16 rounds values below about 2.98×10^{-8} (half the smallest subnormal) to zero; loss scaling preserves gradients across four additional orders of magnitude.

True Gradient Magnitude ($\nabla\mathcal{L}$)	Raw FP16 Result	Status	Scaled ($S = 65536$) FP16	Unscaled Gradient	Error
1.0×10^{-3}	1.000×10^{-3}	Preserved	65.536	1.000×10^{-3}	0.0%
5.0×10^{-5}	5.001×10^{-5}	Subnormal precision loss	3.2768	5.000×10^{-5}	0.0%
1.0×10^{-7}	1.192×10^{-7}	Subnormal, 19% error	0.006554	1.000×10^{-7}	0.0%
2.0×10^{-9}	0.0	**Silent underflow to zero**	0.000131	2.000×10^{-9}	0.0%

23.4.4 Extension 4: Graph Capture and Fusion Compiler

Modern machine learning compilers, including PyTorch Inductor and Google XLA, trace eager execution graphs into static Intermediate Representations to eliminate redundant memory round-trips. By collapsing multi-step elementwise sequences into fused execution kernels, the compiler keeps intermediate values pinned in hardware registers rather than spilling them across the memory bus.

The Systems Bottleneck: In dynamic, define-by-run execution, every mathematical operator is an isolated function call that reads its operands from DRAM, computes results

in registers, and writes the output tensor back to DRAM. Consider an elementwise chain such as:

$$y = (x + \text{bias}) \cdot \text{scale}$$

In eager execution, two separate kernel invocations occur:

1. $t = x + \text{bias}$ (reads x and bias, writes intermediate tensor t to DRAM)
2. $y = t \cdot \text{scale}$ (reads intermediate tensor t and scale, writes output y to DRAM) If `bias` and `scale` are full-size arrays of N elements, this sequence reads $4N$ floating-point words (x, bias, t, scale) and writes $2N$ (t, y), 24 bytes per element across the memory bus. Because modern processor ALUs compute far faster than memory buses can transfer bytes, eager execution is severely memory-bandwidth bound.

Architectural Strategy: A graph compiler intercepts the dynamic define-by-run tape and traces it into a static Intermediate Representation (IR) Abstract Syntax Tree (AST). It then fuses adjacent elementwise operations into a single combined kernel loop (figure 23.1). In the fused loop, the intermediate sum $t = x + \text{bias}$ remains in a CPU or GPU register and is multiplied by scale immediately, writing to DRAM only once.

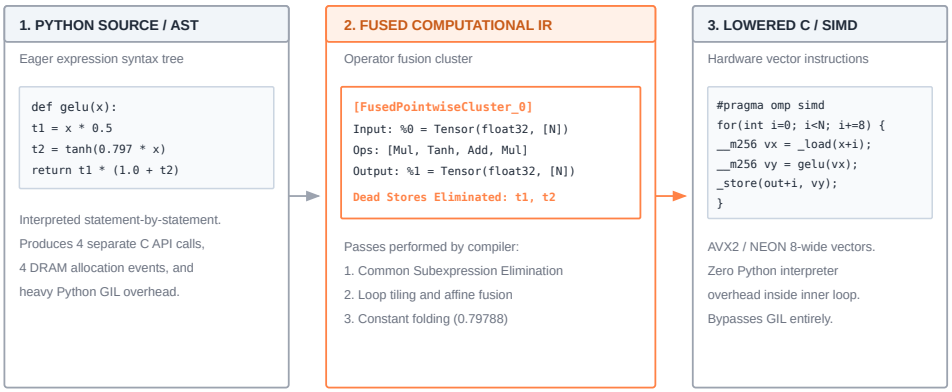


Figure 23.1: **Lowering and fusing an elementwise computation graph.** Eager execution materializes the intermediate sum t in DRAM; the fusion compiler collapses the AST into a single loop, eliminating DRAM traffic.

The Implementation: `compile_graph` traces symbolic `TracedNode` objects into a single Python expression string. It demonstrates graph capture; the emitted function still runs NumPy operators, which allocate the intermediate:

```
def compile_graph(func, *input_names):
    """
    Traces a define-by-run function into one expression and returns it as
    Python source for a function of the named inputs.
    """
    # 1. Tracing phase: pass placeholder nodes through the function
    traced_inputs = [TracedNode(name) for name in input_names]
    output = func(*traced_inputs)

```

```
    # 2. Code generation phase: emit the traced expression as one function
    # (it still runs one operation at a time; a fusing backend would emit a loop)
    code = f"def fused_kernel({', '.join(input_names)}):\n"
    code += f"    # Traced by TinyTorch compile_graph\n"
    code += f"    return {output.name}\n"
    return code
```

The Analytical Scorecard: table 23.4 counts DRAM transfers for eager define-by-run execution and for a hypothetical fused loop that keeps t in registers, on the two-operation elementwise chain ($N = 10^7$ float32 elements, 40 MB per full-size tensor). It assumes `bias` and `scale` are full-size arrays.

Table 23.4: **DRAM traffic counts for an elementwise chain with full-size operands.** A fused loop removes the write and reread of t, 80 MB of 240 MB, a 33% reduction. If `bias` and `scale` are broadcast scalars, eager traffic is 160 MB and fused traffic 80 MB, a 50% reduction. The Python function that `compile_graph` emits still materializes t.

Execution Strategy	DRAM Reads	DRAM Writes	Total DRAM Traffic	Bandwidth Saved
Eager Define-by-Run	x, bias, t, scale (160 MB)	t, y (80 MB)	240 MB	Reference (0.0%)
Fused Single Loop	x, bias, scale (120 MB)	y (40 MB)	160 MB	**33.3% reduction**

23.4.5 Extension 5: Vectorized CPU Execution (SIMD GEMM)

SOURCE CODE MAPPING EXT 21

FILE: tinytorch/extensions/
IMPORT: tinytorch.extensions
SYMBOLS: simd_matmul, mps_matmul, ...

Hand-written, not exported ✓ tested

Source Code Mapping: the extensions live in `tinytorch/extensions/` and are written directly rather than exported from `src/`.

High-performance numerical linear algebra libraries and edge runtimes like llama.cpp achieve near-theoretical CPU compute ceilings by targeting hardware SIMD vector units directly. By restructuring matrix traversals to match memory cache hierarchies and register widths (such as Intel AVX2 or ARM NEON), native kernels overcome interpreter overhead and saturate processor compute capacity.

The Systems Bottleneck: Python is an interpreted language. Executing matrix multiplication using naive triple-nested Python loops incurs catastrophic bytecode evaluation overhead:

```
def matmul_loops(A, B):
    M, K, N = len(A), len(A[0]), len(B[0])
    C = [[0.0] * N for _ in range(M)]
    for i in range(M):
        for k in range(K):
            a_ik = A[i][k]
            for j in range(N):
                C[i][j] += a_ik * B[k][j]
```

```
    return C
```

On a 128 × 128 product, this loop runs at just 0.12 GFLOP/s. For a standard 1024 × 1024 matrix ($2 \times 1024^3 \approx 2.15$ billion FLOPs), Python would take approximately 18 seconds. In contrast, NumPy's underlying BLAS library computes it in 1.33 ms.

Architectural Strategy: To bridge this gap, `simd_matmul` bypasses the Python interpreter via `ctypes` FFI:

1. **Contiguous Pointer Passing:** Extracts raw memory addresses (`.data.ctypes.data_as`) without copying memory.
2. **Cache-Blocked Tiling:** Tiles the matrix traversal in 64 × 64 blocks, ensuring that active working sets (48 KB) reside inside the 128 KB L1 data cache.
3. **SIMD Vectorization:** Arranges inner loops in i, k, j order so that memory access along `b_row[j]` and `c_row[j]` is contiguous, allowing compiler auto-vectorization (AVX2 on x86, ARM NEON on Apple Silicon).[94]
4. **OpenMP Multithreading:** Distributes output tiles across CPU cores using `#pragma omp parallel for collapse(2)`.

Pedagogical Choice: Why ctypes instead of pybind11? Modern PyTorch C++ extensions typically rely on pybind11 and `TORCH_LIBRARY` registration macros to bind native operators into Python. While pybind11 provides seamless class wrapping and exception propagation, it also conceals the actual memory boundary behind layers of C++ template metaprogramming. TinyTorch deliberately chooses Python's built-in `ctypes` foreign function interface: by extracting a raw pointer from the underlying buffer (`tensor.data.ctypes.data_as(ctypes.POINTER(ctypes.c_float))`), the developer directly confronts the C-ABI memory contract, pointer aliasing, and 64-byte cache-line alignment without any framework magic in the middle.

The Implementation:

The Python wrapper passes array pointers to the compiled C++ library:

```
def simd_matmul(a, b):
    """Cache-blocked, SIMD-vectorized matrix multiply (multi-threaded
    ↪  when built with OpenMP).
    """
    is_tensor = _is_tensor(a) or _is_tensor(b)
    a_arr = a.data if _is_tensor(a) else a
    b_arr = b.data if _is_tensor(b) else b

    a_c = np.ascontiguousarray(a_arr, dtype=np.float32)
    b_c = np.ascontiguousarray(b_arr, dtype=np.float32)
    M, K = a_c.shape
    K_b, N = b_c.shape
    if K != K_b:
        raise ValueError(f"Incompatible matrix dimensions: ({M}, {K})
        ↪  x ({K_b}, {N})")

    lib = compile_and_load_simd()
    if lib is None:
        out = np.matmul(a_c, b_c).astype(np.float32)
    else:
```

[94] **SIMD register widths across architectures**: A single Instruction Multiple Data (SIMD) register holds multiple adjacent data values: 128-bit ARM NEON holds 4 float32 values, 256-bit AVX2 holds 8 float32 values, and 512-bit AVX-512 holds 16 float32 values. Modern 2D systolic arrays (such as Google TPUs and Apple Neural Engine) expand vector registers into two-dimensional multiply-accumulate matrices, executing outer products in lock-step across hundreds of processing elements without intermediate register writebacks.

```
        c = np.empty((M, N), dtype=np.float32)
        lib.tinytorch_cpp_gemm(_ptr(a_c), _ptr(b_c), _ptr(c), M, K, N)
        out = c

    if is_tensor:
        from tinytorch.core.tensor import Tensor
        return Tensor(out)
    return out
```

The build system dynamically compiles the C++ kernel on the student's machine with architecture-specific vector flags:

```
def _compile():
    """Build the shared library, with OpenMP if possible. Returns True
    ↪  on success."""
    tmp = _LIB_PATH + f".{os.getpid()}.tmp"
    attempts = []
    omp = _openmp_flags()
    if omp is not None:
        attempts.append((True, _BASE_FLAGS + _arch_flags() + omp))
    attempts.append((False, _BASE_FLAGS + _arch_flags()))

    for openmp, flags in attempts:
        cmd = ["c++"] + flags + [_CPP_SOURCE, "-o", tmp]
        try:
            res = subprocess.run(cmd, capture_output=True, text=True,
↪   encoding="utf-8", errors="replace")
        except OSError as e:  # no compiler on PATH
            _build_info["error"] = str(e)
            return False
        if res.returncode == 0:
            os.replace(tmp, _LIB_PATH)  # atomic, so a concurrent
↪   import never sees half a file
            _build_info.update(built=True, openmp=openmp, command="
↪   ".join(cmd[:-2] + [_LIB_PATH]))
            return True
        _build_info["error"] = res.stderr.strip()
    return False
```

The C++ kernel performs tiled, vectorized matrix multiplication:

```
void tinytorch_cpp_gemm(
    const float* __restrict__ A,
    const float* __restrict__ B,
    float* __restrict__ C,
    int M, int K, int N
) {
    constexpr int BLOCK_SIZE = 64;

    // Zero output buffer
    std::memset(C, 0, sizeof(float) * M * N);
```

```
    #pragma omp parallel for collapse(2) schedule(static)
    for (int sj = 0; sj < N; sj += BLOCK_SIZE) {
        for (int si = 0; si < M; si += BLOCK_SIZE) {
            for (int sk = 0; sk < K; sk += BLOCK_SIZE) {

                int i_max = std::min(si + BLOCK_SIZE, M);
                int j_max = std::min(sj + BLOCK_SIZE, N);
                int k_max = std::min(sk + BLOCK_SIZE, K);

                for (int i = si; i < i_max; ++i) {
                    const float* a_row = &A[i * K];
                    float* c_row = &C[i * N];

                    for (int k = sk; k < k_max; ++k) {
                        float a_ik = a_row[k];
                        const float* b_row = &B[k * N];

                        #pragma omp simd
                        for (int j = sj; j < j_max; ++j) {
                            c_row[j] += a_ik * b_row[j];
                        }
                    }
                }
            }
        }
    }
}
```

A Worked Trace: Consider $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. With $M = K = N = 2$, table 23.5 traces the inner loop step by step. Each row scalar `a_ik` multiplies an entire contiguous row of `B`, accumulating into `c_row`.

Table 23.5: **Inner loops of `tinytorch_cpp_gemm` on a 2 × 2 product.** Row i of C is complete after its last k step, giving rows $(19, 22)$ and $(43, 50)$.

i	k	a_ik	b_row	c_row after the step
0	0	1	$(5, 6)$	$(5, 6)$
0	1	2	$(7, 8)$	$(19, 22)$
1	0	3	$(5, 6)$	$(15, 18)$
1	1	4	$(7, 8)$	$(43, 50)$

The Empirical Scorecard: table 23.6 times a 1024 × 1024 `float32` matrix multiplication across implementations on an Apple M5 Max processor.

Table 23.6: **Matrix multiply, 1024 × 1024, on one Apple M5 Max.** `simd_matmul` built without libomp (one thread); `np.matmul` used Accelerate's default threading. Speedups are ratios of GFLOP/s; outputs agree to within float32 rounding.

Implementation	Time	GFLOP/s	Speedup vs. Python
`matmul_loops`, Python (scaled from 128 × 128)	≈17.9 s	0.12	1.0× (Baseline)
C++ kernel, vectorizer switched off	336 ms	6.4	53×
`simd_matmul` (Vectorized C++)	99.4 ms	21.6	180×
`tiled_matmul` (NumPy blocks, Chapter 18)	18.5 ms	116	967×
`np.matmul` (Accelerate/BLAS)	1.33 ms	1,611	13,425×

Compiling the loop gains a factor of 50 over Python, and auto-vectorization adds another factor of 3.4. Notice that vendor BLAS (`np.matmul`) remains 75 times faster than the basic tiled kernel: tuned production libraries employ register-level micro-kernels and memory packing to keep sub-blocks resident in CPU registers rather than reloading from L1 cache.

23.4.6 Extension 6: Operator Fusion in Silicon (`simd_fused_bias_gelu`)

The Systems Bottleneck: In transformer feedforward networks, adding a bias vector and applying the Gaussian Error Linear Unit (GELU) activation is executed as two consecutive steps:

$$y = \text{GELU}(x + \text{bias})$$

In NumPy, `x + bias` writes the full intermediate tensor to DRAM and GELU reads it back. For memory-bound elementwise operations, DRAM memory bus bandwidth limits execution throughput.

Architectural Strategy: The C++ kernel `simd_fused_bias_gelu` fuses the bias addition, polynomial approximation, and hyperbolic tangent into a single pass over memory. Each element is loaded from X, added to the corresponding bias column, transformed through the polynomial approximation, and written directly to the output array. The intermediate sum $x + \text{bias}$ resides exclusively in processor registers. The listing calls scalar `std::tanh`; on macOS the build adds `-fveclib=Accelerate`, which lets the compiler substitute a vector tanh. That flag is the difference between the 22.1 ms and 3.06 ms rows, so the $8.6\times$ gain may not carry over to a build without a vector tanh, such as a default Linux build.

The Implementation:

```
void tinytorch_cpp_fused_bias_gelu(
    const float* __restrict__ X,
    const float* __restrict__ bias,
    float* __restrict__ Y,
    int total_elements,
    int inner_dim
```

```
) {
    constexpr float SQRT_2_OVER_PI = 0.7978845608028654f;
    constexpr float COEFF = 0.044715f;

    #pragma omp parallel for schedule(static)
    for (int idx = 0; idx < total_elements; ++idx) {
        int col = idx % inner_dim;
        float val = X[idx] + bias[col];
        float cube = val * val * val;
        float inner = SQRT_2_OVER_PI * (val + COEFF * cube);

        float tanh_val = std::tanh(inner);
        Y[idx] = 0.5f * val * (1.0f + tanh_val);
    }
}
```

The Empirical Scorecard: table 23.7 evaluates performance on a 4096 × 768 `float32` activation tensor (12.6 MB), representing the output of a 768-wide transformer layer across 4,096 tokens.

Table 23.7: **Bias plus GELU, 4096 × 768, on one Apple M5 Max.** Outputs agree to within 5×10^{-7}.

Implementation	Time	Speedup
`fused_gelu` (Chapter 18), after `x + bias`	26.5 ms	1.0×
NumPy expression with `z * z * z` in place of `z ** 3`	4.33 ms	6.1×
C++ fused kernel, scalar `std::tanh`	22.1 ms	1.2×
`simd_fused_bias_gelu` (vectorized tangent)	3.06 ms	8.6×
Piecewise breakdown: `x ** 3` alone	16.6 ms	-
Piecewise breakdown: `x * x * x` alone	0.46 ms	-
Piecewise breakdown: `x + bias` alone	0.37 ms	-
Piecewise breakdown: `np.tanh(x)` alone	2.10 ms	-

Notice the critical systems lesson: rewriting `z ** 3` as `z * z * z` in NumPy yielded a 6× speedup by avoiding general exponentiation routines. Fusing the operation into compiled C++ with vectorized tangent routines yielded a total 8.6× **speedup** over the unfused baseline.

23.4.7 Extension 7: Hardware Accelerator Offloading (MPS & Triton)

Modern deep learning runtimes scale beyond general-purpose CPUs by targeting specialized parallel accelerators, using backend frameworks like Apple Metal Performance Shaders (MPS) and OpenAI Triton. Maximizing accelerator efficiency requires balancing high-throughput parallel compute against the latency of host-device synchronization and memory bus transfers.

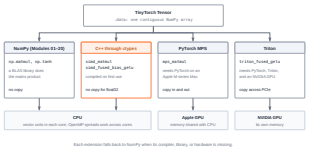
The Systems Bottleneck: Specialized hardware accelerators (such as Apple Silicon GPUs or NVIDIA GPUs) provide massive parallel throughput. However, moving data to and from the accelerator incurs data transfer latency and bus transfer overhead.

Architectural Strategy:

1. **Apple MPS GEMM:** Runs `mps_matmul` through PyTorch's MPS backend (requires `torch`) on the Apple Silicon GPU, not through a C FFI.
2. **OpenAI Triton Fused GELU:** Uses SPMD block-level GPU kernel programming (`triton_fused_gelu`) for NVIDIA GPUs.

The Implementation:

The Apple MPS wrapper queues matrix multiplication on the Apple GPU:



Dispatch Pathways: TinyTorch tensors can be evaluated via NumPy, compiled C++ SIMD, Apple MPS, or NVIDIA Triton.

```
def mps_matmul(a, b):
    """Matrix multiply on the Apple GPU via MPS (the GPU, not the
    ↪   Neural Engine)."""
    is_tensor = _is_tensor(a) or _is_tensor(b)
    a_arr = a.data if _is_tensor(a) else a
    b_arr = b.data if _is_tensor(b) else b

    if not _HAS_MPS:
        out = np.matmul(a_arr, b_arr).astype(np.float32)
        if is_tensor:
            from tinytorch.core.tensor import Tensor
            return Tensor(out)
        return out

    a_c = np.ascontiguousarray(a_arr, dtype=np.float32)
    b_c = np.ascontiguousarray(b_arr, dtype=np.float32)
    M, K = a_c.shape
    K_b, N = b_c.shape
    if K != K_b:
        raise ValueError(f"Incompatible matrix dimensions: ({M}, {K})
        ↪   x ({K_b}, {N})")

    a_t = torch.from_numpy(a_c).to("mps")
    b_t = torch.from_numpy(b_c).to("mps")
    c_t = torch.matmul(a_t, b_t)
    out = c_t.cpu().numpy().astype(np.float32)

    if is_tensor:
        from tinytorch.core.tensor import Tensor
        return Tensor(out)
    return out
```

The Triton kernel executes block programs across streaming multiprocessors on NVIDIA hardware:

```
@triton.jit
def _fused_bias_gelu_kernel(
    x_ptr,
    bias_ptr,
```

```
    out_ptr,
    total_elements,
    inner_dim,
    BLOCK_SIZE: tl.constexpr,
):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < total_elements

    # Compute bias offset (modulo inner_dim)
    bias_offsets = offsets % inner_dim

    # Adjacent programs read adjacent addresses, so the loads coalesce
    x = tl.load(x_ptr + offsets, mask=mask)
    bias = tl.load(bias_ptr + bias_offsets, mask=mask)

    # The whole formula stays in registers
    val = x + bias
    sqrt_2_over_pi = 0.7978845608028654
    coeff = 0.044715
    inner = sqrt_2_over_pi * (val + coeff * val * val * val)
    # tanh(z) = 2*sigmoid(2z) - 1; tl.sigmoid exists in every Triton
    # release, while the libdevice tanh import path has moved between
    ↪   them.
    tanh_val = 2.0 * tl.sigmoid(2.0 * inner) - 1.0
    out = 0.5 * val * (1.0 + tanh_val)

    # One write of the result; x + bias is never stored
    tl.store(out_ptr + offsets, out, mask=mask)
```

Count the work an operation does for each byte it moves across the bus:

- **Matrix multiply ($n \times n$ by $n \times n$ in float32):** $2n^3$ operations on $3n^2$ values of 4 bytes each:

$$\text{Arithmetic Intensity} = \frac{2n^3}{12n^2} = \frac{n}{6} \text{ FLOP/byte}$$

  For $n = 256$, arithmetic intensity is ≈ 43 FLOP/byte. For $n = 4096$, it reaches ≈ 683 FLOP/byte. The larger the product, the more arithmetic covers transfer latency.

- **Bias-plus-GELU (any size):** Approximately a dozen arithmetic operations per element on 8 bytes transferred (4 bytes read, 4 bytes written):

$$\text{Arithmetic Intensity} \approx \frac{12}{8} = 1.5 \text{ FLOP/byte}$$

  The ratio remains constant regardless of array size. A lone GELU call almost never pays for its own device transfer round-trip.

The Empirical Scorecard: table 23.8 compares `mps_matmul` against `np.matmul` across matrix dimensions n.

Table 23.8: **Matrix multiply on Apple GPU through MPS vs. CPU.** At small sizes ($n = 256$), transfer overhead makes GPU offloading $30\times$ slower; at large sizes ($n = 4096$), GPU throughput makes it $6\times$ faster even including round-trip copies.

Matrix Dimension (n)	`np.matmul` (CPU)	`mps_matmul` (GPU with copies)	GPU Kernel Execution Alone
256	0.02 ms	0.72 ms	0.21 ms
1024	1.28 ms	1.06 ms	0.37 ms
2048	10.4 ms	2.53 ms	1.39 ms
4096	86.9 ms	14.2 ms	10.1 ms

As predicted by the napkin math ($n/6$ FLOP/byte), offloading small matrices loses performance to bus latency. As n reaches 4096, arithmetic intensity dominates, and the GPU executes $6\times$ **faster** than the CPU.

23.5 Evaluating an Extension: From Micro-benchmarks to End-to-End Milestones

Authoring a new systems extension requires rigorous verification of correctness, performance benchmarking, and evaluation of real-world impact. In production systems engineering, an optimization cannot be evaluated in a vacuum. TinyTorch provides a structured **3-Tier Evaluation Ladder** from unit verification to full end-to-end model training:

```
[ Tier 3: Custom Ingestion ]
  └─ Train with Trainer on custom data via DataLoader
     ▲
[ Tier 2: Historical Milestones ]
  └─ Macro-benchmark on Milestone 4 (CNN) & 5 (GPT)
     ▲
[ Tier 1: Micro-benchmarking ]
  └─ Unit parity (pytest) & isolated latency (precise_timer)
```

23.5.1 Tier 1: Numerical Correctness and Synthetic Micro-benchmarks

Before measuring execution time, numerical correctness must be certified. An incorrect operator must never be optimized:

1. **Unit Test Parity:** Write unit tests in `tests/extensions/test_<name>.py`. Compare forward outputs and backward gradients against a high-precision reference (such as float64 NumPy):

```python
# Verify outputs agree within float32 tolerance
assert np.allclose(
    out_extension.data, out_reference, atol=1e-5
)
```

2. **Synthetic Micro-benchmarks:** Use Chapter 20's `precise_timer` (for latency) and Chapter 15's `Profiler.measure_memory` (for traced peak allocation) to isolate the operator across power-of-two shapes ($N \in \{128, 256, 512, 1024, 2048\}$):

```
from tinytorch.perf.benchmarking import precise_timer
from tinytorch.perf.profiling import Profiler

with precise_timer() as t:
    y = my_extension_op(x)
print(f"Latency:     {t.elapsed * 1000:.2f} ms")

mem = Profiler().measure_memory(model, (1, 1024))
print(f"Peak traced: {mem['peak_memory_mb']:.2f} MB")
```

   Quantify execution speedup, memory reduction, and arithmetic intensity.

23.5.2 Tier 2: Macro-benchmarking on Stock Milestones

Isolated micro-benchmarks do not capture end-to-end training dynamics, cache contention, or framework overhead. TinyTorch provides complete, reproducible historical milestones under `milestones/` to evaluate extensions in real network architectures:

1. **Milestone 4: Convolutional Vision (TinyDigits):**
   - *Script:* `milestones/04_1998_cnn/01_lecun_tinydigits.py`
   - *Target Extensions:* Custom activations, loss scalers, or optimizers.
   - *Workflow:* Substitute the custom activation or optimizer into the LeNet-style architecture. Train on the TinyDigits classification task and compare epoch training time and final validation accuracy against standard SGD/Adam.
2. **Milestone 5: Generative Transformers (TinyShakespeare & TinyCopilot):**
   - *Scripts:* `milestones/05_2017_transformer/01_tinygpt_shakespeare.py` (prose), `03_tinycopilot.py` (Python code)
   - *Target Extensions:* Low-rank adaptation, activation checkpointing, custom attention, compiler gating, or fused kernels.
   - *Workflow for LoRA:* Freeze the pretrained GPT weights and replace attention projection layers with `LoRALinear`. Fine-tune on Shakespearean text or Python code and verify that the adapter achieves comparable cross-entropy loss while updating less than 2% of the parameters.
   - *Workflow for Activation Checkpointing:* Wrap each `TransformerBlock` in `checkpoint()`. Measure traced peak memory with `Profiler.measure_memory` to demonstrate that batch size can be doubled without triggering out-of-memory errors.
3. **Classroom MLPerf-style check (Chapter 20):**
   - *Module:* `tinytorch.perf.benchmarking`
   - *Workflow:* Run `MLPerf().run_standard_benchmark(model, "keyword_spotting", test_inputs=inputs, labels=labels)` to report p_{50}, p_{90} (gated), and p_{99} latency, inferences per second, and accuracy against a classroom threshold.

23.5.3 Tier 3: Custom Datasets and Pipeline Ingestion

Evaluating an extension on domain-specific workloads integrates directly with TinyTorch's ingestion pipeline:

1. **Vision or Tabular Data:** Subclass `Dataset` from `tinytorch.core.dataloader`, implement `__len__()` and `__getitem__()`, and instantiate a `DataLoader`:

```
from tinytorch.core.dataloader import (
    Dataset,
    DataLoader,
)

class CustomArrayDataset(Dataset):
    def __init__(self, features, targets):
        self.x, self.y = features, targets
    def __len__(self):
        return len(self.x)
    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

loader = DataLoader(
    CustomArrayDataset(x_data, y_data),
    batch_size=32,
    shuffle=True,
)
```

2. **Text Corpora:** Pass raw text to `Tokenizer` from `tinytorch.core.tokenization` to construct tokenized input-target sequences for autoregressive language modeling.
3. **End-to-End Training:** Pass the loader, model (with the active extension), optimizer, and loss function to `Trainer` (`tinytorch.core.training.Trainer`). Calling `trainer.train_epoch(loader)` once per epoch, and `trainer.evaluate(val_loader)` after it, reports loss and accuracy.

23.6 Building on the Foundations: Next-Level Extensions

These seven extensions span key architectural boundaries, establishing clear templates for deeper systems work. Rather than attempting disconnected projects, the most effective path to systems mastery is building directly upon these canonical extension points.

Extending TinyTorch uses intentionally clean and minimalist interfaces:

- To implement custom forward/backward math or recomputation patterns, subclass `Function` and implement `forward` and `backward`.
- To inject custom parameter adaptations or layer hooks, subclass `Layer` and register parameter tensors.
- To bypass the Python interpreter for custom hardware or low-level kernels, pass contiguous buffer pointers (`tensor.data.ctypes.data_as(...)`) across the `ctypes` foreign function interface.
- To simulate hardware execution, trace tensor shapes and access streams into architectural modeling tools.

Each project below addresses a real systems bottleneck and demonstrates how to resolve it or interface TinyTorch with external systems infrastructure:

23.6.1 Dynamic Loss Scaling

- **Systems Limitation:** The stock `LossScaler` applies a fixed scale factor ($S = 65,536$). If training encounters large activations, gradient magnitudes can spike, causing multiplied values to exceed the float16 maximum ($65,504$) and overflow to `inf` or `nan`.
- **The Project:** Implement adaptive loss scaling. After calling `loss.backward()`, scan parameter gradients for `np.isinf` or `np.isnan`. If an overflow is detected:
  1. Skip the `optimizer.step()` call to prevent corrupting parameter weights.
  2. Halve the scale factor ($S \leftarrow S/2$).
  3. Reset a consecutive-success step counter. If M consecutive steps complete without any overflow (e.g., $M = 2000$), double the scale factor ($S \leftarrow 2S$) to maintain maximum numerical dynamic range. Evaluate on Milestone 4 or 5 in FP16.

23.6.2 Register Micro-Tiling

- **Systems Limitation:** Stock `simd_matmul` tiles matrices at the L1 cache level (64×64). However, the innermost loop accumulates scalar products back into memory, keeping CPU vector execution units waiting on L1 data cache load-store latency.
- **The Project:** Implement an 8×4 register micro-kernel in `cpp_simd_gemm.cpp`. Unroll the inner loop so that an 8×4 block of intermediate accumulation registers is kept alive in CPU vector registers (AVX2/NEON) throughout the K-dimension reduction, storing results back to DRAM only when the tile is complete. Benchmark the micro-kernel against `simd_matmul` and quantify the GFLOP/s improvement.

23.6.3 Model-Wide LoRA Injection Hook

- **Systems Limitation:** Stock `LoRALinear` requires manually instantiating adapter layers in the model definition. In deep models with dozens of projection matrices, manual replacement is tedious and fragile.
- **The Project:** Author an automated helper function `inject_lora(model, target_layers, rank=8)`:
  1. Recursively traverse the model's module hierarchy.
  2. For each layer matching a target name, replace the existing `Linear` layer with a corresponding `LoRALinear` instance initialized with the pretrained weights.
  3. Set `requires_grad = False` on all non-adapter parameters. Test `inject_lora` on Milestone 5's GPT model to demonstrate fine-tuning a full transformer with a single function call.

23.6.4 FlashAttention and SRAM-Aware Tiling

- **Systems Limitation:** Standard attention (Chapter 12) evaluates $O = \text{softmax}(QK^T/\sqrt{d_k})V$ by materializing the full intermediate $S \times S$ attention score matrix in high-bandwidth memory (DRAM). For a sequence length of $S = 4096$ with 32 heads, storing the attention score and weight matrices requires about 4 GB per sequence in float32. The operation is severely memory-bandwidth

bound: the processor spends most of its execution cycles reading and writing the $S \times S$ matrix across the memory bus rather than performing floating-point math.

- **The Project:** Implement a custom `FlashAttentionFunction` subclassing `Function`. Tile the Q, K, V matrices into blocks ($B_r \times d$ and $B_c \times d$) sized to fit entirely inside fast on-chip memory (CPU L1/L2 cache or GPU shared memory). Instead of materializing the $S \times S$ matrix, use the **online softmax** algorithm (maintaining running maximum scalars m_i and running sum scalars l_i per row block) to incrementally update the output accumulator O_i as column blocks of K and V stream through:

$$m_i^{\text{new}} = \max(m_i, \max(S_i)), \quad l_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} l_i + \sum e^{S_i - m_i^{\text{new}}}$$

$$O_i^{\text{new}} = \text{diag}(e^{m_i - m_i^{\text{new}}}) O_i + e^{S_i - m_i^{\text{new}}} V_j$$

Once all column blocks have been processed, normalize the output accumulator by dividing each row by its final running sum:

$$O_i^{\text{final}} = \text{diag}(l_i^{\text{final}})^{-1} O_i$$

In the backward pass, recompute the attention weights on-the-fly from the cached Q, K, V blocks, completely eliminating the $O(S^2)$ memory footprint. Benchmark the tiled attention function against stock `attention.py` across sequence lengths $S \in \{512, 1024, 2048, 4096\}$ using `precise_timer` and `Profiler.measure_memory`.

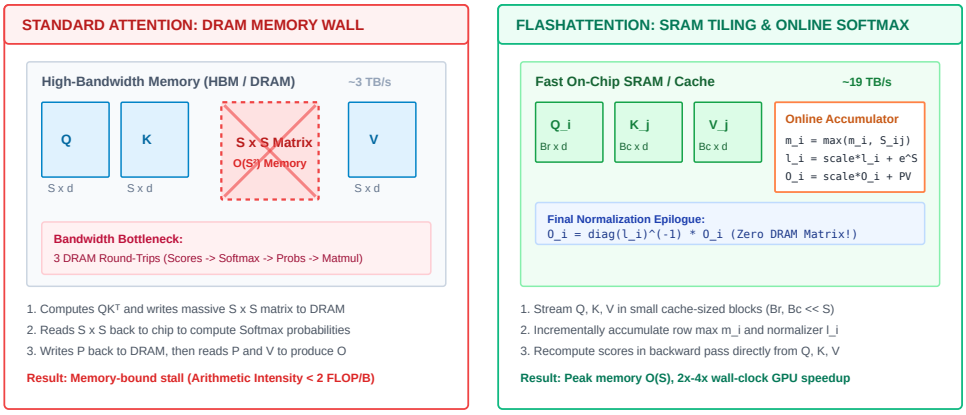
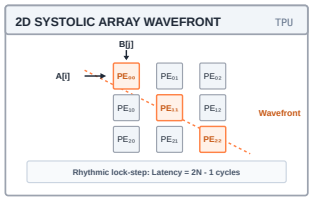


Figure 23.2: **FlashAttention SRAM-aware tiling vs. standard attention DRAM wall**: Standard attention incurs multiple high-latency DRAM round-trips to materialize the full $S \times S$ score and weight matrices. FlashAttention partitions Q, K, V into small SRAM-sized blocks ($B_r, B_c \ll S$) and employs online softmax updates directly in on-chip fast memory, completely eliminating the $O(S^2)$ DRAM memory footprint.

23.6.5 Hardware Simulation and Memory Tracing (SCALE-Sim Integration)

- **Systems Limitation:** Profiling on a host CPU or GPU measures total runtime, but cannot answer architectural design questions: How many multiply-accumulate (MAC) units in a 2D systolic array are actually used? Would doubling the on-chip SRAM buffer eliminate DRAM stall cycles? What array aspect ratio (128×128 vs. 256×64) achieves optimal utilization for a specific model?



2D Systolic Wavefront: Processing elements stream activation and weight operands in rhythmic lock-step, achieving peak compute density with $2N - 1$ cycle latency.

- **The Project:** Build an architectural memory-tracing extension that interfaces TinyTorch with **SCALE-Sim** (Systolic CNN Accelerator Simulator) or an equivalent cycle-accurate hardware simulator:
  1. Author a tracer hook or custom `SimulatedLinear` layer that intercepts GEMM operations in TinyTorch models.
  2. For every matrix multiplication $Y = XW$, extract the target dimensions (M, K, N) and generate memory read/write address request streams for matrices A, B, and C.
  3. Export a SCALE-Sim-compatible workload topology file listing layer dimensions and dataflows (e.g., Weight Stationary `WS`, Output Stationary `OS`, or Input Stationary `IS`).
  4. Run the SCALE-Sim simulation from Python via `subprocess` to obtain cycle counts, buffer hit rates, and DRAM bandwidth requirements. Compare the simulated cycles and bandwidth stalls of `TinyGPT` projections across different array dimensions (32×32 edge accelerator vs. 128×128 datacenter systolic array).

23.6.6 The Data Systems Extension: Offline Micro-Datasets and Compiler Gating (TinyVerse)

- **Systems Limitation:** Real-world training loops frequently stall on storage I/O, network deserialization, and multi-process IPC serialization overhead. Furthermore, in generative code models, traditional NLP metrics (like BLEU, ROUGE, or character cross-entropy loss) suffer from the "syntactic illusion": a generated function can achieve low cross-entropy loss while failing basic compiler parsing due to a single missing colon, unmatched delimiter, or invalid indentation.
- **The Project:** Build an end-to-end data systems extension modeled after the TinyVerse dataset architecture:
  1. **Contiguous Binary Layouts:** Package training samples into contiguous memory-mapped serialization buffers (`.pkl` / `.npy` / flat binary) to allow instantaneous, zero-copy batch slicing directly into `Tensor` without CPython object allocation overhead.
  2. **Closed-World Algorithmic Curricula:** Hand-curate and programmatically generate closed-world, syntactically dense micro-corpora (such as `TinyPy` algorithms or `TinyTalks` concept Q&A) sized under 100 KB that saturate single-core CPU training in under 60 seconds without third-party network downloads.
  3. **Compiler-in-the-Loop Evaluation:** Author a live verification hook that streams generated model tokens directly into Python's `ast.parse()` compiler, tracking the **AST Validity Rate (%)** as a strict compiler gate rather than relying solely on loss. Evaluate this data pipeline against Milestone 05 Part 3 (TinyCopilot, `03_tinycopilot.py`), scoring only prompts absent from the training text so the rate measures learned syntax rather than recall.

23.7 From TinyTorch to Production

The mechanisms demonstrated in TinyTorch extensions reflect the exact techniques deployed in production deep learning engines. What differs in production is the degree of surrounding infrastructure:

Table 23.9: **Architectural differences between TinyTorch extensions and production frameworks.** TinyTorch exposes the bare mathematical and memory contracts; production adds compiler generation and dispatch orchestration.

Dimension	TinyTorch Extension	Production Framework (PyTorch/JAX)
Calling Compiled Code	`ctypes` FFI with raw memory pointers	C++ operators registered with framework dispatcher (`TORCH_LIBRARY`)
Matrix Multiplication	Tiled, auto-vectorized C++ loop	Vendor-tuned BLAS (cuBLAS, CUTLASS, Accelerate, oneMKL)
Elementwise Fusion	Hand-written C++ and Triton kernels	JIT compilers (`torch.compile`/Inductor) generating Triton code
Accelerator Memory	Copied in and out per operator call	Resident on device across layers throughout entire training loop
Autograd Integration	Manual `Function` forward/backward	Dispatcher-registered backward formulas with automated symbolic differentiation

Production systems build atop these contracts through three key architectural evolutions:

1. **Vendor Libraries and Polyhedral Generation:** While `simd_matmul` relies on fixed 64×64 cache tiling, production GEMM runtimes dispatch to vendor-tuned microkernels (NVIDIA cuBLAS, CUTLASS, or Apple Accelerate). These libraries dynamically select tile sizes, pack submatrices into contiguous memory layouts to avoid cache associativity thrashing, and schedule register allocations around pipeline latency.
2. **Persistent Device Memory Pipelines:** TinyTorch extensions copy data between host memory and accelerator buffers on every invocation. Industrial training loops allocate tensor storage directly on high-bandwidth device memory (HBM), keeping weights, activations, and gradient accumulators resident on-chip across thousands of successive training iterations. Host communication is restricted to asynchronous checkpointing and metric reporting.
3. **Automated Compiler Lowering:** Rather than manually authoring separate AST tracers and custom kernels for each operator permutation, production frameworks rely on automated JIT compilation pipelines that synthesize optimal kernels directly from execution graphs.

In TinyTorch, calling native code relies on the Python `ctypes` Foreign Function Interface (FFI) and explicit manual AST pattern matching. While pedagogically transparent,

this approach incurs Python-to-C dispatch latency (~1 to 5 microseconds per call) and requires manual authoring for every fused operator variant.

Modern production engines bridge this gap through multi-tiered compiler infrastructure:

- **Centralized Operator Dispatchers:** Frameworks like PyTorch route operations through a unified runtime dispatcher (`c10::Dispatcher`). The dispatcher inspects tensor metadata (device, memory layout, and autodiff state) to select the optimal implementation without Python interpreter overhead. Custom C++ kernels register once via declarative macros (`TORCH_LIBRARY`) and automatically integrate with autograd and multi-device backends.
- **Dynamic Graph Capture (TorchDynamo):** Imperative Python code is converted into computational graphs at runtime using frame evaluation hooks (PEP 523). TorchDynamo intercepts frame execution, parses standard Python bytecode, and constructs an intermediate FX graph while maintaining fallback compatibility for non-tensor Python control flow.
- **Intermediate Representation Lowering (TorchInductor):** The captured FX graph lowers to a loop-level intermediate representation. TorchInductor analyzes data dependencies, groups compatible elementwise and reduction nodes, allocates shared memory buffers, and generates vendor-specific code, emitting optimized Triton kernels for GPUs and vector-parallel C++ loops for CPUs.

The foundational principle remains invariant across all layers: whether hand-crafted through FFI or synthesized automatically by an optimizing JIT compiler, peak systems efficiency requires eliminating redundant DRAM round-trips, maximizing compute density in fast SRAM and registers, and aligning execution strides with hardware memory hierarchies.

23.8 Summary

The extensions ecosystem expands TinyTorch beyond its core 20 modules into the realm of production systems performance:

- `LoRALinear` freezes base weights and trains rank-r adapters, reducing Adam optimizer state memory by more than 98%.
- `checkpoint` drops intermediate forward activations and recomputes them during backpropagation, reducing peak activation memory scaling from $O(L)$ to $O(\sqrt{L})$.
- `LossScaler` shifts gradient values into the normalized exponent range of IEEE 754 float16, preventing catastrophic gradient underflow during mixed-precision training.
- `compile_graph` traces define-by-run computation into a single fused Python expression, the capture step that a fusion compiler lowers into one loop; for the example chain such a loop would remove a third of DRAM traffic with full-size operands, or half with broadcast scalars.
- `simd_matmul` and `simd_fused_bias_gelu` pass raw memory pointers across `ctypes` to compiled C++ kernels, using cache tiling and CPU SIMD vector registers to deliver up to a $180\times$ speedup over interpreted Python and an $8.6\times$ speedup over unfused activations.

- `mps_matmul` and `triton_fused_gelu` demonstrate hardware accelerator offloading, validating through napkin math why arithmetic intensity ($n/6$ FLOP/byte) dictates when offloading pays for data transfer costs.

The central lesson of this chapter is the empirical invariant of systems engineering: **an optimization is only verified when its numerical outputs match a trusted reference and its performance is measured directly on target hardware.**

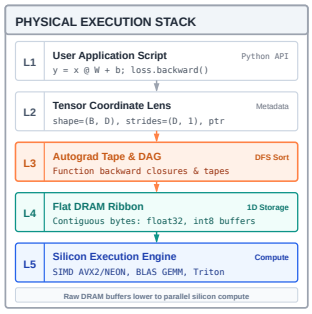
23.9 Further Reading

- **Goto and van de Geijn** (2008). "Anatomy of high-performance matrix multiplication." *ACM TOMS*. Register blocking and packing, the two techniques that separate `simd_matmul` from a tuned library.
- **Van Zee and van de Geijn** (2015). "BLIS: a framework for rapidly instantiating BLAS functionality." *ACM TOMS*. How a matrix library is built from one small hand-tuned kernel.
- **Hu et al.** (2021). "LoRA: Low-Rank Adaptation of Large Language Models." *ICLR*. The mathematical foundation of parameter-efficient adapter fine-tuning.
- **Chen et al.** (2016). "Training Deep Nets with Sublinear Memory Cost." *arXiv*. The original formulation of activation checkpointing.
- **Micikevicius et al.** (2018). "Mixed Precision Training." *ICLR*. The analysis of float16 exponent ranges and loss scaling.
- **Tillet, Kung, and Cox** (2019). "Triton: an intermediate language and compiler for tiled neural network computations." *MAPL*. The design behind the block programs in `triton_gelu.py`.
- **Dao et al.** (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *NeurIPS*. IO-aware tiling algorithms for modern attention.

Conclusion: Building Systems That Learn

We began TinyTorch with a fundamental premise: you cannot truly understand a machine learning system by treating it as a black box. Deep learning frameworks like PyTorch, JAX, and TensorFlow are often taught through high-level API calls, invoking `nn.Linear`, calling `.backward()`, or wrapping models in `torch.compile()`. To the developer, tensors appear as magical multidimensional entities, autograd feels like symbolic calculus, and GPU acceleration seems like free speed.

Over the course of twenty modules, seven historical milestones, and three synthesis chapters, you took that magic apart. You built a working deep learning framework using only Python and NumPy memory buffers. By refusing to take abstractions for granted, you found what a machine learning framework is: **a specialized operating system that manages tensor memory, schedules dynamic directed acyclic graphs, and keeps parallel hardware busy.**



The Complete System Stack: From physical DRAM bytes to autoregressive transformers, built entirely from scratch.

What We Built: The Complete Arc

The architecture of TinyTorch unfolded across four deliberate tiers of progressive disclosure:

1. The Core Engine (Part I)

Before a single neuron could fire, you had to define the physical substrate of machine learning:

- **Tensors and Strides (Chapter 1)**: You proved that multidimensional tensors do not exist in physical hardware. Memory is a flat, one-dimensional ribbon of byte addresses in DRAM. A tensor is simply a coordinate lens defined by a pointer, a shape tuple, and a stride vector. You worked out the stride arithmetic that turns an index into an offset, saw how a zero-valued stride expresses broadcasting without moving bytes, and separated owning a buffer from laying it out contiguously. TinyTorch's own constructor then takes the opposite path from a production framework and copies on every construction, which is the choice that made the ownership rule simple enough to reason about and the cost visible enough to measure.
- **Differentiable Layers and Numerical Stability (Chapter 2 through Chapter 3, Chapter 4)**: You constructed linear projections, layer normalizations, and loss functions. In doing so, you confronted IEEE 754 floating-point realities, engineering numerically stable Softmax and LogSumExp formulations that prevent catastrophic exponential overflow.
- **Autograd and Dynamic Graph Execution (Chapter 6)**: You implemented automatic differentiation from first principles. Rather than symbolic algebra or numeri-

cal perturbation, you built a reverse-mode automatic differentiation engine using dynamic execution tapes. Gradients flow backwards through an acyclic computational graph via depth-first topological sorting, correctly handling fan-out branches and in-place gradient accumulation.

- **Optimization and Training (Chapter 7, Chapter 8, Synthesis I)**: You engineered the `AdamW` optimizer, deriving first- and second-moment exponential moving averages with bias corrections and decoupled weight decay. You closed the foundation tier by training `DigitMLP` from scratch to classify handwritten digits, verifying that your home-built autograd engine trains a working classifier.

2. Deep Architectures (Part II)

With the mathematical and systems engine verified, you scaled the framework to modern perception and language workloads:

- **Spatial Convolutions (Chapter 9)**: You implemented 2D spatial feature extraction with the seven nested loops that pick an output position and form its dot product, and you priced what those loops cost. Chapter 18 then built the `im2col` lowering that reduces the same filtering to one high-performance general matrix multiplication (GEMM), and timed it against your loops.
- **Tokenization and Discrete Semantics (Chapter 10, Chapter 11)**: You built a Byte-Pair Encoding (BPE) tokenizer from the ground up, learning how vocabulary tables and byte merges convert raw unicode strings into discrete token IDs, and how embedding tables project discrete categorical indices into continuous vector spaces.
- **Attention and Generative Transformers (Chapter 12, Chapter 13, Synthesis II)**: You constructed causal multi-head self-attention, wrestling with coordinate axis permutations (`[Batch, Seq, Heads, Dim]`), lower-triangular causal masking with $-\infty$ additive offsets, and scaled dot-product routing. In Milestone 05, you validated basic attention sequence routing on synthetic tasks. You then stacked these blocks into `TinyGPT` (our performance-oriented flagship model), establishing its baseline autoregressive decoding loop on Shakespeare, generating Python source code in TinyCopilot verified by compiler gates, and exposing its quadratic recomputation bottleneck.

3. Systems, Acceleration, and Verification (Part III)

Building a model is only half the systems discipline; the other half is making it fast, lean, and deployable:

- **Profiling and the Measurement Invariant (Chapter 15)**: You learned that unmeasured code is unoptimized code. You built a framework profiler that uses warmup passes and repeated medians to reduce scheduling jitter, isolating true system bottlenecks from measuring artifacts.
- **Quantization and Compression (Chapter 16, Chapter 17)**: You mapped 32-bit floating-point parameters into 8-bit integer affine codes with scale factors and zero points. You derived why symmetric quantization eliminates integer accumulation cross-terms, and proved why naive magnitude pruning yields zero speedup unless paired with compressed index representations like Compressed Sparse Row (CSR).
- **Kernel Acceleration and KV Cache Memoization (Chapter 18, Chapter 19)**: You broke the memory bandwidth wall. You implemented loop tiling to fit active

matrix blocks into L1/L2 cache hierarchies, fused activation chains to eliminate DRAM round trips, and built an autoregressive Key-Value Cache that removes the re-projection of earlier tokens, turning per-step projection work from $O(N)$ into $O(1)$ while each new query still attends over the growing prefix.

- **The MLPerf Optimization Olympics and the Pareto Frontier (Chapter 21, Synthesis III)**: You measured optimized candidates against their baselines under MLPerf-style discipline, saw why a single-metric ranking rewards the degenerate `ZeroModel`, and computed which candidates lie on the Pareto frontier, comparing them on accuracy retention floors ($R \geq 0.99$), memory compression ratios (C), and measured inference speedup (S).

4. Extensions (Part IV)

Finally, in Chapter 23, you stepped below the Python interpreter:

- You crossed the language boundary via `ctypes`, passing contiguous float32 buffers directly into compiled C++ SIMD loops on ARM NEON and x86 AVX2.
- You read a Triton kernel as a block-level SPMD program, with `tl.program_id` selecting a block and a mask switching off the lanes that would run past the end of the array. Chapter 23 notes that the coalesced loads come from adjacent programs reading adjacent addresses; the memory scheduling around them is what the Triton compiler generates for you, which is the point: the kernel in that chapter is the kind a compiler writes.
- You offloaded matrix multiplication to the Apple GPU through PyTorch's MPS backend, uncovering the exact latency break-even point where compute intensity covers the fixed overhead of device memory transfer.

Five Enduring Principles of Machine Learning Systems

As you step from TinyTorch into production frameworks and custom hardware, five universal principles will govern every system you design:

Tensors are coordinate lenses over linear byte ribbons. In modern hardware, floating-point arithmetic is virtually free, but moving bytes across memory buses is exceptionally expensive. High-performance machine learning engineering is rarely about optimizing arithmetic; it is about maximizing arithmetic intensity, structuring algorithms so that every byte loaded from DRAM into SRAM or register files is reused dozens of times before eviction.

Automatic differentiation is not magic and not symbolic calculus. It is an operational recording of primitive operations executed during a forward pass. Every dynamically captured node represents an allocated memory buffer that must remain pinned until its corresponding backward vector-Jacobian product is computed. Understanding this graph lifecycle is the key to preventing out-of-memory (OOM) errors during training.

The distinction between compute-bound and memory-bandwidth-bound execution is the central reality of deep learning hardware. Prompt prefill in large language models processes hundreds of tokens in parallel, saturating matrix multiplication units (compute-bound). Autoregressive decoding processes a single token at a time, reading billions of parameter and KV cache bytes just to execute a few thousand arithmetic operations (memory-bandwidth-bound). Modern serving optimizations attack this bottleneck from

two directions. FlashDecoding and INT4 weight-only quantization reduce the bytes that must cross the bus, which is the bandwidth problem Chapter 15 taught you to measure. PagedAttention attacks capacity rather than bandwidth: Chapter 19 shows it reclaiming the 60 to 80 percent of high-bandwidth memory that reservation, internal, and external fragmentation waste, bringing that waste under 4 percent. Both directions matter, and confusing them is how a serving system ends up optimizing the wrong resource.

A model that runs in zero milliseconds by predicting zero has an infinite speedup and zero utility. A model pruned to 90% sparsity that runs on dense matrix multipliers wastes memory on explicit zeros and saves zero time. Optimization without verification is speculation. Every system transformation must be empirical: measured on identical hardware, tested on identical workloads, and evaluated against strict, non-negotiable accuracy retention floors.

Python provides productive syntax at the expense of substantial runtime overhead: interpreter dispatch, reference counting, dynamic type resolution, and non-contiguous memory allocations. Production runtimes (PyTorch Inductor, TensorRT, vLLM, ExecuTorch) bypass this overhead by tracing graphs, fusing operator sequences into monolithic kernels, and compiling directly to low-level hardware targets (CUDA, Triton, Metal, C++). When you know what lies beneath the abstraction, you know how to work with the compiler rather than against it.

The Bridge to Industrial Engines: From TinyTorch to PyTorch and JAX

A central philosophy of this book is the **Minix-to-Linux principle** (Chapter 23): when Linus Torvalds set out to author Linux, he did not begin by deciphering a proprietary, million-line Unix operating system. He studied Andrew Tanenbaum's Minix: a compact teaching kernel small enough to be held completely in mind. Understanding how Minix managed memory, handled interrupts, and scheduled processes gave Linus the foundational mental model to author, debug, and scale Linux.

TinyTorch is the Minix and xv6 of deep learning systems. Industrial engines like PyTorch and JAX comprise tens of millions of lines of C++, CUDA, Python, LLVM compiler backends, and distributed runtime orchestrators. Diving into their codebases without a foundational mental model is overwhelming. But every single abstraction in PyTorch and JAX is a direct architectural descendant of the contracts you built in TinyTorch:

From TinyTorch to PyTorch

PyTorch's design mirrors TinyTorch's imperative, define-by-run architecture, wrapped in high-performance C++ and JIT compiler dispatch:

- **The Tensor Substrate:** In PyTorch, `torch.Tensor` is a lightweight Python wrapper around the C++ `at::Tensor` class, which holds an intrusive pointer to `c10::TensorImpl`. The fields of `c10::TensorImpl` correspond to what you worked out in Chapter 1: a pointer to a `c10::Storage` raw byte buffer, an array of dimension sizes, and an array of dimension strides. When you call `.view()` or `.transpose()` in PyTorch, it creates a new `TensorImpl` pointing to the same

`Storage` with altered strides, the striding algebra you worked out in Chapter 1, where TinyTorch deliberately copies instead.

- **The Autograd Graph:** PyTorch's autograd engine (`torch.autograd`) operates identically to TinyTorch's tape (Chapter 6). When a differentiable tensor participates in an operation, the framework instantiates a C++ `Node` subclassing `torch.autograd.Node` (the production equivalent of TinyTorch's `Function`). The `Node` records the input pointers and saves necessary intermediate tensors on its forward tape. When you invoke `loss.backward()`, PyTorch's `Engine::evaluate` executes a topological depth-first search traversal, invoking `Node::apply()` to propagate vector-Jacobian products backward and accumulate `.grad` tensors.
- **Compiler Lowering and Kernel Fusion:** TinyTorch's `compile_graph` (Chapter 23) traced define-by-run nodes into a single fused expression, showing the capture step that a real fusion compiler lowers into one loop. PyTorch 2.0's `torch.compile` implements this exact workflow at industrial scale through three coordinated stages:
  1. *TorchDynamo* intercepts Python bytecode at runtime using CPython frame evaluation hooks (`PEP 523`) and extracts computational subgraphs into an FX graph (the industrial counterpart to TinyTorch's `TracedNode`).
  2. *AOTAutograd* traces through the autograd boundary, capturing forward and backward subgraphs before execution.
  3. *TorchInductor* compiles the captured graph, generating optimized OpenAI Triton GPU kernels and OpenMP-parallelized C++ CPU kernels, matching the exact Triton and SIMD patterns you authored in Chapter 23.

From TinyTorch to JAX

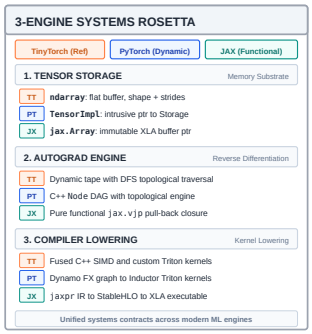
JAX approaches machine learning systems from a functional, compiler-first philosophy rather than an object-oriented imperative tape:

- **Pure Functions and Immutable Arrays:** While TinyTorch and PyTorch treat tensors as mutable objects with in-place `.grad` accumulation, JAX operates on pure mathematical functions: $y = f(x, \theta)$. A `jax.Array` is immutable, backed directly by an on-device XLA buffer pointer.
- **Vector-Jacobian Products (`vjp`):** JAX does not maintain an object tape on tensor instances. Instead, its reverse-mode autodiff primitive `jax.vjp` takes a Python function f and an input x, evaluates the forward pass, and returns the primal output y alongside a *pull-back function* (the VJP). Evaluating the pull-back function with incoming cotangent $\bar{y}$ propagates gradients backward through the chain rule, making explicit the mathematical derivative contract you defined in `Function.backward`. When programmers call the familiar `jax.grad(f)`, JAX simply wraps `jax.vjp(f)` and evaluates the pull-back closure with an incoming cotangent of `1.0` (for scalar loss).
- **Traced IR (`jaxpr`):** When you decorate a function with `@jax.jit`, JAX executes the Python code once using abstract tracer objects (`ShapedArray`). These tracers record every mathematical operation into a linear Directed Acyclic Graph called a **jaxpr** (JAX expression), analogous to TinyTorch's computational graph.
- **Whole-Program Silicon Lowering:** Rather than dispatching kernels operator-by-operator across a Python runtime, JAX lowers the `jaxpr` graph to StableHLO (High-Level Optimizer bytecode) and hands the entire computation graph to the XLA (Accelerated Linear Algebra) compiler. XLA performs whole-program memory

layout assignment, inter-operator buffer reuse, and fusion across entire transformer blocks, targeting Google TPUs or NVIDIA GPUs with zero Python interpreter overhead.

The Structural Rosetta Stone

table 23.10 bridges each foundational systems boundary across TinyTorch, PyTorch, and JAX.



Three-Engine Architecture: The same systems contracts govern TinyTorch, PyTorch, and JAX.

Table 23.10: **The Systems Rosetta Stone: Bridging TinyTorch to Industrial Frameworks.** Every abstraction you authored maps directly to production runtime primitives in PyTorch and JAX.

Systems Boundary	TinyTorch (Educational Reference)	PyTorch (Industrial Dynamic Engine)	JAX (Industrial Functional Compiler)
Tensor Storage	NumPy `ndarray` buffer with shape & strides	`c10::TensorImpl` over `c10::Storage` byte buffer	`jax.Array` backed by XLA device memory buffer
Autograd Engine	Dynamic tape with DFS topological traversal	Dynamic C++ `Engine::evaluate` over `Node` DAG	Pure functional `jax.vjp` returning a pull-back closure
Custom Operator	Subclass `Function(forward, backward)`	Subclass `autograd.Function` or `TORCH_LIBRARY`	Pure Python function transformed via `jax.custom_vjp`
Graph Capture	Symbolic `TracedNode` AST capture	TorchDynamo frame evaluation extracting FX graph	Abstract `ShapedArray` tracing extracting `jaxpr` IR
Kernel Lowering	Hand-written C++ SIMD & Triton kernels	TorchInductor generating Triton & OpenMP C++	XLA compiling StableHLO to machine code/TPU HLO
Model State	Stateful `Layer` with `.parameters()` list	Stateful `torch.nn.Module` with `named_parameters()`	Pure functional `PyTree` dictionary of parameter weights
Serving Cache	Stateful `KVCache` preallocated buffer with write cursor (Chapter 19)	PagedAttention (vLLM)/Static Tensor Cache	Static shape-padded XLA buffer allocation

When you encounter a cryptic PyTorch Dynamo guard error, an ATen stride mismatch during a tensor view operation, a CUDA out-of-memory crash caused by uncheckpointed autograd tape allocations, or an XLA recompilation warning from dynamic shapes, you will not see them as inexplicable framework mysteries. You know the memory ribbons, the topological sorting algorithms, the cache block sizes, and the arithmetic intensities that govern them.

The Road Ahead: Beyond Single-Node TinyTorch

TinyTorch gave you a complete, inspectable reference implementation of a single-node deep learning engine. Where does the field go from here?

1. **Distributed Training at Planetary Scale**: Modern foundation models exceed the memory capacity of any single GPU by orders of magnitude. The principles of tensor striding, autograd tapes, and memory allocation you mastered here extend directly to distributed execution:
   - **Data Parallelism (DDP)**: Replicating the model across workers while computing synchronous all-reduce reductions over gradient buffers.
   - **Tensor Parallelism (Megatron-LM)**: Splitting weight matrices across row and column dimensions within GEMM kernels, communicating intermediate activations across high-speed NVLink interconnects.
   - **Pipeline Parallelism**: Partitioning transformer layers sequentially across separate accelerator nodes and scheduling micro-batches via bubble-minimizing 1F1B (one-forward-one-backward) schedules.
   - **Fully Sharded Data Parallel (FSDP/ZeRO)**: Sharding optimizer states, gradients, and model parameters across the cluster, all-gathering parameters on demand during forward/backward execution and releasing them immediately to maintain minimal memory footprints.
2. **Hardware-Software Co-Design and Custom Silicon**: The boundaries between software compilers and hardware architectures have dissolved. Google TPUs, AWS Trainium, and Groq LPUs eschew traditional cache hierarchies in favor of software-managed scratchpad memories and 2D systolic arrays. Writing software for these targets requires the exact systems intuition you developed in Chapter 18 and Chapter 23: choreographing data movement so that matrix execution units never starve for operands.
3. **Compiler Lowering and Graph Capture**: Modern deep learning frameworks are rapidly becoming optimizing compilers. Systems like `torch.compile` capture computational subgraphs using Dynamo, trace tensor metadata with AOTAutograd, and lower intermediate representations (IR) into hardware-tuned Triton or C++ kernels via Inductor. Because you built `Tensor`, `Function`, and `KVCache` by hand, you can read the generated intermediate representations not as obscure compiler output, but as the direct automated counterpart of the kernels you wrote.
4. **Speculative Decoding and Advanced Serving Engines**: Autoregressive generation is fundamentally memory-bandwidth bound ($O(1)$ arithmetic intensity per token generated). Speculative decoding breaks this memory wall by pairing a small, fast draft model with a large target model. The draft model speculatively generates a sequence of K candidate tokens at high speed, which the target model verifies in parallel within a single forward pass, converting sequential memory-bound token steps into a compute-dense parallel batch verification. Production engines like vLLM and TensorRT-LLM combine speculative decoding, virtual memory paging for key-value tensors (PagedAttention), and continuous chunked prefill to maximize accelerator saturation.

The Builder's Manifesto

When you first opened this book, modern machine learning may have appeared as an impenetrable tower of complex software frameworks, opaque CUDA libraries, and billion-parameter models.

You have now worked up from the lowest layer of the stack. You know how a float32 byte is laid out in DRAM. You know how a stride translates a coordinate into a memory offset. You know how an autograd tape tracks forward computations and walks backward through graph topologies. You know how attention masks out the future, how a KV cache eliminates redundant projections, and how integer quantization squeezes neural networks into constrained edge devices.

Across twenty modules, seven milestones, and three synthesis chapters, you implemented a working framework in Python and NumPy and measured where its time and memory go.